Chapter 1. Introducing DFSORT

DFSORT overview

This chapter introduces IBM z/OS DFSORT Licensed Program 5655-ZOS.

DFSORT is intended to run in problem state and in a user key (that is, key 8 or higher).

DFSORT is a program you use to sort, merge, and copy information.

- When you sort records, you arrange them in a particular sequence, choosing an order more useful to you than the original one.
- When you merge records, you combine the contents of two or more previously sorted data sets into one.
- When you copy records, you make an exact duplicate of each record in your data set.

Merging records first requires that the input data sets are identically sorted for the information you will use to merge them and that they are in the same order required for output. You can merge up to 100 different data sets at a time.

In addition to the three basic functions, you can perform other processing simultaneously:

You can control which records to keep in the final output data set of a DFSORT run by using INCLUDE and OMIT statements in your application. These statements work like filters, testing each record against criteria that you supply and retaining only the ones you want for the output data set. For example, you might choose to work only with records that have a value of “Kuala Lumpur” in the field reserved for office location. Or perhaps you want to leave out any record dated after 2001 if it also contains a value greater than 20 for the number of employees.

You can parse, edit, and reformat your records before or after other processing by using INREC and OUTREC statements. INREC and OUTREC statements support a wide variety of reformatting tasks including:

- The use of fixed position/length fields or variable position/length fields. For fixed fields, you specify the starting position and length of the field directly. For variable fields, such as delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings (and many other types), you define rules that allow DFSORT to extract the relevant data into fixed parsed fields, and then use the parsed fields as you would use fixed fields.
- Insertion of blanks, zeros, strings, current date, future date, past date, current time, sequence numbers, decimal constants, and the results of arithmetic expressions before, between, and after the input fields in the reformatted records.
- Sophisticated conversion capabilities, such as find and replace, hexadecimal display, bit display, translation of EBCDIC letters from lowercase to uppercase or uppercase to lowercase, translation of characters from EBCDIC to ASCII and from ASCII to EBCDIC, translation of characters using the ALTSEQ translation table, conversion of numeric values from one format to another, left-justify or left-squeeze (remove leading blanks or all blanks and shift left), and right-justify or right-squeeze (remove trailing blanks or all blanks and shift right).
- Sophisticated editing capabilities, such as control of the way numeric fields are presented with respect to length, leading or suppressed zeros, thousands separators, decimal points, leading and trailing positive and negative signs, and so on.

  Twenty-seven pre-defined editing masks are available for commonly used numeric editing patterns, encompassing many of the numeric notations used throughout the world. In addition, a virtually unlimited number of numeric editing patterns are available via user-defined editing masks.
- Transformation of SMF, TOD, and ETOD date and time values to more usable forms.
- Conversion of input date fields of one type (CH, ZD, PD, 2-digit year, 4-digit year, Julian, Gregorian) to corresponding output date fields of another type or to a corresponding day of the week.

© Copyright IBM Corp. 1973, 2026

- Various types of arithmetic operations for input date fields.
- Selection of a character constant, hexadecimal constant, or input field from a lookup table for output, based on a character, hexadecimal, or bit string as input (that is, lookup and change).

You can create the reformatted INREC or OUTREC records in one of the following ways:

- By building the entire record one item at a time.
- By only overlaying specific columns.
- By performing find and replace operations.
- By using sophisticated conditional logic or group operations to choose how different records are reformatted.

You can sum numeric information from many records into one record with the SUM statement. For example, if you want to know the total amount of a yearly payroll, you can add the values for a field containing salaries from the records of all your employees.

You can create one or more output data sets for a sort, copy, or merge application from a single pass over one or more input data sets by using OUTFIL control statements. You can use multiple OUTFIL statements, with each statement specifying the OUTFIL processing to be performed for one or more output data sets. OUTFIL processing begins after all other processing ends (that is, after processing for exits, options, and other control statements). OUTFIL statements support a wide variety of output data set tasks, including:

- Creation of multiple output data sets containing unedited or edited records from a single pass over one or more input data sets.
- Creation of multiple output data sets containing different ranges or subsets of records from a single pass over one or more input data sets. In addition, records that are not selected for any subset can be saved in a separate output data set.
- Conversion of variable-length record data sets to fixed-length record data sets.
- Conversion of fixed-length record data sets to variable-length record data sets.
- A wide variety of parsing, editing and reformatting tasks including:
  - The use of fixed position/length fields or variable position/length fields. For fixed fields, you specify the starting position and length of the field directly. For variable fields, such as delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings (and many other types), you define rules that allow DFSORT to extract the relevant data into fixed parsed fields, and then use the parsed fields as you would use fixed fields.
  - Insertion of blanks, zeros, strings, current date, future date, past date, current time, sequence numbers, decimal constants, and the results of arithmetic expressions before, between, and after the input fields in the reformatted records.
  - Sophisticated conversion capabilities, such as find and replace, hexadecimal display, bit display, translation of EBCDIC letters from lowercase to uppercase or uppercase to lowercase, translation of characters from EBCDIC to ASCII or from ASCII to EBCDIC, translation of characters using the ALTSEQ translation table, conversion of numeric values from one format to another, left-justify or left-squeeze (remove leading blanks or all blanks and shift left), and right-justify or right-squeeze (remove trailing blanks or all blanks and shift right).
  - Sophisticated editing capabilities, such as control of the way numeric fields are presented with respect to length, leading or suppressed zeros, thousands separators, decimal points, leading and trailing positive and negative signs, and so on.

    Twenty-seven pre-defined editing masks are available for commonly used numeric editing patterns, encompassing many of the numeric notations used throughout the world. In addition, a virtually unlimited number of numeric editing patterns are available via user-defined editing masks.
  - Transformation of SMF, TOD, and ETOD date and time values to more usable forms.
  - Conversion of input date fields of one type (CH, ZD, PD, 2-digit year, 4-digit year, Julian, Gregorian) to corresponding output date fields of another type or to a corresponding day of the week.

  - Various types of arithmetic operations for input date fields.
  - Selection of a character constant, hexadecimal constant, or input field from a lookup table for output, based on a character, hexadecimal, or bit string as input (that is, lookup and change).
- Creation of the reformatted records in one of the following ways:
  - By building the entire record one item at a time.
  - By only overlaying specific columns.
  - By performing find and replace operations.
  - By using sophisticated conditional logic or group operations to choose how different records are reformatted.
- Highly detailed three-level (report, page, and section) reports containing a variety of report elements you can specify (for example, current date, current time, edited or converted page numbers, character strings, and blank lines) or derive from the input records (for example, character fields; unedited, edited, or converted numeric input fields; edited or converted record counts; and edited or converted totals, maximums, minimums, and averages for numeric input fields).
- Creation of multiple output records from each input record, with or without intervening blank output records.
- Repetition and sampling of data records.
- Splitting of data records in rotation among a set of output data sets.

You can perform various "join" operations on two files by one or more keys. A JOINKEYS application allows you to create joined records in a variety of ways including inner join, full outer join, left outer join, right outer join and unpaired combinations. The two input files can be of different types (fixed, variable, VSAM, and so on) and have keys in different locations. The records from the two input files can be processed in a variety of ways before and after they are joined.

You can control DFSORT functions with other control statements by specifying alternate collating sequences, invoking user exit routines, overriding installation defaults, and so on.

You can direct DFSORT to pass control during run time to routines you design and write yourself. For example, you can write user exit routines to summarize, insert, delete, shorten, or otherwise alter records during processing. However, keep in mind that the extensive editing capabilities provided by the INCLUDE, OMIT, INREC, OUTREC, SUM, and OUTFIL statements can eliminate the need to write user exit routines. You can write your own routines to correct I/O errors that DFSORT does not handle, or to perform any necessary abnormal end-of-task operation before DFSORT terminates.

You can write an EFS (Extended Function Support) program to intercept DFSORT control statements and PARM options for modification prior to use by DFSORT or to provide alternate sequence support for user-defined data.

You can define and use a symbol for any field, constant, or output column that is recognized in a DFSORT control statement or ICETOOL operator. This makes it easy to create and reuse collections of symbols (that is, mappings) representing information associated with various record layouts. You can use system symbols (for example, &JOBNAME.) in your symbol constants. You can use SET and PROC symbols in your symbol constants. See Chapter 8, “Using symbols for fields and constants,” on page 681.

DFSORT on the Web

For articles, online documents, news, tips, techniques, examples, and more, visit the DFSORT Home Page (www.ibm.com/storage/dfsort).

Invoking DFSORT

You can invoke DFSORT processing in the following ways:

- With an EXEC job control statement in the input stream using the name of the program (for example, PGM=ICEMAN or PGM=SORT) or the name of a cataloged procedure (for example, SORTD). See Chapter 2, “Invoking DFSORT with Job Control Language,” on page 27.

  TSO users can allocate the needed ddnames (for example, SYSOUT, SORTIN, SORTOUT and SYSIN), and invoke DFSORT using a calling method equivalent to PGM=ICEMAN. For example:

  ```
  call *(iceman)
  ```

  Restriction: TSO users cannot invoke DFSORT using `iceman`.

  or any other alias for DFSORT (for example, SORT) in this form.

  See Chapter 11, “Examples of DFSORT job streams,” on page 761 for examples of invoking DFSORT from REXX and CLISTs.
- With a program written in basic assembler language using a system macro instruction. See Chapter 6, “Invoking DFSORT from a program,” on page 513.
- With programs written in either COBOL or PL/I with a special facility of the language. See the programmer's guide describing the compiler version available at your location.
- With the ICETOOL utility. See Chapter 7, “Using ICETOOL,” on page 533.

In this document, the term *directly invoked* means that DFSORT is not initiated from another program. The term *program invoked* means that DFSORT is initiated from another program.

How DFSORT works

This section contains a list of the operating systems supported by DFSORT and an explanation of how DFSORT uses control fields and collating sequences to sort, merge, and copy the records of a data set.

The Blockset technique is DFSORT's most efficient technique for sorting, merging and copying data sets. DFSORT uses the Blockset technique whenever possible to take advantage of its highly optimized internal algorithms and efficient utilization of IBM hardware. If Blockset cannot be used, DFSORT uses another of its techniques — Peerage/Vale or Conventional.

Operating systems

DFSORT runs under control of your z/OS operating system and must be initiated according to the appropriate conventions.

Additionally, DFSORT runs under z/OS when it is running as a guest under z/VM®.

DFSORT is compatible with all of the IBM processors supported by z/OS. In addition to any device supported by z/OS for program residence, DFSORT also operates with any device QSAM or VSAM uses for input or output.

Control fields and collating sequences

You define control fields to identify the information you want DFSORT to sort or merge. When thinking of the contents of your data sets, you probably think of names, dates, account numbers, or similar pieces of useful information. For example, when sorting your data sets, you might choose to arrange your records in alphabetical order, by family name. By using the byte position and length (in bytes) of the portion of each record containing a family name, you can define it as a control field to manipulate with DFSORT.

DFSORT uses the control fields you define as keys in processing. A *key* is a concept, such as family name, that you have in mind when you design a record processing strategy for a particular application. A control field, on the other hand, is a discrete portion of a record that contains the text or symbols corresponding to that information in a form that can be used by DFSORT to identify and sort, or merge the records. For all practical purposes, you can think of keys as equivalent to the control fields DFSORT uses in processing.

To arrange your records in a specific order, identify one or more control fields of your records to use as keys. The sequence in which you list the control fields becomes the order of priority DFSORT uses to arrange your records. The first control field you specify is called the *major control field*. Subsequent control fields are called *minor control fields*, as in first, second, third minor control fields, and so on.

If two or more records have identical values for the first control field, they are arranged according to the values in the second. Records with identical values for the first and second are arranged according to the third, and so on, until a difference is found or no more control fields are available.

Records with identical values for all the control fields specified retain their original input order or are arranged randomly, depending upon which of the two options, EQUALS or NOEQUALS, is in effect. You can direct DFSORT to retain the original input order for records with identical values for all control fields by specifying EQUALS.

Control fields may overlap, or be contained within other control fields (such as a three-digit area code, within a 10-digit telephone number). They do not need to be contiguous but must be located within the first 32752 bytes of the record (see Figure 1 on page 5).

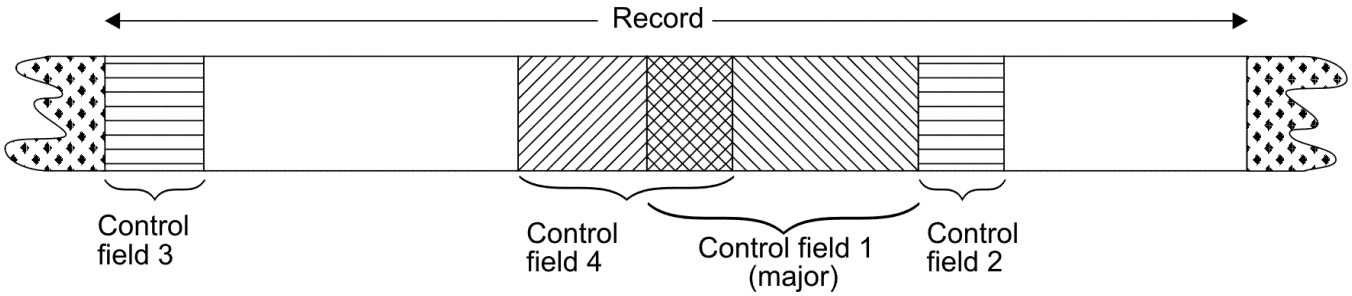


Figure 1. Control fields

DFSORT offers several standard collating sequences. You can choose to arrange your records according to these standard collating sequences or according to a collating sequence defined in the active locale. Conceptually, a collating sequence is a specific arrangement of character priority used to determine which of two values in the same control field of two different records should come first. DFSORT uses EBCDIC, the standard IBM collating sequence, or the ASCII collating sequence when sorting or merging records. If locale processing is in effect, DFSORT will use the collating sequence defined in the active locale.

The collating sequence for character data and binary data is absolute; character and binary fields are not interpreted as having signs. For packed decimal, zoned decimal, fixed-point, normalized floating-point, and the signed numeric data formats, collating is algebraic; each quantity is interpreted as having an algebraic sign.

You can modify the standard EBCDIC sequence to collate differently if, for example, you want to allow alphabetic collating of national characters. An alternate collating sequence can be defined with the ALTSEQ installation option, or you can define it yourself at run-time with the ALTSEQ program control statement. You can also specify a modified collating sequence with an E61 user exit or with an EFS program.

You can specify the LOCALE installation or run-time option to use an active locale's collating rules.

Cultural environment considerations

DFSORT's collating behavior can be modified according to your cultural environment. Your cultural environment is defined to DFSORT using the X/Open** locale model. A locale is a collection of data grouped into categories that describes the information about your cultural environment.

The collate category of a locale is a collection of sequence declarations that defines the relative order between collating elements (single character and multi-character collating elements). The sequence declarations define the collating rules.

The cultural environment is established by selecting the active locale. The active locale affects the behavior of locale-sensitive functions. In particular, the active locale's collating rules affect DFSORT's SORT, MERGE, INCLUDE, and OMIT processing as follows:

- Sort and Merge

  DFSORT produces sorted or merged records for output according to the collating rules defined in the active locale. This provides sorting and merging for single- or multi-byte character data based on defined collating rules that retain the cultural and local characteristics of a language.

- Include and Omit

  DFSORT includes or omits records for output according to the collating rules defined in the active locale. This provides inclusion or omission for single- or multi-byte character data based on defined collating rules that retain the cultural and local characteristics of a language.

  Note: Locale processing is not used for IFTRAIL TRLID or IFTHEN WHEN, BEGIN or END constants or compare fields.

The DFSORT option LOCALE specifies whether locale processing is to be used and, if so, designates the active locale. Only one locale can be active at a time for any DFSORT application.

Unicode

The Unicode Standard provides a single character set that covers the languages of the world, and a small number of machine-friendly encoding forms and schemes to fit the needs of existing applications and protocols. It is designed for best interoperability with both ASCII and ISO-8859-1, the most widely used character sets, to make it easier for Unicode to be used in applications and protocols.

A special number is assigned to every single character. You can refer to The Unicode Consortium (www.unicode.org) to find out more about Unicode. You can find the code charts, which show you which number is assigned to which character within the Unicode standard on the Unicode Character Code Charts (www.unicode.org/charts). If you prefer to have a list of all available characters on your workstation so that you do not have to be online every time you want to find a specific character in the Unicode code page, you can download the Unibook Character Browser (www.unicode.org/unibook).

Currently, the following three forms of Unicode encoding are supported:

UTF8:

Unicode **T**ransformation **F**ormat in 8 bits that uses 1 to 4 bytes, depending on the character.

UTF16:

Unicode **T**ransformation **F**ormat in 16 bits that uses either 2 or 4 bytes to represent a character.

UTF32:

Unicode **T**ransformation **F**ormat in 32 bits.

Unicode Environment Considerations

DFSORT's Unicode Collation behavior can be modified according to your cultural environment. A new COLLKEY parameter allows specifying the Unicode standard character suite for national collation sequences. Your Unicode environment is defined to DFSORT using the COLLKEY=value

Collation supports customization, which means that collation service might behave according to some specific collation rules. Collation rules can be specified using a Locale or User Collation Rules (UCR).

The active collation rules affects the behavior of locale-sensitive functions. In particular, the active locale's collating rules affect DFSORT's SORT and MERGE processing as follows:

Sort and Merge

DFSORT produces sorted or merged records for output according to the collating rules defined in the active locale. This provides sorting and merging for single-or multi-byte character data based on defined collating rules that retain the cultural and local characteristics of a language.

For supported Locales settings, see Appendix E. Locales for collation in *z/OS Unicode Services User's Guide and Reference*

IBM Integrated Accelerator for Z Sort

The IBM z15 provides a sort accelerator that is known as the IBM Integrated Accelerator for Z Sort. The SORTL hardware instruction allows use of a hardware accelerated approach to sorting. To take advantage of the IBM Integrated Accelerator for Z Sort, a z15 or later processor is required.

The IBM Integrated Accelerator for Z Sort feature (ZSORT) helps to reduce the CPU costs and improve the elapsed time for eligible workloads. One of the primary requirements for ZSORT is providing enough virtual, real, and auxiliary storage. Sort jobs that run in memory-constrained environments where the amount of memory available to be used by DFSORT jobs is restricted might not achieve optimal performance results. They might also not be able to exploit ZSORT.

The 64-bit memory objects (above the bar storage) are used to exploit the ZSORT accelerator for sort workloads for optimal results. Since ZSORT is part of the CPU and memory latency is much lower than disk latency, sorting in memory is more efficient than sorting with both memory and disk workspace.

By allowing ZSORT to process the input completely in memory, it can achieve the best results in both elapsed time and CPU time. It might usually require a memory object size to be approximately 2x the input size (the actual size might vary depending on record format, record length (LRECL), and key length) to process the sort completely in memory. For simplicity, the installation default MOSIZE=MAX is recommended, which allows obtaining the optimal size of the memory object. At a minimum, the memory object size must be at least 1.5 times the input file size, or an amount that is greater than 1.5 times the input file size, to be eligible for ZSORT.

When a sort workload cannot be processed completely in memory, disk workspace is used in combination with memory object sorting. When using ZSORT, enhanced I/O technologies are used. If the sort workload cannot be contained entirely in memory, then it might require work data sets that support zHPF (z High Performance FICON). This is needed to provide the benefit of reducing elapsed and CPU time.

When using ZSORT with disk workspace, make sure that enough disk workspace is allocated and available. The amount of disk workspace that is required is at least the size of the input file. For jobs that define the sort disk workspace by using SORTWKxx DD statements, you must specify enough primary and secondary space in the JCL space parameters.

If enough 64-bit storage for ZSORT cannot be allocated, or enough disk space when using sort works, DFSORT will transition to traditional sort processing. Message ICE267I indicates whether the ZSORT accelerator was used with a reason code to explain the condition the sort job met.

The exploitation of ZSORT is shipped as disabled by default. Enabling the new function via the DFSORT installation option allows you to quickly use the new function without having to modify JCL and programs. Consider first enabling ZSORT for individual jobs. This allows you to evaluate ZSORT in your overall batch framework. These options can be activated using one of the following methods in:

- ICEPRMxx members of concatenated parmlib
- An ICEMAC macro option
- OPTION control statement

For the first two methods, see *z/OS DFSORT Installation and Customization*. For the third method, see “OPTION control statement” on page 171.

IBM’s zBNA tool (V2.2) provides modeling support for identifying potential ZSORT-eligible candidate jobs and estimates the benefits of ZSORT. The tool uses information in the SMF type 16 records.

Not all sorts are eligible to use ZSORT. Following is a list of restrictions that will disable ZSORT and will revert to using traditional sorting technique:

- SORTL facility is not enabled or is unavailable on the processor.
- ZSORT is not enabled.
- OPTION COPY or SORT FIELDS=COPY is specified

- Usage of INREC
- Usage of JOINKEYS
- Usage of MERGE FIELDS
- Usage of MODS(EXITS) statements
- Usage of OUTREC
- Usage of OUTFIL
- Usage of SUM FIELDS
- Program invoked sorts
- Memory objects cannot be created
- Insufficient memory object storage available (required more than currently available)
- Unsupported sort fields specified (examples Unicode, Locale, ALTSEQ ...).
- Unknown file size or file size=0.
- SIZE/FILSZ=Uxxxxxx is specified.
- SORTIN/SORTOUT is a VSAM Cluster.
- Sort control field positions are beyond 4092 and VLSHRT is specified.
- Usage of the EXCP access method was requested.
- Insufficient storage (example above the line or below the line).
- Sorting key greater than 4088 bytes or greater than 4080 bytes if EQUALS is specified.
- For variable records, the record length (LRECL) must be greater than 24.
- ZHPF is unavailable for a sort that could not be performed entirely in memory.
- Insufficient amount of sort workspace.

Note:

1. Because ZSORT's goal is to reduce CPU time and elapsed time, it can require more storage than a DFSORT application that does not use ZSORT. It is recommended to have REGION=0M and MEMLIMIT=NOLIMIT for ZSORT jobs.
2. For below the bar memory requirements, DFSORT must be able to obtain enough memory for its various control blocks that reside both below and above the 16 MB line. Specifying a SIZE/MAINSIZE=MAX is recommended to achieve the best performance.

   For sorts under 100 GB, the shipped default DSA value of 128 is enough. For sorts larger than 100 GB, consider increasing DSA to 256. Note that this is only a limit. DFSORT’s dynamic storage adjustment calculates the optimal virtual storage allocation for each sort. Increasing the DSA limit impacts only very large sorts that DFSORT determines would benefit from increased virtual storage.

DFSORT processing

You must prepare job control language (JCL) statements and DFSORT program control statements to invoke DFSORT processing. JCL statements (see Chapter 6, “Invoking DFSORT from a program,” on page 513) are processed by your operating system. They describe your data sets to the operating system and initiate DFSORT processing. DFSORT program control statements (see Chapter 3, “Using DFSORT program control statements,” on page 77) are processed by the DFSORT program. They describe the functions you want to perform and invoke the processing you request.

A sort application usually requires intermediate storage as working space during the program run. This storage can be one of the following:

1. Hiperspace, using DFSORT's Hipersorting feature.
2. Work data sets—either allocated dynamically by DFSORT's DYNALLOC facility or specified by the user, using JCL DD statements. If specified by the user, the intermediate storage devices and the amount of work space must be indicated. Methods for determining the amount of work space to allocate are explained in Appendix A, “Using work space,” on page 793.

3. A combination of Hiperspace and work data sets.

Merge and copy applications do not require intermediate storage.

Figure 2 on page 10 illustrates the processing order for record handling, exits, statements, and options. Use this diagram with the text following it to understand the order DFSORT uses to run your job.

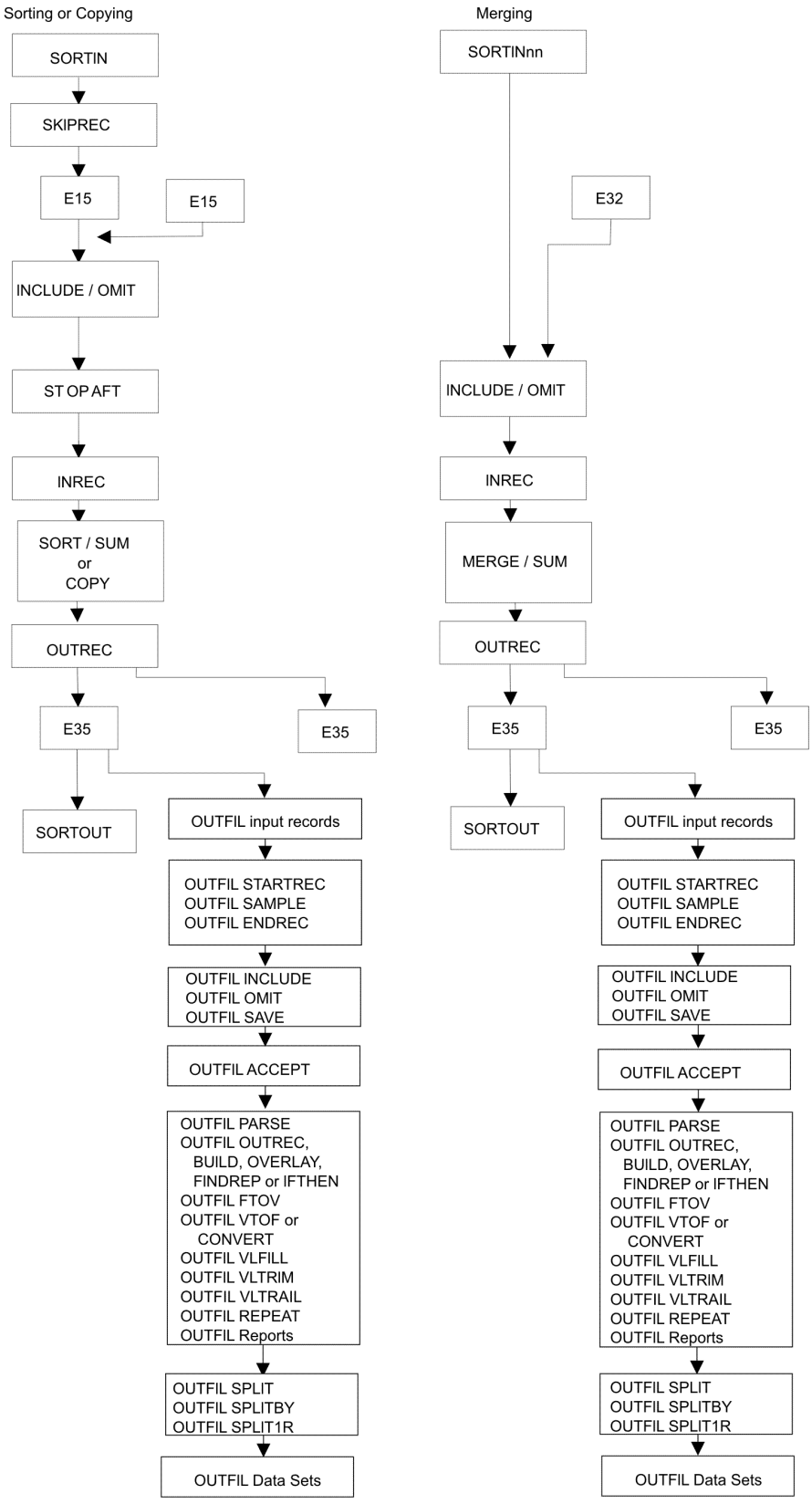


Figure 2. Record Processing Order

As shown in Figure 2 on page 10, DFSORT processing follows this order:

1. DFSORT first checks whether you supplied a SORTIN data set for SORT and COPY jobs or SORTINnn data sets for MERGE jobs. If so, DFSORT reads the input records from them.
   - If no SORTIN data set is present for a SORT or COPY job, you must use an E15 user exit to insert all the records. (This is also true if you invoke DFSORT from a program with the address of an E15 user exit in the parameter list, because SORTIN will be ignored.) DFSORT can use a COBOL E15 routine if you specified the E15 user exit in the MODS statement.
   - If no SORTINnn data sets are present for a MERGE job, you must use an E32 user exit to insert all the records.
2. If input records for SORT or COPY jobs are read from a SORTIN data set, DFSORT performs processing specified with the SKIPREC option. DFSORT deletes records until the SKIPREC count is satisfied. Eliminating records before a SORT or COPY gives better performance.
3. If the input records for a SORT or COPY job are read from a SORTIN data set, DFSORT checks whether you specified an E15 user exit. If so, DFSORT transfers control to the user exit routine. You can use a COBOL E15 routine if the E15 user exit is specified in the MODS statement. The E15 routine can insert, delete, or reformat records.
4. DFSORT performs processing specified on an INCLUDE or OMIT statement. If you used an E15 user exit routine to reformat the records, the INCLUDE/OMIT fields you specify must apply to the current format rather than to the original format. If you use the INCLUDE or OMIT statements to delete unnecessary records before SORT, MERGE, or COPY processing, your jobs run more efficiently.
5. For SORT or COPY jobs, DFSORT performs processing specified with the STOPAFT option. Record input stops after the maximum number of records (n) you specify have been accepted. DFSORT accepts records for processing if they are:
   - Read from SORTIN or inserted by E15
   - Not deleted by SKIPREC
   - Not deleted by E15
   - Not deleted by an INCLUDE or OMIT statement.
6. DFSORT performs processing specified in an INREC statement. Data records are parsed, edited and reformatted according to the options specified. If you reformatted the records before this step, the INREC fields you specify must apply to the current format rather than to the original format.
7. DFSORT performs processing specified in the SORT, MERGE, or OPTION COPY statement.
   - For SORT, all input records are processed before any output record is processed.
   - For COPY or MERGE, an output record is processed after an input record is processed.
   - For SORT or MERGE, if a SUM statement is present, DFSORT processes it during the SORT or MERGE processing. DFSORT summarizes the records and deletes duplicates. If you reformatted the records before this step, the SORT or MERGE and SUM fields you specify must apply to the current format rather than to the original format.
8. DFSORT performs processing specified in an OUTREC statement. Data records are parsed, edited and reformatted according to the options specified. If you reformatted the records before this step, the OUTREC fields you specify must apply to the current format rather than to the original format.
9. If an E35 user exit is present, DFSORT transfers control to your user exit routine after all statement processing (except OUTFIL) is completed. If you reformatted the records, the E35 user exit receives the records in the current format rather than in the original format. You can use a COBOL E35 routine if you specify the E35 user exit in the MODS statement. You can use the E35 exit routine to add, delete, or reformat records.

   If SORTOUT and OUTFIL data sets are not present, the E35 exit must dispose of all the records because DFSORT treats these records as deleted. (This is also true if you do not specify OUTFIL data sets and DFSORT is invoked with the address of an E35 user exit in the parameter list, because SORTOUT will be ignored.)
10. DFSORT writes your records to the SORTOUT data set, if present.
11. DFSORT performs processing specified in one or more OUTFIL statements, if present:

- DFSORT performs processing specified with the STARTREC, SAMPLE, and ENDREC options. Record input for the OUTFIL data sets starts with the record indicated by STARTREC, ends with the record indicated by ENDREC, and is sampled according to the records indicated by SAMPLE.
- DFSORT performs processing specified with the INCLUDE, OMIT, or SAVE option. Records are included or omitted from the OUTFIL data sets according to the criteria specified.
- DFSORT performs processing specified with the ACCEPT option. Record processing ends when the ACCEPT limit is reached.
- DFSORT performs processing specified with the PARSE, OUTREC (or BUILD), OVERLAY, FINDREP, IFTHEN, FTOV, VTOF (or CONVERT), VLFILL, VLTRIM, VLTRAIL and REPEAT options. Data records are parsed, edited, reformatted, converted and repeated according to the options specified.
- DFSORT performs processing specified with the LINES, HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS, NODETAIL, BLKCCH1, BLKCCH2, BLKCCT1 and REMOVECC options. Data records are reformatted and report records are generated for the OUTFIL data sets.
- DFSORT performs SPLIT, SPLITBY, or SPLIT1R processing. Records are distributed among the OUTFIL data sets as evenly as possible.
- DFSORT writes your OUTFIL records to the appropriate OUTFIL data sets.

Input data sets—SORTIN and SORTINnn

DFSORT processes two types of input data sets, referred to as the SORTIN data set (or just SORTIN) and the SORTINnn data sets (or just SORTINnn).

The SORTIN DD statement specifies the input data set (or concatenated input data sets) for a sort or copy application. If a SORTIN DD statement is present, it will be used by default for a sort or copy application unless you invoke DFSORT from a program with the address of an E15 user exit in the parameter list.

The SORTINnn DD statements (where nn can be 00 to 99) specify the data sets for a merge application. If a SORTINnn DD statement is present, it will be used by default for a merge application unless you invoke DFSORT from a program with the address of an E32 user exit in the parameter list.

“Data set considerations ” on page 13 contains general information about input data sets. For specific information about the SORTIN data set, see “SORTIN DD statement” on page 66. For specific information about the SORTINnn data sets, see “SORTINnn DD statement” on page 68.

Output data sets—SORTOUT and OUTFIL

DFSORT processes two types of output data sets, referred to as the SORTOUT data set (or just SORTOUT) and the OUTFIL data sets.

The SORTOUT DD statement specifies the single non-OUTFIL output data set for a sort, copy, or merge application. OUTFIL processing does not apply to SORTOUT. If a SORTOUT DD statement is present, it will be used by default for a sort, copy, or merge application unless you invoke DFSORT from a program with the address of an E35 user exit in the parameter list.

The FNAMES and FILES parameters of one or more OUTFIL statements specify the ddnames of the OUTFIL data sets for a sort, copy, or merge application. The parameters specified for each OUTFIL statement define the OUTFIL processing to be performed for the OUTFIL data sets associated with that statement. Each ddname specified must have a corresponding DD statement.

Although the ddname SORTOUT can actually be used for an OUTFIL data set, the term "SORTOUT" will be used to denote the single non-OUTFIL output data set.

“Data set considerations ” on page 13 contains general information about output data sets. For specific information about the SORTOUT data set, see “SORTOUT and OUTFIL DD statements” on page 71. For specific information about the OUTFIL data sets, see “SORTOUT and OUTFIL DD statements” on page 71 and “OUTFIL control statements” on page 219.

Chapter 3. Using DFSORT program control statements

Using program control statements

Program control statements direct DFSORT in processing your records. Some program control statements are required while others are optional. You use the control statements to:

- Indicate whether a sort, merge, or copy is performed.
- Describe the control fields to be used.
- Indicate program exits for transferring control to your own routines.
- Describe DFSORT functions you want to have invoked.
- Describe input and output files.
- Indicate various options you want to use during processing.

You can supply program control statements to DFSORT from:

- A SYSIN data set
- A SORTCNTL data set
- A DFSPARM data set
- A 24-Bit parameter list
- An extended parameter list

See Appendix B, “Specification/override of DFSORT options,” on page 801 for an explanation of when to use each source.

This chapter begins with a summary of DFSORT program control statements and coding rules. A detailed description of each statement follows.

Control statement summary

Describing the primary task

The only required program control statement in a DFSORT application is a SORT, MERGE, or OPTION statement that specifies whether you want to sort, merge, or copy records. (Copying can be specified on any of the three statements.)

SORT

Describes control fields if you are coding a sort application, or specifies a copy application. Indicates whether you want ascending or descending order for the sort.

MERGE

Describes control fields if you are coding a merge application, or specifies a copy application. Indicates whether you want ascending or descending order for the merge.

OPTION

Overrides installation defaults (such as EQUALS, CHALT, and CHECK) and supplies optional information (such as DYNALLOC and SKIPREC). Can specify a copy application.

Including or omitting records

You can specify whether certain records are included in the output data sets or omitted from them.

© Copyright IBM Corp. 1973, 2026

INCLUDE

Specifies that only records whose fields meet certain criteria are included.

For more information, see "INCLUDE control statement" on page 91 and "Including records in the output data set—comparison examples" on page 103.

OMIT

Specifies that any records whose fields meet certain criteria are deleted.

For more information, see "OMIT control statement" on page 168 and "Example" on page 170.

OUTFIL

Specifies the records to be included or omitted in multiple output data sets.

Reformatting and editing records

You can modify individual records by deleting and reordering fields and inserting blanks, zeros, or constants.

INREC

Specifies how records are reformatted before they are sorted, copied, or merged.

OUTREC

Specifies how records are reformatted after they are sorted, copied, or merged.

OUTFIL

Specifies how records are reformatted in multiple output data sets.

Producing multiple output and reports and converting records

You can produce multiple output data sets and reports, convert variable-length records to fixed-length records, and convert fixed-length records to variable-length records.

OUTFIL
Specifies the output data sets and which records are to appear in each. Specifies how records are to be converted from variable-length to fixed-length or from fixed-length to variable-length.

Joining two files

You can perform various types of "join" applications on two files (F1 and F2) by one or more keys.

JOINKEYS
One JOINKEYS statement is required for each input file to indicate the ddname of the file, describe the keys, indicate whether the file is already sorted by those keys, and so on.

JOIN
An inner join is performed by default, but a JOIN statement can be used to specify a different type of join.

REFORMAT
Describes the fields from the two files to be included in the joined records, and optionally an indicator of where the key was found ('B' for both files, '1' for file1 only or '2' for file2 only.

The three JOINKEYS application control statements (JOINKEYS, JOIN and REFORMAT) are completely described in Chapter 4, "Using a JOINKEYS application for joining two files," on page 437.

Invoking additional functions and options

You can use the remaining control statements to perform a variety of tasks.

ALTSEQ

Specifies changes to the ALTSEQ translation table to be used for SORT, MERGE, INCLUDE or OMIT fields with format AQ, for INREC, OUTREC, and OUTFIL fields with TRAN=ALTSEQ, and for INREC, OUTREC, and OUTFIL fields with format AQ in logical expressions.

DEBUG

Specifies various diagnostic options.

END

Causes DFSORT to discontinue reading SYSIN, SORTCNTL, or DFSPARM.

MODS

Specifies use of one or more user exit routines in a DFSORT application. See Chapter 5, "Using your own user exit routines," on page 465 for information about user exit routines.

RECORD

Can be used to supply length and type information.

SUM

Specifies that numeric summary fields in records with equal control fields are summed in one record and that the other records are deleted.

Using symbols

You can define and use a symbol for any field or constant in the following DFSORT control statements: INCLUDE, INREC, JOINKEYS, MERGE, OMIT, OUTFIL, OUTREC, REFORMAT, SORT and SUM. You can use a symbol for a number associated with various keywords in the following DFSORT control statements: INREC, JOINKEYS, MERGE, OPTION, OUTFIL, OUTREC and SORT. You can also use a symbol for an output column in the following DFSORT control statements: INREC, OUTFIL and OUTREC. This makes it easy to create and reuse collections of symbols (that is, mappings) representing information associated with various record layouts. You can use system symbols (for example, &JOBNAME.) in your symbol constants. You can use SET and PROC symbols in your symbol constants. See Chapter 8, "Using symbols for fields and constants," on page 681 for complete details.

General coding rules

See "Inserting comment statements" on page 83 for an explanation of how to use comment statements, blank statements, and remarks. DFSORT program control statements and EXEC PARM options can also be specified together in a user-defined DD data set. See "DFSPARM DD statement" on page 74 for special coding conventions that apply to this DD source.

All other DFSORT control statements have the same general format, shown in Figure 4 on page 80. The illustrated format does not apply to control statements you supply in a parameter list. See Chapter 6, "Invoking DFSORT from a program," on page 513 for information on the special rules that apply.

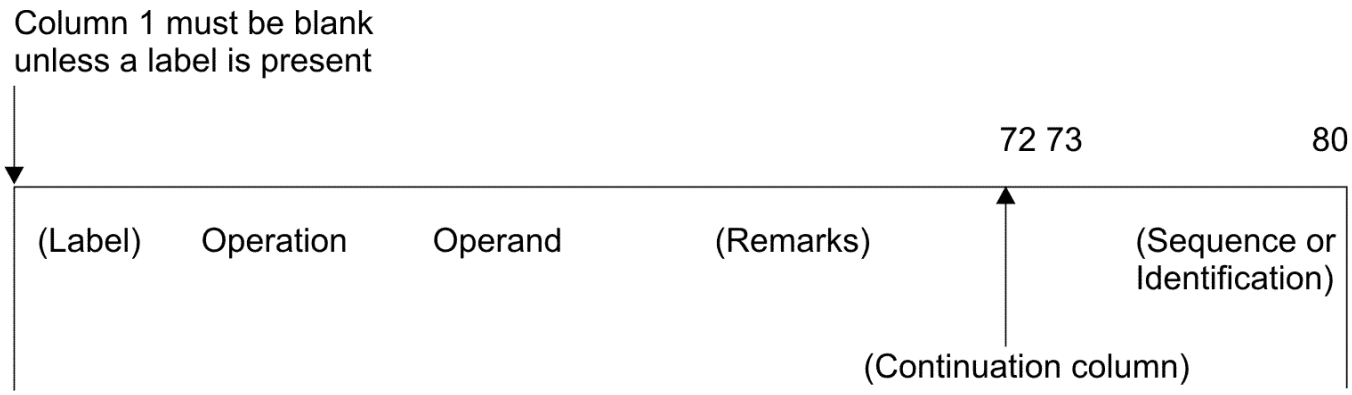


Figure 4. Control Statement Format

The control statements are free-form; that is, the operation definer, operands, and comment field can appear anywhere in a statement, provided they appear in the proper order and are separated by one or more blank characters. Column 1 of each control statement must be blank, unless the first field is a label or a comment statement (see "Inserting comment statements" on page 83).

- **Label Field**

  A label can be specified on any control statement in SYSIN or SORTCNTL. A label is never required. If present, a label must begin in column 1 with any character other than a blank or asterisk (*). A label can be 1 to 70 characters and ends when a blank character is found. Any character can be used in a label. A label followed only by blanks is printed but otherwise not processed.

  Labels cannot be specified in the parameter list, in DFSPARM or in continuation lines.

  To skip the label, specify one or more blanks starting in column 1.

  The following illustrates the use of control statements with and without labels:

```
    OPTION EQUALS
MYSORT SORT FIELDS=(5,4,CH,A)
  OUTREC FIELDS=(1,20,51,30)
OUT_1  OUTFIL FNAMES=OUT1,INCLUDE=(5,1,CH,EQ,C'A')
OUT_2  OUTFIL FNAMES=OUT2,INCLUDE=(5,1,CH,EQ,C'B')
```

- **Operation Field**

  This field can appear anywhere between column 2 and column 71 of the first line. It contains a word (for example, SORT or MERGE) that identifies the statement type to the program. In the example shown later in this section, the operation definer, SORT, is in the operation field of the sample control statement.

- **Operand Field**

  The operand field is composed of one or more operands separated by commas or semicolons. This field must follow the operation field, and be separated from it by at least one blank. No blanks are allowed within the parameters, but a blank is required at the end of all parameters. If the statement occupies more than one line, the operand must begin on the first line. Each operand has an operand definer, or parameter (a group of characters that identifies the operand type to DFSORT). A value or a list of values can be associated with a parameter. The five possible operand forms shown in this chapter are:

  – parameter. Examples: CHALT, NOCHALT, REMOVECC
  – parameter(c). Examples: DATE1(/), TIME2(:), Y2W(-)
  – parameter=value. An operand shown in the form parameter=value can also be specified in the equivalent form parameter(value) or parameter=(value). Examples: AVGRLEN=100, AVGRLEN(100), and AVGRLEN=(100) are equivalent, DYNALLOC=SYSDA, DYNALLOC(SYSDA), and DYNALLOC=(SYSDA) are equivalent, SAMPLE=20, SAMPLE(20), and SAMPLE=(20) are equivalent.

- parameter=(list). An operand shown in the form parameter=(list) can also be specified in the equivalent form parameter(list). Examples: FIELDS=(21,5,CH,A) and FIELDS(21,5,CH,A) are equivalent, BUILD=(1,20,HEX) and BUILD(1,20,HEX) are equivalent.
- parameter=(value). An operand shown in the form parameter=(value) can also be specified in the equivalent form parameter(value). Examples: DATE=(4MD/) and DATE(4MD/) are equivalent, TIMENS=(12) and TIMENS(12) are equivalent, NOMATCH=(C'***') and NOMATCH(C'***') are equivalent.

The following example illustrates the parameter, parameter=value and parameter=(list) forms.

```
SORT EQUALS,FORMAT=CH,FIELDS=(10,30,A),STOPAFT=1000
```

The following example illustrates the parameter, parameter(value), parameter(list) and parameter=(value) forms.

```
SORT EQUALS,FORMAT(CH),FIELDS(10,30,A),STOPAFT=(1000)
```

The two previous SORT statements are equivalent.

- **Remark Field**

  This field can contain any information. It is not required, but if it is present, it must be separated from the last operand field by at least one blank.

- **Continuation Column (72)**

  Any character other than a blank in this column indicates that the present statement is continued on the next line. However, as long as the last character of the operand field on a line is a comma or semicolon or colon followed by a blank, the program assumes that the next line is a continuation line. The nonblank character in column 72 is required only when a remark field is to be continued or when an operand is broken at column 71.

- **Columns 73 through 80**

  This field can be used for any purpose.

Continuation lines

The format of the DFSORT continuation line is shown in Figure 5 on page 81.

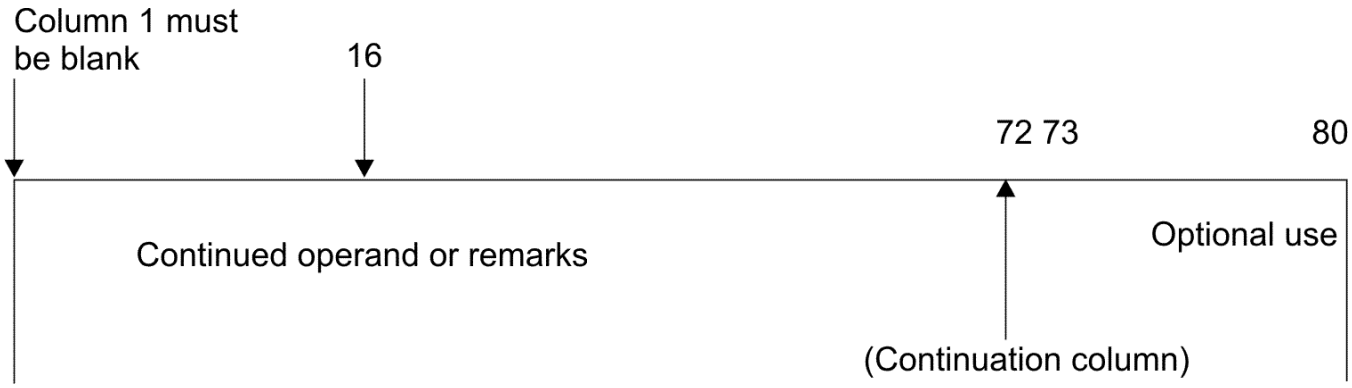


Figure 5. Continuation Line Format

The continuation column and columns 73 through 80 of a continuation line have the same purpose as they do on the first line of a control statement. Column 1 must be blank.

A continuation line is treated as a logical extension of the preceding line. Either an operand or a remark field can begin on one line (referred to as "line 1" in the bullets that follow) and continue on the next line (referred to as "line 2" in the bullets that follow). The following are the rules for continuation illustrated with examples (these different types of continuation can be intermixed):

- **Implicit continuation in 2-71:** If line 1 breaks at a comma-blank or semicolon-blank or colon-blank, DFSORT continues on line 2 with the first nonblank character it finds in columns 2-71. For example:

```
*        1         2         3         4         5         6         7
*234567890123456789012345678901234567890123456789012345678901234567890123456789012
 INCLUDE COND=(5,4,CH,EQ,
  C'ABCD')
 SORT FIELDS=(9,
              3,
              ZD,
              A)
 OUTREC FIELDS=(1,27,2X,       FIRST FIELD AND TWO BLANKS
                51,2,BI,M11,   SECOND FIELD
                60:9,3,ZD,PD)  THIRD FIELD
```

  The previous statements will be treated as if they were specified as:

```
 INCLUDE COND=(5,4,CH,EQ,C'ABCD')
 SORT FIELDS=(9,3,ZD,A)
 OUTREC FIELDS=(1,27,2X,51,2,BI,M11,60:9,3,ZD,PD)
```

- **Explicit continuation in 16:** If line 1 breaks at column 71 with a nonblank in column 72, and columns 2-15 of line 2 are blank, DFSORT continues on line 2 with whatever character it finds in column 16 (blank or nonblank). For example:

```
*        1         2         3         4         5         6         7
*234567890123456789012345678901234567890123456789012345678901234567890123456789012
                                                  INCLUDE COND=(5,4,CH,E*
               Q,C'ABCD')
                                                       SORT FIELDS=(9,3,*
               ZD,A)
                                    OUTREC FIELDS=(1,80,C'BLANK WITHIN A*
                LITERAL')
```

  The previous statements will be treated as if they were specified as:

```
 INCLUDE COND=(5,4,CH,EQ,C'ABCD')
 SORT FIELDS=(9,3,ZD,A)
 OUTREC FIELDS=(1,80,C'BLANK WITHIN A LITERAL')
```

  Attention: You should only start with a blank in column 16 of line 2 if you need a blank as the first character of the continued operand, as shown in the previous OUTREC statement. A blank in column 16 of line 2 will be included in the operand and will result in invalid syntax if incorrectly placed. For example:

```
*        1         2         3         4         5         6         7
*234567890123456789012345678901234567890123456789012345678901234567890123456789012
                                                    SORT FIELDS=(5,4,Z*
               D,A)
                                                     SUM FIELDS=(5,4,Z*
                D)
```

  The previous statements will be treated as if they were specified as:

```
 SORT FIELDS=(5,4,ZD,A)
 SUM FIELDS=(5,4,Z D)
```

  With the 'D' in column 16 of line 2, we get 'ZD' in the SORT statement. But with the 'D' in column 17 of line 2, we get 'Z D' in the SUM statement instead of 'ZD', resulting in a syntax error.

- **Explicit continuation in 2-15:** If line 1 breaks at column 71 with a nonblank in column 72, and columns 2-15 of line 2 are nonblank, DFSORT continues on line 2 with the first nonblank character it finds in columns 2-15. For example:

```
*        1         2         3         4         5         6         7
*234567890123456789012345678901234567890123456789012345678901234567890123456789012
                                          INCLUDE COND=(5,4,CH,EQ,C'AB*
   CD')
                                                       SORT FIELDS=(9,3,*
```

```
    ZD,A)
                                                  OUTREC FIELDS=(5,4,2X*
      ,9,3,ZD,M26,80:X)
```

The previous statements will be treated as if they were specified as:

```
INCLUDE COND=(5,4,CH,EQ,C'ABCD')
SORT FIELDS=(9,3,ZD,A)
OUTREC FIELDS=(5,4,2X,9,3,ZD,M26,80:X)
```

- **Remark continuation in 2-71:** If a statement ends on line 1 with a blank before column 72 and a nonblank in column 72, DFSORT treats the first nonblank character it finds in columns 2-71 of line 2 as the start of a remark. For example:

```
*        1         2         3         4         5         6         7
*2345678901234567890123456789012345678901234567890123456789012345678901234567890123456789012
   SORT FIELDS=(9,3,ZD,A)                THIS IS A                    *
                                           CONTINUED REMARK
```

Tip: A simpler way to do the same thing (without continuation) is to use a comment statement for line 2. For example:

```
*        1         2         3         4         5         6         7
*2345678901234567890123456789012345678901234567890123456789012345678901234567890123456789012
   SORT FIELDS=(9,3,ZD,A)                THIS IS A
*                                          CONTINUED REMARK
```

Inserting comment statements

- Specify comment statements by coding an asterisk (*) in column 1. A comment statement is printed along with other DFSORT program control statements but is not otherwise processed.
- A statement with blanks in columns 1 through 71 is treated as a comment statement.
- Comment statements are allowed only in the DFSPARM, SYSIN, and SORTCNTL data sets.

Coding restrictions

The following rules apply to control statement preparation:

- Operation definers and operands must be in uppercase EBCDIC.
- Column 1 of each control statement can be used only for a label or for a comment statement that begins with an asterisk in column 1.
- If present, a label must begin in column 1. Labels are allowed only in the SYSIN and SORTCNTL data sets.
- The entire operation definer must be contained on the first line of a control statement.
- The first operand must begin on the first line of a control statement. The last operand in a statement must be followed by at least one blank.
- Blanks are not allowed in operands. Anything following a blank is considered part of the remark field.
- Remarks are allowed only in the DFSPARM, SYSIN, and SORTCNTL data sets.
- Commas, semicolons, and blanks can be used only as delimiters. They can be used in values only if the values are constants.
- Each type of program control statement can appear only once within a single source (for example, the SYSIN data set).

EFS restrictions when an EFS program is in effect

In addition to the previous items, the following restrictions apply to control statement preparation for an EFS program.

- Non-DFSORT operation definers can be up to 8 bytes long.

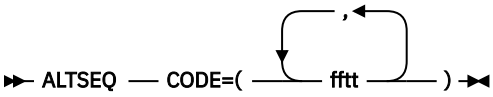
- An operation definer with no operands is allowed only if:
  - It is supplied through SYSIN, SORTCNTL, or DFSPARM.
  - It is the only operation definer on a line; column 72 must contain a blank.

Using control statements from other IBM programs

The INPFIL control statement, which is used by other IBM sort programs, is accepted but not processed. However, control statement errors can result from continuation of an INPFIL statement. The information contained in the INPFIL statement for other IBM sort programs is supplied to DFSORT with DD statements.

Because DFSORT uses the OPTION control statement, OPTION control statements in any job streams from other IBM sort programs cause DFSORT to terminate unless the parameters from the other program conform to the DFSORT OPTION control statement parameters.

ALTSEQ control statement



The ALTSEQ control statement can be used to change the alternate translation table (ALTSEQ table). Any modifications you specify are applied to the standard EBCDIC translation table. The modified ALTSEQ table overrides the installation default ALTSEQ table (the shipped default is the EBCDIC translation table).

The ALTSEQ table can be used in two ways as follows:

- To apply an alternate collating sequence for SORT, MERGE, INCLUDE, or OMIT fields, or for INREC, OUTREC, or OUTFIL fields in logical expressions with format AQ (or format CH with CHALT in effect). In this case, the ALTSEQ table is used to change only the order in which data is collated, not the data itself. If you specify AQ (or CH with CHALT) without specifying an ALTSEQ control statement, DFSORT uses the installation default ALTSEQ table.

  For example, if you want to specify that the character $ (X'5B') is to collate at position X'EA', after uppercase Z (X'E9'), you should specify:

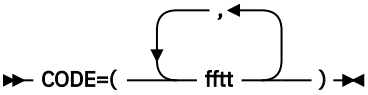
  ```
  ALTSEQ CODE=(5BEA)
  ```

- To convert characters for INREC, OUTREC, or OUTFIL OUTREC fields with TRAN=ALTSEQ. In this case, the ALTSEQ table is used to change the actual data. If you specify TRAN=ALTSEQ without specifying an ALTSEQ control statement, DFSORT uses the installation default ALTSEQ table.

  For example, if you want to change the character $ (X'5B') to the character * (X'5C'), you should specify:

  ```
  ALTSEQ CODE=(5B5C)
  ```

CODE



Specifies the original and modified EBCDIC collating positions.

ff
specifies, in hexadecimal, the character whose position is to be changed in the ALTSEQ table.

tt
specifies, in hexadecimal, the new position the character is to occupy in the ALTSEQ table.

The order in which the parameters are specified is not important.

Note:

1. If CHALT is in effect, control fields with format CH are collated using the ALTSEQ table, in addition to those with format AQ.
2. If you use locale processing for SORT, MERGE, INCLUDE, or OMIT fields, you must not use CHALT. If you need alternate sequence processing for a particular field, use format AQ.
3. Using ALTSEQ can degrade performance.

Default: Usually the installation option. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

Altering EBCDIC collating sequence—examples

Example 1

```
SORT FIELDS=(18,20,AQ,A)
ALTSEQ CODE=(5BEA)
```

The character $ (X'5B') is to collate at position X' EA', that is, after uppercase Z (X'E9').

Example 2

```
MERGE FIELDS=(25,7,A,1,10,D),FORMAT=CH
OPTION CHALT
ALTSEQ CODE=(F0B0,F1B1,F2B2,F3B3,F4B4,F5B5,F6B6,
     F7B7,F8B8,F9B9)
```

The numerals 0 through 9 are to collate before uppercase letters (but after lowercase letters).

Example 3

```
SORT FIELDS=(55,8,AQ,A)
ALTSEQ CODE=(C1F1,C2F2)
```

The uppercase A (X'C1') is to collate at the **same** position as the numeral 1 (X'F1') and the uppercase B (X'C2') is to collate at the **same** position as the numeral 2 (X'F2').

Note that this ALTSEQ statement does NOT cause collating of A before or after 1, or of B before or after 2.

Example 4

```
SORT FIELDS=(55,8,AQ,A)
ALTSEQ   CODE=(81C1,82C2,83C3,84C4,85C5,86C6,87C7,
         88C8,89C9,91D1,92D2,93D3,94D4,95D5,96D6,
         97D7,98D8,99D9,A2E2,A3E3,A4E4,A5E5,A6E6,
         A7E7,A8E8,A9E9)
```

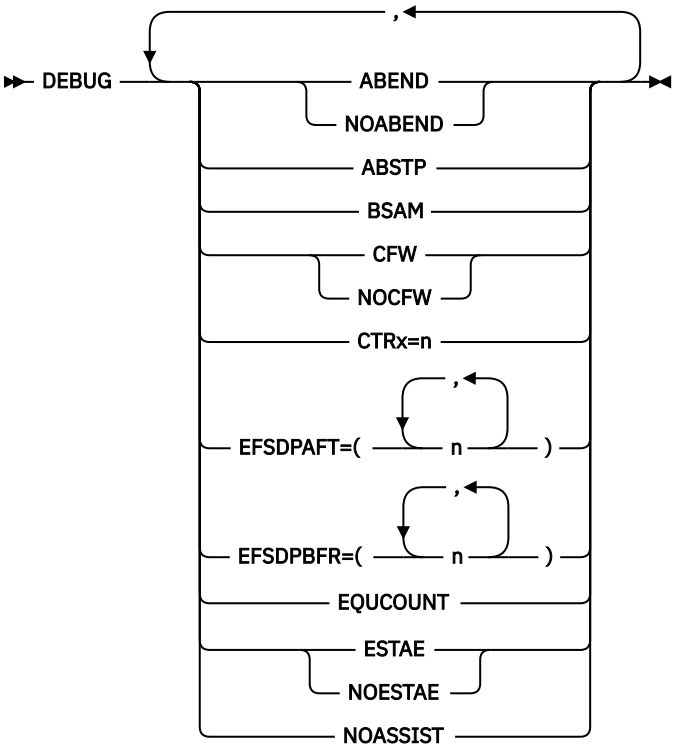
Each lowercase letter is to collate at the **same** position as the corresponding uppercase letter. For example, the lowercase a (X'81') is to collate at the same position as the uppercase A (X'C1'). This results in case-insensitive collating.

Example 5

```
OPTION COPY
ALTSEQ CODE=(0040)
OUTREC FIELDS=(1,80,TRAN=ALTSEQ)
```

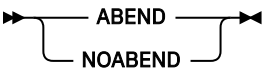
Each binary zero (X'00') is changed to a space (X'40').

DEBUG control statement



The DEBUG control statement is not intended for regular use; only ABEND, NOABEND, and BSAM are of general interest. For a tape work sort or a Conventional merge, only the ABEND or NOABEND parameters of the DEBUG statement are used. For more information about problem diagnosis, see *z/OS DFSORT Messages, Codes and Diagnosis Guide*.

ABEND or NOABEND



Temporarily overrides the ERET installation option, which specifies whether DFSORT abends or terminates with a return code of 16, if your sort, copy, or merge is unsuccessful.

ABEND
Specifies that if your sort, copy, or merge is unsuccessful, DFSORT abends with a user completion code equal to the appropriate message number or with a user-defined number between 1 and 99, as set during installation with installation option ABCODE=n.

When DEBUG ABEND is in effect, a user abend code of zero might be issued when a tape work data set sort or Conventional merge is unsuccessful.

NOABEND
Specifies that an unsuccessful sort, copy, or merge terminates with a return code of 16.

Default: Usually the installation default. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

ABSTP

Prevents loss of needed information in a dump when Blockset terminates. This option overrides ERET, ABEND, and NOABEND. If the DFSORT application is unsuccessful, an abend is forced with a completion code equal to the appropriate message number, or with the user ABEND code set with installation option ABCODE=MSG or ABCODE=n. The message is not written if NOESTAE is in effect.

Default: None; optional. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

BSAM



Temporarily bypasses the EXCP access method for input and output data sets. BSAM is ignored for VSAM input and output data sets.

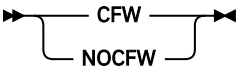
If Blockset is not selected and BSAM processing is used with concatenated SORTIN input, and both null and non-null data sets are specified, all null data sets must precede all non-null data sets; otherwise, the results are unpredictable.

Attention: This option might degrade performance.

Default: None; optional. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

CFW or NOCFW



Temporarily overrides the CFW installation option, which specifies whether DFSORT can use cache fast write when processing SORTWKdd data sets that reside on devices connected to cached 3990 control units.

CFW
Specifies that DFSORT can use cache fast write when processing SORTWKdd data sets.

NOCFW
Specifies that DFSORT cannot use cache fast write.

Attention: This option might degrade performance.

Default: Usually the installation default. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

CTRx



Keeps a count of the input and output records, and abends with code 0C1 when the count reaches n. The numbers that can be assigned to x are:

2
Counts the input records being moved from the input buffer (not used for a copy).

3
Counts the output records being moved to the output buffer (not used for a copy or merge).

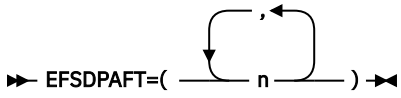
4
Counts the input records inserted by E15 (not used for Blockset).

5
Counts the output records deleted by E35 (not used for Blockset).

Default: None; optional. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

EFSDPAFT



Initiates a SNAP dump after a Major Call to an EFS program. Any combination of the numbers can be specified.

The numbers have the following meanings:

2
Takes the SNAP dump after Major Call 2 to the EFS program.

3
Takes the SNAP dump after Major Call 3 to the EFS program.

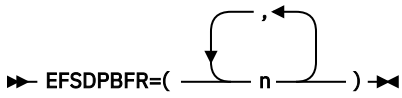
4
Takes the SNAP dump after Major Call 4 to the EFS program.

5
Takes the SNAP dump after Major Call 5 to the EFS program.

Default: None; optional. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

EFSDPBFR



Initiates a SNAP dump before a Major Call to an EFS program. Any combination of the numbers can be specified.

The numbers have the following meanings:

2
Takes the SNAP dump before Major Call 2 to the EFS program.

3
Takes the SNAP dump before Major Call 3 to the EFS program.

4
Takes the SNAP dump before Major Call 4 to the EFS program.

5
Takes the SNAP dump before Major Call 5 to the EFS program.

Default: None; optional. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

EQUCOUNT



Determines the number of records having equal keys (that is, duplicate keys) which have been sorted by the Blockset technique (printed in message ICE184I). For variable-length records, EQUCOUNT can only be used with either Hiperspace (when Hipersorting is used) or work data sets.

Note:

1. Using EQUCOUNT can degrade performance.
2. ICETOOL's UNIQUE and OCCUR operators provide unique and non-unique key reporting capabilities that may be more useful for your application than EQUCOUNT.
3. If VLSHRT is in effect, EQUCOUNT will not be used.

Default: None; optional. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

ESTAE or NOESTAE



Temporarily overrides the ESTAE installation option, which determines whether DFSORT should delete its ESTAE recovery routine early or use it for the entire run.

DFSORT normally establishes an ESTAE recovery routine at the beginning of a run. If an abend occurs and the ESTAE option is in effect, the system passes control to the recovery routine. The routine terminates the run after attempting to:

- Print additional abend information
- Continue a sort, merge, or copy application after successful SORTOUT output
- Call the EFS program at Major Calls 4 and 5 for cleanup and housekeeping
- Write an SMF record
- Call the ICETEXIT termination exit.

If an abend occurs and the ESTAE option is not in effect, these functions might not be performed.

ESTAE
specifies that DFSORT can use its ESTAE recovery routine for the entire run.

NOESTAE
specifies that DFSORT is to delete its ESTAE recovery routine at a point early in its processing. If DFSORT terminates or abends before this point is reached, it will not delete its ESTAE recovery routine; that is, NOESTAE will not be in effect.

Note: See Appendix E, “DFSORT abend processing,” on page 841 for more information on the DFSORT ESTAE recovery routine.

Default: Usually the installation default. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

NOASSIST



DFSORT uses Sorting Instructions when possible. If you do not want to use these instructions, you can temporarily bypass them by specifying this parameter.

Attention: This option might degrade performance.

Default: None; optional. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

Specifying diagnostic options—examples

Example 1

```
SORT FIELDS=(1,4,CH,A)
DEBUG EQUCOUNT
```

If the input records contain the following keys:

- KEYA, KEYA, KEYB, KEYB, KEYC, KEYD, KEYD, KEYE

the following message will be issued:

- ICE184I THE NUMBER OF RECORDS SORTED WITH EQUAL KEYS IS 3

The three equal keys are KEYA, KEYB, and KEYD.

Note: ICETOOL's UNIQUE and OCCUR operators provide full equal key reporting capabilities and should be used instead of EQUCOUNT.

Example 2

```
SORT FIELDS=(12,2,BI,D)
DEBUG BSAM,ABEND
```

Directs DFSORT to use the BSAM access method for the SORTIN and SORTOUT data sets and to abend if the sort application is unsuccessful.

END control statement



The END control statement allows DFSORT to discontinue reading SYSIN, DFSPARM, or SORTCNTL before end of file (EOF).

When you link-edit user exit routines dynamically, the END statement marks the end of the DFSORT control statements and the beginning of exit routine object decks in SYSIN.

Discontinue reading control statements—examples

Example 1

```
//SYSIN DD *
  SORT FIELDS=(1,6,A,28,5,D),FORMAT=CH
  RECORD TYPE=V,LENGTH=(200,,,,80)
  END
  OPTION DYNALLOC
```

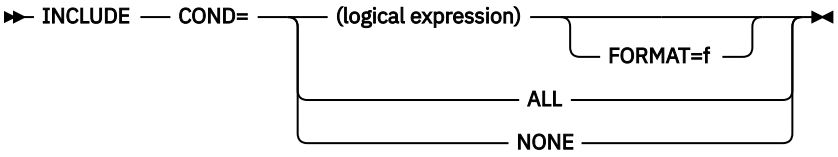
Because the OPTION statement appears after the END statement, it is not read.

Example 2

```
//SYSIN DD *
  SORT FIELDS=(5,8,CH,A)
  MODS E15=(E15,1024,SYSIN,T)
  END
←object deck for E15 user exit here→
```

The END statement precedes the E15 user exit routine object deck in SYSIN.

INCLUDE control statement

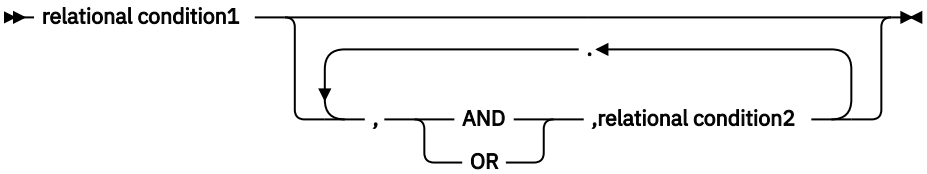


Use an INCLUDE statement if you want only certain records to appear in the output data set. The INCLUDE statement selects the records you want to include.

You can specify either an INCLUDE statement or an OMIT statement in the same DFSORT run, but not both.

The way in which DFSORT processes short INCLUDE/OMIT compare fields depends on the settings for VLSCMP/NOVLSCMP and VLSHRT/NOVLSHRT. A short field is one where the variable-length record is too short to contain the entire field, that is, the field extends beyond the record. For details about including or omitting short records, see the discussion of the VLSCMP and NOVLSCMP options in “OPTION control statement” on page 171.

A logical expression is one or more relational conditions logically combined, based on fields in the input record, and can be represented at a high level as follows:



If the logical expression is true for a given record, the record is included in the output data set.

Seven types of relational conditions can be used as follows:

1. **Comparisons:**

   Compare two compare fields or a compare field and a decimal, hexadecimal, character, or current, future, or past date constant.

   For example, you can compare the first 6 bytes of each record with its last 6 bytes, and include only those records in which those fields are identical. Or you can compare a date field with today's date, yesterday's date, or tomorrow's date, and include records accordingly.

   See “Comparisons” on page 94 for information about comparisons.

2. **Substring Comparison Tests:**

   You can search for field values using one of the combinations:

   - A constant within a field value.
   - Regular expression within a field value.
   - Field value within a constant.

For example, you can search the value in a 6-byte field for the character constant C'OK', and include only those records for which C'OK' is found somewhere in the field. Or you can search the character constant C'J69,L92,J82' for the value in a 3-byte field, and include only those records for which C'J69', C'L92', or C'J82' appears in the field.

See "Substring comparison tests" on page 105 for information about substring comparison tests.

3. **Bit Logic Tests:**

   Test the state (on or off) of selected bits in a binary field using a bit or hexadecimal mask or a bit constant.

   For example, you can include only those records which have bits 0 and 2 on in a 1-byte field. Or you can include only those records which have bits 3 and 12 on and bits 6 and 8 off in a 2-byte field.

   See "Bit logic tests" on page 110 for information about bit logic tests.

4. **Date Comparisons:**

   Compare a two-digit year date field to a two-digit year date constant, a current, future or past two-digit year date, or to another two-digit year date field, using the century window in effect.

   For example, you can include only those records for which a Z'yymm' date field is between January 1996 and March 2005. Or you can include only those records for which a P'dddyy' field is less than another P'dddyy' field. Or you can include only those records for which a C'yyddd' field is between today's date and 5 days earlier than today's date.

   See "Date comparisons" on page 115 for information about date comparisons.

5. **Numeric Tests:**

   Test a field for numerics or non-numerics in character, zoned decimal or packed decimal format.

   For example, you can include only those records in which a 5-byte field contains only '0'-'9' characters (that is, numerics). Or you can include only those records in which a 9-byte field contains invalid ZD numeric data (that is, non-numerics). Or you can include only those records in which a 12-byte field contains valid PD numeric data (that is, numerics).

   See "Numeric tests" on page 119 for information about numeric tests.

6. **Alphanumeric Tests:**

   Test a field for alphanumerics or non-alphanumerics in character format. Various combinations of uppercase characters (A-Z), lowercase characters (a-z) and numeric characters (0-9) can be used.

   For example, you can include only those records in which a 10-byte field contains only 'A'-'Z' characters (that is, uppercase characters) or '0'-'9' characters (that is, numeric characters). Or you can include only those records in which a 20-byte field contains characters other than 'a'-'z' (that is, lowercase characters).

   See "Alphanumeric tests" on page 120 for information about alphanumeric tests.

7. **Unicode Comparisons:**

   Compare a field in Unicode format to another field in Unicode format. For example, you can include only those records for which a UTF8 field is less than another UTF8 field. Or you can exclude those records for which a UTF16 field is equal to another UTF16 field. Or you can include only those records for which a UTF32 field is greater than another UTF32 field.

   See "Unicode comparisons" on page 122 for information about Unicode comparisons.

By nesting relational conditions within parentheses, you can create logical expressions of higher complexity.

Although comparisons, substring comparison tests, bit logic tests, date comparisons, numeric tests, and alphanumeric tests are explained separately later in this section for clarity, they can be combined to form logical expressions.

The INCLUDE control statement differs from the INCLUDE parameter of the OUTFIL statement in the following ways:

- The INCLUDE statement applies to all input records; the INCLUDE parameter applies only to the OUTFIL input records for its OUTFIL group.
- FORMAT=f can be specified with the INCLUDE statement but not with the INCLUDE parameter. Thus, you can use FORMAT=f and p,m or p,m,f fields with the INCLUDE statement, but you must only use p,m,f fields with the INCLUDE parameter. For example:

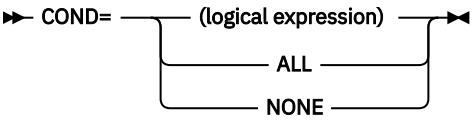
```
INCLUDE FORMAT=BI,
   COND=(5,4,LT,11,4,OR,21,4,EQ,31,4,OR,
      61,20,SS,EQ,C'FLY')

OUTFIL INCLUDE=(5,4,BI,LT,11,4,BI,OR,21,4,BI,EQ,31,4,BI,OR,
      61,20,SS,EQ,C'FLY')
```

- D2 format can be specified with the INCLUDE statement but not with the INCLUDE parameter.

See "OUTFIL control statements" on page 219 for more details on the OUTFIL INCLUDE parameter.

COND



logical expression
specifies one or more relational conditions logically combined, based on fields in the input record. If the logical expression is true for a given record, the record is included in the output data sets.

ALL
specifies that all of the input records are to be included in the output data sets.

NONE
specifies that none of the input records are to be included in the output data sets.

Default: ALL. See Appendix B, "Specification/override of DFSORT options," on page 801 for full override details.

Applicable Functions: See Appendix B, "Specification/override of DFSORT options," on page 801.

FORMAT



FORMAT=f can be used to specify a particular format for one or more compare fields. f from FORMAT=f is used for p,m fields. f from FORMAT=f is ignored for p,m,f fields. For example, the following are all equivalent:

```
INCLUDE COND=(5,5,ZD,EQ,12,3,PD,OR,21,3,PD,NE,35,5,ZD)
INCLUDE FORMAT=ZD,COND=(5,5,EQ,12,3,PD,OR,21,3,PD,NE,35,5)
INCLUDE COND=(5,5,ZD,EQ,12,3,OR,21,3,NE,35,5,ZD),FORMAT=PD
```

The permissible field formats for comparisons are shown in Table 10 on page 95. SS (substring) is the only permissible field format for substring comparison tests. BI (unsigned binary) is the only permissible field format for bit logic tests. The Y2x formats are the only permissible field formats for date comparisons. The FS or CSF (floating sign), ZD (zoned decimal), and PD (packed decimal) formats are the only permissible field formats for numeric tests. BI (unsigned binary) is the only permissible format for alphanumeric tests.

Default: None. FORMAT=f must be specified if any field is specified as p,m rather than p,m,f. See Appendix B, "Specification/override of DFSORT options," on page 801 for full override details.

Applicable Functions: See Appendix B, "Specification/override of DFSORT options," on page 801.

Note: DFSORT issues an informational message and ignores FORMAT=f if all of the fields are specified as p,m,f.

Relational condition

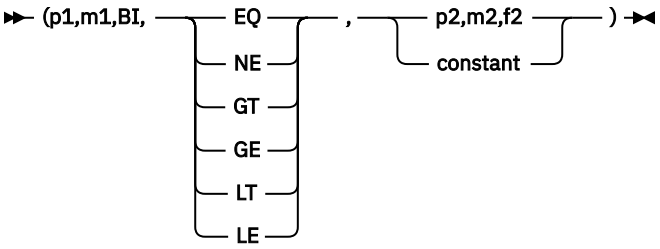
A relational condition specifies a comparison, substring comparison test, bit logic test, date comparison, numeric test or alphanumeric test to be performed. Relational conditions can be logically combined, with AND or OR, to form a logical expression. If they are combined, the following rules apply:

- AND statements are evaluated before OR statements unless parentheses are used to change the order of evaluation; expressions inside parentheses are always evaluated first. (Nesting of parentheses is limited only by the amount of storage available.)
- The symbols & (AND) and | (OR) can be used instead of the words.

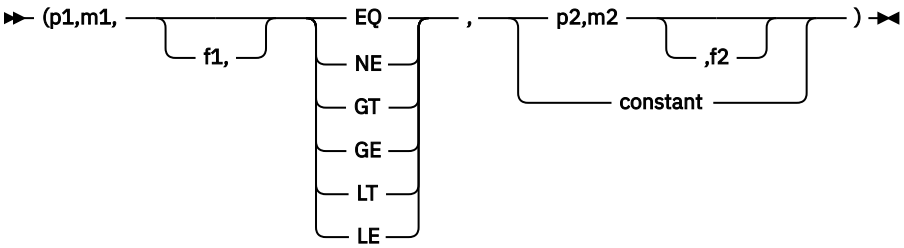
Comparisons

Relational condition format

Two formats for the relational condition can be used:



Or, if the FORMAT=f operand is used:



Comparison operators are as follows:

EQ
Equal to

NE
Not equal to

GT
Greater than

GE
Greater than or equal to

LT
Less than

LE
Less than or equal to.

Fields

p1,m1,f1

These variables specify a field in the input record to be compared either to another field in the input record or to a constant.

- p1 specifies the first byte of the compare field relative to the beginning of the input record.[4] The first data byte of a fixed-length record (FLR) has relative position 1. The first data byte of a variable-length (VLR) record has relative position 5 (because the first 4 bytes contain the record descriptor word). All compare fields must start on a byte boundary, and no compare field can extend beyond byte 32752.
- m1 specifies the length of the compare field. Acceptable lengths for different formats are in Table 10 on page 95.
- f1 specifies the format of the data in the compare field. Permissible formats are given in Table 10 on page 95.

  You can use p1,m1 rather than p1,m1,f1 if you use FORMAT=f to supply the format for the field.

Table 10. Compare field formats and lengths

Format Code	Length	Description
CH	1 to 256 bytes	Character [5]
AQ	1 to 256 bytes	Character with alternate collating sequence
ZD	1 to 256 bytes	Signed zoned decimal
PD	1 to 255 bytes	Signed packed decimal
PD0	2 to 8 bytes	Packed decimal with sign and first digit ignored
FI	1 to 256 bytes	Signed fixed-point
BI	1 to 256 bytes	Unsigned binary
AC	1 to 256 bytes	ASCII character
CSF or FS	1 to 32 bytes	Signed numeric with optional leading floating sign
UFF	1 to 44 bytes	Unsigned free form numeric
SFF	1 to 44 bytes	Signed free form numeric
CSL or LS	2 to 256 bytes	Signed numeric with leading separate sign
CST or TS	2 to 256 bytes	Signed numeric with trailing separate sign
CLO or OL	1 to 256 bytes	Signed numeric with leading overpunch sign
CTO or OT	1 to 256 bytes	Signed numeric with trailing overpunch sign
ASL	2 to 256 bytes	Signed ASCII numeric with leading separate sign

[4] If your E15 user exit routine formats the record, p1 must refer to the record as reformatted by the exit.

Table 10. Compare field formats and lengths (continued)

Format Code	Length	Description
AST	2 to 256 bytes	Signed ASCII numeric with trailing separate sign
D2	1 to 256 bytes	User-defined data type (requires an EFS program)
AUF	1 to 44 bytes	Unsigned ASCII free form numeric
ASF	1 to 44 bytes	Signed ASCII free form numeric

Note: See Appendix C, "Data format descriptions," on page 819 for detailed format descriptions.

p2,m2,f2

These variables specify another field in the input record with which the p1,m1,f1 field will be compared. Permissible comparisons between compare fields with different formats are shown in Table 11 on page 96 and Table 12 on page 96.

AC, ASL, and AST formats sequence EBCDIC data using the ASCII collating sequence.

You can use p2,m2 rather than p2,m2,f2 if you use FORMAT=f to supply the format for the field.

Table 11. Permissible field-to-field comparisons for INCLUDE/OMIT (Group 1)

Field Format	BI	CH	ZD	PD	FI	CSF or FS	UFF	SFF	CSL or LS	CST or TS	AUF	ASF
BI	X	X										
CH	X	X										
ZD			X	X								
PD			X	X								
FI					X							
CSF or FS						X	X	X	X	X	X	X
UFF						X	X	X	X	X	X	X
SFF						X	X	X	X	X	X	X
CSL or LS						X	X	X	X	X	X	X
CST or TS						X	X	X	X	X	X	X
AUF						X	X	X	X	X	X	X
ASF						X	X	X	X	X	X	X

Table 12. Permissible field-to-field comparisons for INCLUDE/OMIT (Group 2)

Field Format	PD0	AC	ASL	AST	CLO or OL	CTO or OT	AQ	D2
PD0	X							
AC		X						
ASL			X	X				

[5] If CHALT is in effect, CH is treated as AQ.

Table 12. Permissible field-to-field comparisons for INCLUDE/OMIT (Group 2) (continued)

Field Format	PD0	AC	ASL	AST	CLO or OL	CTO or OT	AQ	D2
AST			X	X				
CLO or OL					X	X		
CTO or OT					X	X		
AQ							X	
D2								X

Note: D2 field formats are user-defined.

Constants

A constant can be a decimal number (n, +n, -n), character string (C'xx...x'), or hexadecimal string (X'yy...yy').

The **current date** can be used as a decimal number (DATE1P, &DATE1P, DATE2P, &DATE2P, DATE3P, &DATE3P) or character string (DATE1, &DATE1, DATE1(c), &DATE1(c), DATE2, &DATE2, DATE2(c), &DATE2(c), DATE3, &DATE3, DATE3(c), &DATE3(c), DATE4, &DATE4, DATE5, &DATE5).

A **future date** can be used as a decimal number (DATE1P+d, &DATE1P+d, DATE2P+m, &DATE2P+m, DATE3P+d, &DATE3P+d) or character string (DATE1+d, &DATE1+d, DATE1(c)+d, &DATE1(c)+d, DATE2+m, &DATE2+m, DATE2(c)+m, &DATE2(c)+m, DATE3+d, &DATE3+d, DATE3(c)+d, &DATE3(c)+d).

A **past date** can be used as a decimal number (DATE1P-d, &DATE1P-d, DATE2P-m, &DATE2P-m, DATE3P-d, &DATE3P-d) or character string (DATE1-d, &DATE1-d, DATE1(c)-d, &DATE1(c)-d, DATE2-m, &DATE2-m, DATE2(c)-m, &DATE2(c)-m, DATE3-d, &DATE3-d, DATE3(c)-d, &DATE3(c)-d).

The different constants are explained in detail in the following table. Permissible comparisons between compare fields and constants are shown in Table 13 on page 97.

Table 13. Permissible field-to-constant comparisons for INCLUDE/OMIT

Field format	Self-defining term		
	Decimal number	Character string	Hexadecimal string
BI	X	X	X
CH		X	X
ZD	X		
PD	X		
PD0			X
FI	X		
AC		X	X
ASL	X		
AST	X		
CSF or FS	X		
UFF	X		
SFF	X		
CSL or LS	X		
CST or TS	X		
CLO or OL	X		

Table 13. Permissible field-to-constant comparisons for INCLUDE/OMIT (continued)

Field format	Self-defining term		
	Decimal number	Character string	Hexadecimal string
CTO or OT	X		
AQ		X	X
D2	X	X	X

Note: D2 field formats are user-defined.

Decimal number format

The format for coding a decimal constant is:

[±]**n**

When an FI field is compared with a decimal constant, n or +n cannot be larger than +9223372036854775807 and -n cannot be smaller than -9223372036854775808.

When a BI field is compared with a decimal constant, n or +n cannot be larger than +18446744073709551615 nor smaller than +0. A BI field cannot be compared to a negative number (-n). A BI field cannot be compared to -0 even if NOSZERO is in effect.

Examples of valid and invalid decimal constants are:

Table 14. Valid and invalid decimal constants

Valid	Invalid	Explanation
15	++15	Too many sign characters
+15	15+	Sign in wrong place
-15	1.5	Contains invalid character
18000000	1,500	Contains invalid character

Current date as decimal number

DATE1P, &DATE1P, DATE2P, &DATE2P, DATE3P, or &DATE3P can be used to generate a decimal number for the current date of the run.

Table 15 on page 98 shows the form of the decimal number constant generated for each current date operand along with an example of the actual decimal number generated when the date of the run is June 21, 2005. yyyy represents the year, mm represents the month (01-12), dd represents the day (01-31) and ddd represents the day of the year (001-366).

Table 15. Decimal numbers for current date

Format of operand	Format of constant	Example of constant
DATE1P	+yyyymmdd	+20050621
DATE2P	+yyyymm	+200506
DATE3P	+yyyyddd	+2005172

Note: You can precede each of the operands in the table with an & with identical results.

Future date as decimal number

DATE1P+d, &DATE1P+d, DATE2P+m, &DATE2P+m, DATE3P+d, or &DATE3P+d can be used to generate a decimal number for a future date relative to the current date of the run. d is days in the future and m is months in the future. d and m can be 0 to 9999.

Table 16 on page 99 shows the form of the decimal number constant generated for each future date operand along with an example of the actual decimal number generated when the date of the run is June 21, 2005. yyyy represents the year, mm represents the month (01-12) , dd represents the day (01-31) and ddd represents the day of the year (001-366).

Table 16. Decimal numbers for future dates

Operand	Format of constant	Example of operand	Example of constant
DATE1P+d	+yyyymmdd	DATE1P+11	+20050702
DATE2P+m	+yyyymm	DATE2P+2	+200508
DATE3P+d	+yyyyddd	DATE3P+200	+2006007

Note: You can precede each of the operands in the table with an & with identical results.

Past date as decimal number

DATE1P-d, &DATE1P-d, DATE2P-m, &DATE2P-m, DATE3P-d, or &DATE3P-d can be used to generate a decimal number for a past date relative to the current date of the run. d is days in the past and m is months in the past. d and m can be 0 to 9999.

Table 17 on page 99 shows the form of the decimal number constant generated for each past date operand along with an example of the actual decimal number generated when the date of the run is June 21, 2005. yyyy represents the year, mm represents the month (01-12), dd represents the day (01-31) and ddd represents the day of the year (001-366).

Table 17. Decimal numbers for past dates

Format of operand	Format of constant	Example of operand	Example of constant
DATE1P-d	+yyyymmdd	DATE1P-30	+20050522
DATE2P-m	+yyyymm	DATE2P-12	+200406
DATE3P-d	+yyyyddd	DATE3P-172	+2004366

Note: You can precede each of the operands in the table with an & with identical results.

Character string format

The format for coding a character string constant is:

```
C'xx...x'
```

The value x may be any EBCDIC character (the EBCDIC character string is translated appropriately for comparison to an AC or AQ field). You can specify up to 256 characters.

If you want to include a single apostrophe in the character string, you must specify it as two single apostrophes. Thus:

```
Required:  O'NEILL  Specify: C'O''NEILL'
```

Examples of valid and invalid character string constants are shown in Table 18 on page 100:

Table 18. Valid and invalid character string constants		
Valid	**Invalid**	**Explanation**
C'JDCO'	C'''''	Apostrophes not paired
C'$@#'	'ABCDEF'	C identifier missing
C'+0.193'	C'ABCDEF	Apostrophe missing
C'Frank''s'	C'Frank's'	Two single apostrophes needed for one

Double-byte data may be used in a character string for INCLUDE/OMIT comparisons. The start of double-byte data is delimited by the shift-out (SO) control character (X'0E'), and the end by the shift-in (SI) control character (X'0F'). SO and SI control characters are part of the character string and must be paired with zero or an even number of intervening bytes. Nested shift codes are not allowed. All characters between SO and SI must be valid double-byte characters. No single-byte meaning is drawn from the double-byte data.

Examples of valid and invalid character string constants containing double-byte characters are shown in Table 19 on page 100 using:

- < to represent SO
- > to represent SI
- Dn to represent a double-byte character

Table 19. Valid and invalid strings with double-byte data		
Valid	**Invalid**	**Explanation**
C'Q<D1D2>T'	C'Q<R>S'	Single-byte data within SO/SI
C'<D1D2D3>'	C'D1D2D3'	Missing SO/SI; treated as single-byte data
C'Q<D1>R<D2>'	C'Q<D1<D2>>'	Nested SO/SI

Tip: X'0E', X'0F', and X'7D' are treated as the special characters shift-out, shift-in, and single apostrophe in a character string. If you don't want to treat one or more of these characters as special in a particular value, use a hexadecimal string instead of a character string. For example, if you want to treat the binary value 000E0E7D as its decimal equivalent of 921213, use X'000E0E7D'; 0E will not be treated as shift-out and 7D will not be treated as a single apostrophe.

Current date as character string

DATE1, &DATE1, DATE1(c), &DATE1(c), DATE2, &DATE2, DATE2(c), &DATE2(c), DATE3, &DATE3, DATE3(c), &DATE3(c), DATE4, &DATE4, DATE5 or &DATE5 can be used to generate a character string for the current date of the run.

Table 20 on page 100 shows the form of the character string constant generated for each current date operand along with an example of the actual character string generated when the date of the run is June 21, 2005 at 04:42:45 PM, using (/) for (c) where relevant. *yyyy* represents the year, mm (for date) represents the month (01-12), *dd* represents the day (01-31), *ddd* represents the day of the year (001-366), *hh* represents the hour (00-23), mm (for time) represents the minutes (00-59), *ss* represents the seconds (00-59), *nnnnnn* represents the microseconds (000000-999999) and *c* can be any character except a blank.

Table 20. Character strings for current date		
Operand	**Format of Constant**	**Example of Constant**
DATE1	C'yyyymmdd'	C'20050621'
DATE1(c)	C'yyyycmmcdd'	C'2005/06/21'

Table 20. Character strings for current date (continued)

Operand	Format of Constant	Example of Constant
DATE2	C'yyyymm'	C'200506'
DATE2(c)	C'yyyycmm'	C'2005/06'
DATE3	C'yyyyddd'	C'2005172'
DATE3(c)	C'yyyycddd'	C'2005/172'
DATE4	C'yyyy-mm-dd-hh.mm.ss'	C'2005-06-21-16.52.45'
DATE4	C'yyyy-mm-dd-hh.mm.ss'	C'2005-06-21-16.52.45'
DATE5	C'yyyy-mm-dd-hh.mm.ss.nnnnnn'	C'2005-06-21-16.52.45.582013'

You can precede each of the operands in the table with an & with identical results.

Tip: When a field is shorter than the character string it is compared to, DFSORT truncates the string on the right. You can take advantage of this to compare a field to only part of the DATE4 timestamp when appropriate. For example:

```
INCLUDE COND=(1,13,CH,GT,DATE4)
```

would compare the field in positions 1-13 to the truncated DATE4 constant `C'yyyy-mm-dd-hh'`.

Future date as character string

DATE1+d, &DATE1+d, DATE1(c)+d, &DATE1(c)+d, DATE2+m, &DATE2+m, DATE2(c)+m, &DATE2(c)+m, DATE3+d, &DATE3+d, DATE3(c)+d or &DATE3(c)+d can be used to generate a character string for a future date relative to the current date of the run. d is days in the future and m is months in the future. d and m can be 0 to 9999.

Table 21 on page 101 shows the form of the character string constant generated for each future date operand along with an example of the actual character string generated when the date of the run is June 21, 2005. yyyy represents the year, mm represents the month (01-12), dd represents the day (01-31), ddd represents the day of the year (001-366), and c can be any character except a blank.

Table 21. Character set

Operand	Format of constant	Example of operand	Example of constant
DATE1+d	C'yyyymmdd'	DATE1+11	C'20050702'
DATE1(c)+d	C'yyyycmmcdd'	DATE1(/)+90	C'2005/09/19'
DATE2+m	C'yyyymm'	DATE2+2	C'200508'
DATE2(c)+m	C'yyyycmm'	DATE2(.)+25	C'2007.07'
DATE3+d	C'yyyyddd'	DATE3+200	C'2006007'
DATE3(c)+d	C'yyyycddd'	DATE3(-)+1	C'2005-171'

Note: You can precede each of the operands in the table with an & with identical results.

Past date as character string

DATE1-d, &DATE1-d, DATE1(c)-d, &DATE1(c)-d, DATE2-m, &DATE2-m, DATE2(c)-m, &DATE2(c)-m, DATE3-d, &DATE3-d, DATE3(c)-d or &DATE3(c)-d can be used to generate a character string for a past date relative to the current date of the run. d is days in the past and m is months in the past. d and m can be 0 to 9999.

Table 22 on page 102 shows the form of the character string constant generated for each past date operand along with an example of the actual character string generated when the date of the run is June

21, 2005. yyyy represents the year, mm represents the month (01-12), dd represents the day (01-31), ddd represents the day of the year (001-366), and c can be any character *except* a blank.

Table 22. Character strings for past dates

Operand	Format of constant	Example of operand	Example of constant
DATE1-d	C'yyyymmdd'	DATE1-1	C'20050620'
DATE1(c)-d	C'yyyycmmcdd'	DATE1(-)-60	C'2005-04-22'
DATE2-m	C'yyyymm'	DATE2-6	C'200412'
DATE2(c)-m	C'yyyycmm'	DATE2(/)-1	C'2005/05'
DATE3-d	C'yyyyddd'	DATE3-300	C'2004238'
DATE3(c)-d	C'yyyycddd'	DATE3(.)-21	C'2005.151'

Note: You can precede each of the operands in the table with an & with identical results.

Hexadecimal string format

The format for coding a hexadecimal string constant is:

X'**yy...yy**'

The value yy represents any pair of hexadecimal digits. You can specify up to 256 pairs of hexadecimal digits.

Because the first digit and sign are ignored in a PD0 field, you should not include the first digit or sign in a hexadecimal constant to be compared to a PD0 field. For example, 3-byte PD0 values like X'01234C' and X'01234D' would be equal to a hexadecimal constant of X'1234'.

Examples of valid and invalid hexadecimal constants are shown in the following table.

Table 23. Valid and invalid hexadecimal constants

Valid	Invalid	Explanation
X'ABCD'	X'ABGD'	Invalid hexadecimal digit
X'BF3C'	X'BF3'	Incomplete pair of digits
X'AF050505'	'AF050505'	Missing X identifier
X'BF3C'	'BF3C'X	X identifier in wrong place

Padding and truncation

In a field-to-field comparison, the shorter compare field is padded appropriately. In a field-to-constant comparison, the constant is padded or truncated to the length of the compare field.

Character and hexadecimal strings are truncated and padded on the right.

The padding characters are:

- X'40' For a character string
- X'00' For a hexadecimal string.

Decimal constants are padded and truncated on the left. Padding is done with zeros in the proper format.

Cultural environment considerations

DFSORT's collating behavior can be modified according to your cultural environment. The cultural environment is established by selecting the active locale. The active locale's collating rules affect INCLUDE and OMIT processing as follows:

- DFSORT includes or omits records for output according to the collating rules defined in the active locale. This provides inclusion or omission for single- or multi-byte character data, based on defined collating rules which retain the cultural and local characteristics of a language.

If locale processing is to be used, the active locale will only be used to process character (CH) compare fields and character and hexadecimal constants compared to character (CH) compare fields.

For more information on locale processing, see “Cultural environment considerations ” on page 5 or LOCALE in “OPTION control statement” on page 171.

Including records in the output data set—comparison examples

Example 1

```
INCLUDE  COND=(5,8,GT,13,8,|,105,4,LE,1000),FORMAT=CSF
```

This example illustrates how to only include records in which:

- The floating sign number in bytes 5 through 12 is greater than the floating sign number in bytes 13 through 20

  OR

- The floating sign number in bytes 105 through 108 is less than or equal to 1000.

Note that all three compare fields have the same format.

Example 2

```
INCLUDE   COND=(1,10,CH,EQ,C'STOCKHOLM',
          AND,21,8,ZD,GT,+50000,
          OR,31,4,CH,NE,C'HERR')
```

This example illustrates how to only include records in which:

- The first 10 bytes contain STOCKHOLM (this nine-character string was padded on the right with a blank) AND the zoned-decimal number in bytes 21 through 28 is greater than 50 000

  OR

- Bytes 31 through 34 do not contain HERR.

Note that the AND is evaluated before the OR. (“Omitting records from the output data set—example” on page 170 illustrates how parentheses can be used to change the order of evaluation.) Also note that ending a line with a comma or semicolon followed by a blank indicates that the parameters continue on the next line, starting in any position from columns 2 through 71.

Example 3

```
INCLUDE  FORMAT=CH,
  COND=((5,1,EQ,8,1),&,
        ((20,1,EQ,C'A',&,30,1,FI,GT,10),|,
         (20,1,EQ,C'B',&,30,1,FI,LT,100),|,
         (20,1,NE,C'A',&,20,1,NE,C'B')))
```

This example illustrates how to only include records in which:

- Byte 5 equals byte 8

  AND

- At least one of the following is true:
  - Byte 20 equals 'A' and byte 30 is greater than 10
  - Byte 20 equals 'B' and byte 30 is less than 100
  - Byte 20 is not equal to 'A' or 'B'.

Note that p,m,FI is used for the FI fields, and p,m with FORMAT=CH is used for all of the CH fields. With FORMAT=f, you can mix p,m and p,m,f fields when that's convenient such as when all or most of the fields have the same format (although you can always code p,m,f for all fields and not use FORMAT=f, if you prefer).

Example 4

```
INCLUDE COND=(7,2,CH,EQ,C'T1',OR,
              (1,2,BI,GE,X'001A',AND,20,2,CH,EQ,25,2,CH))
```

This example shows the effects of VLSCMP/NOVLSCMP and VLSHRT/NOVLSHRT on INCLUDE processing when short records are present.

Consider the records shown in Figure 6 on page 104:

- If VLSCMP is in effect, the first record is included because bytes 7-8 are equal to C'T1', even though the comparison of bytes 20-21 to 25-26 involves short fields. The second record is included or omitted based on the comparison of bytes 20-21 to bytes 25-26.
- If NOVLSCMP and VLSHRT are in effect, the first record is omitted because the comparison of bytes 20-21 to 25-26 involves short fields. The second record is included or omitted based on the comparison of bytes 20-21 to bytes 25-26.
- If NOVLSCMP and NOVLSHRT are in effect, the first record causes message ICE015A or ICE218A to be issued because the comparison of bytes 20-21 to bytes 25-26 involves short fields.

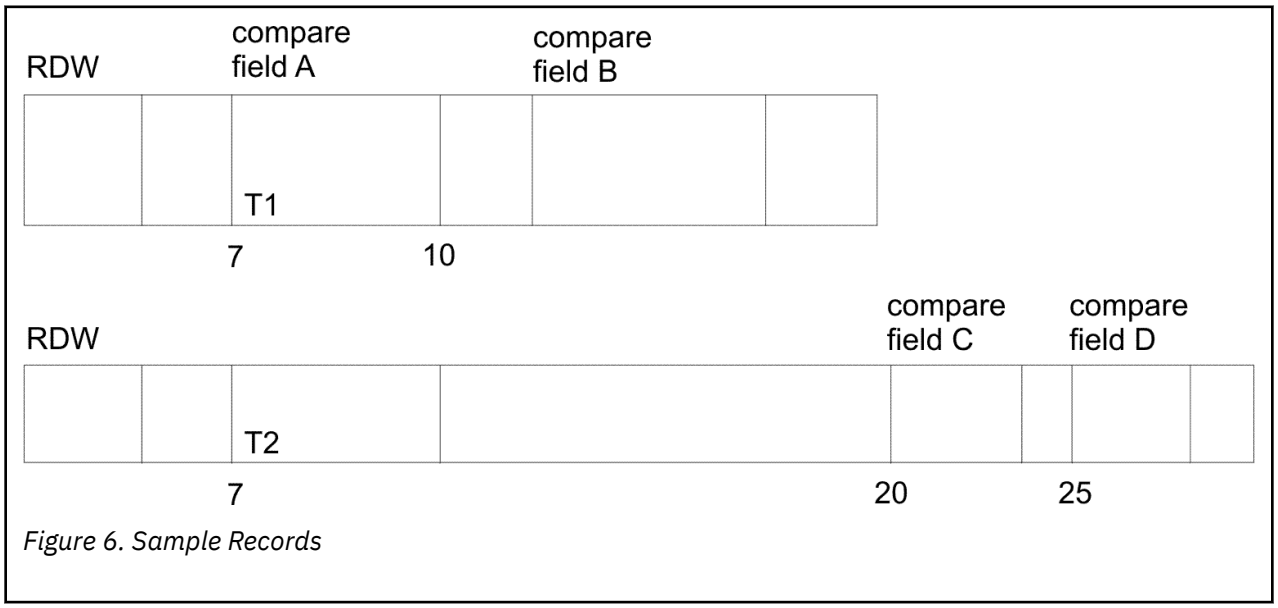


Figure 6. Sample Records

Example 5

```
INCLUDE COND=(21,8,ZD,GT,DATE1P)
```

This example illustrates how to include records in which a zoned-decimal date of the form Z'yyyymmdd' in bytes 21-28 is greater than today's date. DATE1P generates a decimal number for the current date in the form +yyyymmdd.

Example 6

```
INCLUDE COND=(15,7,CH,GE,DATE3-7,AND,15,7,CH,LE,DATE3+7)
```

This example illustrates how to include records in which a character date of the form C'yyyyddd' in bytes 15-21 is between 7 days in the past and 7 days in the future, relative to the current date. DATE3-7 generates a character constant in the form C'yyyyddd' where yyyyddd is the current date minus 7 days. DATE3+7 generates a character constant in the form C'yyyyddd' where yyyyddd is the current date plus 7 days.

Example 7

```
INCLUDE COND=(21,10,CH,GE,DATE1(-)-365)
```

This example illustrates how to include records in which a character date of the form C'yyyy-mm-dd' in bytes 21-30 is within 365 days of the current date. DATE1(-)-365 generates a character constant in the form C'yyyy-mm-dd' where yyyymmdd is the current date minus 365 days.

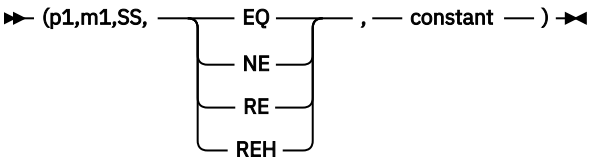
Substring comparison tests

Four types of substring comparison tests are offered, as follows:

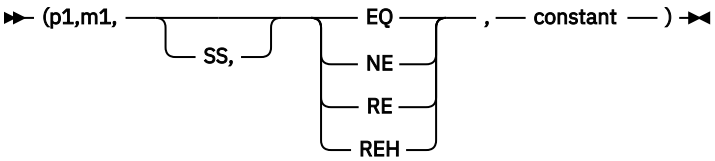
1. Find a constant within a field value. For example, you can search the value in a 6-byte field for the character constant C'OK'. If the field value is, for example, C'**OK**' or C'****OK', the relational condition is true; if the field value is C'**ERR*', the relational condition is false.
2. Find a field value within a constant. For example, you can search the character constant C'J69,L92,J82' for the value in a 3-byte field. If the field value is C'J69', C'L92', or C'J82', the relational condition is true; if the field value is C'X24', the relational condition is false. Note that the comma is used within the constant to separate the valid 3-character values; any character that will not appear in the field value can be used as a separator in the constant.
3. Find a field value or a constant within a field value using Regular expressions. For example, you can search the value in a 20 byte field for the character constant C'(iP).*[0-9]'. If the field value for example C'IP' or C'ip' or C'Ip' or C'iP' and a numeric 0-9, the relational condition is true; if the field value is C'It', the relational condition is false.
4. Find a field value or a constant within a field value using Regular expressions with hexadecimal strings. For example, you can search the value in a 20 byte field for the character constant C'(.*Z+)|(.*\XC2\XF1.*)'. If the field value for example C'Fuzzy' or C'AnZ}' or X'C2F1', the relational condition is true; if the field value is C'EAN', the relational condition is false.

Relational condition format

Four formats for the relational condition can be used:



Or, if the FORMAT=SS operand is used:



Restriction: FORMAT=SS can precede COND but cannot follow it.

Substring comparison operators are as follows:

EQ
Equal to

NE
Not equal to

RE
Regular expressions

REH
Regular expressions with hexadecimal string

Fields

p1,m1

These variables specify the character field in the input record for the substring test.

- p1 specifies the first byte of the character input field for the substring test, relative to the beginning of the input record.[6] The first data byte of a fixed-length record (FLR) has relative position 1. The first data byte of a variable-length (VLR) record has relative position 5 (because the first 4 bytes contain the record descriptor word). All fields to be tested must start on a byte boundary and must not extend beyond byte 32752.
- m1 specifies the length of the field to be tested. The length can be 1 to 32752 bytes.

Constant

The constant can be a character string or a hexadecimal string. See "Character string format" on page 99 and "Hexadecimal string format" on page 102 for details.

If m1 is greater than the length of the constant, the field value will be searched for the constant and the condition will be true if a match is found when the EQ comparison operator is specified or if a match is not found when the NE comparison operator is specified.

If m1 is smaller than the length of the constant, the constant will be searched for the field value and the condition will be true if a match is found when the EQ comparison operator is specified or if a match is not found when the NE comparison operator is specified.

Regular expressions

A *regular expression* is a pattern which specifies a string of characters used to match certain strings. DFSORT only treats extended regular expressions as *case-insensitive*. The input regular expression can be in any case (lowercase, uppercase or mixed case). DFSORT compares the NULL-terminated string specified by INCLUDE/OMIT field against the compiled regular expression. For comparison DFSORT only adds NULL byte to the specified regular expression. The specified regular expression is then compiled and if an error is detected, DFSORT terminates with an error message.

This support is based on the following Standards/Extensions:

- XPG4 (X/Open Common Applications Environment Specification, System Interfaces and Headers, Issue 4.)
- XPG4.2 (X/Open Common Applications Environment Specification, System Interfaces and Headers, Issue 4, Version 2.)
- Single UNIX Specification, Version 3 (IEEE Std 1003.1-2001.)
- z/OS UNIX (functions that provide z/OS UNIX support beyond the defined standards.)
- The default code page (CCSID) is IBM-1047.

[6] If your E15 user exit routine formats the record, p1 must refer to the record as reformatted by the exit.

Regular expressions can be made up of normal characters or special characters, sometimes called metacharacters.

The extended regular expression metacharacters supported are:

```
| ¬ $ . * + ? ( ) [ { } \ ]
```

Table 24. Regular expression metacharacters

Symbol	Description
.	The period symbol matches any one character except the terminal newline character.
[character–character]	The hyphen symbol, within square brackets, means "through." It fills in the intervening characters according to the current collating sequence. For example, [a–z] can be equivalent to [abc...xyz] or [aAbBcC...xXyYzZ].
[string]	A string within square brackets specifies any of the characters in string. Thus [abc], if compared to other strings, would match any that contained a, b, or c.
{n} {n,} {n,u}	Integer values enclosed in {} indicate the number of times to apply the preceding regular expression. n is the minimum number, and u is the maximum number. If you specify only n, it indicates the exact number of times to apply the regular expression. {n,} is equivalent to {n,u}. They both match n or more occurrences of the expression.
*	The asterisk symbol indicates 0 or more of any characters. For example, [a*e] is equivalent to any of the following: 99ae9, aaaaae, a999e99.
^	The caret symbol matches the beginning of the string.
$	The dollar symbol matches the end of the string.
character+	The plus symbol specifies one or more occurrences of a character. Thus, smith+ern is equivalent to, for example, smithhhern.
[^string]	The caret symbol, when inside square brackets, negates the characters within the square brackets. Thus [^abc], if compared to other strings, would fail to match any that contains even one a, b, or c.
(expression)	Groups a sub-expression allowing an operator, such as *, +, or [].], to work on the sub-expression enclosed in parentheses. For example, (a*(cb+)*)$0.

Note:

1. Do not use multibyte characters.
2. You can use the] (right square bracket) alone within a pair of square brackets, but only if it immediately follows either the opening left square bracket or if it immediately follows [^. For example: []–] matches the] and – characters.
3. All the preceding symbols are special. You precede them with \ to use the symbol itself. For example, a\.e is equivalent to a.e.

4. You can use the – (hyphen) by itself, but only if it is the first or last character in the expression. For example, the expression []--0] matches either the] or else the characters – through 0. Otherwise, use \–.

The following patterns are given as examples, along with descriptions of what they match:

abc
Matches any record containing the three letters abc in that order.

a.c
Matches any string beginning with the letter a, followed by any character, followed by the letter c.

^.$
Matches any record containing exactly one character.

.* [a–z]+ .*
Matches any record containing a word, consisting of lowercase/uppercase/mixed case alphabetic characters, delimited by at least one space on each side.

Johny.*Johny
Matches any record containing at least two occurrences of the string Johny.

^ibm
Matches records beginning with "ibm"

ibm$
Matches records ending with "ibm"

^ibm$
Matches records with exactly "ibm"

Regular expressions application notes

Can only be used if Blockset technique is selected.

The following existing DFSORT functions are not compatible when used with Regular expressions:

- Locale
- EFS
- COBOL E15 and E35 user exits with COPY
- COBOL E15 user exit when regular expression is used in INCLUDE/OMIT control statement
- COBOL E35 user exit when regular expression is used in OUTFIL's INCLUDE/OMIT control statement.
- For a JOINKEYS application, no more than one task can use regular expressions. Regular expressions can only be used in the main task or subtask 1 or subtask 2, and not in any combination of tasks.

Regular expressions application examples

Example 1: Include all records that contain string 'Department'(case-insensitive) using Regular expressions:

```
//STEP0100 EXEC PGM=SORT
//SYSOUT   DD SYSOUT=*
//SORTDIAG DD DUMMY
//SORTIN   DD *
lower case              florida department of Palm trees
UPPER CASE                 FLORIDA DEPARTMENT OF ORANGE GROVES
exclude this record     California state
alternate case            FlOrIdA DePaRtMeNt Of PaLm TrEeS
sentence                     Florida Department Of Revenue
Mixed case              ARIZONA DEPARTment of Motor Vehicles
//SORTOUT  DD SYSOUT=*
//SYSIN    DD *
  OPTION COPY
  INCLUDE COND=(25,50,SS,RE,C'DEPARTMENT')
//*
```

Using the string C'department' or C'DePaRtMeNt' will also produce the same exact result.

This example illustrates how to include only records in which:

- DEPARTMENT (case-insensitive) is found somewhere within bytes 25 through 74
- Using the string C’department’ or C'DePaRtMeNt' will also produce the same exact results.

The Output from the above job is:

```
lower case              florida department of Palm trees
UPPER CASE                 FLORIDA DEPARTMENT OF ORANGE GROVES
alternate case           FlOrIdA DePaRtMeNt Of PaLm TrEeS
sentence                    Florida Department Of Revenue
Mixed case              ARIZONA DEPARTment of Motor Vehicles
```

Example 2: Omit records which contain one or more occurrences of a character or hex strings using Regular expressions.

```
//STEP0100 EXEC PGM=SORT
//SYSOUT   DD SYSOUT=*
//SORTIN   DD *
record - 1 1234500\x00a ANZ]'' string CD0
record - 2 12345111EAND'45678901234567890
record - 3 123452289012RRAA"890....567890
record - 4 1234533\x00a SIMPLE string CD0
record - 5 12345441EANx000!            B1
record - 6 123456789012345fizZZ
record - 7 12345678901234567890123456789O
/*
//SORTOUT  DD SYSOUT=*
//SYSIN    DD *
  OPTION COPY
  OUTFIL OMIT=(25,032,SS,REH,C'(.*Z+)|(.*\XC2\XF1.*)')
//*
```

This example illustrates how to exclude records in which:

- One or more character ‘z’ (Case-insensitive) is followed by any character OR the records that contain hex values of X’C2F1’ which is equivalent to characters ‘B1’ are found somewhere within bytes 25 through 56.

The output from this job is:

```
record - 2 12345111EAND'45678901234567890
record - 3 123452289012RRAA"890....567890
record - 4 1234533\x00a SIMPLE string CD0
record - 7 12345678901234567890123456789O
```

Example 3: Include all records that contain string 'ER'(case-insensitive) followed by a numeric (0-9) using Regular expressions.

```
//STEP0100 EXEC PGM=SORT
//SYSOUT   DD SYSOUT=*
//SORTIN   DD *
MORGAN STANLEY
 ERIC
      HOLMER CA 90201
  WALTER  ALBUQUERQUENEW MEXICO
TONY NY 10000
 JERRY 10029 NEW YORK
SHERLOCK
//SORTOUT  DD SYSOUT=*
//SYSIN    DD *
  OPTION COPY
  INCLUDE COND=(1,50,SS,RE,C'(ER).*[0-9]')
//*
```

This example illustrates how to include only records in which:

- ER (case-insensitive) is found somewhere within bytes 1 through 50 and have a number (0-9) following the string ER.

The output from this job is:

```
    HOLMER CA 90201
   JERRY 10029 NEW YORK
```

Example 4: Show the usage of Regular expressions with JOINKEYS application.

```
//STEP0100 EXEC PGM=SORT
//SYSOUT   DD SYSOUT=*
//INA      DD DISP=SHR,DSN=Your.input.file1
//INB      DD DISP=SHR,DSN=Your.input.file2
//SORTOUT  DD SYSOUT=*
//SYSIN    DD *
  OPTION COPY
  JOINKEYS F1=F1,FIELDS=(10,4,A)
  JOINKEYS F2=F2,FIELDS=(10,4,A)
  JOIN UNPAIRED,F1,F2
  REFORMAT FIELDS=(F1:10,4,F2:10,4,F1:3,6,F2:3,6)
/*
//JNF1CNTL DD *
  OMIT COND=(10,4,SS,RE,C'8D[0-9A-F]{2,2}')
/*
//JNF2CNTL DD *
  OMIT COND=(10,4,CH,EQ,C' ')
/*
```

Note:

Regular expressions can only be used in the main task or subtask 1 or subtask 2, and not in any combination of tasks.

Including records in the output data set—substring comparison example

Example

```
  INCLUDE FORMAT=SS,COND=(11,6000,EQ,C'OK',OR,5,3,EQ,C'J69,L92,J82')
```

This example illustrates how to include only records in which:

- OK is found somewhere within bytes 11 through 6010

  OR

- Bytes 5 through 7 contain J69, L92 or J82.

Bit logic tests

Two methods for bit logic testing are offered as follows:

- Bit operator with hexadecimal or bit mask
- Bit comparison tests

While any bit logic test can be specified using either of the two methods, each of them offers unique advantages not found with the other.

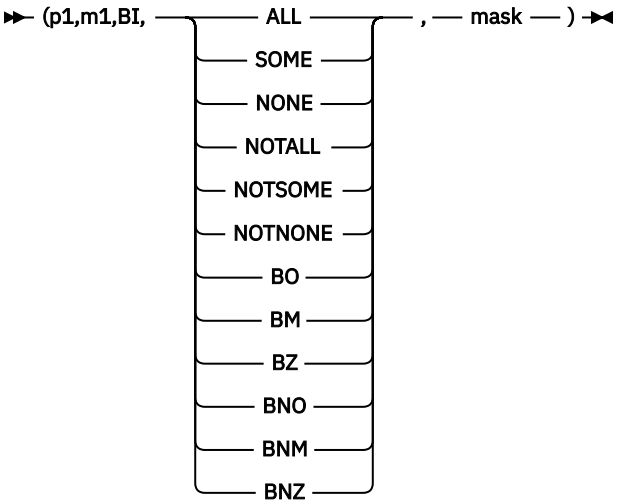
The ability to specify selected bits in a field, by either of the two methods, can greatly reduce the number of INCLUDE conditions that must be specified to achieve a given result, because the need to account for unspecified bits is eliminated.

Method 1: Bit operator tests

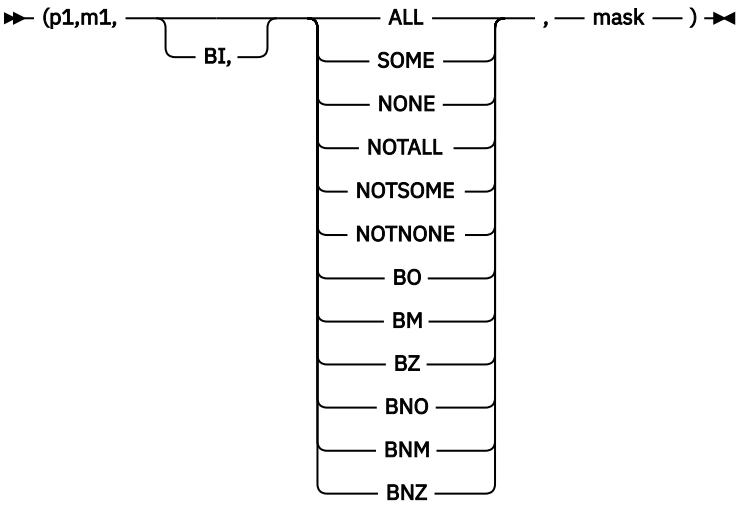
This method of bit logic testing allows you to test whether selected bits in a binary field are all on, all off, in a mixed on-off state, or in selected combinations of these states. While this method allows you to test many different possible bit combinations with a single operation, similar to the Test Under Mask (TM) machine instruction, it is less suited to determine if a field contains exactly one particular combination of on and off bits than Method 2 described later in this section.

Relational condition format

Two formats for the relational condition can be used:



Or, if the FORMAT=BI operand is used:



Bit operators describe the input field to mask relationship to be tested as follows:

ALL or BO
All mask bits are on in the input field

SOME or BM
Some, but not all mask bits are on in the input field

NONE or BZ
No mask bits are on in the input field

NOTALL or BNO
Some or no mask bits are on in the input field

NOTSOME or BNM
All or no mask bits are on in the input field

NOTNONE or BNZ
All or some mask bits are on in the input field

The first set of operators (ALL, SOME, and so on) are intended for those who like meaningful mnemonics. The second set of operators (BO, BM, and so on) are intended for those familiar with the conditions associated with the Test Under Mask (TM) instruction.

Fields

p1,m1

These variables specify the binary field in the input record to be tested against the mask.

- p1 specifies the first byte of the binary input field to be tested against the mask, relative to the beginning of the input record.[7] The first data byte of a fixed-length record (FLR) has relative position 1. The first data byte of a variable-length (VLR) record has relative position 5 (because the first 4 bytes contain the record descriptor word). All fields to be tested must start on a byte boundary and must not extend beyond byte 32752.
- m1 specifies the length of the field to be tested. The length can be 1 to 256 bytes.

Mask

A hexadecimal string or bit string that indicates the bits in the field selected for testing. If a mask bit is on (1), the corresponding bit in the field is tested. If a mask bit is off (0), the corresponding bit in the field is ignored.

Hexadecimal string format

The format for coding a hexadecimal string mask is:

X'**yy...yy**'

The value *yy* represents any pair of hexadecimal digits that constitute a byte (8 bits). Each bit must be 1 (test bit) or 0 (ignore bit). You can specify up to 256 pairs of hexadecimal digits.

Bit string format

The format for coding a bit string mask is:

B'**bbbbbbbb...bbbbbbbb**'

The value bbbbbbbb represents 8 bits that constitute a byte. Each bit must be 1 (test bit) or 0 (ignore bit). You can specify up to 256 groups of 8 bits. The total number of bits in the mask must be a multiple of 8. A bit mask string can only be used with a bit operator.

Padding and truncation

The hexadecimal or bit mask is truncated or padded on the right to the byte length of the binary field. The padding character is X'00' (all bits off and thus not tested).

Including records in the output data set—bit operator test examples

Example 1

```
  INCLUDE   COND=(27,1,CH,EQ,C'D',AND,18,1,BI,ALL,B'10000000')
```

This example illustrates how to only include records in which:

[7] If your E15 user exit routine formats the record, p1 must refer to the record as reformatted by the exit.

- Byte 27 contains D

  AND

- Byte 18 has bit 0 on.

Example 2

```
INCLUDE   COND=(11,1,BI,BM,X'85')
```

This example illustrates how to only include records in which byte 11 has some, but not all of bits 0, 5 and 7 on. Results for selected field values are shown as follows:

Table 25. Bit comparison example 2: Results for selected field values

Value	11,1,BI result	Action
X'85'	False	Omit Record
X'C1'	True	Include Record
X'84'	True	Include Record
X'00'	False	Omit Record

Example 3

```
INCLUDE   COND=(11,2,ALL,B'0001001000110100',
                OR,21,1,NONE,B'01001100'),FORMAT=BI
```

This example illustrates how to only include records in which:

- Bytes 11 through 12 have all of bits 3, 6, 10, 11 and 13 on

  OR

- Byte 21 has none of bits 1, 4, or 5 on.

Results for selected field values are shown as follows:

Table 26. Bit comparison example 3: Results for selected field values

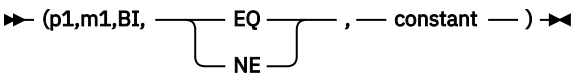
Value	11,2,BI 4esult	21,1,BI value	21,1,BI result	Action
X'1234'	True	X'4C'	False	Include Record
X'02C4'	False	X'81'	True	Include Record
X'0204'	False	X'40'	False	Omit Record
X'F334'	True	X'00'	True	Include Record
X'1238'	False	X'4F'	False	Omit Record

Method 2: Bit comparison tests

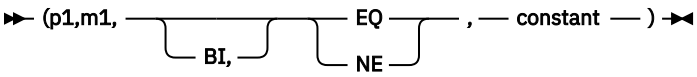
This method of bit logic testing allows you to test whether selected bits in a binary field are either in an exact pattern of on and off bits, or not in that exact pattern. Unlike Method 1 described previously, only "equal" and "unequal" comparisons are allowed; however, this method has the advantage of being able to test for a precise combination of on and off bits.

Relational condition format

Two formats for the relational condition can be used:



Or, if the FORMAT=BI operand is used:



Bit comparison operators are as follows:

EQ
Equal to

NE
Not equal to

Fields

p1,m1

These variables specify the binary field in the input record to be compared to the bit constant.

- p1 specifies the first byte of the binary input field to be compared to the bit constant, relative to the beginning of the input record.[8] The first data byte of a fixed-length record (FLR) has relative position 1. The first data byte of a variable-length (VLR) record has relative position 5 (because the first 4 bytes contain the record descriptor word). All fields to be tested must start on a byte boundary and must not extend beyond byte 32752.
- m1 specifies the length of the field to be tested. The length can be 1 to 256 bytes.

Bit constant

A bit string constant that specifies the pattern to which the binary field is compared. If a bit in the constant is 1 or 0, the corresponding bit in the field is compared to 1 or 0, respectively. If a bit in the constant is . (period), the corresponding bit in the field is ignored.

Bit string format

The format for coding a bit string constant is:

B'**bbbbbbbb...bbbbbbbb**'

The value bbbbbbbb represents 8 bits that constitute a byte. Each bit must be 1 (test bit for 1), 0 (test bit for 0) or . (ignore bit). You can specify up to 256 groups of 8 bits. The total number of bits in the mask must be a multiple of 8. A bit constant can only be used for bit comparison tests (BI format and EQ or NE operator).

Padding and truncation

The bit constant is truncated or padded on the right to the byte length of the binary field. The padding character is B'00000000' (all bits equal to 0). Note that the padded bytes are compared to the excess bytes in the binary field.

Recommendation: To ensure that comparison of the padded bytes to the excess bytes in the binary field does not cause unwanted results, shorten the field length to eliminate the padding characters, or increase the length of the bit constant to specify the exact test pattern you want.

[8] If your E15 user exit routine formats the record, p1 must refer to the record as reformatted by the exit.

Including records in the output data set—bit comparison test examples

Example 1

```
INCLUDE   COND=(27,1,CH,EQ,C'D',AND,18,1,BI,EQ,B'1.......')
```

This example illustrates how to only include records in which:

- Byte 27 contains D

  AND
- Byte 18 is equal to the specified pattern of bit 0 on.

Example 2

```
INCLUDE   COND=(11,1,BI,NE,B'10...1.1')
```

This example illustrates how to only include records in which byte 11 is not equal to the specified pattern of bit 0 on, bit 1 off, bit 5 on and bit 7 on. Results for selected field values are shown :

Table 27. Bit Comparison example 2: Results for selected field values

Value	11,1,BI result	Action
X'85'	False	Omit Record
X'C1'	True	Include Record
X'84'	True	Include Record
X'97'	False	Omit Record

Example 3

```
INCLUDE   COND=(11,2,EQ,B'..01....0......1',
               OR,21,1,EQ,B'01......'),FORMAT=BI
```

This example illustrates how to only include records in which:

- Bytes 11 through 12 are equal to the specified pattern of bit 2 off, bit 3 on, bit 8 off and bit 15 on

  OR
- Byte 21 is equal to the specified pattern of bit 0 off and bit 1 on.

Results for selected field values are shown in Table 28 on page 115

Table 28. Bit comparison example 3: Results for selected field values

Value	11,2,BI 4esult	21,1,BI value	21,1,BI 4esult	Action
X'1221'	True	X'C0'	False	Include Record
X'02C4'	False	X'41'	True	Include Record
X'1234'	False	X'00'	False	Omit Record
X'5F7F'	True	X'7F'	True	Include Record
X'FFFF'	False	X'2F'	False	Omit Record

Date comparisons

You can use DFSORT's Y2 formats in conjunction with the century window in effect, as follows:

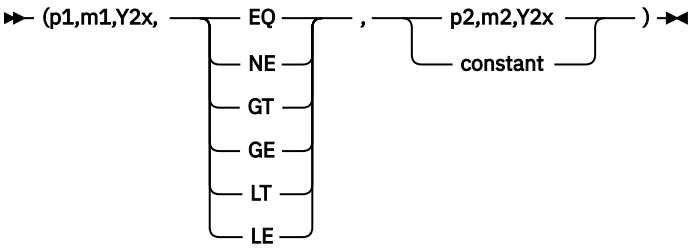
- Use the full date formats (Y2T, Y2U, Y2V, Y2W, Y2X and Y2Y) to compare a two-digit year date field to a two-digit year date constant, a current, future or past two-digit year date (Y constant), or to another two-digit year date field.
- Use the year formats (Y2C, Y2Z, Y2S, Y2P, Y2D and Y2B) to compare a two-digit year field to a two-digit year constant (Y constant) or to another two-digit year field.

For example, you can include only those records for which a Z'yymm' date field is between January 1996 and March 2005. Or you can include only those records for which a P'dddyy' field is less than another P'dddyy' field. Or you can include only those records for which a C'yyddd' field is between today's date and 5 days earlier than today's date.

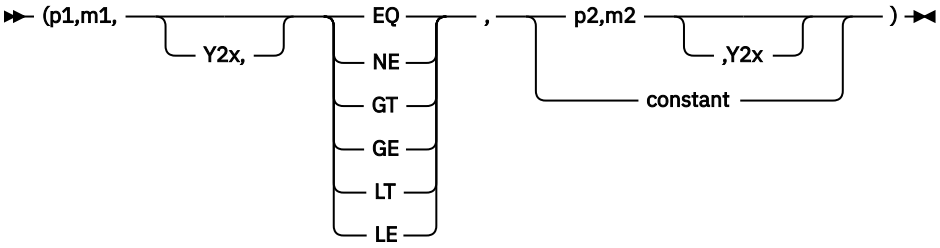
The ordering of dates and special indicators used for comparisons with Y2 fields and Y constants is the same as the ascending orders for sorting and merging Y2 fields (see “SORT control statement” on page 421 for details).

Relational condition format

Two formats for the relational condition can be used:



Or, if the FORMAT=Y2x operand is used:



Comparison operators are as follows:

EQ
Equal to

NE
Not equal to

GT
Greater than

GE
Greater than or equal to

LT
Less than

LE
Less than or equal to.

Fields

p1,m1,Y2x

These variables specify a two-digit year date field in the input record to be compared either to another two-digit year date field in the input record or to a two-digit year date constant.

- p1 specifies the first byte of the date field relative to the beginning of the input record.[9] The first data byte of a fixed-length record (FLR) has relative position 1. The first data byte of a variable-length (VLR) record has relative position 5 (because the first 4 bytes contain the record descriptor word). All date fields must start on a byte boundary, and no date field can extend beyond byte 32752.
- m1 specifies the length of the date field. Appendix C, “Data format descriptions,” on page 819 describes the length and format for each type of date field.
- Y2x specifies the Y2 format. Appendix C, “Data format descriptions,” on page 819 describes the length (m) and format (Y2x) for each type of date field.

You can use p1,m1 rather than p1,m1,Y2x if you use FORMAT=Y2x to supply the format for the date field.

p2,m2,Y2x

These variables specify another two-digit year date field in the input record with which the p1,m1,Y2x field will be compared.

You can use p2,m2 rather than p2,m2,Y2x if you use FORMAT=Y2x to supply the format for the date field.

Constant

A two-digit year date constant in the form Y'string', Y'DATE1', Y'DATE1'+d, Y'DATE1'-d, Y'DATE2', Y'DATE2'+m, Y'DATE2'-m, Y'DATE3', Y'DATE3'+d, or Y'DATE3'-d, with which the p1,m1,Y2x field will be compared.

Comparisons

A date field can be compared to a date constant or another date field with the same number of non-year (x) digits. Table 29 on page 117 shows the type of field-to-field and field-to-constant comparisons you can use. The fields shown for any type of date (for example, yyx and xyy) can be compared to any other fields shown for that type of date or to the Y constant shown for that type of date.

Table 29. Permissible comparisons for dates

Type of Date	Fields (m,f)		Y constant
yyx and xyy	3,Y2T 3,Y2W	2,Y2U 2,Y2X	Y'yyx'
yyxx and xxyy	4,Y2T 4,Y2W	3,Y2V 3,Y2Y	Y'yyxx' Y'DATE2' Y'DATE2'+m Y'DATE2'-m
yyxxx and xxxyy	5,Y2T 5,Y2W	3,Y2U 3,Y2X	Y'yyxxx' Y'DATE3' Y'DATE3'+d Y'DATE3'-d

[9] If your E15 user exit routine formats the record, p1 must refer to the record as reformatted by the exit.

Table 29. Permissible comparisons for dates (continued)

Type of Date	Fields (m,f)		Y constant
yyxxxx and xxxxyy	6,Y2T 6,Y2W	4,Y2V 4,Y2Y	Y'yyxxxx' Y'DATE1' Y'DATE1'+d Y'DATE1'-d
yy	2,Y2C 2,Y2S 1,Y2D	2,Y2Z 2,Y2P 1,Y2B	Y'yy'

Y constants for current, future, and past two-digit year dates are as follows. d can be 0 to 9999 days and m can be 0 to 9999 months.

- Y'DATE1' generates a Y constant for the current date in the form Y'yymmdd'
- Y'DATE1'+d generates a Y constant for the current date plus d days in the form Y'yymmdd'
- Y'DATE1'-d generates a Y constant for the current date minus d days in the form Y'yymmdd'
- Y'DATE2' generates a Y constant for the current date in the form Y'yymm'
- Y'DATE2'+m generates a Y constant for the current date plus m months in the form Y'yymm'
- Y'DATE2'-m generates a Y constant for the current date minus m months in the form Y'yymm'
- Y'DATE3' generates a Y constant for the current date in the form Y'yyddd'
- Y'DATE3'+d generates a Y constant for the current date plus d days in the form Y'yyddd'
- Y'DATE3'-d generates a Y constant for the current date minus d days in the form Y'yyddd'.

You must use the same number of digits in a Y constant as the type of date; leading zeros must be specified (for example, for Y'yymm', use Y'0001' for January 2000 and Y'0501' for January 2005).

You can also use Y constants for special indicators as follows:

- Y'0...0' (CH/ZD/PD zeros) and Y'9...9' (CH/ZD/PD nines) can be used with Y2T, Y2U, Y2V, Y2W, Y2X and Y2Y dates. You must use the same number of digits as the type of date (for example, Y'000' for yyq or qyy, Y'0000' for yymm or mmyy, and so forth).
- Y'LOW' (BI zeros), Y'BLANKS' (blanks) and Y'HIGH' (BI ones) can be used with Y2T, Y2W and Y2S dates.

Including records in the output data set—date comparisons

Example 1

```
INCLUDE FORMAT=Y2T,
  COND=(3,4,GE,Y'9901',AND,
        3,4,LE,Y'0312',OR,
        3,4,LE,Y'0000')
```

This example illustrates how to only include records in which:

- A C'yymm' date field in bytes 3 through 6 is between January 1999 and December 2003

  OR

- Bytes 3 through 6 contain CH zeros (C'0000'), ZD zeros (Z'0000') or BI zeros (X'00000000').

Note that the century window in effect will be used to interpret the Y'9901' and Y'0312' date constants, as well as real dates in the C'yymm' date field. However, the century window will not be used to interpret the Y'0000' special indicator constant or special indicators in the C'yymm' date field.

Example 2

```
INCLUDE COND=(2,3,Y2X,LT,36,5,Y2T)
```

This example illustrates how to only include records in which a P'dddyy' date field in bytes 2 through 4 is less than a Z'yyddd' date field in bytes 36 through 40.

Note that the century window in effect will be used to interpret real dates in the P'dddyy' and Z'yyddd' date fields. However, the century window will not be used to interpret special indicators in the P'dddyy' and Z'yyddd' date fields.

Numeric tests

You can test a field for numerics or non-numerics in character, zoned decimal or packed decimal format.

For example, you can include only those records in which a 5-byte field contains only '0'-'9' characters (that is, character numerics). Or you can include only those records in which a 9-byte field contains invalid zoned decimal data (that is, zoned decimal non-numerics). Or you can include only those records in which a 12-byte field contains valid packed decimal data (that is, packed decimal numerics).

A field to be tested for numerics in character format looks like this in hexadecimal:

```
FdFd...Fd
```

The field is considered to be character numeric if every d is 0-9. (This is equivalent to '0'-'9' for each character.) Otherwise, the field is considered to be character non-numeric. For example, '1234', '0001' and '9999' are all considered to be character numeric, whereas 'A234', '12.3', ' 1' and '123D' are all considered to be character non-numeric.

A field to be tested for numerics in zoned decimal format looks like this in hexadecimal:

```
zdzd...sd
```

The field is considered to be zoned decimal numeric if every z is F, every d is 0-9, and s is C, D or F. Otherwise, the field is considered to be zoned decimal non-numeric. For example, '1234' (X'F1F2F3F4'), '123D' (X'F1F2F3C4') and '123M' (X'F1F2F3D4') are all considered to be zoned decimal numeric, whereas 'A234' (X'C1F2F3F4'), '12.3' (X'F1F24BF3') and '123X' (X'F1F2F3E7') are all considered to be zoned decimal non-numeric.

A field to be tested for numerics in packed decimal format looks like this in hexadecimal:

```
dddd...ds
```

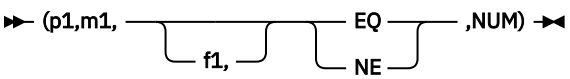
The field is considered to be packed decimal numeric if every d is 0-9, and s is C, D or F. Otherwise, the field is considered to be packed decimal non-numeric. For example, X'12345C', X'12345D' and X'12345F' are all considered to be packed decimal numeric, whereas X'A2345C', X'1B345D' and X'12F45F' are all considered to be packed decimal non-numeric.

Relational condition format

Two formats for the relational condition can be used:



Or, if the FORMAT=f operand is used:



Numeric test operators are as follows:

EQ
Equal to numerics

NE
Not equal to numerics (non-numerics)

Fields

p1,m1,f1

These variables specify the field in the input record for the numeric test.

- p1 specifies the first byte of the field relative to the beginning of the input record [10]. The first data byte of a fixed-length record (FLR) has relative position 1. The first data byte of a variable-length (VLR) record has relative position 5 (because the first 4 bytes contain the record descriptor word). All fields must start on a byte boundary, and no field can extend beyond byte 32752.
- m1 specifies the length of the field. The length can be 1 to 256 bytes.
- f1 specifies the type of numerics the field is to be tested for as follows:

  FS
  test for numerics in character format ('0'-'9' (X'F0'-X'F9') in all bytes).

  Note: CSF can be used instead of FS.

  ZD
  test for numerics in zoned decimal format ('0'-'9' (X'F0'-X'F9') in all non-sign bytes; X'F0'-X'F9', X'D0'-X'D9' or X'C0'-X'C9' in the sign byte)

  PD
  test for numerics in packed decimal format (0-9 for all digits; F, D or C for the sign).

You can use p1,m1 rather than p1,m1,f1, if you use FORMAT=f to supply the format for the field.

NUM

Specifies a test for numerics or non-numerics. The condition will be true if the field is numeric when the EQ operator is specified or if the field is non-numeric when the NE operator is specified.

Including records in the output data set--numeric tests

Example 1

```
INCLUDE COND=(1,20,FS,EQ,NUM)
```

This example illustrates how to only include records in which the field in bytes 1 through 20 contains valid character numeric data (that is, '0'-'9' in all bytes).

Example 2

```
INCLUDE COND=(21,8,ZD,NE,NUM,OR,31,5,PD,NE,NUM)
```

This example illustrates how to only include records in which the field in bytes 21 through 28 contains invalid zoned decimal data, or the field in bytes 31 through 35 contains invalid packed decimal data (that is, one of the fields is non-numeric).

Alphanumeric tests

You can test a field for alphanumerics or non-alphanumerics in character format. Various combinations of uppercase characters (A-Z), lowercase characters (a-z) and numeric characters (0-9) can be used.

[10] If your E15 user exit routine formats the record, p1 must refer to the record as reformatted by the exit.

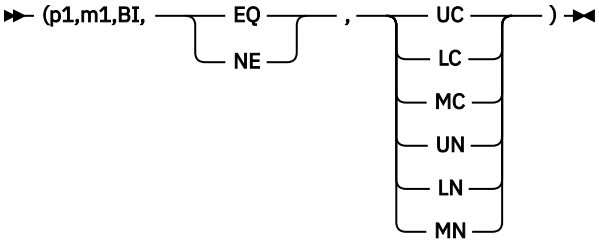
For example, you can include only those records in which a 10-byte field contains only 'A'-'Z' characters (that is, uppercase characters) or '0'-'9' characters (that is, numeric characters). Or you can include only those records in which a 20-byte field contains characters other than 'a'-'z' (that is, lowercase characters).

The following combinations of alphanumeric characters can be used:

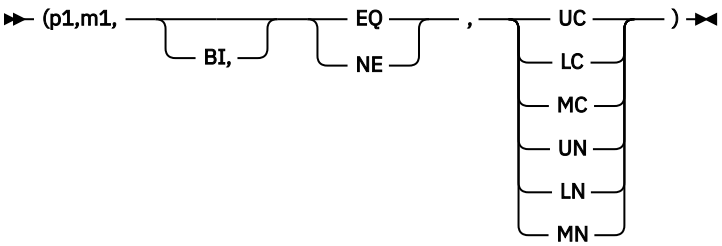
- Uppercase characters (A-Z)
- Lowercase characters (a-z)
- Mixed case characters (A-Z, a-z)
- Uppercase and numeric characters (A-Z, 0-9)
- Lowercase and numeric characters (a-z, 0-9)
- Mixed case and numeric characters (A-Z, a-z, 0-9)

Relational condition format

Two formats for the relational condition can be used:



Or, if the FORMAT=BI operand is used:



Alphanumeric test operators are as follows:

EQ
Equal to specified set of alphanumerics

NE
Not equal to specified set of alphanumerics (non-alphanumerics)

Fields:

p1,m1
These variables specify the field in the input record for the alphanumeric test.

- *p1* specifies the first byte of the field relative to the beginning of the input record[11]. The first data byte of a fixed-length record (FLR) has relative position 1. The first data byte of a variable-length (VLR) record has relative position 5 (because the first 4 bytes contain the record descriptor word). All fields must start on a byte boundary, and no field can extend beyond byte 32752.
- *m1* specifies the length of the field. The length can be 1 to 256 bytes.

[11] If your E15 user exit routine formats the record, p1 must refer to the record as reformatted by the exit.

UC
Specifies a test for the set of uppercase characters (A-Z). The condition will be true if the field is all uppercase characters when the EQ operator is specified or if the field is not all uppercase characters when the NE operator is specified.

LC
Specifies a test for the set of lowercase characters (a-z). The condition will be true if the field is all lowercase characters when the EQ operator is specified or if the field is not all lowercase characters when the NE operator is specified.

MC
Specifies a test for the set of mixed case characters (A-Z and a-z). The condition will be true if the field is all mixed case characters when the EQ operator is specified or if the field is not all mixed case characters when the NE operator is specified.

UN
Specifies a test for the set of uppercase and numeric characters (A-Z and 0-9). The condition will be true if the field is all uppercase or numeric characters when the EQ operator is specified or if the field is not all uppercase or numeric characters when the NE operator is specified.

LN
Specifies a test for the set of lowercase and numeric characters (a-z and 0-9). The condition will be true if the field is all lowercase or numeric characters when the EQ operator is specified or if the field is not all lowercase or numeric characters when the NE operator is specified.

MN
Specifies a test for the set of mixed case and numeric characters (A-Z, a-z or 0-9). The condition will be true if the field is all mixed case or numeric characters when the EQ operator is specified or if the field is not all mixed case or numeric characters when the NE operator is specified.

Including records in the output data set--alphanumeric tests

Example 1

```
INCLUDE COND=(11,10,BI,EQ,MC)
```

This example illustrates how to only include records in which the field in bytes 11 through 20 contains only mixed case characters (that is, 'A'-'Z' or 'a'-'z' in all bytes). A record with 'AaBbZRStuv' in 11-20 would be included, whereas a record with 'Aa7BtuvZQR' would not be included.

Example 2

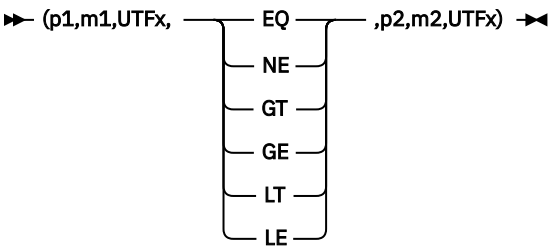
```
INCLUDE COND=(21,4,BI,NE,LN)
```

This example illustrates how to only include records in which the field in bytes 21 through 24 does not contain all lowercase or numeric characters (that is, one of the bytes is not 'a'-'z' or '0'-'9'). A record with 'a,23' would be included, whereas a record with 'a2b9' would not be included.

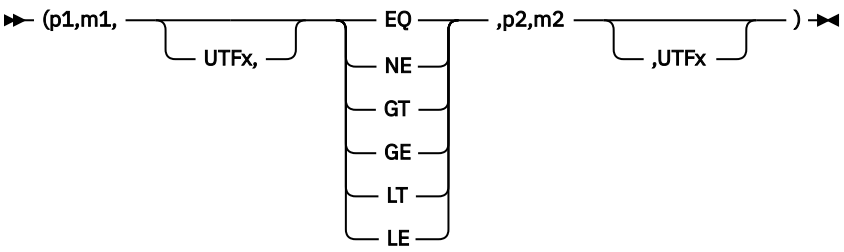
Unicode comparisons

Compare a field in Unicode format to another field in Unicode format. For example, you can include only those records for which a UTF8 field is less than another UTF8 field. Or you can exclude those records for which a UTF16 field is equal to another UTF16 field. Or you can include only those records for which a UTF32 field is greater than another UTF32 field.

Relational condition format:



Or, if the FORMAT=UTFx operand is used:



Comparison operators are as follows:

EQ
Equal to

NE
Not equal to

GT
Greater than

GE
Greater than or equal to

LT
Less than

LE
Less than or equal to

Fields:

p1,m1, UTFx
These variables specify a Unicode data field in the input record to be compared either to another Unicode data field in the input record.

- *p1* specifies the starting position in bytes (octets) of the Unicode data field relative to the beginning of the input record. The first data byte of a fixed-length record (FLR) has relative position 1. The first data byte of a variable-length (VLR) record has relative position 5 (because the first 4 bytes contain the record descriptor word). All Unicode data fields must start on a byte boundary, and no Unicode data field can extend beyond byte 32752. Note that for UTF8 format, a single Unicode character may be represented by 1 to 4 bytes.
- *m1* specifies the length as the number of Unicode characters (not bytes) of the Unicode data field to be compared. Acceptable lengths for different formats are shown in Table 30 on page 124.
- *UTFx* specifies the UTF format. Permissible formats are shown in Table 30 on page 124.

Note that you can use p1,m1 rather than p1,m1,UTFx if you use FORMAT=UTFx to supply the format for the Unicode data field.

Table 30. Compare field formats and lengths		
Format code	**Length**	**Description**
UTF8	1 to 256 Unicode Characters	Unicode Data Format UTF-8
UTF16	1 to 256 Unicode Characters	Unicode Data Format UTF16
UTF32	1 to 256 Unicode Characters	Unicode Data Format UTF-32

p2,m2, UTFx

These variables specify another Unicode data field in the input record with which the p1,m1,UTFx field will be compared. Permissible comparisons between compare fields with different formats are shown in Table 31 on page 124.

You can use p2,m2 rather than p2,m2,UTFx if you use FORMAT=UTFx to supply the format for the Unicode data field.

Table 31. Permissible field-to-field comparisons for INCLUDE/OMIT			
Field format	**UTF8**	**UTF16**	**UTF32**
UTF8	X		
UTF16		X	
UTF32			X

Note:

- When specifying the length of the compared fields, you need to specify the number of Unicode characters, but not the number of bytes.
- The lengths of the two fields with Unicode data being compared must be identical.
- Unicode comparisons use binary collating sequence based on the Unicode code point values. The comparison is performed character by character using the numeric values of the Unicode code points, independent of locale settings.
- The following operands are not supported with INCLUDE/OMIT processing of UTF8/UTF16/UTF32 data formats:
  - LOCALE
  - EFS
  - SUM
  - E61 exit
  - VLSHRT
  - INREC
  - OUTREC

Examples:

```
INCLUDE FORMAT=UTF8,COND=(5,4,LT,11,4)
```

Explanation: Include the records if UTF8 format data starting at byte position 5 for 4 Unicode characters is less than UTF8 format data starting at byte position 11 for 4 Unicode characters. The comparison uses binary collating sequence based on Unicode code point values.

```
OMIT COND=(21,40,UTF16,EQ,151,40,UTF16)
```

Explanation: Omit the records if UTF16 format data at position 21 for 40 characters is equal to UTF16 format data at 151 for 40 characters.

```
OUTFIL INCLUDE=(21,4,UTF32,NE,31,4,UTF32)
```

Explanation: Include the records if UTF32 format data at position 21 for 4 characters is not equal to UTF32 format data at 31 for 4 characters.

INCLUDE/OMIT statement notes

- Floating point compare fields cannot be referenced in INCLUDE or OMIT statements.
-
- Any selection can be performed with either an INCLUDE or an OMIT statement. INCLUDE and OMIT are mutually exclusive.
- If several relational conditions are joined with a combination of AND and OR logical operators, the AND statement is evaluated first. The order of evaluation can be changed by using parentheses inside the COND expression.
- If any changes are made to record formats by user exits E15 or E32, the INCLUDE or OMIT statement must apply to the newest formats.
- DFSORT issues a message and terminates if an INCLUDE or OMIT statement is specified for a tape work data set sort or conventional merge application.
- If SZERO is in effect, -0 compares as less than +0 when numeric fields and constants are used. If NOSZERO is in effect, -0 compares as equal to +0 when numeric fields and constants are used.

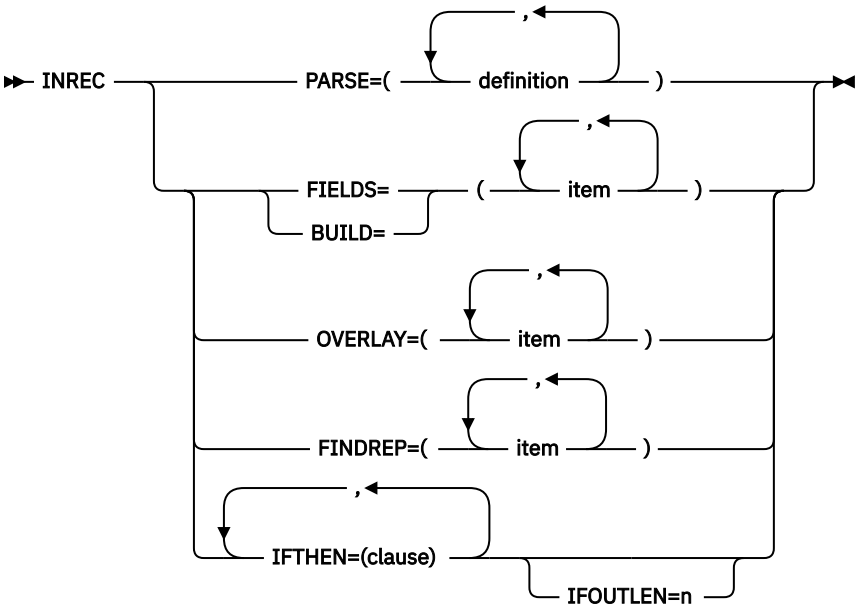
Table 32 on page 125 shows how DFSORT reacts to the result of a relational condition comparison, depending on whether the statement is INCLUDE or OMIT and whether the relational condition is followed by an AND or an OR logical operator.

When writing complex statements, the table in Table 32 on page 125 helps you get the result that you want.

Table 32. Logic Table for INCLUDE/OMIT.

	Relational Condition	**Program action if next logical operator is:**	
Statement	**Compare**	**AND**	**OR**
OMIT	True	Check next compare, or if last compare OMIT record.	OMIT record
OMIT	False	INCLUDE record	Check next compare, or if last compare, INCLUDE record.
INCLUDE	True	Check next compare, or if last compare, INCLUDE record.	INCLUDE record
INCLUDE	False	OMIT record	Check compare, or if last compare, OMIT record.

INREC control statement



The INREC control statement allows you to reformat the input records before they are sorted, merged, or copied.

The INREC control statement supports a wide variety of parsing, editing, and reformatting tasks, including:

- The use of fixed position/length fields or variable position/length fields. For fixed fields, you specify the starting position and length of the field directly. For variable fields, such as delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings (and many other types), you define rules that allow DFSORT to extract the relevant data into fixed parsed fields, and then use the parsed fields as you would use fixed fields.
- Insertion of blanks, zeros, strings, current date, future date, past date, current time, sequence numbers, decimal constants, and the results of arithmetic expressions before, between, and after the input fields in the reformatted records.
-
- Sophisticated conversion capabilities, such as find and replace, hexadecimal display, bit display, translation of EBCDIC letters from lowercase to uppercase or uppercase to lowercase, translation of characters from EBCDIC to ASCII and from ASCII to EBCDIC, translation of characters using the ALTSEQ translation table, conversion of numeric values from one format to another, left-justify or left-squeeze (remove leading blanks or all blanks and shift left), and right-justify or right-squeeze (remove trailing blanks or all blanks and shift right).
- Sophisticated editing capabilities, such as control of the way numeric fields are presented with respect to length, leading or suppressed zeros, thousands separators, decimal points, leading and trailing positive and negative signs, and so on.

  Twenty-seven pre-defined editing masks are available for commonly used numeric editing patterns, encompassing many of the numeric notations used throughout the world. In addition, a virtually unlimited number of numeric editing patterns are available via user-defined editing masks.
- Transformation of SMF, TOD, and ETOD date and time values to more usable forms.
- Conversion of input date fields of one type (CH, ZD, PD, 2-digit year, 4-digit year, Julian, Gregorian) to corresponding output date fields of another type or to a corresponding day of the week.
- Various types of arithmetic operations for input date fields.
- Selection of a character constant, hexadecimal constant, or input field from a lookup table, based on a character, hexadecimal, or bit string as input (that is, lookup and change).

You can create the reformatted INREC records in one of the following ways using unedited, edited, or converted input fields (p,m for fixed fields, or %nn for parsed fields - see PARSE), and a variety of constants:

- **BUILD or FIELDS:** Reformat each record by specifying all of its items one by one. Build gives you complete control over the items you want in your reformatted INREC records and the order in which they appear. You can delete, rearrange and insert fields and constants. Example:

```
INREC BUILD=(1,20,C'ABC',26:5C'*',
       15,3,PD,EDIT=(TTT.TT),21,30,80:X)
```

- **OVERLAY:** Reformat each record by specifying just the items that overlay specific columns. Overlay lets you change specific existing columns without affecting the entire record. Example:

```
INREC OVERLAY=(45:45,8,TRAN=LTOU)
```

- **FINDREP**: Reformat each record by doing various types of find and replace operations. Example:

```
INREC FINDREP=(IN=C'Mr.',OUT=C'Mister')
```

- **IFTHEN clauses:** Reformat different records in different ways by specifying how build, overlay, find/replace, or group operation items are applied to records that meet given criteria. IFTHEN clauses let you use sophisticated conditional logic to choose how different record types are reformatted. Example:

```
INREC IFTHEN=(WHEN=(1,5,CH,EQ,C'TYPE1'),
         BUILD=(1,40,C'**',+1,TO=PD)),
      IFTHEN=(WHEN=(1,5,CH,EQ,C'TYPE2'),
         BUILD=(1,40,+2,TO=PD,X'FFFF')),
      IFTHEN=(WHEN=NONE,OVERLAY=(45:C'NONE'))
```

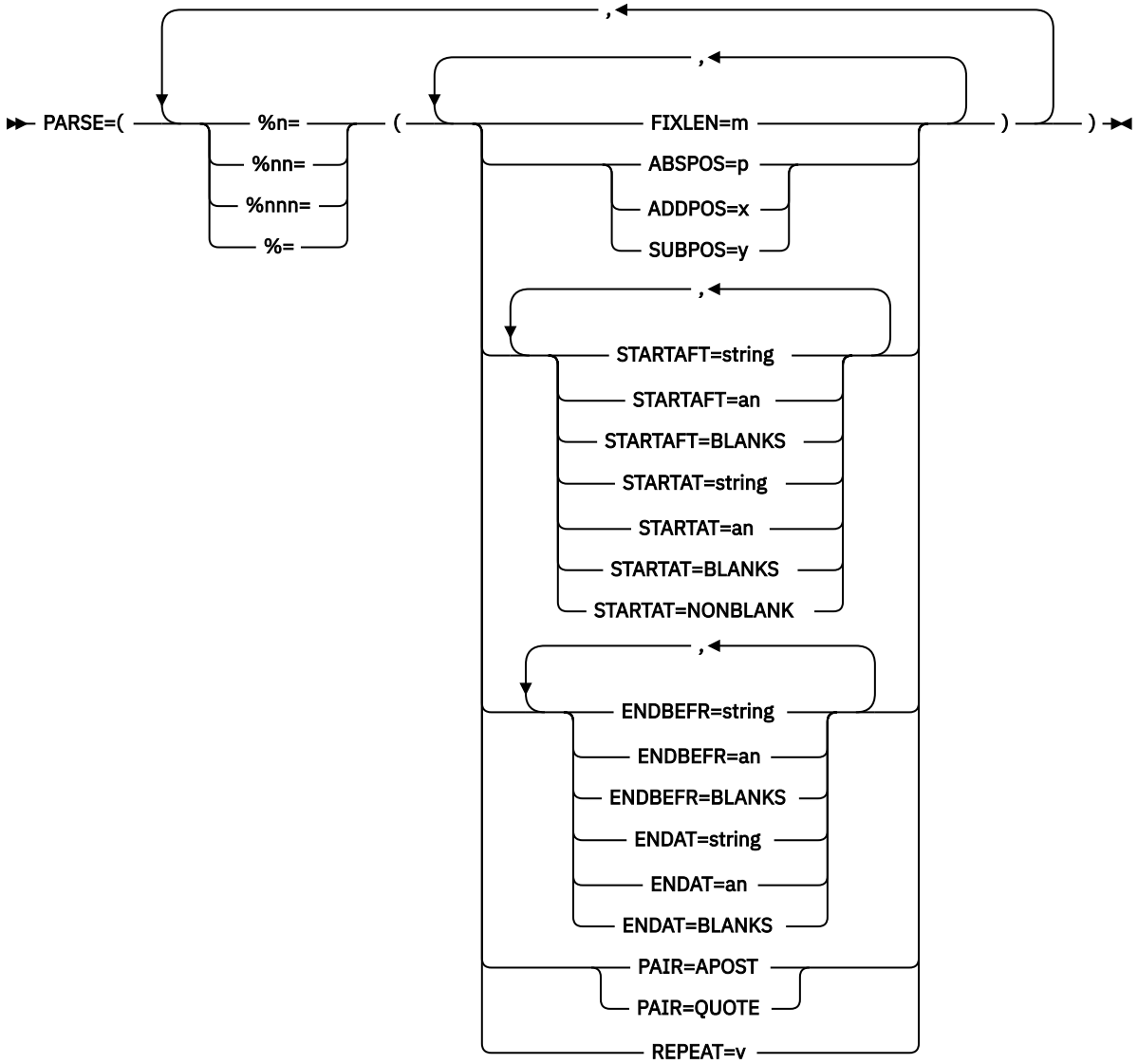
You can choose to include any or all of the following items in your reformatted INREC records:

- Fixed position/length fields or variable position/length fields. For fixed fields, you specify the starting position and length of the field directly. For variable fields, such as delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings (and many other types), you define rules that allow DFSORT to extract the relevant data into fixed parsed fields, and then use the parsed fields as you would use fixed fields.
- Blanks, binary zeros, character strings, and hexadecimal strings
- Current date, future date, past date, and current time in various forms
- Unedited input fields aligned on byte, halfword, fullword, and doubleword boundaries
- Replaced or removed strings.
- Hexadecimal or bit representations of binary input fields
- Characters translated from uppercase to lowercase, lowercase to uppercase, ASCII to EBCDIC or EBCDIC to ASCII.
- Left-justified, right-justified, left-squeezed, or right-squeezed input fields.
- Numeric input fields of various formats converted to different numeric formats, or to character format edited to contain signs, thousands separators, decimal points, leading zeros or no leading zeros, and so on.
- Decimal constants converted to different numeric formats, or to character format edited to contain signs, thousands separators, decimal points, leading zeros or no leading zeros, and so on.
- The results of arithmetic expressions combining fields, decimal constants, operators (MIN, MAX, MUL, DIV, MOD, ADD and SUB) and parentheses converted to different numeric formats, or to character format edited to contain signs, thousands separators, decimal points, leading zeros or no leading zeros, and so on.
- SMF, TOD and ETOD date and time fields converted to different numeric formats, or to character format edited to contain separators, leading zeros or no leading zeros, and so on.
- Input date fields of one type (CH, ZD, PD, 2-digit year, 4-digit year, Julian, Gregorian) converted to corresponding output date fields of another type or to a corresponding day of the week.

- The results of various types of arithmetic operations for input date fields.
- Sequence numbers in various formats.
- A character constant, hexadecimal constant or input field selected from a lookup table, based on a character, hexadecimal or bit constant as input.
- A zoned decimal group identifier, a zoned decimal group sequence number, or a field propagated from the first record of a group to all of the records of a group.

For information concerning the interaction of INREC and OUTREC, see "INREC statement notes" on page 148.

PARSE



This operand allows you to extract variable position/length fields into fixed parsed fields. Parsed fields (%n, %nn or %nnn) can be used where fixed position/length fields (p,m) can be used in the BUILD (or FIELDS) or OVERLAY operands as described later in this section.

Note: Although you can use %n (%0-%9), %nn (%00-%99) or %nnn (%000-%999) for a parsed field, for convenience in this book %nn will be used in general when referring to a parsed field. %n, %0n or %00n can be used interchangeably for parsed field n (for example, %1, %01 or %001 for parsed field 1). %nn or %0nn can be used interchangeably for parsed field nn (for example, %12 or %012 for parsed field 12).

PARSE can be used for many different types of variable fields including delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings, and so on. You can assign up to 1000 parsed fields (%0-%999) to the variable fields you want to extract.

Note that if all of the fields in your records have fixed positions and lengths, you don't need to use PARSE. But if any of the fields in your records have variable positions or lengths, you can use PARSE to treat them as fixed parsed fields in BUILD or OVERLAY. You can mix p,m fields (fixed fields) and %nn fields (parsed fields) in BUILD and OVERLAY.

See PARSE under "OUTFIL Control Statements" for complete details.

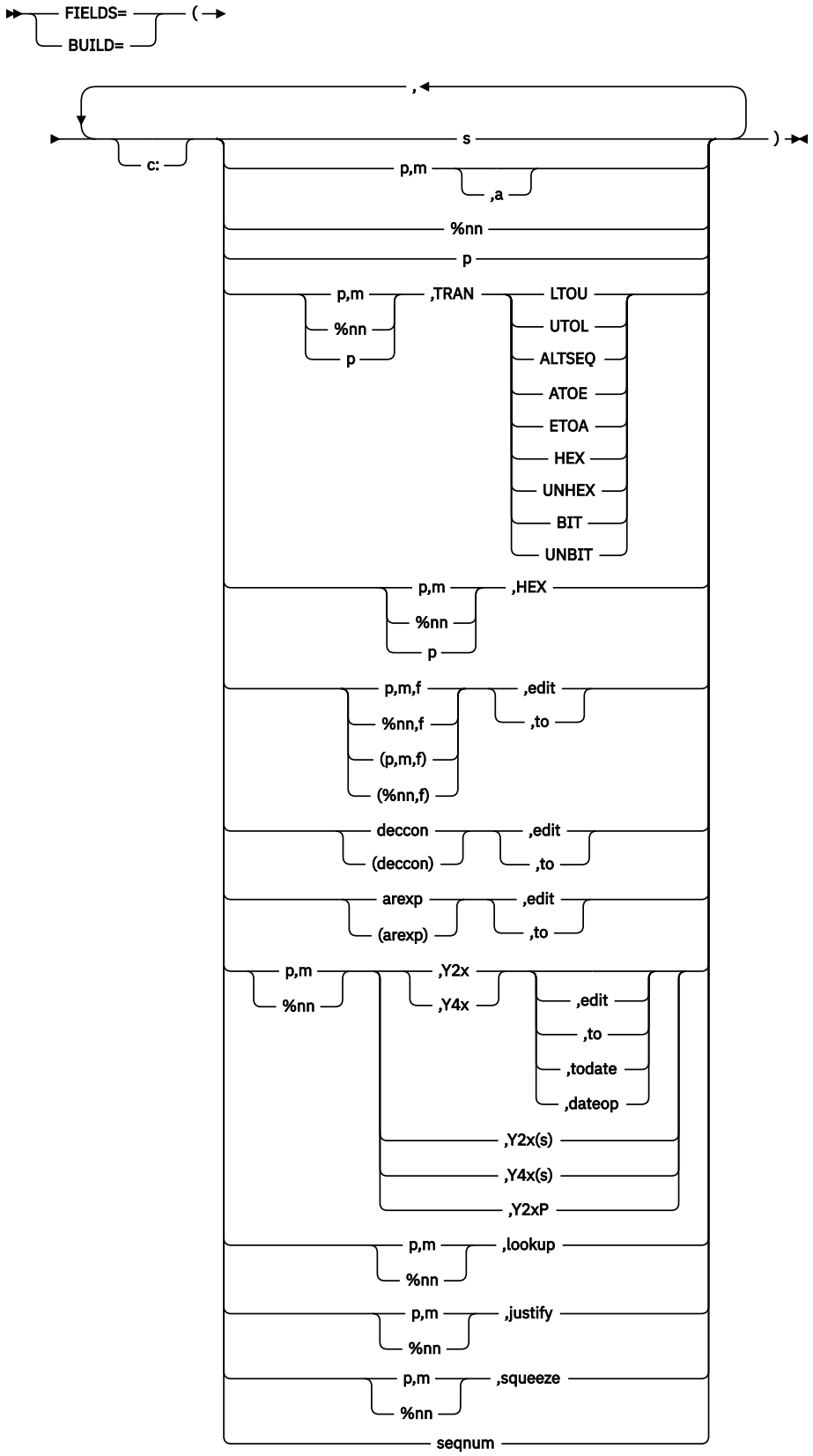
Sample syntax:

```
INREC PARSE=(%00=(ENDBEFR=C'*',FIXLEN=3),
    %01=(ENDBEFR=BLANKS,FIXLEN=6),
    %02=(STARTAT=C'MAX',FIXLEN=8),
    %03=(STARTAFT=C'(',ENDBEFR=C')',FIXLEN=6),
    %04=(STARTAFT=BLANKS,FIXLEN=5)),
  BUILD=(%03,X,%03,HEX,21:%02,31:%01,SFF,M26,LENGTH=7,
         18,6,%00,UFF,M11,LENGTH=3,%04,JFY=(SHIFT=RIGHT))
```

Default for PARSE: None; must be specified. See Appendix B, “Specification/override of DFSORT options,” on page 801.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

FIELDS or BUILD



Specifies all of the items in the reformatted INREC record in the order in which they are to be included. The reformatted INREC record consists of the separation fields, edited and unedited input fields (p,m for fixed fields, or %nn for parsed fields - see PARSE),, edited decimal constants, edited

results of arithmetic expressions, and sequence numbers you select, in the order in which you select them, aligned on the boundaries or in the columns you indicate.

For variable-length records, the first item in the BUILD or FIELDS parameter must specify or include the unedited 4-byte record descriptor word (RDW), that is, you must start with 1,m with m equal to or greater than 4. If you want to include the bytes from a specific position to the end of each input record at the end of each reformatted output record, you can specify that starting position (p) as the last item in the BUILD or FIELDS parameter. For example:

```
INREC FIELDS=(1,4,          unedited RDW
   1,2,BI,TO=ZD,LENGTH=5,   display RDW length in decimal
   C'|',                    | separator
   5)                       display input positions 5 to end
```

For fixed-length records, the first input and output data byte starts at position 1. For variable-length records, the first input and output data byte starts at position 5, after the RDW in positions 1-4.

c:

Specifies the position (column) for a separation field, input field, decimal constant, arithmetic expression, or sequence number, relative to the start of the reformatted input record. Unused space preceding the specified column is padded with EBCDIC blanks. The following rules apply:

- c must be a number between 1 and 32752.
- c: must be followed by a separation field, input field, decimal constant, or arithmetic expression.
- c must not overlap the previous input field or separation field in the reformatted input record.
- for variable-length records, c: must not be specified before the first input field (the record descriptor word) nor after the variable part of the input record.
- The colon (:) is treated like the comma (,) or semicolon (;) for continuation to another line.

Both valid and invalid examples are shown in Table 33 on page 131.

Table 33. Examples of valid and invalid column alignment . Examples of valid and invalid column alignment

Validity	Specified	Result
Valid	33:C'State '	Columns 1-32 — blank Columns 33-38 — 'State '
Valid	20:5,4,30:10,8	Columns 1-19 — blank Columns 20-23 — input field (5,4) Columns 24-29 — blank Columns 30-37 — input field (10,8)
Invalid	0:5,4	Column value cannot be zero.
Invalid	:25Z	Column value must be specified.
Invalid	32753:21,8	Invalid — column value must be less than 32753.
Invalid	5:10:2,5	Column values cannot be adjacent.
Invalid	20,10,6:C'AB'	Column value overlaps previous field.

s

Specifies that a separation field (blanks, zeros, character string, hexadecimal string, current date, future date, past date, or current time) is to appear in the reformatted input record. It can be specified before or after any input field. Consecutive separation fields can be specified. For variable-length records, separation fields must not be specified before the first input field (the record descriptor word), or after the variable part of the input record. Permissible values are nX, nZ, nC'xx...x', nX'yy...yy', and various date and time constants.

nX

Blank separation. n bytes of EBCDIC blanks (X'40') are to appear in the reformatted input records. n can range from 1 to 4095. If n is omitted, 1 is used.

Examples of valid and invalid blank separation are shown in Table 34 on page 132.

Table 34. Examples of valid and invalid blank separation

Status	Specified	Result
Valid	X or 1X	1 blank
Valid	4095X	4095 blanks
Invalid	5000X	Too many repetitions. Use two adjacent separation fields instead (2500X,2500X, for example)
Invalid	0X	0 is not allowed.

nZ

Binary zero separation. n bytes of binary zeros (X'00') are to appear in the reformatted input records. n can range from 1 to 4095. If n is omitted, 1 is used.

Examples of valid and invalid binary zero separation are shown in Table 35 on page 132.

Table 35. Examples of valid and invalid binary zero separation

Status	Specified	Result
Valid	Z or 1Z	1 binary zero
Valid	4095Z	4095 binary zeros
Invalid	4450Z	Too many repetitions. Use two adjacent separation fields instead (4000Z,450Z for example).
Invalid	0Z	0 is not allowed.

nC'xx...x'

Character string separation. n repetitions of the character string constant (C'xx...x') are to appear in the reformatted input records. n can range from 1 to 4095. If n is omitted, 1 is used. x can be any EBCDIC character. You can specify from 1 to 256 characters.

If you want to include a single apostrophe in the character string, you must specify it as two single apostrophes:

```
Required:  O'NEILL     Specify:  C'O''NEILL'
```

Examples of valid and invalid character string separation are shown in Table 36 on page 132.

Table 36. Examples of valid and invalid character string separation

Status	Specified	Result	Length
Valid	C'John Doe'	John Doe	8

Table 36. Examples of valid and invalid character string separation (continued)

Status	Specified	Result	Length
Valid	C'JOHN DOE'	JOHN DOE	8
Valid	C'$@#'	$@#	3
Valid	C'+0.193'	+0.193	6
Valid	4000C' '	8000 blanks	8000
Valid	20C'**FILLER**'	**FILLER** repeated 20 times	200
Valid	C'Frank''s'	Frank's	7
Invalid	C'''''	Apostrophes not paired	n/a
Invalid	'ABCDEF'	C identifier missing	n/a
Invalid	C'ABCDE	Apostrophe missing	n/a
Invalid	4450C'1'	Too many repetitions. Use two adjacent separation fields instead (4000C'1',450C'1', for example).	n/a
Invalid	0C'ABC'	0 is not allowed	n/a
Invalid	C''	No characters specified	n/a
Invalid	C'Frank's'	Two single apostrophes needed for one	n/a

nX'yy...yy'

Hexadecimal string separation. n repetitions of the hexadecimal string constant (X'yy...yy') are to appear in the reformatted input records. n can range from 1 to 4095. If n is omitted, 1 is used.

The value yy represents any pair of hexadecimal digits. You can specify from 1 to 256 pairs of hexadecimal digits. Examples of valid and invalid hexadecimal string separation are shown in Table 37 on page 133.

Table 37. Examples of valid and invalid hexadecimal string separation

Status	Specified	Result	Length
Valid	X'FF'	FF	1
Valid	X'BF3C'	BF3C	2
Valid	3X'00000F'	00000F00000F00000F	9
Valid	4000X'FFFF'	FF repeated 8000 times	8000
Invalid	X'ABGD'	G is not a hexadecimal digit	n/a
Invalid	X'F1F'	Incomplete pair of digits	n/a
Invalid	'BF3C'	X identifier missing	n/a
Invalid	'F2F1'X	X in wrong place	n/a
Invalid	8000X'01'	Too many repetitions. Use two adjacent separation fields instead (4000X'01',4000X'01', for example).	n/a
Invalid	0X'23AB'	0 is not allowed	n/a

Table 37. Examples of valid and invalid hexadecimal string separation (continued)

Status	Specified	Result	Length
Invalid	X''	No hexadecimal digits specified	n/a

DATEn, DATEn(c), DATEnP
Constant for current date. The current date of the run is to appear in the reformatted input records. See DATEn, DATEn(c), DATEnP under OUTFIL OUTREC for details.

&DATEn, &DATEn(c), &DATEnP
Can be used instead of DATEn, DATEn(c) and DATEnP, respectively.

DATEn+r, DATEn(c)+r, DATEnP+r
Constant for future date. A future date relative to the current date of the run is to appear in the reformatted input records. See DATEn+r, DATEn(c)+r, DATEnP+r under OUTFIL OUTREC for details.

&DATEn+r, &DATEn(c)+r, &DATEnP+r
Can be used instead of DATEn+r, DATEn(c)+r and DATEnP+r, respectively.

DATEn-r, DATEn(c)-r, DATEnP-r
Constant for past date. A past date relative to the current date of the run is to appear in the reformatted input records. See DATEn-r, DATEn(c)-r, DATEnP-r under OUTFIL OUTREC for details.

&DATEn-r, &DATEn(c)-r, &DATEnP-r
Can be used instead of DATEn-r, DATEn(c)-r and DATEnP-r, respectively.

TIMEn, TIMEn(c), TIMEnP
Constant for current time. The time of the run is to appear in the reformatted input records. See TIMEn, TIMEn(c), TIMEnP under OUTFIL OUTREC for details.

&TIMEn, &TIMEn(c), &TIMEnP
Can be used instead of TIMEn, TIMEn(c) and TIMEnP, respectively.

DATE
Specifies that the current date is to appear in the reformatted input records in the form 'mm/dd/yy'. See DATE under OUTFIL OUTREC for details.

&DATE
Can be used instead of DATE.

DATE=(abcd)
Specifies that the current date is to appear in the reformatted input records in the form 'adbdc'. See DATE=(abcd) under OUTFIL OUTREC for details.

&DATE=(abcd)
Can be used instead of DATE=(abcd).

DATENS=(abc)
Specifies that the current date is to appear in the reformatted input records in the form 'abc'. See DATENS=(ab) under OUTFIL OUTREC for details.

&DATENS=(abc)
Can be used instead of DATENS=(abc).

YDDD=(abc)
specifies that the current date is to appear in the reformatted input records in the form 'acb'. See YDDD=(abc) under OUTFIL OUTREC for details.

&YDDD=(abc)
Can be used instead of YDDD=(abc).

YDDDNS=(ab)
specifies that the current date is to appear in the reformatted input records in the form 'ab'. See YDDDNS=(ab) under OUTFIL OUTREC for details.

&YDDDNS=(ab)
can be used instead of YDDDNS=(ab).

TIME
specifies that the current time is to appear in the reformatted input records in the form 'hh:mm:ss'. See TIME under OUTFIL OUTREC for details.

&TIME
Can be used instead of TIME.

TIME=(abc)
specifies that the current time is to appear in the reformatted input records in the form 'hhcmmcss' (24-hour time) or 'hhcmmcss xx' (12-hour time). See TIME=(abc) under OUTFIL OUTREC for details.

&TIME=(abc)
Can be used instead of TIME=(abc).

TIMENS=(ab)
specifies that the current time is to appear in the reformatted input record in the form 'hhmmss' (24-hour time) or 'hhmmss xx' (12-hour time). See TIMENS=(ab) under OUTFIL OUTREC for details.

&TIMENS=(ab)
Can be used instead of TIMENS=(ab).

p,m,a

Specifies that an unedited input field is to appear in the reformatted input record.

p
Specifies the first byte of the input field relative to the beginning of the input record.[12] The first data byte of a fixed-length record has relative position 1. The first data byte of a variable-length record has relative position 5 (because the first 4 bytes contain the RDW). All fields must start on a byte boundary, and no field can extend beyond byte 32752. For special rules concerning variable-length records, see “INREC statement notes” on page 148.

m
Specifies the length of the input field. It must include the sign if the data is signed, and must be an integer number of bytes. See “INREC statement notes” on page 148 for more information.

a
Specifies the alignment (displacement) of the input field in the reformatted input record relative to the start of the reformatted input record.

Permissible values of **a** are:

H
Halfword aligned. The displacement (p-1) of the field from the beginning of the reformatted input record, in bytes, is a multiple of two (that is, position 1, 3, 5, and so forth).

F
Fullword aligned. The displacement is a multiple of four (that is, position 1, 5, 9, and so forth).

D
Doubleword aligned. The displacement is a multiple of eight (that is, position 1, 9, 17, and so forth).

Alignment can be necessary if, for example, the data is to be used in a COBOL application program where items are aligned through the SYNCHRONIZED clause. Unused space preceding aligned fields will always be padded with binary zeros.

[12] If your E15 user exit reformats the record, p must refer to the record as reformatted by the exit.

%nn
specifies that an unedited parsed input field is to appear in the reformatted input record. See PARSE for details of parsed fields. See p,m,a for further details. Note that alignment (H, F, D) is not permitted for %nn fields (for example, %nn,F results in an error message and termination).

p
specifies that the unedited variable part of the input record (that part beyond the minimum record length), is to appear in the reformatted input record, as the last field. p without m can only be used for variable-length records; not for fixed-length records.

Attention: If 1,4,p is specified (only RDW and variable part of record), "null" records containing only an RDW will result if the record length is less than p.

A value must be specified for p that is less than or equal to the minimum record length (RECORD statement L4 value) plus 1 byte.

p,m,TRAN=keyword
Specifies that an input field is to be translated as indicated by the keyword. See p,m,TRAN=keyword under OUTFIL OUTREC for details.

%nn,TRAN=keyword
Specifies that a parsed input field is to be translated as indicated by the keyword. See %nn,TRAN=keyword under OUTFIL OUTREC for details.

p,TRAN=keyword
Specifies that the variable part of the input record is to be translated as indicated by the keyword. See p,TRAN=keyword under OUTFIL OUTREC for details.

p,m,HEX
Can be used instead of p,m,TRAN=HEX. See p,m,HEX under OUTFIL OUTREC for details.

%nn,HEX
Can be used instead of %nn,TRAN=HEX. See %nn,HEX under OUTFIL OUTREC for details.

p,HEX
Can be used instead of p,TRAN=HEX. See p,HEX under OUTFIL OUTREC for details.

p,m,f,edit or (p,m,f),edit
specifies that an edited numeric input field is to appear in the reformatted input record. You can edit BI, FI, PD, PD0, ZD, FL, CSF, FS, UFF, SFF, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3 or TM4 fields using either pre-defined edit masks (M0-M26) or specific edit patterns you define. You can control the way the edited fields look with respect to length, leading or suppressed zeros, thousands separators, decimal points, leading and trailing positive and negative signs, and so on.

See p,m,f,edit under OUTFIL OUTREC for details.

Sample syntax:

```
INREC FIELDS=(5:21,8,ZD,M19,X,46,5,ZD,M13,
              31:(35,6,FS),SIGNS=(,,+,-),LENGTH=10,
              51:8,4,PD,EDIT=(**II,IIT.TTXS),SIGNS=(,,+,-))
```

%nn,f,edit or (%nn,f),edit
specifies that an edited numeric parsed input field is to appear in the reformatted input record. See PARSE for details of parsed fields. See p,m,f,edit or (p,m,f),edit for further details.

p,m,f,to or (p,m,f),to
specifies that a converted numeric input field is to appear in the reformatted input record. You can convert BI, FI, PD, PD0, ZD, FL, CSF, FS, UFF, SFF, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3, or TM4 fields to BI, FI, PD, PDC, PDF, ZD, ZDF, ZDC, or CSF/FS fields.

See p,m,f,to under OUTFIL OUTREC for details.

Sample syntax:

```
INREC FIELDS=(21,5,ZD,TO=PD,X,(8,4,ZD),FI,LENGTH=2,X,55,4,FL,TO=FS)
```

%nn,f,to or (%nn,f),to
specifies that a converted numeric parsed input field is to appear in the reformatted input record. See PARSE for details of parsed fields. See p,m,f,to or (p,m,f),to for further details.

deccon,edit or (deccon),edit
specifies that an edited decimal constant is to appear in the reformatted input record. The decimal constant must be in the form +n or -n where n is 1 to 31 decimal digits. The sign (+ or -) must be specified. A decimal constant produces a signed, 31-digit zoned decimal (ZD) result to be edited as specified.

See deccon,edit under OUTFIL OUTREC for details.

Sample syntax:

```
INREC FIELDS=(5:+5000,EDIT=(T,TTT),X,
  (-25500),M18,LENGTH=8)
```

deccon,to or (deccon),to
specifies that a converted decimal constant is to appear in the reformatted input record. The decimal constant must be in the form +n or -n where n is 1 to 31 decimal digits. The sign (+ or -) must be specified. A decimal constant produces a signed, 31-digit zoned decimal (ZD) result to be converted as specified.

See deccon,to under OUTFIL OUTREC for details.

Sample syntax:

```
INREC FIELDS=(+0,TO=PD,LENGTH=6,3Z,(-512000),FI)
```

arexp,edit or (arexp),edit
specifies that the edited result of an arithmetic expression is to appear in the reformatted input record. The arithmetic expression can consist of input fields, decimal constants, operators and parentheses. An arithmetic expression produces a signed, 31-digit zoned decimal (ZD) result to be edited as specified.

See arexp,edit under OUTFIL OUTREC for details.

Sample syntax:

```
INREC FIELDS=(C'**',27,2,FI,MIN,
                  83,4,PD,EDIT=(STTTTTTT),SIGNS=(+,-),
  15:(((15,5,ZD,ADD,+1),MUL,+100),DIV,62,2,PD),M25,LENGTH=10)
```

arexp,to or (arexp),to
specifies that the converted result of an arithmetic expression is to appear in the reformatted input record. The arithmetic expression can consist of input fields, decimal constants, operators and parentheses. An arithmetic expression produces a signed, 31-digit zoned decimal (ZD) result to be converted as specified.

See arexp,to under OUTFIL OUTREC for details.

Sample syntax:

```
INREC FIELDS=((15,6,FS,SUB,+5),ADD,(-1,MUL,36,6,FS),ZD,X,
   3,2,FI,MIN,-6,LENGTH=4,TO=PD)
```

p,m,Y2x or p,m,Y4x
Specifies that an input date field is to be edited. Real Y2x dates are edited using the century window established by the Y2PAST option in effect. Y2x and Y4x dates with special indicators are expanded appropriately (for example, p,6,Y2T transforms C'000000' to C'00000000').

See "p,m,Y2x or p,m,Y4x" under OUTFIL OUTREC for details.

Sample syntax:

```
INREC BUILD=(21,3,Y2U,X,3,8,Y4W)
```

%nn,Y2x or %nn,Y4x
Specifies that a parsed input date field is to be edited. See PARSE for details of parsed fields. See "p,m,Y2x or p,m,Y4x" for further details.

p,m,Y2x,edit or p,m,Y4x,edit
Specifies that the output for a p,m,Yxx input date field is to be edited according to the edit parameters you specify. For example, if you specify:

```
INREC BUILD=(28,5,Y4V,EDIT=(TTTT-TT-TT)
```

The P'ccyymmdd' (X'0ccyymmddC') date value will be transformed to a C'ccyymmdd' date value and then edited to a C'ccyy-mm-dd' date value.

See "p,m,Y2x or p,m,Y4x" and "p,m,f,edit" for related details.

%nn,Y2x,edit or %nn,Y4x,edit
Specifies that the output for a parsed Yxx input date field is to be edited according to the edit parameters you specify. See PARSE for details of parsed fields. See "p,m,Y2x,edit or p,m,Y4x,edit" for further details.

p,m,Y2x,to or p,m,Y4x,to
Specifies that an input date field is to be converted according to the to parameters you specify. For example, if you specify:

```
INREC BUILD=(5,6,Y2W,TO=PD,LENGTH=5)
```

The C'mmddyy' date value will be transformed to a Z'mmddccyy' date value and then converted to a P'mmddccyy' (X'0mmddccyyC') date value.

See "p,m,Y2x or p,m,Y4x" and "p,m,f,to" for related details.

%nn,Y2x,to or %nn,Y4x,to
Specifies that a parsed input date field is to be converted according to the to parameters you specify. See PARSE for details of parsed fields. See "p,m,Y2x,to or p,m,Y4x,to" for further details.

p,m,Y2x,todate or p,m,Y4x,todate
Specifies that an input date field of one type is to be converted to a corresponding output date field of another type. You can convert date fields between combinations of 2-digit and 4-digit year dates, CH/ZD and PD dates, and Julian and Gregorian dates. You can also convert a date field to a corresponding day of the week in several forms.

See "p,m,Y2x,todate or p,m,Y4x,todate" under OUTFIL OUTREC for details.

Sample syntax:

```
* Convert a P'dddyy' input date to a C'ccyy/mm/dd' output date
  INREC BUILD=(21,3,Y2X,TOGREG=Y4T(/),X,
* Convert a C'ccyymmdd' input date to a P'ccyyddd' output date
     42,8,Y4T,TOJUL=Y4U,X,
* Convert a C'mmddyy' input date to a C'yymmdd' output date
     11,6,Y2W,TOGREG=Y2T)
```

%nn,Y2x,todate or %nn,Y4x,todate
Specifies that an input date field of one type is to be converted to a corresponding output date field of another type. See PARSE for details of parsed fields. See "p,m,Y2x,todate or p,m,Y4x,todate" for further details.

p,m,Y2x,dateop or p,m,Y4x,dateop
Specifies an arithmetic operation for an input date field. See "p,m,Y2x,dateop or p,m,Y4x,dateop" under OUTFIL OUTREC for details.

%nn,Y2x,dateop or %nn,Y4x,dateop
Specifies an arithmetic operation for a parsed input date field. See "%nn,Y2x,dateop or %nn,Y4x,dateop" under OUTFIL OUTREC for details.

p,m,Y2x(s) or p,m,Y4x(s)
Specifies that an input date field is to be edited with separators. s can be any character except a blank. Real Y2x dates are edited using the century window established by the Y2PAST option in effect. Y2x and Y4x dates with special indicators are expanded appropriately (for example, p,8,Y4T(/) transforms C'00000000' to C'0000/00/00').

See "p,m,Y2x(s) or p,m,Y4x(s)" under OUTFIL OUTREC for details.

Sample syntax:

```
* Convert a Z'dddccyy' date to a C'ddd/ccyy' date.
  INREC BUILD=(19,7,Y4W(/),X,
* Convert a P'ccyymmdd' date to a C'ccyy-mm-dd' date.
     43,5,Y4V(-))
```

%nn,Y2x(s) or %nn,Y4x(s)
Specifies that a parsed input date field is to be edited with separators. See PARSE for details of parsed fields. See "p,m,Y2x(s) or p,m,Y4x(s)" for further details.

p,m,Y2xP
Specifies that an input date field is to be converted to a packed decimal output date field. Real Y2x dates are edited using the century window established by the Y2PAST option in effect. Y2x and Y4x dates with special indicators are expanded appropriately (for example, p,6,Y2TP transforms C'000000' to P'00000000').

See "p,m,Y2xP" under OUTFIL OUTREC for details.

Sample syntax:

```
  INREC BUILD=(11,3,Y2XP,X,21,4,Y2WP)
```

%nn,Y2xP
Specifies that a parsed input date field is to be converted to a packed decimal output date field. See PARSE for details of parsed fields. See "p,m,Y2xP" for further details.

p,m,lookup or %nn,lookup
specifies that a character constant, hexadecimal constant, input field (p,m), or parsed input field (%nn) from a lookup table is to appear in the reformatted input record. You can use p,m,lookup or %nn,lookup to select a specified character set constant (that is, a character or hexadecimal string) or set field (that is, an input field or parsed input field) based on matching an input value against find constants (that is, character, hexadecimal, or bit constants).

See p,m,lookup or %nn,lookup under OUTFIL OUTREC for details.

Sample syntax:

```
  INREC FIELDS=(11,1,
               CHANGE=(6,
                 C'R',C'READ',
                 C'U',C'UPDATE',
                 X'FF',C'EMPTY',
                 C'A',C'ALTER'),
               NOMATCH=(11,6),
               4X,
               21,1,
               CHANGE=(10,
                 B'.1......',C'VSAM',
                 B'.0......',C'NON-VSAM'))
```

p,m,justify
specifies that a left-justified or right-justified input field is to appear in the reformatted input record. For a left-justified field, leading blanks are removed and the characters from the first nonblank to the last nonblank are shifted left, with blanks inserted on the right if needed. For a

right-justified field, trailing blanks are removed and the characters from the last nonblank to the first nonblank are shifted right, with blanks inserted on the left if needed. Optionally:

- specific leading and trailing characters can be changed to blanks before justification begins
- a leading string can be inserted
- a trailing string can be inserted
- the output length can be changed (it's equal to the input length by default)

See p,m,justify under OUTFIL OUTREC for details.

Sample syntax:

```
INREC FIELDS=(1,10,
   21,20,JFY=(SHIFT=RIGHT),5X,
   52,12,JFY=(SHIFT=LEFT,PREBLANK=C'()',LEAD=C'VOL=SER=',
          LENGTH=16))
```

%nn,justify
specifies that a left-justified or right-justified parsed input field is to appear in the reformatted input record. See PARSE for details of parsed fields. See p,m,justify for further details.

p,m,squeeze
specifies that a left-squeezed or right-squeezed input field is to appear in the reformatted input record. For a left-squeezed field, all blanks are removed and the characters from the first nonblank to the last nonblank are shifted left, with blanks inserted on the right if needed. For a right-squeezed field, all blanks are removed and the characters from the last nonblank to the first nonblank are shifted right, with blanks inserted on the left if needed. Optionally:

- specific characters can be changed to blanks before squeezing begins
- a leading string can be inserted
- a trailing string can be inserted
- a string (for example, a comma delimiter) can be inserted wherever a group of blanks is removed between the first nonblank and the last nonblank
- blanks can be kept as is between paired apostrophes ('AB CD EF') or paired quotes ("AB CD EF")
- the output length can be changed (it's equal to the input length by default)

See p,m,squeeze under OUTFIL OUTREC for details.

Sample syntax:

```
INREC FIELDS=(21,20,SQZ=(SHIFT=LEFT),5X,
   152,18,SQZ=(SHIFT=RIGHT,PREBLANK=X'00',
     LEAD=C'<',MID=C',',TRAIL=C'>',PAIR=APOST))
```

%nn,squeeze
specifies that a left-squeezed or right-squeezed parsed input field is to appear in the reformatted input record. See PARSE for details of parsed fields. See p,m,squeeze for further details.

seqnum
specifies that a sequence number is to appear in the reformatted input record. The sequence numbers are assigned in the order in which the records are received for INREC processing. You can create BI, PD, ZD, CSF, or FS sequence numbers and control their lengths, starting values and increment values. You can restart the sequence number at the start value each time a specified input field (p,m) or parsed input field (%nn) changes.

See seqnum under OUTFIL OUTREC for details.

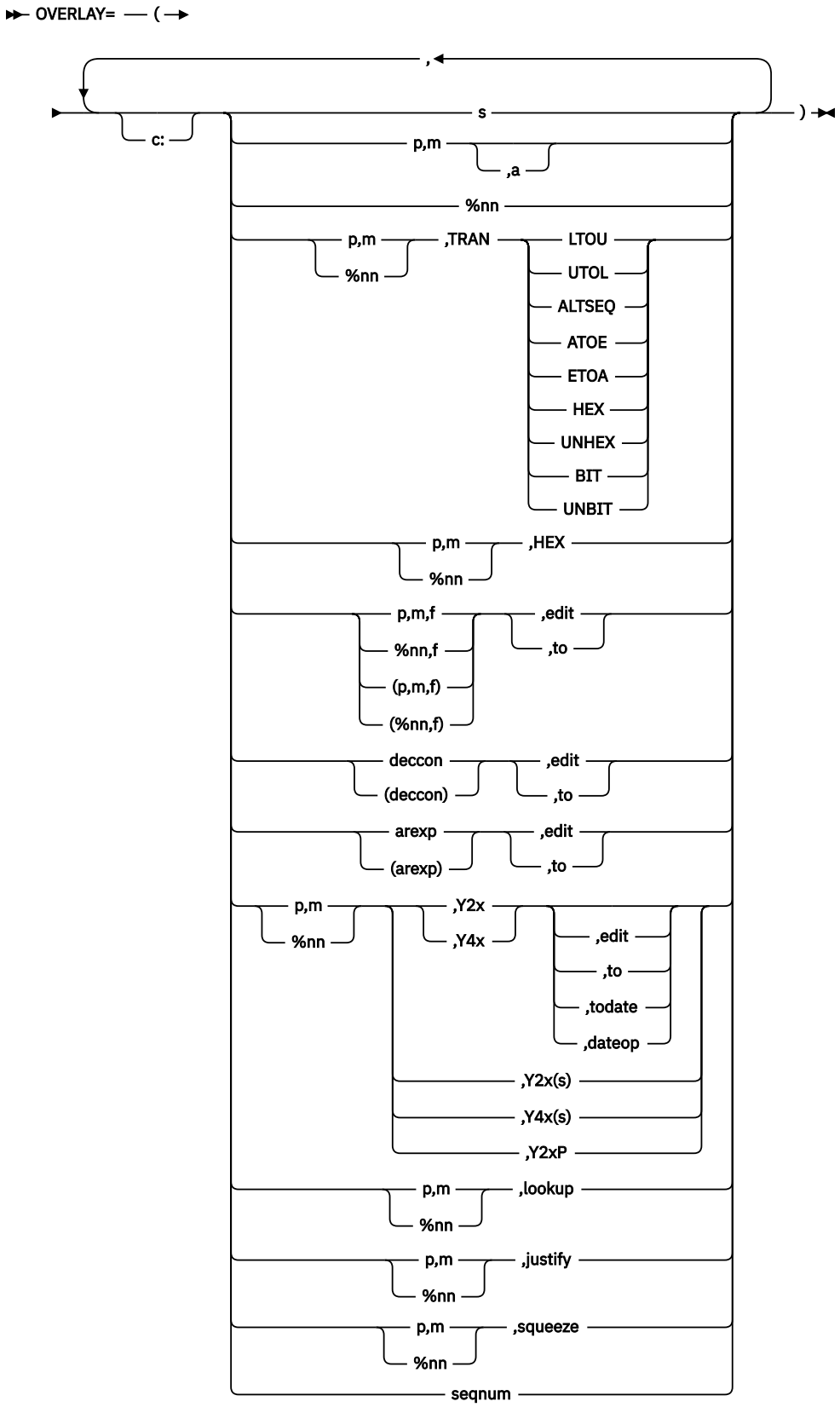
Sample syntax:

```
  INREC FIELDS=(1,80,SEQNUM,8,ZD)
```

Default for BUILD or FIELDS: None; must be specified. See Appendix B, “Specification/override of DFSORT options,” on page 801.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

OVERLAY



Specifies each item that is to overlay specific columns in the reformatted record. Columns that are not overlaid remain unchanged. If you want to insert, rearrange, or delete fields, use BUILD or FIELDS rather than OVERLAY. Use OVERLAY only to overlay existing columns or to add fields at the end of every record. OVERLAY can be easier to use then BUILD or FIELDS when you just want to change a few fields without rebuilding the entire record.

For fixed-length records, the first input and output data byte starts at position 1. For variable-length records, the first input and output data byte starts at position 5, after the RDW in positions 1-4.

Use c: (column) to specify the output positions to be overlaid. If you do not specify c: for the first item, it defaults to 1:. If you do not specify c: for any other item, it starts after the previous item. For example, if you specify:

```
INREC OVERLAY=(25,2,11:C'A',15,3,C'**')
```

Input positions 25-26 are placed at output positions 1-2; C'A' is placed at output position 11; input positions 15-17 are placed at output positions 12-14; and C'**' is placed at output positions 15–16. The rest of the record remains unchanged.

You can specify items in any order, you can change the same item multiple times and you can overlap output columns. **Changes to earlier items affect changes to later items.** For example, say you specify:

```
INREC OVERLAY=(21:8,4,ZD,MUL,+10,TO=ZD,LENGTH=6,
    5:5,1,TRAN=UTOL,
    5:5,1,CHANGE=(1,C'a',C'X',C'b',C'Y'),NOMATCH=(5,1))
```

and input position 5 has 'A'. The second item (UTOL) would change 'A' to 'a' and the third item (CHANGE) would change 'a' again to 'X'.

If you specify an OVERLAY item that extends the overlay record beyond the end of the input record, the reformatted record length is automatically increased to that length, and blanks are filled in on the left as needed. For variable-length records, the RDW length is also increased to correspond to the larger reformatted record length after all of the OVERLAY items are processed. For example, if your input record has a length of 40 and you specify:

```
INREC OVERLAY=(16:C'ABC',51:5C'*',35:15,2)
```

the output record is given a length of 55. Blanks are filled in from columns 41-50. For variable-length records, the length in the RDW is changed from 40 to 55 after all of the OVERLAY items are processed.

Missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

See INREC FIELDS for details of the items listed in the OVERLAY syntax diagram previously. You can specify all of the items for OVERLAY in the same way that you can specify them for BUILD or FIELDS with the following exceptions:

- You cannot specify p or p,HEX or p,TRAN=keyword for OVERLAY.
- For p,m,H or p,m,F or p,m,D fields specified for OVERLAY, fields are aligned as necessary without changing the preceding bytes.
- For variable-length records, you must not overlay positions 1-4 (the RDW) for OVERLAY, so be sure to specify the first column (c:) as 5 or greater. If you do not specify the first column, it will default to 1: which is invalid for variable-length records with OVERLAY. Whereas FIELDS=(1,m,...) is required, OVERLAY=(1,m) is not allowed, since it would overlay the RDW.

Sample syntax:

Fixed input records:

```
INREC OVERLAY=(21:21,4,ZD,TO=PD,LENGTH=4,
         2:5,8,HEX,45:C'*',32,4,C'*',81:SEQNUM,5,ZD)
```

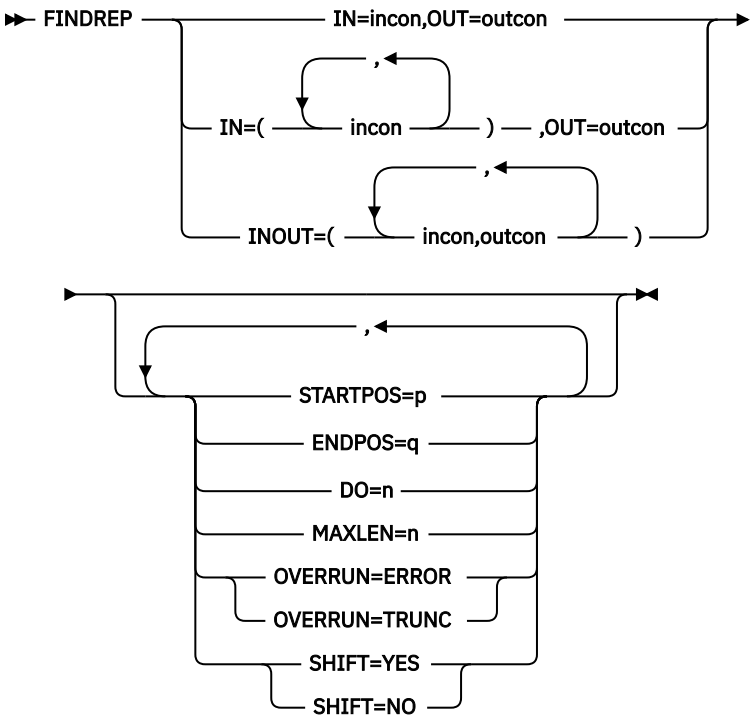
Variable input records:

```
INREC OVERLAY=(5:X'0001',28:C'Date is ',YDDDNS=(4D),
        17:5C'*')
```

Default for OVERLAY: None; must be specified. See Appendix B, “Specification/override of DFSORT options,” on page 801.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

FINDREP



You can use FINDREP to find constants anywhere in a record and replace them with other constants of the same or different lengths. You can find character or hexadecimal input constants anywhere in your records and replace them with character, hexadecimal or null output constants. As appropriate, bytes can be shifted left or right, blank padding can be added for fixed-length records, and the length can be changed for variable-length records.

Various options of FINDREP allow you to define one or more input constants and a corresponding output constant, define one or more pairs of input and output constants, start and end the find scan at specified positions, stop after a specified number of constants are replaced, increase or decrease the length of the output record, define the action to be taken if nonblank characters overrun the end of the record, and specify whether output constants are to replace or overlay input constants.

See FINDREP under "OUTFIL Control Statements" for complete details.

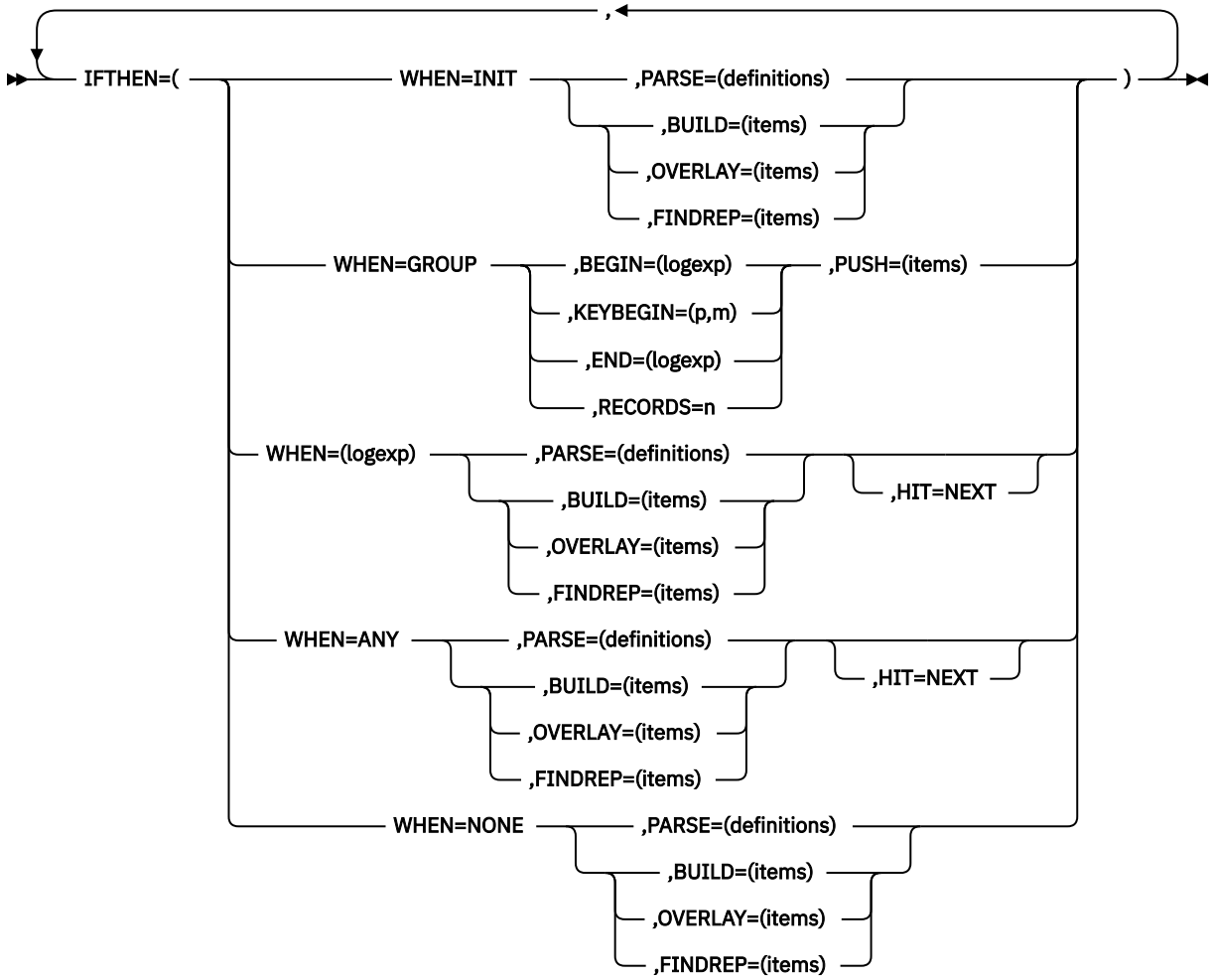
Sample syntax:

```
INREC FINDREP=(IN=C'Goodbye',OUT=C'Bye')
```

Default for FINDREP: None; must be specified. See Appendix B, “Specification/override of DFSORT options,” on page 801.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

IFTHEN



IFTHEN clauses allow you to reformat different records in different ways by specifying how build, overlay, find/replace or group operation items are to be applied to records that meet given criteria. IFTHEN clauses let you use simple or complex conditional logic to choose how different record types are reformatted.

If you want to insert, rearrange, or delete fields in the same way for every record, use BUILD or FIELDS rather than IFTHEN. If you want to overlay existing columns in the same way for every record, use OVERLAY rather than IFTHEN. If you want to do find and replace operations in the same way for every record, use FINDREP rather than IFTHEN. Use IFTHEN clauses if you want to insert, rearrange, delete or overlay fields, or perform find/replace operations in different ways for different records., or if you want to perform operations on groups of records.

You can use five types of IFTHEN clauses as follows:

- **WHEN=INIT:** Use one or more WHEN=INIT clauses to apply build, overlay or find/replace items to all of your input records. WHEN=INIT clauses and WHEN=GROUP clauses are processed before any of the other IFTHEN clauses.
- **WHEN=GROUP**: Use one or more WHEN=GROUP clauses to propagate fields, identifiers and sequence numbers within specified groups of records. WHEN=INIT clauses and WHEN=GROUP clauses are processed before any of the other IFTHEN clauses.
- **WHEN=(logexp):** Use one or more WHEN=(logexp) clauses to apply build, overlay or find/replace items to your input records that meet specified criteria. A WHEN=(logexp) clause is satisfied when the logical expression evaluates as true.

- **WHEN=ANY:** Use a WHEN=ANY clause after multiple WHEN=(logexp) clauses to apply additional build, overlay or find/replace items to your input records if they satisfied the criteria for any of the preceding WHEN=(logexp) clauses.
- **WHEN=NONE:** Use one or more WHEN=NONE clauses to apply build, overlay or find/replace items to your input records that did not meet the criteria for any of the WHEN=(logexp) clauses. WHEN=NONE clauses are processed after any of the other IFTHEN clauses. If you do not specify a WHEN=NONE clause, only the WHEN=INIT and WHEN=GROUP changes (if any) are applied to input records that do not meet the criteria for any of the WHEN=(logexp) clauses.

IFTHEN clauses are processed in the following order:

- WHEN=INIT clauses and WHEN=GROUP clauses
- WHEN=(logexp) clauses and WHEN=ANY clauses
- WHEN=NONE clauses

Processing of IFTHEN clauses continues unless one of the following occurs:

- A WHEN=(logexp) or WHEN=ANY clause is satisfied, and HIT=NEXT is not specified.
- There are no more IFTHEN clauses to process. When processing of IFTHEN clauses stops, the IFTHEN record created so far is used as the output record.

Example:

```
  INREC IFTHEN=(WHEN=(12,1,BI,ALL,X'3F'),OVERLAY=(18:C'Yes')),
        IFTHEN=(WHEN=(35,2,PD,EQ,+14),BUILD=(1,40,45,3,HEX),HIT=NEXT),
        IFTHEN=(WHEN=(35,2,PD,GT,+17),BUILD=(1,40,41,5,HEX),HIT=NEXT),
        IFTHEN=(WHEN=ANY,BUILD=(1,55,C'ABC',70:X)),
        IFTHEN=(WHEN=(63,2,CH,EQ,C'AB'),OVERLAY=(18:C'No')),
        IFTHEN=(WHEN=NONE,BUILD=(1,40,51,8,TRAN=LTOU))
```

For this example, the IFTHEN clauses are processed as follows:

- If IFTHEN clause 1 is satisfied, its overlay item is applied and IFTHEN processing stops.
- If IFTHEN clause 1 is not satisfied, its overlay item is not applied and IFTHEN processing continues.
- If IFTHEN clause 2 is satisfied, its build items are applied and IFTHEN processing continues.
- If IFTHEN clause 2 is not satisfied, its build items are not applied and IFTHEN processing continues.
- If IFTHEN clause 3 is satisfied, its build items are applied and IFTHEN processing continues.
- If IFTHEN clause 3 is not satisfied, its build items are not applied and IFTHEN processing continues.
- If IFTHEN clause 4 is satisfied, its build items are applied and IFTHEN processing stops.
- If IFTHEN clause 4 is not satisfied, its build items are not applied and IFTHEN processing continues.
- If IFTHEN clause 5 is satisfied, its overlay item is applied and IFTHEN processing stops.
- If IFTHEN clause 5 is not satisfied, its overlay item is not applied and IFTHEN processing continues.
- If IFTHEN clause 6 is satisfied, its build items are applied and IFTHEN processing stops.
- If IFTHEN clause 6 is not satisfied, its build items are not applied and IFTHEN processing stops.

All of the IFTHEN clauses operate sequentially on an IFTHEN record. The IFTHEN record is created initially from the input record. Each IFTHEN clause tests and changes the IFTHEN record, as appropriate. Thus, **changes made by earlier IFTHEN clauses are "seen" by later IFTHEN clauses.** For example, if you have a 40-byte input record and specify:

```
  INREC IFTHEN=(WHEN=INIT,OVERLAY=(8:8,4,ZD,ADD,+1,TO=ZD,LENGTH=4)),
        IFTHEN=(WHEN=(8,4,ZD,EQ,+27),OVERLAY=(28:C'Yes')),
        IFTHEN=(WHEN=NONE,OVERLAY=(28:C'No'))
```

The WHEN=INIT clause adds 1 to the ZD value and stores it in the IFTHEN record. The WHEN=(8,4,ZD,EQ,+27) clause tests the incremented ZD value in the IFTHEN record rather than the original ZD value in the input record.

The IFTHEN record is adjusted as needed for the records created or changed by the IFTHEN clauses. For fixed-length records, blanks are filled in on the left as needed. For variable-length records, the RDW length is adjusted as needed each time the IFTHEN record is changed.

Missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

DFSORT sets an appropriate LRECL (or reformatted record length if the INREC record is further modified) for the output records based on the build, overlay, find/replace and group operation items specified by the IFTHEN clauses. However, DFSORT does not analyze the possible results of WHEN=(logexp) conditions when determining an appropriate LRECL. When you use INREC IFTHEN clauses, you can override the INREC LRECL determined by DFSORT with the INREC IFOUTLEN parameter.

If SEQNUM is used in multiple IFTHEN clauses, the sequence number will be incremented for each record that satisfies the IFTHEN clause, that is, a separate SEQNUM counter will be kept for each IFTHEN clause. For example, if your input is:

```
RECORD A 1
RECORD B 1
RECORD B 2
RECORD C 1
RECORD A 2
RECORD C 2
RECORD B 3
RECORD D 1
```

and you specify:

```
  INREC IFTHEN=(WHEN=(8,1,CH,EQ,C'A'),OVERLAY=(15:SEQNUM,4,ZD)),
        IFTHEN=(WHEN=(8,1,CH,EQ,C'B'),OVERLAY=(16:SEQNUM,4,ZD)),
        IFTHEN=(WHEN=NONE,OVERLAY=(17:SEQNUM,4,ZD))
```

your output will be:

```
RECORD A 1    0001
RECORD B 1     0001
RECORD B 2     0002
RECORD C 1      0001
RECORD A 2    0002
RECORD C 2      0002
RECORD B 3     0003
RECORD D 1      0003
```

Separate SEQNUM counters are kept for the 'A' record, for the 'B' record, and for the NONE records.

WHEN=INIT clause

See "WHEN=INIT clause" under OUTFIL IFTHEN for details. Note that / cannot be used to create blank records or new records.

Sample syntax:

```
  INREC IFTHEN=(WHEN=INIT,
                BUILD=(1,20,21:C'Department',31:3X,21,60)),
        IFTHEN=(WHEN=(5,2,CH,EQ,C'D1'),OVERLAY=(31:8,3)),
        IFTHEN=(WHEN=(5,2,CH,EQ,C'D2'),OVERLAY=(31:12,3))
```

WHEN=GROUP clause

See "WHEN=GROUP clause" under OUTFIL IFTHEN for details.

Sample syntax:

```
  INREC IFTHEN=(WHEN=GROUP,
       BEGIN=(1,40,SS,EQ,C'J82',OR,1,40,SS,EQ,C'M72'),
       PUSH=(41:ID=5))
```

WHEN=(logexp) clause

See "WHEN=(logexp) clause" under OUTFIL IFTHEN for details. Note that although / can be used to create blank records and new records with OUTFIL, it cannot be used with INREC.

Sample Syntax:

```
 INREC IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,
          18,4,ZD,LE,+2000),OVERLAY=(42:C'Type1 <= 2000'),HIT=NEXT),
       IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,6,1,BI,BO,X'03'),
          BUILD=(1,21,42,13)),
       IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,
          18,4,ZD,GT,+2000),OVERLAY=(42:C'Type1 > 2000 '),HIT=NEXT),
       IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,6,1,BI,BO,X'01'),
          BUILD=(1,25,42,13))
```

WHEN=ANY clause

See "WHEN=ANY clause" under OUTFIL IFTHEN for details. Note that although / can be used to create blank records and new records with OUTFIL, it cannot be used with INREC.

Sample Syntax:

```
INREC IFTHEN=(WHEN=(1,3,SS,EQ,C'T01,T02,T03'),
              BUILD=(C'Group A',X,1,80),HIT=NEXT),
      IFTHEN=(WHEN=(1,3,SS,EQ,C'T04,T05,T06'),
              BUILD=(C'Group B',X,1,80),HIT=NEXT),
      IFTHEN=(WHEN=(1,3,SS,EQ,C'T07,T08,T09,T10'),
              BUILD=(C'Group C',X,1,80),HIT=NEXT),
      IFTHEN=(WHEN=ANY,OVERLAY=(16:C'Group Found'))
```

WHEN=NONE clause

See "WHEN=NONE clause" under OUTFIL IFTHEN for details. Note that although / can be used to create blank records and new records with OUTFIL, it cannot be used with INREC.

Sample Syntax:

```
INREC IFTHEN=(WHEN=INIT,BUILD=(1,20,21:C'Department',31:3X,21,60)),
      IFTHEN=(WHEN=(5,2,CH,EQ,C'D1'),OVERLAY=(31:8,3)),
      IFTHEN=(WHEN=(5,2,CH,EQ,C'D2'),OVERLAY=(31:12,3)),
      IFTHEN=(WHEN=NONE,OVERLAY=(31:C'***'))
```

Default for IFTHEN clauses: None; must be specified. See Appendix B, "Specification/Override of DFSORT Options".

Applicable Functions: See Appendix B, "Specification/Override of DFSORT Options".

IFOUTLEN



Overrides the INREC LRECL (or reformatted record length if the INREC record is further modified) determined by DFSORT from your INREC IFTHEN clauses. DFSORT sets an appropriate LRECL for the output records based on the build, overlay, find/replace and group operation items specified by the IFTHEN clauses. However, DFSORT does not analyze the possible results of WHEN=(logexp) conditions when determining an appropriate INREC LRECL. When you use INREC IFTHEN clauses, you can override the INREC LRECL determined by DFSORT with the INREC IFOUTLEN parameter.

Fixed-length records longer than the IFOUTLEN length are truncated to the IFOUTLEN length. Fixed-length records shorter than the IFOUTLEN are padded with blanks to the IFOUTLEN length. Variable-length records longer than the IFOUTLEN length are truncated to the IFOUTLEN length.

n

specifies the length to use for the INREC LRECL (or for the reformatted record length if the INREC record is further modified) . The value for n must be between 1 and 32767, but must not be larger than the maximum LRECL allowed for the RECFM, and must not conflict with the specified or retrieved LRECL for the fixed-length output data set.

Sample syntax:

```
INREC IFOUTLEN=70,
  IFTHEN=(WHEN=(5,1,CH,EQ,C'1',AND,8,3,ZD,EQ,+10),
      BUILD=(1,40,C'T01-GROUP-A',65)),
  IFTHEN=(WHEN=(5,1,CH,EQ,C'2',AND,8,3,ZD,EQ,+12),
      BUILD=(1,40,C'T02-GROUP-B',65))
```

Default for IFOUTLEN: The LRECL determined from the IFTHEN clauses.

INREC statement notes

- When INREC is specified, DFSORT reformats the input records after user exit E15 or INCLUDE/OMIT statement processing is finished. Thus, references to fields by your E15 user exit and INCLUDE/OMIT statements are not affected, whereas your SORT, OUTREC, and SUM statements must refer to fields in the reformatted input records. Your E35 user exit must refer to fields in the reformatted output record.
- In general, OUTREC should be used rather than INREC so your SORT and SUM statements can refer to fields in the original input records.
- If you use locale processing for SORT, MERGE, INCLUDE, or OMIT fields, you must not use INREC. Use the OUTREC statement or the OUTFIL statement instead of INREC.
- When you specify INREC, you must be aware of the change in record size and layout of the resulting reformatted input records.
- Performance can be improved if you can significantly reduce the length of your records with INREC. INREC and OUTREC should not be used unless they are actually needed to reformat your records.
- For variable-length records, the first entry in the FIELDS, BUILD, or IFTHEN BUILD parameter must specify or include the unedited 4-byte record descriptor word (RDW), that is, the first field must be 1,4 or 1,m with m greater than 4. DFSORT sets the length of the reformatted record in the RDW.

  If the first field in the data portion of the input record is to appear unedited in the reformatted record immediately following the RDW, the entry in the FIELDS, BUILD, or IFTHEN BUILD parameter can specify both RDW and data field in one (1,m,...). Otherwise, the RDW must be specifically included in the reformatted record (for example, 1,4,1,4,HEX).
- For variable-length records, OVERLAY, IFTHEN OVERLAY or IFTHEN PUSH items must not overlay the RDW in bytes 1-4. You must ensure that 1:, 2:, 3: or 4: is not specified or defaulted for any OVERLAY or PUSH item. Note that the default for the first OVERLAY or PUSH item is 1:, so you must override it.
- If the SORTOUT LRECL is specified or available, DFSORT will use it even if it does not match the reformatted INREC record length; this can cause padding or truncation of the reformatted INREC records, or termination. If the SORTOUT LRECL is not specified or available, DFSORT can automatically use the reformatted INREC record length as the SORTOUT LRECL, when appropriate. See the discussion of the SOLRF and NOSOLRF options in “OPTION control statement” on page 171.

  For VSAM data sets, the maximum record size defined in the cluster is equivalent to the LRECL when processing fixed-length records, and is four bytes less than the LRECL when processing variable-length records. See “VSAM considerations ” on page 15 for more information.
- With FIELDS, BUILD, or IFTHEN BUILD, the variable part of the input record (that part beyond the minimum record length) can be included in the reformatted record, and if included, must be the last part. In this case, a value must be specified for p*n* that is less than or equal to the minimum record length (see L4 of the RECORD control statement) plus 1 byte; m*n* and a*n* must be omitted. For example:

  ```
  INREC FIELDS=(1,8,20C'*',9)
  ```

  With OVERLAY, the variable part of the input record must not be included in the reformatted record.
- If INREC with FIELDS or BUILD and OUTREC with FIELDS and BUILD are specified, either both must specify position-only for the last part, or neither must specify position-only for the last part. For example:

  ```
  INREC BUILD=(1,8,20C'*',9)
  OUTREC BUILD=(1,4,3Z,5)
  ```

  or:

```
 INREC FIELDS=(1,40,45,5)
 OUTREC FIELDS=(1,45,C'****')
```

OVERLAY or IFTHEN, and FIELDS or BUILD, can differ with respect to position-only. For example:

```
INREC BUILD=(1,24,32:25)
OUTREC IFTHEN=(WHEN=(8,1,ZD,GT,+5),
  BUILD=(1,24,25:C'Yes',28,10)),
  IFTHEN=(WHEN=NONE,
    BUILD=(1,24,25:C'No ',28,10))
```

or:

```
INREC FIELDS=(1,18,8C'*',23)
OUTREC OVERLAY=(24:C'A')
```

- If the reformatted record includes only the RDW and the variable part of the input record, "null" records containing only an RDW could result.
-
- The input records are reformatted before processing, as specified by INREC. The output records are in the format specified by INREC, unless OUTREC is also specified.
- Fields referenced in INREC statements can overlap each other and control fields or both.
- If input is variable records, the output is also variable. This means that each record is given the correct RDW by DFSORT before output.
-
- If overflow might occur during summation, INREC can be used to create a larger SUM field in the reformatted input record (perhaps resulting in a larger record for sorting or merging) so that overflow does not occur. “Example 5” on page 435 illustrates this technique.
- DFSORT issues a message and terminates if an INREC statement is specified for a tape work data set sort or conventional merge application.
- If SZERO is in effect, -0 is treated as negative and +0 is treated as positive for edited or converted input fields, decimal constants, and the results of arithmetic expressions. If NOSZERO is in effect, -0 and +0 are treated as positive for edited or converted input fields, decimal constants, and the results of arithmetic expressions.
- If SZERO is in effect, -0 compares as less than +0 when numeric fields and constants are used. If NOSZERO is in effect, -0 compares as equal to +0 when numeric fields and constants are used.

  Note: OPTION SZERO or OPTION NOSZERO is ignored for INREC IFTHEN=(WHEN=(logexp),...) unless the OPTION statement is found in a higher source (for example, DFSPARM is a higher source than SYSIN) or before the INREC statement in the same source. For example, NOSZERO will be used in both of the following cases:

  – Case 1:

  ```
  //DFSPARM DD *
    OPTION COPY,NOSZERO
  /*
  //SYSIN DD *
    INREC IFTHEN=(WHEN=(1,2,FS,EQ,+0),OVERLAY=(22:C'Yes')),
          IFTHEN=(WHEN=NONE,OVERLAY=(22:C'No '))
  /*
  ```

  – Case 2:

  ```
  //SYSIN DD *
    OPTION COPY,NOSZERO
    INREC IFTHEN=(WHEN=(1,2,FS,EQ,+0),OVERLAY=(28:C'A')),
          IFTHEN=(WHEN=NONE,OVERLAY=(28:C'B'))
  /*
  ```

Reformatting records before processing — examples

Both INREC and OUTREC can be used to reformat records, either separately or together. The two control statements perform similar functions, although INREC reformats records before sort, merge, or copy processing, whereas OUTREC reformats records after sort, merge, or copy processing. This section shows INREC examples. See “Reformatting records after processing — examples” on page 405 for OUTREC examples.

Example 1

INREC method

```
INCLUDE COND=(5,1,GE,C'M'),FORMAT=CH
INREC FIELDS=(10,3,20,8,33,11,5,1)
SORT FIELDS=(4,8,CH,A,1,3,FI,A)
SUM FIELDS=(17,4,BI)
```

OUTREC method

```
INCLUDE COND=(5,1,GE,C'M'),FORMAT=CH
OUTREC FIELDS=(10,3,20,8,33,11,5,1)
SORT FIELDS=(20,8,CH,A,10,3,FI,A)
SUM FIELDS=(38,4,BI)
```

These examples illustrate how a fixed-length input data set is sorted and reformatted for output. Unnecessary fields are eliminated from the output records using INREC or OUTREC. The SORTIN LRECL is 80.

Records are also included or excluded by means of the INCLUDE statement, and summed by means of the SUM statement.

The reformatted input records are fixed length with a record size of 23 bytes. SOLRF (the IBM-supplied default) is in effect, so unless the SORTOUT LRECL is specified or available, it will automatically be set to the reformatted record length of 23. The reformatted records look as follows after INREC or OUTREC processing:

Position
Contents

1-3
Input positions 10 through 12

4-11
Input positions 20 through 27

12-22
Input positions 33 through 43

23
Input position 5

Identical results are achieved with INREC or OUTREC. However, use of OUTREC makes it easier to code the SORT and SUM statements. In either case, the INCLUDE COND parameters must refer to the fields of the original input records. However, with INREC, the SUM and SORT FIELDS parameters must refer to the fields of the reformatted input records, while with OUTREC, the SUM and SORT FIELDS parameters must refer to the fields of the original input records.

Example 2

```
INREC FIELDS=(1,35,2Z,36,45)
MERGE FIELDS=(20,4,CH,D,10,3,CH,D),FILES=3
SUM FIELDS=(36,4,BI,40,8,PD)
RECORD TYPE=F,LENGTH=(80,,82)
```

This example illustrates how overflow of a summary field can be prevented when three fixed-length data sets are merged and reformatted for output. The input record size is 80 bytes. To illustrate the use of the RECORD statement, assume that SORTIN and SORTOUT are not present (that is, all input/output is handled by user exits).

The reformatted input records are fixed-length with a record size of 82 bytes (an insignificant increase from the original size of 80 bytes). They look as follows:

Position
Contents

1-35
Input positions 1 through 35

36-37
Binary zeros (to prevent overflow)

38-82
Input positions 36 through 80

The MERGE and SUM statements must refer to the fields of the reformatted input records.

The reformatted output records are identical to the reformatted input records.

Thus, the 2-byte summary field at positions 36 and 37 in the original input records expands to a 4-byte summary field in positions 36 through 39 of the reformatted input/output record before merging. This prevents overflow of this summary field.

If OUTREC were used instead of INREC, the records would be reformatted after merging, and the 2-byte summary field might overflow.

Restriction: This method of preventing overflow cannot be used for negative FI summary fields because padding with zeros rather than ones would change the sign.

Example 3

```
INREC BUILD=(20,4,12,3)
SORT FIELDS=(1,4,D,5,3,D),FORMAT=CH
OUTREC BUILD=(5X,1,4,H,19:1,2,5,3,DATE1(-),80X'FF')
```

This example illustrates how a fixed-length input data set can be sorted and reformatted for output. A more efficient sort is achieved by using INREC before sorting to reduce the input records as much as possible, and using OUTREC after sorting to add padding, the current date and repeated fields. The SORTIN LRECL is 80 bytes.

The reformatted input records are fixed-length, and have a record size of seven bytes (a significant reduction from the original size of 80 bytes). They look as follows:

Position
Contents

1-4
Input positions 20 through 23

5-7
Input positions 12 through 14

The SORT and OUTREC statements must refer to the fields of the reformatted input records.

The reformatted output records are fixed length with a record size of 113 bytes. SOLRF (the IBM-supplied default) is in effect, so unless the SORTOUT LRECL is specified or available, it will automatically be set to the reformatted record length of 113. The reformatted output records look as follows:

Position
Contents

1-5
EBCDIC blanks

6
Binary zero (for H alignment)

7-10
Input positions 20 through 23

11-18
EBCDIC blanks

19-20
Input positions 20 through 21

21-23
Input positions 12 through 14

24-33
The current date in the form C'yyyy-mm-dd'

34-113
Hexadecimal FF's

Thus, the use of INREC and OUTREC allows sorting of 7-byte records rather than 80-byte records, even though the output records are 113 bytes long.

Example 4

```
OPTION COPY,Y2PAST=1985
INREC FIELDS=(SEQNUM,4,BI,
              8,5,ZD,TO=PD,
              31,2,PD,TO=FI,LENGTH=2,
              15,6,Y2TP,
              25,3,CHANGE=(1,C'L92',X'01',C'M72',X'02',C'J42',X'03'),
              NOMATCH=(X'FF'))
```

This example illustrates how a sequence number can be generated, how values in one numeric or date format can be converted to another format, and how a lookup table can be used.

The reformatted input records will look as follows:

Position
Contents

1-4
A binary sequence number that starts at 1 and increments by 1.

5–7
A PD field containing the converted ZD field from input positions 8 through 12.

8–9
An FI field containing the converted PD field from input positions 31 through 32.

10–14
A P'yyyymmdd' date field containing the C'yymmdd' date field from input positions 15-20 transformed according to the specified century window of 1985-2084.

15
A BI field containing X'01', X'02', X'03' or X'FF' as determined by using a lookup table for the input field in positions 25-27.

The SORT statement can now refer to the "sort" field in the reformatted input records. The OUTREC statement is used to restore the records to their original format.

Example 5

```
INREC OVERLAY=(61:21,11,SFF,ADD,41,11,SFF,TO=PD,LENGTH=5)
SORT FIELDS=(61,5,PD,A)
OUTREC OVERLAY=(61:61,5,PD,EDIT=(SIII,IIT.TT),SIGNS=(+,-))
```

This example illustrates how you can use the OVERLAY parameter with INREC and OUTREC to change certain columns in your records without affecting other columns.

Positions 61-65 of the reformatted input records are overlaid with a 5-byte PD value derived from adding the sddd,ddd.dd field at positions 21-31 to the sddd,ddd.dd field at positions 41-51. The records are then sorted by this 5-byte PD field. Positions 61-71 of the reformatted output records are overlaid with an sddd,ddd.dd field derived from the 5-byte PD value. The data before positions 61-71 and after positions 61-71 are not affected

Example 6

```
  OPTION COPY
  INREC IFTHEN=(WHEN=(5,2,CH,EQ,C'GP',AND,2,1,BI,EQ,+1),
                BUILD=(1,6,16,20,C'T1',X'0003',1,7,20C'1')),
        IFTHEN=(WHEN=(5,2,CH,EQ,C'GP',AND,2,1,BI,EQ,+2),
                BUILD=(1,6,45,20,C'T2',X'0008',16,7,20C'2')),
        IFTHEN=(WHEN=(5,2,CH,EQ,C'GP',AND,2,1,BI,EQ,+3),
                BUILD=(1,6,31,20,C'T3',X'0005',25,7,20C'3')),
        IFTHEN=(WHEN=NONE,OVERLAY=(27:C'00',X'FFFF')),
     IFOUTLEN=57
```

This example illustrates how you can use IFTHEN clauses with INREC to reformat different records in different ways. IFOUTLEN=57 is used to set the reformatted record length to 57.

Records with 'GP' in positions 5-6 and X'01' in position 2 are reformatted as follows:

Position
Contents

1-6
Input positions 1-6

7-26
Input positions 16-35

27-28
'T1'

29-30
X'0003'

31-37
Input positions 1-7

38-57
20 '1's

Records with 'GP' in positions 5-6 and X'02' in position 2 are reformatted as follows:

Position
Contents

1-6
Input positions 1-6

7-26
Input positions 45-64

27-28
'T2'

29-30
X'0008'

31-37
Input positions 16-22

38-57
20 '2's

Records with 'GP' in positions 5-6 and X'03' in position 2 are reformatted as follows:

Position
Contents

1-6
Input positions 1-6

7-26
Input positions 31-50

27-28
'T3'

29-30
X'0005'

31-37
Input positions 25-31

38-57
20 '3's

Records without 'GP' in positions 5-6 or without X'01', X'02', or X'03' in position 2 are reformatted as follows:

Position
Contents

1-26
Input positions 1-26

27-28
'00'

29-30
X'FFFF'

31-57
Input positions 31-57

Example 7

```
  INREC OVERLAY=(16:1,15,JFY=(SHIFT=LEFT))
  SORT FIELDS=(16,15,CH,A)
  OUTREC BUILD=(1,15)
```

This example illustrates how you can left-justify characters in an input field so they can be sorted without regard to the leading blanks.

The 15-byte input records might look like this:

```
   CARRIE
       VICKY
          FRANK
      SAM
DAVID
    MARTIN
```

Note that if we sort these records using just this control statement:

```
   SORT FIELDS=(1,15,CH,A)
```

The 15-byte output records are as follows:

```
          FRANK
       VICKY
      SAM
    MARTIN
   CARRIE
DAVID
```

Because of the different number of leading blanks in the input records, we don't get what we want. To fix that, while keeping the leading blanks in the original records, we use the JFY function of INREC to make a left-justified copy of the 15-byte input field at positions 16-30, SORT on it and use OUTREC to remove the left-justified field. With the INREC, SORT and OUTREC control statements shown previously, the output records are:

```
   CARRIE
DAVID
          FRANK
    MARTIN
      SAM
       VICKY
```

If we wanted the output to contain the sorted left-justified fields, we could use these control statements:

```
  INREC BUILD=(1,15,JFY=(SHIFT=LEFT))
  SORT FIELDS=(1,15,CH,A)
```

The output records would then be:

```
CARRIE
DAVID
FRANK
MARTIN
SAM
VICKY
```

Example 8

```
  INREC PARSE=(%00=(ENDBEFR=C',',FIXLEN=11),
               %01=(ENDBEFR=C',',FIXLEN=5),
               %02=(FIXLEN=6)),
        OVERLAY=(31:%00,42:%01,47:%02)
  SORT FIELDS=(31,11,CH,A,42,5,UFF,A,47,6,SFF,D)
  OUTREC BUILD=(1,30)
```

This example illustrates how you can sort FB input records with variable position/length fields, such as comma separated values.

The 30-byte input records might look like this:

```
Marketing,96218,+27365
Development,3807,+1275
Research,7283,+5001
Development,1700,-5316
Research,978,+13562
Development,3807,-158
Research,7283,+5002
Marketing,52,-8736
Development,5781,+2736
Marketing,52,+1603
Research,16072,-2022
```

We want to sort the first field as character ascending, the second field as unsigned numeric ascending and the third field as signed numeric descending.

Note that each record has three variable fields in comma separated value format. The fields do not start and end in the same position in every record and do not have the same length in every record. The first and second fields end with a comma and the third field ends with a blank.

In order to sort variable fields like these, we use the PARSE and OVERLAY functions of INREC to create a fixed parsed copy of each variable field. We use %00 to create an 11-byte fixed parsed field into which we extract the value before the first comma. We use %01 to create a 5-byte fixed parsed field into which we extract the value after the first comma and before the second comma. We use %02 to create a 6-byte fixed parsed field into which we extract the value after the second comma. Then we SORT on the fixed parsed fields. Finally, we use OUTREC to remove the fixed parsed fields.

After the INREC statement is processed, the records look like this:

```
Marketing,96218,+27365        Marketing  96218+27365
Development,3807,+1275        Development3807 +1275
Research,7283,+5001           Research   7283 +5001
Development,1700,-5316        Development1700 -5316
Research,978,+13562           Research   978  +13562
Development,3807,-158         Development3807 -158
Research,7283,+5002           Research   7283 +5002
Marketing,52,-8736            Marketing  52   -8736
Development,5781,+2736        Development5781 +2736
Marketing,52,+1603            Marketing  52   +1603
Research,16072,-2022          Research   16072-2022
```

Since the second fixed parsed field is unsigned and left-justified, we sort it with the UFF format. Since the third fixed parsed field is signed and left-justified, we sort it with the SFF format.

After the SORT and OUTREC statements are processed, the output records look like this:

```
Development,1700,-5316
Development,3807,+1275
Development,3807,-158
Development,5781,+2736
Marketing,52,+1603
Marketing,52,-8736
Marketing,96218,+27365
Research,978,+13562
Research,7283,+5002
Research,7283,+5001
Research,16072,-2022
```

Example 9

```
  INREC PARSE=(%00=(ENDBEFR=C',',FIXLEN=11),
               %01=(ENDBEFR=C',',FIXLEN=5),
               %02=(FIXLEN=6)),
        BUILD=(1,4,5:%00,16:%01,21:%02,27:5)
  SORT FIELDS=(5,11,CH,A,16,5,UFF,A,21,6,SFF,D)
  OUTREC BUILD=(1,4,27)
```

This example illustrates how you can sort VB input records with variable position/length fields, such as comma separated values. This example is very similar to the previous example for FB records, except that with VB records we need to copy the fixed parsed fields after the 4-byte RDW rather than at the end of the records.

The VB input records might look like this:

```
Length|Data
    26|Marketing,96218,+27365
    26|Development,3807,+1275
    23|Research,7283,+5001
    26|Development,1700,-5316
    23|Research,978,+13562
    25|Development,3807,-158
    23|Research,7283,+5002
    22|Marketing,52,-8736
    26|Development,5781,+2736
    22|Marketing,52,+1603
    24|Research,16072,-2022
```

After the INREC statement is processed, the records look like this:

```
Length|Data
    48|Marketing  96218+27365Marketing,96218,+27365
    48|Development3807 +1275 Development,3807,+1275
    45|Research   7283 +5001 Research,7283,+5001
    48|Development1700 -5316 Development,1700,-5316
    45|Research   978  +13562Research,978,+13562
    47|Development3807 -158  Development,3807,-158
    45|Research   7283 +5002 Research,7283,+5002
    44|Marketing  52   -8736 Marketing,52,-8736
    48|Development5781 +2736 Development,5781,+2736
    44|Marketing  52   +1603 Marketing,52,+1603
    46|Research   16072-2022 Research,16072,-2022
```

After the SORT and OUTREC statements are processed, the output records look like this:

```
Length|Data
    26|Development,1700,-5316
    26|Development,3807,+1275
    25|Development,3807,-158
    26|Development,5781,+2736
    22|Marketing,52,+1603
    22|Marketing,52,-8736
    26|Marketing,96218,+27365
    23|Research,978,+13562
    23|Research,7283,+5002
    23|Research,7283,+5001
    24|Research,16072,-2022
```

Example 10

```
  OPTION COPY
  INREC FINDREP=(IN=(X'00',X'FF'),OUT=C'')
```

This example illustrates how you can remove characters from FB or VB records.

The VB input records might look like this in hexadecimal:

```
RDW----|Data
000F0000D1E4D5C5**00**C1D7D9C9D3**FF**
00100000C2C5E3E3E8**00**C4C1C9E2E8**FF**
```

We want to remove each X'00' and X'FF' character. We use IN=(X'00',X'FF') to indicate we want to find each X'00' and X'FF' character, and OUT=C'' (null) to indicate we want to remove it.

The output records look like this:

```
RDW----|Data
000D0000D1E4D5C5C1D7D9C9D3
000E0000C2C5E3E3E8C4C1C9E2E8
```

Note that the X'00' and X'FF' characters have been removed and the RDW length decreased accordingly. For VB input records, FINDREP processing automatically starts at position 5 after the RDW so the X'00' characters in the RDW are not affected.

Example 11

```
  OPTION COPY
  INREC FINDREP=(IN=C'BALANCE',
                OUT=C'BALANCE 1000',SHIFT=NO,DO=1)
```

This example illustrates how you can find a value in FB or VB records and overlay it with a larger value without shifting other bytes in the records.

The FB input records might look like this:

```
CUSTOMER1 10100
MNTHLY STMT BALANCE 2000
CUSTOMER2 11100
MNTHLY STMT REQUIRES NO MODIFICATION ACCT BALANCE 5000
CUSTOMER3 11111
YOUR INFO ENCLOSED
MNTHLY STMT REQUIRES MODIFICATION ACCT BALANCE 7000
```

We want to replace every instance of 'BALANCE dddd' with 'BALANCE 1000' where dddd can be any value. We use IN=C'BALANCE' to indicate we want to find each instance of 'BALANCE'. We use OUT='BALANCE 1000' to indicate we want to replace it with 'BALANCE 1000'. We use SHIFT=NO to do an overlay, overriding the default of shifting bytes. Since we only have at most one instance of 'BALANCE dddd' in a record, we can use DO=1 to stop processing a record after one replacement of 'BALANCE dddd' with 'BALANCE 1000'. In this case, DO=1 is more efficient than the default of continuing to look for more

instances of 'BALANCE dddd' after the first instance. We would get the same result without DO=1, just less efficiently

The output records look like this:

```
CUSTOMER1 10100
MNTHLY STMT BALANCE 1000
CUSTOMER2 11100
MNTHLY STMT REQUIRES NO MODIFICATION ACCT BALANCE 1000
CUSTOMER3 11111
YOUR INFO ENCLOSED
MNTHLY STMT REQUIRES MODIFICATION ACCT BALANCE 1000
```

Example 12

```
  INREC IFTHEN=(WHEN=GROUP,BEGIN=(2,4,CH,EQ,C'RPT.'),
    PUSH=(31:6,8))
  OPTION EQUALS
  SORT FIELDS=(31,8,CH,A)
  OUTFIL INCLUDE=(31,8,CH,EQ,C'FRANK',OR,
     31,8,CH,EQ,C'SRIHARI'),BUILD=(1,30)
```

This example illustrates how you can SORT and INCLUDE groups of FB records depending on a value in the first record of each group. We propagate the value in the first record of the group to every record of the group, SORT and INCLUDE on the value, and then remove it.

The 30-byte FBA input records might look like this:

```
1RPT.SRIHARI
 LINE  1 FOR REPORT 1
 LINE  2 FOR REPORT 1
 ...
1RPT.VICKY
 LINE  1 FOR REPORT 2
 LINE  2 FOR REPORT 2
 ...
1RPT.FRANK
 LINE  1 FOR REPORT 3
 LINE  2 FOR REPORT 3
 ...
1RPT.DAVID
 LINE  1 FOR REPORT 4
 LINE  2 FOR REPORT 4
 ...
```

Each report starts with 'RPT.reptname' in positions 2-13. In the output data set we only want to include records for reports with specific reptname values, and the reptname values we want can change from run to run. We also want to sort by the reptname values in ascending order. For this example, let's say we just want the SRIHARI and FRANK reports.

We use an IFTHEN WHEN=GROUP clause to propagate the reptname value to each record of the group. BEGIN indicates a group starts with 'RPT.' in positions 2-5. PUSH overlays the reptname value from the first record of the group (the 'RPT.reptname' record) at positions 31-38 (after the end of the record) in each record of the group including the first. After the IFTHEN GROUP clause is executed, the intermediate records look like this:

```
1RPT.SRIHARI                 SRIHARI
 LINE  1 FOR REPORT 1        SRIHARI
 LINE  2 FOR REPORT 1        SRIHARI
 ...                         SRIHARI
1RPT.VICKY                   VICKY
 LINE  1 FOR REPORT 2        VICKY
 LINE  2 FOR REPORT 2        VICKY
 ...                         VICKY
1RPT.FRANK                   FRANK
 LINE  1 FOR REPORT 3        FRANK
 LINE  2 FOR REPORT 3        FRANK
 ...                         FRANK
1RPT.DAVID                   DAVID
 LINE  1 FOR REPORT 4        DAVID
```

```
 LINE  2 FOR REPORT 4         DAVID
 ...                          DAVID
```

Note that the records of each group have the reptname value from the first record of that group in positions 31-38.

We use a SORT statement to sort ascending on the reptname in positions 31-38. We use the EQUALS option to ensure that records in the same group (that is, with the same reptname value) are kept in their original order. After the SORT statement is executed, the intermediate records look like this:

```
1RPT.DAVID                    DAVID
 LINE  1 FOR REPORT 4         DAVID
 LINE  2 FOR REPORT 4         DAVID
 ...                          DAVID
1RPT.FRANK                    FRANK
 LINE  1 FOR REPORT 3         FRANK
 LINE  2 FOR REPORT 3         FRANK
 ...                          FRANK
1RPT.SRIHARI                  SRIHARI
 LINE  1 FOR REPORT 1         SRIHARI
 LINE  2 FOR REPORT 1         SRIHARI
 ...                          SRIHARI
1RPT.VICKY                    VICKY
 LINE  1 FOR REPORT 2         VICKY
 LINE  2 FOR REPORT 2         VICKY
 ...                          VICKY
```

We use an OUTFIL statement to only INCLUDE the records with a a reptname of FRANK or SRIHARI in positions 31-38, and to remove the reptname from positions 31-38 so the included output records will be identical to the input records. After the OUTFIL statement is executed, the final output records look like this:

```
1RPT.FRANK
 LINE  1 FOR REPORT 3
 LINE  2 FOR REPORT 3
 ...
1RPT.SRIHARI
 LINE  1 FOR REPORT 1
 LINE  2 FOR REPORT 1
 ...
```

Example 13

```
  OPTION COPY,Y2PAST=1996
  INREC BUILD=(1,6,Y2W,TOJUL=Y4T,X,
     1,6,Y2W,WEEKDAY=CHAR3,X,
     9,7,Y4T,TOGREG=Y4T(/),X,
     9,7,Y4T,WEEKDAY=DIGIT1)
```

This example illustrates how to convert an mmddyy date to a ccyyddd date and a 3-character weekday string, and how to convert a ccyyddd date to a ccyy/mm/dd date and 1-digit weekday string.

The input records might be as follows:

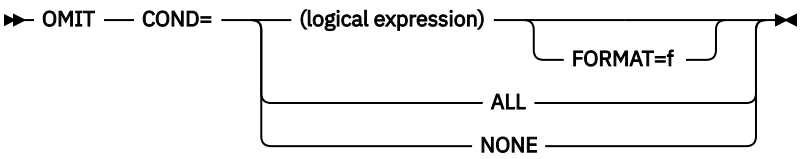
```
120409  1999014
051895  2003235
999999  0000000
013099  1992343
```

The output records would be as follows:

```
2009338 FRI 1999/01/14 5
2095138 WED 2003/08/23 7
9999999 999 0000/00/00 0
1999030 SAT 1992/12/08 3
```

The Y2PAST=1996 option sets the century window to 1996-2095. The century window is used to transform yy in the Y2W field to ccyy.

OMIT control statement

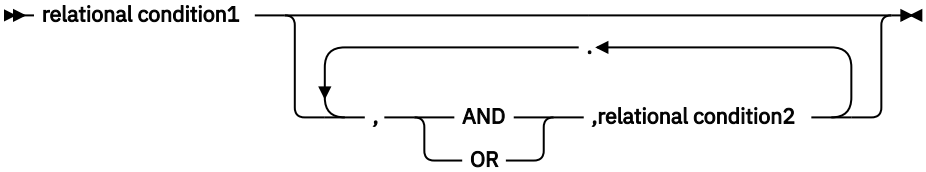


Use an OMIT statement if you do not want all of the input records to appear in the output data sets. The OMIT statement selects the records you do not want to include.

You can specify either an INCLUDE statement or an OMIT statement in the same DFSORT run, but not both.

The way in which DFSORT processes short INCLUDE/OMIT compare fields depends on the settings for VLSCMP/NOVLSCMP and VLSHRT/NOVLSHRT. A short field is one where the variable-length record is too short to contain the entire field, that is, the field extends beyond the record. For details about including or omitting short records, see the discussion of the VLSCMP and NOVLSCMP options in “OPTION control statement” on page 171.

A logical expression is one or more relational conditions logically combined, based on fields in the input record, and can be represented at a high level as follows:



If the logical expression is true for a given record, the record is omitted from the output data set.

Five types of relational conditions can be used as follows:

1.
2. **Comparisons:**

   Compare two compare fields or a compare field and a decimal, hexadecimal, character or current, future, or past date constant.

   For example, you can compare the first 6 bytes of each record with its last 6 bytes, and omit those records in which those fields are identical. Or you can compare a date field with today's date, yesterday's date, or tomorrow's date and omit those records accordingly.
3. **Substring Comparison Tests:**

   Search for a constant within a field value or a field value within a constant.

   For example, you can search the value in a 6-byte field for the character constant C'OK', and omit those records for which C'OK' is found somewhere in the field. Or you can search the character constant C'J69,L92,J82' for the value in a 3-byte field, and omit those records for which C'J69', C'L92', or C'J82' appears in the field.
4. **Bit Logic Tests:**

   Test the state (on or off) of selected bits in a binary field using a bit or hexadecimal mask or a bit constant.

   For example, you can omit those records which have bits 0 and 2 on in a 1-byte field. Or you can omit those records which have bits 3 and 12 on and bits 6 and 8 off in a 2-byte field.
5. **Date Comparisons:**

   Compare a two-digit year date field to a two-digit year date constant, the current two-digit year date or another two-digit year date field, using the century window in effect.

For example, you can omit only those records for which a Z'yymm' date field is between January 1996 and March 2005. Or you can omit only those records for which a P'dddyy' field is less than another P'dddyy' field.

6. **Numeric Tests:**

   Test a field for numerics or non-numerics in character, zoned decimal, or packed decimal format.

   For example, you can omit only those records in which a 5-byte field contains only '0'-'9' characters (that is, numerics). Or you can omit only those records in which a 9-byte field contains invalid ZD numeric data (that is, non-numerics). Or you can omit only those records in which a 12-byte field contains valid PD numeric data (that is, numerics).

7. **Alphanumeric Tests:**

   Test a field for alphanumerics or non-alphanumerics in character format. Various combinations of uppercase characters (A-Z), lowercase characters (a-z) and numeric characters (0-9) can be used.

   For example, you can omit only those records in which a 10-byte field contains only 'A'-'Z' characters (that is, uppercase characters) or '0'-'9' characters (that is, numeric characters). Or you can omit only those records in which a 20-byte field contains characters other than 'a'-'z' (that is, lowercase characters).

For complete details on the parameters of the OMIT control statement, see "INCLUDE control statement" on page 91.

The OMIT control statement differs from the OMIT parameter of the OUTFIL statement in the following ways:

- The OMIT statement applies to all input records; the OMIT parameter applies only to the OUTFIL input records for its OUTFIL group.
- FORMAT=f can be specified with the OMIT statement but not with the OMIT parameter. Thus, you can use FORMAT=f and p,m or p,m,f fields with the OMIT statement, but you must only use p,m,f fields with the OMIT parameter. For example:

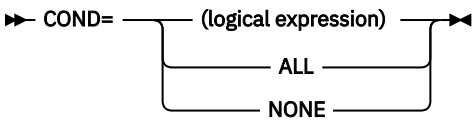
```
OMIT FORMAT=BI,
  COND=(5,4,LT,11,4,OR,21,4,EQ,31,4,OR,
     61,20,SS,EQ,C'FLY')

OUTFIL OMIT=(5,4,BI,LT,11,4,BI,OR,21,4,BI,EQ,31,4,BI,OR,
       61,20,SS,EQ,C'FLY')
```

- D2 format can be specified with the OMIT statement but not with the OMIT parameter.

See "OUTFIL control statements" on page 219 for more details on the OUTFIL OMIT parameter.

COND



logical expression
specifies one or more relational conditions logically combined, based on fields in the input record. If the logical expression is true for a given record, the record is omitted from the output data sets.

ALL
specifies that all of the input records are to be omitted from the output data sets.

NONE
specifies that none of the input records are to be omitted from the output data sets.

Default: NONE. See Appendix B, "Specification/override of DFSORT options," on page 801 for full override details.

Applicable Functions: See Appendix B, "Specification/override of DFSORT options," on page 801.

FORMAT



For details on this parameter, see “INCLUDE control statement” on page 91.

Omitting records from the output data set—example

Example

```
OMIT COND=(27,1,CH,EQ,C'D',&,
           (22,2,BI,SOME,X'C008',|,
            28,1,BI,EQ,B'.1....01'))
```

This statement omits records in which:

- Byte 27 contains D

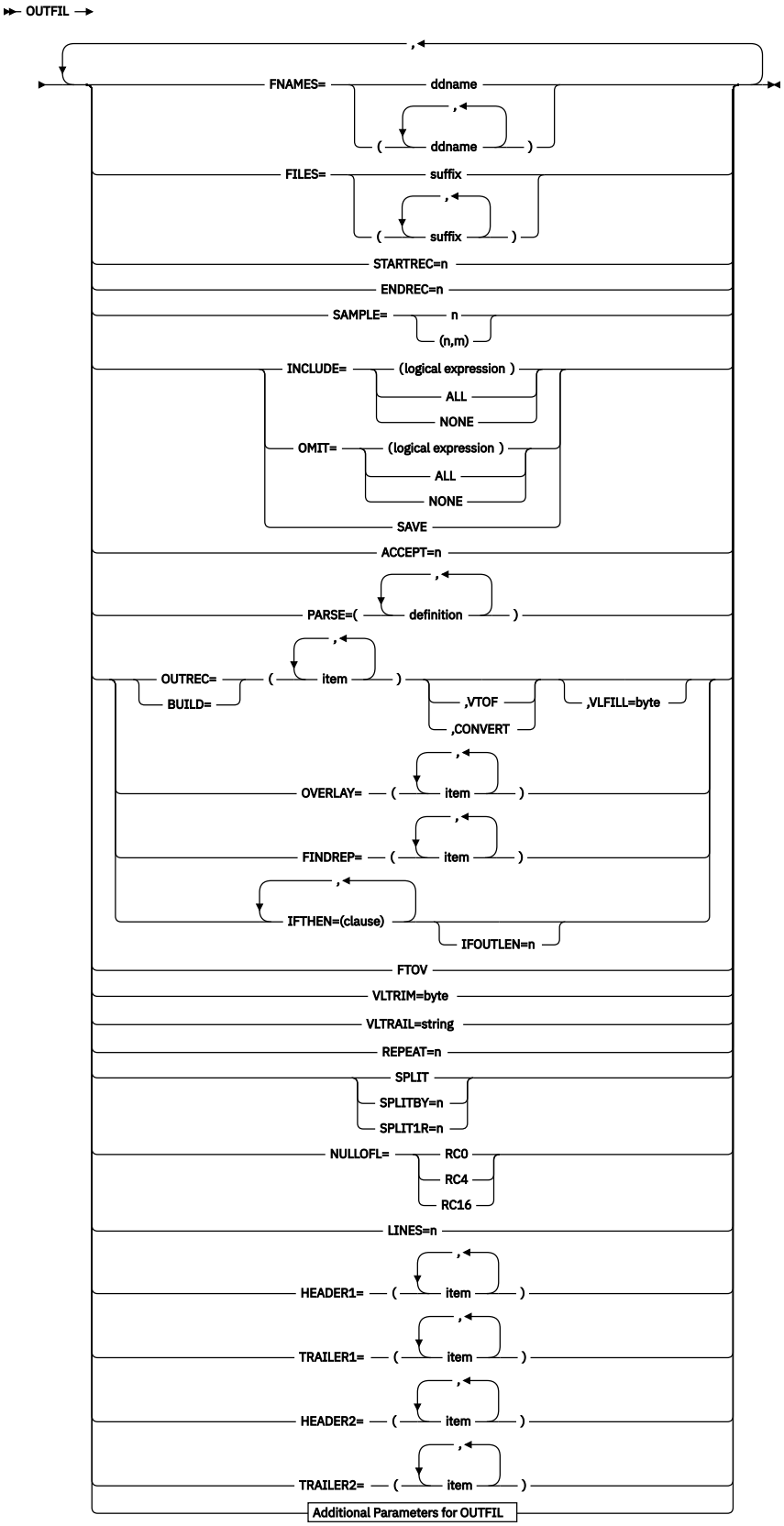
  AND

- Bytes 22 through 23 have some, but not all of bits 0, 1 and 12 on OR byte 28 is equal to the specified pattern of bit 1 on, bit 6 off and bit 7 on.

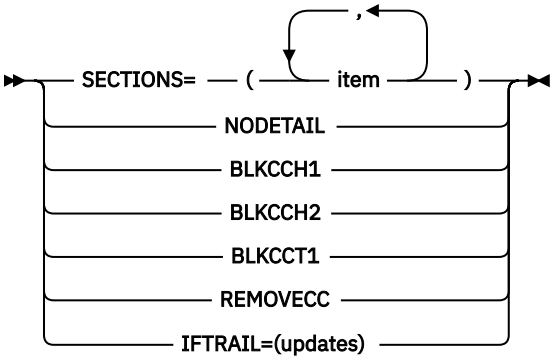
Note that the AND and OR operators can be written with the AND and OR signs, and that parentheses are used to change the order in which AND and OR are evaluated.

For additional examples of logical expressions, see “INCLUDE control statement” on page 91.

OUTFIL control statements



Additional Parameters for OUTFIL



OUTFIL control statements allow you to create one or more output data sets for a sort, copy, or merge application from a single pass over one or more input data sets. You can use multiple OUTFIL statements, with each statement specifying the OUTFIL processing to be performed for one or more output data sets. OUTFIL processing begins after all other processing ends (that is, after processing for exits, options, and other control statements).

OUTFILE can be used as an alias for OUTFIL.

OUTFIL statements support a wide variety of output data set tasks, including:

- Creation of multiple output data sets containing unedited or edited records from a single pass over one or more input data sets.
- Creation of multiple output data sets containing different ranges or subsets of records from a single pass over one or more input data sets. In addition, records that are not selected for any subset can be saved in a separate output data set.
- Conversion of variable-length record data sets to fixed-length record data sets.
- Conversion of fixed-length record data sets to variable-length record data sets.
- A wide variety of parsing, editing, and reformatting tasks including:
  - The use of fixed position/length fields or variable position/length fields. For fixed fields, you specify the starting position and length of the field directly. For variable fields, such as delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings (and many other types), you define rules that allow DFSORT to extract the relevant data into fixed parsed fields, and then use the parsed fields as you would use fixed fields.
  - Insertion of blanks, zeros, strings, current date, future date, past date, current time, sequence numbers, decimal constants, and the results of arithmetic expressions before, between, and after the input fields in the reformatted records.
  - Sophisticated conversion capabilities, such as find and replace, hexadecimal display, bit display, translation of EBCDIC letters from lowercase to uppercase or uppercase to lowercase, translation of characters from EBCDIC to ASCII and from ASCII to EBCDIC, translation of characters using the ALTSEQ translation table, conversion of numeric values from one format to another, left justify or left-squeeze (remove leading blanks or all blanks and shift left), and right-justify or right-squeeze (remove trailing blanks or all blanks and shift right).
  - Sophisticated editing capabilities, such as control of the way numeric fields are presented with respect to length, leading or suppressed zeros, thousands separators, decimal points, leading and trailing positive and negative signs, and so on.
  - Twenty-seven pre-defined editing masks are available for commonly used numeric editing patterns, encompassing many of the numeric notations used throughout the world. In addition, a virtually unlimited number of numeric editing patterns are available via user-defined editing masks.
  -
  - Transformation of SMF, TOD, and ETOD date and time values to more usable forms.

  - Conversion of input date fields of one type (CH, ZD, PD, 2-digit year, 4-digit year, Julian, Gregorian) to corresponding output date fields of another type or to a corresponding day of the week.
  - Various types of arithmetic operations for input date fields.
  - Selection of a character constant, hexadecimal constant, or input field from a lookup table for output, based on a character, hexadecimal, or bit string as input (that is, lookup and change).
- Creation of the reformatted records in one of the following ways:
  - By building the entire record one item at a time.
  - By only overlaying specific columns.
  - By performing find and replace operations.
  - By using sophisticated conditional logic or group operations to choose how different records are reformatted.
- Highly detailed three-level (report, page, and section) reports containing a variety of report elements you can specify (for example, current date, current time, edited or converted page numbers, character strings, and blank lines) or derive from the input records (for example, character fields; unedited, edited or converted numeric input fields; edited or converted record counts; and edited or converted totals, maximums, minimums, and averages for numeric input fields).
- Creation of multiple output records from each input record, with or without intervening blank output records.
- Repetition and sampling of data records.
- Splitting of data records in rotation among a set of output data sets.
- Updating counts and totals in an existing trailer (last) record based on the current data records.

The parameters of OUTFIL are grouped by primary purpose as follows:

-
- **FNAMES** and **FILES** specify the ddnames of the **OUTFIL data sets** to be created. Each OUTFIL data set to be created must be specifically identified using FNAMES or FILES in an OUTFIL statement. By contrast, the **SORTOUT data set** is created by default if a DD statement for it is present. The term "SORTOUT data set" denotes the single non-OUTFIL output data set, but in fact, the SORTOUT ddname can be used for an OUTFIL data set either explicitly or by default.

  If SORTOUT is identified as an OUTFIL ddname, either explicitly (for example, via FILES=OUT) or by default (OUTFIL without FILES or FNAMES), the data set associated with the SORTOUT ddname will be processed as an OUTFIL data set rather than as the SORTOUT data set.

  OUTFIL data sets have characteristics and requirements similar to those for the SORTOUT data set, but there are differences in the way each is processed. The major differences are that an E39 exit routine is not entered for OUTFIL data sets, and that OUTFIL processing does not permit the use of the LRECL value to pad fixed-format OUTFIL records. (DFSORT will automatically determine and set an appropriate RECFM, LRECL, and BLKSIZE for each OUTFIL data set for which these attributes are not specified or available.)

  For a single DFSORT application, OUTFIL data sets can be intermixed with respect to VSAM and non-VSAM, tape and disk, and so on. All of the data sets specified for a particular OUTFIL statement are processed in a similar way and thus are referred to as an **OUTFIL group**. (That is, you group OUTFIL data sets that use the same operands by specifying them on a single OUTFIL statement.) For example, the first OUTFIL statement might have an INCLUDE operand that applies to an OUTFIL group of one non-VSAM data set on disk and another on tape; a second OUTFIL statement might have OMIT and OUTREC operands that apply to an OUTFIL group of one non-VSAM data set on disk and two VSAM data sets.

  Records are processed for OUTFIL as they are for SORTOUT, after all other DFSORT processing is complete. Conceptually, you can think of an **OUTFIL input record** as being intercepted at the point between being passed from an E35 exit and written to SORTOUT, although neither an E35 exit nor

SORTOUT need actually be specified with OUTFIL processing. With that in mind, see Figure 2 on page 10 for details on the processing that occurs prior to processing the OUTFIL input record. In particular:

- Records deleted by an E15 or E35 exit, an INCLUDE, OMIT or SUM statement, or the SKIPREC or STOPAFT parameter are not available for OUTFIL processing
- If records are reformatted by an E15 exit, an INREC or OUTREC statement, or an E35 exit, the resulting reformatted record is the OUTFIL input record to which OUTFIL fields must refer.

- **STARTREC** starts processing for an OUTFIL group at a specific OUTFIL input record. **ENDREC** ends processing for an OUTFIL group at a specific OUTFIL input record. **SAMPLE** selects a sample of OUTFIL input records for an OUTFIL group using a specific interval and number of records in that interval. Separately or together, STARTREC, ENDREC, and SAMPLE select a range of records to which subsequent OUTFIL processing will apply.
- **INCLUDE, OMIT**, and **SAVE** select the records to be included in the data sets of an OUTFIL group. INCLUDE and OMIT operate against the specified fields of each OUTFIL input record to select the output records for their OUTFIL group (all records are selected by default). SAVE selects the records that are not selected for any other OUTFIL group.

  Whereas the INCLUDE and OMIT statements apply to all input records, the INCLUDE and OMIT parameters apply only to the OUTFIL input records for their OUTFIL group. The INCLUDE and OMIT parameters have all of the logical expression capabilities of the INCLUDE and OMIT statements.
- **ACCEPT** limits the number of OUTFIL input records accepted for processing for an OUTFIL group. A record is accepted if it is not deleted by STARTREC, ENDREC, SAMPLE, INCLUDE or OMIT processing. ACCEPT limits the number of records to which subsequent OUTFIL processing will apply.
-
- **PARSE, OUTREC, BUILD, OVERLAY, FINDREP, or IFTHEN** reformat the output records for an OUTFIL group. These parameters allow you to rearrange, edit, replace, remove and change fixed position/length fields or variable position/length fields from the OUTFIL input records and to insert blanks, zeros, strings, current date, future date, past date, current time, sequence numbers, decimal constants, and the results of arithmetic expressions.

  OVERLAY allows you to change specific existing columns without affecting the entire record.

  FINDREP allows you to do various find and replace operations.

  IFTHEN clauses allow you to reformat different records in different ways according to the criteria you specify. IFOUTLEN can be used with IFTHEN clauses to set the output LRECL.

  OUTREC or BUILD gives you complete control over the items in your reformatted records and the order in which they appear, and also allows you to produce multiple reformatted output records from each input record, with or without intervening blank output records.

  VTOF or **CONVERT** can be used with OUTREC or BUILD to convert variable-length input records to fixed-length output records.

  VLFILL can be used to allow processing of variable-length input records which are too short to contain all specified OUTREC or BUILD fields.

  Whereas the FIELDS, BUILD, OVERLAY, FINDREP, and IFTHEN parameters of the OUTREC statement apply to all input records, the OUTREC, BUILD, OVERLAY, FINDREP, and IFTHEN parameters of the OUTFIL statement apply only to the OUTFIL input records for its OUTFIL group. In addition, the OUTREC and BUILD parameters of the OUTFIL statement support the forward slash (/) separator for creating blank records and new records, whereas the FIELDS and BUILD parameters of the OUTREC statement do not.

-

- **FTOV** can be used to convert fixed-length input records to variable-length output records. FTOV can be used with or without OUTREC , BUILD, OVERLAY, FINDREP, or IFTHEN.
- **VLTRIM** can be used to remove the trailing bytes with a specified value, such as blanks, binary zeros or asterisks, from variable-length records. VLTRIM can be used with or without FTOV.
- **VLTRAIL** can be used to insert a character string or hexadecimal string at the end of variable-length records. VLTRAIL can be used with or without FTOV or VLTRIM.
- **REPEAT** can be used to repeat each output record a specified number of times.
- **SPLIT, SPLITBY, or SPLIT1R** splits the output records in rotation among the data sets of an OUTFIL group.

  With SPLIT, the first output record is written to the first OUTFIL data set in the group, the second output record is written to the second data set, and so on. When each OUTFIL data set has one record, the rotation starts again with the first OUTFIL data set.

  SPLITBY can be used to rotate by a specified number of records rather than by one record, for example, records 1-10 to the first OUTFIL data set, records 11-20 to the second OUTFIL data set, and so on. When each OUTFIL data set has n records, the rotation starts again with the first OUTFIL data set.

  SPLIT1R can be used to rotate once by a specified number of records. SPLIT1R continues with the last OUTFIL data set instead of rotating back to the first OUTFIL data set. When each OUTFIL data set has n records, the last OUTFIL data set will have the remainder of the records even if that is more than n.
-
- **LINES, HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS**, and **NODETAIL** indicate that a report is to be produced for an OUTFIL group, and specify the details of the **report records** to be produced for the report. Reports can contain report records for a report header (first page), report trailer (last page), page header and page trailer (at the top and bottom of each page, respectively), and section headers and trailers (before and after each section, respectively).

  Data records for the report result from the inclusion of OUTFIL input records. All of the capabilities of the OUTREC, BUILD, OVERLAY, FINDREP, or IFTHEN parameters are available to create reformatted data records from the OUTFIL input records. Each set of sequential OUTFIL input records, with the same binary value for a specified field, results in a corresponding set of data records that is treated as a section in the report.

  The length for the data records must be equal to or greater than the maximum report record length. OUTFIL data sets used for reports must have or will be given ANSI control character format ('A' as in, for example, RECFM=FBA or RECFM=VBA), and must allow an extra byte in the LRECL for the carriage control character that DFSORT will add to each report and data record. DFSORT uses these carriage control characters to control page ejects and the placement of the lines in your report according to your specifications. DFSORT uses appropriate carriage controls (for example, C'-' for triple space) in header and trailer records when possible, to reduce the number of report records written. DFSORT always uses the single space carriage control (C' ') in data records. Although these carriage control characters may not be shown when you view an OUTFIL data set (depending upon how you view it), they will be used if you print the report.
- **BLKCCH1, BLKCCT1 or BLKCCH2** can be used avoid forcing a page eject for the report header, report trailer or first page header, respectively, by using a blank instead of a '1' for the ANSI control carriage character of its first line.
- **REMOVECC** can be used to remove the ANSI control characters from a report. In this case, an 'A' is not added to or required for the RECFM and an extra byte is not added to or required for the LRECL.
- **IFTRAIL** can be used to update count and total values in an existing trailer (last) record based on the current data records.
- Figure 7 on page 224 illustrates the order in which OUTFIL records and parameters are processed.

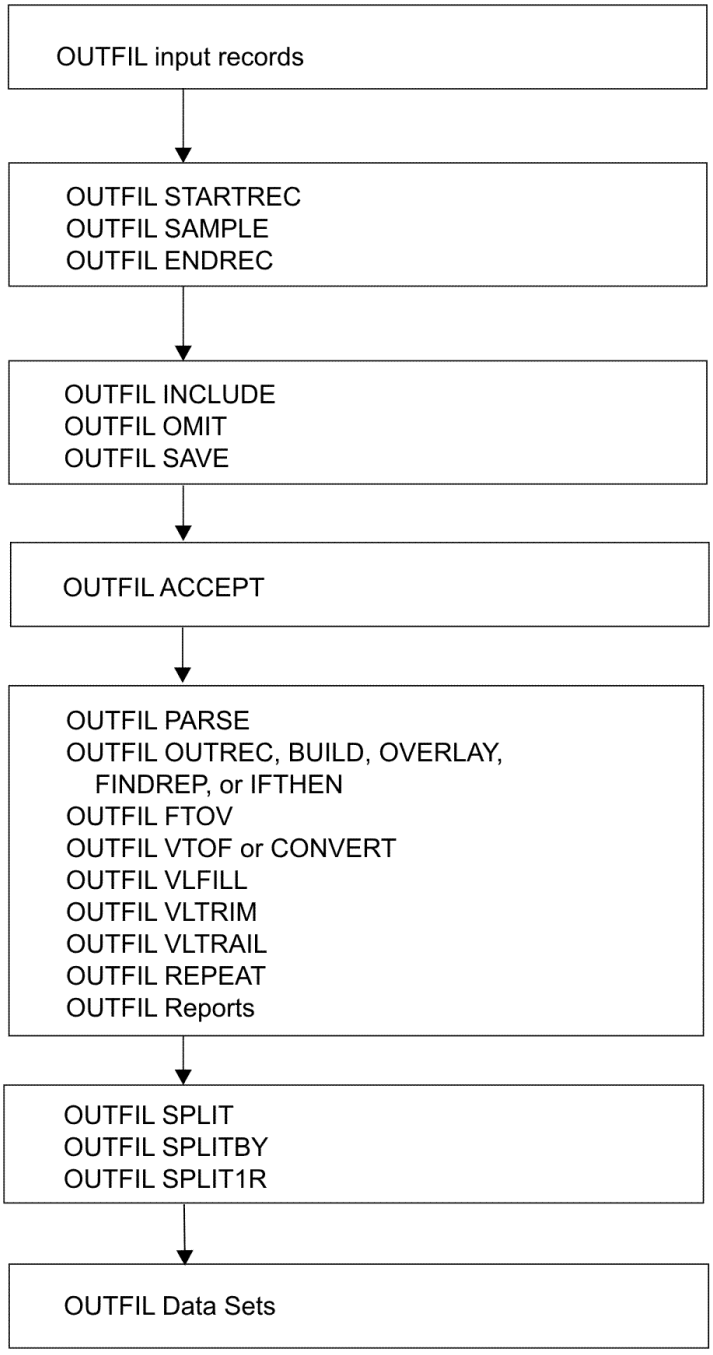
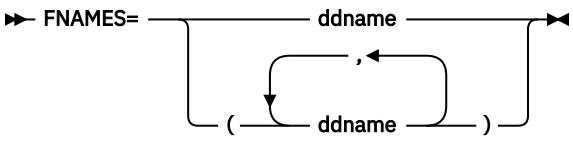


Figure 7. OUTFIL Processing Order

Note:

1. DFSORT accepts but does not process the following OUTFIL operands: BLKSIZE=value, BUFLIM=value, BUFOFF=value, CARDS=value, CLOSE=value, DISK, ESDS, EXIT, FREEOUT, KSDS, LRECL=value, NOTPMK, OPEN=value, OUTPUT, PAGES=value, PRINT, PUNCH, REUSE, RRDS, SPAN, SYSLST, TAPE, and TOL.
2. Sample syntax is shown throughout this section. Complete OUTFIL statement examples are shown and explained under “OUTFIL features—examples” on page 362.

FNAMES



Specifies ddnames associated with the OUTFIL data sets for this OUTFIL statement. The ddnames specified using the FNAMES and FILES parameters constitute the output data sets for this OUTFIL group to which all of the other parameters for this OUTFIL statement apply.

If FNAMES specifies the ddname in effect for the SORTOUT data set (that is, whichever is in effect among ddname from SORTOUT=ddname, ccccOUT from SORTDD=cccc, or SORTOUT) DFSORT will treat the data set associated with that ddname as an OUTFIL data set rather than as the SORTOUT data set.

ddname
specifies a 1- through 8-character ddname. A DD statement must be present for this ddname.

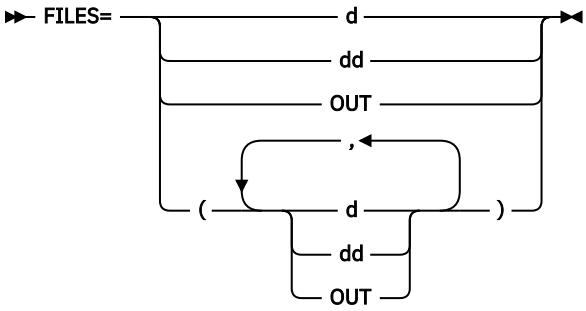
Sample Syntax:

```
OUTFIL FNAMES=(OUT1,OUT2,PRINTER,TAPE)
OUTFIL FNAMES=BACKUP
```

Default for FNAMES: If neither FNAMES nor FILES is specified for an OUTFIL statement, the default ddname is:

- ddname if SORTOUT=ddname is in effect
- ccccOUT if SORTDD=cccc is in effect and SORTOUT=ddname is not in effect
- SORTOUT if neither SORTOUT=ddname nor SORTDD=cccc is in effect.

FILES



Specifies suffixes for ddnames to be associated with the OUTFIL data sets for this OUTFIL statement. The ddnames specified using the FNAMES and FILES parameters constitute the output data sets for this OUTFIL group to which all of the other parameters for this OUTFIL statement apply.

If FILES specifies the ddname in effect for the SORTOUT data set (that is, whichever is in effect among ddname from SORTOUT=name, ccccOUT from SORTDD=cccc, or SORTOUT), DFSORT will treat the data set associated with that ddname as an OUTFIL data set rather than as the SORTOUT data set.

d
specifies the 1-character suffix to be used to form the ddname SORTOFd or ccccOFd if SORTDD=cccc is in effect. A DD statement must be present for this ddname.

dd
specifies the 2-character suffix to be used to form the ddname SORTOFdd or ccccOFdd if SORTDD=cccc is in effect. A DD statement must be present for this ddname.

OUT
specifies the suffix OUT is to be used to form the ddname SORTOUT or ccccOUT if SORTDD=cccc is in effect. A DD statement must be present for this ddname.

Sample Syntax:

```
OUTFIL FILES=(1,2,PR,TP)
OUTFIL FILES=OUT
```

Default for FILES: If neither FNAMES nor FILES is specified for an OUTFIL statement, the default ddname is:

- ddname if SORTOUT=ddname is in effect
- ccccOUT if SORTDD=cccc is in effect and SORTOUT=ddname is not in effect
- SORTOUT if neither SORTOUT=ddname nor SORTDD=cccc is in effect.

STARTREC



Specifies the OUTFIL input record at which OUTFIL processing is to start for this OUTFIL group. OUTFIL input records before this starting record are not included in the data sets for this OUTFIL group.

n
specifies the relative record number. The value for n starts at 1 (the first record) and is limited to 28 digits (15 significant digits).

Sample Syntax:

```
OUTFIL FNAMES=SKIP20,STARTREC=21
```

Default for STARTREC: 1.

ENDREC



Specifies the OUTFIL input record at which OUTFIL processing is to end for this OUTFIL group. OUTFIL input records after this ending record are not included in the data sets for this OUTFIL group.

The ENDREC value must be equal to or greater than the STARTREC value if both are specified on the same OUTFIL statement.

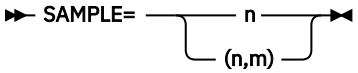
n
specifies the relative record number. The value for n starts at 1 (the first record) and is limited to 28 digits (15 significant digits).

Sample Syntax:

```
OUTFIL FNAMES=TOP10,ENDREC=10
OUTFIL FNAMES=FRONT,ENDREC=500
OUTFIL FNAMES=MIDDLE,STARTREC=501,ENDREC=2205
OUTFIL FNAMES=BACK,STARTREC=2206
```

Default for ENDREC: The last OUTFIL input record.

SAMPLE



Specifies a sample of OUTFIL input records to be processed for this OUTFIL group. The sample consists of the first m records in every nth interval.

n
specifies the interval size. The value for n starts at 2 (sample every other record) and is limited to 28 digits (15 significant digits).

m
specifies the number of records to be processed in each interval. The value for m starts at 1 (process the first record in each interval) and is limited to 28 digits (15 significant digits). If m is not specified, 1 is used for m. If m is specified, it must be less than n.

Sample Syntax:

```
* PROCESS RECORDS 1, 6, 11, ...
  OUTFIL FNAMES=OUT1,SAMPLE=5

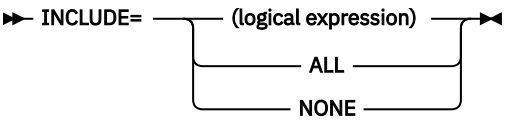
* PROCESS RECORDS 1, 2, 1001, 1002, 2001, 2002
  OUTFIL FNAMES=OUT2,SAMPLE=(1000,2),ENDREC=2500

* PROCESS RECORDS 23, 48, 73
  OUTFIL FNAMES=OUT3,STARTREC=23,ENDREC=75,SAMPLE=25

* PROCESS RECORDS 1001, 1002, 1003, 1101, 1102, 1103, ...
  OUTFIL FNAMES=OUT4,STARTREC=1001,SAMPLE=(100,3)
```

Default for SAMPLE: None; must be specified.

INCLUDE



Selects the records to be included in the data sets for this OUTFIL group.

The INCLUDE parameter operates in the same way as the INCLUDE statement, except that:

- The INCLUDE statement applies to all input records; the INCLUDE parameter applies only to the OUTFIL input records for its OUTFIL group.
- FORMAT=f can be specified with the INCLUDE statement, but not with the INCLUDE parameter. Thus, you can use FORMAT=f and p,m or p,m,f fields with the INCLUDE statement, but you must only use p,m,f fields with the INCLUDE parameter. For example:

```
INCLUDE FORMAT=BI,
    COND=(5,4,LT,11,4,OR,21,4,EQ,31,4,OR,
       61,20,SS,EQ,C'FLY')

OUTFIL INCLUDE=(5,4,BI,LT,11,4,BI,OR,21,4,BI,EQ,31,4,BI,OR,
       61,20,SS,EQ,C'FLY')
```

- D2 format can be specified with the INCLUDE statement, but not with the INCLUDE parameter.

See “INCLUDE control statement” on page 91 for complete details.

logical expression
specifies one or more relational conditions logically combined based on fields in the OUTFIL input record. If the logical expression is true for a given record, the record is included in the data sets for this OUTFIL group.

ALL
specifies that all of the OUTFIL input records are to be included in the data sets for this OUTFIL group.

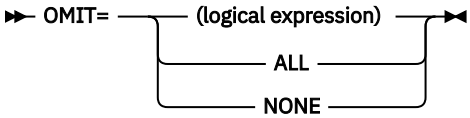
NONE
specifies that none of the OUTFIL input records are to be included in the data sets for this OUTFIL group.

Sample Syntax:

```
OUTFIL FNAMES=J69,INCLUDE=(5,3,CH,EQ,C'J69')
OUTFIL FNAMES=J82,INCLUDE=(5,3,CH,EQ,C'J82')
```

Default for INCLUDE: ALL.

OMIT



Selects the records to be omitted from the data sets for this OUTFIL group.

The OMIT parameter operates in the same way as the OMIT statement, except that:

- The OMIT statement applies to all input records; the OMIT parameter applies only to the OUTFIL input records for its OUTFIL group.
- FORMAT=f can be specified with the OMIT statement, but not with the OMIT parameter. Thus, you can use FORMAT=f and p,m or p,m,f fields with the OMIT statement, but you must only use p,m,f fields with the OMIT parameter. For example:

```
OMIT FORMAT=BI,
  COND=(5,4,LT,11,4,OR,21,4,EQ,31,4,OR,
            61,20,SS,EQ,C'FLY')

OUTFIL OMIT=(5,4,BI,LT,11,4,BI,OR,21,4,BI,EQ,31,4,BI,OR,
       61,20,SS,EQ,C'FLY')
```

- The D2 format can be specified with the OMIT statement, but not with the OMIT parameter.

See “OMIT control statement” on page 168 and “INCLUDE control statement” on page 91 for complete details.

logical expression
specifies one or more relational conditions logically combined based on fields in the OUTFIL input record. If the logical expression is true for a given record, the record is omitted from the data sets for this OUTFIL group.

ALL
specifies that all of the OUTFIL input records are to be omitted from the data sets for this OUTFIL group.

NONE
specifies that none of the OUTFIL input records are to be omitted from the data sets for this OUTFIL group.

Sample Syntax:

```
OUTFIL FILES=01,OMIT=NONE
OUTFIL OMIT=(5,1,BI,EQ,B'110.....')
OUTFIL FNAMES=(OUT1,OUT2),
  OMIT=(7,2,CH,EQ,C'32',OR,18,3,CH,EQ,C'XYZ')
```

Default for OMIT: NONE.

SAVE



Specifies that OUTFIL input records not included by STARTREC, ENDREC, ACCEPT, SAMPLE, INCLUDE or OMIT for any other OUTFIL group are to be included in the data sets for this OUTFIL group. SAVE operates in a global fashion over all of the other OUTFIL statements for which SAVE is not specified,

enabling you to keep any OUTFIL input records that would not be kept otherwise. SAVE will include the same records for each group for which it is specified.

Sample Syntax:

```
OUTFIL INCLUDE=(8,6,CH,EQ,C'ACCTNG'),FNAMES=GP1
OUTFIL INCLUDE=(8,6,CH,EQ,C'DVPMNT'),FNAMES=GP2
OUTFIL SAVE,FNAMES=NOT1OR2
```

Default for SAVE: None; must be specified.

ACCEPT



Specifies the number of OUTFIL input records to be accepted for processing for this OUTFIL group. A record is accepted if it is not deleted by STARTREC, ENDREC, SAMPLE, INCLUDE or OMIT processing for the group. After n OUTFIL input records have been accepted, additional OUTFIL input records are not included in the data sets for this OUTFIL group.

n
specifies the number of records to be accepted. The value for n starts at 1 and is limited to 28 digits (15 significant digits).

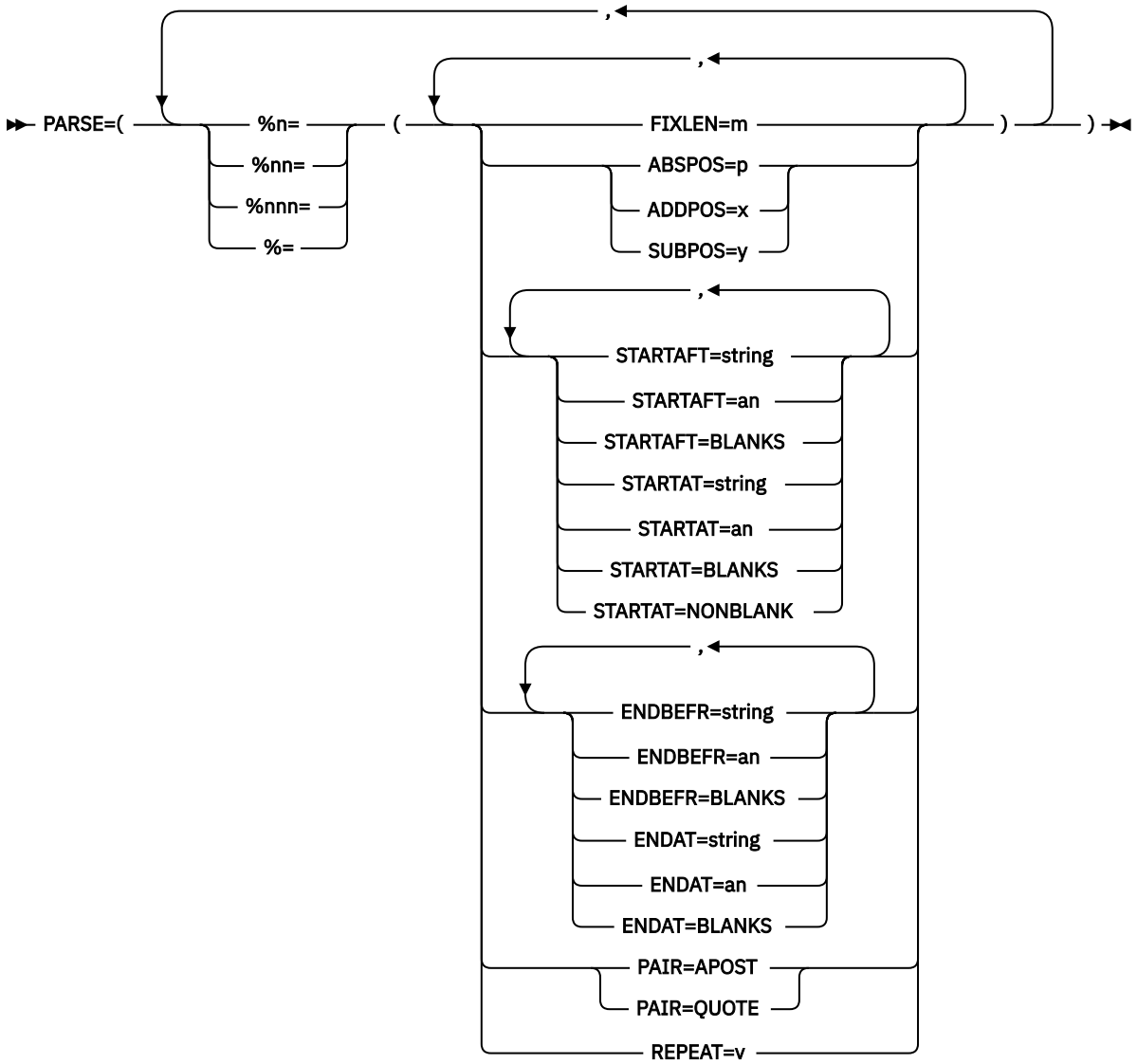
ACCEPT=n operates in a similar way to the ENDREC=n option. However, whereas ENDREC=n stops processing OUTFIL input records for a group at relative record n, ACCEPT=n stops processing OUTFIL input records for a group after n records have been "accepted". If both ACCEPT=n and ENDREC=n are specified, processing will stop when the first criteria is met.

Sample Syntax:

```
OUTFIL FNAMES=OUT1,INCLUDE=(11,3,CH,EQ,C'D51'),ACCEPT=3
OUTFIL FNAMES=OUT2,STARTREC=5,SAMPLE=1000,ACCEPT=12
```

Default for ACCEPT: None; must be specified.

PARSE



This operand allows you to extract variable position/length fields into fixed parsed fields. Parsed fields (%n, %nn or %nnn) can be used where fixed position/length fields (p,m) can be used in the BUILD (or OUTREC) or OVERLAY operands as described later in this section.

Note: Although you can use %n (%0-%9), %nn (%00-%99) or %nnn (%000-%999) for a parsed field, for convenience in this book %nn will be used in general when referring to a parsed field. %n, %0n or %00n can be used interchangeably for parsed field n (for example, %1, %01 or %001 for parsed field 1). %nn or %0nn can be used interchangeably for parsed field nn (for example, %12 or %012 for parsed field 12).

PARSE can be used for many different types of variable fields including delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings, and so on. You can assign up to 1000 parsed fields (%0-%999) to the variable fields you want to extract.

Note that if all of the fields in your records have fixed positions and lengths, you don't need to use PARSE. But if any of the fields in your records have variable positions or lengths, you can use PARSE to treat them as fixed parsed fields in BUILD or OVERLAY. You can mix p,m fields (fixed fields) and %nn fields (parsed fields) in BUILD and OVERLAY.

For each %nn parsed field, you define where the data to be extracted starts and ends, and the length (m) of the fixed field to contain the extracted data. For example, if your input records contained CSV data as follows:

```
AA,BBBB,C,DDDDD
EEEEE,,F,GG
HHH,IIIII,JJ,K
```

and you wanted to reformat the data into fixed positions like this:

```
AA        BBBB      C         DDDDD
EEEEE               F         GG
HHH       IIIII     JJ        K
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%01=(ENDBEFR=C',',FIXLEN=5),
                %02=(ENDBEFR=C',',FIXLEN=5),
                %03=(ENDBEFR=C',',FIXLEN=5),
                %04=(FIXLEN=5)),
         BUILD=(1:%01,11:%02,21:%03,31:%04)
```

The PARSE operand:

- assigns %01 to the first parsed field, and defines it as starting at position 1 and ending before the next (first) comma with a fixed length of 5 bytes
- assigns %02 to the second parsed field, and defines it as starting after the (first) comma and ending before the next (second) comma with a fixed length of 5 bytes
- assigns %03 to the third parsed field, and defines it as starting after the (second) comma and ending before the next (third) comma with a fixed length of 5 bytes
- assigns %04 to the fourth parsed field, and defines it as starting after the (third) comma and ending after 5 bytes with a fixed length of 5 bytes

You can start extracting data:

- at a specific position (ABSPOS=p), a relative position (ADDPOS=x or SUBPOS=y)
- after the start of one or more specific strings (STARTAFT=string), alphanumeric character sets (STARTAFT=an) or blanks (STARTAFT=BLANKS)
- at the start of one or more specific strings (STARTAT=string), alphanumeric character sets (STARTAT=an) or blanks (STARTAT=BLANKS) or a nonblank (STARTAT=NONBLANK).

You can end extracting data:

- before the start of one or more specific strings (ENDBEFR=string), alphanumeric character sets (ENDBEFR=an) or blanks (ENDBEFR=BLANKS),
- at the end of one or more specific strings (ENDAT=string), alphanumeric character sets (ENDAT=an) or blanks (ENDAT=BLANKS)
- after a specified number of bytes (FIXLEN=m without ENDBEFR or ENDAT).

For STARTAFT=an, STARTAT=an, ENDBEFR=an and ENDAT=an, you can specify one of the following for an to indicate the alphanumeric character set you want to use:

- UC: Uppercase characters (A-Z)
- LC: Lowercase characters (a-z)
- MC: Mixed case characters (A-Z, a-z)
- UN: Uppercase and numeric characters (A-Z, 0-9)
- LN: Lowercase and numeric characters (a-z, 0-9)
- MN: Mixed case and numeric characters (A-Z, a-z, 0-9)
- NUM: Numeric characters (0-9)

Multiple STARTAFT/STARTAT strings, UC, LC, MC, UN, LN, MN, NUM, BLANKS or NONBLANK can be used for a %nn parsed field to search for more than one string, set of characters, blanks or a nonblank. The first search satisfied is used.

Multiple ENDBEFR/ENDAT strings, UC, LC, MC, UN, LN, MN, NUM, or BLANKS can be used for a %nn parsed field to search for more than one string, set of characters or blanks. The first search satisfied is used.

For each record, each %nn and % parsed field you define is processed according to the suboperands you define, in the following order:

- ABSPOS or ADDPOS or SUBPOS
- STARTAFT, STARTAT and PAIR
- ENDBEFR, ENDAT and PAIR
- FIXLEN

As each %nn parsed field is processed, data is extracted according to the suboperands you specify.

In addition, as each %nn or % parsed field is processed, the Start Pointer for the next parsed field advances through the record according to the suboperands you specify. By default, for the first parsed field, the Start Pointer is set to position 1 for fixed-length records or to position 5 for variable-length records, and the Start Pointer for each subsequent %nn parsed field uses the Start Pointer as set by the previous %nn parsed field. For any parsed field, the Start Pointer advances as follows depending on the suboperands specified, and may advance more than once (for example, if STARTAFT and ENDBEFR are both specified, or if STARTAT and FIXLEN are both specified):

- if ABSPOS=p is specified, the Start Pointer is set to position p.
- if ADDPOS=x is specified, the Start Pointer is incremented by x.
- if SUBPOS=y is specified, the Start Pointer is decremented by y.
- if STARTAFT=string or STARTAT=string is specified, the search for string starts at the Start Pointer. If string is found, the Start Pointer is set to the byte after the end of the string. If string is not found, the current parsed field and any subsequent parsed fields contain all blanks.
- if STARTAFT=an or STARTAT=an is specified, the search for the character in the specified alphanumeric set starts at the Start Pointer. If a character in the set is found, the Start Pointer is set to the byte after the character. If a character in the set is not found, the current parsed field and any subsequent parsed fields contain all blanks.
- if STARTAFT=BLANKS or STARTAT=BLANKS is specified, the search for blanks starts at the Start Pointer. If a blank is found, the Start Pointer is set to the next nonblank. If a blank is not found, the current parsed field and any subsequent parsed fields contain all blanks.
- if STARTAT=NONBLANK is specified, the search for a nonblank starts at the Start Pointer. If a nonblank is found, the Start Pointer is set to the nonblank. If a nonblank is not found, the current parsed field and any subsequent parsed fields contain all blanks.
- if ENDBEFR=string or ENDAT=string is specified, the search for string starts at the Start Pointer. If string is found, the Start Pointer is set to the byte after the end of the string. If string is not found, data for this parsed field is extracted up to the end of the record. Subsequent parsed fields contain all blanks.
- if ENDBEFR=an or ENDAT=an is specified, the search for the character in the specified alphanumeric set starts at the Start Pointer. If a character in the set is found, the Start Pointer is set to the byte after the character. If a character in the set is not found, data for this parsed field is extracted up to the end of the record. Subsequent parsed fields contain all blanks.
- if ENDBEFR=BLANKS or ENDAT=BLANKS is specified, the search for blanks starts at the Start Pointer. If a blank is found, the Start Pointer is set to the next nonblank. If a blank is not found, data for this parsed field is extracted up to the end of the record. Subsequent parsed fields contain all blanks.
- if ENDBEFR and ENDAT are not specified and FIXLEN=m is specified, the Start Pointer is set m bytes past the start of the extracted data.
- if the Start Pointer advances past the end of the record, PARSE processing stops.

For example, if you had these records as input:

```
11*+23  NEW  MAXCC=00   (Monica) 18
1*-6 OLD  MAXCC=04 (Joey)           -2735
232*+86342 SHR MAXCC=04  (Rachel)  0
```

and you wanted this output:

```
Monica    MAXCC=00  +000023  011     18
Joey      MAXCC=04  -000006  001  -2735
Rachel    MAXCC=04  +086342  232      0
```

you could use this OUTFIL statement:

```
  OUTFIL FNAMES=OUT1,
    PARSE=(%00=(ENDBEFR=C'*',FIXLEN=3),
      %01=(ENDBEFR=BLANKS,FIXLEN=6),
      %02=(STARTAT=C'MAX',FIXLEN=8),
      %03=(STARTAFT=C'(',ENDBEFR=C')',FIXLEN=6),
      %04=(STARTAFT=BLANKS,FIXLEN=5)),
    BUILD=(%03,11:%02,21:%01,SFF,M26,LENGTH=7,
           30:%00,UFF,M11,LENGTH=3,35:%04,JFY=(SHIFT=RIGHT))
```

For this example, the Start Pointer advances as follows for the first record:

- For %00: Set Start Pointer to position 1. Start searching for '*' end string at position 1. Extract '11' (per ENDBEFR=C'*'). Set Start Pointer after '*' (at '+').
- For %01: Start searching for end blank at the Start Pointer (at '+'). Extract '+23' (per ENDBEFR=BLANKS). Set Start Pointer after the blanks (at 'N').
- For %02: Start searching for 'MAX' start string at the Start Pointer (at 'N'). Extract 'MAXCC=00' (per FIXLEN=8). Increment Start Pointer by m (8) bytes from the start of the extracted string (at blank after '00').
- For %03: Start searching for '(' start string at Start Pointer (at blank after '00'). Set Start Pointer after '(' (at 'M'). Start searching for ')' end string at Start Pointer (at 'M'). Extract 'Monica' (per ENDBEFR=C')'). Set Start Pointer after ')' (at blank after ')').
- For %04: Start searching for start blank at the Start Pointer (at blank after ')'). Extract '18 ' (per FIXLEN=5).

If the Start Pointer advances past the end of the record, PARSE processing stops. For example, if you had these input records:

```
First=George Last=Washington
First=John Middle=Quincy Last=Adams
```

and you used this OUTFIL statement:

```
  OUTFIL FNAMES=OUT1,
     PARSE=(%00=(STARTAFT=C'First=',ENDBEFR=C' ',FIXLEN=12),
            %01=(STARTAFT=C'Middle=',ENDBEFR=C' ',FIXLEN=12),
            %02=(STARTAFT=C'Last=',ENDBEFR=C' ',FIXLEN=12)),
     BUILD=(%00,%01,%02)
```

the %01 parsed field (middle name) is not found in the first record, so PARSE processing stops and the %02 parsed field is set to blanks. You can handle this kind of possible missing field situation by using IFTHEN PARSE instead of PARSE. For example:

```
  OUTFIL FNAMES=OUT1,
     IFTHEN=(WHEN=INIT,
       PARSE=(%00=(STARTAFT=C'First=',ENDBEFR=C' ',FIXLEN=12))),
     IFTHEN=(WHEN=INIT,
       PARSE=(%01=(STARTAFT=C'Middle=',ENDBEFR=C' ',FIXLEN=12))),
     IFTHEN=(WHEN=INIT,
       PARSE=(%02=(STARTAFT=C'Last=',ENDBEFR=C' ',FIXLEN=12)),
     BUILD=(%00,%01,%02))
```

By default, the Start Pointer is set to 1 (F) or 5 (V) for each IFTHEN PARSE, so the missing middle name in the first record does not prevent the last name from being extracted.

See the discussion of "IFTHEN" for more information on IFTHEN PARSE.

You must define each %nn field with PARSE before you use it in BUILD or OVERLAY, so generally you would want to specify PARSE before BUILD or OVERLAY. If you define a %nn field with PARSE, but do not use it in BUILD or OVERLAY, data will not be extracted for that parsed field.

Each %nn parsed field (%0-%999) can only be defined once per run with PARSE, but can be used as many times as needed in BUILD or OVERLAY.

The %nn parsed fields defined for a particular source (INREC, OUTREC or OUTFIL) can only be used in the BUILD or OVERLAY operands for that source. Generally, you will only use PARSE for one source (for example, INREC) per run. But you could define a separate set of %nn parsed fields for each source, if needed. For example, you could define %21 and %22 for the INREC statement, %101, %102 and %103 for the OUTREC statement, %40 and %45 for OUTFIL statement 1, and %30, %31 and %32 for OUTFIL statement 2. You could then use %21 and %22 for INREC BUILD or OVERLAY, %102 and %103 for OUTREC BUILD or OVERLAY, %40 and %45 for OUTFIL statement 1 BUILD or OVERLAY, and %30, %31 and %32 for OUTFIL statement 2 BUILD or OVERLAY. Note that you could not, for example, use %21 and %22 for OUTREC BUILD or OVERLAY.

If you specify PARSE=(...),IFTHEN=(...) or IFTHEN=(...),PARSE=(...), DFSORT will terminate. If you need to specify PARSE with IFTHEN, specify it within one or more individual IFTHEN clauses, as appropriate. For details, see the description of "IFTHEN".

%n, %nn or %nnn
Assigns %n, %nn or %nnn to a parsed field. The variable-length data defined for the parsed field is extracted to a fixed length area (padded with blanks or truncated as appropriate) and can be used where a p,m field can be used in BUILD or OVERLAY.

n can be 0 to 9, nn can be 00 to 99, and nnn can be 000 to 999 but each %n, %nn or %nnn value can only be defined once per run. This gives you a maximum of 1000 parsed fields per run.

%0 to %9 will be treated as equivalent to %00 to %09. %010 to %099 will be treated as equivalent to %10 to %99. You can define parse field n with %n, %0n or %00n; you can use %n, %0n and %00n interchangeably. You can define parse field nn with %nn or %0nn; you can use %nn and %0nn interchangeably.

Sample Syntax

```
OUTFIL PARSE=(%99=(ABSPOS=6,STARTAT=C'KW=(',ENDAT=C')',FIXLEN=20),
  %10=(ENDBEFR=UC,ENDBEFR=C'$',FIXLEN=5),
  %03=(STARTAFT=BLANKS,FIXLEN=10)),
  OVERLAY=(1:1,5,%3,C'=',%99,1:1,36,SQZ=(SHIFT=LEFT),%010)
```

%
Specifies a parsed field to be ignored. No data is extracted, but the starting point for the next parsed field advances according to the suboperands specified. Use % when you don't need the data from a particular field, but you do need to get to the next field. For example, if we had the four CSV fields shown earlier as input, but we only wanted to extract the first and fourth fields, we could use this OUTFIL statement:

```
OUTFIL PARSE=(%01=(ENDBEFR=C',',FIXLEN=5),
              %=(ENDBEFR=C','),
              %=(ENDBEFR=C','),
              %04=(FIXLEN=5)),
       BUILD=(1:%01,11:%04,21:%01,HEX)
```

Data is extracted for %01 (first field) and %04 (fourth field), but not for % (second and third fields).

FIXLEN=m
Specifies the length (m) of the fixed area to contain the extracted variable-length data for this %nn fixed parsed field. m can be 1 to 32752. You must specify FIXLEN=m for each %nn parsed field. FIXLEN=m is optional for a % ignored field.

When %nn is specified in BUILD or OVERLAY, the variable-length data is extracted to the fixed area and then used for BUILD or OVERLAY processing. The fixed area always contains m bytes. If the extracted data is longer than m, the fixed area contains the first m bytes of the extracted

data (the data is truncated on the right to m bytes). If the extracted data is shorter than m, the fixed area contains the extracted data padded with blanks on the right to m bytes. If no data is extracted, the fixed area contains all blanks.

The length m from FIXLEN=m for fixed parsed fields (%nn) corresponds to the length (m) for fixed fields (p,m). Keep in mind that %nn fields are left-aligned and may be padded on the right with blanks. You must ensure that the data in each %nn field and its length (m) are valid for the way you want to use that %nn field in BUILD or OVERLAY. For example, if you had these records as input:

```
123,8621
1,302
18345,17
```

and you wanted this output:

```
  1.23   86.21
  0.01    3.02
183.45    0.17
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(ENDBEFR=C',',FIXLEN=5),
                %01=(FIXLEN=5)),
         BUILD=(%00,UFF,EDIT=(IIT.TT),2X,%01,UFF,EDIT=(IIT.TT))
```

Note that the 5-byte extracted parsed fields for %00 are left-aligned and padded with blanks (b) like this:

```
123bb
1bbbb
18345
```

and the 5-byte extracted parsed fields for %01 are left-aligned and padded with blanks (b) like this:

```
8621b
302bb
17bbb
```

Thus, you cannot use numeric formats like ZD or FS for these parsed fields, but you can use UFF. Note also that you must ensure that m from FIXLEN=m for each %nn parsed field is valid for the specific BUILD or OVERLAY item in which you use %nn. For example, you could use 1-44 as m for UFF, but you couldn't use 45 as m.

If ENDBEFR and ENDAT are not specified, the Start Pointer is set m bytes past the start of the extracted data. For example, if you had this record as input:

```
    MAX=(ABCDEF)
```

and you specified:

```
    OUTFIL PARSE=(%100=(STARTAT=C'MAX',FIXLEN=8),
                 %101=(FIXLEN=3),%102=(FIXLEN=1)),
           BUILD=(%100,12:%101,20:%102)
```

The output record would be:

```
MAX=(ABC   DEF     )
```

Sample Syntax

```
  OUTFIL PARSE=(%00=(STARTAFT=C'KW=',FIXLEN=12)),
      BUILD=(%00)
```

ABSPOS=p

Sets the Start Pointer for this parsed field to p. p can be 1 to 32752. By default, the Start Pointer for the first %nn parsed field is position 1 for fixed-length records or position 5 for variable-length records, and the Start Pointer for each subsequent %nn parsed field is the Start Pointer set by the previous %nn field. You can use ABSPOS=p to set the Start Pointer to position p to override the default Start Pointer. If the resulting Start Pointer is less than position 5 for variable length records, it will be set to position 5.

Example

If your input is:

```
****|BB|CCCC|
****|EEEE|FF|
```

and you wanted to reformat the data into fixed positions like this:

```
**** BB   CCCC
**** EEEE FF
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%01=(ABSPOS=6,ENDBEFR=C'|',FIXLEN=4),
       %02=(ENDBEFR=C'|',FIXLEN=4)),
     BUILD=(1,4,X,%01,X,%02)
```

The initial Start Pointer for the %01 parsed field is set to position 6 instead of to position 1.

ADDPOS=x

Increments the Start Pointer for this parsed field by x. x can be 1 to 32752. By default, the Start Pointer for the first %nn parsed field is position 1 for fixed-length records or position 5 for variable-length records, and the Start Pointer for each subsequent %nn parsed field is the Start Pointer set by the previous %nn field. You can use ADDPOS=x to increment the Start Pointer by x to override the default Start Pointer.

Sample Syntax

```
   OUTFIL PARSE=(%00=(ENDAT=C'||',FIXLEN=10),
        %01=(ADDPOS=5,STARTAFT=C';',FIXLEN=6)),
      BUILD=(%00,TRAN=ALTSEQ,%01)
```

SUBPOS=y

Decrements the Start Pointer for this parsed field by y. y can be 1 to 32752. By default, the Start Pointer for the first %nn parsed field is position 1 for fixed-length records or position 5 for variable-length records, and the Start Pointer for each subsequent %nn parsed field is the Start Pointer set by the previous %nn field. You can use SUBPOS=y to decrement the Start Pointer by x to override the default Start Pointer. If the resulting Start Pointer is less than position 1 for fixed-length records, it will be set to position 1. If the resulting Start Pointer is less than position 5 for variable-length records, it will be set to position 5.

Sample Syntax

```
  OUTFIL PARSE=(%00=(ENDBEFR=C'|;|',FIXLEN=10),
       %01=(SUBPOS=1,STARTAT=C'|',ENDAT=C'|',FIXLEN=12)),
     BUILD=(%00,TRAN=ALTSEQ,%01)
```

STARTAFT=string

Data is to be extracted for this parsed field starting after the last byte of the specified string. The search for the string begins at the Start Pointer. If the specified string is not found, data is not extracted for this parsed field or any subsequent parsed fields. If the specified string is found, data is extracted to the fixed area for this parsed field starting at the byte after the end of the string, and the Start Pointer is set to the byte after the end of the string.

string can be 1 to 256 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See "INCLUDE control statement" on page 91 for details of coding character and hexadecimal string constants.

Example

If your input is:

```
         MAX=25,832        MIN=2,831  $4
  MAX=1,275    MIN=17 %3
```

and you wanted your output to contain MAX-MIN, you could use this OUTFIL statement:

```
OUTFIL PARSE=(%00=(STARTAFT=C'MAX=',ENDBEFR=X'40',FIXLEN=6),
    %01=(STARTAFT=C'MIN=',ENDBEFR=X'40',FIXLEN=6)),
  OVERLAY=(50:C'DELTA:  ',%00,UFF,SUB,%01,UFF,TO=ZD,LENGTH=8)
```

STARTAFT=an

Data is to be extracted for this parsed field starting after the first character found from the specified alphanumeric set. The search for a character in the specified set begins at the Start Pointer. If a character in the set is not found, data is not extracted for this parsed field or any subsequent parsed field. If a character in the set is found, data is extracted to the fixed area for this parsed field starting at the byte after that character, and the Start Pointer is set to the byte after that character.

an can be one of the following alphanumeric character sets:

- **UC**: Uppercase characters (A-Z)
- **LC**: Lowercase characters (a-z)
- **MC**: Mixed case characters (A-Z, a-z)
- **UN**: Uppercase and numeric characters (A-Z, 0-9)
- **LN**: Lowercase and numeric characters (a-z, 0-9)
- **MN**: Mixed case and numeric characters (A-Z, a-z, 0-9)
- **NUM**: Numeric characters (0-9)

Note that STARTAFT=an is equivalent to specifying a STARTAFT=c operand for each character in the set. For example:

```
%=(STARTAFT=NUM)
```

is equivalent to specifying

```
      %=(STARTAFT=C'0',STARTAFT=C'1',STARTAFT=C'2',STARTAFT=C'3',
         STARTAFT=C'4',STARTAFT=C'5',STARTAFT=C'6',STARTAFT=C'7',
         STARTAFT=C'8',STARTAFT=C'9')
```

You can add more characters to a set with additional STARTAFT operands. For example, if you wanted the lowercase characters and the $ and @ characters for the set, you could use:

```
%05=(STARTAFT=LC,STARTAFT=C'$',STARTAFT=C'@',FIXLEN=10)
```

Example

If your input is:

```
          a123    1Stop
     Z0056    2rest
      q8    3go
```

and you wanted your output to be:

```
000123 STOP
000056 REST
000008 GO
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(STARTAFT=MC,ENDBEFR=C' ',FIXLEN=5),
                %01=(STARTAFT=NUM,FIXLEN=8)),
         BUILD=(%00,UFF,EDIT=(TTTTTT),X,%01,TRAN=LTOU)
```

STARTAFT=BLANKS

Data is to be extracted for this parsed field starting at the first nonblank after one or more blanks. The search for a blank begins at the Start Pointer. If a blank is not found, data is not extracted for this parsed field or any subsequent parsed fields. If a blank is found, data is extracted to the fixed area for this parsed field starting at the first nonblank, and the Start Pointer is set to the first nonblank.

Example

If your input is:

```
         Frank     D28
  Vicky                    D52
```

and you wanted your output to be:

```
Frank        D28
Vicky        D52
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(STARTAFT=BLANKS,FIXLEN=8),
     %01=(STARTAFT=BLANKS,FIXLEN=3)),
    BUILD=(%00,14:%01)
```

STARTAT=string

Data is to be extracted for this parsed field starting at the first byte of the specified string. The search for the string begins at the Start Pointer. If the specified string is not found, data is not extracted for this parsed field or any subsequent parsed fields. If the specified string is found, data is extracted to the fixed area for this parsed field starting at the first byte of the string, and the Start Pointer is set to the byte after the end of the string.

string can be 1 to 256 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

Example

If your input is:

```
          MAX=58321        MIN=00273
  MAX=01275    MIN=00017
```

and you wanted your output to be:

```
MAX=58321 MIN=00273
MAX=01275 MIN=00017
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(STARTAT=C'MAX=',FIXLEN=9),
     %01=(STARTAT=C'MIN=',FIXLEN=9)),
    BUILD=(%00,X,%01)
```

STARTAT=an

Data is to be extracted for this parsed field starting at the first character found from the specified alphanumeric set. The search for a character in the specified set begins at the Start Pointer. If a character in the set is not found, data is not extracted for this parsed field or any subsequent parsed field. If a character in the set is found, data is extracted to the fixed area for this parsed field starting at that character, and the Start Pointer is set to the byte after that character.

See STARTAFT=an previously in this section for more information on using the available alphanumeric character sets.

Example

If your input is:

```
$@$321%@%$Frank
@053$Susan
%%%$$836%$Vicky
```

and you wanted your output to contain the amount and name fields:

```
321 Frank
053 Susan
836 Vicky
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(STARTAT=NUM,FIXLEN=3),
      %01=(STARTAT=UC,FIXLEN=10)),
    BUILD=(%00,X,%01)
```

STARTAT=BLANKS
Data is to be extracted for this parsed field starting at the first blank. The search for a blank begins at the Start Pointer. If a blank is not found, data is not extracted for this parsed field or any subsequent parsed fields. If a blank is found, data is extracted to the fixed area for this parsed field starting at the first blank, and the Start Pointer is set to the first nonblank.

STARTAT=NONBLANK
Data is to be extracted for this parsed field starting at the first nonblank. The search for a nonblank begins at the Start Pointer. If a nonblank is not found, data is not extracted for this parsed field or any subsequent parsed fields. If a nonblank is found, data is extracted to the fixed area for this parsed field starting at the nonblank, and the Start Pointer is set to the nonblank.

Example

If your input is:

```
Frank     D28
   Victoria           D52
  Holly D15
Roberta     D52
```

and you wanted your output to be:

```
Frank        D28
Victoria     D52
Holly        D15
Roberta      D52
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(STARTAT=NONBLANK,ENDBEFR=BLANKS,FIXLEN=8),
      %01=(FIXLEN=3)),
    BUILD=(%00,14:%01)
```

Note: Multiple STARTAFT/STARTAT strings, UC, LC, MC, UN, LN, MN, NUM, BLANKS or NONBLANK can be used for a %nn parsed field to search for more than one string, set of characters, blanks or a nonblank. The first search satisfied is used.

Example

If your input is:

```
  12   MAXCC=08
58 MINCC=00
         MAXCC=04
```

and you wanted your output to be:

```
MAXCC=08
MINCC=00
MAXCC=04
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(STARTAT=C'MAXCC=',STARTAT=C'MINCC=',
               FIXLEN=8)),BUILD=(%00)
```

ENDBEFR=string

Data is to be extracted for this parsed field ending before the first byte of the specified string. The search for the string begins at the Start Pointer. If the specified string is not found, data is extracted to the fixed area for this parsed field up to the end of the record, but data is not extracted for any subsequent parsed fields. If the specified string is found, data is extracted to the fixed area for this parsed field up to the byte before the start of the string, and the Start Pointer is set to the byte after the end of the string.

string can be 1 to 256 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See "INCLUDE control statement" on page 91 for details of coding character and hexadecimal string constants.

Example

If your input is:

```
Morgan Hill;California;5000:
San Jose;California;2000:
Austin;Texas;8000:
```

and you wanted your output to be:

```
Morgan Hill   California  5000
San Jose      California  2000
Austin        Texas       8000
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(ENDBEFR=C';',FIXLEN=14),
      %01=(ENDBEFR=C';',FIXLEN=12),
      %02=(ENDBEFR=C':',FIXLEN=4)),
    BUILD=(%00,%01,%02)
```

ENDBEFR=X'00' can be used to extract null-terminated strings to fixed parsed fields.

Example

If your input contains null-terminated strings that look like this in hex:

```
F0F5F800
F1F2F3F4F500
F9F7F2F000
F500
F0F0F3F700
```

and you wanted to convert the null-terminated strings to right-aligned numeric values like this:

```
   58
12345
 9720
    5
   37
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%01=(ENDBEFR=X'00',FIXLEN=5)),
    BUILD=(%01,UFF,EDIT=(IIIIT))
```

ENDBEFR=an

Data is to be extracted for this parsed field ending before the first character found from the specified alphanumeric set. The search for a character in the specified set begins at the Start Pointer. If a character in the set is not found, data is extracted to the fixed area for this parsed field up to the end of the record, but data is not extracted for any subsequent parsed fields. If a character in the set is found, data is extracted to the fixed area for this parsed field up to the byte before that character, and the Start Pointer is set to the byte after that character.

See STARTAFT=an previously in this section for more information on using the available alphanumeric character sets.

Example

If your input is:

```
$$$$Samantha
$052
$$$$$cat
$$358721
```

and you wanted to extract the leading $ characters in each input record, you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%10=(ENDBEFR=MN,FIXLEN=10)),
     BUILD=(%10)
```

ENDBEFR=BLANKS

Data is to be extracted for this parsed field ending before the first blank. The search for a blank begins at the Start Pointer. If a blank is not found, data is extracted to the fixed area for this parsed field up to the end of the record, but data is not extracted for any subsequent parsed fields. If a blank is found, data is extracted to the fixed area for this parsed field up to the byte before the blank, and the Start Pointer is set to the first nonblank.

Example

If your input is:

```
1     Frank     D28 123   No
2 Loretta            D52   58 Yes
```

and you wanted your output to be:

```
0123 Frank   D28
0058 Loretta D52
```

you could use this OUTFIL statement:

```
   OUTFIL PARSE=(%00=(STARTAFT=BLANKS,ENDBEFR=BLANKS,FIXLEN=8),
      %01=(ENDBEFR=BLANKS,FIXLEN=3),
      %02=(ENDBEFR=C' ',FIXLEN=4)),
    BUILD=(%02,UFF,EDIT=(TTTT),X,%00,%01)
```

ENDAT=string

Data is to be extracted for this parsed field ending at the last byte of the specified string. The search for the string begins at the Start Pointer. If the specified string is not found, data is extracted to the fixed area for this parsed field up to the end of the record, but data is not extracted for any subsequent parsed fields. If the specified string is found, data is extracted to the fixed area for this parsed field up to the last byte of the string, and the Start Pointer is set to the byte after the end of the string

string can be 1 to 256 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

Example

If your input is:

```
12,(June)
5852,(Gracie)
```

and you wanted your output to be:

```
(June)
(Gracie)
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(STARTAT=C'(',ENDAT=C')',FIXLEN=12)),
    BUILD=(%00)
```

ENDAT=an

Data is to be extracted for this parsed field ending at the first character found from the specified alphanumeric set. The search for a character in the specified set begins at the Start Pointer. If a character in the set is not found, data is extracted to the fixed area for this parsed field up to the end of the record, but data is not extracted for any subsequent parsed fields. If a character in the set is found, data is extracted to the fixed area for this parsed field up to that character, and the Start Pointer is set to the byte after that character.

See STARTAFT=an previously in this section for more information on using the available alphanumeric character sets.

Example

If your input is:

```
        A0005X2
           Z081789Q331
R3M05
    S52J06834
```

and you wanted your output to be:

```
A0005X     2
Z081789Q   331
R3M        05
S52J       06834
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%01=(STARTAT=UC,ENDAT=UC,FIXLEN=10),
                %02=(FIXLEN=5)),
         BUILD=(%01,X,%02)
```

ENDAT=BLANKS

Data is to be extracted for this parsed field ending at the last blank before a nonblank. The search for a blank begins at the Start Pointer. If a blank is not found, data is extracted to the fixed area for this parsed field up to the end of the record, but data is not extracted for any subsequent parsed fields. If a blank is found, data is extracted to the fixed area for this parsed field up to the byte before the first nonblank, and the Start Pointer is set to the first nonblank.

Note: Multiple ENDBEFR/ENDAT strings, UC, LC, MC, UN, LN, MN, NUM, or BLANKS can be used for a %nn parsed field to search for more than one string, set of characters or blanks. The first search satisfied is used.

Example

If your input is:

```
  12;
58:
         04/
```

and you wanted your output to be:

```
12
58
04
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%01=(ENDBEFR=C';',ENDBEFR=C':',ENDBEFR=C'/',
                FIXLEN=10)),
     BUILD=(%01,UFF,TO=ZD,LENGTH=2,80:X)
```

PAIR=APOST

Do not search for strings or blanks between apostrophe (') pairs. Use POST=APOST when you might have strings you are searching for within literals that should not satisfy the search.

Once an apostrophe is found, searching is discontinued until another apostrophe is found. Searching then continues after the second apostrophe of the pair. If an unpaired apostrophe is found, strings or BLANKS will not be recognized from that point on, so be careful not to set the Start Pointer in the middle of an apostrophe pair.

Do not specify PAIR=APOST if you want to search for a string containing an apostrophe, because the string will not be searched for within the paired apostrophes.

Example

If your input is:

```
'23','12,567,823','5,032'
'9,903','18,321','8'
```

and you wanted your output to be:

```
        23 12,567,823      5,032
    9,903     18,321          8
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(ENDBEFR=C',',FIXLEN=12,PAIR=APOST),
                %01=(ENDBEFR=C',',FIXLEN=12,PAIR=APOST),
                %02=(FIXLEN=12)),
         BUILD=(%00,UFF,EDIT=(II,III,IIT),X,
                %01,UFF,EDIT=(II,III,IIT),X,
                %02,UFF,EDIT=(II,III,IIT))
```

With PAIR=APOST, the commas outside the apostrophe pairs satisfy the search, but the commas within the apostrophe pairs do not satisfy the search.

PAIR=QUOTE

Do not search for strings or blanks between quote (") pairs. Use POST=QUOTE when you might have strings you are searching for within literals that should not satisfy the search.

Once a quote is found, searching is discontinued until another quote is found. Searching then continues after the second quote of the pair. If an unpaired quote is found, strings or BLANKS will not be recognized from that point on, so be careful not to set the Start Pointer in the middle of a quote pair.

Do not specify PAIR=QUOTE if you want to search for a string containing a quote, because the string will not be searched for within the paired quotes.

Example

If your input is:

```
"23","12,567,823","5,032"
"9,903","18,321","8"
```

and you wanted your output to be:

```
       23 12,567,823      5,032
    9,903     18,321          8
```

you could use this OUTFIL statement:

```
OUTFIL PARSE=(%00=(ENDBEFR=C',',FIXLEN=12,PAIR=QUOTE),
              %01=(ENDBEFR=C',',FIXLEN=12,PAIR=QUOTE),
              %02=(FIXLEN=12)),
       BUILD=(%00,UFF,EDIT=(II,III,IIT),X,
              %01,UFF,EDIT=(II,III,IIT),X,
              %02,UFF,EDIT=(II,III,IIT))
```

With PAIR=QUOTE, the commas outside the quote pairs satisfy the search, but the commas within the quote pairs do not satisfy the search.

Default for PARSE: None; must be specified.

REPEAT=v

Repeat this parsed field v times.

REPEAT=v can be used with % to specify v identically defined consecutive parsed fields to be ignored. v can be 2 to 1000. For example, to ignore five consecutive comma delimited fields, you can use:

```
%=(ENDBEFR=C',',REPEAT=5),
```

which is equivalent to using:

```
%=(ENDBEFR=C','),
%=(ENDBEFR=C','),
%=(ENDBEFR=C','),
%=(ENDBEFR=C','),
%=(ENDBEFR=C','),
```

REPEAT=v can be used with %nn to specify v identically defined consecutive parsed fields for which data is to be extracted. v can be 2 to 1000. The parsed fields will start with the %nn field you select and be incremented by 1 for each repeated parsed field.

For example, to extract four consecutive 10-byte comma delimited fields as %11, %12, %13 and %14, you can use:

```
%11=(ENDBEFR=C',',FIXLEN=10,REPEAT=4),
```

which is equivalent to using:

```
%11=(ENDBEFR=C',',FIXLEN=10),
%12=(ENDBEFR=C',',FIXLEN=10),
%13=(ENDBEFR=C',',FIXLEN=10),
%14=(ENDBEFR=C',',FIXLEN=10),
```

You must ensure that the %nn fields resulting from the use of REPEAT=v are unique for the run. For example, %21=(ENDBEFR=C',',FIXLEN=8,REPEAT=10) defines %21-%30, so if you used any of %21-%30 separately, DFSORT would issue an error message and terminate; the next available %nn field would be %31.

You must also ensure that any %nnn field resulting from the use of REPEAT=v does not exceed %999. For example, %998=(STARTAFT=C'$',FIXLEN=4,REPEAT=3) defines %998-%1000; since %1000 is invalid, DFSORT would issue an error message and terminate.

Example

If your input is the following period delimited values:

```
11.2.30.4.55.160.7.82.95.37
10.22.3.41.5.16.72.2.5.42
3.18.33.12.7.9.52.12
```

and you wanted to extract the first value, skip the next three values and then extract the last six values to produce output like this:

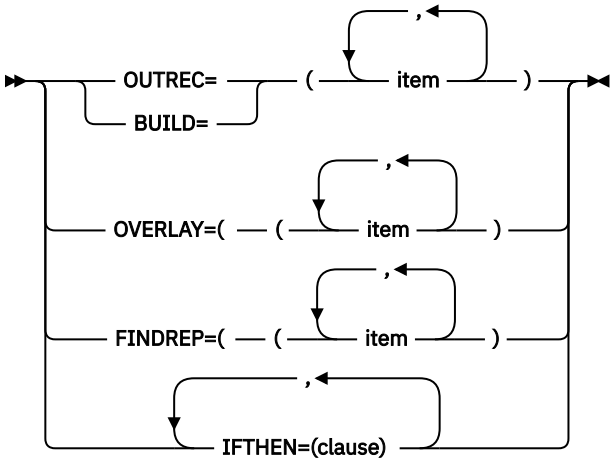
```
011 055 160 007 082 095 037
010 005 016 072 002 005 042
003 007 009 052 012 000 000
```

you could use this OUTFIL statement:

```
  OUTFIL PARSE=(%00=(ENDBEFR=C'.',FIXLEN=3),
            %=(ENDBEFR=C'.',REPEAT=3),
            %01=(ENDBEFR=C'.',FIXLEN=3,REPEAT=6)),
         BUILD=(%00,UFF,EDIT=(TTT),X,
            %01,UFF,EDIT=(TTT),X,
            %02,UFF,EDIT=(TTT),X,
            %03,UFF,EDIT=(TTT),X,
            %04,UFF,EDIT=(TTT),X,
            %05,UFF,EDIT=(TTT),X,
            %06,UFF,EDIT=(TTT))
```

Default for PARSE: None; must be specified.

OUTREC, BUILD, OVERLAY, FINDREP, or IFTHEN



These operands allow you to reformat the OUTFIL input records in this OUTFIL group.

You can create the reformatted OUTFIL records in one of the following ways using unedited, edited, or converted input fields (p,m for fixed fields, or %nn for parsed fields - see PARSE) and a variety of constants:

- **BUILD or OUTREC:** Reformat each record by specifying all of its items one by one. Build gives you complete control over the items you want in your reformatted OUTFIL records and the order in which they appear. You can delete, rearrange and insert fields and constants. Examples:

```
 OUTFIL FNAMES=OUT1,BUILD=(1,20,C'ABC',26:5C'*',
 15,3,PD,EDIT=(TTT.TT),21,30,80:X)

OUTFIL FNAMES=OUT2,
  PARSE=(%00=(ENDBEFR=C',',FIXLEN=6),
         %01=(ENDBEFR=C',',FIXLEN=5),
         %=(ENDBEFR=C','),
         %02=(FIXLEN=6)),
  BUILD=(C'|',%01,SFF,ADD,%02,SFF,M4,LENGTH=12,
         C'|',%00,C'|')
```

- **OVERLAY:** Reformat each record by specifying just the items that overlay specific columns. Overlay lets you change specific existing columns without affecting the entire record. Example:

```
  OUTFIL OVERLAY=(45:45,8,TRAN=LTOU)
```

- **FINDREP**: Reformat each record by doing various types of find and replace operations. Example:

```
OUTREC FINDREP=(INOUT=(C'Pigeon',C'Dove',C'Hawk',C'Eagle'))
```

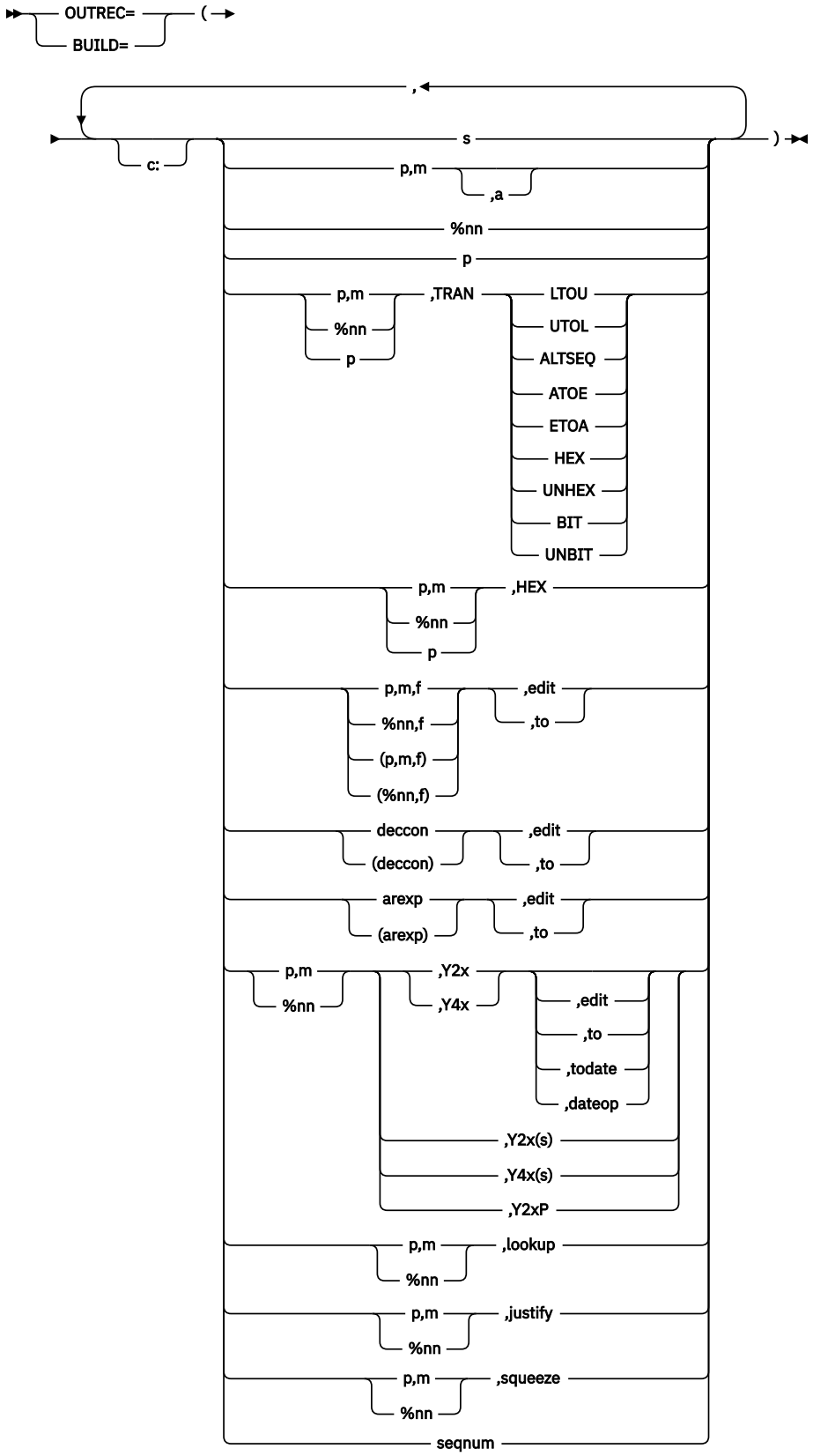
- **IFTHEN clauses:** Reformat different records in different ways by specifying how build, overlay, find/replace, or group operation items are applied to records that meet given criteria. IFTHEN clauses let you use sophisticated conditional logic to choose how different record types are reformatted. Example:

```
OUTFIL IFTHEN=(WHEN=(1,5,CH,EQ,C'TYPE1'),
         BUILD=(1,40,C'**',+1,TO=PD)),
       IFTHEN=(WHEN=(1,5,CH,EQ,C'TYPE2'),
         BUILD=(1,40,+2,TO=PD,X'FFFF')),
       IFTHEN=(WHEN=NONE,OVERLAY=(45:C'NONE'))
```

You can choose to include any or all of the following items in your reformatted OUTFIL records:

- Fixed position/length fields or variable position/length fields. For fixed fields, you specify the starting position and length of the field directly. For variable fields, such as delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings (and many other types), you define rules that allow DFSORT to extract the relevant data into fixed parsed fields, and then use the parsed fields as you would use fixed fields.
- Blanks, binary zeros, character strings, and hexadecimal strings.
- Current date, future date, past date, and current time in various forms.
- Replaced or removed strings.
- Unedited input fields aligned on byte, halfword, fullword, and doubleword boundaries.
- Hexadecimal or bit representations of binary input fields.
- Characters translated from uppercase to lowercase, lowercase to uppercase, ASCII to EBCDIC or EBCDIC to ASCII.
- Left-justified, right-justified, left-squeezed or right-squeezed input fields.
- Numeric input fields of various formats converted to different numeric formats, or to character format edited to contain signs, thousands separators, decimal points, leading zeros or no leading zeros, and so on.
- Decimal constants converted to different numeric formats, or to character format edited to contain signs, thousands separators, decimal points, leading zeros or no leading zeros, and so on.
- The results of arithmetic expressions combining fields, decimal constants, operators (MIN, MAX, MUL, DIV, MOD, ADD and SUB) and parentheses converted to different numeric formats, or to character format edited to contain signs, thousands separators, decimal points, leading zeros or no leading zeros, and so on.
- SMF, TOD, and ETOD date and time fields converted to different numeric formats, or to character format edited to contain separators, leading zeros or no leading zeros, and so on.
- The results of various types of arithmetic operations for input date fields.
- Input date fields of one type (CH, ZD, PD, 2-digit year, 4-digit year, Julian, Gregorian) converted to corresponding output date fields of another type or to a corresponding day of the week.
- Sequence numbers in various formats.
- A character constant, hexadecimal constant or input field selected from a lookup table, based on a character, hexadecimal or bit constant as input.
- A zoned decimal group identifier, a zoned decimal group sequence number, or a field propagated from the first record of a group to all of the records of a group.

OUTREC or BUILD



Specifies all of the items in the reformatted OUTFIL record in the order in which they are to be included. The reformatted OUTFIL record consists of the separation fields, edited and unedited input fields (p,m for fixed fields, or %nn for parsed fields - see PARSE), edited decimal constants, edited

results of arithmetic expressions, and sequence numbers you select, in the order in which you select them, aligned on the boundaries or in the columns you indicate.

For variable-length records, the first item in the BUILD or OUTREC parameter must specify or include the unedited 4-byte record descriptor word (RDW), that is, you must start with 1,m with m equal to or greater than 4. If you want to include the bytes from a specific position to the end of each input record at the end of each reformatted output record, you can specify that starting position (p) as the last item in the BUILD or OUTREC parameter. For example:

```
OUTFIL OUTREC=(1,4,          unedited RDW
   1,2,BI,TO=ZD,LENGTH=5,    display RDW length in decimal
   C'|',                     | separator
   5)                        display input positions 5 to end
```

For fixed-length records, the first input and output data byte starts at position 1. For variable-length records, the first input and output data byte starts at position 5, after the RDW in positions 1-4.

You can use the BUILD or OUTREC parameter to produce multiple reformatted output records for each OUTFIL input record, with or without intervening blank output records.

You can use the BUILD or OUTREC parameter in conjunction with the VTOF or CONVERT parameter to convert variable-length record data sets to fixed-length record data sets.

You can use the BUILD or OUTREC parameter with the FTOV parameter to convert fixed-length record data sets to variable-length record data sets.

You can use the VLFILL parameter to allow processing of variable-length input records which are too short to contain all specified BUILD or OUTREC fields.

You can use the VLTRIM parameter in conjunction with the BUILD or OUTREC parameter to remove specified trailing bytes from the end of variable-length records.

You can use the VLTRAIL parameter in conjunction with the BUILD or OUTREC parameter to insert a string at the end of variable-length records.

The BUILD or OUTREC parameter can be used with any or all of the report parameters (LINES, HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS, BLKCCH1, BLKCCH2, BLKCCT1, and NODETAIL) to produce reports. The report parameters specify the report records to be produced, while the BUILD or OUTREC parameter specifies the reformatted data records to be produced. DFSORT uses ANSI carriage control characters to control page ejects and the placement of the lines in your report, according to your specifications. You can use the REMOVECC parameter to remove the ANSI carriage control characters.

When you create an OUTFIL report, the length for the longest or only data record must be equal to or greater than the maximum report record length. You can use the BUILD or OUTREC parameter to force a length for the data records that is longer than any report record; you can then either let DFSORT compute and set the LRECL, or ensure that the computed LRECL is equal to the existing or specified LRECL. Remember to allow an extra byte in the LRECL for the ANSI carriage control character.

For example, if your data records are 40 bytes, but your longest report record is 60 bytes, you can use a BUILD or OUTREC parameter such as:

```
OUTREC=(1,40,80:X)
```

DFSORT will then set the LRECL to 81 (1 byte for the ANSI carriage control character plus 80 bytes for the length of the data records), and pad the data records with blanks on the right.

If you don't want the ANSI carriage control characters to appear in the output data set, use the REMOVECC parameter to remove them. For example, if you specify:

```
OUTREC=(1,40,80:X),REMOVECC
```

DFSORT will set the LRECL to 80 instead of 81 and remove the ANSI carriage control character from each record before it is written.

The BUILD or FIELDS parameter of the OUTREC statement differs from the BUILD or OUTREC parameter of the OUTFIL statement in the following ways:

- The BUILD or FIELDS parameter of the OUTREC statement applies to all input records; the BUILD or OUTREC parameter of the OUTFIL statement only applies to the OUTFIL input records for its OUTFIL group.
- The BUILD or OUTREC parameter of the OUTFIL statement supports the slash (/) separator for creating blank records and new records; the BUILD or FIELDS parameter of the OUTREC statement does not.

The reformatted OUTFIL output record consists of the separation fields, edited and unedited input fields, (p,m for fixed fields, or %nn for parsed fields - see PARSE), edited decimal constants, edited results of arithmetic expressions, and sequence numbers you select, in the order in which you select them, aligned on the boundaries or in the columns you indicate.

c:

specifies the position (column) for a separation field, input field, decimal constant, arithmetic expression, or sequence number, relative to the start of the reformatted OUTFIL output record. Count the RDW (variable-length output records only) but not the carriage control character (for reports) when specifying c:. That is, 1: indicates the first byte of the data in fixed-length output records and 5: indicates the first byte of the data in variable-length output records.

Unused space preceding the specified column is padded with EBCDIC blanks. The following rules apply:

- c must be a number between 1 and 32752.
- c: must be followed by a separation field, input field, decimal constant, or arithmetic expression.
- c must not overlap the previous input field or separation field in the reformatted OUTFIL output record.
- For variable-length records, c: must not be specified before the first input field (the record descriptor word) nor after the variable part of the OUTFIL input record.
- The colon (:) is treated like the comma (,) or semicolon (;) for continuation to another line.

See Table 33 on page 131 for examples of valid and invalid column alignment.

s

specifies that a separation field (blanks, zeros, character string, hexadecimal string, current date, future date, past date, or current time) is to appear in the reformatted OUTFIL output record, or that a new output record is to be started, with or without intervening blank output records. These separation elements (separation fields, new record indicators, and blank record indicators) can be specified before or after any input field. Consecutive separation elements may be specified. For variable-length records, separation elements must not be specified before the first input field (the record descriptor word) or after the variable part of the OUTFIL input record. Permissible values are nX, nZ, nC'xx...x', nX'yy...yy', various date and time constants, /.../ and n/.

nX

Blank separation. n bytes of EBCDIC blanks (X'40') are to appear in the reformatted OUTFIL output records. n can range from 1 to 4095. If n is omitted, 1 is used.

See Table 34 on page 132 for examples of valid and invalid blank separation.

nZ

Binary zero separation. n bytes of binary zeros (X'00') are to appear in the reformatted OUTFIL output records. n can range from 1 to 4095. If n is omitted, 1 is used.

See Table 35 on page 132 for examples of valid and invalid binary zero separation.

nC'xx...x'

Character string separation. n repetitions of the character string constant (C'xx...x') are to appear in the reformatted OUTFIL output records. n can range from 1 to 4095. If n is omitted, 1 is used. x can be any EBCDIC character. You can specify from 1 to 256 characters.

If you want to include a single apostrophe in the character string, you must specify it as two single apostrophes:

```
Required:  O'NEILL Specify:  C'O''NEILL'
```

See Table 36 on page 132 for examples of valid and invalid character string separation.

nX'yy...yy'

Hexadecimal string separation. n repetitions of the hexadecimal string constant (X'yy...yy') are to appear in the reformatted OUTFIL output records. n can range from 1 to 4095. If n is omitted, 1 is used.

The value yy represents any pair of hexadecimal digits. You can specify from 1 to 256 pairs of hexadecimal digits.

See Table 37 on page 133 for examples of valid and invalid hexadecimal string separation.

DATEn, DATEN(c), DATEnP

Constant for current date. The current date of the run is to appear in the reformatted OUTFIL output records.

DATE1, &DATE1, DATE1(c), &DATE1(c), DATE2, &DATE2, DATE2(c), &DATE2(c), DATE3, &DATE3, DATE3(c) or &DATE3(c) can be used to generate a character constant for the current date of the run. DATE1P, &DATE1P, DATE2P, &DATE2P, DATE3P or &DATE3P can be used to generate a packed decimal constant for the current date of the run.

Table 43 on page 250 shows the form of the constant generated for each current date operand and an example of the actual constant generated when the date of the run is June 21, 2005 at 4:42:45 PM, using (/) for (c) where relevant. yyyy represents the year, mm (for date) represents the month (01-12), dd represents the day (01-31), ddd represents the day of the year (001-366), hh represents the hour (00-23), mm (for time) represents the minutes (00-59), ss represents the seconds (00-59), and c can be any character *except* a blank.

Table 43. Current date constants

Format of Operand	Format of Constant	Length (bytes)	Example of Constant
DATE1	C'yyyymmdd'	8	C'20050621'
DATE1(c)	C'yyyycmmcdd'	10	C'2005/06/21'
DATE1P	P'yyyymmdd'	5	P'20050621'
DATE2	C'yyyymm'	6	C'200506'
DATE2(c)	C'yyyycmm'	7	C'2005/06'
DATE2P	P'yyyymm'	4	P'200506'
DATE3	C'yyyyddd'	7	C'2005172'
DATE3(c)	C'yyyycddd'	8	C'2005/172'
DATE3P	P'yyyyddd'	4	P'2005172'
DATE4	C'yyyy-mm-dd-hh.mm.ss'	19	C'2005-06-21-16.52.45'
DATE5	C'yyyy-mm-dd-hh.mm.ss.nnnnnn'	26	C'2005-06-21-16.52.45.602837'

Note: You can precede each of the operands in the table with an & with identical results.

&DATEn, &DATEN(c), &DATEnP
Can be used instead of DATEn, DATEn(c) and DATEnP, respectively

DATEn+r, DATEn(c)+r, DATEnP+r
Constant for future date. A future date relative to the current date of the run is to appear in the reformatted OUTFIL output records.

DATE1+d, &DATE1+d, DATE1(c)+d, &DATE1(c)+d, DATE2+m, &DATE2+m, DATE2(c)+m, &DATE2(c)+m, DATE3+d, &DATE3+d, DATE3(c)+d or &DATE3(c)+d can be used to generate a character constant for a future date relative to the current date of the run. DATE1P+d, &DATE1P+d, DATE2P+m, &DATE2P+m, DATE3P+d or &DATE3P+d can be used to generate a packed decimal constant for a future date relative to the current date of the run. d is days in the future and m is months in the future. d and m can be 0 to 9999.

Table 44 on page 251 shows the form of the constant generated for each future date operand and an example of the actual constant generated when the date of the run is June 21, 2005, using (/) for (c) where relevant. yyyy represents the year, mm (for date) represents the month (01-12), dd represents the day (01-31), ddd represents the day of the year (001-366), and c can be any character *except* a blank.

Table 44. Future date constants

Format of Operand	Format of Constant	Length (bytes)	Example of Operand	Example of Constant
DATE1+d	C'yyyymmdd'	8	DATE1+11	C'20050702'
DATE1(c)+d	C'yyyycmmcdd'	10	DATE1(/)+90	C'2005/09/19'
DATE1P+d	P'yyyymmdd'	5	DATE1P+11	P'20050702'
DATE2+m	C'yyyymm'	6	DATE2+2	C'200508'
DATE2(c)+m	C'yyyycmm'	7	DATE2(.)+25	C'2007.07'
DATE2P+m	P'yyyymm'	4	DATE2P+2	P'200508'
DATE3+d	C'yyyyddd'	7	DATE3+200	C'2006007'
DATE3(c)+d	C'yyyycddd'	8	DATE3(-)+1	C'2005-171'
DATE3P+d	P'yyyyddd'	4	DATE3P+200	P'2006007'

Note: You can precede each of the operands in the table with an & with identical results.

&DATEn+r, &DATEn(c)+r, &DATEnP+r
Can be used instead of DATEn+r, DATEn(c)+r and DATEnP+r, respectively.

DATEn-r, DATEn(c)-r, DATEnP-r
Constant for past date. A past date relative to the current date of the run is to appear in the reformatted OUTFIL output records.

DATE1-d, &DATE1-d, DATE1(c)-d, &DATE1(c)-d, DATE2-m, &DATE2-m, DATE2(c)-m, &DATE2(c)-m, DATE3-d, &DATE3-d, DATE3(c)-d or &DATE3(c)-d can be used to generate a character constant for a past date relative to the current date of the run. DATE1P-d, &DATE1P-d, DATE2P-m, &DATE2P-m, DATE3P-d or &DATE3P-d can be used to generate a packed decimal constant for a past date relative to the current date of the run. d is days in the past and m is months in the past. d and m can be 0 to 9999.

Table 45 on page 252 shows the form of the constant generated for each past date operand and an example of the actual constant generated when the date of the run is June 21, 2005, using (/) for (c) where relevant. yyyy represents the year, mm (for date) represents the month (01-12), dd represents the day (01-31), ddd represents the day of the year (001-366), and c can be any character *except* a blank.

Table 45. Past date constants

Format of Operand	Format of Constant	Length (bytes)	Example of Operand	Example of Constant
DATE1-d	C'yyyymmdd'	8	DATE1-1	C'20050620'
DATE1(c)-d	C'yyyycmmcdd'	10	DATE1(-)-60	C'2005-04-22'
DATE1P-d	P'yyyymmdd'	5	DATE1P-30	P'20050522'
DATE2-m	C'yyyymm'	6	DATE2-6	C'200412'
DATE2(c)-m	C'yyyycmm'	7	DATE2(/)-1	C'2005/05'
DATE2P-m	P'yyyymm'	4	DATE2P-12	P'200406'
DATE3-d	C'yyyyddd'	7	DATE3-300	C'2004238'
DATE3(c)-d	C'yyyycddd'	8	DATE3(.)-21	C'2005.151'
DATE3P-d	P'yyyyddd'	4	DATE3P-172	P'2004366'

Note: You can precede each of the operands in the table with an & with identical results.

&DATEn-r, &DATEn(c)-r, &DATEnP-r
Can be used instead of DATEn-r, DATEn(c)-r and DATEnP-r, respectively.

TIMEn, TIMEn(c), TIMEnP
Constant for current time. The time of the run is to appear in the reformatted OUTFIL output records. Table 46 on page 252 shows the constant generated for each separation field you can specify along with its length and an example using (:) for (c) where relevant. hh represents the hour (00-23), mm represents the minutes (00-59), ss represents the seconds (00-59), and c can be any character *except* a blank.

Table 46. Current time constants

Separation Field	Constant	Length (bytes)	01:55:43 PM
TIME1	C'hhmmss'	6	C'135543'
TIME1(c)	C'hhcmmcss'	8	C'13:55:43'
TIME1P	P'hhmmss'	4	P'135543'
TIME2	C'hhmm'	4	C'1355'
TIME2(c)	C'hhcmm'	5	C'13:55'
TIME2P	P'hhmm'	3	P'1355'
TIME3	C'hh'	2	C'13'
TIME3P	P'hh'	2	P'13'

&TIMEn, &TIMEn(c), &TIMEnP
Can be used instead of TIMEn, TIMEn(c), and TIMEnP, respectively.

DATE
specifies that the current date is to appear in the reformatted OUTFIL output records in the form 'mm/dd/yy', where mm represents the month (01-12), dd represents the day (01-31), and yy represents the last two digits of the year (for example, 04).

&DATE
can be used instead of DATE.

DATE=(abcd)
specifies that the current date is to appear in the reformatted OUTFIL output records in the form 'adbdc', where a, b, and c indicate the order in which the month, day, and year are to appear and whether the year is to appear as two or four digits, and d is the character to be used to separate the month, day and year.

For a, b, and c, use M to represent the month (01-12), D to represent the day (01-31), Y to represent the last two digits of the year (for example, 04), or 4 to represent the four digits of the year (for example, 2004). M, D, and Y or 4 can each be specified only once. Examples: DATE=(DMY.) would produce a date of the form 'dd.mm.yy', which on March 29, 2004, would appear as '29.03.04'. DATE=(4MD-) would produce a date of the form 'yyyy-mm-dd', which on March 29, 2004, would appear as '2004-03-29'.

a, b, c, and d must be specified.

&DATE=(abcd)
can be used instead of DATE=(abcd).

DATENS=(abc)
specifies that the current date is to appear in the reformatted OUTFIL output record in the form 'abc', where a, b and c indicate the order in which the month, day, and year are to appear and whether the year is to appear as two or four digits.

For a, b and c, use M to represent the month (01-12), D to represent the day (01-31), Y to represent the last two digits of the year (for example, 04), or 4 to represent the four digits of the year (for example, 2004). M, D, and Y or 4 can each be specified only once. Examples: DATENS=(DMY) would produce a date of the form 'ddmmyy', which on March 29, 2004, would appear as '290304'. DATENS=(4MD) would produce a date of the form 'yyyymmdd', which on March 29, 2004, would appear as '20040329'.

a, b and c must be specified.

&DATENS=(abc)
can be used instead of DATENS=(abc).

YDDD=(abc)
specifies that the current date is to appear in the reformatted OUTFIL output records in the form 'acb', where a and b indicate the order in which the year and day of the year are to appear and whether the year is to appear as two or four digits, and c is the character to be used to separate the year and day of the year.

For a and b, use D to represent the day of the year (001-366), Y to represent the last two digits of the year (for example, 04), or 4 to represent the four digits of the year (for example, 2004). D, and Y or 4 can each be specified only once. Examples: YDDD=(DY-) would produce a date of the form 'ddd-yy', which on April 7, 2004, would appear as '098-04'. YDDD=(4D/) would produce a date of the form 'yyyy/ddd', which on April 7, 2004, would appear as '2004/098'.

a, b and c must be specified.

&YDDD=(abc)
can be used instead of YDDD=(abc).

YDDDNS=(ab)
specifies that the current date is to appear in the reformatted OUTFIL output records in the form 'ab', where a and b indicate the order in which the year and day of the year are to appear and whether the year is to appear as two or four digits.

For a and b, use D to represent the day of the year (001-366), Y to represent the last two digits of the year (for example, 04), or 4 to represent the four digits of the year (for example, 2004). D, and Y or 4 can each be specified only once. Examples: YDDDNS=(DY) would produce a date of the form 'dddyy', which on April 7, 2004, would appear as '09804'. YDDDNS=(4D) would produce a date of the form 'yyyyddd', which on April 7, 2004, would appear as '2004098'.

a and b must be specified.

&YDDDNS=(ab)
can be used instead of YDDDNS=(ab).

TIME
specifies that the current time is to appear in the reformatted OUTFIL output records in the form 'hh:mm:ss', where hh represents the hour (00-23), mm represents the minutes (00-59), and ss represents the seconds (00-59).

&TIME
can be used instead of TIME.

TIME=(abc)
specifies that the current time is to appear in the reformatted OUTFIL output records in the form 'hhcmmcss' (24-hour time) or 'hhcmmcss xx' (12-hour time).

If ab is 24, the time is to appear in the form 'hhcmmcss' (24-hour time) where hh represents the hour (00-23), mm represents the minutes (00-59), ss represents the seconds (00-59), and c is the character used to separate the hours, minutes, and seconds. Example: TIME=(24.) would produce a time of the form 'hh.mm.ss', which at 08:25:13 pm would appear as '20.25.13'.

If ab is 12, the time is to appear in the form 'hhcmmcss xx' (12-hour time) where hh represents the hour (01-12), mm represents the minutes (00-59), ss represents the seconds (00-59), xx is am or pm, and c is the character used to separate the hours, minutes, and seconds. Example: TIME=(12.) would produce a time of the form 'hh.mm.ss xx', which at 08:25:13 pm would appear as '08.25.13 pm'.

ab and c must be specified.

&TIME=(abc)
can be used instead of TIME=(abc).

TIMENS=(ab)
specifies that the current time is to appear in the reformatted OUTFIL output record in the form 'hhmmss' (24-hour time) or 'hhmmss xx' (12-hour time).

If ab is 24, the time is to appear in the form 'hhmmss' (24-hour time) where hh represents the hour (00-23), mm represents the minutes (00-59), and ss represents the seconds (00-59). Example: TIMENS=(24) would produce a time of the form 'hhmmss', which at 08:25:13 pm would appear as '202513'.

If ab is 12, the time is to appear in the form 'hhmmss xx' (12-hour time) where hh represents the hour (01-12), mm represents the minutes (00-59), and ss represents the seconds (00-59). Example: TIMENS=(12) would produce a time of the form 'hhmmss xx', which at 08:25:13 pm would appear as '082513 pm'.

ab must be specified.

&TIMENS=(ab)
can be used instead of TIMENS=(ab).

/.../ or n/
Blank records or a new record. A new output record is to be started with or without intervening blank output records. If /.../ or n/ is specified at the beginning or end of OUTREC, n blank output records are to be produced. If /.../ or n/ is specified in the middle of OUTREC, n-1 blank output records are to be produced (thus, / or 1/ indicates a new output record with no intervening blank output records).

At least one input field or separation field must be specified if you use /.../ or n/. For example, OUTREC=(//) is not allowed, whereas OUTREC=(//X) is allowed.

Either n/ (for example, 5/) or multiple /'s (for example, /////) can be used. n can range from 1 to 255. If n is omitted, 1 is used.

As an example, if you specify:

```
OUTFIL OUTREC=(2/,C'Field 2 contains ',4,3,/,
                  C'Field 1 contains ',1,3)
```

an input record containing:

```
111222
```

would produce the following four output records:

```
Blanks
Blanks
Field 2 contains 222
Field 1 contains 111
```

Note that **four** OUTFIL output records are produced for **each** OUTFIL input record.

p,m,a
specifies that an unedited input field is to appear in the reformatted OUTFIL output record.

p
specifies the first byte of the input field relative to the beginning of the OUTFIL input record. The first data byte of a fixed-length record has relative position 1. The first data byte of a variable-length record has relative position 5, because the first four bytes are occupied by the RDW. All fields must start on a byte boundary, and no field can extend beyond byte 32752. See "OUTFIL statements notes" on page 358 for special rules concerning variable-length records.

m
specifies the length in bytes of the input field.

a
specifies the alignment (displacement) of the input field in the reformatted OUTFIL output record relative to the start of the reformatted OUTFIL output record.

The permissible values of **a** are:

H
Halfword aligned. The displacement (p-1) of the field from the beginning of the reformatted OUTFIL input record, in bytes, is a multiple of 2 (that is, position 1, 3, 5, and so forth).

F
Fullword aligned. The displacement is a multiple of 4 (that is, position 1, 5, 9, and so forth).

D
Doubleword aligned. The displacement is a multiple of 8 (that is, position 1, 9, 17, and so forth).

Alignment can be necessary if, for example, the data is used in a COBOL application program where items are aligned through the SYNCHRONIZED clause. Unused space preceding aligned fields are always padded with binary zeros.

%nn
specifies that an unedited parsed input field is to appear in the reformatted OUTFIL output record. See PARSE for details of parsed fields. See p,m,a for further details. Note that alignment (H, F, D) is not permitted for %nn fields (for example, %nn,F results in an error message and termination).

p
specifies the unedited variable part of the OUTFIL input record (that part beyond the minimum record length) is to appear in the reformatted OUTFIL output record as the last field. p without m can only be used for variable-length records; not for fixed-length records.



Attention: If 1,4,p is specified (only RDW and variable part of record), "null" records containing only an RDW will result if the record length is less than p.

A value must be specified for p that is less than or equal to the minimum OUTFIL input record length plus 1 byte.

p,m,TRAN=keyword
Specifies that an input field is to be translated as indicated by the keyword.

Attention: If TRAN=keyword is used for non-character data (for example, PD), it may have unintended consequences.

p
See p under p,m,a

m
specifies the length in bytes of the numeric field and depends on the keyword used as indicated.

TRAN=keyword
specifies the type of translation to be performed. The keyword can be one of the following:

TRAN=LTOU
translates lowercase EBCDIC letters (that is, a-z) to uppercase EBCDIC letters (that is, A-Z). Other characters are not changed. For example, the characters 'Vicky-123,x' would be replaced by 'VICKY-123,X'.

m can be 1 to 32752. The output length will be m bytes.

TRAN=UTOL
translates uppercase EBCDIC letters (that is, A-Z) to lowercase EBCDIC letters (that is, a-z). Other characters are not changed. For example, the characters 'CARRIE-005, CA' would be replaced by 'carrie-005, ca'.

m can be 1 to 32752. The output length will be m bytes.

TRAN=ALTSEQ
translates characters according to the ALTSEQ translation table in effect. For example, the characters X'5B5C' would be replaced by X'4040' if ALTSEQ CODE=(5B40,5C40) was in effect.

m can be 1 to 32752. The output length will be m bytes.

TRAN=ATOE
translates characters from ASCII to EBCDIC using the default standard TCP/IP service ASCII-to-EBCDIC translation table. For example, with TRAN=ATOE, the ASCII characters aB2 (X'614232') would be replaced by the EBCDIC characters aB2 (X'81C2F2').

m can be 1 to 32752. The output length will be m bytes.

Table 47 on page 256 is the default standard TCP/IP table used for TRAN=ATOE. Each column shows the ASCII (A) character value on the left and its equivalent EBCDIC (E) character value on the right.

Table 47. TRAN=ATOE ASCII-to-EBCDIC translation

Part 1 - TRAN=ATOE ASCII-to-EBCDIC Translation

A	E	A	E	A	E	A	E	A	E	A	E	A	E	A	E
00	00	10	10	20	40	30	F0	40	7C	50	D7	60	79	70	97
01	01	11	11	21	5A	31	F1	41	C1	51	D8	61	81	71	98
02	02	12	12	22	7F	32	F2	42	C2	52	D9	62	82	72	99
03	03	13	13	23	7B	33	F3	43	C3	53	E2	63	83	73	A2
04	37	14	3C	24	5B	34	F4	44	C4	54	E3	64	84	74	A3
05	2D	15	3D	25	6C	35	F5	45	C5	55	E4	65	85	75	A4
06	2E	16	32	26	50	36	F6	46	C6	56	E5	66	86	76	A5
07	2F	17	26	27	7D	37	F7	47	C7	57	E6	67	87	77	A6
08	16	18	18	28	4D	38	F8	48	C8	58	E7	68	88	78	A7
09	05	19	19	29	5D	39	F9	49	C9	59	E8	69	89	79	A8
0A	25	1A	3F	2A	5C	3A	7A	4A	D1	5A	E9	6A	91	7A	A9
0B	0B	1B	27	2B	4E	3B	5E	4B	D2	5B	AD	6B	92	7B	C0
0C	0C	1C	22	2C	6B	3C	4C	4C	D3	5C	E0	6C	93	7C	4F
0D	0D	1D	1D	2D	60	3D	7E	4D	D4	5D	BD	6D	94	7D	D0
0E	0E	1E	35	2E	4B	3E	6E	4E	D5	5E	5F	6E	95	7E	A1
0F	0F	1F	1F	2F	61	3F	6F	4F	D6	5F	6D	6F	96	7F	07

Part 2 - TRAN=ATOE ASCII-to-EBCDIC Translation

Table 47. TRAN=ATOE ASCII-to-EBCDIC translation (continued)

Part 1 - TRAN=ATOE ASCII-to-EBCDIC Translation

```
| A  E| A  E| A  E| A  E| A  E| A  E| A  E| A  E|
|-- --|-- --|-- --|-- --|-- --|-- --|-- --|-- --|
|80 00|90 10|A0 40|B0 F0|C0 7C|D0 D7|E0 79|F0 97|
|81 01|91 11|A1 5A|B1 F1|C1 C1|D1 D8|E1 81|F1 98|
|82 02|92 12|A2 7F|B2 F2|C2 C2|D2 D9|E2 82|F2 99|
|83 03|93 13|A3 7B|B3 F3|C3 C3|D3 E2|E3 83|F3 A2|
|84 37|94 3C|A4 5B|B4 F4|C4 C4|D4 E3|E4 84|F4 A3|
|85 2D|95 3D|A5 6C|B5 F5|C5 C5|D5 E4|E5 85|F5 A4|
|86 2E|96 32|A6 50|B6 F6|C6 C6|D6 E5|E6 86|F6 A5|
|87 2F|97 26|A7 7D|B7 F7|C7 C7|D7 E6|E7 87|F7 A6|
|88 16|98 18|A8 4D|B8 F8|C8 C8|D8 E7|E8 88|F8 A7|
|89 05|99 19|A9 5D|B9 F9|C9 C9|D9 E8|E9 89|F9 A8|
|8A 25|9A 3F|AA 5C|BA 7A|CA D1|DA E9|EA 91|FA A9|
|8B 0B|9B 27|AB 4E|BB 5E|CB D2|DB AD|EB 92|FB C0|
|8C 0C|9C 22|AC 6B|BC 4C|CC D3|DC E0|EC 93|FC 4F|
|8D 0D|9D 1D|AD 60|BD 7E|CD D4|DD BD|ED 94|FD D0|
|8E 0E|9E 35|AE 4B|BE 6E|CE D5|DE 5F|EE 95|FE A1|
|8F 0F|9F 1F|AF 61|BF 6F|CF D6|DF 6D|EF 96|FF 07|
```

TRAN=ETOA

translates characters from EBCDIC to ASCII using the default standard TCP/IP service EBCDIC-to-ASCII translation table. For example, with TRAN=ETOA, the EBCDIC characters aB2 (X'81C2F2') would be replaced by the ASCII characters aB2 (X'614232').

m can be 1 to 32752. The output length will be m bytes.

Table 48 on page 257is the default standard TCP/IP table used for TRAN=ETOA. Each column shows the EBCDIC (E) character value on the left and its equivalent ASCII (A) character value on the right.

Table 48. TRAN=ETOA EBCDIC-to-ASCII translation

Part 1 - TRAN=ETOA EBCDIC-to-ASCII translation

```
| E  A| E  A| E  A| E  A| E  A| E  A| E  A| E  A|
|-- --|-- --|-- --|-- --|-- --|-- --|-- --|-- --|
|00 00|10 10|20 1A|30 1A|40 20|50 26|60 2D|70 D7|
|01 01|11 11|21 1A|31 1A|41 A6|51 A9|61 2F|71 88|
|02 02|12 12|22 1C|32 16|42 E1|52 AA|62 DF|72 94|
|03 03|13 13|23 1A|33 1A|43 80|53 9C|63 DC|73 B0|
|04 1A|14 1A|24 1A|34 1A|44 EB|54 DB|64 9A|74 B1|
|05 09|15 0A|25 0A|35 1E|45 90|55 A5|65 DD|75 B2|
|06 1A|16 08|26 17|36 1A|46 9F|56 99|66 DE|76 FC|
|07 7F|17 1A|27 1B|37 04|47 E2|57 E3|67 98|77 D6|
|08 1A|18 18|28 1A|38 1A|48 AB|58 A8|68 9D|78 FB|
|09 1A|19 19|29 1A|39 1A|49 8B|59 9E|69 AC|79 60|
|0A 1A|1A 1A|2A 1A|3A 1A|4A 9B|5A 21|6A BA|7A 3A|
|0B 0B|1B 1A|2B 1A|3B 1A|4B 2E|5B 24|6B 2C|7B 23|
|0C 0C|1C 1C|2C 1A|3C 14|4C 3C|5C 2A|6C 25|7C 40|
|0D 0D|1D 1D|2D 05|3D 15|4D 28|5D 29|6D 5F|7D 27|
|0E 0E|1E 1E|2E 06|3E 1A|4E 2B|5E 3B|6E 3E|7E 3D|
|0F 0F|1F 1F|2F 07|3F 1A|4F 7C|5F 5E|6F 3F|7F 22|
```

Part 2 - TRAN=ETOA EBCDIC-to-ASCII translation

Table 48. TRAN=ETOA EBCDIC-to-ASCII translation (continued)

Part 1 - TRAN=ETOA EBCDIC-to-ASCII translation

```
| E  A| E  A| E  A| E  A| E  A| E  A| E  A| E  A|
|-- --|-- --|-- --|-- --|-- --|-- --|-- --|-- --|
|80 F8|90 8C|A0 C8|B0 B5|C0 7B|D0 7D|E0 5C|F0 30|
|81 61|91 6A|A1 7E|B1 B6|C1 41|D1 4A|E1 E7|F1 31|
|82 62|92 6B|A2 73|B2 FD|C2 42|D2 4B|E2 53|F2 32|
|83 63|93 6C|A3 74|B3 B7|C3 43|D3 4C|E3 54|F3 33|
|84 64|94 6D|A4 75|B4 B8|C4 44|D4 4D|E4 55|F4 34|
|85 65|95 6E|A5 76|B5 B9|C5 45|D5 4E|E5 56|F5 35|
|86 66|96 6F|A6 77|B6 E6|C6 46|D6 4F|E6 57|F6 36|
|87 67|97 70|A7 78|B7 BB|C7 47|D7 50|E7 58|F7 37|
|88 68|98 71|A8 79|B8 BC|C8 48|D8 51|E8 59|F8 38|
|89 69|99 72|A9 7A|B9 BD|C9 49|D9 52|E9 5A|F9 39|
|8A 96|9A 97|AA EF|BA 8D|CA CB|DA A1|EA A0|FA B3|
|8B A4|9B 87|AB C0|BB D9|CB CA|DB AD|EB 85|FB F7|
|8C F3|9C CE|AC DA|BC BF|CC BE|DC F5|EC 8E|FC F0|
|8D AF|9D 93|AD 5B|BD 5D|CD E8|DD F4|ED E9|FD FA|
|8E AE|9E F1|AE F2|BE D8|CE EC|DE A3|EE E4|FE A7|
|8F C5|9F FE|AF F9|BF C4|CF ED|DF 8F|EF D1|FF FF|
```

TRAN=HEX
translates binary values to their equivalent EBCDIC hexadecimal values. For example, with TRAN=HEX, X'C1F1' (C'A1') would be replaced by C'C1F1'.

m can be 1 to 16376. The output length will be m*2 bytes

TRAN=UNHEX
translates EBCDIC hexadecimal values to their equivalent binary values. For example, with TRAN=UNHEX, C'C1F1' would be replaced by X'C1F1' (C'A1').

m can be 1 to 32752. The output length will be (m+1)/2 bytes.

If m is odd, the last nibble will be 0. As an example, C'C1F1F' would translate to X'C1F1F0' (C'A10').

If an input character is not 0-9 or A-F, the equivalent nibble will be set to 0. As an example, C'G2' will be translated to X'02'.

TRAN=BIT
translates binary values to their equivalent EBCDIC bit values. For example, with TRAN=BIT, X'C1F1' (C'A1') would be replaced by C'1100000111110001'.

m can be 1 to 4094. The output length will be m*8 bytes

TRAN=UNBIT
translates EBCDIC bit values to their equivalent binary values. For example, with TRAN=UNBIT, C'1100000111110001' would be replaced by X'C1F1' (C'A1').

m can be 1 to 32752. The output length will be (m+7)/8 bytes

If m is not a multiple of 8, the missing bits on the right will be set to 0. As an example, C'1100000111111' would translate to X'C1F8'.

If an input character is not 0 or 1, the equivalent bit will be set to 0. As an example, C'11100005' will be translated to X'E0'.

Sample Syntax:

```
OUTFIL FNAMES=OUT1,
  BUILD=(5,6,TRAN=UNHEX,2X,20,15,TRAN=UTOL)
```

%nn,TRAN=keyword
Specifies that a parsed input field is to be translated as indicated by the keyword. See PARSE for details of PARSED fields. See p,m,TRAN=keyword for details of the translation functions.

p,TRAN=keyword
Specifies that the variable part of the input record is to be translated as indicated by the keyword. p,TRAN=keyword must be the last item and can only be used for variable-length records; not for fixed-length records. A value must be specified for p that is less than or equal to the minimum input record length plus 1 byte. See p,m,TRAN=keyword for details of the translation functions.



Attention: If 1,4,p,TRAN=keyword is specified (only RDW and variable part of record), "null" records containing only an RDW will result if the record length is less than p.

Sample Syntax:

```
OUTFIL FNAMES=OUT2,BUILD=(1,4,5,TRAN=ETOA)
```

p,m,HEX
Can be used instead of p,m,TRAN=HEX. See p,m,TRAN=HEX for details.

%nn,HEX
Can be used instead of %nn,TRAN=HEX. See %nn,TRAN=HEX for details.

p,HEX
Can be used instead of p,TRAN=HEX. See p,TRAN=HEX for details.

p,m,f,edit or (p,m,f),edit
specifies that an edited numeric input field is to appear in the reformatted OUTFIL output record. You can edit BI, FI, PD, PD0, ZD, FL,CSF, FS, UFF, SFF, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3 or TM4 fields using either pre-defined edit masks (M0-M26) or specific edit patterns you define. You can control the way the edited fields look with respect to length, leading or suppressed zeros, thousands separators, decimal points, leading and trailing positive and negative signs, and so on.

p
See p under p,m,a.

m
specifies the length in bytes of the numeric field. The length must include the sign, if the data is signed. See Table 49 on page 259 for permissible length values.

f
specifies the format of the numeric field:

Table 49. Edit field formats and lengths

Format Code	Length	Description
BI	1 to 8 bytes	Unsigned binary
FI	1 to 8 bytes	Signed fixed-point
PD	1 to 16 bytes	Signed packed decimal
PD0	2 to 8 bytes	Packed decimal with sign and first digit ignored
ZD	1 to 31 bytes	Signed zoned decimal
FL	4 or 8 bytes	Signed hexadecimal floating-point converted to signed integer
CSF or FS	1 to 32 bytes (31 digit limit)	Signed numeric with optional leading floating sign
UFF	1 to 44 bytes (31 digit limit)	Unsigned free form numeric
SFF	1 to 44 bytes (31 digit limit)	Signed free form numeric
DT1	4 bytes	SMF date interpreted as Z'yyyymmdd'

Table 49. Edit field formats and lengths (continued)

Format Code	Length	Description
DT2	4 bytes	SMF date interpreted as Z'yyyymm'
DT3	4 bytes	SMF date interpreted as Z'yyyyddd'
DC1	8 bytes	TOD date interpreted as Z'yyyymmdd'
DC2	8 bytes	TOD date interpreted as Z'yyyymm'
DC3	8 bytes	TOD date interpreted as Z'yyyyddd'
DE1	8 bytes	ETOD date interpreted as Z'yyyymmdd'
DE2	8 bytes	ETOD date interpreted as Z'yyyymm'
DE3	8 bytes	ETOD date interpreted as Z'yyyyddd'
TM1	4 bytes	SMF time interpreted as Z'hhmmss'
TM2	4 bytes	SMF time interpreted as Z'hhmm'
TM3	4 bytes	SMF time interpreted as Z'hh'
TM4	4 bytes	SMF time interpreted as Z'hhmmssxx'
TC1	8 bytes	TOD time interpreted as Z'hhmmss'
TC2	8 bytes	TOD time interpreted as Z'hhmm'
TC3	8 bytes	TOD time interpreted as Z'hh'
TC4	8 bytes	TOD time interpreted as Z'hhmmssxx'
TE1	8 bytes	ETOD time interpreted as Z'hhmmss'
TE2	8 bytes	ETOD time interpreted as Z'hhmm'
TE3	8 bytes	ETOD time interpreted as Z'hh'
TE4	8 bytes	ETOD time interpreted as Z'hhmmssxx'
Note: See Appendix C, "Data Format Descriptions" for detailed format descriptions.		

For a CSF or FS format field:

- A maximum of 31 digits is allowed. If a CSF or FS value with 32 digits is found, the leftmost digit will be treated as a positive sign indicator.

For a UFF or SFF format field:

- A maximum of 31 digits is allowed. If a UFF or SFF value with more than 31 digits is found, the leftmost digits will be ignored.

For a ZD or PD format field:

- An invalid digit results in a data exception (0C7 ABEND) or incorrect numeric output; A-F are invalid digits. ICETOOL's VERIFY or DISPLAY operator can be used to identify decimal values with invalid digits.
- A value is treated as positive if its sign is F, E, C, A, 8, 6, 4, 2, or 0.
- A value is treated as negative if its sign is D, B, 9, 7, 5, 3, or 1.

For a PD0 format field:

- The first digit is ignored.
- An invalid digit other than the first results in a data exception (0C7 ABEND) or incorrect numeric output; A-F are invalid digits.
- The sign is ignored and the value is treated as positive.

For an FL format field:

- The normalized or unnormalized FL (hexadecimal floating-point) value is converted to a signed integer in the range -9223372036854775808 to 9223372036854775807. The fractional part of the FL value is lost, and in some cases the signed integer may be one of a number of possible signed integers for the FL value depending on its precision. Converted values less than -9223372036854775808 are set to -9223372036854775808. Converted values greater than 9223372036854775807 are set to 9223372036854775807.
- If you are not running in z/Architecture mode, specifying an FL format field results in an error message and termination.

For a DT1, DT2, or DT3 format field:

- An invalid SMF date can result in a data exception (0C7 ABEND) or an incorrect ZD date.
- SMF date values are always treated as positive.

For a DC1, DC2, DC3, DE1, DE2, or DE3 format field:

- TOD and ETOD date values are always treated as positive.

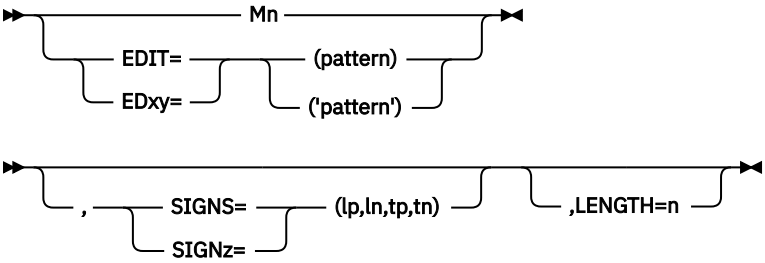
For a TM1, TM2, TM3, or TM4 format field:

- An invalid SMF time can result in an incorrect ZD time.
- SMF time values are always treated as positive.

For a TC1, TC2, TC3, TC4, TE1, TE2, TE3, or TE4 format field:

- TOD and ETOD time values are always treated as positive.

edit



Specifies how the numeric field is to be edited for output. If an Mn, EDIT, or EDxy parameter is not specified:

- a DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3, or TM4 field is edited using the M11 edit mask.
- a BI, FI, PD, PD0, ZD, FL, CSF, FS, UFF, or SFF field is edited using the M0 edit mask.

Mn
specifies one of twenty-seven pre-defined edit masks (M0-M26) for presenting numeric data. If these pre-defined edit masks are not suitable for presenting your numeric data, the EDIT parameter gives you the flexibility to define your own edit patterns.

The twenty-seven pre-defined edit masks can be represented as follows:

Table 50. Edit mask patterns

Mask	Pattern	Examples	
		Value	**Result**
M0	IIIIIIIIIIIIIIIIIIIIIIIIIIIITS	+01234	1234
		-00001	1-
M1	TTTTTTTTTTTTTTTTTTTTTTTTTTTTTTS	-00123	00123-
		+00123	00123
M2	II,III,III,III,III,III,III,III,III,IIT.TTS	+123450	1,234.50
		-000020	0.20-
M3	II,III,III,III,III,III,III,III,III,IIT.TTCR	-001234	12.34CR
		+123456	1,234.56
M4	SII,III,III,III,III,III,III,III,III,IIT.TT	+0123456	+1,234.56
		-1234567	-12,345.67
M5	SII,III,III,III,III,III,III,III,III,IIT.TTS	-001234	(12.34)
		+123450	1,234.50
M6	III-TTT-TTTT	00123456	012-3456
		12345678	1-234-56788
M7	TTT-TT-TTTT	00123456	000-12-3456
		12345678	012-34-5678
M8	IT:TT:TT	030553	3:05:53
		121736	12:17:36
M9	IT/TT/TT	123004	12/30/04
		083104	8/31/04
M10	IIIIIIIIIIIIIIIIIIIIIIIIIIIIT	01234	1234
		00000	0
M11	TTTTTTTTTTTTTTTTTTTTTTTTTTTTTT	00010	00010
		01234	01234
M12	SI,III,III,III,III,III,III,III,III,III,IIT	+1234567	1,234,567
		-0012345	-12,345
M13	SI.III.III.III.III.III.III.III.III.III.IIT	+1234567	1.234.567
		-0012345	-12.345
M14	SI III III III III III III III III III IITS	+1234567	1 234 567
		-0012345	(12 345)
M15	I III III III III III III III III III IITS	+1234567	1 234 567
		-0012345	12 345-

Table 50. Edit mask patterns (continued)

Mask	Pattern	Examples	
		Value	Result
M16	SI III III III III III III III III III IIT	+1234567	1 234 567
		-0012345	-12 345
M17	SI'III'III'III'III'III'III'III'III'III'IIT	+1234567	1'234'567
		-0012345	-12'345
M18	SII,III,III,III,III,III,III,III,III,IIT.TT	+0123456	1,234.56
		-1234567	-12,345.67
M19	SII.III.III.III.III.III.III.III.IIT,TT	+0123456	1.234,56
		-1234567	-12.345,67
M20	SI III III III III III III III III IIT,TTS	+0123456	1 234,56
		-1234567	(12 345,67)
M21	II III III III III III III III III IIT,TTS	+0123456	1 234,567
		-1234567	12 345,67-
M22	SI III III III III IIII III III III IIT,TT	+0123456	1 234,56
		-1234567	-12 345,67
M23	SII'III'III'III'III'III'III'III'IIT.TT	+0123456	1'234.56
		-1234567	-12'345.67
M24	SII'III'III'III'III'III'III'III'IIT,TT	+0123456	1'234,56
		-1234567	-12'345,67
M25	SIIIIIIIIIIIIIIIIIIIIIIIIIIIIIT	+01234	1234
		-00001	-1
M26	STTTTTTTTTTTTTTTTTTTTTTTTTTTTTT	1234	+01234
		-1	-00001

The elements used in the representation of the edit masks in Table 50 on page 262 are as follows:

- **I** indicates a leading insignificant digit. If zero, this digit will not be shown.
- **T** indicates a significant digit. If zero, this digit will be shown.
- **CR** (in M3) is printed to the right of the digits if the value is negative; otherwise, two blanks are printed to the right of the digits.
- **S** indicates a sign. If it appears as the first character in the pattern, it is a leading sign. If it appears as the last character in the pattern, it is a trailing sign. If S appears as both the first and last characters in a pattern (example: M5), the first character is a leading sign and the last character is a trailing sign. Four different sign values are used: leading positive sign (lp), leading negative sign (ln), trailing positive sign (tp) and trailing negative sign (tn). Their applicable values for the Mn edit masks are:

Table 51. Edit mask signs

Mask	lp	ln	tp	tn
M0	none	none	blank	-

Table 51. Edit mask signs (continued)

Mask	lp	ln	tp	tn
M1	none	none	blank	-
M2	none	none	blank	-
M3	none	none	none	none
M4	+	-	none	none
M5	blank	(	blank	)
M6	none	none	none	none
M7	none	none	none	none
M8	none	none	none	none
M9	none	none	none	none
M10	none	none	none	none
M11	none	none	none	none
M12	blank	-	none	none
M13	blank	-	none	none
M14	blank	(	blank	)
M15	none	none	blank	-
M16	blank	-	none	none
M17	blank	-	none	none
M18	blank	-	none	none
M19	blank	-	none	none
M20	blank	(	blank	)
M21	none	none	blank	-
M22	blank	-	none	none
M23	blank	-	none	none
M24	blank	-	none	none
M25	blank	-	none	none
M26	+	-	none	none

- any other character (for example, /) will be printed as shown, subject to certain rules to be subsequently discussed.

The implied length of the edited output field depends on the number of digits and characters needed for the pattern of the particular edit mask used. The LENGTH parameter can be used to change the implied length of the edited output field.

The number of digits needed depends on the format and length of the numeric field as follows:

Table 52. Digits needed for numeric fields

Format	Input Length	Digits Needed
ZD	m	m

Table 52. Digits needed for numeric fields (continued)

Format	Input Length	Digits Needed
PD	m	2m-1
PD0	m	2m-2
BI, FI	1	3
BI, FI	2	5
BI, FI	3	8
BI, FI	4	10
BI, FI	5	13
BI, FI	6	15
BI, FI	7	17
BI, FI	8	20
FL	4 or 8	20
CSF or FS	32	31
CSF or FS	m (less than 32)	m
UFF, SFF	32 to 44	31
UFF, SFF	m (less than 32)	m

The length of the output field can be represented as follows for each pattern, where d is the number of digits needed, as shown in Table 52 on page 264, and the result is rounded down to the nearest integer:

Table 53. Edit mask output field lengths

Mask	Output Field Length	Example	
		Input (f,m)	Output Length
M0	d + 1	ZD,3	4
M1	d + 1	PD,10	20
M2	d + 1 + d/3	BI,4	14
M3	d + 2 + d/3	UFF,20	28
M4	d + 1 + d/3	PD,8	21
M5	d + 2 + d/3	FI,3	12
M6	12	ZD,10	12
M7	11	PD,5	11
M8	8	ZD,6	8
M9	8	PD,4	8
M10	d	BI,6	15
M11	d	PD,5	9
M12	d + 1 + (d - 1)/3	PD,3	7
M13	d + 1 + (d - 1)/3	FS,5	7

Table 53. Edit mask output field lengths (continued)

Mask	Output Field Length	Example	
		Input (f,m)	Output Length
M14	d + 2 + (d - 1)/3	ZD,5	8
M15	d + 1 + (d - 1)/3	FI,3	11
M16	d + 1 + (d - 1)/3	SFF,41	42
M17	d + 1 + (d - 1)/3	FI,4	14
M18	d + 1 + d/3	BI,4	14
M19	d + 1 + d/3	PD,8	21
M20	d + 2 + d/3	FI,3	12
M21	d + 1 + d/3	ZD,3	5
M22	d + 1 + d/3	BI,2	7
M23	d + 1 + d/3	PD,6	15
M24	d + 1 + d/3	ZD,21	29
M25	d + 1	CSF,16	17
M26	d + 1	FL,4	21

To illustrate conceptually how DFSORT produces the edited output from the numeric value, consider the following example:

```
OUTFIL OUTREC=(5,7,ZD,M5)

with ZD values of C'0123456'(+0123456)
and C'000302J' (-0003021)
```

As shown in the preceding tables, it is determined that:

- The general pattern for M5 is: SI,III,...,IIT.TTS
- The signs to be used are blank for leading positive sign, C'(' for leading negative sign, blank for trailing positive sign and C')' for trailing negative sign
- The number of digits needed is 7
- The length of the output field is 11 (7 + 2 + 7/3)
- The specific pattern for the output field is thus: C'SII,IIT.TTS'

The digits of C'0123456' are mapped to the pattern, resulting in C'S01,234.56S'. Because the value is positive, the leading sign is replaced with blank and the trailing sign is replaced with blank, resulting in C' 01,234.56 '. Finally, all digits before the first non-zero digit (1 in this case), are replaced with blanks, resulting in the final output of C' 1,234.56 '.

The digits of C'000302J' are mapped to the pattern, resulting in C'S00,030.21S'. Because the value is negative, the leading sign is replaced with C'(' and the trailing sign is replaced with C')' resulting in C'(00,030.21)'. All digits before the first non-zero digit (3 in this case), are replaced with blanks, resulting in C'(30.21)'. Finally, the leading sign is "floated" to the right, next to the first non-zero digit, resulting in the final output of C' (30.21)'.

To state the rules in more general terms, the steps DFSORT takes conceptually to produce the edited output from the numeric value are as follows:

- Determine the specific pattern and its length, using the preceding tables.
- Map the digits of the numeric value to the pattern.

- If the value is positive, replace the leading and trailing signs (if any) with the characters for positive values shown in Table 51 on page 263. Otherwise, replace the leading and trailing signs (if any) with the characters for negative values shown in that same table.
- Replace all digits before the first non-zero (I) or significant digit (T) with blanks.
- Float the leading sign (if any) to the right, next to the first non-zero (I) or significant digit (T).

The following additional rule applies to edit masks:

- The specific pattern is determined from the general pattern by including signs, the rightmost digits needed as determined from the input format and length, and any characters in between those rightmost digits. This may unintentionally truncate significant digits (T). As an example, if you specify 5,1,ZD,M4, the length of the output field will be 2 (1 + 1 + 1/3). The general pattern for M4 is SI,III,...,IIT.TT, but the specific pattern will be ST (the leading sign and the rightmost digit).

EDIT
specifies an edit pattern for presenting numeric data. If the pre-defined edit masks (M0-M26) are not suitable for presenting your numeric data, EDIT gives you the flexibility to define your own edit patterns. The elements you use to specify the pattern are the same as those used for the edit masks: I, T, S, and printable characters. However, S will not be recognized as a sign indicator unless the SIGNS parameter is also specified.

pattern
specifies the edit pattern to be used. Not enclosing the pattern in single apostrophes restricts you from specifying the following characters in the pattern: blank, apostrophe, unbalanced left or right parentheses, and hexadecimal digits 20, 21, and 22. For example, EDIT=((IIT.TT)) is valid, whereas EDIT=(C)ITT.TT), EDIT=(I / T) and EDIT=(S'II.T) are not.

The maximum number of digits (I's and T's) you specify in the pattern must not exceed 31. The maximum length of the pattern must not exceed 44 characters.

'pattern'
specifies the edit pattern to be used. Enclosing the pattern in single apostrophes allows you to specify any character in the pattern except hexadecimal digits 20, 21, or 22. If you want to include a single apostrophe in the pattern, you must specify it as two single apostrophes, which will be counted as a single character in the pattern. As examples, EDIT=('C)ITT.TT'), EDIT=('I / T'), and EDIT=('S'II.T') are all valid.

The maximum number of digits (I's and T's) you specify in the pattern must not exceed 31. The maximum length of the pattern must not exceed 44 characters.

The implied length of the edited output field is the same as the length of the pattern. The LENGTH parameter can be used to change the implied length of the edited output field.

To illustrate conceptually how DFSORT produces the edited output from the numeric value, consider the following example:

```
OUTFIL OUTREC=(1,5,ZD,EDIT=(**I/ITTTCR))

with ZD values of C'01230'(+1230)
and C'0004J' (-41)
```

The digits of C'01230' are mapped to the pattern, resulting in C'**0/1230CR'. Because the value is positive, the characters (C'CR') to the right of the last digit are replaced with blanks, resulting in C'**0/1230 '. All digits before the first non-zero digit (1 in this case) are replaced with blanks, resulting in C'** /1230 '. Finally, all characters before the first digit in the pattern are floated to the right, next to the first non-zero digit, resulting in C' **1230 '.

The digits of C'0004J' are mapped to the pattern, resulting in C'**0/0041CR'. Because the value is negative, the characters (C'CR') to the right of the last digit are kept. All digits before the first T digit are replaced with blanks, resulting in C'** / 041CR'. Finally, all characters before the first digit in the pattern are floated to the right, next to the first non-zero digit, resulting in C' **041CR'.

In general terms, the steps DFSORT takes conceptually to produce the edited output from the numeric value are as follows:

- Map the digits of the numeric value to the pattern, padding on the left with zeros, if necessary.
- If the value is positive, replace the leading and trailing signs (if any) with the characters for positive values specified by the SIGNS parameter and replace any characters between the last digit and the trailing sign (if any) with blanks. Otherwise, replace the leading and trailing signs (if any) with the characters for negative values specified by the SIGNS parameter and keep any characters between the last digit and the trailing sign (if any).
- Replace all digits before the first non-zero (I), significant digit (T), or significant decimal point with blanks.
- Float all characters (if any) before the first digit in the pattern to the right, next to the first non-zero (I) or significant digit (T).

The following additional rules apply to edit patterns:

- An insignificant digit (I) after a significant digit (T) or significant decimal point is treated as a significant digit.
- If SIGNS is specified, an S in the first or last character of the pattern is treated as a sign; an S anywhere else in the pattern is treated as the letter S. If SIGNS is not specified, an S anywhere in the pattern is treated as the letter S.
- If the pattern contains fewer digits than the value, the leftmost digits of the value will be lost, intentionally or unintentionally. As an example, if you specify 5,5,ZD,EDIT=(IIT) for a value of C'12345', the result will be C'345'. As another example, if you specify 1,6,ZD,EDIT=($IIT.T) for a value of C'100345', the result will be C' $34.5'.

EDxy
specifies an edit pattern for presenting numeric data. EDxy is a special variation of EDIT that allows other characters to be substituted for I and T in the pattern. For example, if you use EDAB instead of EDIT, you must use A in the pattern instead of I and use B instead of T to represent digits. x and y must not be the same character. If SIGNS is specified, x and y must not be S. If SIGNz is specified, x and y must not be the same character as z. You can select x and y from: A-Z, #, $, @, and 0-9.

SIGNS
specifies the sign values to be used when editing numeric values according to the edit mask (Mn) or pattern (EDIT or EDxy). You can specify any or all of the four sign values. Any value not specified must be represented by a comma. Blank will be used for any sign value you do not specify. As examples, SIGNS=(+,-) specifies + for lp, - for ln, blank for tp, and blank for tn; SIGNS=(,,+,-) specifies blank for lp, blank for ln, + for tp, and - for tn.

lp
specifies the value for the leading positive sign. If an S is specified as the first character of the edit mask or pattern and the value is positive, the lp value will be used as the leading sign.

ln
specifies the value for the leading negative sign. If an S is specified as the first character of the edit mask or pattern and the value is negative, the ln value will be used as the leading sign.

tp
specifies the value for the trailing positive sign. If an S is specified as the last character of the edit mask or pattern and the value is positive, the tp value will be used as the trailing sign.

tn
specifies the value for the trailing negative sign. If an S is specified as the last character of the edit mask or pattern and the value is negative, the tn value will be used as the trailing sign.

If you want to use any of the following characters as sign values, you must enclose them in single apostrophes: comma, blank, or unbalanced left or right parentheses. A single apostrophe must be specified as four single apostrophes (that is, two single apostrophes enclosed in single apostrophes).

A semicolon cannot be substituted for a comma as the delimiter between sign characters.

SIGNz
specifies the sign values to be used when editing numeric values according to the edit pattern (EDIT or EDxy). SIGNz is a special variation of SIGNS which allows another character to be substituted for S

in the pattern. For example, if you use SIGNX instead of SIGNS, you must use X in the pattern instead of S to identify a sign. If EDIT is specified, z must not be I or T. If EDxy is specified, z must not be the same character as either x or y. You can select z from: A-Z, #, $, @, and 0-9.

LENGTH
specifies the length of the edited output field. If the implied length of the edited output field produced using an edit mask or edit pattern is not suitable for presenting your numeric data, LENGTH can be used to make the edited output field shorter or longer.

n
specifies the length of the edited output field. The value for n must be between 1 and 44.

LENGTH does not change the pattern used, only the length of the resulting edited output field. For example, as discussed previously for Mn, if you specify:

```
OUTFIL OUTREC=(5,1,ZD,M4)
```

the pattern will be C'ST' rather than C'ST.TT' because the digit length is 1. Specifying:

```
OUTFIL OUTREC=(5,1,ZD,M4,LENGTH=5)
```

will change the pattern to C' ST', not to C'ST.TT'.

If you specify a value for n that is shorter than the implied length, truncation will occur on the left after editing. For example, if you specify:

```
OUTFIL OUTREC=(1,5,ZD,EDIT=($IIT.TT),LENGTH=5)
```

with a value of C'12345', editing according to the specified $IIT.TT pattern will produce C'$123.45', but the specified length of 5 will truncate this to C'23.45'.

If you specify a value for n that is longer than the implied length, padding on the left with blanks will occur after editing. For example, if you specify:

```
OUTFIL OUTREC=(1,5,ZD,EDIT=($IIT.TT),LENGTH=10)
```

with a value of C'12345', editing according to the specified $IIT.TT pattern will produce C'$123.45', but the specified length of 10 will pad this to C' $123.45'.

Sample Syntax:

```
OUTFIL FNAMES=OUT1,OUTREC=(5:21,19,ZD,M19,35:46,5,ZD,M13)
OUTFIL FILES=1,OUTREC=(4,8,BI,C' * ',13,8,BI,80:X),
   ENDREC=10,OMIT=(4,8,BI,EQ,13,8,BI)
OUTFIL FILES=(2,3),
  OUTREC=(11:55,6,FS,SIGNS=(,,+,-),LENGTH=10,
          31:(41,4,PD),EDIT=(**II,IIT.TTXS),SIGNS=(,,+,-))
```

%nn,f,edit or (%nn,f),edit
specifies that an edited numeric parsed input field is to appear in the reformatted OUTFIL output record. See PARSE for details of parsed fields. See p,m,f,edit or (p,m,f),edit for further details.

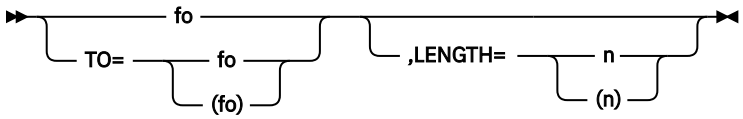
p,m,f,to or (p,m,f),to
specifies that a converted numeric input field is to appear in the reformatted OUTFIL output record. You can convert BI, FI, PD, PD0, ZD, FL, CSF, FS, UFF, SFF, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3, or TM4 fields to BI, FI, PD, PDC, PDF, ZD, ZDF, ZDC, or CSF/FS fields.

p
See p under p,m,f,edit.

m
See m under p,m,f,edit.

f
See f under p,m,f,edit.

to



specifies how the numeric field is to be converted for output.

fo

specifies the format for the output field, which can be BI, FI, PD, PDC, PDF, ZD, ZDF, ZDC, CSF or FS. Any one of these output field formats (fo) can be used with any one of the input field formats (f).

If you do not specify the LENGTH parameter, DFSORT will determine the implied length of the output field from the length (m) and format (f) of the input field and the format (fo) of the output field. The implied length of the output field can be represented as follows for each output format, where d is the number of digits needed for the input field as shown in Table 52 on page 264, and the result is rounded down to the nearest integer:

Table 54. To output field lengths

Output Format	Output Length	Example Input (f,m)	Example Output Length
BI with d <= 9	4	FS,9	4
BI with d > 9	8	FS,10	8
FI with d <= 9	4	ZD,7	4
FI with d > 9	8	ZD,12	8
PD, PDC, or PDF	d/2 + 1	BI,4	6
ZD, ZDF, or ZDC	d	PD,9	17
CSF or FS	d + 1	FI,4	21

For ZD or ZDF output, F is used as the positive sign and D is used as the negative sign. For ZDC output, C is used as the positive sign.

For PD or PDC output, C is used as the positive sign and D is used as the negative sign. For PDF output, F is used as the positive sign.

For CSF or FS output, blank is used as the positive sign, - is used as the negative sign and leading zeros are suppressed.

For ZD, ZDF, ZDC, PD, PDC, PDF, CSF, or FS output, the maximum output value is 9999999999999999999999999999999 (31 digits) and the minimum output value is -9999999999999999999999999999999 (31 digits), which correspond to the maximum and minimum input values, respectively.

For BI output:

- An input value greater than 18446744073709551615 (X'FFFFFFFFFFFFFFFF') produces an output value of 18446744073709551615 (X'FFFFFFFFFFFFFFFF').
- An input value less than zero produces an absolute output value. For example, an input value of P'-5000' produces a BI output value of 5000 (X'1388').

For FI output, an input value greater than 9223372036854775807 (X'7FFFFFFFFFFFFFFF') produces an output value of 9223372036854775807 (X'7FFFFFFFFFFFFFFF'), and an input value less than -9223372036854775808 (X'8000000000000000') produces an output value of -9223372036854775808 (X'8000000000000000').

fo, TO=fo and TO=(fo) are interchangeable except that:

- fo must be specified before the LENGTH parameter whereas TO can be specified before or after the LENGTH parameter.
- TO=fo or TO=(fo) should be used after a symbol rather than fo to prevent the misinterpretation of fo as f. See the discussion of OUTFIL OUTREC in Chapter 7 for details.

LENGTH
specifies the length of the converted output field. If the implied length of the output field is not suitable, LENGTH can be used to make the output field shorter or longer.

n
specifies the length of the converted output field. The value for n must be between 1 and 44.

If you specify a value for n that is shorter than the implied length, truncation on the left will occur after conversion. For example, if you specify:

```
OUTFIL OUTREC=(1,8,ZD,TO=PD,LENGTH=3)
```

with values of ZL8'-12345678' (X'F1F2F3F4F5F6F7D8') and ZL8'58' (X'F0F0F0F0F0F0F5F8'), conversion with the implied length (5) will produce PL5'-12345678' (X'012345678D') and PL5'58' (X'0000000058C'). The specified length of 3 will then result in truncation to PL3'-45678' (X'45678D') and PL3'58' (X'00058C').

If you specify a value for n that is longer than the implied length, padding on the left will occur after conversion using:

- Blanks for CSF and FS output values
- Character zeros for ZD output values
- Binary zeros for PD and BI output values
- Binary zeros for positive FI output values
- Binary ones for negative FI output values

For example, if you specify:

```
OUTFIL OUTREC=(1,4,ZD,TO=FI,LENGTH=6)
```

with values of ZL4'-1234' (X'F1F2F3D4') and ZL4'58' (X'F0F0F5F8'), conversion with the implied length (4) will produce FL4'-1234' (X'FFFFFB2E') and FL4'58' (X'000004D2'). The specified length of 6 will then result in padding to FL6'-1234' (X'FFFFFFFFFB2E') and FL6'58' (X'00000000003A').

Sample Syntax:

```
OUTFIL OUTREC=((21,5,ZD),PD,X,8,4,ZD,TO=FI,LENGTH=2)
```

%nn,f,to or (%nn,f),to
specifies that a converted numeric parsed input field is to appear in the reformatted OUTFIL output record. See PARSE for details of parsed fields. See p,m,f,to or (p,m,f),to for further details.

deccon,edit or (deccon),edit
specifies that an edited decimal constant is to appear in the reformatted OUTFIL output record. The decimal constant must be in the form +n or -n where n is 1 to 31 decimal digits. The sign (+ or -) must be specified. A decimal constant produces a signed, 31-digit zoned decimal (ZD) result to be edited as specified. If an Mn, EDIT, or EDxy parameter is not specified, the decimal constant is edited using the M0 edit mask.

The default number of digits (d) used for editing is 15 for a decimal constant with 1 to 15 significant digits, or 31 for a decimal constant with 16 to 31 significant digits. If EDIT or EDxy is specified, the number of digits in the pattern (I's and T's) is used.

See edit under p,m,f,edit for further details on the edit fields you can use.

Sample Syntax:

```
OUTFIL OUTREC=(5,8,+4096,2X,-17,M18,LENGTH=7,2X,
  (+2000000),EDIT=(STTTTT.TT),SIGNS=(+))
```

deccon,to or (deccon),to

specifies that a converted decimal constant is to appear in the reformatted OUTFIL output record. The decimal constant must be in the form +n or -n where n is 1 to 31 decimal digits. The sign (+ or -) must be specified. A decimal constant produces a signed, 31-digit zoned decimal (ZD) result to be converted as specified.

The default number of digits (d) used for conversion is 15 for a decimal constant with 1 to 15 significant digits, or 31 for a decimal constant with 16 to 31 significant digits.

See to under p,m,f,to for further details on the to fields you can use.

Sample Syntax:

```
OUTFIL FNAMES=OUT1,
  OUTREC=(6:+0,TO=PD,LENGTH=6,+0,TO=PD,LENGTH=6,/,
         6:(-4096),ZD,LENGTH=12)
```

arexp,edit or (arexp),edit

Specifies that the edited result of an arithmetic expression is to appear in the reformatted OUTFIL output record. An arithmetic expression takes the form:

```
term,operator,term<,operator,...>
```

where:

- **term** is a field (p,m,f), a parsed field (%nn,f), or a decimal constant (+n or -n). See p,m,f under p,m,f,edit for details on the fields you can use. See deccon under deccon,edit for details on the decimal constants you can use.
- **operator** is MIN (minimum), MAX (maximum), MUL (multiplication), DIV (division), MOD (modulus), ADD (addition) or SUB (subtraction).

The order of evaluation precedence for the operators is as follows unless it is changed by parentheses:

1. MIN and MAX
2. MUL, DIV and MOD
3. ADD and SUB

The intermediate or final result of a DIV operation is rounded down to the nearest integer. The intermediate or final result of a MOD operation is an integer remainder with the same sign as the dividend. If an intermediate or final result of an arithmetic expression overflows 31 digits, the overflowing intermediate or final result will be truncated to 31 digits, intentionally or unintentionally. If an intermediate or final result of an arithmetic expression requires division or modulus by 0, the intermediate or final result will be set to 0, intentionally or unintentionally.

An arithmetic expression produces a signed, 31-digit zoned decimal (ZD) result to be edited as specified. If an Mn, EDIT, or EDxy parameter is not specified, the result is edited using the M0 edit mask.

The default number of digits (d) used for editing is 15 if every term in the expression is one of the following:

- a 1-4 byte BI or FI field
- a 1-8 byte PD field
- a 1-15 byte ZD, FS, CSF, UFF or SFF field
- a decimal constant with 1-15 significant digits.

The default number of digits (d) used for editing is 31 if any term in the expression is one of the following:

- a 5-8 byte BI or FI field
- a 9-16 byte PD field
- a 16-31 byte ZD field
- a 4-byte or 8-byte FL field
- a 16-32 byte FS or CSF field
- a 16-44 byte UFF or SFF field
- a decimal constant with 16-31 significant digits.

If EDIT or EDxy is specified, the number of digits in the pattern (I's and T's) is used.

See edit under p,m,f,edit for further details on the edit fields you can use.

Sample Syntax:

```
OUTFIL OUTREC=(5:C'% REDUCTION FOR ',21,8,C' IS ',
  ((11,6,ZD,SUB,31,6,ZD),MUL,+1000),DIV,11,6,ZD,
  EDIT=(SIIT.T),SIGNS=(+,-))
```

arexp,to or (arexp),to

specifies that the converted result of an arithmetic expression is to appear in the reformatted OUTFIL output record. See arexp under arexp,edit for further details on arithmetic expressions.

An arithmetic expression produces a signed, 31-digit zoned decimal (ZD) result to be converted as specified.

The default number of digits (d) used for conversion is 15 if **every** term in the expression is one of the following:

- a 1-4 byte BI or FI field
- a 1-8 byte PD field
- a 1-15 byte ZD, FS, CSF, UFF or SFF field
- a decimal constant with 1-15 significant digits.

The default number of digits (d) used for conversion is 31 if any term in the expression is one of the following:

- a 5-8 byte BI or FI field
- a 9-16 byte PD field
- a 16-31 byte ZD field
- a 4-byte or 8-byte FL field
- a 16-32 byte FS or CSF field
- a 16-44 byte UFF or SFF field
- a decimal constant with 16-31 significant digits.

See to under p,m,f,to for further details on the to fields you can use.

Sample Syntax:

```
OUTFIL FNAMES=OUT,
  OUTREC=(61,3,X,
          35,6,FS,ADD,45,6,FS,ADD,55,6,FS,TO=FS,LENGTH=7,X,
          (5,11,PD,MIN,112,11,PD),PD,LENGTH=11,X,
          64,5,SEQNUM,5,ZD)
```

p,m,Y2x or p,m,Y4x

Specifies that an input date field is to be edited. Real Y2x dates are edited using the century window established by the Y2PAST option in effect. Y2x and Y4x dates with special indicators are expanded appropriately (for example, p,6,Y2T transforms C'000000' to C'00000000').

Editing involving an input date with an invalid digit (A-F) can result in a data exception (0C7 ABEND) or an incorrect output value. Editing involving an invalid input date can result in an invalid output value.

p
See p under p,m,a.

m
Specifies the length in bytes of the Y2x (two-digit year) or Y4x (four-digit year) date field.

Y2x or Y4x
Specifies the Y2 or Y4 format for the date field. See Appendix C, “Data format descriptions,” on page 819 for detailed format descriptions.

Sample Syntax:

```
OUTFIL BUILD=(21,3,Y2U,X,3,8,Y4W)
```

Table 55 on page 274 shows the output produced for each type of edited date.

Table 55. Input and result fields for Yxx date editing

m,Yxx	Input date	Output for p,m,Yxx
3,Y2T	C'yyx' or Z'yyx'	C'ccyyx'
4,Y2T	C'yyxx' or Z'yyxx'	C'ccyyxx'
5,Y2T	C'yyddd' or Z'yyddd'	C'ccyyddd'
6,Y2T	C'yymmdd' or Z'yymmdd'	C'ccyymmdd'
7,Y4T	C'ccyyddd' or Z'ccyyddd'	C'ccyyddd'
8,Y4T	C'ccyymmdd' or Z'ccyymmdd'	C'ccyymmdd'
2,Y2U	P'yyx'	C'ccyyx'
3,Y2V	P'yyxx'	C'ccyyxx'
3,Y2U	P'yyddd'	C'ccyyddd'
4,Y2V	P'yymmdd'	C'ccyymmdd'
4,Y4U	P'ccyyddd'	C'ccyyddd'
5,Y4V	P'ccyymmdd'	C'ccyymmdd'
3,Y2W	C'xyy' or Z'xyy'	C'xccyy'
4,Y2W	C'xxyy' or Z'xxyy'	C'xxccyy'
5,Y2W	C'dddyy' or Z'dddyy'	C'dddccyy'
6,Y2W	C'mmddyy' or Z'mmddyy'	C'mmddccyy'
7,Y4W	C'dddccyy' or Z'dddccyy'	C'dddccyy'
8,Y4W	C'mmddccyy' or Z'mmddccyy'	C'mmddccyy'
2,Y2X	P'xyy'	C'xccyy'
3,Y2Y	P'xxyy'	C'xxccyy'
3,Y2X	P'dddyy'	C'dddccyy'
4,Y2Y	P'mmddyy'	C'mmddccyy'
4,Y4X	P'dddccyy'	C'dddccyy'
5,Y4Y	P'mmddccyy'	C'mmddccyy'
2,Y2C	C'yy'	C'ccyy'
2,Y2Z	Z'yy'	C'ccyy'
2,Y2S	C'yy'	C'ccyy'
2,Y2P	P'yy'	C'ccyy'
1,Y2D	X'yy'	C'ccyy'
1,Y2B	B'yy'	C'ccyy'

%nn,Y2x or %nn,Y4x
Specifies that a parsed input date field is to be edited. See PARSE for details of parsed fields. See "p,m,Y2x or p,m,Y4x" for further details.

p,m,Y2x,edit or p,m,Y4x,edit
Specifies that the output for a p,m,Yxx input date field as shown in Table 55 on page 274 is to be further edited according to the edit parameters you specify. For example, if you specify:

```
OUTFIL BUILD=(28,5,Y4V,EDIT=(TTTT-TT-TT)
```

The P'ccyymmdd' (X'0ccyymmddC') date value will be transformed to a C'ccyymmdd' date value and then edited to a C'ccyy-mm-dd' date value.

See "p,m,Y2x or p,m,Y4x" and "p,m,f,edit" for related details.

nn,Y2x,edit or %nn,Y4x,edit
Specifies that the output for a parsed Yxx input date field as shown in Table 55 on page 274 is to be further edited according to the edit parameters you specify. See PARSE for details of parsed fields. See "p,m,Y2x,edit or p,m,Y4x,edit" for further details.

p,m,Y2x,to or p,m,Y4x,to
Specifies that an input date field as shown in Table 55 on page 274 is to be converted according to the to parameters you specify. For example, if you specify:

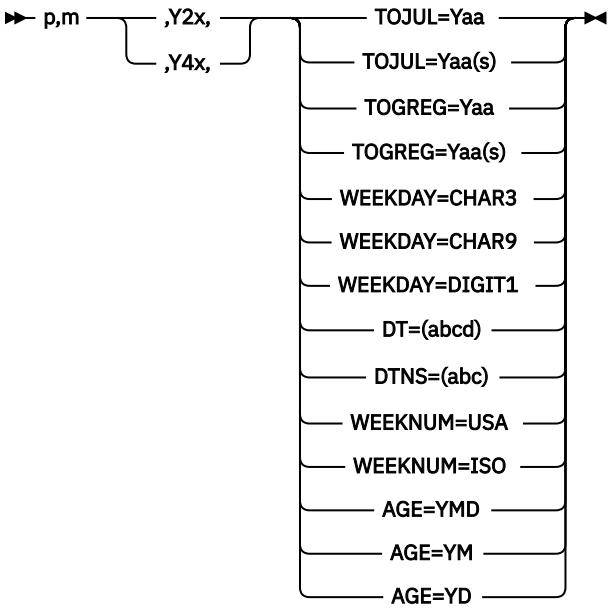
```
OUTFIL BUILD=(5,6,Y2W,TO=PD,LENGTH=5)
```

The C'mmddyy' date value will be transformed to a Z'mmddccyy' date value and then converted to a P'mmddccyy' (X'0mmddccyyC') date value.

See "p,m,Y2x or p,m,Y4x" and "p,m,f,to" for related details.

%nn,Y2x,to or %nn,Y4x,to
Specifies that a parsed input date field as shown in Table 55 on page 274 is to be converted according to the to parameters you specify. See PARSE for details of parsed fields. See "p,m,Y2x,to or p,m,Y4x,to" for further details.

p,m,Y2x,todate or p,m,Y4x,todate



Specifies that an input date field of one type is to be converted to a corresponding output date field of another type. You can convert date fields between combinations of 2-digit and 4-digit year dates, CH/ZD and PD dates, and Julian and Gregorian dates. You can also convert a date field to a corresponding day of the week in several forms.

Each type of date field you can use as input for date conversions is shown in Table 56 on page 276.

Table 56. Input fields for p,m,Yxx date conversion

m,Yxx	Input date
5,Y2T	C'yyddd' or Z'yyddd'
6,Y2T	C'yymmdd' or Z'yymmdd'
7,Y4T	C'ccyyddd' or Z'ccyyddd'
8,Y4T	C'ccyymmdd' or Z'ccyymmdd'
3,Y2U	P'yyddd'
4,Y2V	P'yymmdd'
4,Y4U	P'ccyyddd'
5,Y4V	P'ccyymmdd'
5,Y2W	C'dddyy' or Z'dddyy'
6,Y2W	C'mmddyy' or Z'mmddyy'
7,Y4W	C'dddccyy' or Z'dddccyy'
8,Y4W	C'mmddccyy' or Z'mmddccyy'
3,Y2X	P'dddyy'
4,Y2Y	P'mmddyy'
4,Y4X	P'dddccyy'
5,Y4Y	P'mmddccyy'

p
See p under p,m,a.

m
Specifies the length in bytes of the Y2x (two-digit year) or Y4x (four-digit year) date field.

Y2x or Y4x
Specifies the Y2 or Y4 format for the date field. See Appendix C, “Data format descriptions,” on page 819 for detailed format descriptions.

todate
Specifies the type of date conversion to be performed.

p,m,Yxx,TOJUL=Yaa
Converts the input date to a Julian output date.

p,m,Yxx,TOJUL=Yaa(s)
Converts the input date to a Julian output date with s separators. s can be any character except a blank.

p,m,Yxx,TOGREG=Yaa
Converts the input date to a Gregorian output date.

p,m,Yxx,TOGREG=Yaa(s)
Converts the input date to a Gregorian output date with s separators. s can be any character except a blank.

The output date field created by each valid TOJUL and TOGREG combination is shown in Table 57 on page 277.

Table 57. TOJUL and TOGREG output date fields

Yaa	TOJUL=Yaa	TOJUL=Yaa(s)	TOGREG=Yaa	TOGREG=Yaa(s)
Y2T	C'yyddd'	C'yysddd'	C'yymmdd'	C'yysmmsdd'
Y2W	C'dddyy'	C'dddsyy'	C'mmddyy'	C'mmsddsyy'
Y2U	P'yyddd'	n/a	n/a	n/a
Y2V	n/a	n/a	P'yymmdd'	n/a
Y2X	P'dddyy'	n/a	n/a	n/a
Y2Y	n/a	n/a	P'mmddyy'	n/a
Y4T	C'ccyyddd'	C'ccyysddd'	C'ccyymmdd'	C'ccyysmmsdd'
Y4W	C'dddccyy'	C'dddsccyy'	C'mmddccyy'	C'mmsddsccyy'
Y4U	P'ccyyddd'	n/a	n/a	n/a
Y4V	n/a	n/a	P'ccyymmdd'	n/a
Y4X	P'dddccyy'	n/a	n/a	n/a
Y4Y	n/a	n/a	P'mmddccyy'	n/a

Sample Syntax:

```
* Convert a P'dddyy' input date to a C'ccyy/mm/dd' output date
  OUTFIL BUILD=(21,3,Y2X,TOGREG=Y4T(/),X,
* Convert a C'ccyymmdd' input date to a P'ccyyddd' output date
     42,8,Y4T,TOJUL=Y4U,X,
* Convert a C'mmddyy' input date to a C'yymmdd' output date
     11,6,Y2W,TOGREG=Y2T)
```

p,m,Yxx,WEEKDAY={CHAR3|CHAR9|DIGIT1}

Converts an input date field to a corresponding output day of the week in one of several forms.

Each type of input date field you can use is shown in Table 56 on page 276. The different types of output you can display are shown in Table 58 on page 277.

Table 58. Output for weekdays

Day	CHAR3	CHAR9	DIGIT1
Sunday	C'SUN'	C'SUNDAY '	C'1'
Monday	C'MON'	C'MONDAY '	C'2'
Tuesday	C'TUE'	C'TUESDAY '	C'3'
Wednesday	C'WED'	C'WEDNESDAY'	C'4'
Thursday	C'THU'	C'THURSDAY '	C'5'
Friday	C'FRI'	C'FRIDAY '	C'6'
Saturday	C'SAT'	C'SATURDAY '	C'7'

Sample Syntax:

```
* Convert a P'mmddccyy' input date to a 3-character weekday
  OUTFIL BUILD=(5:15,5,Y4Y,WEEKDAY=CHAR3,
* Convert a C'yyddd' input date to a 1-digit weekday
     18:27,5,Y2T,WEEKDAY=DIGIT1,
* Convert a P'dddccyy' input date to a 9-character weekday
     41:121,4,Y4X,WEEKDAY=CHAR9)
```

p,m,Yxx,DT=(abcd) or p,m,Yxx,DTNS=(abc)

Converts an input date field of one type to a corresponding Gregorian output date field of another type.

Each type of input date field you can use is shown in Table 56 on page 276.

DT=(abcd) creates an output date in the form C'adbdc', where a, b, and c indicate the order in which the month, day, and year are to appear and whether the year is to appear as two or four digits, and d is the character to be used to separate the month, day and year. For a, b, and c, use M to represent the month (01-12), D to represent the day (01-31), Y to represent the

last two digits of the year (for example, 09), or 4 to represent the four digits of the year (for example, 2009). M, D, and Y or 4 can each be specified only once.

DTNS=(abc) creates an output date in the form C'abc', where a, b and c indicate the order in which the month, day, and year are to appear and whether the year is to appear as two or four digits. For a, b and c, use M to represent the month (01-12), D to represent the day (01-31), Y to represent the last two digits of the year (for example, 09), or 4 to represent the four digits of the year (for example, 2009). M, D, and Y or 4 can each be specified only once.

Sample Syntax:

```
* Convert a C'yyddd' input date to a C'dd/mm/ccyy' output date
  OUTFIL BUILD=(92,5,Y2T,DT=(DM4/),X,
* Convert a P'ccyyddd' input date to a C'mmddyy' output date
     53:32,4,Y4U,DTNS=(MDY))
```

p,m,Yxx,WEEKNUM={USA|ISO}

Converts an input date field to a corresponding 2 byte output number of the week in 2 different forms.

USA: The WEEKNUM=USA function returns an integer in the range of 1 to 54 that represents the week of the year. The week starts with Sunday, and January 1 is always in the first week.

ISO: The WEEKNUM=ISO function returns an integer in the range of 1 to 53 that represents the week of the year. The week starts with Monday and includes 7 days. Week 1 is the first week of the year to contain a Thursday, which is equivalent to the first week containing January 4. Thus, it is possible to have up to 3 days at the beginning of the year appear as the last week of the previous year, or to have up to 3 days at the end of a year appear as the first week of the next year.

Each type of input date field you can use is shown in Table 56 on page 276. The different types of output you can display are shown in Table 59 on page 278.

Table 59. Output for weeknum

Input - Date	WEEKNUM=USA	WEEKNUM=ISO
2000-01-01	01	52
2000-01-02	02	52
2000-01-03	02	01
2000-12-31	54	52
2014-01-02	01	01
2014-06-09	24	24
2014-12-31	53	01

Sample Syntax:

```
* Convert a P'mmddccyy' input date to a 2-character weeknum in
* ISO format
  OUTFIL BUILD=(5:15,5,Y4Y,WEEKNUM=ISO,
* Convert a C'yyddd' input date to a 2-digit weeknum in
* USA format
     18:27,5,Y2T,WEEKNUM=USA)
```

p,m,Yxx,AGE={YMD|YM|YD}

Can be used to calculate the date duration that specifies the number of years, months, and days between a given date and the current date. The Input date cannot be greater than the current date.

Age=YMD produces a 8 byte result which has duration in years (0-9999), months (00-12), and days (00-31).

Age=YM produces a 6 byte result which has duration in years (0-9999), months (00-12).

Age=YD produces a 7 byte result which has duration in years (0-9999), days (00-366).

For example, if the input birth date is March 12th 2015 and the current date is September 30th 2015 then Age=YMD will produce a duration of 00000618 (0000 years, 06 months, and 18 days).

Sample Syntax: If the input birth date is March 12th 2015 and the current date is September 30th 2015 then Age=YMD will produce a duration of 00000618 (0000 years, 06 months, and 18 days).

```
* Calculate the date duration of a C'ccyyddd' input date to
* current date as years, months and days
  OUTFIL BUILD=(15:1,7,Y4T,AGE=YMD,
* Calculate the date duration of a P'yymmdd' input date to
* current date as years and days
                                55:27,4,Y2V,AGE=YD)
```

Conversion of Real Dates, Special Indicators and Invalid Dates

For CH/ZD dates (Y2T, Y4T, Y2W, Y4W), the zone and sign are ignored; only the digits are used. For example, X'F2F0F0F9F1F2F3D0' and X'A2B0C0D9E1F21320' are treated as 20091230. For PD dates (Y2U, Y4U, Y2V, Y4V, Y2X, Y4X, Y2Y, Y4Y), the sign is ignored; only the relevant digits are used. For example, X'120091230D' is treated as 20091230.

For CH/ZD dates (Y2T, Y4T, Y2W, Y4W), the special indicators are X'00...00' (BI zeros), X'40...40' (blanks), C'0...0' (CH zeros), Z'0...0' (ZD zeros), C'9...9' (CH nines), Z'9...9' (ZD nines) and X'FF...FF' (BI ones). For PD dates (Y2U, Y4U, Y2V, Y4V, Y2X, Y4X, Y2Y, Y4Y), the special indicators are P'0...0' (PD zeros) and P'9...9' (PD nines).

yy for real 2-digit year dates is transformed to ccyy when appropriate using the century window established by the Y2PAST option in effect. ccyy for real 4-digit year dates is transformed to yy when appropriate by removing cc.

Date conversion is not performed for special indicators; the special indicator is just used appropriately for the output date field. For example, if p,5,Y2T,TOGREG=Y4T(/) is used, an input yyddd special indicator of C'99999' results in an output date field of C'9999/99/99'. However, CH/ZD special indicators of BI zeros, blanks and BI ones cannot be converted to PD special indicators.

Conversion involving an input date with an invalid digit (A-F) will result in a data exception (0C7 ABEND) or an incorrect output value.

Conversion involving an invalid input date or invalid output date will result in an output value of asterisks and an informational message (issued once). A date is considered invalid if any of the following range conditions are not met:

- yy must be between 00 and 99
- ccyy must be between 0001 and 9999
- mm must be between 01 and 12
- dd must be between 01 and 31, and must be valid for the year and month
- ddd must be 001 to 366 for a leap year, or between 001 and 365 for a non-leap year

A date is also considered invalid if the input field is a CH/ZD special indicator of binary zeros, blanks or binary ones, and the output field is PD.

%nn,Y2x,todate or %nn,Y4x,todate

Specifies that an input date field of one type is to be converted to a corresponding output date field of another type. See PARSE for details of parsed fields. See "p,m,Y2x,todate or p,m,Y4x,todate" for further details.

p,m,Y2x,dateop or p,m,Y4x,dateop

Specifies an arithmetic operation for an input date field. The available operations are as follows:

- **ADDDAYS**, **ADDMONS** or **ADDYEARS** can be used to add days, months or years to a date field. As a simple example, you could use the following to add 10 days to a C'ccyymmdd' date and produce the resulting C'ccyy/mm/dd' date:

```
15,8,Y4T,ADDDAYS,+10,TOGREG=Y4T(/)
```

- **SUBDAYS**, **SUBMONS** or **SUBYEARS** can be used to subtract days, months or years from a date field. As a simple example, you could use the following to subtract 3 months from a P'dddyy' date and produce the resulting P'ccyyddd' date:

```
21,3,Y2X,SUBMONS,+3,TOJUL=Y4U
```

- **DATEDIFF** can be used to calculate the number of days between two date fields. As a simple example, you could use the following to calculate the difference in days between two C'ccyymmdd' dates:

```
41,8,Y4T,DATEDIFF,31,8,Y4T
```

- **NEXTDday** can be used to calculate the next specified day for a date field. As a simple example, you could use the following to calculate the next Friday for a C'ccyyddd' date as a C'ccyy.ddd' date:

```
3,7,Y4T,NEXTDFRI,TOJUL=Y4T(.)
```

- **PREVDday** can be used to calculate the previous specified day for a date field. As a simple example, you could use the following to calculate the previous Wednesday for a P'yyddd' date as a C'ccyymmdd' date:

```
51,3,Y2U,PREVDWED,TOGREG=Y4T
```

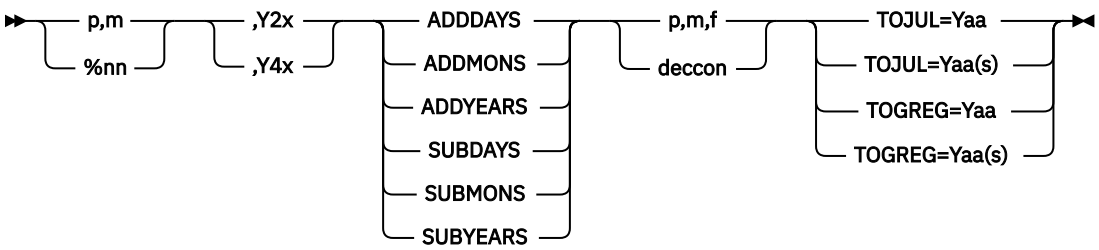
- **LASTDAYW**, **LASTDAYM**, **LASTDAYQ** or **LASTDAYY** can be used to calculate the last day of the week, month, quarter or year for a date field. As a simple example, you could use the following to calculate the last day of the month for a C'mmddccyy' date as a C'mmddccyy' date:

```
28,8,Y4W,LASTDAYM,TOGREG=Y4W
```

%nn,Y2x,dateop or %nn,Y4x,dateop

Specifies an arithmetic operation for a parsed input date field.

Adding or Subtracting Days, Months or Years



p,m,Yxx,keyword,numeric,todate

Can be used to add n days, months or years to an input date field, or subtract n days, months or years from an input date field. The valid keywords are ADDDAYS, ADDMONS, ADDYEARS, SUBDAYS, SUBMONS and SUBYEARS.

The numeric value specifies the number of days, months or years to be added to or subtracted from the input date field. The resulting output date field is converted to the form indicated by todate.

p,m,Yxx specifies the starting position (p), length (m) and format (Yxx) of a 2-digit or 4-digit year input date field. The valid length and format for each type of date field you can use is shown in Table 60 on page 281.

Table 60. p,m,Yxx fields for date arithmetic

m,Yxx	Type of date
5,Y2T	C'yyddd' or Z'yyddd'
6,Y2T	C'yymmdd' or Z'yymmdd'
7,Y4T	C'ccyyddd' or Z'ccyyddd'
8,Y4T	C'ccyymmdd' or Z'ccyymmdd'
5,Y2W	C'dddyy' or Z'dddyy'
6,Y2W	C'mmddyy' or Z'mmddyy'
7,Y4W	C'dddccyy' or Z'dddccyy'
8,Y4W	C'mmddccyy' or Z'mmddccyy'
3,Y2U	P'yyddd'
4,Y2V	P'yymmdd'
4,Y4U	P'ccyyddd'
5,Y4V	P'ccyymmdd'
3,Y2X	P'dddyy'
4,Y2Y	P'mmddyy'
4,Y4X	P'dddccyy'
5,Y4Y	P'mmddccyy'

ADDDAYS adds n days to the p,m,Yxx date field.

ADDMONS adds n months to the p,m,Yxx date field. If necessary, the resulting date is adjusted backwards to the last valid day of the month. For example, if 1 month is added to 20101031, the resulting date will be 20101130 rather than 20101131.

ADDYEARS adds n years to the p,m,Yxx date field. If the resulting date is February 29 of a non-leap year, it will be adjusted backwards to a valid date of February 28.

SUBDAYS subtracts n days from the p,m,Yxx date field.

SUBMONS subtracts n months from the p,m,Yxx date field. If necessary, the resulting date is adjusted backwards to the last valid day of the month. For example, if 1 month is subtracted from 20101031, the resulting date will be 20100930 rather than 20100931.

SUBYEARS subtracts n years from the p,m,Yxx date field. If the resulting date is February 29 of a non-leap year, it will be adjusted backwards to a valid date of February 28.

numeric can be a decimal constant (+n or -n), or a valid numeric field (p,m,f) with BI, FI, ZD, PD, FS, UFF or SFF format and an appropriately corresponding length.

For ADDDAYS or SUBDAYS, the numeric constant or field value can be from -3652058 to +3652058.

For ADDMONS or SUBMONS, the numeric constant or field value can be from -119987 to +119987.

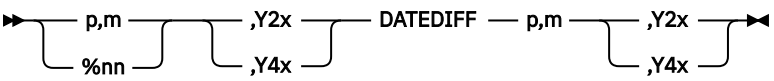
For ADDYEARS or SUBYEARS, the numeric constant or field value can be from -9998 to +9998.

TOJUL and TOGREG
See TOJUL and TOGREG under "p,m,Y2x,todate or p,m,Y4x,todate" for details.

%nn,Yxx,keyword,numeric,todate
Can be used to add n days, months or years to a parsed input date field, or subtract n days, months or years from a parsed input date field. See PARSE for details of parsed fields. See "p,m,Yxx,keyword,numeric,todate" for further details.

%nn cannot be used for the numeric field.

Calculating Difference in Days



p,m,Yxx,DATEDIFF,p,m,Yxx
Can be used to calculate the number of days difference between two input date fields. The result is an 8-byte value consisting of a sign and 7 digits (sddddddd). If the first date is greater than or equal to the second date, the sign is + (plus). If the first date is less than the second date, the sign is - (minus).

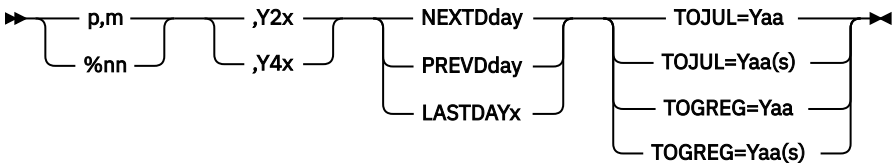
p,m,Yxx specifies the starting position (p), length (m) and format (Yxx) of a 2-digit or 4-digit year input date field. The valid length and format for each type of date field you can use is shown in Table 60 on page 281.

DATEDIFF calculates the number of days that result when the second date is subtracted from the first date. The result is in the form sddddddd with + or - for s as appropriate and leading zeros. The result can be between -3652058 and +3652058 days.

%nn,Yxx,DATEDIFF,p,m,Yxx
Can be used to calculate the number of days difference between a parsed input date field and another input date field. See PARSE for details of parsed fields. See "p,m,Yxx,DATEDIFF,p,m,Yxx" for further details.

%nn cannot be used for the second date field.

Next, Previous or Last Day



p,m,Yxx,keyword,todate
Can be used to specify the next specified day, previous specified day, or last day of the week, month, quarter or year for an input date. The valid keywords are NEXTDday, PREVDday or LASTDAYx (where day can be SUN, MON, TUE, WED, THU, FRI or SAT and x can be W, M, Q or Y).

The result is an output date field which is converted to the form indicated by todate.

p,m,Yxx specifies the starting position (p), length (m) and format (Yxx) of a 2-digit or 4-digit year input date field. The valid length and format for each type of date field you can use is shown in Table 60 on page 281.

NEXTDSUN calculates the next Sunday for a date field.

NEXTDMON calculates the next Monday for a date field.

NEXTDTUE calculates the next Tuesday for a date field.

NEXTDWED calculates the next Wednesday for a date field.

NEXTDTHU calculates the next Thursday for a date field.

NEXTDFRI calculates the next Friday for a date field.

NEXTDSAT calculates the next Saturday for a date field.

PREVDSUN calculates the previous Sunday for a date field.

PREVDMON calculates the previous Monday for a date field.

PREVDTUE calculates the previous Tuesday for a date field.

PREVDWED calculates the previous Wednesday for a date field

PREVDTHU calculates the previous Thursday for a date field.

PREVDFRI calculates the previous Friday for a date field.

PREVDSAT calculates the previous Saturday for a date

LASTDAYW calculates the last day of the week (Friday) for a date field.

LASTDAYM calculates the last day of the month for a date field.

LASTDAYQ calculates the last day of the quarter for a date field.

LASTDAYY calculates the last day of the year for a date field.

TOJUL and TOGREG
See TOJUL and TOGREG under "p,m,Y2x,todate or p,m,Y4x,todate" for details.

%nn,Yxx,keyword,todate
Can be used to specify the next specified day, previous specified day, or last day of the week, month, quarter or year for a parsed input date. See PARSE for details of parsed fields. See “p,m,Yxx,keyword,todate” for further details.

Arithmetic with Real Dates, Special Indicators and Invalid Dates
For CH/ZD dates (Y2T, Y4T, Y2W, Y4W), the zone and sign are ignored; only the digits are used. For example, X'F2F0F0F9F1F2F3D0' and X'A2B0C0D9E1F21320' are treated as 20091230. For PD dates (Y2U, Y4U, Y2V, Y4V, Y2X, Y4X, Y2Y, Y4Y), the sign is ignored; only the relevant digits are used. For example, X'120091230D' is treated as 20091230.

For CH/ZD dates (Y2T, Y4T, Y2W, Y4W), the special indicators are X'00...00' (BI zeros), X'40...40' (blanks), C'0...0' (CH zeros), Z'0...0' (ZD zeros), C'9...9' (CH nines), Z'9...9' (ZD nines) and X'FF...FF' (BI ones). For PD dates (Y2U, Y4U, Y2V, Y4V, Y2X, Y4X, Y2Y, Y4Y), the special indicators are P'0...0' (PD zeros) and P'9...9' (PD nines).

yy for real 2-digit year dates is transformed to ccyy when appropriate using the century window established by the Y2PAST option in effect. ccyy for real 4-digit year dates is transformed to yy when appropriate by removing cc.

If a special indicator is used for DATEDIFF, the result will be an output value of asterisks and an informational message (issued once).

Date arithmetic is not performed for special indicators; the special indicator is just used appropriately for the output date field. For example, if p,5,Y2T,ADDDAYS,+10,TOGREG=Y4T(/) is used, an input yyddd special indicator of C'99999' results in an output date field of C'9999/99/99'. However, CH/ZD special indicators of BI zeros, blanks and BI ones cannot be converted to PD special indicators.

Arithmetic involving an input date with an invalid digit (A-F) will result in a data exception (0C7 ABEND) or an incorrect output value.

Arithmetic involving an invalid input date or invalid output date will result in an output value of asterisks and an informational message (issued once). A date is considered invalid if any of the following range conditions are not met:

- yy must be between 00 and 99
- ccyy must be between 0001 and 9999
- mm must be between 01 and 12
- dd must be between 01 and 31, and must be valid for the year and month
- ddd must be between 001 and 366 for a leap year, or between 001 and 365 for a non-leap year.

A date is also considered invalid if the input field is a CH/ZD special indicator of binary zeros, blanks or binary ones, and the output field is PD.

p,m,Y2x(s) or p,m,Y4x(s)
Specifies that an input date field is to be edited with separators. s can be any character except a blank. Real Y2x dates are edited using the century window established by the Y2PAST option in effect. Y2x and Y4x dates with special indicators are expanded appropriately (for example, p,8,Y4T(s) transforms C'00000000' to C'0000/00/00').

Editing involving an input date with an invalid digit (A-F) can result in a data exception (0C7 ABEND) or an incorrect output value. Editing involving an invalid input date can result in an invalid output value.

p
See p under p,m,a.

m
Specifies the length in bytes of the Y2x (two-digit year) or Y4x (four-digit year) date field.

Y2x(s) or Y4x(s)
Specifies the Y2 or Y4 format for the date field and the separator character(s). See Appendix C, "Data format descriptions," on page 819 for detailed format descriptions.

Sample Syntax:

```
* Convert a Z'dddccyy' date to a C'ddd/ccyy' date.
  OUTFIL BUILD=(19,7,Y4W(/),X,
* Convert a P'ccyymmdd' date to a C'ccyy-mm-dd' date.
     43,5,Y4V(-))
```

Table 61 on page 284 shows the output produced for each type of edited date.

Table 61. Input and result fields for Yxx(s) date editing

m,Yxx	Input date	Output for p,m,Yxx(s)
3,Y2T(s)	C'yyx' or Z'yyx'	C'ccyysx'
4,Y2T(s)	C'yyxx' or Z'yyxx'	C'ccyysxx'
5,Y2T(s)	C'yyddd' or Z'yyddd'	C'ccyysddd'
6,Y2T(s)	C'yymmdd' or Z'yymmdd'	C'ccyysmmsdd'
7,Y4T(s)	C'ccyyddd' or Z'ccyyddd'	C'ccyysddd'
8,Y4T(s)	C'ccyymmdd' or Z'ccyymmdd'	C'ccyysmmsdd'
2,Y2U(s)	P'yyx'	C'ccyysx'
3,Y2V(s)	P'yyxx'	C'ccyysxx'
3,Y2U(s)	P'yyddd'	C'ccyysddd'
4,Y2V(s)	P'yymmdd'	C'ccyysmmsdd'
4,Y4U(s)	P'ccyyddd'	C'ccyysddd'
5,Y4V(s)	P'ccyymmdd'	C'ccyysmmsdd'
3,Y2W(s)	C'xyy' or Z'xyy'	C'xsccyy'
4,Y2W(s)	C'xxyy' or Z'xxyy'	C'xxsccyy'
5,Y2W(s)	C'dddyy' or Z'dddyy'	C'dddsccyy'
6,Y2W(s)	C'mmddyy' or Z'mmddyy'	C'mmsddsccyy'
7,Y4W(s)	C'dddccyy' or Z'dddccyy'	C'dddsccyy'
8,Y4W(s)	C'mmddccyy' or Z'mmddccyy'	C'mmsddsccyy'
2,Y2X(s)	P'xyy'	C'xsccyy'
3,Y2Y(s)	P'xxyy'	C'xxsccyy'
3,Y2X(s)	P'dddyy'	C'dddsccyy'
4,Y2Y(s)	P'mmddyy'	C'mmsddsccyy'
4,Y4X(s)	P'dddccyy'	C'dddsccyy'
5,Y4Y(s)	P'mmddccyy'	C'mmsddsccyy'

%nn,Y2x(s) or %nn,Y4x(s)
Specifies that a parsed input date field is to be edited with separators. See PARSE for details of parsed fields. See "p,m,Y2x(s) or p,m,Y4x(s)" for further details.

p,m,Y2xP
Specifies that an input date field is to be converted to a packed decimal output date field. Real Y2x dates are edited using the century window established by the Y2PAST option in effect. Y2x and Y4x dates with special indicators are expanded appropriately (for example, p,6,Y2TP transforms C'000000' to P'00000000').

Editing involving an input date with an invalid digit (A-F) can result in a data exception (0C7 ABEND) or an incorrect output value. Editing involving an invalid input date can result in an invalid output value.

p
See p under p,m,a.

m
Specifies the length in bytes of the Y2x (two-digit year) or Y4x (four-digit year) date field.

Y2xP
Specifies the Y2 format for the date field. See Appendix C, “Data format descriptions,” on page 819 for detailed format descriptions.

Sample Syntax:

```
OUTFIL BUILD=(11,3,Y2XP,X,21,4,Y2WP)
```

Table 62 on page 285 shows the output produced for each type of date.

Table 62. Input and result fields for Yxx(s) date editing

m,Y2xP	Input date	Output for p,m,Y2xP
3,Y2TP	C'yyx' or Z'yyx'	P'ccyyx'
4,Y2TP	C'yyxx' or Z'yyxx'	P'ccyyxx'
5,Y2TP	C'yyddd' or Z'yyddd'	P'ccyyddd'
6,Y2TP	C'yymmdd' or Z'yymmdd'	P'ccyymmdd'
2,Y2UP	P'yyx'	P'ccyyx'
3,Y2VP	P'yyxx'	P'ccyyxx'
3,Y2UP	P'yyddd'	C'ccyyddd'
4,Y2VP	P'yymmdd'	C'ccyymmdd'
3,Y2WP	C'xyy' or Z'xyy'	C'xccyy'
4,Y2WP	C'xxyy' or Z'xxyy'	C'xxccyy'
5,Y2WP	C'dddyy' or Z'dddyy'	C'dddccyy'
6,Y2WP	C'mmddyy' or Z'mmddyy'	C'mmddccyy'
2,Y2XP	P'xyy'	C'xccyy'
3,Y2YP	P'xxyy'	C'xxccyy'
3,Y2XP	P'dddyy'	C'dddccyy'
4,Y2YP	P'mmddyy'	C'mmddccyy'
2,Y2PP	P'yy'	C'ccyy'
1,Y2DP	X'yy'	C'ccyy'

%nn,Y2xP
Specifies that a parsed input date field is to be converted to a packed decimal output date field. See PARSE for details of parsed fields. See "p,m,Y2xP" for further details

p,m,lookup or %nn,lookup
specifies that a character constant, hexadecimal constant, input field (p,m), or parsed input field (%nn) from a lookup table is to appear in the reformatted OUTFIL output record. You can use p,m,lookup or %nn,lookup to select a specified set constant (that is, a character or hexadecimal string) or set field (that is, an input field or parsed input field) based on matching an input value against find constants (that is, character, hexadecimal, or bit constants).

p
See p under p,m,a.

m
specifies the length in bytes of the input field to be compared to the find constants. The value for m must be 1 to 64 if character or hexadecimal find constants are used, or 1 if bit find constants are used.

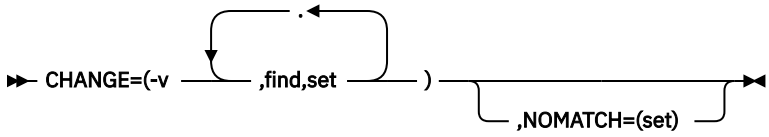
%nn

specifies a parsed input field to be compared to the find constants. See PARSE for details of parsed fields.

lookup

Specifies how the input field or parsed input field is to be changed to the output field, using a lookup table.

CHANGE



specifies a list of change pairs, each consisting of a find constant to be compared to the input field value or parsed input field value and a set constant or set field to use as the output field when a match occurs.

v
specifies the length in bytes of the output field to be inserted in the reformatted OUTFIL output record. The value for v must be between 1 and 64.

find
specifies a find constant to be compared to the input field value or parsed input field value. If the input field value or parsed input field value matches the find constant, the corresponding set constant or set field is used for the output field.

The find constants can be either character string constants, hexadecimal string constants, or bit constants:

- Character string constants (C'xx...x') and hexadecimal string constants (X'yy...yy') can be 1 to m bytes and can be intermixed with each other, but not with bit constants. See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

  If the string is less than m bytes, it will be padded on the right to a length of m bytes, using blanks (X'40') for a character string constant or zeros (X'00') for a hexadecimal string constant.
- Bit constants (B'bbbbbbbb') must be 1 byte and cannot be intermixed with character or string constants. See “INCLUDE control statement” on page 91 for details of coding bit constants.

  For bit constants, because of the specification of bits to be ignored, more than one find constant can match an input field value; the set constant for the first match found will be used as the output field. For example, if you specify:

```
OUTFIL OUTREC=(5,1,
  CHANGE=(1,B'11......',C'A',B'1.......',C'B'))
```

  input field value X'C0' (B'11000000') matches both bit constants, but C'A' will be used for the set constant, because its find constant is the first match.

set
specifies a set constant or set field to be used as the output field if the corresponding find constant matches the input field value or parsed input field value. Set constants and set fields can be intermixed.

Set constants can be either character string constants (C'xx...x') or hexadecimal string constants (X'yy...yy') of 1 to v bytes. See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

If the string is less than v bytes, it will be padded on the right to a length of v bytes, using blanks (X'40') for a character string constant or zeros (X'00') for a hexadecimal string constant.

Set fields are specified as q,n or %nn. q specifies the input position. See p under p,m,a for details of coding q. n specifies the input length of 1 to v bytes. %nn specifies a parsed input field. See PARSE for details of parsed fields. If the set field length is less than v, the input field or parsed input field will be padded on the right to a length of v bytes, using blanks (X'40').

NOMATCH
specifies the action to be taken if an input field value or parsed input field value does not match any of the find constants. If you do not specify NOMATCH, and a match is not found for any input value, DFSORT terminates processing.

If you specify NOMATCH, it must follow CHANGE.

set
specifies a set constant or set field to be used as the output field if a match is not found.

Set constants can be either character string constants (C'xx...x') or hexadecimal string constants (X'yy...yy') of 1 to v bytes and can be intermixed. See "INCLUDE control statement" on page 91 for details of coding character and hexadecimal string constants.

If the string is less than v bytes, it will be padded on the right to a length of v bytes, using blanks (X'40') for a character string constant or zeros (X'00') for a hexadecimal string constant.

Set fields are specified as q,n or %nn. q specifies the input position. See p under p,m,a for details of coding q. n specifies the input length of 1 to v bytes. %nn specifies a parsed input field. See PARSE for details of parsed fields. If the set field length is less than v, the input field or parsed input field will be padded on the right to a length of v bytes, using blanks (X'40').

Sample Syntax:

```
OUTFIL FILES=1,
 OUTREC=(11,1,
         CHANGE=(6,
           C'R',C'READ',
           C'U',C'UPDATE',
           X'FF',C'EMPTY',
           C'X',35,6,
           C'A',C'ALTER'),
         NOMATCH=(11,6),
         4X,
         21,1,
         CHANGE=(10,
           B'.1......',C'VSAM',
           B'.0......',C'NON-VSAM'))
```

p,m,justify
specifies that a left-justified or right-justified input field is to appear in the reformatted OUTFIL output record. For a left-justified field, leading blanks are removed and the characters from the first nonblank to the last nonblank are shifted left, with blanks inserted on the right if needed. For a right-justified field, trailing blanks are removed and the characters from the last nonblank to the first nonblank are shifted right, with blanks inserted on the left if needed.

Optionally:

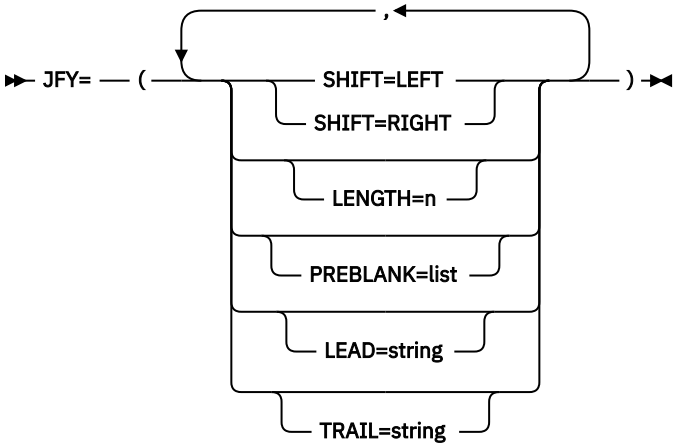
- specific leading and trailing characters can be changed to blanks before justification begins
- a leading string can be inserted
- a trailing string can be inserted
- the output length can be changed (it's equal to the input length by default)

p
See p under p,m,a.

m
See m under p,m,a.

justify
specifies how the input field is to be justified for output.

JFY



Note: For clarity, in the examples in this section, b is used to represent a blank in the input fields.

SHIFT=LEFT
specifies that a left-justified input field is to appear in the reformatted OUTFIL output record. Leading blanks are removed and the characters from the first nonblank to the last nonblank are shifted left, with blanks inserted on the right if needed. For example, with:

```
1,15,JFY=(SHIFT=LEFT)
```

an input field of:

```
bbbbABbCDbbEFbb
```

results in an output field of:

```
ABbCDbbEFbbbbbb
```

m is used for the output field length unless LENGTH=n is specified. If the left-justified input field is shorter than the output field length, blanks are inserted on the right. If the left-justified input field is longer than the output field length, characters are truncated on the right. You can use LENGTH=n to prevent padding or truncation.

SHIFT=RIGHT
specifies that a right-justified input field is to appear in the reformatted OUTFIL output record. Trailing blanks are removed and the characters from the last nonblank to the first nonblank are shifted right, with blanks inserted on the left if needed. For example, with:

```
1,15,JFY=(SHIFT=RIGHT)
```

an input field of:

```
bbbbABbCDbbEFbb
```

results in an output field of:

```
bbbbbbABbCDbbEF
```

m is used for the output field length unless LENGTH=n is specified. If the right-justified input field is shorter than the output field length, blanks are inserted on the left. If the right-justified input field is longer than the output field length, characters are truncated on the left. You can use LENGTH=n to prevent padding or truncation.

LENGTH=n
specifies the length of the justified output field. The value for n must be between 1 and 32752. If LENGTH=n is not specified, m (the input field length) is used for the length of the justified output field. You can use LENGTH=n to prevent padding or truncation.

PREBLANK=list
specifies a list of one or more characters to be changed to blanks **before** justify processing begins. Only leading and trailing characters are changed to blanks. Scanning from left to right, each nonblank character at the start of the input field that matches a character in the list is replaced by a blank character, until a nonblank character not in the list is found. Scanning from right to left, each nonblank character at the end of the input field that matches a character in the list will be replaced by a blank character, until a nonblank character not in the list is found.

list can be 1 to 10 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See "INCLUDE control statement" on page 91 for details of coding character and hexadecimal string constants.

Each character in the list is treated as one character to be preblanked. Thus PREBLANK=C'*' specifies an asterisk is to be preblanked, and PREBLANK=C',;$' specifies a comma, a semicolon and a dollar sign are each to be preblanked.

For example, let's say we have an input field of:

```
**b<*ABbCD*bEF>*b**
```

If we specify:

```
1,19,JFY=(SHIFT=LEFT,PREBLANK=C'*')
```

each leading or trailing asterisk is changed to a blank before left- justify processing begins. So the output field is:

```
<*ABbCD*bEF>bbbbbbb
```

If we specify:

```
1,19,JFY=(SHIFT=RIGHT,PREBLANK=C'*<>')
```

each leading or trailing asterisk, less than sign and greater than sign is changed to a blank before right-justify processing begins. So the output field is:

```
bbbbbbbbbbABbCD*bEF
```

LEAD=string
specifies a string to be inserted in the output field before the first nonblank character in the input field.

string can be 1 to 50 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See "INCLUDE control statement" on page 91 for details of coding character and hexadecimal string constants.

For example, let's say we have an input field of:

```
bABCbEbbbbb
```

If we specify:

```
1,11,JFY=(SHIFT=RIGHT,LEAD=C'XYZ')
```

the output field is:

```
bbbXYZABCbE
```

When we add characters with LEAD=string, it's often necessary to specify LENGTH=n to avoid truncation. For example, let's say we have an input field of:

```
AbBbC
```

If we specify:

```
1,5,JFY=(SHIFT=LEFT,LEAD=C'XYZ')
```

the output field is:

```
XYZAb
```

Since the output field length is defaulted to the input field length of 5, the resulting 8 characters (XYZAbBbC) are truncated on the right to 5 characters (XYZAb) for output. If we instead specify:

```
1,5,JFY=(SHIFT=LEFT,LEAD=C'XYZ',LENGTH=8)
```

the output field is:

```
XYZAbBbC
```

LENGTH=8 increases the output field by the 3 LEAD characters and truncation is prevented.

TRAIL=string
specifies a string to be inserted in the output field after the last nonblank character in the input field.

string can be 1 to 50 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

For example, let's say we have an input field of:

```
bABCbEbbbbb
```

If we specify:

```
1,11,JFY=(SHIFT=LEFT,TRAIL=C'XYZ')
```

the output field is:

```
ABCbEXYZbbb
```

When we add characters with TRAIL=string, it's often necessary to specify LENGTH=n to avoid truncation. For example, let's say we have an input field of:

```
AbBbC
```

If we specify:

```
1,5,JFY=(SHIFT=RIGHT,TRAIL=C'XYZ')
```

the output field is:

```
bCXYZ
```

Since the output field length is defaulted to the input field length of 5, the resulting 8 characters (AbBbCXYZ) are truncated on the left to 5 characters (bCXYZ) for output. If we instead specify:

```
1,5,JFY=(SHIFT=RIGHT,TRAIL=C'XYZ',LENGTH=8)
```

the output field is:

```
AbBbCXYZ
```

LENGTH=8 increases the output field by the 3 TRAIL characters and truncation is prevented.

Sample Syntax:

```
OUTFIL BUILD=(5:16,20,JFY=(SHIFT=LEFT,PREBLANK=C'*',
   LEAD=C'<A>',TRAIL=C'</A>',LENGTH=22))
```

%nn,justify
specifies that a left-justified or right-justified parsed input field is to appear in the reformatted OUTFIL output record. See PARSE for details of parsed fields. See p,m,justify for further details.

p,m,squeeze
specifies that a left-squeezed or right-squeezed input field is to appear in the reformatted OUTFIL output record. For a left-squeezed field, all blanks are removed and the characters from the first nonblank to the last nonblank are shifted left, with blanks inserted on the right if needed. For a right-squeezed field, all blanks are removed and the characters from the last nonblank to the first nonblank are shifted right, with blanks inserted on the left if needed. Optionally:

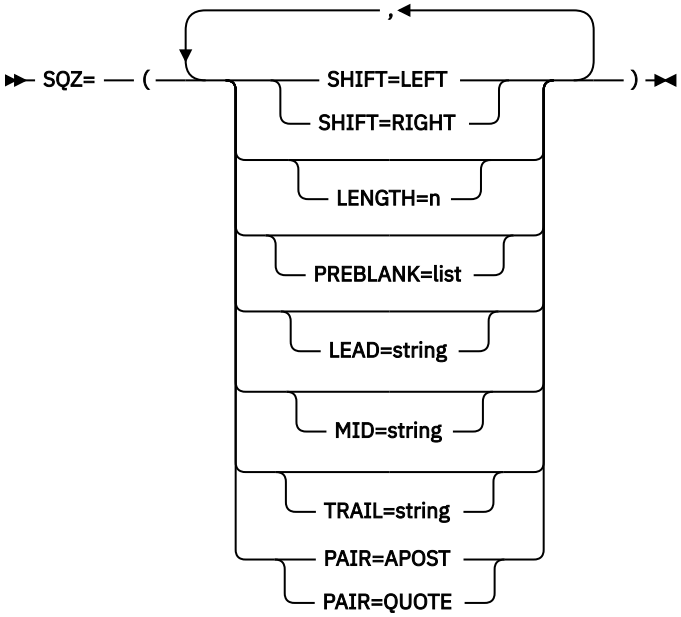
- specific characters can be changed to blanks before squeezing begins
- a leading string can be inserted
- a trailing string can be inserted
- a string (for example, a comma delimiter) can be inserted wherever a group of blanks is removed between the first nonblank and the last nonblank
- blanks can be kept as is between paired apostrophes ('AB CD EF') or paired quotes ("AB CD EF")
- the output length can be changed (it is equal to the input length by default)

p
See p under p,m,a.

m
See m under p,m,a.

squeeze
Specifies how the input field is to be squeezed for output.

SQZ



Note: For clarity, in the examples in this section, b is used to represent a blank in the input fields.

SHIFT=LEFT
specifies that a left-squeezed input field is to appear in the reformatted OUTFIL output record. All blanks are removed and the characters from the first nonblank to the last nonblank are shifted left, with blanks inserted on the right if needed. For example, with:

```
1,15,SQZ=(SHIFT=LEFT)
```

an input field of:

```
bbbbABbCDbbEFbb
```

results in an output field of:

```
ABCDEFbbbbbbbbb
```

m is used for the output field length unless LENGTH=n is specified. If the left-squeezed input field is shorter than the output field length, blanks are inserted on the right. If the left-squeezed input field is longer than the output field length, characters are truncated on the right.

SHIFT=RIGHT
specifies that a right-squeezed input field is to appear in the reformatted OUTFIL output record. All blanks are removed and the characters from the last nonblank to the first nonblank are shifted right, with blanks inserted on the left if needed. For example, with:

```
1,15,SQZ=(SHIFT=RIGHT)
```

an input field of:

```
bbbbABbCDbbEFbb
```

results in an output field of:

```
bbbbbbbbbABCDEF
```

m is used for the output field length unless LENGTH=n is specified. If the right-squeezed input field is shorter than the output field length, blanks are inserted on the left. If the right-squeezed input field is longer than the output field length, characters are truncated on the left.

LENGTH=n
specifies the length of the squeezed output field. The value for n must be between 1 and 32752. If LENGTH=n is not specified, m (the input field length) is used for the length of the squeezed output field.

PREBLANK=list
specifies a list of one or more characters to be changed to blanks **before** squeeze processing begins. Each nonblank character in the input field that matches a character in the list is replaced by a blank character.

list can be 1 to 10 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

Each character in the list is treated as one character to be preblanked. Thus PREBLANK=C'*' specifies an asterisk is to be preblanked, and PREBLANK=C',;$' specifies a comma, a semicolon and a dollar sign are each to be preblanked.

For example, let's say we have an input field of:

```
**b<*ABbC<>D*bEF>*b**
```

If we specify:

```
1,21,SQZ=(SHIFT=LEFT,PREBLANK=C'*',LENGTH=12)
```

each asterisk is changed to a blank before left-squeeze processing begins. The output field is:

```
<ABC<>DEF>bb
```

If we specify:

```
1,21,SQZ=(SHIFT=RIGHT,PREBLANK=C'*<>')
```

each asterisk, less than sign and greater than sign is changed to a blank before right-squeeze processing begins. The output field is:

```
bbbbbbbbbbbbbbbbABCDEF
```

LEAD=string
specifies a string to be inserted in the output field before the first nonblank character in the input field.

string can be 1 to 50 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

For example, let's say we have an input field of:

```
bABCbEbbbbb
```

If we specify:

```
1,11,SQZ=(SHIFT=RIGHT,LEAD=C'XYZ')
```

the output field is:

```
bbbbXYZABCE
```

When we add characters with LEAD=string, it's often necessary to specify LENGTH=n to avoid truncation. For additional information on this, see "LEAD=string" for JFY shown previously in this section.

MID=string
specifies a string to be inserted in the output field wherever one or more blanks is removed between the first nonblank and the last nonblank.

string can be 1 to 10 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

For example, let's say we have an input field of:

```
bABbbCbbbDbE
```

If we specify:

```
1,12,SQZ=(SHIFT=LEFT,MID=C',')
```

the output field is:

```
AB,C,D,Ebbbb
```

When we add characters with MID=string, it's often necessary to specify LENGTH=n to avoid truncation. For example, let's say we have an input field of:

```
AbBbC
```

If we specify:

```
1,5,SQZ=(SHIFT=LEFT,MID=C'XY')
```

the output field is:

```
AXYBX
```

Since the output field length is defaulted to the input field length of 5, the resulting 7 characters (AXYBXYC) are truncated on the right to 5 characters (AXYBX) for output. If we instead specify:

```
1,5,SQZ=(SHIFT=LEFT,MID=C'XY',LENGTH=7)
```

the output field is:

```
AXYBXYC
```

LENGTH=7 increases the output field to accommodate the MID strings and truncation is prevented.

TRAIL=string
specifies a string to be inserted in the output field after the last nonblank character in the input field.

string can be 1 to 50 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

For example, let's say we have an input field of:

```
bABCbEbbbbb
```

If we specify:

```
1,11,SQZ=(SHIFT=LEFT,LEAD=X'7D',TRAIL=X'7D')
```

the output field is:

```
'ABCE'bbbbb
```

When we add characters with TRAIL=string, it's often necessary to specify LENGTH=n to avoid truncation. For additional information on this, see "TRAIL=string" for JFY as shown previously in this section.

PAIR=APOST
specifies that blanks and PREBLANK characters between apostrophe (') pairs are to be kept as is. Use PAIR=APOST when you have literals that should not be squeezed.

For example, let's say we have an input field of:

```
b*b'ABbb*''*C'bbb'D*Ebb'*
```

If we specify:

```
1,25,SQZ=(SHIFT=LEFT,PREBLANK=C'*',MID=C',')
```

all of the blanks and asterisks, including those inside the apostrophe pairs, are squeezed out. The output field is:

```
'AB,'',C','D,E,'bbbbbbbbb
```

However, if we specify:

```
 1,25,SQZ=(SHIFT=LEFT,PREBLANK=C'*',MID=C',',
           PAIR=APOST)
```

only the blanks and asterisks outside of the apostrophe pairs are squeezed out. The output field is:

```
'ABbb*''*C','D*Ebb'bbbbbb
```

If an apostrophe is specified in the PREBLANK list (for example, PREBLANK=C'''' or PREBLANK=X'7D'), it is ignored, that is, an apostrophe in the input field is not replaced by a blank. So if you want each apostrophe to be replaced by a blank, do not specify PAIR=APOST.

For SHIFT=LEFT, scanning is left to right. If an apostrophe is found with no subsequent apostrophe, all of the characters from the apostrophe to the end of the input field are ignored. For example, let's say we have an input field of:

```
bbXbY'ABCbbbDbb
```

If we specify:

```
1,15,SQZ=(SHIFT=LEFT,PAIR=APOST)
```

there's an unpaired apostrophe, so all of the characters from the apostrophe to the end of the input field are ignored. The output field is:

```
XY'ABCbbbDbbbbb
```

For SHIFT=RIGHT, scanning is right to left. If an apostrophe is found with no previous apostrophe, all of the characters from the apostrophe to the beginning of the input field are ignored. For example, let's say we have an input field of:

```
bbXbY'ABCbbbDbb
```

If we specify:

```
1,15,SQZ=(SHIFT=RIGHT,PAIR=APOST)
```

there's an unpaired apostrophe, so all of the characters from the apostrophe to the beginning of the input field are ignored. The output field is:

```
bbbbbbbXbY'ABCD
```

PAIR=QUOTE

specifies that blanks and PREBLANK characters between quote (") pairs are to be kept as is. Use PAIR=QUOTE when you have literals that should not be squeezed.

For example, let's say we have an input field of:

```
b*b"ABbb*""*C"bbb"D*Ebb"*
```

If we specify:

```
1,25,SQZ=(SHIFT=LEFT,PREBLANK=C'*',MID=C',')
```

all of the blanks and asterisks, including those inside the quote pairs, are squeezed out. The output field is:

```
"AB,"",C","D,E,"bbbbbbbbb
```

However, if we specify:

```
1,25,SQZ=(SHIFT=LEFT,PREBLANK=C'*',MID=C',',
          PAIR=QUOTE)
```

only the blanks and asterisks outside of the quote pairs are squeezed out. The output field is:

```
"ABbb*""*C","D*Ebb'bbbbbb
```

If a quote is specified in the PREBLANK list (for example, PREBLANK=C'"' or PREBLANK=X'7F'), it is ignored, that is, a quote in the input field is not replaced by a blank. So if you want each quote to be replaced by a blank, do not specify PAIR=QUOTE.

For SHIFT=LEFT, scanning is left to right. If a quote is found with no subsequent quote, all of the characters from the quote to the end of the input field are ignored. For example, let's say we have an input field of:

```
bbXbY"ABCbbbDbb
```

If we specify:

```
1,15,SQZ=(SHIFT=LEFT,PAIR=QUOTE)
```

there's an unpaired quote, so all of the characters from the quote to the end of the input field are ignored. The output field is:

```
XY"ABCbbbDbbbbb
```

For SHIFT=RIGHT, scanning is right to left. If a quote is found with no previous quote, all of the characters from the quote to the beginning of the input field are ignored. For example, let's say we have an input field of:

```
bbXbY"ABCbbbDbb
```

If we specify:

```
1,15,SQZ=(SHIFT=RIGHT,PAIR=QUOTE)
```

there's an unpaired quote, so all of the characters from the quote to the beginning of the input field are ignored. The output field is:

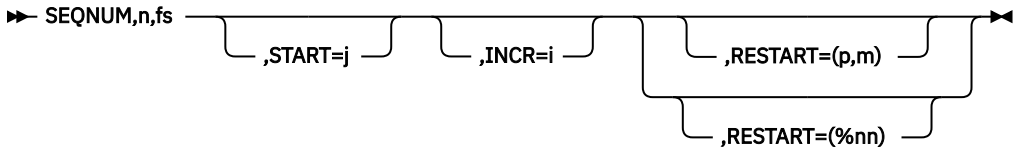
```
bbbbbbbXbY"ABCD
```

Sample Syntax:

```
   OUTFIL BUILD=(5:16,20,SQZ=(SHIFT=LEFT,PAIR=QUOTE,PREBLANK=C'<>'))
```

%nn,squeeze
specifies that a left-squeezed or right-squeezed parsed input field is to appear in the reformatted OUTFIL output record. See PARSE for details of parsed fields. See p,m,squeeze for further details.

seqnum



specifies that a sequence number is to appear in the reformatted OUTFIL output record. The sequence numbers are assigned in the order in which the records are received for OUTFIL OUTREC processing. You can create BI, PD, ZD, CSF, or FS sequence numbers and control their lengths, starting values and increment values. You can restart the sequence number at the start value each time a specified OUTFIL input field (p,m) or parsed input field (%nn) changes.

n
specifies the length of the sequence number. The value for n must be between 1 and 16.

fs
specifies the format for the sequence number, which can be BI, PD, ZD, CSF, or FS.

For a ZD format sequence number, F is used as the sign.

For a PD format sequence number, C is used as the sign.

For a CSF or FS format sequence number, blank is used as the sign and leading zeros are suppressed.

For a PD, ZD, CSF, or FS format sequence number, the maximum value DFSORT can create is limited to the lesser of 15 decimal digits or the output field length (n). If a sequence number overflows this limit, it will be truncated to the lesser of 15 decimal digits or the output field length, and then subsequently incremented as usual.

For a BI format sequence number, the maximum value DFSORT can create is limited to the lesser of 8 bytes of ones (X'FFFFFFFFFFFFFFFF') or the number of ones that will fit in the specified output field length (n). If a sequence number overflows this limit, it will be truncated to the lesser of 8 bytes or the output field length, and then subsequently incremented as usual.

START
specifies the starting value for the sequence number.

j
specifies the starting value. The value for j must be between 0 and 100000000000. The default for j is 1.

INCR
specifies the increment value for the sequence number.

i
specifies the increment value. The value for i must be between 1 and 10000000. The default for i is 1.

RESTART
specifies that DFSORT is to restart the sequence number at the starting value (START=j) when the binary value for the specified OUTFIL input field or parsed input field changes. This allows you to sequence each set of records with the same value (that is, duplicate records) separately. For example: Without RESTART, if you had six OUTFIL input records with 'A', 'A', 'A', 'B', 'B' and 'C', respectively, in position 1, the output records would be given the sequence numbers 1, 2, 3, 4, 5 and 6. But with RESTART=(1,1), the output records are given the sequence numbers 1, 2, 3, 1, 2 and 1; DFSORT restarts at the starting value (1, by default) when the input value at position 1 changes from 'A' to 'B' and again when it changes from 'B' to 'C'.

p,m or %nn
specifies the OUTFIL input field or parsed input field to be used to determine when the sequence number is to be restarted at the starting value. If a variable-length OUTFIL input record is too short to contain a specified restart field, binary zeros (or blanks for a parsed field) will be used for the missing bytes, intentionally or unintentionally.

p
See p under p,m,a

m
specifies the length in bytes of the input field. The value for m must be between 1 and 256.

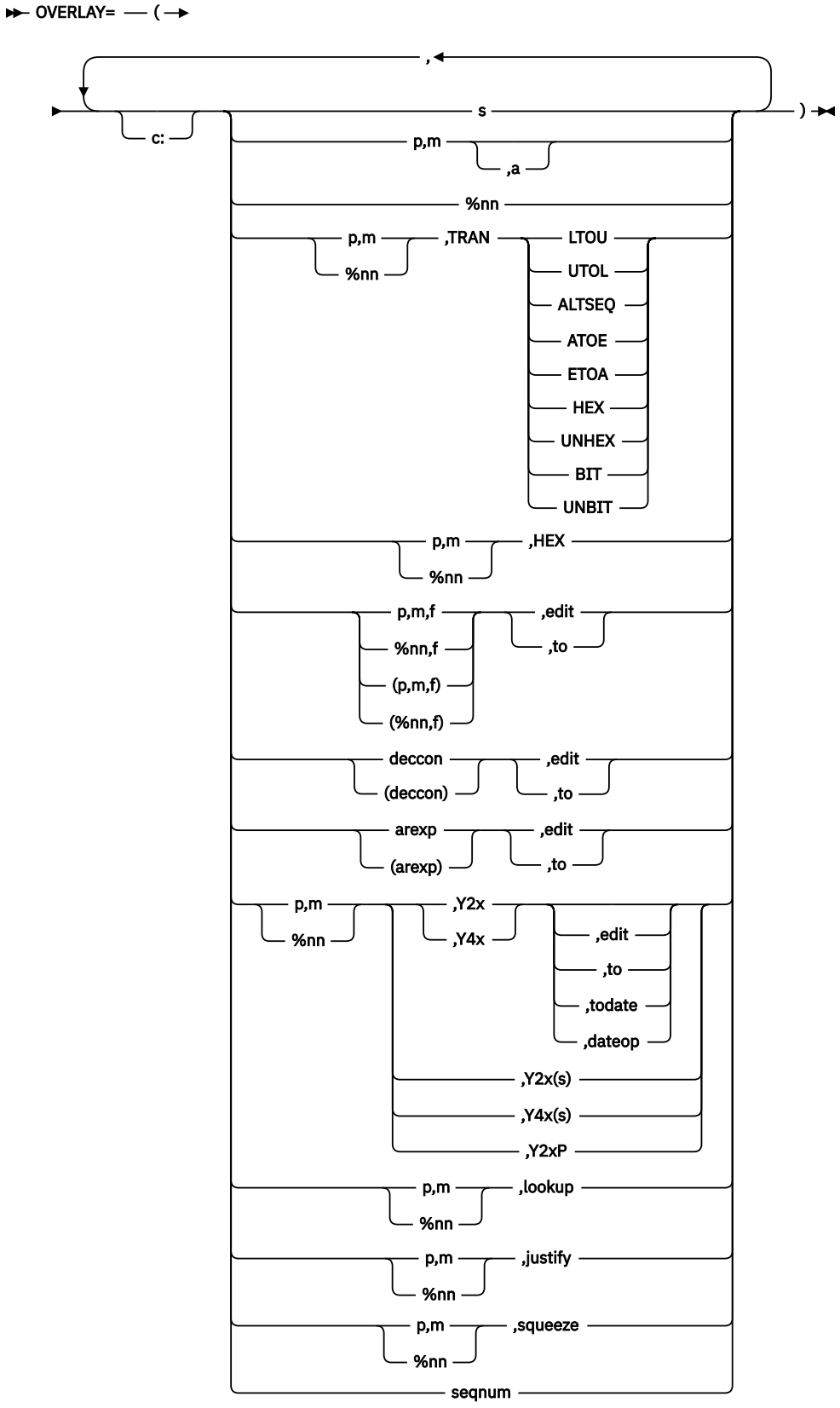
%nn
specifies the parsed input field. See PARSE for details of parsed fields.

Sample Syntax:

```
OUTFIL FNAMES=O1,OUTREC=(SEQNUM,6,ZD,START=1000,INCR=50,
         X,22,8,X,13,5)
OUTFIL FNAMES=O2,OUTREC=(1,12,SEQNUM,4,BI)
OUTFIL FNAMES=O3,OUTREC=(1,80,81:SEQNUM,8,ZD,START=21,INCR=20,
              RESTART=(35,8))
```

Default for BUILD or OUTREC: None; must be specified.

OVERLAY



Specifies each item that is to overlay specific columns in the reformatted record. Columns that are not overlaid remain unchanged. If you want to insert, rearrange, or delete fields, use BUILD or OUTREC rather than OVERLAY. Use OVERLAY only to overlay existing columns or to add fields at the end of

every record. OVERLAY can be easier to use than BUILD or OUTREC when you just want to change a few fields without rebuilding the entire record.

For fixed-length records, the first input and output data byte starts at position 1. For variable-length records, the first input and output data byte starts at position 5, after the RDW in positions 1-4.

Use c: (column) to specify the output positions to be overlaid. If you do not specify c: for the first item, it defaults to 1:. If you do not specify c: for any other item, it starts after the previous item. For example, if you specify:

```
  OUTFIL OVERLAY=(25,2,11:C'A',15,3,C'**')
```

Input positions 25-26 are placed at output positions 1-2; C'A' is placed at output position 11; input positions 15-17 are placed at output positions 12-14; and C'**' is placed at output positions 15-16. The rest of the record remains unchanged.

You can specify items in any order, you can change the same item multiple times and you can overlap output columns. **Changes to earlier items affect changes to later items.** For example, say you specify:

```
OUTFIL OVERLAY=(21:8,4,ZD,MUL,+10,TO=ZD,LENGTH=6,
5:5,1,TRAN=UTOL,
5:5,1,CHANGE=(1,C'a',C'X',C'b',C'Y'),NOMATCH=(5,1))
```

and input position 5 has 'A'. The second item (UTOL) would change 'A' to 'a' and the third item (CHANGE) would change 'a' again to 'X'.

If you specify an OVERLAY item that extends the overlay record beyond the end of the input record, the reformatted record length is automatically increased to that length, and blanks are filled in on the left as needed. For variable-length records, the RDW length is increased to correspond to the larger reformatted record length after all of the OVERLAY items are processed. For example, if your OUTFIL input record has a length of 40 and you specify:

```
OUTFIL OVERLAY=(16:C'ABC',51:5C'*',35:15,2)
```

the OUTFIL output record is given a length of 55. Blanks are filled in from columns 41-50. For variable-length records, the length in the RDW is changed from 40 to 55 after all of the OVERLAY items are processed.

Missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

You can use the OVERLAY parameter with the FTOV parameter to convert fixed-length record data sets to variable-length record data sets.

You can use the VLTRIM parameter with the OVERLAY parameter to remove specified trailing bytes from the end of variable-length records.

You can use the VLTRAIL parameter with the OVERLAY parameter to insert a string at the end of variable-length records.

You can use the OVERLAY parameter with any or all of the report parameters in the same way as for the BUILD or OUTREC parameter.

The OVERLAY parameter of the OUTREC statement applies to all input records whereas the OVERLAY parameter of the OUTFIL statement only applies to the OUTFIL input records for its OUTFIL group.

See OUTFIL OUTREC for details of the items listed in the OVERLAY syntax diagram shown previously in this section. You can specify all of the items for OVERLAY in the same way that you can specify them for BUILD or OUTREC with the following exceptions:

- You cannot specify p or p,HEX or p,TRAN=keyword for OVERLAY.
- You cannot specify / for OVERLAY.
- For p,m,H or p,m,F or or p,m,D fields specified for OVERLAY, fields are aligned as necessary without changing the preceding bytes.

- For variable-length records, you must not overlay positions 1-4 (the RDW) for OVERLAY, so be sure to specify the first column (c:) as 5 or greater. Do not specify 1:, 2:, 3: or 4: anywhere in your OVERLAY parameter. If you do not specify the first column, it will default to 1: which is invalid for variable-length records with OVERLAY. Whereas OUTREC=(1,m,...) is required, OVERLAY=(1,m) is not allowed since it would overlay the RDW.

Sample Syntax:

Fixed input records:

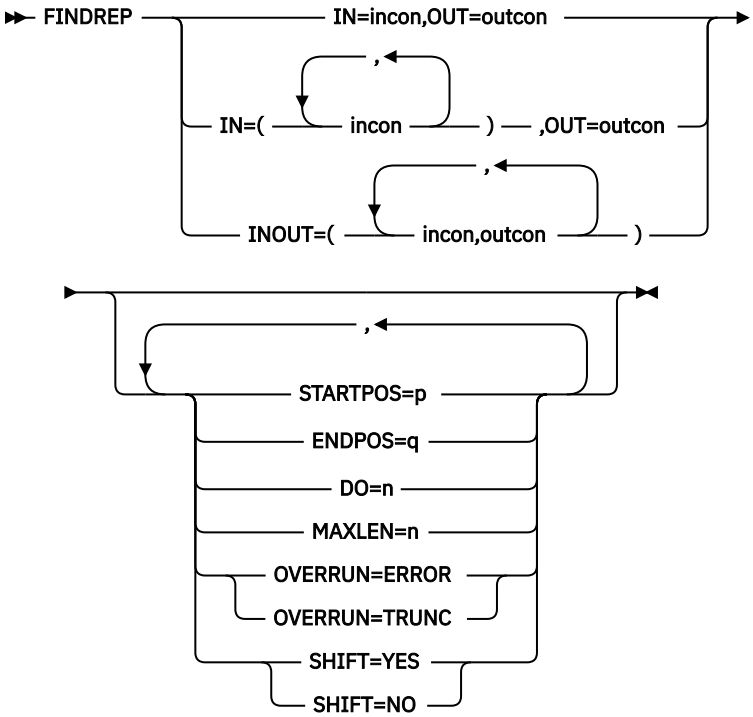
```
OUTFIL FNAMES=FLR,
     OVERLAY=(21:21,4,ZD,TO=PD,LENGTH=4,
         2:5,8,HEX,45:C'*',32,4,C'*',81:SEQNUM,5,ZD)
```

Variable input records:

```
OUTFIL FNAMES=VLR,
    OVERLAY=(5:X'0001',28:C'Date is ',YDDDNS=(4D),
       17:5C'*')
```

Default for OVERLAY: None; must be specified.

FINDREP



You can use FINDREP to find constants anywhere in a record and replace them with other constants of the same or different lengths. You can find character or hexadecimal input constants anywhere in your records and replace them with character, hexadecimal or null output constants. As appropriate, bytes can be shifted left or right, blank padding can be added for fixed-length records, and the length can be changed for variable-length records.

Various options of FINDREP allow you to define one or more input constants and a corresponding output constant, define one or more pairs of input and output constants, start and end the find scan at specified positions, stop after a specified number of constants are replaced, increase or decrease the length of the output record, define the action to be taken if nonblank characters overrun the end of the record, and specify whether output constants are to replace or overlay input constants.

You can use the FINDREP parameter with the FTOV parameter to convert fixed-length record data sets to variable-length record data sets.

You can use the VLTRIM parameter with the FINDREP parameter to remove specified trailing bytes from the end of variable-length records.

You can use the VLTRAIL parameter with the FINDREP parameter to insert a string at the end of variable-length records.

You can use the FINDREP parameter with any or all of the report parameters in the same way as for the BUILD or OUTREC parameter.

The FINDREP parameter of the OUTREC statement applies to all input records whereas the FINDREP parameter of the OUTFIL statement only applies to the OUTFIL input records for its OUTFIL group.

Input and output constants for find and replace processing are defined as follows:

input constant
An input constant can be specified as a single character string, a repeated character string, a single hexadecimal string, or a repeated hexadecimal string. The syntax is: C'string', nC'string', X'string' or nX'string'. n can be 1 to 256. The string can be 1 to 256 characters, or 1 to 256 pairs of hexadecimal digits. The total length of the constant must not exceed 256 bytes. Use two apostrophes for a single apostrophe.

output constant
An output constant can be specified as a null string, a single character string, a repeated character string, a single hexadecimal string, or a repeated hexadecimal string. The syntax is: C'' (null), C'string', nC'string', X'string' or nX'string'. n can be 1 to 256. The string can be a null, or 1 to 256 characters, or 1 to 256 pairs of hexadecimal digits. The total length of the constant must not exceed 256 bytes. Use two apostrophes for a single apostrophe. A null string can be used to remove input constants.

You must specify the input and output constants to be used for find and replace processing in one of the following ways. The default processing for each method described later in this section can be changed with various options described later.

IN=incon,OUT=outcon
Specifies one input constant and one output constant. Position 1 (for fixed-length records) or 5 (for variable-length records) will be set as the current position. The current position of the input record will be checked for the input constant. If a match is not found at the current position, the current position will be incremented by 1, and the process will be repeated. If a match is found at the current position, the output constant will replace the input constant, the current position will be incremented past the input constant, and the process will be repeated. Bytes after the replaced constants up to the end of the record will be shifted left or right as needed. Processing will stop when the current position is beyond the end of the input record.

Example:

```
OUTFIL IFTHEN=(WHEN=(11,1,CH,EQ,C'3'),
         FINDREP=(IN=C'YES',OUT=C'NO'))
```

Replaces every C'YES' input constant in records with a C'3' in position 11 with a C'NO' output constant, and shifts the bytes after each replaced constant to the left.

IN=(incon1,incon2,...,inconx),OUT=outcon
Specifies multiple input constants and one output constant. Position 1 (for fixed-length records) or 5 (for variable-length records) will be set as the current position. The current position of the input record will be checked for each input constant in turn until a match is found or all of the input constants have been checked. If a match is not found at the current position for any input constant, the current position will be incremented by 1, and the process will be repeated. If a match is found at the current position, the output constant will replace the input constant, the current position will be incremented past the input constant, and the process will be repeated. Bytes after the replaced constants up to the end of the record will be shifted left or right as needed. Processing will stop when the current position is beyond the end of the input record.

Example:

```
OUTFIL FINDREP=(IN=(X'FF',3X'00'),OUT=C'')
```

Removes every X'FF' and X'000000' input constant, and shifts the bytes after each removed constant to the left.

INOUT=(incon1,outcon1,incon2,outcon2,...,inconx,outconx)
Specifies one or more pairs of input and output constants. Position 1 (for fixed-length records) or 5 (for variable-length records) will be set as the current position. The current position of the input record will be checked for each input constant in turn until a match is found or all of the input constants have been checked. If a match is not found at the current position for any input constant, the current position will be incremented by 1, and the process will be repeated. If a match is found at the current position for an input constant, the corresponding output constant will replace the input constant, the current position will be incremented past the input constant, and the process will be repeated. Bytes after the replaced constants up to the end of the record will be shifted left or right as needed. Processing will stop when the current position is beyond the end of the input record.

Example:

```
OUTFIL FINDREP=(INOUT=(C'SAT',C'SATURDAY',C'SUN',C'SUNDAY'))
```

Replaces every C'SAT' input constant with a C'SATURDAY' output constant, and every C'SUN' input constant with a C'SUNDAY' output constant, and shifts the bytes after each replaced constant to the right.

You can specify any or all of the following options to change the default processing described earlier:

STARTPOS=p
Specifies the starting position in the input record for the find scan, overriding the default of position 1 for a fixed-length record or position 5 for a variable-length record. Use STARTPOS=p if you want to start your find scan at a particular position. p can be 1 to 32752. If p is less than 5 for a variable-length record, 5 will be used for p. If p is beyond the end of the input record, find and replace processing will not be performed for the record.

Example:

```
OUTFIL FINDREP=(IN=C'Yes',OUT=C'YES',STARTPOS=11)
```

Replaces every C'Yes' input constant found starting at or after position 11 with a C'YES' output constant.

ENDPOS=q
Specifies the ending position in the input record for the find scan, overriding the default of the end of the record. Use ENDPOS=q if you want to end your find scan at a particular position. ENDPOS=q only applies to the end of the find scan; bytes will still be shifted up to the end of the record as needed. q can be 1 to 32752. If q is less than 5 for a variable-length record, 5 will be used for q. If q is beyond the end of the input record, the end of the record will be used for q. If STARTPOS=p and ENDPOS=q are both specified, and p is greater than q, find and replace processing will not be performed for the record.

Example:

```
OUTFIL FINDREP=(IN=(C'D27',C'A52',C'X31'),OUT=C'INVALID',
   ENDPOS=2015)
```

Replaces every C'D27', C'A52' and C'X31' input constant found before or at position 2015 with a C'INVALID' output constant, and shifts the bytes after each replaced constant to the right (past 2015 up to the end of the record).

DO=n

Specifies the maximum number of times find and replace is to be performed for a record, overriding the default of every time. Scanning for the input constant stops when n input constants have been found and replaced. Use DO=n if you want to stop after a particular number of constants have been replaced. n can be 1 to 1000.

Example:

```
OUTFIL FNAMES=OUT1,FINDREP=(IN=X'015C',OUT=X'015D',DO=3)
```

Replaces the first three X'015C' input constants found with X'015D' output constants.

MAXLEN=n

Specifies the maximum length to be used for the output record created by find and replace processing, overriding the default of using the maximum length of the input record. Use MAXLEN=n if you want to increase or decrease the output record length. (For IFTHEN FINDREP, MAXLEN=n can only be used to increase the output record length, not decrease it. See "Notes" later in this section for more information.)

If an output constant is larger than a corresponding input constant, MAXLEN=n can be used to increase the size of the output record to allow for shifting characters to the right. n can be 1 to 32752. MAXLEN=n will be used to set the LRECL of the output data set, when appropriate.

Example:

```
OUTFIL FINDREP=(INOUT=(C'01',C'January',C'02',C'February',
  C'03',C'March'),MAXLEN=150)
```

Replaces every C'01' input constant with a C'January' output constant, every C'02' input constant with a C'February' output constant, and every C'03' input constant with a C'March' output constant, and shifts the bytes after each replaced constant to the right, allowing the record to expand to 150 bytes.

OVERRUN=ERROR or OVERRUN=TRUNC

Specifies the action DFSORT is to take if an overrun occurs, that is:

- if nonblank bytes are shifted to the right past the end of the output record as specified by MAXLEN=n, or as defaulted to the input record length, or
- if MAXLEN=n is used to make the output record smaller than the input record, and nonblank bytes are found in the input record past the end of the output record.

OVERRUN=ERROR is the default; it tells DFSORT to issue an error message and terminate if an overrun occurs.

Use OVERRUN=TRUNC if you want DFSORT to truncate the output record to the MAXLEN=n or input record length if an overrun occurs, rather than terminating. Bytes beyond the end of the output record are lost.

Example:

```
OUTFIL FINDREP=(IN=X'FFFF',OUT=C'INVALID',MAXLEN=50,OVERRUN=TRUNC)
```

Replaces every X'FFFF' input constant with a C'INVALID' output constant, and shifts the bytes after the replaced constants to the right. 50 byte output records are created. Bytes shifted past position 50 are lost. Without OVERRUN=TRUNC, if nonblank characters were shifted past position 50, an overrun error message would be issued and the job would terminate.

SHIFT=YES or SHIFT=NO

Specifies the action DFSORT is to take if an input constant is to be replaced by an output constant of a different length.

SHIFT=YES is the default; it tells DFSORT to shift the bytes after each replaced input constant to the left or right as needed.

Use SHIFT=NO if you want DFSORT to overlay the input constant with the output constant instead of shifting bytes. If a matching input constant is found and the output constant is smaller than the input constant, the current position will be incremented past the output constant rather than past the input constant before processing continues.

Example:

```
  OUTFIL FINDREP=(IN=(C'KW1=YES,',C'KW1=NO,'),OUT=C'KW2',SHIFT=NO)
```

Replaces every C'KW1=YES,' input constant with a C'KW2=YES,' output constant. Replaces every C'KW1=NO,' input constant with a C'KW2=NO,' output constant. SHIFT=NO ensures that C'KW1' is replaced with C'KW2' and the '=YES,' and '=NO,' bytes are kept. Without SHIFT=NO, the '=YES,' and '=NO,' bytes would be removed.

Default for FINDREP: None; must be specified.

FINDREP notes:

- For a single FINDREP operand, when a constant is replaced at the current position, no further checks are performed at that position. So a single FINDREP cannot be used to change a constant and then change it again. For example, if you had an input record with:

  ```
  ABC
  ```

  and used this OUTFIL statement:

  ```
    OUTFIL FINDREP=(INOUT=(C'AB',C'XY',C'XYC',C'RST'))
  ```

  the output record would be:

  ```
  XYC
  ```

  After C'AB' is changed to C'XY', the current pointer is advanced to C'C' so C'XYZ' is not found. However, since each IFTHEN clause starts at the beginning of the record, multiple IFTHEN clauses with FINDREP can be used to change a constant and then change it again. If you used:

  ```
    OUTFIL IFTHEN=(WHEN=INIT,FINDREP=(INOUT=(C'AB',C'XY'))),
      IFTHEN=(WHEN=INIT,FINDREP=(INOUT=(C'XYC',C'RST')))
  ```

  for the ABC record, the output would be:

  ```
  RST
  ```

  The first IFTHEN clause would change C'AB' to C'XY'. The second IFTHEN clause would start over from the first position and change C'XYC' to C'RST'.

- Duplicates and supersets of the same input constant after the first are effectively ignored, whereas subsets of the same input constant after the first are processed. For example, if you had this input record:

  ```
  ABCD ABQ
  ```

  and you used this OUTFIL statement:

  ```
    OUTFIL FINDREP=(INOUT=(C'AB',C'XY',C'ABCD',C'RSTU'))
  ```

  the output record would be:

  ```
  XYCD XYQ
  ```

  Since C'AB' is changed to C'XY', C'ABCD' is not found. If you wanted to change C'ABCD' to C'RSTU' and other instances of C'AB' to C'XY', you would need to specify C'ABCD' first. If you used this OUTFIL statement:

```
  OUTFIL FINDREP=(INOUT=(C'ABCD',C'RSTU',C'AB',C'XY'))
```

the output record would be:

```
RSTU XYQ
```

If C'ABCD' is found, it is changed to C'RSTU'. Otherwise, if C'AB' is found, it is changed to C'XY'.

- For fixed-length records, FINDREP in an IFTHEN clause operates against the maximum record padded with blanks on the right as needed. If a blank constant is being replaced, the blanks on the right will be replaced. For example, if we have an 80 byte FB input record and use:

```
  OUTFIL IFTHEN=(WHEN=INIT,BUILD=(1,20)),
         IFTHEN=(WHEN=(2,1,CH,EQ,C'R'),
           FINDREP=(IN=C' ',OUT=C'ABC'))
```

Although in this case BUILD=(1,20) will result in a 20-byte output record, IFTHEN FINDREP will operate against the maximum 80-byte record padded with 60 blanks on the right. For a record with 'R' in position 2, each of those 60 blanks on the right will be replaced with C'ABC'. This will cause an overrun of the 80 byte record, thus resulting in termination (OVERRUN=ERROR) or truncation (OVERRUN=TRUNC). ENDPOS=20 could be used to stop FINDREP from replacing the 60 blanks on the right.

Note that if we have an 80-byte input record with 'ABC' in positions 1-3 and we use:

```
  OUTFIL IFTHEN=(WHEN=INIT,BUILD=(1,3)),
         IFTHEN=(WHEN=INIT,FINDREP=(IN=C'ABC',OUT=C'12345'))
```

an overrun will not occur because IFTHEN FINDREP will operate against an 80-byte record padded with 77 blanks on the right, rather than against a 3-byte record even though the output record will be 3 bytes.

- For FINDREP in an IFTHEN clause, MAXLEN=n can be used to increase the maximum output length, but cannot be used to decrease the maximum output length. For example, with:

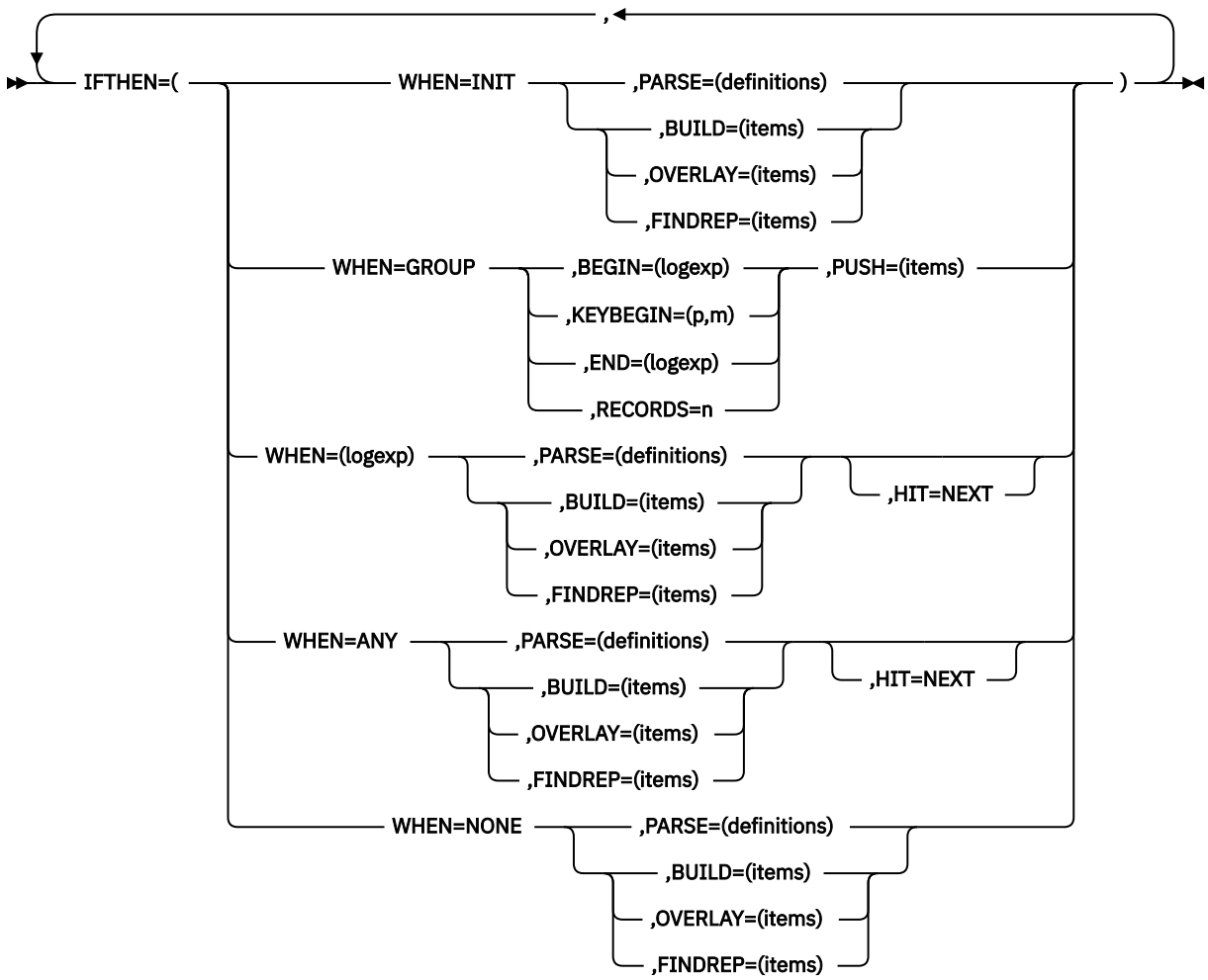
```
  OUTFIL IFTHEN=(WHEN=INIT,BUILD=(1,100)),
         IFTHEN=(WHEN=INIT,
           FINDREP=(IN=C'A',OUT=C'B',MAXLEN=150))
```

the output length will be set to 150, whereas with:

```
  OUTFIL IFTHEN=(WHEN=INIT,BUILD=(1,100)),
         IFTHEN=(WHEN=INIT,
           FINDREP=(IN=C'A',OUT=C'B',MAXLEN=70))
```

the output length will be set to 100.

IFTHEN



IFTHEN clauses allow you to reformat different records in different ways by specifying how build, overlay, find/replace or group operation items are to be applied to records that meet given criteria. IFTHEN clauses let you use simple or complex conditional logic to choose how different record types are reformatted.

If you want to insert, rearrange, or delete fields in the same way for every record, use BUILD or OUTREC rather than IFTHEN. If you want to do find and replace operations in the same way for every record, use FINDREP rather than IFTHEN. Use IFTHEN clauses if you want to insert, rearrange, delete, or overlay fields, or perform find/replace operations in different ways for different records, or if you want to perform operations on groups of records.

You can use five types of IFTHEN clauses as follows:

- **WHEN=INIT:** Use one or more WHEN=INIT clauses to apply build, overlay, or find/replace items to all of your input records. WHEN=INIT clauses and WHEN=GROUP clauses are processed before any of the other IFTHEN clauses.
- **WHEN=GROUP**: Use one or more WHEN=GROUP clauses to propagate fields, identifiers and sequence numbers within specified groups of records. WHEN=INIT clauses and WHEN=GROUP clauses are processed before any of the other IFTHEN clauses.
- **WHEN=(logexp):** Use one or more WHEN=(logexp) clauses to apply build, overlay or find/replace items to your input records that meet specified criteria. A WHEN=(logexp) clause is satisfied when the logical expression evaluates as true.
- **WHEN=ANY:** Use a WHEN=ANY clause after multiple WHEN=(logexp) clauses to apply additional build, overlay or find/replace items to your input records if they satisfied the criteria for any of the preceding WHEN=(logexp) clauses.

- **WHEN=NONE:** Use one or more WHEN=NONE clauses to apply build, overlay or find/replace items to your input records that did not meet the criteria for any of the WHEN=(logexp) clauses. WHEN=NONE clauses are processed after any of the other IFTHEN clauses. If you do not specify a WHEN=NONE clause, only the WHEN=INIT and WHEN=GROUP changes (if any) are applied to input records that do not meet the criteria for any of the WHEN=(logexp) clauses.

IFTHEN clauses are processed in the following order:

- WHEN=INIT clauses and WHEN=GROUP clauses
- WHEN=(logexp) clauses and WHEN=ANY clauses
- WHEN=NONE clauses

Processing of IFTHEN clauses continues unless one of the following occurs:

- A WHEN=(logexp) or WHEN=ANY clause is satisfied, and HIT=NEXT is not specified.
- A WHEN=(logexp), WHEN=ANY or WHEN=NONE clause is satisfied, and BUILD with / is specified (multiple output records).
- There are no more IFTHEN clauses to process. When processing of IFTHEN clauses stops, the IFTHEN record created so far is used as the output record.

  Example:

```
OUTFIL IFTHEN=(WHEN=(12,1,BI,ALL,X'3F'),OVERLAY=(18:C'Yes')),
       IFTHEN=(WHEN=(35,2,PD,EQ,+14),BUILD=(1,40,45,3,HEX),HIT=NEXT),
       IFTHEN=(WHEN=(35,2,PD,GT,+17),BUILD=(1,40,41,5,HEX),HIT=NEXT),
       IFTHEN=(WHEN=ANY,BUILD=(1,55,C'ABC',70:X)),
       IFTHEN=(WHEN=(35,2,PD,EQ,+8),BUILD=(1,40,2/,45,10)),
       IFTHEN=(WHEN=(63,2,CH,EQ,C'AB'),OVERLAY=(18:C'No')),
       IFTHEN=(WHEN=NONE,BUILD=(1,40,51,8,TRAN=LTOU))
```

  For this example, the IFTHEN clauses are processed as follows:

  – If IFTHEN clause 1 is satisfied, its overlay item is applied and IFTHEN processing stops.
  – If IFTHEN clause 1 is not satisfied, its overlay item is not applied and IFTHEN processing continues.
  – If IFTHEN clause 2 is satisfied, its build items are applied and IFTHEN processing continues.
  – If IFTHEN clause 2 is not satisfied, its build items are not applied and IFTHEN processing continues.
  – If IFTHEN clause 3 is satisfied, its build items are applied and IFTHEN processing continues.
  – If IFTHEN clause 3 is not satisfied, its build items are not applied and IFTHEN processing continues.
  – If IFTHEN clause 4 is satisfied, its build items are applied and IFTHEN processing stops.
  – If IFTHEN clause 4 is not satisfied, its build items are not applied and IFTHEN processing continues.
  – If IFTHEN clause 5 is satisfied, its build items are applied and IFTHEN processing stops.
  – If IFTHEN clause 5 is not satisfied, its build items are not applied and IFTHEN processing continues.
  – If IFTHEN clause 6 is satisfied, its overlay item is applied and IFTHEN processing stops.
  – If IFTHEN clause 6 is not satisfied, its overlay item is not applied and IFTHEN processing continues.
  – If IFTHEN clause 7 is satisfied, its build items are applied and IFTHEN processing stops.
  – If IFTHEN clause 7 is not satisfied, its build items are not applied and IFTHEN processing stops.

All of the IFTHEN clauses operate sequentially on an IFTHEN record. The IFTHEN record is created initially from the input record. Each IFTHEN clause tests and changes the IFTHEN record, as

appropriate. Thus, **changes made by earlier IFTHEN clauses are "seen" by later IFTHEN clauses.** For example, if you have a 40-byte input record and specify:

```
  OUTFIL IFTHEN=(WHEN=INIT,OVERLAY=(8:8,4,ZD,ADD,+1,TO=ZD,LENGTH=4)),
         IFTHEN=(WHEN=(8,4,ZD,EQ,+27),OVERLAY=(28:C'Yes')),
         IFTHEN=(WHEN=NONE,OVERLAY=(28:C'No'))
```

The WHEN=INIT clause adds 1 to the ZD value and stores it in the IFTHEN record. The WHEN=(8,4,ZD,EQ,+27) clause tests the incremented ZD value in the IFTHEN record rather than the original ZD value in the input record.

The IFTHEN record is adjusted as needed for the records created or changed by the IFTHEN clauses. For fixed-length records, blanks are filled in on the left as needed. For variable-length records, the RDW length is adjusted as needed each time the IFTHEN record is changed.

Missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

DFSORT sets an appropriate LRECL for the OUTFIL output records based on the build, overlay, find/replace and group operation items specified by the IFTHEN clauses. However, DFSORT does not analyze the possible results of WHEN=(logexp) conditions when determining an appropriate LRECL. When you use OUTFIL IFTHEN clauses, you can override the OUTFIL LRECL determined by DFSORT with the OUTFIL IFOUTLEN parameter.

If SEQNUM is used in multiple IFTHEN clauses, the sequence number will be incremented for each record that satisfies the IFTHEN clause, that is, a separate SEQNUM counter will be kept for each IFTHEN clause. For example, if your input is:

```
RECORD A 1
RECORD B 1
RECORD B 2
RECORD C 1
RECORD A 2
RECORD C 2
RECORD B 3
RECORD D 1
```

and you specify:

```
  OUTFIL IFTHEN=(WHEN=(8,1,CH,EQ,C'A'),OVERLAY=(15:SEQNUM,4,ZD)),
         IFTHEN=(WHEN=(8,1,CH,EQ,C'B'),OVERLAY=(16:SEQNUM,4,ZD)),
         IFTHEN=(WHEN=NONE,OVERLAY=(17:SEQNUM,4,ZD))
```

your output will be:

```
RECORD A 1    0001
RECORD B 1     0001
RECORD B 2     0002
RECORD C 1      0001
RECORD A 2    0002
RECORD C 2      0002
RECORD B 3     0003
RECORD D 1      0003
```

Separate SEQNUM counters are kept for the 'A' record, for the 'B' record, and for the NONE records.

You can use IFTHEN clauses with the FTOV parameter to convert fixed-length record data sets to variable-length record data sets.

You can use the VLTRIM parameter with IFTHEN clauses to remove specified trailing bytes from the end of variable-length records.

You can use the VLTRAIL parameter with IFTHEN clauses to insert a string at the end of variable-length records.

You can use IFTHEN clauses with any or all of the report parameters (LINES, HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS, BLKCCH1, BLKCCH2, BLKCCT1, and NODETAIL) in the same way as for the BUILD or OUTREC parameter.

The IFTHEN clauses of the OUTREC statement apply to all input records whereas the IFTHEN clauses of the OUTFIL statement only apply to the OUTFIL input records for its OUTFIL group.

WHEN=INIT clause

Specifies build, overlay or find/replace items to be applied to all records. Multiple WHEN=INIT clauses and WHEN=GROUP clauses may be intermixed. They are processed before any other type of IFTHEN clauses and in the order in which they are specified.

WHEN=INIT

Identifies a WHEN=INIT clause.

PARSE=(definitions)

Defines %nn fixed parsed fields into which variable position/length fields are extracted for all records. See OUTFIL PARSE for details. You can use the %nn parsed fields defined in a WHEN=INIT clause in the BUILD or OVERLAY operand of that clause and all subsequent IFTHEN clauses.

Sample Syntax:

```
OUTFIL IFOUTLEN=50,
       IFTHEN=(WHEN=INIT,
        PARSE=(%01=(ABSPOS=11,STARTAT=NONBLANK,
                    ENDBEFR=C',',FIXLEN=12),
               %02=(ENDBEFR=C',',FIXLEN=10),
               %03=(FIXLEN=12))),
       IFTHEN=(WHEN=(5,2,CH,EQ,C'US'),BUILD=(%02,%03)),
       IFTHEN=(WHEN=(5,2,CH,EQ,C'UK'),BUILD=(%01,%02)),
       IFTHEN=(WHEN=NONE,BUILD=(%03,%01))
```

BUILD=(items)

Specifies the build items to be applied to each record. See OUTFIL BUILD for details. You can specify all of the items in the same way that you can specify them for OUTFIL BUILD, except that you cannot specify / to create blank records or new records.

OVERLAY=(items)

Specifies the overlay items to be applied to each record. See OUTFIL OVERLAY for details. You can specify all of the items in the same way that you can specify them for OUTFIL OVERLAY.

Sample Syntax:

```
OUTFIL FNAMES=OUT1,
       IFTHEN=(WHEN=INIT,BUILD=(1,20,21:C'Department',31:3X,21,60)),
       IFTHEN=(WHEN=(5,2,CH,EQ,C'D1'),OVERLAY=(31:8,3)),
       IFTHEN=(WHEN=(5,2,CH,EQ,C'D2'),OVERLAY=(31:12,3))
```

FINDREP=(items)

Specifies find and replace operations to be applied to each record. See OUTFIL FINDREP for details. You can specify all of the items in the same way that you can specify them for OUTFIL FINDREP.

Sample Syntax

```
OUTFIL FNAMES=OUT2,
  IFTHEN=(WHEN=INIT,FINDREP=(IN=X'00',OUT=X'40')),
  IFTHEN=(WHEN=INIT,OVERLAY=(81:SEQNUM,8,ZD))
```

WHEN=GROUP clause

Identifies groups of records in various ways and propagates fields, identifiers and sequence numbers to the records of each group. Fields, identifiers and sequence numbers are not propagated to records before, between or after identified groups. Multiple WHEN=INIT clauses and WHEN=GROUP clauses may be intermixed. They are processed before any other type of IFTHEN clauses and in the order in which they are specified.

You can specify the BEGIN, KEYBEGIN, END and RECORDS operands in any combination to define the groups, but you must specify at least one of these operands.

BEGIN=(logexp)

Specifies the criteria to be tested to determine if a record starts a group. See the INCLUDE statement for details of the logical expressions you can use. You can specify all of the logical expressions in the same way that you can specify them for the INCLUDE statement except that:

- You cannot specify FORMAT=f with BEGIN=(logexp).
- You cannot specify D2 format in BEGIN=(logexp).
- Locale processing is not used for BEGIN=(logexp).
- VLSCMP and VLSHRT are not used with BEGIN=(logexp). Instead, missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

A new group starts with a record that satisfies the BEGIN condition, that is, when the specified logical expression is true for that record. If BEGIN is specified without END or RECORDS, all of the records from the begin record up to but not including the next begin record belong to a group. Here's an example of groups with

```
BEGIN=(1,1,CH,EQ,C'A'):

H
R
A group 1
B group 1
C group 1
A group 2
A group 3
B group 3
```

Example:

```
  OUTFIL IFTHEN=(WHEN=GROUP,
    BEGIN=(1,40,SS,EQ,C'J82',OR,1,40,SS,EQ,C'M72'),
    PUSH=(41:ID=5))
```

Starts a new group each time C'J82' or 'M72' is found anywhere in positions 1-40 of a record. Overlays positions 41-45 of each record of a group with a 5-byte ZD identifier. The records before the first group are not changed.

KEYBEGIN=(p,m)

Specifies a field to be used to determine if a record starts a group. A new group starts when the binary value in the specified key field (p,m) changes. The first input record will start a group. KEYBEGIN=(p,m) operates in a similar way to BEGIN=(logexp) but uses a key change as a trigger rather than a condition to be satisfied. Use KEYBEGIN=(p,m) if you want to start a new group for each consecutive set of key values in your records.

p is the starting position of the key and can be from 1 to 32752.

m is the length of the key and can be from 1 to 256.

If KEYBEGIN=(p,m) is specified, the first input record will start a group.

Here is an example of groups with

```
KEYBEGIN=(1,1)

A group 1
A group 1
A group 1
B group 2
B group 2
C group 3
```

Example:

```
  OUTFIL IFTHEN=(WHEN=GROUP,
     KEYBEGIN=(11,5),PUSH=(81:21,8))
```

Starts a new group each time the value in positions 11-15 changes. Overlays positions 81-88 of each record of a group with positions 21-28 from the first record of that group.

END=(logexp)

Specifies the criteria to be tested to determine if a record ends a group. See the INCLUDE statement for details of the logical expressions you can use. You can specify all of the logical expressions in the same way that you can specify them for the INCLUDE statement except that:

- You cannot specify FORMAT=f with END=(logexp).
- You cannot specify D2 format in END=(logexp).
- Locale processing is not used for END=(logexp).
- VLSCMP and VLSHRT are not used with END=(logexp). Instead, missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

A group ends whenever a record satisfies the END condition, that is, whenever the specified logical expression is true for that record. If END is specified without BEGIN or KEYBEGIN, all of the records up to the end record (or the last record of the data set) belong to a group. Here's an example of groups with

```
END=(1,1,CH,EQ,C'T')

A group 1
B group 1
T group 1
T group 2
A group 3
T group 3
M group 4
```

If END is specified with BEGIN or KEYBEGIN, all of the records from the begin record up to the end record (or the last record of the data set) belong to a group. Here's an example of groups with

```
BEGIN=(1,1,CH,EQ,C'H'),END=(1,1,CH,EQ,C'T')

H group 1
B group 1
T group 1
T
H group 2
T group 2
M
N
H group 3
A group 3
```

Example:

```
  OUTFIL IFTHEN=(WHEN=GROUP,
           BEGIN=(1,2,CH,EQ,C'02',AND,8,3,CH,EQ,C'YES'),
           END=(11,5,CH,EQ,C'PAGE:'),PUSH=(61:SEQ=3))
```

Starts a new group each time C'02' is found in positions 1-2 and C'YES' is found in positions 8-10 of a record. Ends the group when C'PAGE:' is found in positions 11-15 of a record. Overlays positions 61-63 of each record of a group with a 3-byte ZD sequence number. The records between the groups are not changed.

RECORDS=n

Specifies the maximum number of records in a group. n can be 1 to 2000000000. If RECORDS is specified without BEGIN, KEYBEGIN or END, a new group starts after n records. Here's an example of groups with

```
RECORDS=3

H group 1
B group 1
```

```
T group 1
H group 2
B group 2
T group 2
M group 3
N group 3
```

If RECORDS is specified with BEGIN or KEYBEGIN, up to n records starting with the begin record belong to a group. Here's an example of groups with

```
BEGIN=(1,1,CH,EQ,C'H'),RECORDS=3

H group 1
B group 1
T group 1
A
H group 2
B group 2
H group 3
M group 3
N group 3
P
```

The records between and after the groups are not changed.

If RECORDS is specified with END, the group ends after up to n records or with the end record (or the last record of the data set), whichever comes first. Here's an example of groups with

```
BEGIN=(1,1,CH,EQ,C'H'),END=(1,1,CH,EQ,C'T'),RECORDS=4

H group 1
B group 1
T group 1
A
H group 2
B group 2
C group 2
D group 2
E
H group 3
M group 3
```

The records between the groups are not changed.

You must specify the PUSH operand to define at least one field, identifier or sequence number to be added to the records of each group. You can specify any combination or number of fields, identifiers and sequence numbers.

PUSH=(c:item,...)

Specifies the position where each field, identifier or sequence number is to be overlaid in the records of each group.

For fixed-length records, the first input and output data byte starts at position 1. For variable-length records, the first input and output data byte starts at position 5, after the RDW in positions 1-4.

You can use the following in PUSH:

c:

Specifies the output position (column) to be overlaid. If you do not specify c: for the first item, it defaults to 1:. If you do not specify c: for any other item, it starts after the previous item. You can specify items in any order and overlap output columns. c can be 1 to 32752.

If you specify an item that extends the output record beyond the end of the input record, the record length is automatically increased to that length, and blanks are filled in on the left as needed. For variable-length records, the RDW length is increased to correspond to the larger record length after all of the items are processed. Missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

p,m
Specifies a field in the first input record of each group to be propagated to every record of the group. p specifies the starting position of the field in the input record and m specifies its length. A field must not extend beyond position 32752.

ID=n
Specifies a ZD identifier of length n is to be added to every record of each group. The identifier starts at 1 for the first group and is incremented by 1 for each subsequent group. n can be 1 to 15.

SEQ=n
Specifies a ZD sequence number of length n is to be added to every record of each group. The sequence number starts at 1 for the first record of each group and is incremented by 1 for each subsequent record of the group. n can be 1 to 15.

Sample Syntax

```
OUTFIL IFTHEN=(WHEN=GROUP,
         BEGIN=(1,5,CH,EQ,C'DATE:'),
         RECORDS=3,
         PUSH=(15:ID=3,31:21,8))
```

WHEN=(logexp) clause
Specifies build, overlay or find/replace items to be applied to records for which the specified logical expression is true. If multiple WHEN=(logexp) clauses are specified, they are processed in the order in which they are specified.

WHEN=(logexp)
Identifies a WHEN=(logexp) clause and specifies the criteria to be tested to determine if the build, overlay or find/replace items are to be applied to the record. See the INCLUDE statement for details of the logical expressions you can use. You can specify all of the logical expressions in the same way that you can specify them for the INCLUDE statement except that:

- You cannot specify FORMAT=f with WHEN=(logexp).
- You cannot specify D2 format in WHEN=(logexp).
- Locale processing is not used for WHEN=(logexp).
- VLSCMP and VLSHRT are not used with WHEN=(logexp). Instead, missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

PARSE=(definitions)
Defines %nn fixed parsed fields into which variable position/length fields are extracted for each record for which the logical expression is true. See OUTFIL PARSE for details. You can only use the %nn parsed fields defined in a WHEN=(logexp) clause in the BUILD or OVERLAY operand of that WHEN=(logexp) clause.

Sample Syntax:

```
OUTFIL IFTHEN=(WHEN=(45,2,CH,EQ,C'AL'),
         PARSE=(%01=(ENDBEFR=C',',FIXLEN=12),
                %02=(FIXLEN=10)),
         OVERLAY=(21:%02,51:%01))
```

BUILD=(items)
Specifies the build items to be applied to each record for which the logical expression is true. See OUTFIL BUILD for details. You can specify all of the items in the same way that you can specify them for OUTFIL BUILD.

OVERLAY=(items)
Specifies the overlay items to be applied to each record for which the logical expression is true. See OUTFIL OVERLAY for details. You can specify all of the items in the same way that you can specify them for OUTFIL OVERLAY.

FINDREP=(items)
Specifies find and replace operations to be applied to each record for which the logical expression is true. See OUTFIL FINDREP for details. You can specify all of the items in the same way that you can specify them for OUTFIL FINDREP.

Sample Syntax

```
  OUTFIL IFTHEN=(WHEN=(11,2,CH,EQ,C'**'),
    FINDREP=(IN=C'**',OUT=C''))
```

HIT=NEXT
Specifies that IFTHEN processing should continue even if the logical expression is true. By default (if HIT=NEXT is not specified), IFTHEN processing stops if the logical expression is true.

Sample Syntax:

```
OUTFIL FNAMES=OUT2,
  IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,
     18,4,ZD,LE,+2000),OVERLAY=(42:C'Type1 <= 2000'),HIT=NEXT),
  IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,6,1,BI,BO,X'03'),
      BUILD=(1,21,42,13)),
  IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,
     18,4,ZD,GT,+2000),OVERLAY=(42:C'Type1 > 2000 '),HIT=NEXT),
  IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,6,1,BI,BO,X'01'),
      BUILD=(1,25,42,13))
```

WHEN=ANY clause
Specifies build, overlay or find/replace items to be applied to records for which the logical expression in any "preceding" WHEN=(logexp) clause is true. For the first WHEN=ANY clause, the "preceding" WHEN=(logexp) clauses are those before this WHEN=ANY clause. For the second or subsequent WHEN=ANY clause, the "preceding" WHEN=(logexp) clauses are those between the previous WHEN=ANY clause and this WHEN=ANY clause. At least one WHEN=(logexp) clause must be specified before a WHEN=ANY clause. A WHEN=ANY clause can be used without any build, overlay or find/replace items to just stop IFTHEN processing if any preceding WHEN=(logexp) clause is satisfied.

PARSE=(definitions)
Defines %nn fixed parsed fields into which variable position/length fields are extracted for each record for which the logical expression in any preceding WHEN=(logexp) clause is true. See OUTFIL PARSE for details. You can only use the %nn parsed fields defined in a WHEN=ANY clause in the BUILD or OVERLAY operand of that WHEN=ANY clause.

BUILD=(items)
Specifies the build items to be applied to each record for which the logical expression in any preceding WHEN=(logexp) clause is true. See OUTFIL BUILD for details. You can specify all of the items in the same way that you can specify them for OUTFIL BUILD.

OVERLAY=(items)
Specifies the overlay items to be applied to each record for which the logical expression in any preceding WHEN=(logexp) clause is true. See OUTFIL OVERLAY for details. You can specify all of the items in the same way that you can specify them for OUTFIL OVERLAY.

FINDREP=(items)
Specifies find and replace operations to be applied to each record for which the logical expression in any preceding WHEN=(logexp) clause is true. See OUTFIL FINDREP for details. You can specify all of the items in the same way that you can specify them for OUTFIL FINDREP.

HIT=NEXT
Specifies that IFTHEN processing should continue even if the logical expression in a preceding WHEN=(logexp) clause is true. By default (if HIT=NEXT is not specified), IFTHEN processing stops if the logical expression in a preceding WHEN=(logexp) clause is true.

Sample Syntax:

```
OUTFIL FNAMES=OUT3,
 IFTHEN=(WHEN=(1,3,SS,EQ,C'T01,T02,T03'),
           BUILD=(C'Group A',X,1,80),HIT=NEXT),
 IFTHEN=(WHEN=(1,3,SS,EQ,C'T04,T05,T06'),
           BUILD=(C'Group B',X,1,80),HIT=NEXT),
 IFTHEN=(WHEN=(1,3,SS,EQ,C'T07,T08,T09,T10'),
           BUILD=(C'Group C',X,1,80),HIT=NEXT),
 IFTHEN=(WHEN=ANY,OVERLAY=(16:C'Group Found'))
```

WHEN=NONE clause

Specifies build, overlay or find/replace items to be applied to records for which none of the logical expressions in any WHEN=(logexp) clause is true. If there are no WHEN=(logexp) clauses, the build, overlay or find/replace items are applied to all of the records. If multiple WHEN=NONE clauses are specified, they are processed in the order in which they are specified. WHEN=NONE clauses are processed after any other type of IFTHEN clauses.

PARSE=(definitions)

Defines %nn fixed parsed fields into which variable position/length fields are extracted for each record for which no logical expression was true. See OUTFIL PARSE for details. You can only use the %nn parsed fields defined in a WHEN=NONE clause in the BUILD or OVERLAY operand of that WHEN=NONE clause.

Sample Syntax:

```
OUTFIL IFTHEN=(WHEN=(5,2,ZD,GT,+5),
  PARSE=(%01=(STARTAT=C'<',ENDAT=C'>',FIXLEN=12),
         %02=(STARTAFT=BLANKS,FIXLEN=10)),
  BUILD=(5,2,21:%01,X,%02,HEX)),
 IFTHEN=(WHEN=NONE,
  PARSE=(%03=(STARTAFT=C'(',ENDBEFR=C')',FIXLEN=8)),
  BUILD=(5,2,%03,SFF,M12))
```

BUILD=(items)

Specifies the build items to be applied to each record for which no logical expression was true. See OUTFIL BUILD for details. You can specify all of the items in the same way that you can specify them for OUTFIL BUILD.

OVERLAY=(items)

Specifies the overlay items to be applied to each record for which no logical expression was true. See OUTFIL OVERLAY for details. You can specify all of the items in the same way that you can specify them for OUTFIL OVERLAY.

Sample Syntax:

```
OUTFIL FNAMES=OUT4,
  IFTHEN=(WHEN=INIT,BUILD=(1,20,21:C'Department',31:3X,21,60)),
  IFTHEN=(WHEN=(5,2,CH,EQ,C'D1'),OVERLAY=(31:8,3)),
  IFTHEN=(WHEN=(5,2,CH,EQ,C'D2'),OVERLAY=(31:12,3)),
  IFTHEN=(WHEN=NONE,OVERLAY=(31:C'***'))
```

FINDREP=(items)

Specifies find and replace operations to be applied to each record for which no logical expression was true. See OUTFIL FINDREP for details. You can specify all of the items in the same way that you can specify them for OUTFIL FINDREP.

Default for IFTHEN clauses: None; must be specified.

IFOUTLEN



Overrides the OUTFIL LRECL determined by DFSORT from your OUTFIL IFTHEN clauses. DFSORT sets an appropriate LRECL for the output records based on the build, overlay, find/replace and group operation items specified by the IFTHEN clauses. However, DFSORT does not analyze the possible results of WHEN=(logexp) conditions when determining an appropriate OUTFIL LRECL. When you use

OUTFIL IFTHEN clauses, you can override the OUTFIL LRECL determined by DFSORT with the OUTFIL IFOUTLEN parameter.

Fixed-length records longer than the IFOUTLEN length are truncated to the IFOUTLEN length. Fixed-length records shorter than the IFOUTLEN are padded with blanks to the IFOUTLEN length. Variable-length records longer than the IFOUTLEN length are truncated to the IFOUTLEN length.

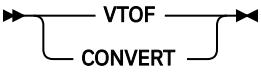
n
specifies the length to use for the OUTFIL LRECL. The value for n must be between 1 and 32767, but must not be larger than the maximum LRECL allowed for the RECFM, and must not conflict with the specified or retrieved LRECL for the fixed-length OUTFIL data set.

Sample Syntax:

```
OUTFIL FNAMES=OUT5,IFOUTLEN=70,
  IFTHEN=(WHEN=(5,1,CH,EQ,C'1',AND,8,3,ZD,EQ,+10),
    BUILD=(1,40,C'T01-GROUP-A',65)),
  IFTHEN=(WHEN=(5,1,CH,EQ,C'2',AND,8,3,ZD,EQ,+12),
    BUILD=(1,40,C'T02-GROUP-B',65))
```

Default for IFOUTLEN: The LRECL determined from the IFTHEN clauses.

VTOF or CONVERT



Specifies that variable-length OUTFIL input records are to be converted to fixed-length OUTFIL output records for this OUTFIL group.

You must specify a BUILD or OUTREC parameter. The items you specify produce a reformatted fixed-length OUTFIL output record without an RDW (the data starts at position 1). Any BUILD or OUTREC fields you specify apply to the variable-length OUTFIL input records (the data starts at position 5 after the 4-byte RDW). However, you cannot specify the variable-part of the OUTFIL input records (for example, p or p,HEX). Any BUILD or OUTREC columns you specify apply to the reformatted fixed-length OUTFIL output records.

By default, VTOF or CONVERT automatically uses VLFILL=X'40' (blank fill byte) to allow processing of variable-length input records which are too short to contain all specified BUILD or OUTREC fields. You can specify VLFILL=byte to change the fill byte.

If you do not specify a RECFM for the OUTFIL data set, it will be given a record format of FB.

If you specify a RECFM for the OUTFIL data set, it must have a fixed-length record format (for example, FB).

If VTOF or CONVERT is specified for fixed-length input records, it will not be used.

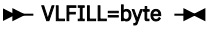
If VTOF or CONVERT is specified with FTOV, IFTRAIL, IFTHEN, FINDREP or OVERLAY, DFSORT will terminate.

Sample Syntax:

```
OUTFIL FNAMES=FIXOUT,VTOF,
  OUTREC=(1:5,14,35:32,8,50:22,6,7c'*')
```

Default for VTOF or CONVERT: None; must be specified.

VLFILL



Allows DFSORT to continue processing if a variable-length OUTFIL input record is found to be too short to contain all specified OUTFIL BUILD or OUTREC fields for this OUTFIL group. Without VLFILL=byte, a short record causes DFSORT to issue message ICE218A and terminate. With

VLFILL=byte, missing bytes in OUTFIL BUILD or OUTREC fields are replaced with fill bytes so the filled fields can be processed.

If VLFILL=byte is specified for fixed-length input records, it will not be used.

If VLFILL=byte is specified with FTOV, IFTRAIL, IFTHEN, FINDREP or OVERLAY, DFSORT will terminate.

byte
specifies the fill byte. Permissible values are C'x' and X'yy'.

C'x'
Character byte: The value x must be one EBCDIC character. If you want to use an apostrophe as the fill byte, you must specify it as C''''.

X'yy'
Hexadecimal byte: The value yy must be one pair of hexadecimal digits (00-FF).

Sample Syntax:

```
OUTFIL FNAMES=FIXOUT,VTOF,OUTREC=(5,20,2X,35,10),VLFILL=C'*'
OUTFIL FNAMES=OUT1,VLFILL=X'FF',OUTREC=(1,4,15,5,52)
```

Default for VLFILL: VLFILL=X'40' (blank fill byte) if VTOF or CONVERT is specified. Otherwise, none; must be specified.

FTOV



Specifies that fixed-length OUTFIL input records are to be converted to variable-length OUTFIL output records for this OUTFIL group.

If you do not specify an OUTREC, BUILD, OVERLAY, FINDREP or IFTHEN parameter, the fixed-length OUTFIL input record is converted to a variable-length OUTFIL output record. A 4-byte RDW is prepended to the fixed-length record before it is written.

If you specify an OUTREC, BUILD, OVERLAY, FINDREP or IFTHEN parameter, the items you specify produce a reformatted fixed-length record that is converted to a variable-length OUTFIL output record. Any OUTREC, BUILD, OVERLAY, FINDREP or IFTHEN fields you specify apply to the fixed-length OUTFIL input records (the data starts at position 1). A 4-byte RDW is prepended to the reformatted fixed-length record before it is written.

If you do not specify a RECFM for the OUTFIL data set, it will be given a record format of VB.

If you specify a RECFM for the OUTFIL data set, it must have a variable-length record format (for example, VB or VBS).

If you do not specify an LRECL for the OUTFIL data set, it will be given an LRECL that can contain the largest variable-length output record to be produced, up to a maximum of 32756 for an unspanned record format (for example, VB) or up to 32767 for a spanned record format (for example, VBS).

If you specify an LRECL for the OUTFIL data set, it must be big enough to contain the largest variable-length output record to be produced.

If your largest variable-length output record is between 32757 and 32767 bytes, you'll need to specify a spanned record format (for example, VBS) for the output data set.

If FTOV is specified for variable-length input records, it will not be used.

If FTOV is specified with VTOF, IFTRAIL, CONVERT or VLFILL=byte, DFSORT will terminate.

Sample Syntax:

```
OUTFIL FNAMES=VAROUT,FTOV
OUTFIL FNAMES=V1,FTOV,OUTREC=(1,20,26:21,10,6C'*')
OUTFIL FNAMES=V2,FTOV,
```

```
     IFTHEN=(WHEN=(12,5,ZD,GT,+20000),OVERLAY=(25:C'Yes')),
     IFTHEN=(WHEN=NONE,OVERLAY=(25:C'No'))
```

Default for FTOV: None; must be specified.

VLTRIM



VLTRIM=byte specifies that the trailing bytes are to be removed from the end of variable-length OUTFIL output records for this OUTFIL group before the records are written.

The trim byte can be any value, such as blank, binary zero, or asterisk. If DFSORT finds one or more trim bytes at the end of a variable-length OUTFIL data record or report record, it will decrease the length of the record accordingly, effectively removing the trailing trim bytes. However, VLTRIM=byte will not remove the RDW, the ANSI carriage control character (if produced), or the first data byte.

For example, say that you have the following 17-byte fixed-length data records that you want to convert to variable-length data records:

```
123456**********
0003**********
ABCDEFGHIJ*****22
*****************
```

If you use:

```
OUTFIL FTOV
```

the following variable-length output records will be written (4-byte RDW followed by data):

```
Length  |  Data
  21       123456**********
  21       0003**********
  21       ABCDEFGHIJ*****22
  21       *****************
```

but if you use:

```
OUTFIL FTOV,VLTRIM=C'*'
```

the following variable-length output records will be written (4-byte RDW followed by data):

```
Length  |  Data
  10       123456
   8       0003
  21       ABCDEFGHIJ*****22
   5       *
```

VLTRIM=C'*' removed the trailing asterisks from the first and second records. The third record did not have any trailing asterisks to remove. The fourth record had all asterisks, so one asterisk was kept.

If VLTRIM=byte is specified for fixed-length output records, it will not be used.

If VLTRIM is specified with IFTRAIL, DFSORT will terminate.

byte
specifies the trim byte. Permissible values are C'x' and X'yy'.

C'x'
Character byte: The value x must be one EBCDIC character. If you want to use an apostrophe as the trim byte, you must specify it as C''''.

X'yy'
Hexadecimal byte: The value yy must be one pair of hexadecimal digits (00-FF)

Sample Syntax:

```
 Fixed input:
  OUTFIL FNAMES=TRIM1,FTOV,VLTRIM=C' '

 Variable input:
  OUTFIL FNAMES=TRIM2,VLTRIM=X'00'
  OUTFIL FNAMES=TRIM3,VLTRIM=C'*',
     OUTREC=(1,15,5X,16,8,5X,28)
```

Default for VLTRIM: None; must be specified.

VLTRAIL



VLTRAIL=string specifies that the indicated string is to be inserted at the end of variable-length OUTFIL output records for this OUTFIL group before the records are written.

string can be 1 to 50 characters specified using a character string constant (C'xx...x') or a hexadecimal string constant (X'yy...yy'). See “INCLUDE control statement” on page 91 for details of coding character and hexadecimal string constants.

DFSORT will insert the specified string at the end of each variable-length OUTFIL data record or report record, and increase the length of the record accordingly. You must ensure that the LRECL of the OUTFIL data set is large enough to contain the added bytes. If the added bytes increase the length of a record beyond the LRECL for the data set, DFSORT will terminate. To avoid this, you can specify a larger LRECL on the DD statement, when necessary.

For example, say that you have the following variable-length input records:

```
Length | Data
    28   Professional Development
    18   Psychoanalysis
    25   Theory of Computation
```

and you want to add X'0D0A' (CR,LF) to the end of each record. If you use the following OUTFIL statement:

```
  OUTFIL FNAMES=OUT1,VLTRAIL=X'0D0A'
```

X'0D0A' will be inserted at the end of each record and the length of each record will be increased by 2 bytes. If we represent X'0D0A' by CL, the OUT1 records will look like this:

```
Length | Data
    30   Professional DevelopmentCL
    20   PsychoanalysisCL
    27   Theory of ComputationCL
```

Note that if the LRECL of the output data set is less than the maximum output record (30 bytes for this example), we must increase it to at least the maximum (LRECL=30 or greater for this example).

If VLTRIM=byte is used with VLTRAIL=string, the indicated bytes will be trimmed from the end of the record before the indicated string is added. For example, say that you have the following variable-length input records:

```
Length | Data
    30   Abby is a sweet rat*******
    31   Samantha is a curious rat**
    43   Rachel is our oldest rat***************
    29   Cathy is a mischief maker
```

and you want to remove any trailing asterisks, and add '|March 21|' at the end of each record. If you use the following OUTFIL statement:

```
  OUTFIL FNAMES=OUT2,VLTRIM=C'*',VLTRAIL=C'|March 21|'
```

The trailing asterisks will be removed, 'March 21' will be inserted, and the length of each record will be updated appropriately. The OUT2 records will look like this:

```
Length | Data
    33   Abby is a sweet rat|March 21|
    39   Samantha is a curious rat|March 21|
    38   Rachel is our oldest rat|March 21|
    39   Cathy is a mischief maker|March 21|
```

If VLTRAIL=string is specified for fixed-length output records, it will not be used.

If VLTRAIL is specified with IFTRAIL, DFSORT will terminate.

Sample Syntax:

```
Variable input:
  OUTFIL FNAMES=ADD1,VLTRIM=X'00',VLTRAIL=C'$$ADD$$'
  OUTFIL FNAMES=(ADD2A,ADD2B),VLTRAIL=X'FFFF',SPLIT

Fixed input:
   OUTFIL FNAMES=ADD3,FTOV,VLTRIM=C' ',VLTRAIL=C'end'
```

Default for VLTRAIL: None; must be specified.

REPEAT



Specifies the number of times each OUTFIL output record is to be repeated for this OUTFIL group. Each OUTFIL output record is written n times.

If SEQNUM is used in the OUTREC, BUILD, OVERLAY, or IFTHEN parameter for this OUTFIL group, the sequence number will be incremented for each repeated record. For example, if your input is:

```
RECORD A
RECORD B
```

and you specify:

```
  OUTFIL OUTREC=(1,8,X,SEQNUM,5,ZD),REPEAT=2
```

your output will be:

```
RECORD A 00001
RECORD A 00002
RECORD B 00003
RECORD B 00004
```

If SEQNUM is used in multiple IFTHEN clauses for this OUTFIL group, the sequence number will be incremented for each repeated record that satisfies the IFTHEN clause. For example, if your input is:

```
RECORD A 1
RECORD B 1
RECORD C 1
RECORD A 2
RECORD C 2
RECORD B 2
RECORD B 3
```

and you specify:

```
  OUTFIL REPEAT=2,
    IFTHEN=(WHEN=(8,1,CH,EQ,C'A'),OVERLAY=(15:SEQNUM,4,ZD)),
    IFTHEN=(WHEN=(8,1,CH,EQ,C'B'),OVERLAY=(15:SEQNUM,4,ZD)),
    IFTHEN=(WHEN=NONE,OVERLAY=(15:SEQNUM,4,ZD))
```

your output will be:

```
RECORD A 1    0001
RECORD A 1    0002
RECORD B 1    0001
RECORD B 1    0002
RECORD C 1    0001
RECORD C 1    0002
RECORD A 2    0003
RECORD A 2    0004
RECORD C 2    0003
RECORD C 2    0004
RECORD B 2    0003
RECORD B 2    0004
RECORD B 3    0005
RECORD B 3    0006
```

If you specify REPEAT=n with / in the OUTREC, BUILD, or IFTHEN BUILD parameter for this OUTFIL group, the first line is written n times, then the second line is written n times, and so on. (For IFTHEN, this means each record that has the SEQNUM item applied to it.) If SEQNUM is used, all lines for the same record are given the same sequence number. For example, if your input is:

```
RECORD A
RECORD B
```

and you specify:

```
 OUTFIL OUTREC=(C'P1>',X,1,6,X,SEQNUM,4,ZD,/,
   C'P2>',X,8,1,X,SEQNUM,4,ZD),REPEAT=2
```

your output will be:

```
P1> RECORD 0001
P1> RECORD 0002
P2> A 0001
P2> A 0002
P1> RECORD 0003
P1> RECORD 0004
P2> B 0003
P2> B 0004
```

The REPEAT parameter cannot be used with any of the following parameters: IFTRAIL, LINES, HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS, and NODETAIL.

n
specifies the number of times each OUTFIL output record is to be repeated. The value for n starts at 2 (write record twice) and is limited to 28 digits (15 significant digits).

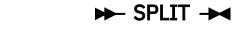
Sample Syntax:

```
* WRITE EACH OUTPUT RECORD 12 TIMES.
  OUTFIL FNAMES=OUT1,REPEAT=12

* WRITE EACH INCLUDED AND REFORMATTED OUTPUT RECORD 50 TIMES.
* (THE SEQUENCE NUMBER WILL BE INCREMENTED FOR EACH REPETITION.)
  OUTFIL FNAMES=OUT2,INCLUDE=(5,2,SS,EQ,C'B2,C5,M3'),
```

Default for REPEAT: None; must be specified.

SPLIT



Splits the output records one record at a time in rotation among the data sets of this OUTFIL group until all of the output records have been written. As a result, the records will be split as evenly as possible among all of the data sets in the group.

As an example, for an OUTFIL group with three data sets:

- the first OUTFIL data set in the group will receive records 1, 4, 7, and so on.

- the second OUTFIL data set in the group will receive records 2, 5, 8, and so on.
- the third OUTFIL data set in the group will receive records 3, 6, 9, and so on.

The records are **not** contiguous in each OUTFIL data set.

SPLIT is equivalent to SPLITBY=1.

The SPLIT parameter cannot be used with any of the following parameters: IFTRAIL, LINES, HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS, and NODETAIL.

Sample Syntax:

```
* WRITE RECORD 1 TO PIPE1, RECORD 2 TO PIPE2, RECORD 3 TO PIPE3,
* RECORD 4 TO PIPE4, RECORD 5 TO PIPE1, RECORD 6 TO PIPE2, AND SO ON.
  OUTFIL FNAMES=(PIPE1,PIPE2,PIPE3,PIPE4),SPLIT

* SPLIT THE INCLUDED AND REFORMATTED OUTPUT RECORDS EVENLY BETWEEN
* TAPE1 AND TAPE2.
  OUTFIL FNAMES=(TAPE1,TAPE2),SPLIT,
    INCLUDE=(8,2,ZD,EQ,27),OUTREC=(5X,1,75)
```

Default for SPLIT: None; must be specified.

SPLITBY



Splits the output records n records at a time in rotation among the data sets of this OUTFIL group until all of the output records have been written.

As an example, if SPLITBY=10 is specified for an OUTFIL group with three data sets:

- the first OUTFIL data set in the group will receive records 1-10, 31-40, and so on.
- the second OUTFIL data set in the group will receive records 11-20, 41-50, and so on.
- the third OUTFIL data set in the group will receive records 21-30, 51-60, and so on.

The records are **not** contiguous in each OUTFIL data set.

SPLITBY=1 is equivalent to SPLIT.

The SPLITBY parameter cannot be used with any of the following parameters: IFTRAIL, LINES, HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS, and NODETAIL.

n
specifies the number of records to split by. The value for n starts at 1 and is limited to 28 digits (15 significant digits).

Sample Syntax:

```
* WRITE RECORDS 1-5 TO PIPE1, RECORDS 6-10 TO PIPE2, RECORDS 11-15 TO
* PIPE3, RECORDS 16-20 TO PIPE4, RECORDS 21-25 TO PIPE1, RECORDS 26-30
* TO PIPE2, AND SO ON.
   OUTFIL FNAMES=(PIPE1,PIPE2,PIPE3,PIPE4),SPLITBY=5

* SPLIT THE INCLUDED AND REFORMATTED OUTPUT RECORDS 100 AT A TIME
* BETWEEN TAPE1 AND TAPE2.
   OUTFIL FNAMES=(TAPE1,TAPE2),SPLITBY=100,
     INCLUDE=(8,2,ZD,EQ,27),OUTREC=(5X,1,75)
```

Default for SPLITBY: None; must be specified.

SPLIT1R



Splits the output records n records at a time for one rotation among the data sets of this OUTFIL group until all of the output records have been written.

As an example, if SPLIT1R=10 is specified for an input data set with 35 records and an OUTFIL group with three data sets:

- the first OUTFIL data set in the group will receive records 1-10.
- the second OUTFIL data set in the group will receive records 11-20.
- the third OUTFIL data set in the group will receive records 21-35.

The records **are** contiguous in each OUTFIL data set.

The SPLIT1R parameter cannot be used with any of the following parameters: IFTRAIL, LINES, HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS, and NODETAIL.

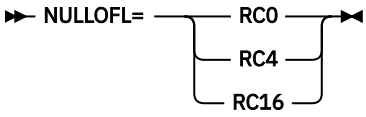
n
specifies the number of records to split by. The value for n starts at 1 and is limited to 28 digits (15 significant digits).

Sample Syntax:

```
  * WRITE RECORDS 1-20 TO PIPE1, RECORDS 21-40 TO PIPE2,
  * RECORDS 41-60 TO PIPE3 AND RECORDS 61-85 TO PIPE4.
     OUTFIL FNAMES=(PIPE1,PIPE2,PIPE3,PIPE4),SPLIT1R=20
  * SPLIT THE INCLUDED AND REFORMATTED OUTPUT RECORDS ONCE
  * CONTIGUOUSLY BETWEEN TAPE1 AND TAPE2.
     OUTFIL FNAMES=(TAPE1,TAPE2),SPLIT1R=100,
      INCLUDE=(8,2,ZD,EQ,27),OUTREC=(5X,1,75)
```

Default for SPLIT1R: None; must be specified.

NULLOFL



specifies the action to be taken by DFSORT when there are no data records for a data set associated with this OUTFIL statement, as indicated by a DATA count of 0 in message ICE227I. OUTFIL report records have no affect on the action taken as a result of this option.

RC0
specifies that DFSORT should issue message ICE174I, set a return code of 0, and continue processing when there are no data records for a data set associated with this OUTFIL statement.

RC4
specifies that DFSORT should issue message ICE174I, set a return code of 4, and continue processing when there are no data records for the a data set associated with this OUTFIL statement.

RC16
specifies that DFSORT should issue message ICE209A, terminate, and give a return code of 16 when there are no data records for a data set associated with this OUTFIL statement.

Default for NULLOFL: The NULLOFL installation default.

Note:

1. The NULLOFL value specified for each OUTFIL statement applies to the data sets associated with that OUTFIL statement. If a NULLOFL value is not specified for an OUTFIL statement, the NULLOFL installation default value applies to the data sets associated with that OUTFIL statement. For example, if the installation default is NULLOFL=RC0 (IBM's shipped default) and the following is specified:

```
OUTFIL FNAMES=OUT1,NULLOFL=RC16,INCLUDE=(5,3,CH,EQ,C'D01')
OUTFIL FNAMES=(OUT2,OUT3),INCLUDE=(5,3,CH,EQ,C'D02')
OUTFIL FNAMES=OUT4,NULLOFL=RC4,SAVE
```

then NULLOFL=RC16 applies to the data set for OUT1, NULLOFL=RC0 (the installation default) applies to the data sets for OUT2 and OUT3, and NULLOFL=RC4 applies to the data set for OUT4.

2. If you receive message ICE174I or ICE209A, a DATA count of 0 in message ICE227I identifies an OUTFIL data set for which there are no data records.
3. The return code of 0 or 4 resulting from NULLOFL=RC0 or NULLOFL=RC4, respectively, can be overridden by a higher return code set for some other reason, including a return code of 16 resulting from NULLOFL=RC16.
4. NULLOUT applies to the SORTOUT data set. NULLOFL applies to OUTFIL data sets

LINES



Specifies the number of lines per page to be used for the reports produced for this OUTFIL group. DFSORT uses ANSI carriage control characters to control page ejects and the placement of the lines in your report, according to your specifications.

The LINES parameter cannot be used with the IFTRAIL parameter.

n
specifies the number of lines per page. The value for n must be between 1 and 255. However, n--or the default for n if LINES is not specified--must be greater than or equal to the number of lines needed for each of the following:

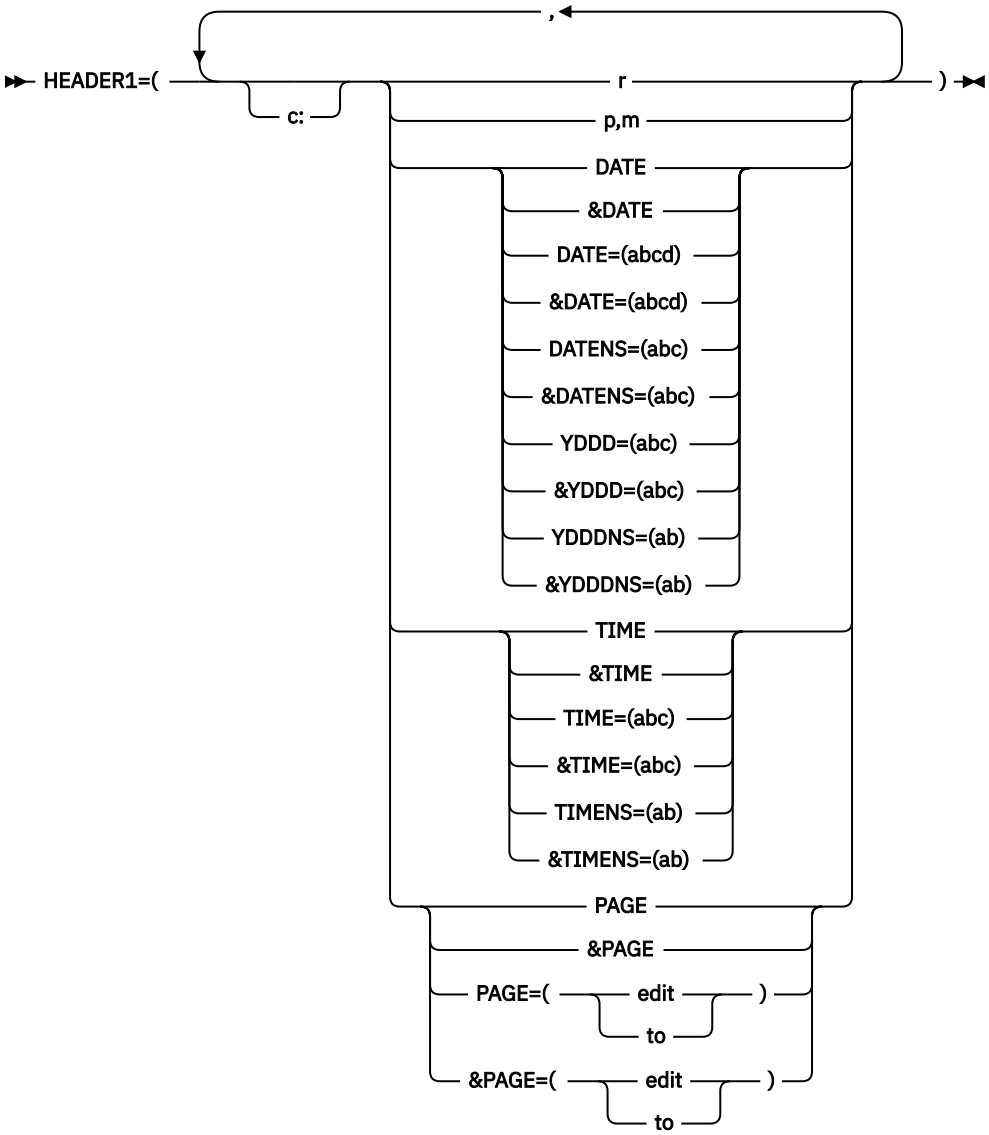
- The HEADER1 lines
- The TRAILER1 lines
- The sum of all lines for HEADER2, TRAILER2, HEADER3s, TRAILER3s, and the data lines and blank lines produced from an input record.

Sample Syntax:

```
OUTFIL FNAMES=RPT1,LINES=50
```

Default for LINES: None; must be specified, unless HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS, or NODETAIL is specified, in which case the default for LINES is 60.

HEADER1



Specifies the report header to be used for the reports produced for this OUTFIL group. The report header appears by itself as the first page of the report. DFSORT uses ANSI carriage control characters to control page ejects and the placement of the lines in your report, according to your specifications. You can use BLKCCH1 to replace the '1' (page eject) for the ANSI carriage control character in the first line of the report header with a blank, thus avoiding forcing a page eject. You can use REMOVECC to remove the ANSI carriage control characters from a report.

You can choose to include any or all of the following report elements in your report header:

- Blanks, character strings, and hexadecimal strings
- Unedited input fields from the first OUTFIL input record
- Current date in a variety of different forms
- Current time in a variety of different forms
- Page number, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on.

The HEADER1 parameter cannot be used with the IFTRAIL parameter.

The report header consists of the elements you select, in the order in which you specify them, and in the columns or lines you specify.

c:

specifies the column in which the first position of the associated report element is to appear, relative to the start of the data in the report record. Ignore the RDW (variable-length report records only) and carriage control character when specifying c:. That is, 1: indicates the first byte of the data in the report record for both fixed-length and variable-length report records.

Unused space preceding the specified column is padded with EBCDIC blanks. The following rules apply:

- c must be a number between 1 and 32752.
- c: must be followed by a report element, but must not precede / or n/.
- c must not overlap the previous report element in the report record.
- The colon (:) is treated like the comma (,) or semicolon (;) for continuation to another line.

r

specifies that blanks, a character string, or a hexadecimal string are to appear in the report record, or that a new report record is to be started in the header, with or without intervening blank lines. These report elements can be specified before or after any other report elements. Consecutive report elements can be specified. Permissible values are nX, n'xx...x', nC'xx...x', nX'yy...yy', /.../ and n/.

nX

Blanks. n bytes of EBCDIC blanks (X'40') are to appear in the report record. n can range from 1 to 4095. If n is omitted, 1 is used.

n'xx...x'

Character string. n repetitions of the character string constant ('xx...x') are to appear in the report record. n can range from 1 to 4095. If n is omitted, 1 is used. x can be any EBCDIC character. You can specify 1 to 256 characters.

nC'xx...x' can be used instead of n'xx...x'.

If you want to include a single apostrophe in the character string, you must specify it as two single apostrophes:

```
Required:  O'NEILL     Specify:  'O''NEILL' or C'O''NEILL'
```

nX'yy...yy'

Hexadecimal string. n repetitions of the hexadecimal string constant (X'yy...yy') are to appear in the report record. n can range from 1 to 4095. If n is omitted, 1 is used.

The value yy represents any pair of hexadecimal digits. You can specify from 1 to 256 pairs of hexadecimal digits

/.../ or n/

Blank lines or a new line. A new report record is to be started in the header with or without intervening blank lines. If /.../ or n/ is specified at the beginning or end of the header, n blank lines are to appear in the header. If /.../ or n/ is specified in the middle of the header, n-1 blank lines are to appear in the header (thus, / or 1/ indicates a new line with no intervening blank lines).

Either n/ (for example, 5/) or multiple /'s (for example, /////) can be used. n can range from 1 to 255. If n is omitted, 1 is used.

As an example, if you specify:

```
  OUTFIL HEADER1=(2/,'First line of text',/,
                     'Second line of text',2/,
                     'Third line of text',2/)
```

the report header appears as follows when printed:

```
blank line
blank line
First line of text
Second line of text
blank line
Third line of text
blank line
blank line
```

p,m
specifies that an unedited input field, from the first OUTFIL input record for which a data record appears in the report, is to appear in the report record.

p
specifies the first byte of the input field relative to the beginning of the OUTFIL input record. The first data byte of a fixed-length record has relative position 1. The first data byte of a variable-length record has relative position 5, because the first four bytes are occupied by the RDW. All fields must start on a byte boundary, and no field can extend beyond byte 32752. See "OUTFIL statements notes" on page 358 for special rules concerning variable-length records.

m
specifies the length in bytes of the input field. The value for m must be between 1 and 32752.

DATE
specifies that the current date is to appear in the report record in the form 'mm/dd/yy', where mm represents the month (01-12), dd represents the day (01-31), and yy represents the last two digits of the year (for example, 95).

&DATE
can be used instead of DATE.

DATE=(abcd)
specifies that the current date is to appear in the report record in the form 'adbdc', where a, b, and c indicate the order in which the month, day, and year are to appear and whether the year is to appear as two or four digits, and d is the character to be used to separate the month, day and year.

For a, b, and c, use M to represent the month (01-12), D to represent the day (01-31), Y to represent the last two digits of the year (for example, 05), or 4 to represent the four digits of the year (for example, 2005). M, D, and Y or 4 can each be specified only once. Examples: DATE=(DMY.) would produce a date of the form 'dd.mm.yy', which on March 29, 2005, would appear as '29.03.05'. DATE=(4MD-) would produce a date of the form 'yyyy-mm-dd', which on March 29, 2005, would appear as '2005-03-29'.

a, b, c, and d must be specified.

&DATE=(abcd)
can be used instead of DATE=(abcd).

DATENS=(abc)
specifies that the current date is to appear in the report record in the form 'abc', where a, b and c indicate the order in which the month, day, and year are to appear and whether the year is to appear as two or four digits.

For a, b and c, use M to represent the month (01-12), D to represent the day (01-31), Y to represent the last two digits of the year (for example, 02), or 4 to represent the four digits of the year (for example, 2002). M, D, and Y or 4 can each be specified only once. Examples: DATENS=(DMY) would produce a date of the form 'ddmmyy', which on March 29, 2002, would appear as '290302'. DATENS=(4MD) would produce a date of the form 'yyyymmdd', which on March 29, 2002, would appear as '20020329'.

a, b and c must be specified.

&DATENS=(abc)
can be used instead of DATENS=(abc).

YDDD=(abc)
specifies that the current date is to appear in the report record in the form 'acb', where a and b indicate the order in which the year and day of the year are to appear and whether the year is to appear as two or four digits, and c is the character to be used to separate the year and day of the year.

For a and b, use D to represent the day of the year (001-D366), Y to represent the last two digits of the year (for example, 04), or 4 to represent the four digits of the year (for example, 2004). D, and Y or 4 can each be specified only once. Examples: YDDD=(DY-) would produce a date of the form 'ddd-yy' which on April 7, 2004, would appear as '098-04'. YDDD=(4D/) would produce a date of the form 'yyyy/ddd' which on April 7, 2004, would appear as '2004/098'.

a, b and c must be specified.

&YDDD=(abc)
can be used instead of YDDD=(abc).

YDDDNS=(ab)
specifies that the current date is to appear in the report record in the form 'ab', where a and b indicate the order in which the year and day of the year are to appear and whether the year is to appear as two or four digits.

For a and b, use D to represent the day of the year (001-366), Y to represent the last two digits of the year (for example, 04), or 4 to represent the four digits of the year (for example, 2004). D, and Y or 4 can each be specified only once. Examples: YDDDNS=(DY) would produce a date of the form 'dddyy' which on April 7, 2004, would appear as '09804'. YDDDNS=(4D) would produce a date of the form 'yyyyddd' which on April 7, 2004, would appear as '2004098'.

a and b must be specified.

&YDDDNS=(ab)
can be used instead of YDDDNS=(ab).

TIME
specifies that the current time is to appear in the report record in the form 'hh:mm:ss', where hh represents the hour (00-23), mm represents the minutes (00-59), and ss represents the seconds (00-59).

&TIME
can be used instead of TIME.

TIME=(abc)
specifies that the current time is to appear in the report record in the form 'hhcmmcss' (24-hour time) or 'hhcmmcss xx' (12-hour time).

If ab is 24, the time is to appear in the form 'hhcmmcss' (24-hour time) where hh represents the hour (00-23), mm represents the minutes (00-59), ss represents the seconds (00-59), and c is the character used to separate the hours, minutes, and seconds. Example: TIME=(24.) would produce a time of the form 'hh.mm.ss' which at 08:25:13 pm would appear as '20.25.13'.

If ab is 12, the time is to appear in the form 'hhcmmcss xx' (12-hour time) where hh represents the hour (01-12), mm represents the minutes (00-59), ss represents the seconds (00-59), xx is am or pm, and c is the character used to separate the hours, minutes, and seconds. Example: TIME=(12.) would produce a time of the form 'hh.mm.ss xx' which at 08:25:13 pm would appear as '08.25.13 pm'.

ab and c must be specified.

&TIME=(abc)
can be used instead of TIME=(abc).

TIMENS=(ab)
specifies that the current time is to appear in the report record in the form 'hhmmss' (24-hour time) or 'hhmmss xx' (12-hour time).

If ab is 24, the time is to appear in the form 'hhmmss' (24-hour time) where hh represents the hour (00-23), mm represents the minutes (00-59), and ss represents the seconds (00-59). Example: TIMENS=(24) would produce a time of the form 'hhmmss' which at 08:25:13 pm would appear as '202513'.

If ab is 12, the time is to appear in the form 'hhmmss xx' (12-hour time) where hh represents the hour (01-12), mm represents the minutes (00-59), and ss represents the seconds (00-59). Example: TIMENS=(12) would produce a time of the form 'hhmmss xx' which at 08:25:13 pm would appear as '082513 pm'.

ab must be specified.

&TIMENS=(ab)
can be used instead of TIMENS=(ab).

PAGE
specifies that the page number is to appear in the report record. The page number for the report header appears as ' 1'.

If HEADER1 is specified with PAGE, PAGE for the report header (first page) will be ' 1' and PAGE for the next page (second page) will be ' 2'. If HEADER1 is specified without PAGE, PAGE for the page after the report header (second page) will be ' 1' (typical of a report with a cover sheet).

&PAGE
can be used instead of PAGE.

PAGE=(edit) or PAGE=(to)
same as PAGE except that the 15-digit page number appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use.

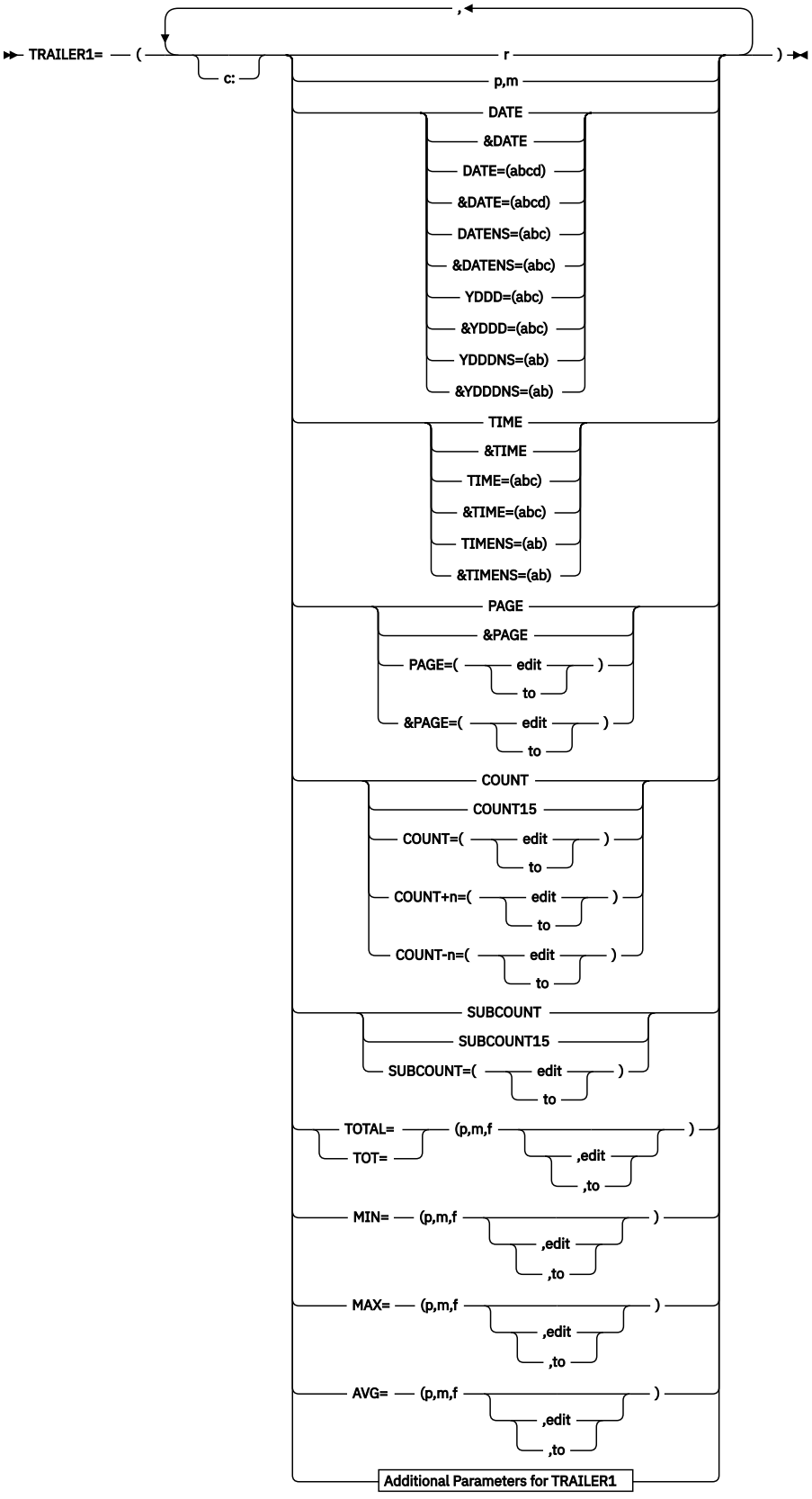
&PAGE=(edit) or &PAGE=(to)
can be used instead of PAGE=(edit) or PAGE=(to).

Sample Syntax:

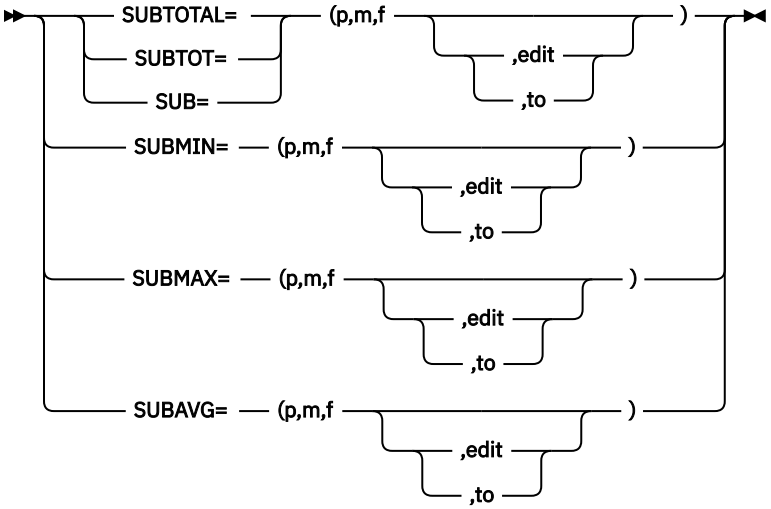
```
OUTFIL FNAMES=(RPT1,RPT2),
   HEADER1=(30:'January Report',4/,
            28:'Prepared on ',DATE,//,
            32:'at ',TIME,//,
            28:'using DFSORT''S OUTFIL',5/,
            10:'Department: ',12,8,50:'Page:',PAGE=(EDIT=(TTT)))
```

Default for HEADER1: None; must be specified.

TRAILER1


TRAILER1=
(
,
c:
r
p,m
DATE
&DATE
DATE=(abcd)
&DATE=(abcd)
DATENS=(abc)
&DATENS=(abc)
YDDD=(abc)
&YDDD=(abc)
YDDDNS=(ab)
&YDDDNS=(ab)
TIME
&TIME
TIME=(abc)
&TIME=(abc)
TIMENS=(ab)
&TIMENS=(ab)
PAGE
&PAGE
PAGE=(edit to)
&PAGE=(edit to)
COUNT
COUNT15
COUNT=(edit to)
COUNT+n=(edit to)
COUNT-n=(edit to)
SUBCOUNT
SUBCOUNT15
SUBCOUNT=(edit to)
TOTAL=
TOT=
(p,m,f ,edit ,to)
MIN= (p,m,f ,edit ,to)
MAX= (p,m,f ,edit ,to)
AVG= (p,m,f ,edit ,to)
Additional Parameters for TRAILER1
)

Additional Parameters for TRAILER1



Specifies the report trailer to be used for the reports produced for this OUTFIL group. The report trailer appears by itself as the last page of the report. DFSORT uses ANSI carriage control characters to control page ejects and the placement of the lines in your report, according to your specifications. You can use BLKCCT1 to replace the '1' (page eject) for the ANSI carriage control character in the first line of the report trailer with a blank, thus avoiding forcing a page eject. You can use REMOVECC to remove the ANSI carriage control characters from a report.

You can choose to include any or all of the following report elements in your report trailer:

- Blanks, character strings, and hexadecimal strings
- Unedited input fields from the last OUTFIL input record
- Current date in a variety of different forms
- Current time in a variety of different forms
- Page number, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on
- Any or all of the following statistics:
  - Count of data records in the report, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on. You can add a decimal number to the count before converting or editing it (for example, add 1 to account for writing a trailer record, or add 2 to account for writing a header and trailer record).
  - Total, minimum, maximum, or average for each specified ZD, PD, BI, FI, FL, CSF, FS, UFF, or SFF numeric input field in the data records of the report, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on.

The TRAILER1 parameter cannot be used with the IFTRAIL parameter.

The report trailer consists of the elements you select, in the order in which you specify them, and in the columns or lines you specify.

c:
See c: under HEADER1.

r
specifies that blanks, a character string, or a hexadecimal string are to appear in the report record, or that a new report record is to be started in the trailer, with or without intervening blank lines. These report elements can be specified before or after any other report elements. Consecutive report elements can be specified. Permissible values are nX, n'xx...x', nC'xx...x', nX'yy...yy', /.../, and n/.

nX
Blanks. See nX under r for HEADER1.

n'xx...x'
Character string. See n'xx...x' under r for HEADER1. nC'xx...x' can be used instead of n'xx...x'

nX'xx...x'
Hexadecimal string. See nX'xx...x' under r for HEADER1.

/.../ or n/
Blank lines or a new line. A new report record is to be started in the trailer, with or without intervening blank lines. If /.../ or n/ is specified at the beginning or end of the trailer, n blank lines are to appear in the trailer. If /.../ or n/ is specified in the middle of the trailer, n-1 blank lines are to appear in the trailer (thus, / or 1/ indicates a new line with no intervening blank lines).

Either n/ (for example, 5/) or multiple /'s (for example, /////) can be used. n can range from 1 to 255. If n is omitted, 1 is used.

p,m
specifies that an unedited input field, from the last OUTFIL input record for which a data record appears in the report, is to appear in the report record.

p
See p under HEADER1.

m
See m under HEADER1.

DATE
See DATE under HEADER1.

&DATE
can be used instead of DATE.

DATE=(abcd)
See DATE=(abcd) under HEADER1.

&DATE=(abcd)
can be used instead of DATE=(abcd).

DATENS=(abc)
See DATENS=(abc) under HEADER1.

&DATENS=(abc)
can be used instead of DATENS=(abc).

YDDD=(abc)
See YDDD=(abc) under HEADER1.

&YDDD=(abc)
can be used instead of YDDD=(abc).

YDDDNS=(ab)
See YDDDNS=(ab) under HEADER1.

&YDDDNS=(ab)
can be used instead of YDDDNS=(ab).

TIME
See TIME under HEADER1.

&TIME
can be used instead of TIME.

TIME=(abc)
See TIME=(abc) under HEADER1.

&TIME=(abc)
can be used instead of TIME=(abc).

TIMENS=(ab)
See TIMENS=(ab) under HEADER1.

&TIMENS=(ab)
can be used instead of TIMENS=(ab).

PAGE
specifies that the current page number is to appear in the report record. The page number for the trailer appears as 6 digits, right-justified, with leading zeros suppressed. For example, if the page is numbered 12, it appears as '   12'.

&PAGE
can be used instead of PAGE.

PAGE=(edit) or PAGE=(to)
same as PAGE except that the 15-digit page number appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use.

&PAGE=(edit) or &PAGE=(to)
can be used instead of PAGE=(edit) or PAGE=(to).

COUNT
specifies that the count of data records in the report is to appear in the report record as 8 digits, right-justified, with leading zeros suppressed. For example, if there are 6810 input records in the report, the count appears as '   6810'.

If slash (/) is used in OUTREC or BUILD to produce multiple data records, COUNT only counts the number of data records processed as input to OUTREC or BUILD. For example, if OUTREC processes 3 input records and creates 2 output records for each input record, the count is 3, not 6

COUNT15
same as COUNT except that the count appears as 15 digits.

COUNT=(edit) or COUNT=(to)
same as COUNT except that the 15–digit count appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use. For example, if there are 6810 input records, COUNT=(M11,LENGTH=6) produces a count of '006810'.

COUNT+n=(edit) or COUNT+n=(to)
same as COUNT=(edit) or COUNT=(to) except that n is added to the 15-digit count before it is edited or converted. n can be 1 to 3 decimal digits. For example, if there are 6810 input records, COUNT+2=(M11,LENGTH=6) produces a count of '006812'. One important use for this parameter is to add 1 for the TRAILER1 record to the count of data records.

COUNT-n=(edit) or COUNT-n=(to)
same as COUNT=(edit) or COUNT=(to) except that n is subtracted from the 15-digit count before it is edited or converted. n can be 1 to 3 decimal digits. For example, if there are 6810 input records, COUNT-1=(M11,LENGTH=6) produces a count of '006809'. One important use for this parameter is to subtract 1 for a header record from the count of all records.

SUBCOUNT
specifies that the running count of input records in the report is to appear in the report record as 8 digits, right-justified, with leading zeros suppressed.

For TRAILER1, the running count is the same as the count, so SUBCOUNT produces the same value as COUNT.

If slash (/) is used in OUTREC or BUILD to produce multiple data records, SUBCOUNT counts only the number of data records processed as input to OUTREC or BUILD. For example, if OUTREC processes 3 input records and creates 2 output records for each input record, the running count is 3, not 6.

SUBCOUNT15
same as SUBCOUNT except that the running count appears as 15 digits.

SUBCOUNT=(edit) or SUBCOUNT=(to)
same as SUBCOUNT except that the 15–digit running count appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use.

TOTAL
specifies that an edited or converted total, for the values of a numeric input field in all data records of the report, is to appear in the report record.

TOT can be used instead of TOTAL.

p,m,f,edit or p,m,f,to
specifies the numeric input field for which the total is to be produced and how the output field (that is, the total) is to be edited or converted.

See p,m,f,edit under OUTREC for further details. However, note that PD0, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3, and TM4 are not allowed for TOTAL and that for TOTAL, the default number of digits (d) used for editing or conversion is as follows:

Table 63. Digits for TOTAL fields

Format (f)	Length (m)	Digits (d)
ZD	1-15	15
ZD	16-31	31
PD	1-8	15
PD	9-16	31
BI	1-4	10
BI	5-8	20
FI	1-4	10
FI	5-8	20
FL	4 or 8	20
CSF or FS	1-15	15
CSF or FS	16-32	31
UFF	1-15	15
UFF	16-44	31
SFF	1-15	15
SFF	16-44	31

If EDIT or EDxy is specified, the number of digits in the pattern (I's and T's) is used.

MIN
specifies that an edited or converted minimum, for the values of a numeric input field in all data records of the report, is to appear in the report record.

p,m,f,edit or p,m,f,to
specifies the numeric input field for which the minimum is to be produced and how the output field (that is, the minimum) is to be edited or converted.

See p,m,f,edit or p,m,f,to under OUTREC for further details. However, note that PD0, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3 and TM4 are not allowed for MIN.

MAX
specifies that an edited or converted maximum, for the values of a numeric input field in all data records of the report, is to appear in the report record.

p,m,f,edit or p,m,f,to
specifies the numeric input field for which the maximum is to be produced and how the output field (that is, the maximum) is to be edited or converted.

See p,m,f,edit or p,m,f,to under OUTREC for further details. However, note that PD0, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3 and TM4 are not allowed for MAX.

AVG
specifies that an edited or converted average, for the values of a numeric input field in all data records of the report, is to appear in the report record. The average (or mean) is calculated by dividing the total by the count and rounding down to the nearest integer. For example:

```
+2305 / 152 = +15
-2305 / 152 = -15
```

p,m,f,edit or p,m,f,to
specifies the numeric input field for which the average is to be produced and how the output field (that is, the average) is to be edited or converted.

See p,m,f,edit or p,m,f,to under OUTREC for further details. However, note that PD0, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3 and TM4 are not allowed for AVG.

SUBTOTAL
specifies that an edited or converted running total, for the values of a numeric input field in all data records of the report, is to appear in the report record.

SUBTOT or SUB can be used instead of SUBTOTAL.

For TRAILER1, the running total is the same as the total, so SUBTOTAL produces the same value as TOTAL.

p,m,f,edit or p,m,f,to
specifies the numeric input field for which the running total is to be produced and how the output field (that is, the running total) is to be edited or converted.

See p,m,f,edit or p,m,f,to under TOTAL for further details.

SUBMIN
specifies that an edited or converted running minimum, for the values of a numeric input field in all data records of the report, is to appear in the report record.

For TRAILER1, the running minimum is the same as the minimum, so SUBMIN produces the same value as MIN.

p,m,f,edit or p,m,f,to
specifies the numeric input field for which the running minimum is to be produced and how the output field (that is, the running minimum) is to be edited or converted.

See p,m,f,edit or p,m,f,to under OUTREC for further details. However, note that PD0, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3 and TM4 are not allowed for SUBMIN.

SUBMAX
specifies that an edited or converted running maximum, for the values of a numeric input field in all data records of the report, is to appear in the report record.

For TRAILER1, the running maximum is the same as the maximum, so SUBMAX produces the same value as MAX.

p,m,f,edit or p,m,f,to
specifies the numeric input field for which the running maximum is to be produced and how the output field (that is, the running maximum) is to be edited or converted.

See p,m,f,edit or p,m,f,to under OUTREC for further details. However, note that PD0, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3 and TM4 are not allowed for SUBMAX.

SUBAVG
specifies that an edited or converted running average, for the values of a numeric input field in all data records of the report, is to appear in the report record.

For TRAILER1, the running average is the same as the average, so SUBAVG produces the same value as AVG.

p,m,f,edit or p,m,f,to
specifies the numeric input field for which the running average is to be produced and how the output field (that is, the running average) is to be edited or converted.

See p,m,f,edit or p,m,f,to under OUTREC for further details. However, note that PD0, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3 and TM4 are not allowed for SUBAVG.

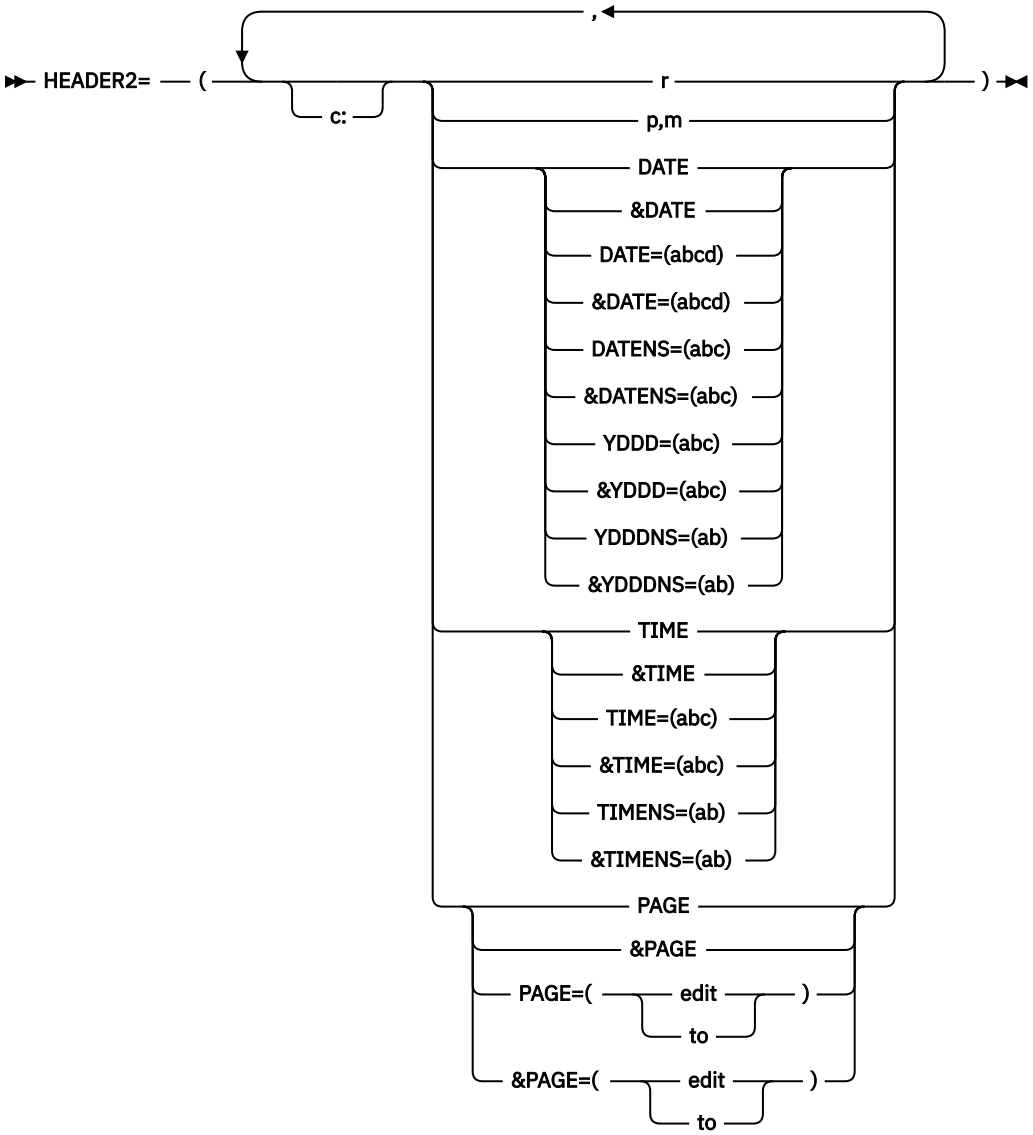
Sample Syntax:

```
OUTFIL FNAMES=RPT1,
  TRAILER1=(5/,
   10:'Summary of Report for Division Revenues',3/,
   10:'Number of divisions reporting: ',COUNT,2/,
   10:'Total revenue: ',TOTAL=(25,5,PD,M5),2/,
   10:'Lowest revenue: ',MIN=(25,5,PD,M5),2/,
   10:'Highest revenue: ',MAX=(25,5,PD,M5),2/,
   10:'Average revenue: ',AVG=(25,5,PD,M5))

OUTFIL FNAMES=RPT2,BLKCCT1,
       TRAILER1=(/,10:'Grand Total: ',TOTAL=(25,5,PD,M5))
```

Default for TRAILER1: None; must be specified.

HEADER2



Specifies the page header to be used for the reports produced for this OUTFIL group. The page header appears at the top of each page of the report, except for the report header page (if any) and report trailer page (if any). DFSORT uses ANSI carriage control characters to control page ejects and the placement of the lines in your report, according to your specifications. You can use BLKCCH2 to replace the '1' (page eject) for the ANSI carriage control character in the first line of the first page header with a blank, thus avoiding forcing a page eject. You can use REMOVECC to remove the ANSI carriage control characters from a report.

You can choose to include any or all of the following report elements in your page header:

- Blanks, character strings, and hexadecimal strings
- Unedited input fields from the first OUTFIL input record for which a data record appears on the page
- Current date in a variety of different forms
- Current time in a variety of different forms
- Page number, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on.

The HEADER2 parameter cannot be used with the IFTRAIL parameter.

The page header consists of the elements you select, in the order in which you specify them, and in the columns or lines you specify.

c:
See c: under HEADER1.

r
See r under HEADER1.

p,m
specifies that an unedited input field, from the first OUTFIL input record for which a data record appears on the page, is to appear in the report record. See p,m under HEADER1 for further details.

DATE
See DATE under HEADER1.

&DATE
can be used instead of DATE.

DATE=(abcd)
See DATE=(abcd) under HEADER1.

&DATE=(abcd)
can be used instead of DATE=(abcd).

DATENS=(abc)
See DATENS=(abc) under HEADER1.

&DATENS=(abc)
can be used instead of DATENS=(abc).

YDDD=(abc)
See YDDD=(abc) under HEADER1.

&YDDD=(abc)
can be used instead of YDDD=(abc).

YDDDNS=(ab)
See YDDDNS=(ab) under HEADER1.

&YDDDNS=(ab)
can be used instead of YDDDNS=(ab).

TIME
See TIME under HEADER1.

&TIME
can be used instead of TIME.

TIME=(abc)
See TIME=(abc) under HEADER1.

&TIME=(abc)
can be used instead of TIME=(abc).

TIMENS=(ab)
See TIMENS=(ab) under HEADER1.

&TIMENS=(ab)
can be used instead of TIMENS=(ab).

PAGE
specifies that the current page number is to appear in the OUTFIL report record. The page number for the header appears as 6 digits, right-justified, with leading zeros suppressed. For example, if the page is numbered 3, it appears as '   3'.

If HEADER1 is specified with PAGE and HEADER2 is specified with PAGE, the page number for the first page header will be '   2'. If HEADER1 is not specified or is specified without PAGE and HEADER2 is specified with PAGE, the page number for the first page header will be '   1'.

&PAGE
can be used instead of PAGE.

PAGE=(edit) or PAGE=(to)
same as PAGE except that the 15-digit page number appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use.

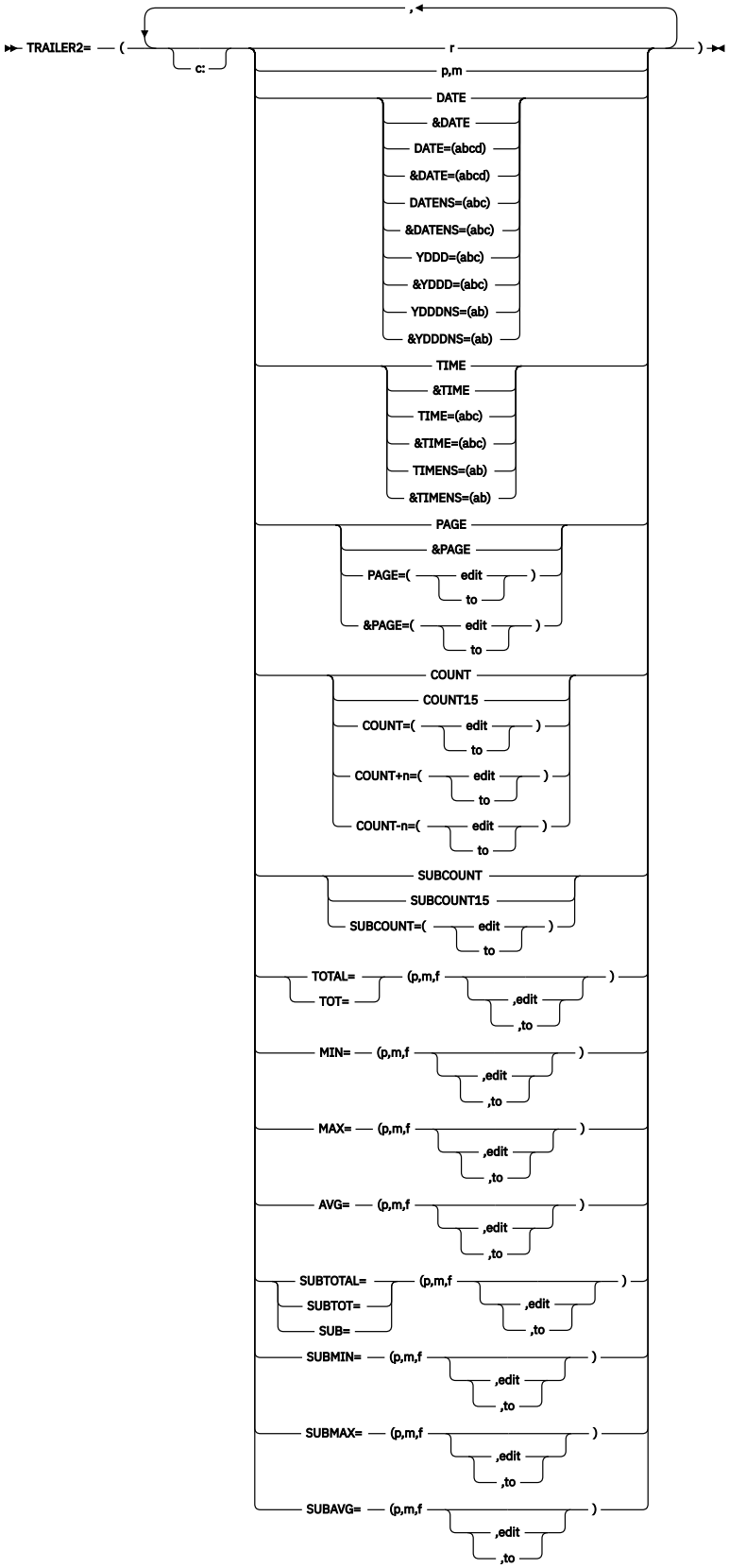
&PAGE=(edit) or &PAGE=(to)
can be used instead of PAGE=(edit) or PAGE=(to).

Sample Syntax:

```
OUTFIL FNAMES=STATUS,
  HEADER2=(5:'Page ',PAGE,' of Status Report for ',DATE=(MD4/),
           ' at ',TIME=(12:),2/,
           10:'Item ',20:'Status      ',35:'Count',/,
           10:'-----',20:'------------',35:'-----'),
   OUTREC=(10:6,5,
           20:14,1,CHANGE=(12,
                           C'S',C'Ship',
                           C'H',C'Hold',
                           C'T',C'Transfer'),
                   NOMATCH=(C'*Check Code*'),
           36:39,4,ZD,M10,
           132:X)
```

Default for HEADER2: None; must be specified.

TRAILER2



Specifies the page trailer to be used for the reports produced for this OUTFIL group. The page trailer appears at the very bottom of each page of the report (as specified or defaulted by the LINES value), except for the report header page (if any) and report trailer page (if any). DFSORT uses ANSI carriage

control characters to control page ejects and the placement of the lines in your report, according to your specifications.

You can choose to include any or all of the following report elements in your page trailer:

- Blanks, character strings, and hexadecimal strings
- Unedited input fields from the last OUTFIL input record for which a data record appears on the page
- Current date in a variety of different forms
- Current time in a variety of different forms
- Page number, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on
- Any or all of the following statistics:
  - Count of data records on the page, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on. You can add a decimal number to the count before converting or editing it (for example, add 1 to account for writing a trailer record, or add 2 to account for writing a header and trailer record).
  - Total, minimum, maximum, or average for each specified ZD, PD, BI, FI, FL, CSF, FS, UFF, or SFF numeric input field in the data records on the page, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on.
  - Running total, minimum, maximum, or average for each specified ZD, PD, BI, FI, FL, CSF, FS, UFF, or SFF numeric input field in the data records up to this point, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on.

The TRAILER2 parameter cannot be used with the IFTRAIL parameter.

The page trailer consists of the elements you select, in the order in which you specify them, and in the columns or lines you specify.

c:
See c: under HEADER1.

r
See r under TRAILER1.

p,m
specifies that an unedited input field, from the last OUTFIL input record for which a data record appears on the page, is to appear in the report record. See p,m under TRAILER1 for further details.

DATE
See DATE under HEADER1.

&DATE
can be used instead of DATE.

DATE=(abcd)
See DATE=(abcd) under HEADER1.

&DATE=(abcd)
can be used instead of DATE=(abcd).

DATENS=(abc)
See DATENS=(abc) under HEADER1.

&DATENS=(abc)
can be used instead of DATENS=(abc).

YDDD=(abc)
See YDDD=(abc) under HEADER1.

&YDDD=(abc)
can be used instead of YDDD=(abc).

YDDDNS=(ab)
See YDDDNS=(ab) under HEADER1.

&YDDDNS=(ab)
can be used instead of YDDDNS=(ab).

TIME
See TIME under HEADER1.

&TIME
can be used instead of TIME.

TIME=(abc)
See TIME=(abc) under HEADER1.

&TIME=(abc)
can be used instead of TIME=(abc).

TIMENS=(ab)
See TIMENS=(ab) under HEADER1.

&TIMENS=(ab)
can be used instead of TIMENS=(ab).

PAGE
See PAGE under TRAILER1.

&PAGE
can be used instead of PAGE.

PAGE=(edit) or PAGE=(to)
See PAGE=(edit) or PAGE=(to) under TRAILER1.

&PAGE=(edit) or &PAGE=(to)
can be used instead of PAGE=(edit) or PAGE=(to).

COUNT
specifies that the count of data records on the page is to appear in the report record as 8 digits, right-justified, with leading zeros suppressed. For example, if page 1 has 40 input records, page 2 has 40 input records, and page 3 has 26 input records, COUNT will show ' 40' for page 1, ' 40' for page 2, and ' 26' for page 3.

If slash (/) is used in OUTREC or BUILD to produce multiple data records, COUNT counts only the number of data records processed as input to OUTREC or BUILD. For example, if OUTREC processes 3 input records and creates 2 output records for each input record, the count is 3, not 6.

COUNT15
same as COUNT except that the count appears as 15 digits.

COUNT=(edit) or COUNT=(to)
same as COUNT except that the 15–digit count appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use.

COUNT+n=(edit) or COUNT+n=(to)
same as COUNT=(edit) or COUNT=(to) except that n is added to the 15-digit count before it is edited or converted. n can be 1 to 3 decimal digits.

COUNT-n=(edit) or COUNT-n=(to)
same as COUNT=(edit) or COUNT=(to) except that n is subtracted from the 15-digit count before it is edited or converted. n can be 1 to 3 decimal digits.

SUBCOUNT
specifies that the count of input records up to this point in the report is to appear in the report record as 8 digits, right-justified, with leading zeros suppressed. The running count accumulates the count for all pages up to and including the current page. For example, if page 1 has 40 input records, page 2 has 40 input records, and page 3 has 26 input records, SUBCOUNT will show ' 40' for page 1, ' 80' for page 2, and ' 106' for page 3.

If slash (/) is used in OUTREC or BUILD to produce multiple data records, SUBCOUNT counts only the number of data records processed as input to OUTREC or BUILD. For example, if OUTREC

processes 3 input records and creates 2 output records for each input record, the running count is 3, not 6.

SUBCOUNT15
same as SUBCOUNT except that the running count appears as 15 digits.

SUBCOUNT=(edit) or SUBCOUNT=(to)
same as SUBCOUNT except that the 15–digit running count appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use.

TOTAL
specifies that an edited or converted total, for the values of a numeric input field in the data records on the page, is to appear in the report record.

TOT can be used instead of TOTAL.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under TOTAL for TRAILER1.

MIN
specifies that an edited or converted minimum, for the values of a numeric input field in the data records on the page, is to appear in the report record.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under MIN for TRAILER1.

MAX
specifies that an edited or converted maximum, for the values of a numeric input field in the data records on the page, is to appear in the report record.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under MAX for TRAILER1.

AVG
specifies that an edited or converted average, for the values of a numeric input field in the data records on the page, is to appear in the report record.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under AVG for TRAILER1.

SUBTOTAL
specifies that an edited or converted running total, for the values of a numeric input field in the data records up to this point in the report, is to appear in the report record. The running total accumulates the total for all pages up to and including the current page. For example, if the total for a selected numeric field is +200 for page 1, -250 for page 2, and +90 for page 3, SUBTOTAL will be +200 for page 1, -50 for page 2, and +40 for page 3.

SUBTOT or SUB can be used instead of SUBTOTAL.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under SUBTOTAL for TRAILER1.

SUBMIN
specifies that an edited or converted running minimum, for the values of a numeric input field in the data records up to this point in the report, is to appear in the report record. The running minimum selects the minimum from all pages up to and including the current page. For example, if the minimum for a selected numeric field is +200 for page 1, -250 for page 2, and +90 for page 3, SUBMIN will be +200 for page 1, -250 for page 2, and -250 for page 3.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under SUBMIN for TRAILER1.

SUBMAX
specifies that an edited or converted running maximum, for the values of a numeric input field in the data records up to this point in the report, is to appear in the report record. The running maximum selects the maximum from all pages up to and including the current page. For example,

if the maximum for a selected numeric field is -100 for page 1, +250 for page 2, and +90 for page 3, SUBMAX will be -100 for page 1, +250 for page 2, and +250 for page 3.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under SUBMAX for TRAILER1.

SUBAVG
specifies that an edited or converted running average, for the values of a numeric input field in the data records up to this point in the report, is to appear in the report record. The running average computes the average for all pages up to and including the current page. For example, if the count of data records and total for a selected numeric field are 60 and +2205 for page 1, respectively, 60 and -6252 for page 2, respectively, and 23 and -320 for page 3, respectively, SUBAVG will be +36 for page 1, -33 for page 2, and -30 for page 3.

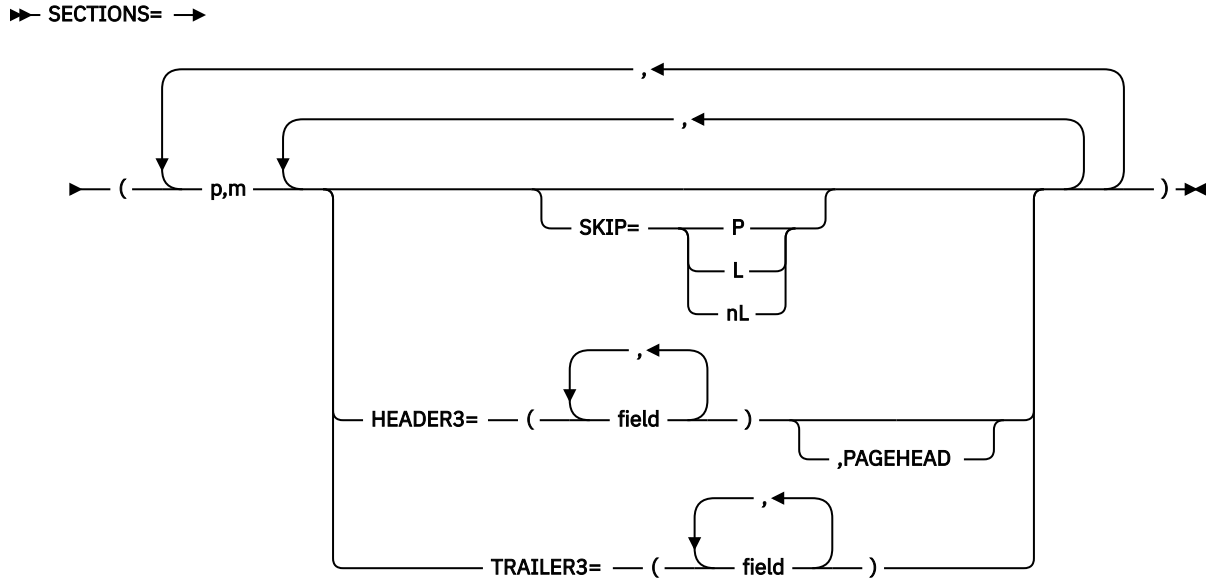
p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under SUBAVG for TRAILER1.

Sample Syntax:

```
OUTFIL FNAMES=STATS,
   STARTREC=3,
    OUTREC=(20:23,3,PD,M16,
            30:40,3,PD,M16,
            80:X),
   TRAILER2=(/,2:'Average on page:',
             20:AVG=(23,3,PD,M16),
             30:AVG=(40,3,PD,M16),/,
             2:'Average so far:',
             20:SUBAVG=(23,3,PD,M16),
             30:SUBAVG=(40,3,PD,M16))
```

Default for TRAILER2: None; must be specified.

SECTIONS



Specifies the section break processing to be used for the reports produced for this OUTFIL group. A section break field divides the report into sets of sequential OUTFIL input records with the same binary value for that field, which result in corresponding sets of data records (that is, sections) in the report. A break is said to occur when the binary value changes. Of course, because a break can occur in any record, the data records of a section can be split across pages in your report.

For each section break field you specify, you can choose to include any or all of the following:

- A page eject between sections.

- Zero, one or more blank lines to appear between sections on the same page.
- A section header to appear before the first data record of each section and optionally, at the top of each page. When a page header and section header are both to appear at the top of a page, the section header will follow the page header.
- A section trailer to appear after the last data record of each section. When a page trailer and section trailer are both to appear at the bottom of a page, the page trailer will follow the section trailer.

The SECTIONS parameter cannot be used with the IFTRAIL parameter.

DFSORT uses ANSI carriage control characters to control page ejects and the placement of the lines in your report, according to your specifications.

If multiple section break fields are used, they are processed in first-to-last order, in the same way they would be sorted by these fields. In fact, the input data set is generally sorted by the section break fields, to group the records with the same section break values together for the report. This sorting can be done by the same application that produces the report or by a previous application.

A break in section break field 1 results in a break in section break fields 2 through n. A break in section break 2 results in a break in section break fields 3 through n, and so on. The section headers appear before each section in first-to-last order, whereas the section trailers appear in last-to-first order. For example, if section break fields represented by B1 with header H3A and trailer T3A, B2 with header H3B and trailer T3B, and B3 with header H3C and trailer T3C are specified in order, the following can appear:

```
H3A (header for B1=1 section)
  H3B (header for B2=1 section)
    H3C (header for B3=1 section)
      data records for B1=1, B2=1, B3=1 (new B1, B2, and B3 section)
    T3C (trailer for B3=1 section)
    H3C (header for B3=2 section)
      data records for B1=1, B2=1, B3=2 (new B3 section)
    T3C (trailer for B3=2 section)
  T3B (trailer for B2=1 section)
  H3B (header for B2=2 section)
    H3C (header for B3=1 section)
      data records for B1=1, B2=2, B3=1 (new B2 and B3 section)
    T3C (trailer for B3=1 section)
  T3B (trailer for B2=2 section)
T3A (trailer for B1=1 section)
H3A (header for B1=2 section)
  H3B (header for B2=2 section)
    H3C (header for B3=0 section)
      data records for B1=2, B2=2, B3=0 (new B1, B2, and B3 section)
    T3C (trailer for B3=0 section)
    H3C (header for B3=1 section)
      data records for B1=2, B2=2, B3=1 (new B3 section)
    T3C (trailer for B3=1 section)
  T3B (trailer for B2=2 section)
T3A (trailer for B1=2 section)
```

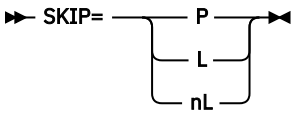
p,m
specifies a section break field in the OUTFIL input records to be used to divide the report into sections. Each set of sequential OUTFIL input records, with the same binary value for the section break field, results in a corresponding set of data records. Each such set of data records is treated as a section in the report. A break is said to occur when the binary value changes.

p
specifies the first byte of the input field relative to the beginning of the OUTFIL input record. The first data byte of a fixed-length record has relative position 1. The first data byte of a variable-length record has relative position 5, because the first four bytes are occupied by the RDW. All fields must start on a byte boundary, and no field can extend beyond byte 32752. See “OUTFIL statements notes” on page 358 for special rules concerning variable-length records.

m
specifies the length in bytes of the input field. The value for m must be between 1 and 256.

You can specify any, all or none of SKIP, HEADER3 and TRAILER3 after p,m. If you do not specify SKIP, HEADER3 or TRAILER3 after p,m, SKIP=0L is used as the default.

SKIP



Specifies, for reports produced for this OUTFIL group, that either:

- Each section for the associated section break field is to appear on a new page, or
- Zero, one or more blank lines to appear after each section associated with this section break field, when it is followed by another section on the same page.

Thus, you can use SKIP to specify how sections will be separated from each other.

P
specifies that each section is to appear on a new page.

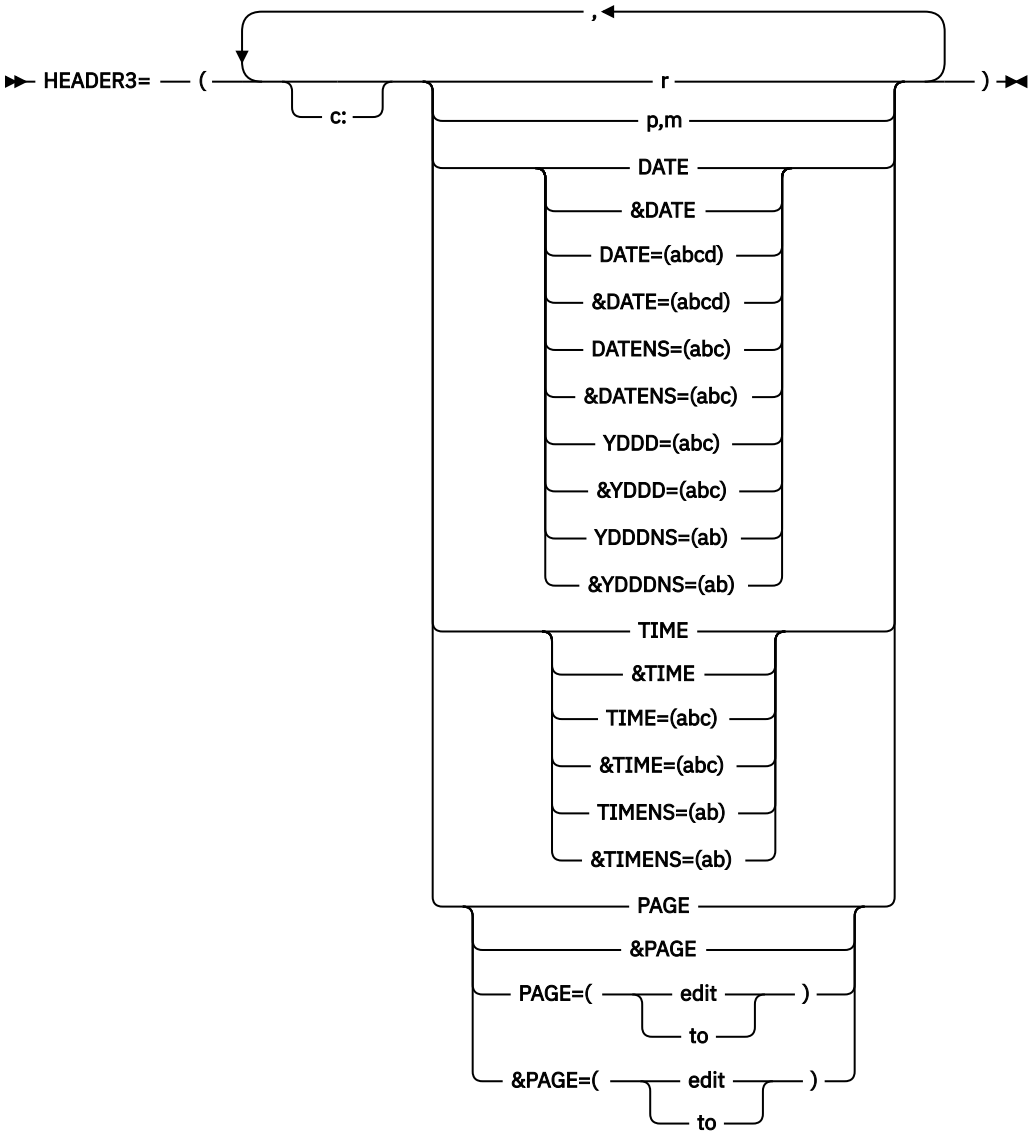
L
specifies that one blank line is to appear between sections on the same page. L is the same as 1L.

nL
specifies that n blank lines are to appear between sections on the same page. You can specify from 0 to 255 for n.

Sample Syntax:

```
OUTFIL FNAMES=(PRINT,TAPE),
   SECTIONS=(10,20,SKIP=P,
             42,10,SKIP=3L)
```

HEADER3



Specifies the section header to be used with the associated section break field for the reports produced for this OUTFIL group. The section header appears before the first data record of each section. DFSORT uses ANSI carriage control characters to control page ejects and the placement of the lines in your report, according to your specifications.

You can choose to include any or all of the following report elements in your section header:

- Blanks, character strings, and hexadecimal strings
- Unedited input fields from the first OUTFIL input record for which a data record appears in the section
- Current date in a variety of different forms
- Current time in a variety of different forms
- Page number, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on.

The section header consists of the elements you select, in the order in which you specify them, and in the columns or lines you specify.

c:
See c: under HEADER1.

r
See r under HEADER1.

p,m
specifies that an unedited input field, from the first OUTFIL input record for which a data record appears in the section, is to appear in the report record. See p,m under HEADER1 for further details.

DATE
See DATE under HEADER1.

&DATE
can be used instead of DATE.

DATE=(abcd)
See DATE=(abcd) under HEADER1.

&DATE=(abcd)
can be used instead of DATE=(abcd).

DATENS=(abc)
See DATENS=(abc) under HEADER1.

&DATENS=(abc)
can be used instead of DATENS=(abc).

YDDD=(abc)
See YDDD=(abc) under HEADER1.

&YDDD=(abc)
can be used instead of YDDD=(abc).

YDDDNS=(ab)
See YDDDNS=(ab) under HEADER1.

&YDDDNS=(ab)
can be used instead of YDDD=(ab).

TIME
See TIME under HEADER1.

&TIME
can be used instead of TIME.

TIME=(abc)
See TIME=(abc) under HEADER1.

&TIME=(abc)
can be used instead of TIME=(abc).

TIMENS=(ab)
See TIMENS=(ab) under HEADER1.

&TIMENS=(ab)
can be used instead of TIMENS=(ab).

PAGE
specifies that the current page number is to appear in the OUTFIL report record. The page number for the header appears as 6 digits, right-justified, with leading zeros suppressed. For example, if the page is numbered 3, it appears as '   3'.

&PAGE
can be used instead of PAGE.

PAGE=(edit) or PAGE=(to)
same as PAGE except that the 15-digit page number appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use.

&PAGE=(edit) or &PAGE=(to)
can be used instead of PAGE=(edit) or PAGE=(to).

Sample Syntax:

```
OUTFIL FNAMES=STATUS1,
         HEADER2=(10:'Status Report for all departments',5X,
                        '- ',&PAGE,' -'),
         SECTIONS=(10,8,
          HEADER3=(2/,10:'Report for department ',10,8,' on ',&DATE,2/,
                    10:'   Number',25:'Average Time',/,
                    10:'Completed',25:'   (in days)',/,
                    10:'---------',25:'------------')),
            OUTREC=(10:21,5,ZD,M10,LENGTH=9,
                       25:38,4,ZD,EDIT=(III.T),LENGTH=12,
                       132:X)
```

PAGEHEAD



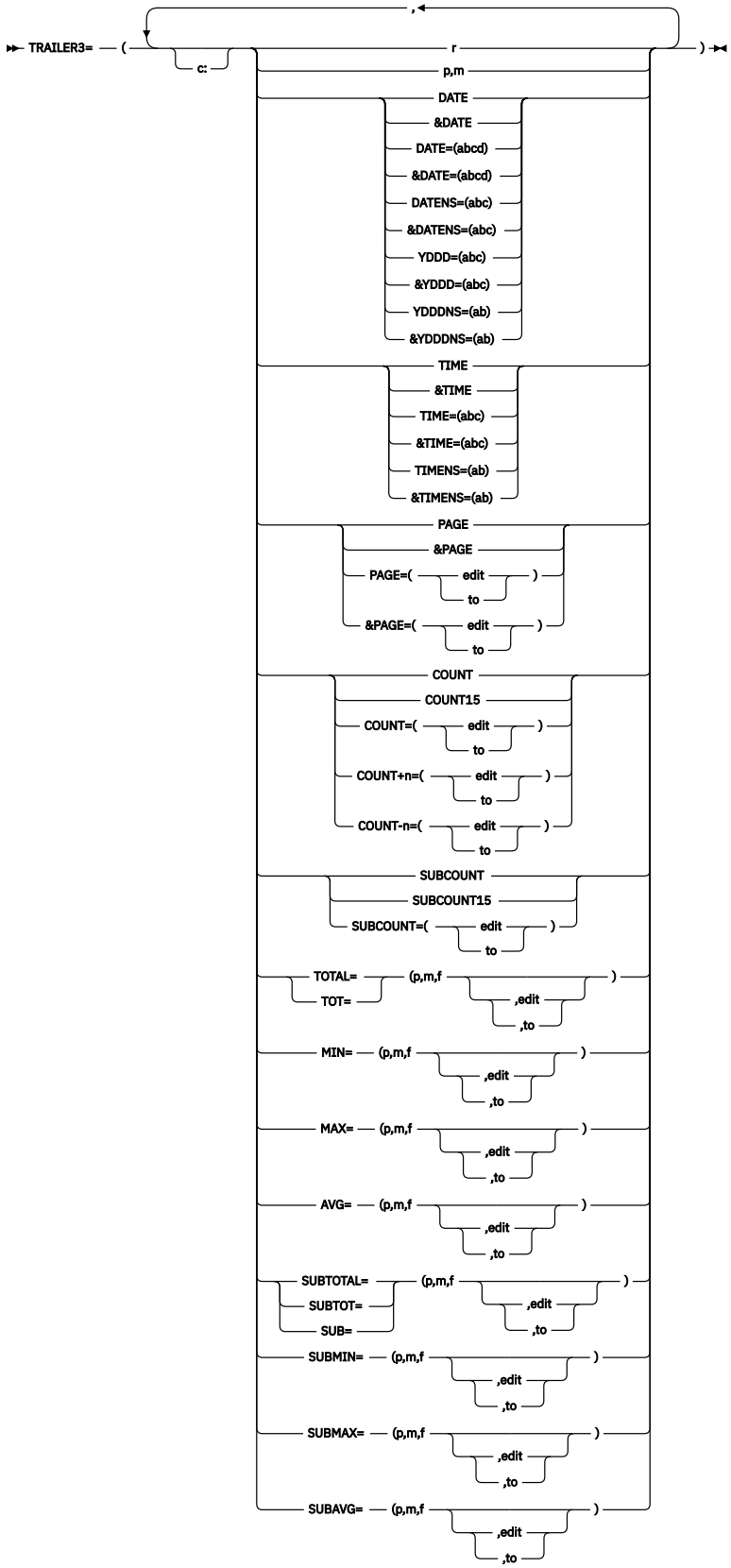
Specifies that the section header to be used with the associated section break field is to appear at the top of each page of the report, except for the report header page (if any) and report trailer page (if any), as well as before each section. If you do not specify PAGEHEAD, the section header appears only before each section; so if a section is split between pages, the section header appears only in the middle of the page. PAGEHEAD can be used when you want HEADER3 to be used as a page header as well as a section header.

If PAGEHEAD is specified for a section break field for which HEADER3 is not also specified, PAGEHEAD will not be used.

Sample Syntax:

```
  OUTFIL FNAMES=STATUS2,
     HEADER2=(10:'Status Report for all departments',5X,
                  '- ',&PAGE,' -'),
     SECTIONS=(10,8,
      HEADER3=(2/,10:'Report for department ',10,8,' on ',&DATE,2/,
               10:'   Number',25:'Average Time',/,
               10:'Completed',25:'   (in days)',/,
               10:'---------',25:'------------'),
          PAGEHEAD,SKIP=P),
       OUTREC=(10:21,5,ZD,M10,LENGTH=9,
               25:38,4,ZD,EDIT=(III.T),LENGTH=12,
               132:X)
```

TRAILER3



Specifies the section trailer to be used with the associated section break field for the reports produced for this OUTFIL group. The section trailer appears after the last data record of each section. DFSORT uses ANSI carriage control characters to control page ejects and the placement of the lines in your report, according to your specifications.

You can choose to include any or all of the following report elements in your section trailer:

- Blanks, character strings, and hexadecimal strings
- Unedited input fields from the last OUTFIL input record for which a data record appears in the section
- Current date in a variety of different forms
- Current time in a variety of different forms
- Page number, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on
- Any or all of the following statistics:
  - Count of data records in the section, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on. You can add a decimal number to the count before converting or editing it (for example, add 1 to account for writing a trailer record, or add 2 to account for writing a header and trailer record).
  - Total, minimum, maximum, or average for each specified ZD, PD, BI, FI, FL, CSF, FS, UFF, or SFF numeric input field in the data records in the section, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on
  - Running total, minimum, maximum, or average for each specified ZD, PD, BI, FI, FL, CSF, FS, UFF, or SFF numeric input field in the data records up to this point, converted to different numeric formats, or edited to contain signs, decimal points, leading zeros or no leading zeros, and so on.

The section trailer consists of the elements you select, in the order in which you specify them, and in the columns or lines you specify.

c:
See c: under HEADER1.

r
See r under TRAILER1.

p,m
specifies that an unedited input field, from the last OUTFIL input record for which a data record appears in the section, is to appear in the report record. See p,m under TRAILER1 for further details.

DATE
See DATE under HEADER1.

&DATE
can be used instead of DATE.

DATE=(abcd)
See DATE=(abcd) under HEADER1.

&DATE=(abcd)
can be used instead of DATE=(abcd).

DATENS=(abc)
See DATENS=(abc) under HEADER1.

&DATENS=(abc)
can be used instead of DATENS=(abc).

YDDD=(abc)
See YDDD=(abc) under HEADER1.

&YDDD=(abc)
can be used instead of YDDD=(abc).

YDDDNS=(ab)
See YDDDNS=(ab) under HEADER1.

&YDDDNS=(ab)
can be used instead of YDDDNS=(ab).

TIME
See TIME under HEADER1.

&TIME
can be used instead of TIME.

TIME=(abc)
See TIME=(abc) under HEADER1.

&TIME=(abc)
can be used instead of TIME=(abc).

TIMENS=(ab)
See TIMENS=(ab) under HEADER1.

&TIMENS=(ab)
can be used instead of TIMENS=(ab).

PAGE
See PAGE under TRAILER1.

&PAGE
can be used instead of PAGE.

PAGE=(edit) or PAGE=(to)
See PAGE=(edit) or PAGE=(to) under TRAILER1.

&PAGE=(edit) or &PAGE=(to)
can be used instead of PAGE=(edit) or PAGE=(to).

COUNT
specifies that the count of data records in the section is to appear in the report record as 8 digits, right-justified, with leading zeros suppressed. For example, if the first section has 40 input records, the second section has 40 input records, and the third section has 26 input records, COUNT will show '   40' for the first section, '   40' for the second section, and '   26' for the third section.

If slash (/) is used in OUTREC or BUILD to produce multiple data records, COUNT counts only the number of data records processed as input to OUTREC or BUILD. For example, if OUTREC or BUILD processes 3 input records and creates 2 output records for each input record, the count is 3, not 6.

COUNT15
same as COUNT except that the count appears as 15 digits.

COUNT=(edit) or COUNT=(to)
same as COUNT except that the 15–digit count appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use.

COUNT+n=(edit) or COUNT+n=(to)
same as COUNT=(edit) or COUNT=(to) except that n is added to the 15-digit count before it is edited or converted. n can be 1 to 3 decimal digits.

COUNT-n=(edit) or COUNT-n=(to)
same as COUNT=(edit) or COUNT=(to) except that n is subtracted from the 15-digit count before it is edited or converted. n can be 1 to 3 decimal digits.

SUBCOUNT
specifies that the running count of input records up to this point in the report is to appear in the report record 8 digits, right-justified, with leading zeros suppressed. The running count accumulates the count for all sections up to and including the current section. For example, if the first section has 40 input records, the second section has 40 input records, and the third section has 26 input records, SUBCOUNT will show '   40' for the first section, '   80' for the second section, and '   106' for the third section.

If slash (/) is used in OUTREC or BUILD to produce multiple data records, SUBCOUNT counts only the number of data records processed as input to OUTREC or BUILD. For example, if OUTREC or BUILD processes 3 input records and creates 2 output records for each input record, the running count is 3, not 6

SUBCOUNT15
same as SUBCOUNT except that the running count appears as 15 digits.

SUBCOUNT=(edit) or SUBCOUNT=(to)
same as SUBCOUNT except that the 15–digit running count appears edited or converted as specified. See p,m,f,edit under OUTREC for further details on the edit fields you can use. See p,m,f,to under OUTREC for further details on the to fields you can use.

TOTAL
specifies that an edited or converted total, for the values of a numeric input field in the data records in the section, is to appear in the report record.

TOT can be used instead of TOTAL.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under TOTAL for TRAILER1.

MIN
specifies that an edited or converted minimum, for the values of a numeric input field in the data records in the section, is to appear in the report record.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,tounder MIN for TRAILER1.

MAX
specifies that an edited or converted maximum, for the values of a numeric input field in the data records in the section, is to appear in the report record.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under MAX for TRAILER1.

AVG
specifies that an edited or converted average, for the values of a numeric input field in the data records in the section, is to appear in the report record.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under AVG for TRAILER1.

SUBTOTAL
specifies that an edited or converted running total, for the values of a numeric input field in the data records up to this point in the report, is to appear in the report record. The running total accumulates the total for all sections up to and including the current section. For example, if the total for a selected numeric field is +200 for the first section, -250 for the second section and +90 for the third section, SUBTOTAL will be +200 for the first section, -50 for the second section and +40 for the third section.

SUBTOT or SUB can be used instead of SUBTOTAL.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under SUBTOTAL for TRAILER1.

SUBMIN
specifies that an edited or converted running minimum, for the values of a numeric input field in the data records up to this point in the report, is to appear in the report record. The running minimum selects the minimum from all sections up to and including the current section. For example, if the minimum for a selected numeric field is +200 for the first section, -250 for the second section and +90 for the third section, SUBMIN will be +200 for the first section, -250 for the second section and -250 for the third section.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under SUBMIN for TRAILER1.

SUBMAX
specifies that an edited or converted running maximum, for the values of a numeric input field in the data records up to this point in the report, is to appear in the report record. The running maximum selects the maximum from all sections up to and including the current section. For example, if the maximum for a selected numeric field is -100 for the first section, +250 for the second section and +90 for the third section, SUBMAX will be -100 for the first section, +250 for the second section and +250 for the third section.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under SUBMAX for TRAILER1.

SUBAVG
specifies that an edited or converted running average, for the values of a numeric input field in the data records up to this point in the report, is to appear in the report record. The running average computes the average for all sections up to and including the current section. For example, if the count of data records and total for a selected numeric field are 60 and +2205 for the first section, respectively, 60 and -6252 for the second section, respectively, and 23 and -320 for the third section, respectively, SUBAVG will be +36 for the first section, -33 for the second section and -30 for the third section.

p,m,f,edit or p,m,f,to
See p,m,f,edit or p,m,f,to under SUBAVG for TRAILER1.

Sample Syntax:

```
OUTFIL FNAMES=SECRPT,
  INCLUDE=(11,4,CH,EQ,C'SSD'),
  SECTIONS=(3,5,SKIP=P,
   HEADER3=(2:'Department: ',3,5,4X,'Date: ',&DATE,2/),
   TRAILER3=(2/,2:'The average for ',3,5,' is ',
                  AVG=(40,3,PD,M12),/,
               2:'The overall average so far is ',
                  SUBAVG=(40,3,PD,M12)),
  45,8,SKIP=3L,
   HEADER3=(4:'Week Ending ',45,8,2/,
              4:'Item Number',20:'Completed',/,
              4:'-',20:'-'),
   TRAILER3=(4:'The item count for week ending ',45,8,
      ' is ',COUNT=(EDIT=(II,IIT)))),
   OUTREC=(11:16,4,22:40,3,PD,M12,120:X)
```

Default for SECTIONS: None; must be specified.

NODETAIL



Specifies that data records are not to be output for the reports produced for this OUTFIL group. With NODETAIL, the data records are completely processed with respect to input fields, statistics, counts, sections breaks, and so on, but are not written to the OUTFIL data set and are not included in line counts for determining the end of a page. You can use NODETAIL to summarize the data records without actually showing them.

The NODETAIL parameter cannot be used with the IFTRAIL parameter.

Sample Syntax:

```
OUTFIL FNAMES=SUMMARY,NODETAIL,
   HEADER2=(' Date: ',DATENS=(DMY.),4X,'Page: ',PAGE,2/,
             10:'Division',25:'    Total Revenue',/,
             10:'--------',25:'-----------------'),
   SECTIONS=(3,5,
    TRAILER3=(10:3,5,
              25:TOTAL=(25,4,FI,M19,
                 LENGTH=17))),
    TRAILER1=(5/,10:'Summary of Revenue ',4/,
                 12:'Number of divisions reporting is ',
                     COUNT,/,
```

```
                    12:'Total revenue is ',
                       TOTAL=(25,4,FI,M19))
```

Default for NODETAIL: None; must be specified.

Default for OUTFIL Statements: None; must be specified. Multiple OUTFIL statements can be specified in the same and different sources; override is at the ddname level.

Applicable Functions for OUTFIL Statements: Sort, merge, and copy.

BLKCCH1

specifies that the ANSI carriage control character of '1' (page eject) in the first line of the report header (HEADER1) is to be replaced with a blank. You can use BLKCCH1 to avoid forcing a page eject at the start of the report header. If BLKCCH1 is specified without HEADER1, it will not be used.

Sample Syntax:

```
  OUTFIL FNAMES=RPT1,BLKCCH1,
         HEADER1=(30:'January Report',4/)
```

Default for BLKCCH1: None; must be specified.

BLKCCH2

specifies that the ANSI carriage control character of '1' (page eject) in the first line of the first page header (HEADER2) is to replaced with a blank. You can use BLKCCH2 to avoid forcing a page eject at the start of the first page header. BLKCHH2 does not prevent a page eject for the second and subsequent page headers. If BLKCCH2 is specified without HEADER2, it will not be used.

Sample Syntax:

```
  OUTFIL FNAMES=RPT2,
* Do page eject for HEADER1, but not for first HEADER2.
     HEADER1=(30:'January Report',2/),
     BLKCCH2,HEADER2=(5:'Account Number',25:'Name')
```

Default for BLKCCH2: None; must be specified.

BLKCCT1

specifies that the ANSI carriage control character of '1' (page eject) in the first line of the report trailer (TRAILER1) is to be replaced with a blank. You can use BLKCCT1 to avoid forcing a page eject at the start of the report trailer. If BLKCCT1 is specified without TRAILER1, it will not be used.

Sample Syntax:

```
  OUTFIL FNAMES=RPT3,BLKCCT1
,        TRAILER1=(5:'Grand Total is ',TOT=(21,10,ZD,M5))
```

Default for BLKCCT1: None; must be specified.

REMOVECC



Specifies that the ANSI carriage control character is to be removed from OUTFIL output records for this OUTFIL group before the records are written. In addition, blank lines are not used to position the page trailer (TRAILER2) at the bottom of the page.

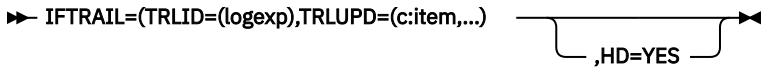
If REMOVECC is specified without any of the following report parameters, it will not be used: LINES, HEADER1, TRAILER1, HEADER2, TRAILER2, SECTIONS, or NODETAIL.

Sample Syntax:

```
OUTFIL FNAMES=RPTWOCC,
  TRAILER1=(3/,'Number of records is ',
     COUNT=(TO=ZD,LENGTH=6)),
  REMOVECC
```

Default for REMOVECC: None; must be specified.

IFTRAIL



IFTRAIL allows you to update count and total values in an existing trailer (last) record to reflect the actual data records in an OUTFIL data set. Whereas TRAILER1 lets you build a new trailer record, IFTRAIL lets you update an existing trailer record using the COUNT and TOTAL features of TRAILER1. When you add, delete or modify input data records to create an OUTFIL data set, you can use IFTRAIL to ensure that the trailer record has accurate updated count and total values.

The count and total values are determined using the data records processed as input to OUTFIL. If slash (/) is used in OUTFIL BUILD to produce multiple output data records, the count and total values are determined using the data records processed as input to OUTFIL. For example, if you use OUTFIL BUILD=(1,80,/,1,80) to create 6 OUTFIL output records from 3 OUTFIL input records, the count is 3, not 6.

You must identify the trailer record using a unique condition (for example, 'TRL' in positions 1-3). You then specify how count and total values in the identified trailer record are to be updated. The trailer record will NOT be treated as a data record, that is, no other OUTFIL processing will be performed against the trailer record. You can optionally specify that the header (first) record will NOT be treated as a data record.

As a simple example, you could use the following OUTFIL statement to write the header (first) record, data records with 'D1', and trailer record (identified by '9' in position 1) in the OUTFIL data set, and set an accurate count and total in the trailer record for the included input data records.

```
OUTFIL FNAMES=OUT1,INCLUDE=(10,2,CH,EQ,C'D1'),
  IFTRAIL=(HD=YES,TRLID=(1,1,CH,EQ,C'9'),
    TRLUPD=(11:COUNT=(M11,LENGTH=8),
     25:TOTAL=(15,6,ZD,EDIT=(IIIIIT))))
```

The trailer record **will not** be treated as a data record; it will not be subject to INCLUDE processing, and will not be used for the updated count or total values.

Since HD=YES is specified, the header record **will not** be treated as a data record; it will not be subject to INCLUDE processing, and will not be used for the updated count or total values.

IFTRAIL
Allows you to update count and total fields in the trailer record of the data sets for an OUTFIL group. You can use IFTRAIL to identify a trailer record and indicate count and total fields to be updated in that record. The identified trailer record will not be used to determine the count or totals. If you specify HD=YES, the header (first) record will not be used to determine the count or totals.

You cannot specify IFTRAIL with any of the following operands in an OUTFIL statement: HEADER1, TRAILER1, HEADER2, TRAILER2, NODETAIL, SECTIONS, LINES, SPLIT, SPLITBY, SPLIT1R, REPEAT, FTOV, VTOF, VLTRIM, VLTRAIL or VLFILL.

You must specify the TRLID and TRLUPD operands. The HD=YES operand is optional.

TRLID=(logexp)
Specifies the criteria to be tested to determine if a record is the trailer record. When a record is found that meets the criteria, it will be treated as the last record, and subsequent records will not

be processed for the OUTFIL group. Thus if you use TRLID=(1,1,CH,EQ,C'9') to identify the trailer record and you have three records with '9' in position 1, the first record with '9' in position 1 will be treated as the trailer record, and records after that will not appear in the OUTFIL data sets.

See the discussion of INCLUDE statement in "INCLUDE control statement" on page 91 for details of the logical expressions that you can use. You can specify all of the logical expressions for TRLID in the same way that you can specify them for the INCLUDE statement, except that:

- You cannot specify FORMAT=f
- You cannot specify D2 format
- Locale processing is not used
- VLSCMP and VLSHRT are not used. The trailer record must be long enough to contain all fields in the logical expression. Trailer record checking will be skipped for any variable-length record that is too short to contain a field in the TRLID logical expression.

The trailer record will not be treated as a data record; all other OUTFIL processing (for example, INCLUDE, OMIT, BUILD, OVERLAY, and so on) will be bypassed for the trailer record. The updated count and total values will not include the trailer record.

TRLUPD=(c:item,...)

Specifies the position where each count or total is to be updated in the trailer record.

For fixed-length records, the first input and output data byte starts at position 1. The trailer record will be truncated or padded with blanks to the length of the data records, if appropriate. You cannot change the length of the trailer record using TRLUPD.

For variable-length records, the first input and output data byte starts at position 5, after the RDW in positions 1-4. The original length of the trailer record will not be changed for output. You cannot change the length of the trailer record using TRLUPD. If you specify an item beyond the end of the trailer record, it will not be updated.

c:

specifies the output position (column) in the trailer record to be updated. If you do not specify c: for the first item, it defaults to 1:. If you do not specify c: for any other item, it starts after the previous item. You must specify items in ascending output column order, and an item must not overlap a previous item in the trailer record. For a variable-length trailer record, you must not specify columns 1-4.

item

specifies the count or total you want to update. Any of the following count and total items can be used as described for TRAILER1:

- COUNT=(edit)
- COUNT=(to)
- COUNT+n=(edit)
- COUNT+n=(to)
- COUNT-n=(edit)
- COUNT-n=(to)
- TOTAL=(p,m,f,edit)
- TOTAL=(p,m,f,to)
- TOT=(p,m,f,edit)
- TOT=(p,m,f,to)

The count and total values are determined using the data records processed as input to OUTFIL.

If TOT or TOTAL is used, the specified p,m,f field from each data record will be totalled, even if the trailer record is not found. OUTFIL BUILD, OVERLAY or IFTHEN processing does NOT affect

the values that are totalled; the values in the original OUTFIL input records are used for the totals.

An invalid field amount can result in a data exception (0C7 ABEND) or incorrect numeric output. Be careful to use HD=YES if the header record does not have valid total fields. For example, if the input file has these records:

```
H***
0001
0002
T***
```

and this IFTRAIL operand is specified:

```
   IFTRAIL=(TRLID=(1,1,CH,EQ,C'T'),
     TRLUPD=(5:TOT=(1,4,ZD,TO=ZD,LENGTH=5))
```

an 0C7 ABEND will result when DFSORT attempts to total the invalid 1,4,ZD field in the header (H***) record. The use of HD=YES would prevent this.

An invalid total field can also be encountered if the trailer record is not found due to erroneous TRLID conditions. For example, if the input file has these records:

```
H***
0001
0002
T***
```

and this IFTRAIL operand is specified:

```
   IFTRAIL=(TRLID=(1,2,CH,EQ,C'TR'),
     TRLUPD=(5:TOT=(1,4,ZD,TO=ZD,LENGTH=5))
```

the last (T***) record will not be identified as the trailer record since it does not have 'TR' in positions 1-2. Thus, an 0C7 ABEND will result when DFSORT attempts to total the invalid 1,4,ZD field in the last (T***) record. The use of the correct condition for identifying the trailer (T***) record would prevent this.

If slash (/) is used in OUTFIL BUILD to produce multiple output data records, the count and total values are determined using the data records processed as input to OUTFIL. For example, if you use OUTFIL BUILD=(1,80,/,1,80) to create 6 OUTFIL output records from 3 OUTFIL input records, the count is 3, not 6.

HD=YES

Specify HD=YES if you want the first record treated as a header record rather than as a data record. With HD=YES, OUTFIL processing (for example, INCLUDE, OMIT, BUILD, OVERLAY, and so on) will be bypassed for the header record. The updated count and total values will not include the header record.

If HD=YES is specified, the TRLID test will not be applied to the first record.

Sample Syntax:

```
OUTFIL IFTRAIL=(HD=YES,
   TRLID=(1,3,CH,EQ,C'TRL'),
   TRLUPD=(11:COUNT=(M11,LENGTH=4),
           21:TOT=(M11,LENGTH=6)))
```

Default for IFTRAIL: None; must be specified.

OUTFIL statements notes

- OUTFIL processing is supported for sort, merge, and copy applications, but only by the Blockset technique.
-

- The ODMAXBF value in effect specifies the maximum buffer space to be used for each OUTFIL data set. ODMAXBF can be specified as an installation or run-time option, or in an ICEIEXIT routine. The default value of 2M is recommended for the ODMAXBF option in effect. Lowering ODMAXBF can cause performance degradation for the application, but might be necessary if you consider the amount of storage used for OUTFIL processing to be a problem. Raising ODMAXBF can improve EXCPs for the application, but can also increase the amount of storage needed.
-
- The storage used for OUTFIL processing will be adjusted automatically according to the total storage available, the storage needed for non-OUTFIL processing, and the number of OUTFIL data sets and their attributes (for example, block size). OUTFIL processing will be subject to the ODMAXBF limit in effect and the system storage limits (for example, IEFUSI), but not to the DFSORT storage limits (that is, SIZE, MAXLIM, and TMAXLIM). DFSORT attempts to use storage above 16MB virtual for OUTFIL processing whenever possible.
- The VSAMBSP option applies to SORTOUT data sets, but not to OUTFIL data sets. The NOBLKSET option will be ignored if OUTFIL data sets are being processed. An E39 exit routine is entered for the SORTOUT data set, but not for OUTFIL data sets.
- For fixed-format OUTFIL data sets: DFSORT will determine each OUTFIL data set LRECL based on the length of the reformatted records for the group, or the length of the OUTFIL input records (if OUTREC, BUILD, OVERLAY, FINDREP and IFTHEN are not specified for the group). For VSAM data sets, the maximum record size defined in the cluster is equivalent to the LRECL.

  If an OUTFIL data set LRECL is not specified or available, DFSORT will set it to the determined LRECL. If an OUTFIL data set LRECL is specified or available, it must not be less than the determined LRECL, or more than the determined LRECL if the OUTREC, BUILD, OVERLAY, FINDREP or IFTHEN parameter is specified. In other words, the LRECL value cannot be used to pad the output records, or to truncate the records produced by OUTREC, BUILD, OVERLAY, FINDREP or IFTHEN parameter processing.

  In general, OUTREC, BUILD, OVERLAY, or IFTHEN processing should be used to pad the records and OUTREC, BUILD, or IFTHEN processing should be used to truncate the records. In either case, the LRECL should either not be specified or set to the length of the reformatted records.
- For variable-format OUTFIL data sets: DFSORT will determine each OUTFIL data set maximum LRECL based on the length of the reformatted records for the group, or the length of the OUTFIL input records (if OUTREC, BUILD, OVERLAY, FINDREP and IFTHEN are not specified for the group). If an OUTFIL data set maximum LRECL is not specified or available, DFSORT will set it to the determined maximum LRECL. For VSAM data sets, the maximum record size defined in the cluster is four bytes more than the maximum LRECL.
- When you create an OUTFIL report, the length for the longest or only data record must be equal to or greater than the maximum report record length. You can use the OUTREC, BUILD, OVERLAY, FINDREP or IFTHEN parameter to force a length for the data records that is longer than any report record; you can then either let DFSORT compute and set the LRECL, or ensure that the computed LRECL is equal to the existing or specified LRECL. Remember to allow an extra byte in the LRECL for the ANSI carriage control character.

  For example, if your data records are 40 bytes, but your longest report record is 60 bytes, you can use an OUTREC parameter such as:

  ```
  OUTREC=(1,40,80:X)
  ```

  DFSORT will then set the LRECL to 81 (1 byte for the ANSI carriage control character plus 80 bytes for the length of the data records), and pad the data records with blanks on the right.

  If you don't want the ANSI carriage control characters to appear in the output data set, use the REMOVECC parameter to remove them. For example, if you specify:

  ```
  OUTREC=(1,40,80:X),REMOVECC
  ```

  DFSORT will set the LRECL to 80 instead of 81 and remove the ANSI carriage control character from each record before it is written.

System errors can result if you print an OUTFIL report containing records longer than your printer can handle.

- DFSORT uses appropriate ANSI carriage controls (for example, C'-' for triple space) in header and trailer records when possible to reduce the number of report records written. DFSORT always uses the single space carriage control (C' ') in data records. Although these carriage control characters may not be shown when you view an OUTFIL data set (depending on how you view it), they will be used if you print the report. If you are creating a report for viewing and want blank lines to appear in headers and trailers, specify a line of blanks instead of using n/. For example, instead of specifying:

```
OUTFIL FNAMES=RPT,
 HEADER2=(2/,'start of header',2/,'next line')
```

  which will result in blank lines for the printer, but not for viewing, specify:

```
OUTFIL FNAMES=RPT,
 HEADER2=(X,/,X,/,'start of header',/,X,/,'next line')
```

  If you don't want the ANSI carriage control characters to appear in the output data set, use the REMOVECC parameter to remove them.
- For variable-length records, the first entry in the OUTREC, BUILD or IFTHEN BUILD parameter must specify or include the unedited 4-byte record descriptor word (RDW), that is, the first field must be 1,4 or 1,m with m greater than 4. DFSORT sets the length of the reformatted record in the RDW.

  If the first field in the data portion of the input record is to appear unedited in the reformatted record immediately following the RDW, the entry in the OUTREC, BUILD, or IFTHEN BUILD parameter can specify both RDW and data field in one (1,m,...). Otherwise, the RDW must be specifically included in the reformatted record (for example, 1,4,1,4,HEX).

  For variable-length OUTFIL header or trailer records, you must not specify the 4-byte RDW at the beginning of the record.
- For variable-length records, OVERLAY, IFTHEN OVERLAY or IFTHEN PUSH items must not overlay the RDW in bytes 1-4. You must ensure that 1:, 2:, 3: or 4: is not specified or defaulted for any OVERLAY or PUSH item. Note that the default for the first OVERLAY or PUSH item is 1:, so you must override it.
- With FIELDS, BUILD or IFTHEN BUILD, the variable part of the input record (that part beyond the minimum record length) can be included in the reformatted record and, if included, must be the last part. For example:

```
OUTFIL BUILD=(1,8,20C'*',9)
```

  With OVERLAY, the variable part of the input record must not be included in the reformatted record.
- For variable-length records, the FIELDS or BUILD parameter of the INREC and OUTREC statement and the OUTREC or FIELDS parameter of the OUTFIL statement must all specify position-only for the last part, or all not specify position-only for the last part. OVERLAY or IFTHEN, and FIELDS, OUTREC or BUILD, can differ with respect to position-only. See “INREC statement notes” on page 148 for more details.
- If there are no OUTFIL input records for an OUTFIL group, the headers and trailers appear without any data records. Blanks will be used for any specified unedited input fields, and zero values will be used for any specified statistics fields.
- If a variable-length OUTFIL input record is too short to contain a specified unedited input field for a report header or trailer, blanks will be used for the missing bytes. If a variable-length OUTFIL input record is too short to contain a specified section break field or statistics field, zeros will be used for the missing bytes, intentionally or unintentionally.
- If a variable-length OUTFIL input record is too short to contain an OUTFIL INCLUDE or OMIT compare field, the action DFSORT takes depends on the settings for VLSCMP/NOVLSCMP and VLSHRT/NOVLSHRT. For details, see the discussion of the VLSCMP and NOVLSCMP options in “OPTION control statement” on page 171.
- If a variable-length OUTFIL input record is too short to contain an OUTFIL OUTREC or BUILD field, DFSORT will terminate unless the VLFILL=byte parameter is specified.

- If a variable-length OUTFIL input record is too short to contain an OUTFIL OVERLAY or IFTHEN field, blanks will be used for the missing bytes.
- If a variable-length OUTFIL output data record is longer than the LRECL of its OUTFIL data set, the action DFSORT takes depends on the settings for VLLONG/NOVLLONG. For details, see the discussion of the VLLONG and NOVLLONG options in "OPTION control statement" on page 171. Note that VLLONG can be used to truncate long OUTFIL data records, but has no effect on long OUTFIL header or trailer records.
- If an unedited page number overflows 6 digits, an edited page number overflows 15 digits, a count or running count overflows 15 digits, or a total or running total overflows 31 digits, the overflowing value will be truncated to the number of digits allowed, intentionally or unintentionally.
- Multiple OUTFIL statements can be specified in the same and different sources. If a ddname occurs more than once in the same source, the ddname is associated with the first OUTFIL group in which it appears. For example, if the following is specified in SYSIN:

```
  OUTFIL FNAMES=(OUT1,OUT2),INCLUDE=(1,1,CH,EQ,C'A')
  OUTFIL FNAMES=(OUT3,OUT1),SAVE
```

  OUT1 and OUT2 are processed as part of the first OUTFIL group, that is, with INCLUDE. OUT3 is processed as part of the second OUTFIL group, that is, with SAVE; but OUT1 is not because it is a duplicate ddname.

  If a ddname occurs in more than one source, the ddname is associated with the highest source OUTFIL group in which it appears. For example, if the following is specified in DFSPARM:

```
  OUTFIL FNAMES=(OUT1,OUT2),INCLUDE=(1,1,CH,EQ,C'A')
```

  and the following is specified in SYSIN:

```
  OUTFIL FNAMES=(OUT3,OUT1),SAVE
```

  OUT1 and OUT2 are processed as part of the DFSPARM OUTFIL group, that is, with INCLUDE. OUT3 is processed as part of the SYSIN OUTFIL group, that is, with SAVE; but OUT1 is not because it is an overridden ddname.
- OUTFIL statements cannot be passed to or returned from an EFS program. The D2 format cannot be specified in the INCLUDE or OMIT parameter of an OUTFIL statement.
- If SZERO is in effect, -0 is treated as negative and +0 is treated as positive for edited or converted input fields, minimums, maximums, decimal constants and the results of arithmetic expressions. If NOSZERO is in effect, -0 and +0 are treated as positive for edited or converted input fields, minimums, maximums, decimal constants, and the results of arithmetic expressions.
- If SZERO is in effect, -0 compares as less than +0 when numeric fields and constants are used. If NOSZERO is in effect, -0 compares as equal to +0 when numeric fields and constants are used.

  Note: OPTION SZERO or OPTION NOSZERO is ignored for OUTFIL INCLUDE=(logexp) or OMIT=(logexp), or for OUTFIL IFTHEN=(WHEN=(logexp),...) unless the OPTION statement is found in a higher source (for example, DFSPARM is a higher source than SYSIN) or **before** the OUTFIL statement in the same source. For example, NOSZERO will be used in both of the following cases:

  Case 1:

```
//DFSPARM DD *
  OPTION COPY,NOSZERO
/*
//SYSIN DD *
  OUTFIL INCLUDE=(1,2,FS,EQ,+0)
/*
```

  Case 2:

```
//SYSIN DD *
  OPTION COPY,NOSZERO
  OUTFIL IFTHEN=(WHEN=(1,2,FS,EQ,+0),OVERLAY=(28:C'A')),
```

```
         IFTHEN=(WHEN=NONE,OVERLAY=(28:C'B'))
/*
```

OUTFIL features—examples

Example 1

```
OPTION COPY
OUTFIL INCLUDE=(15,6,CH,EQ,C'MSG005'),FNAMES=M005
OUTFIL INCLUDE=(15,6,CH,EQ,C'MSG022'),FNAMES=M022
OUTFIL INCLUDE=(15,6,CH,EQ,C'MSG028'),FNAMES=M028
OUTFIL INCLUDE=(15,6,CH,EQ,C'MSG115'),FNAMES=M115
OUTFIL SAVE,FNAMES=UNKNOWN
```

This example illustrates how records can be distributed to different OUTFIL data sets based on criteria you specify:

- Input records with MSG005 in bytes 15 through 20 will be written to the OUTFIL data set associated with ddname M005.
- Input records with MSG022 in bytes 15 through 20 will be written to the OUTFIL data set associated with ddname M022.
- Input records with MSG028 in bytes 15 through 20 will be written to the OUTFIL data set associated with ddname M028.
- Input records with MSG115 in bytes 15 through 20 will be written to the OUTFIL data set associated with ddname M115.
- Input records with anything else in bytes 15 through 20 will be written to the OUTFIL data set associated with ddname UNKNOWN

Example 2

```
SORT FIELDS=(18,5,ZD,D)
OUTFIL FNAMES=(V,VBU1,VBU2)
OUTFIL FNAMES=(F,FBU1),
  CONVERT,OUTREC=(11,3,X,18,5,X,X'0000000F')
OUTFIL FNAMES=VINF,OUTREC=(1,4,C'*',5,20,C'*',25)
```

This example illustrates how multiple sorted output data sets can be created and how a variable-length record data set can be converted to a fixed-length record data set:

- The first OUTFIL statement writes the variable-length input records to the variable-length OUTFIL data sets associated with ddnames V, VBU1, and VBU2.
- The second OUTFIL statement reformats the variable-length input records to fixed-length output records and writes them to the fixed-length OUTFIL data sets associated with ddnames F and FBU1. CONVERT is used to indicate that a variable-length data set is to be converted to a fixed-length data set; OUTREC is used to describe how the variable-length input records are to be reformatted as fixed-length output records.
- The third OUTFIL statement reformats the variable-length input records and writes them to the variable-length OUTFIL data set associated with ddname VINF. OUTREC is used to insert asterisks between fields. 1,4 represents the RDW. 25 represents the variable part at the end of the input record.

Example 3

```
SORT FIELDS=(15,6,ZD,A)
OUTFIL FNAMES=USA,
   HEADER2=(5:'Parts Completion Report for USA',2/,
            5:'Printed on ',DATE,
              ' at ',TIME=(12:),3/,
            5:'Part  ',20:'Completed',35:'   Value ($)',/,
            5:'------',20:'---------',35:'------------'),
   OUTREC=(5:15,6,ZD,M11,
           20:3,4,ZD,M12,LENGTH=9,
           35:38,8,ZD,M18,LENGTH=12,
```

```
            132:X)
  OUTFIL FNAMES=FRANCE,
     HEADER2=(5:'Parts Completion Report for France',2/,
              5:'Printed on ',DATE=(DM4/),
                ' at ',TIME,3/,
              5:'Part  ',20:'Completed',35:'   Value (F)',/,
              5:'------',20:'---------',35:'------------'),
     OUTREC=(5:15,6,ZD,M11,
             20:3,4,ZD,M16,LENGTH=9,
             35:38,8,ZD,M22,LENGTH=12,
             132:X)
  OUTFIL FNAMES=DENMARK,
     HEADER2=(5:'Parts Completion Report for Denmark',2/,
              5:'Printed on ',DATE=(DMY-),
                ' at ',TIME=(24.),3/,
              5:'Part  ',20:'Completed',35:'  Value (kr)',/,
              5:'------',20:'---------',35:'------------'),
     OUTREC=(5:15,6,ZD,M11,
             20:3,4,ZD,M13,LENGTH=9,
             35:38,8,ZD,M19,LENGTH=12,
             132:X)
```

This example illustrates how reports for three different countries can be produced from sorted fixed-length input records. The reports differ only in the way that date, time, and numeric formats are specified:

1. The first OUTFIL statement produces a report that has the date, time, and numeric formats commonly used in the United States.
2. The second OUTFIL statement produces a report that has the date, time, and numeric formats commonly used in France.
3. The third OUTFIL statement produces a report that has the date, time, and numeric formats commonly used in Denmark.

Of course, any one of the three reports can be produced by itself using a single OUTFIL statement instead of three OUTFIL statements. (This may be necessary if you are sorting character data according to a specified locale for that country.)

The FNAMES parameter specifies the ddname (USA, FRANCE, DENMARK) associated with the fixed-length data set for that report.

The HEADER2 parameter specifies the page header to appear at the top of each page for that report, which will consist of:

- A line of text identifying the report. Note that all English text in the report can be replaced by text in the language of that country.
- A blank line (2/).
- A line of text showing the date and time. Note that variations of the DATE, DATE=(abcd), TIME, and TIME=(abc) operands are used to specify the date and time in the format commonly used in that country.
- Two blank lines (3/).
- Two lines of text showing headings for the columns of data. Note that the appropriate currency symbol can be included in the text.

The OUTREC parameter specifies the three columns of data to appear for each input record as follows:

- A 6-byte edited numeric value produced by transforming the ZD value in bytes 15 through 20 according to the pattern specified by M11. M11 is a pattern for showing integers with leading zeros.
- A 9-byte (LENGTH=9) edited numeric value produced by transforming the ZD value in bytes 3 through 6 according to the pattern for integer values with thousands separators commonly used in that country. M12 uses a comma for the thousands separator. M16 uses a blank for the thousands separator. M13 uses a period for the thousands separator.
- A 12-byte (LENGTH=12) edited numeric value produced by transforming the ZD value in bytes 38 through 45 according to the pattern for decimal values with thousands separators and decimal separators commonly used in that country. M18 uses a comma for the thousands separator and a period

for the decimal separator. M22 uses a blank for the thousands separator and a comma for the decimal separator. M19 uses a period for the thousands separator and a comma for the decimal separator.

Table 50 on page 262 shows the twenty-seven pre-defined edit masks (M0-M26) from which you can choose.

132:X is used at the end of the OUTREC parameter to ensure that the data records are longer than the report records. This will result in an LRECL of 132 for the fixed-length OUTFIL data sets (1 byte for the ANSI control character and 131 bytes for the data).

The three reports might look as follows:

```
   Parts Completion Report for USA

   Printed on 03/25/05 at 01:56:20 pm


   Part           Completed         Value ($)
   ------         ---------      ------------
   000310               562          8,317.53
   001184             1,234         23,456.78
   029633                35            642.10
   192199             3,150        121,934.65
   821356               233          2,212.34
```

```
   Parts Completion Report for France

   Printed on 25/03/2005 at 13:56:20


   Part           Completed         Value (F)
   ------         ---------      ------------
   000310               562          8 317,53
   001184             1 234         23 456,78
   029633                35            642,10
   192199             3 150        121 934,65
   821356               233          2 212,34
```

```
   Parts Completion Report for Denmark

   Printed on 25-03-05 at 13.56.20


   Part           Completed        Value (kr)
   ------         ---------      ------------
   000310               562          8.317,53
   001184             1.234         23.456,78
   029633                35            642,10
   192199             3.150        121.934,65
   821356               233          2.212,34
```

Example 4

```
SORT FIELDS=(3,10,A,16,13,A),FORMAT=CH
OUTFIL FNAMES=WEST,
  INCLUDE=(42,6,CH,EQ,C'West'),
  HEADER1=(5/,18:'    Western Region',3/,
              18:'Profit and Loss Report',3/,
              18:'     for  ',&DATE,3/,
              18:'      Page',&PAGE),
  OUTREC=(6:16,13,24:31,10,ZD,M5,LENGTH=20,75:X),
  SECTIONS=(3,10,SKIP=P,
    HEADER3=(2:'Division:  ',3,10,5X,'Page:',&PAGE,2/,
             6:'Branch Office',24:'       Profit/(Loss)',/,
             6:'-------------',24:'--------------------'),
   TRAILER3=(6:'=============',24:'====================',/,
             6:'Total',24:TOTAL=(31,10,ZD,M5,LENGTH=20),/,
             6:'Lowest',24:MIN=(31,10,ZD,M5,LENGTH=20),/,
             6:'Highest',24:MAX=(31,10,ZD,M5,LENGTH=20),/,
             6:'Average',24:AVG=(31,10,ZD,M5,LENGTH=20),/,
             3/,2:'Average for all Branch Offices so far:',
                X,SUBAVG=(31,10,ZD,M5))),
    TRAILER1=(8:'Page ',&PAGE,5X,'Date:  ',&DATE,5/,
```

```
               8:'Total Number of Branch Offices Reporting:  ',
                 COUNT,2/,
               8:'Summary of Profit/(Loss) for all',
                 ' Western Division Branch Offices',2/,
               12:'Total:',
                  22:TOTAL=(31,10,ZD,M5,LENGTH=20),/,
               12:'Lowest:',
                  22:MIN=(31,10,ZD,M5,LENGTH=20),/,
               12:'Highest:',
                  22:MAX=(31,10,ZD,M5,LENGTH=20),/,
               12:'Average:',
                  22:AVG=(31,10,ZD,M5,LENGTH=20))
```

This example illustrates how a report can be produced with a header and trailer page and sections of columns of data, from a sorted subset of fixed-length input records.

The FNAMES parameter specifies the ddname (WEST) associated with the fixed-length data set for the report.

The INCLUDE parameter specifies the records to be selected for the report.

The HEADER1 parameter specifies the report header to appear as the first page of the report, which will consist of five blank lines (5/) followed by four lines of text, each separated by 2 blank lines (3/). The last two lines of text will show the date (&DATE) and page number (&PAGE), respectively.

The OUTREC parameter specifies the two columns of data to appear for each selected input record as follows:

- The character string from bytes 16 through 28 of the input record.
- A 20-byte (LENGTH=20) edited numeric value produced by transforming the ZD value in bytes 31 through 40 according to the pattern specified by M5.

The SECTIONS parameter specifies the section break field (3,10), page ejects between sections (SKIP=P), the header (HEADER3) to appear before each section and the trailer (TRAILER3) to appear after each section. The section header will consist of a line of text showing the page number, a blank line (2/) and two lines of text showing the headings for the columns of data. The section trailer will consist of a line of text separating the data from the trailer, lines of text showing the total (TOTAL), minimum (MIN), maximum (MAX) and average (AVG) for the data in the section as edited numeric values, two blank lines, and a line of text showing the running average (SUBAVG) for all of the data records in the report up to this point.

The TRAILER1 parameter specifies the report trailer to appear as the last page of the report, which will consist of a line of text showing the page and date, four blank lines (5/), a text line showing the count of data records in the report, a blank line, a line of text, a blank line, and lines of text showing the total, minimum maximum and average for all of the data in the report as edited numeric values.

75:X is used at the end of the OUTREC parameter to ensure that the data records are longer than the report records. This will result in an LRECL of 76 for the fixed-length OUTFIL data set (1 byte for the ANSI control character and 75 bytes for the data).

The report might look as follows:

```
                    Western Region

                 Profit and Loss Report

                      for  08/20/05

                       Page     1
```

```
Division:  Chips          Page:     2

      Branch Office            Profit/(Loss)
      -------------     --------------------
      Gilroy                      554,843.42
      Los Angeles                 (22,340.14)
      Morgan Hill                 987,322.32
      Oakland                     234,124.32
      San Francisco               (32,434.31)
      San Jose                  1,232,133.35
      San Martin                  889,022.03
      =============     ====================
      Total                     3,842,670.99
      Lowest                      (32,434.31)
      Highest                   1,232,133.35
      Average                     548,952.99


  Average for all Branch Offices so far:      548,952.99
```

```
Division:  Ice Cream      Page:     3

      Branch Office            Profit/(Loss)
      -------------     --------------------
      Marin                       542,341.23
      Napa                        857,342.83
      San Francisco               922,312.45
      San Jose                       (234.55)
      San Martin                1,003,467.30
      =============     ====================
      Total                     3,325,229.26
      Lowest                         (234.55)
      Highest                   1,003,467.30
      Average                     665,045.85


  Average for all Branch Offices so far:      597,325.02
```

```
Division:  Pretzels       Page:     4

      Branch Office            Profit/(Loss)
      -------------     --------------------
      Marin                     5,343,323.44
      Morgan Hill                 843,843.40
      Napa                      5,312,348.56
      San Francisco             5,412,300.05
      San Jose                  1,234,885.34
      San Martin                   (2,343.82)
      =============     ====================
      Total                    18,144,356.97
      Lowest                       (2,343.82)
      Highest                   5,412,300.05
      Average                   3,024,059.49


  Average for all Branch Offices so far:    1,406,236.51
```

```
         Page      5     Date:  08/20/05



        Total Number of Branch Offices Reporting:        18

        Summary of Profit/(Loss) for all Western Division Branch Offices

            Total:          25,312,257.22
            Lowest:            (32,434.31)
            Highest:         5,412,300.05
            Average:         1,406,236.51
```

Example 5

```
SORT FIELDS=(6,5,CH,A)
OUTFIL FNAMES=STATUS,
  HEADER2=(1:C'PAGE ',&PAGE,C' OF STATUS REPORT FOR ',&DATE,2/,
           6:C'ITEM ',16:C'STATUS      ',31:C'PARTS',/,
           6:C'-----',16:C'------------',31:C'-----'),
   OUTREC=(1,4,
           10:6,5,
           20:14,1,CHANGE=(12,
                           C'1',C'SHIP',
                           C'2',C'HOLD',
                           C'3',C'TRANSFER'),
                   NOMATCH=(C'*CHECK CODE*'),
           37:39,1,BI,M10,
           120:X)
```

This example illustrates how a report can be produced with a page header and columns of data from sorted variable-length input records, using a lookup table.

The FNAMES parameter specifies the ddname (STATUS) associated with the variable-length data set for the report.

The HEADER2 parameter specifies the page header to appear at the top of each page, which will consist of a line of text showing the page number (&PAGE) and date (&DATE), a blank line (2/), and two lines of text showing headings for the columns of data.

The OUTREC parameter specifies the RDW and three columns of data to appear for each input record as follows (remember that byte 5 is the first byte of data for variable-length records):

- The character string from bytes 6 through 10 of the input record
- A character string produced by finding a match for byte 14 of the input record in the table defined by CHANGE (lookup and change). NOMATCH indicates the character string to be used if byte 14 does not match any of the entries in the CHANGE table.
- An edited numeric value produced by transforming the BI value in byte 39 according to the pattern specified by M10.

With variable-length input records, you must account for the RDW when specifying the c: values for OUTREC, but not for headers or trailers. The 1: used for the first line of HEADER2 causes it to start in the first data byte (by contrast, 5: must be used to specify the first OUTREC data byte for variable-length records). Also, because 6: is used for the ITEM heading, 10: must be used for the ITEM data to get the heading and data to line up in columns.

120:X is used at the end of the OUTREC parameter to ensure that the data records are longer than the report records. This will result in a maximum LRECL of 121 for the variable-length OUTFIL data set (1 byte for the ANSI control character and a maximum of 120 bytes for the data).

The first page of the printed report might start as follows:

```
PAGE      1 OF STATUS REPORT FOR 07/18/05

     ITEM      STATUS         PARTS
     -----     ------------   -----
     00082     HOLD              36
     00123     SHIP             106
     00300     *CHECK CODE*      95
     10321     TRANSFER          18
     12140     SHIP             120
```

Example 6

```
OPTION COPY
OUTFIL FNAMES=(A1,A2,A3),SPLIT
OUTFIL FNAMES=(B1,B2,B3),SPLITBY=25
OUTFIL FNAMES=(C1,C2,C3),SPLIT1R=25
```

This example illustrates different ways to split 77 input records among three OUTFIL data sets.

The first OUTFIL statement uses SPLIT to rotate the records among the three OUTFIL data sets one record at a time. A1 will have records 1, 4, ...,76. A2 will have records 2, 5, ...,77. A3 will have records 3, 6, ...,75. A1 will have 26 records. A2 will have 26 records. A3 will have 25 records. A1, A2 and A3 will each have non-contiguous records.

The second OUTFIL statement uses SPLITBY=25 to rotate the records among the three OUTFIL data sets 25 records at a time. B1 will have records 1-25 and 76-77. B2 will have records 26-50. B3 will have records 51-75. B1 will have 27 records. B2 will have 25 records. B3 will have 25 records. B1 will have non-contiguous records. B2 and B3 will have contiguous records.

The third OUTFIL statement uses SPLIT1R=25 to rotate the records once among the three OUTFIL data sets 25 records at a time. C1 will have records 1-25. C2 will have records 26-50. C3 will have records 51-77. C1 will have 25 records. C2 will have 25 records. C3 will have 27 records. C1, C2 and C3 will have contiguous records.

Example 7

```
OPTION COPY
OUTFIL FNAMES=RANGE1,ENDREC=1000000
OUTFIL FNAMES=RANGE2,STARTREC=1000001,ENDREC=2000000
OUTFIL FNAMES=RANGE3,STARTREC=2000001,ENDREC=3000000
OUTFIL FNAMES=RANGE4,STARTREC=3000001,ENDREC=4000000
OUTFIL FNAMES=(RANGE5,EXTRA),STARTREC=4000001
```

This example illustrates how specific ranges of output records can be written to different output data sets. A typical application might be database partitioning.

The first 1 million records will be written to the data set associated with RANGE1, the second million to RANGE2, the third million to RANGE3, and the fourth million to RANGE4. The remaining records will be written to both the data set associated with RANGE5 and the data set associated with EXTRA (SAVE or STARTREC=4000001 will accomplish the same purpose in this case).

Note that the INCLUDE, OMIT, and SAVE parameters of OUTFIL can also be used to select records to be written to different output data sets, based on criteria you specify.

Example 8

```
OPTION COPY,Y2PAST
OUTFIL FNAMES=Y4,
       OUTREC=(1,19,
               21,2,PD0,M11,C'/',  transform mm
               22,2,PD0,M11,C'/',  transform dd
               20,2,Y2P,           transform yy to yyyy
               24,57)
```

This example illustrates how to transform an existing data set with a packed decimal date field of the form P'yymmdd' (X'0yymmddC') in bytes 20-23 into a new data set with a character date field of the form C'mm/dd/yyyy' in bytes 20-29. yy represents the two-digit year, yyyy represents the four-digit year, mm represents the month, dd represents the day, and C represents a positive sign.

The input data set has an LRECL of 80 and the Y4 data set will have an LRECL of 86.

The Y2PAST=26 option sets the century window to be used to transform two-digit years into four-digit years. If the current year is 2006, the century window will be 1980 to 2079. Using this century window, the input and output fields might be as follows:

```
Input Field (HEX)              Output Field (CH)
   20                             20
    |                              |
    0020505F                       05/05/2002
    0950823C                       08/23/1995
    0980316C                       03/16/1998
    0000316F                       03/16/2000
```

Example 9

```
OPTION COPY,Y2PAST=1996
OUTFIL FNAMES=SPCL,
       OUTREC=(1,14, copy positions 1-14
          15,6,Y2T,  transform yy to yyyy - allow blanks
          21,20)     copy positions 21 - 40
```

This example illustrates how to transform an existing data set with a character date field of the form C'yymmdd' and blank special indicators in bytes 15-20, into a new date set with a character date field of the form C'yyyymmdd' and blank special indicators in bytes 15-22.

The input data set has an LRECL of 40 and the SPCL data set will have an LRECL of 42.

The Y2PAST=1996 option sets the century window to 1996-2095. The century window will be used to transform the two-digit years into four-digit years, but will not be used for the special blank indicators.

The input records might be as follows:

```
MORGAN HILL                   CA
SAN JOSE             960512   CA
BOCA RATON           000628   FL
DENVER               951115   CO
```

The output records would be as follows:

```
MORGAN HILL                     CA
SAN JOSE             19960512   CA
BOCA RATON           20000628   FL
DENVER               20951115   CO
```

Example 10

```
OPTION COPY
OUTFIL FNAMES=ALL,OUTREC=(C'US ',1,10,C' is in ',11,15,/,
                          C'WW ',1,10,C' is in ',26,20,2/)
OUTFIL FNAMES=(US,WW),SPLIT,
                      OUTREC=(1,10,C' is in ',11,15,/,
                              1,10,C' is in ',26,20)
```

This example illustrates how multiple OUTFIL output and blank records can be produced from each OUTFIL input record. The input data set has an LRECL of 50 and contains the following three records:

```
Finance   San Francisco  Buenos Aires
Research  New York       Amsterdam
Marketing Los Angeles    Mexico City
```

The first OUTFIL statement creates the data set associated with ddname ALL. This data set will have an LRECL of 40 (the length of the longest output record; the one that includes the 26,20 input field). Each input record will result in two data records followed by two blank records as follows:

`ALL data set`

```
US Finance    is in San Francisco
WW Finance    is in Buenos Aires


US Research   is in New York
WW Research   is in Amsterdam


US Marketing  is in Los Angeles
WW Marketing  is in Mexico City
```

The second OUTFIL statement creates the two data sets associated with ddnames US and WW. These data sets will have an LRECL of 37 (the length of the longest output record; the one that includes the 26,20 input field). Each input record will result in two data records. SPLIT will cause the first data record

to be written to the US data set and the second data record to be written to the WW data set. Thus, each input record will create one record in each OUTFIL data set as follows:

US data set

```
Finance    is in San Francisco
Research   is in New York
Marketing  is in Los Angeles
```

WW data set

```
Finance    is in Buenos Aires
Research   is in Amsterdam
Marketing  is in Mexico City
```

Example 11

```
  SORT FIELDS=(6,3,CH,D)
  OUTFIL FNAMES=SET60,BUILD=(1,60),VLFILL=C' '
  OUTFIL FNAMES=VARFIX,VTOF,BUILD=(5,20,5X,28,20),VLFILL=C'*'
```

This example illustrates how variable-length records that are too short to contain all OUTFIL BUILD fields can be processed successfully.

The input data set has RECFM=VB and LRECL=80. The records in this data set have lengths that vary from 15 bytes to 75 bytes.

The first OUTFIL statement creates the data set associated with ddname SET60. This data set will have RECFM=VB and LRECL=60. Every record in this data set will have a length of 60. The 1,60 field truncates records longer than 60 bytes to 60 bytes. Because VLFILL=C' ' is specified, the 1,60 field pads records shorter than 60 bytes to 60 bytes using a blank (C' ') as the fill byte.

Note: Without VLFILL=byte, this OUTFIL statement would terminate with an ICE218A message because some of the input records are too short to contain the BUILD field.

The second OUTFIL statement creates the data set associated with ddname VARFIX. This data set will have RECFM=FB and LRECL=45. VTOF changes the variable-length input records to fixed-length output records according to the fields specified by BUILD. VLFILL=C'*' allows short input records to be processed. Each missing byte in an OUTFIL BUILD field is replaced with an asterisk (C'*') fill byte.

Note:

1. CONVERT can be used instead of VTOF.
2. VLFILL=C'*' overrides the default of VLFILL=X'40' for VTOF or CONVERT.

Example 12

```
OPTION COPY
  OUTFIL OUTREC=(SEQNUM,4,BI,Z,8,5,ZD,TO=PD,Z,
         31,2,PD,TO=FI,LENGTH=2),Z,
         16,3,ZD,ADD,+1,TO=FI,LENGTH=2,Z,
         (16,3,ZD,MAX,31,2,PD),MUL,+2,TO=ZD,LENGTH=4)
```

This example illustrates how a sequence number can be generated, how values in one numeric format can be converted to another numeric format, and how arithmetic expressions involving fields and decimal constants can be used.

The input data set has an LRECL of 50 and the SORTOUT data set will have an LRECL of 19.

The OUTFIL statement creates output records with the following fields:

- A binary sequence number in bytes 1-4 that starts at 1 and increments by 1.
- X'00' in byte 5.
- A PD field in bytes 6-8 containing the converted ZD field from input bytes 8-12
- X'00' in position 9.

- An FI field in bytes 10-11 containing the converted PD field from input bytes 31-32.
- X'00' in position 12.
- An FI field in bytes 13-14 containing the converted result of the ZD field from input bytes 16-18 incremented by 1.
- X'00' in position 15.
- A ZD field in bytes 16-19 containing the converted result of the maximum of the ZD field from input bytes 16-18 and the PD field from input bytes 31-32, multiplied by 2.

Example 13

```
SORT FIELDS=COPY
OUTFIL FNAMES=VAROUT1,FTOV
OUTFIL FNAMES=VAROUT2,FTOV,
  OUTREC=(20,8,35,10)
OUTFIL FNAMES=VAROUT3,FTOV,VLTRIM=X'40'
```

This example illustrates several ways to convert a fixed-length record data set to a variable-length record data set using the FTOV parameter of OUTFIL.

The input data set has an RECFM=FB and LRECL=60.

- The first OUTFIL statement converts the fixed-length input data set to a variable-length OUTFIL data set associated with ddname VAROUT1. VAROUT1 will have RECFM=VB and LRECL=64. All of its records will be 64 bytes long (4-byte RDW plus 60-byte input record).
- The second OUTFIL statement converts the fixed-length input data set to a variable-length OUTFIL data set associated with ddname VAROUT2. OUTREC is used to select two input fields for the output records, bytes 20-27 and bytes 35-44. VAROUT2 will have RECFM=VB and LRECL=22. All of its records will be 22 bytes long (4-byte RDW plus 8-byte input field plus 10-byte input field).
- The third OUTFIL statement converts the fixed-length input data set to a variable-length OUTFIL data set associated with ddname VAROUT3. VAROUT3 will have RECFM=VB and LRECL=64. VLTRIM=X'40' is used to remove the trailing blanks from the variable-length output records. The records can vary from 5 bytes long to 64 bytes long depending on the number of trailing blanks in each record.

Example 14

```
OPTION COPY
OUTFIL FNAMES=OUT1,OUTREC=(DATE1(/),X,TIME1(:),X,1,80)
OUTFIL FNAMES=OUT2,OUTREC=(DATE2P,TIME3P,1,80)
OUTFIL FNAMES=OUT3,OUTREC=(DATE3(.),X,TIME2,X,1,80)
```

This example illustrates several different ways to insert timestamps into your records.

The input data set has RECFM=FB and LRECL=80.

The first OUTFIL statement creates the data set associated with ddname OUT1. This data set will have LRECL=100. Each output record will have a timestamp consisting of the date and time of the run in the form C'yyyy/mm/dd hh:mm:ss ' (20 bytes), followed by the original input record (80 bytes).

The second OUTFIL statement creates the data set associated with ddname OUT2. This data set will have LRECL=86. Each output record will have a timestamp consisting of the date of the run in the form P'yyyymm' (4 bytes) and the time of the run in the form P'hh' (2 bytes), followed by the original input record (80 bytes).

The third OUTFIL statement creates the data set associated with ddname OUT3. This data set will have LRECL=94. Each output record will have a timestamp consisting of the date and time of the run in the form C'yyyy.ddd hhmm ' (14 bytes), followed by the original input record (80 bytes).

Example 15

```
OPTION COPY
OUTREC FIELDS=(1,4,11,4,DT1,7,4,TM1,60:X)
```

```
OUTFIL NODETAIL,
 TRAILER1=(//,
  3:'Earliest SMF timestamp is  ',
     MIN=(5,14,ZD,EDIT=('TTTT/TT/TT TT:TT:TT')),/,
  3:'Latest SMF timestamp is    '
     MAX=(5,14,ZD,EDIT=('TTTT/TT/TT TT:TT:TT')))
```

This example illustrates how the earliest and latest timestamps from a set of SMF records can be displayed.

The OUTREC statement uses the DT1 format to convert the SMF date in input bytes 11-15 to a Z'yyyymmdd' value in bytes 5-12, and uses the TM1 format to convert the SMF time in input bytes 7-10 to a Z'hhmmss' value in bytes 13-18.

The OUTFIL statement uses the Z'yyyymmddhhmmss' value created by OUTREC in bytes 5-18 to determine the minimum (earliest) and maximum (latest) timestamp, and displays those timestamps in a trailer record in the form C'yyyy/mm/dd hh:mm:ss'.

The report might look as follows:

```
 Earliest SMF timestamp is  2001/01/09 10:27:04
 Latest SMF timestamp is    2001/04/24 06:13:22
```

Example 16

```
SORT FIELDS=(1,20,BI,A)
OUTFIL FNAMES=FUPPER,OUTREC=(1,80,TRAN=LTOU)
OUTFIL FNAMES=FHEX,OUTREC=(1,80,HEX)
```

This example illustrates two types of conversion for fixed length records: lowercase to uppercase conversion and hex conversion.

The input data set has RECFM = FB and LRECL = 80.

The first OUTFIL statement creates the data set associated with ddname FUPPER. This data set will have RECFM = FB and LRECL = 80. All of the lowercase EBCDIC characters (a-z) from byte 1 to byte 80 will be converted to uppercase EBCDIC characters (A-Z). Other characters will remain unchanged. For example, the characters 'san jose, ca 95193' will be converted to 'SAN JOSE, CA 95193'.

The second OUTFIL statement creates the data set associated with ddname FHEX. This data set will have RECFM = FB and LRECL = 160 (2 * 80 data bytes). Each byte from 1 to 80 will be converted to the two bytes representing its hex value. For example, the three characters 'A12' will be converted to the six characters 'C1F1F2'.

Example 17

```
OPTION COPY
OUTFIL FNAMES=VUPPER,OUTREC=(1,4,5,TRAN=UTOL)
OUTFIL FNAMES=VHEX,OUTREC=(1,4,5,HEX)
```

This example illustrates two types of conversion for variable-length records: uppercase to lowercase conversion and hex conversion.

The input data set has RECFM = VB and LRECL = 5000.

The first OUTFIL statement creates the data set associated with ddname VUPPER. This data set will have RECFM = VB and LRECL = 5000. All of the uppercase EBCDIC characters (A-Z) from bytes 5 (after the RDW) to the end of each record will be converted to lowercase EBCDIC characters (a-z). Other characters will remain unchanged. For example, the characters 'SAN JOSE, CA 95193' will be converted to 'san jose, ca 95193'.

The second OUTFIL statement creates the data set associated with ddname VHEX. This data set will have RECFM = VB and LRECL = 9996 (4 for RDW plus 2 * 4996 data bytes). Each byte from 5 (after the RDW) to the end of each record will be converted to the two bytes representing its hex value. For example, the three characters 'A12' will be converted to the six characters 'C1F1F2'.

Example 18

```
SORT FIELDS=(22,8,CH,A)
OUTFIL FNAMES=SAMP1,REMOVECC,HEADER1=('Sample 1',2/),
  STARTREC=120,SAMPLE=(20,4),
  OUTREC=(1,8,9,5,PD,ZD,C'***',14,50)
OUTFIL FNAMES=SAMP2,REMOVECC,HEADER1=('Sample 2',2/),
  SAMPLE=1000,ENDREC=5001
```

This example illustrates how to take different "samples" of sorted output records. Sorted records 120, 121, 122, 123, 140, 141, 142, 143, and so on will be reformatted as indicated by the OUTREC parameter and written to the output data set associated with SAMP1. The heading 'Sample 1' will appear before the sample output records.

Sorted records 1, 1001, 2001, 3001, 4001 and 5001 will be written to the output data set associated with SAMP2. The heading 'Sample 2' will appear before the sample output records.

Example 19

```
OPTION COPY
OUTFIL FNAMES=R500,REPEAT=500
OUTFIL FNAMES=R100,OUTREC=(SEQNUM,4,ZD,80X'FF'),REPEAT=100
```

This example illustrates how one record can be used to generate many records.

The first OUTFIL statement writes each output record 500 times to the data set associated with R500. Each set of 500 records will be identical.

The second OUTFIL statement writes each reformatted output record 100 times to the data set associated with R100. Each set of 100 records will be identical except for the sequence number. The 100 records written from the first output record will have sequence numbers 1-100, the 100 records written from the second output record will have sequence numbers 101-200, and so on.

Example 20

```
OPTION COPY
OUTFIL OMIT=(56,6,CH,EQ,C'******'),
  OVERLAY=(121:SEQNUM,8,ZD,56:56,6,TRAN=UTOL)
```

This example illustrates how you can use the OVERLAY parameter with OUTFIL to add sequence numbers at the end of your records, and to convert uppercase EBCDIC characters to lowercase EBCDIC characters in certain columns, without affecting the rest of the record.

The input data set has RECFM=FB and LRECL=120. The output data set will have RECFM=FB and LRECL=128.

The OMIT parameter removes records which have asterisks in positions 56-61.

121:SEQNUM,8,ZD in the OVERLAY parameter adds an 8-byte sequence number in positions 121-128 of every record. The LRECL is increased from 120 to 128 to hold the sequence number.

56:56,6,TRAN=UTOL in the OVERLAY parameter converts uppercase EBCDIC characters in positions 56-61 to lowercase EBCDIC characters in positions 56-61.

Only the two overlaid fields are changed; all of the other data in the records is unaffected.

Example 21

```
OPTION COPY
OUTFIL FNAMES=RPT1,
  HEADER2=(1:'Type',9:'Date',21:'Time',/,
           1:'----',9:'--------',21:'------'),
  IFTHEN=(WHEN=(5,2,BI,EQ,X'0001'),
    BUILD=(1,4,5:5,2,BI,M11,LENGTH=4,
           13:8,4,DT1,25:15,4,TM1)),
  IFTHEN=(WHEN=(5,2,BI,EQ,X'0002'),
    BUILD=(1,4,5:5,2,BI,M11,LENGTH=4,
```

```
             13:12,8,DC1,25:12,8,TC1)),
    IFTHEN=(WHEN=(5,2,BI,EQ,X'0003'),
      BUILD=(1,4,5:5,2,BI,M11,LENGTH=4,
             13:17,8,DE1,25:17,8,TE1)),
    IFTHEN=(WHEN=NONE,
      BUILD=(1,4,5:5,2,BI,M11,LENGTH=4,
             13:C'n/a',25:C'n/a'))
  OUTFIL FNAMES=RPT2,
    HEADER2=(1:'Type',9:'Date',21:'Time',/,
             1:'----',9:'-------',21:'--------'),
    IFTHEN=(WHEN=(5,2,BI,EQ,X'0001'),
      BUILD=(1,4,5:5,2,BI,M11,LENGTH=4,
            13:8,4,DT3,25:15,4,TM4)),
    IFTHEN=(WHEN=(5,2,BI,EQ,X'0002'),
      BUILD=(1,4,5:5,2,BI,M11,LENGTH=4,
            13:12,8,DC3,25:12,8,TC4)),
    IFTHEN=(WHEN=(5,2,BI,EQ,X'0003'),
      BUILD=(1,4,5:5,2,BI,M11,LENGTH=4,
            13:17,8,DE3,25:17,8,TE4)),
    IFTHEN=(WHEN=NONE,
      BUILD=(1,4,5:5,2,BI,M11,LENGTH=4,
            13:C'n/a',25:C'n/a'))
```

This example illustrates how you can use IFTHEN clauses to reformat different records in different ways.

The input data set has RECFM=VB and consists of several different types of input records as follows:

- Type1: Has X'0001' in positions 5-6, a 4-byte SMF date in positions 8-11 and a 4-byte SMF time in positions 15-18.
- Type2: Has X'0002' in positions 5-6 and an 8–byte TOD date and time in positions 12–19.
- Type3: Has X'0003' in positions 5-6 and an 8–byte ETOD date and time in positions 17–24.
- Other types: Do not have X'0001', X'0002' or X'0003' in positions 5-6, and do not have a date or time value.

The first OUTFIL statement produces a report with RECFM=VBA and LRECL=31 that might look like this:

```
Type    Date        Time
----    --------    ------
0001    20040827    124531
0003    20040907    230603
0008    n/a         n/a
0002    20040901    061559
0001    20040731    152201
0004    n/a         n/a
```

The first IFTHEN clause operates only against Type1 records; it converts the SMF date using DT1 and the SMF time using TM1. The second IFTHEN clause only operates against Type2 records; it converts the TOD date using DC1 and the TOD time using TC1. The third IFTHEN clause only operates against Type3 records; it converts the ETOD date using DE1 and the ETOD time using TE1. The fourth IFTHEN clause operates against all other types of records; it uses 'n/a' for the date and time.

The second OUTFIL statement produces a report with RECFM=VBA and LRECL=33 that looks like this:

```
Type    Date        Time
----    -------     --------
0001    2004240     12453184
0003    2004251     23060373
0008    n/a         n/a
0002    2004245     06155903
0001    2004213     15220150
0004    n/a         n/a
```

The first IFTHEN clause operates only against Type1 records; it converts the SMF date using DT3 and the SMF time using TM4. The second IFTHEN clause operates only against Type2 records; it converts the TOD date using DC3 and the TOD time using TC4. The third IFTHEN clause operates only against Type3 records; it converts the ETOD date using DE3 and the ETOD time using TE4. The fourth IFTHEN clause operates against all other types of records; it uses 'n/a' for the date and time.

Example 22

```
  OPTION COPY
  OUTFIL BUILD=(1,80,SQZ=(SHIFT=LEFT,PAIR=APOST,MID=C','))
```

This example illustrates how you can create FB output records with comma separated values from FB input records containing fields in fixed positions.

The 80-byte FB input records might look like this:

```
  'John Lewis'         -12.83  'Research'
  'Ted Blank'         +128.37  'Manufacturing'
  'Marilyn Carlson'   -282.83  'Technical Support'
  'Rex Otis'            +2.83  'Marketing'
```

Note that the data has three fields in fixed positions. The first field is a character string surrounded by apostrophes. The second field is a numeric value. The third field is another character string surrounded by apostrophes.

The 80-byte FB output records are in the form of comma separated values as follows:

```
'John Lewis',-12.83,'Research'
'Ted Blank',+128.37,'Manufacturing'
'Marilyn Carlson',-282.83,'Technical Support'
'Rex Otis',+2.83,'Marketing'
```

We use OUTFIL BUILD to build the output records with comma separated values. We use SQZ to squeeze out the blanks between the fields, shift the remaining characters to the left and insert commas between the fields. SHIFT=LEFT shifts the characters to the left. PAIR=APOST ensures that blanks within paired apostrophes are not squeezed out (for example, we want to keep the blank in the 'John Lewis' field.) MID=C',' inserts a comma for each group of blanks removed between the fields (for example, between 'John Lewis' and -12.83).

Example 23

```
   OPTION COPY
   OUTFIL IFTHEN=(WHEN=INIT,
     OVERLAY=(5:5,18,JFY=(SHIFT=LEFT,LEAD=C'''',TRAIL=C''''),
              34:34,19,JFY=(SHIFT=LEFT,LEAD=C'''',TRAIL=C''''))),
    IFTHEN=(WHEN=INIT,
     BUILD=(1,4,5,60,SQZ=(SHIFT=LEFT,PAIR=APOST,MID=C','))),
    VLTRIM=C' '
```

This example illustrates how you can create VB output records with comma separated values from VB input records containing fields in fixed positions.

The 84-byte VB input records might look like this (4-byte RDW followed by data):

```
Length   |  Data
    45       John Lewis          -12.83  Research
    50       Ted Blank          +128.37  Manufacturing
    54       Marilyn Carlson    -282.83  Technical Support
    51       Rex Otis             +2.83  Marketing
```

Note that the data has three fields in fixed positions. The first field is a character string. The second field is a numeric value. The third field is another character string. (In the previous example, the character strings were surrounded by apostrophes; in this example the apostrophes must be added around the character strings.)

The 84-byte VB output records are in the form of comma separated values as follows:

```
Length   |  Data
    34      'John Lewis',-12.83,'Research'
    39      'Ted Blank',+128.37,'Manufacturing'
    49      'Marilyn Carlson',-282.83,'Technical Support'
    32      'Rex Otis',+2.83,'Marketing'
```

We use OUTFIL IFTHEN to ensure that short records are padded on the right with blanks. (OUTFIL BUILD would terminate due to the short records.) WHEN=INIT reformats every record. OVERLAY surrounds the character strings with apostrophes. BUILD builds the output records with comma separated values.

We use JFY to surround the character strings with apostrophes without removing embedded blanks (for example, John Lewis is changed to 'John Lewis'). After the first IFTHEN, the records look like this:

```
Length   |  Data
    52      'John Lewis'          -12.83 'Research'
    52      'Ted Blank'          +128.37 'Manufacturing'
    54      'Marilyn Carlson'    -282.83 'Technical Support'
    52      'Rex Otis'             +2.83 'Marketing'
```

We then use SQZ to squeeze out the blanks between the fields, shift the remaining characters to the left and insert commas between the fields. SHIFT=LEFT shifts the characters to the left. PAIR=APOST ensures that blanks within paired apostrophes are not squeezed out (for example, we want to keep the blank in the 'John Lewis' field.) MID=C',' inserts a comma for each group of blanks removed between the fields (for example, between 'John Lewis' and -12.83).

Finally, we use VLRTIM=C' ' to remove trailing blanks at the end of each VB record by adjusting its length appropriately.

Example 24

```
  OPTION COPY
  OUTFIL REMOVECC,
   HEADER1=(C'<?xml version="1.0"?>',/,
     3:C'<booklist>'),
   BUILD=(5:C'<book>',/,
     7:1,20,JFY=(SHIFT=LEFT,LEAD=C'<title>',TRAIL=C'</title>',
       LENGTH=36),/,
     7:24,15,SQZ=(SHIFT=LEFT,LEAD=C'<author>',MID=C', ',
       TRAIL=C'</author>',LENGTH=33),/,
     5:C'</book>'),
   TRAILER1=(3:C'</booklist>')
```

This example illustrates how you can generate XML statements from FB input records containing fields in fixed positions.

The 40-byte FB input records might look like this:

```
Modern Poetry          Friedman    KR
Intro to Computers     Chatterjee  CL
Marketing              Maxwell     G
```

Note that the data has three character fields in fixed positions.

The 42-byte FB output records look like this:

```
<?xml version="1.0"?>
  <booklist>
    <book>
      <title>Modern Poetry</title>
      <author>Friedman, KR</author>
    </book>
    <book>
      <title>Intro to Computers</title>
      <author>Chatterjee, CL</author>
    </book>
    <book>
      <title>Marketing</title>
      <author>Maxwell, G</author>
    </book>
  </booklist>
```

We use OUTFIL HEADER1 to generate the xml and booklist starting tags that precede the set of tags for each record. We use OUTFIL BUILD to generate the set of tags for each record as follows:

- A constant is used to generate the book starting tag.

- JFY is used to generate the title tags and data from the first input field. LEAD generates the title starting tag before the input field from the record. TRAIL generates the title ending tag after the last nonblank character from the input field. JFY keeps embedded blanks between the first nonblank character and the last nonblank character of the input field. LENGTH ensures that the addition of the LEAD and TRAIL strings does not cause truncation by increasing the output length to 36 bytes (overriding the default of 20 bytes from the input field).
- SQZ is used to generate the author tags and data from the second and third input fields. LEAD generates the author starting tag before the input field from the record. MID replaces the blanks between the second and third input fields with a comma and one blank. TRAIL generates the author ending tag after the last nonblank character from the input fields. LENGTH ensures that the addition of the LEAD, MID and TRAIL strings does not cause truncation by increasing the output length to 33 bytes (overriding the default of 15 bytes from the input field).

We use OUTFIL TRAILER1 to generate the booklist ending tag that follows the set of tags for each record.

Example 25

```
  OPTION COPY
  OUTFIL HEADER2=(1:'First',13:'Initial',22:'Last',37:'Score',/,
                  1:10'-',13:7'-',22:11'-',35:7'-'),
     PARSE=(%00=(STARTAFT=C'LAST-> ',ENDBEFR=C' ',FIXLEN=11),
            %01=(STARTAFT=C'FIRST-> ',ENDBEFR=C' ',FIXLEN=11),
            %02=(STARTAFT=C'INITIAL-> ',ENDBEFR=C' ',FIXLEN=7),
            %03=(STARTAFT=C'SCORE-> ',FIXLEN=7)),
     BUILD=(1:%01,13:%02,22:%00,
            35:%03,SFF,EDIT=(SIIIT.T),SIGNS=(,-))
```

This example illustrates how you can create a report from FB input records with variable position/length fields, such as keyword delimited values.

The 70-byte input records might look like this:

```
     LAST-> Clark   FIRST-> Oscar     INITIAL-> D   SCORE-> +98.2
  LAST-> Roberts  FIRST-> Harriet   INITIAL->    SCORE-> -152.6
   LAST-> Stein FIRST-> Gertrude INITIAL-> V SCORE-> +5.1
```

Note that each record has four variable fields each identified by a specific keyword. The fields do not start and end in the same position in every record and they have different lengths in different records.

The output report has RECFM=FBA and LRECL=42 and looks like this:

```
First       Initial  Last          Score
----------  -------  -----------  -------
Oscar       D        Clark          98.2
Harriet              Roberts      -152.6
Gertrude    V        Stein           5.1
```

In order to create a report like this from variable position/length fields, we use HEADER2 to create the page header, and PARSE and BUILD to create a fixed parsed field from each variable field. We use %00 to create an 11-byte fixed parsed field with the extracted 'LAST-> ' value. We use %01 to create an 11-byte fixed parsed field with the extracted 'FIRST-> ' value. We use %02 to create a 7-byte fixed parsed field with the extracted 'INITIAL-> ' value. We use %03 to create a 7-byte fixed parsed field with the extracted 'SCORE-> ' value. Since the fourth field (%03) is extracted as signed and padded on the right with blanks, we treat it as SFF format in order to edit it.

Example 26

```
  INREC IFTHEN=(WHEN=INIT,
     PARSE=(%1=(FIXLEN=10,STARTAFT=C'First="',ENDBEFR=C'"')),
    IFTHEN=(WHEN=INIT,
     PARSE=(%2=(FIXLEN=10,STARTAFT=C'Middle="',ENDBEFR=C'"')),
    IFTHEN=(WHEN=INIT,
     PARSE=(%3=(FIXLEN=10,STARTAFT=C'Last="',ENDBEFR=C'"')),
    IFTHEN=(WHEN=INIT,
     PARSE=(%4=(FIXLEN=10,STARTAFT=C'Wife="',ENDBEFR=C'"')),
     BUILD=(%1,13:%2,25:%3,37:%4))
```

```
  SORT FIELDS=(25,10,CH,A,1,10,CH,A,13,10,CH,A)
  OUTFIL HEADER2=('First',13:'Middle',
      25:C'Last',37:'Wife',/,10C'-',13:10C'-',
      25:10C'-',37:10C'-')
```

This example illustrates how you can create a sorted report from FB input records with keyword values that can occur in any order or not occur at all.

The 80-byte input records might look like this:

```
   Last="Buchanan" First="James"
  Wife="Louisa" First="John" Middle="Quincy" Last="Adams"
   First="George" Last="Washington" Wife="Martha"
Last="Clinton" Wife="Hillary" Middle="Jefferson" First="William"
  First="John" Wife="Abigail" Last="Adams"
```

Note that each record has up to four variable fields each identified by a specific keyword (First, Middle, Last, Wife). In each record, the fields can be in any order, do not start and end in the same position and can have different lengths.

We want to sort the records by last name, first name and middle name, and create a report with fixed headings and values for the first name, middle name, last name and wife's name that looks like this:

```
First       Middle      Last        Wife
----------  ----------  ----------  ----------
John                    Adams       Abigail
John        Quincy      Adams       Louisa
James                   Buchanan
William     Jefferson   Clinton     Hillary
George                  Washington  Martha
```

In order to extract and sort variable fields like these, we use a separate IFTHEN PARSE clause for each keyword. This allows us to find a particular keyword anywhere in the record or ignore it if it is not in the record. If we used PARSE instead of IFTHEN PARSE, a missing keyword would cause any keywords that followed to be ignored. But because each IFTHEN PARSE starts scanning at position 1 by default, we can look for each keyword independently of the others. If a keyword is missing, the parsed field is set to all blanks (for example, %2 and %4 are set to blanks for the James Buchanan record).

We use %1 to create a 10-byte fixed parsed field into which we extract the first name. We use %2 to create a 10-byte fixed parsed field into which we extract the middle name. We use %3 to create a 10-byte fixed parsed field into which we extract the last name. We use %4 to create a 10-byte fixed parsed field into which we extract the wife's name. We use BUILD to reformat the records to contain the fixed parsed fields. Then we SORT on the fixed parsed fields. Finally, we use OUTFIL to create the headings.

Example 27

```
  OPTION COPY
  OUTFIL FINDREP=(INOUT=(C'AM',C'IN THE MORNING',
    C'PM',C'IN THE EVENING'),MAXLEN=70)
```

This example illustrates how you can replace values in FB or VB records with larger values and shift the rest of the bytes to the right.

The 40 byte FB input records might look like this:

```
COFFEE AT 12:28 AM, TOAST AT 06:15 PM
MILK AT 03:17 PM, BAGELS AT 05:03 PM
PUDDING AT 09:32 AM
```

We want to replace each instance of 'AM' with 'IN THE MORNING' and each instance of 'PM' with 'IN THE EVENING' and shift the bytes after 'AM' or 'PM' to the right. We use INOUT to indicate find and replace pairs. Since replacing the smaller values with the larger values can cause the remaining bytes to be shifted beyond the end of the 40 byte record, we use MAXLEN to set the output record to 70 bytes to allow for the expansion, overriding the default of using the input length for the output record.

The 70 byte FB output records look like this:

```
COFFEE AT 12:28 IN THE MORNING, TOAST AT 06:15 IN THE EVENING
MILK AT 03:17 IN THE EVENING, BAGELS AT 05:03 IN THE EVENING
PUDDING AT 09:32 IN THE MORNING
```

Example 28

```
  OPTION COPY
  OUTREC IFTHEN=(WHEN=INIT,BUILD=(1,4,8:5)),
    IFTHEN=(WHEN=GROUP,BEGIN=(8,5,CH,EQ,C'PAGE:'),
    PUSH=(5:SEQ=3))
  OUTFIL INCLUDE=(5,3,ZD,EQ,2,OR,5,3,ZD,EQ,3),
    BUILD=(1,4,5:8)
```

This example illustrates how you can INCLUDE specific relative records from groups of VB records. We insert a sequence number between the RDW and the first data byte of each record. The sequence number restarts at 1 for the first record in each group. We INCLUDE on the sequence number and then remove it.

The VB input records might look like this:

```
Len|Data
 12|PAGE:  1
 22|LINE 1 OF REPORT A
 22|LINE 2 OF REPORT A
 22|LINE 3 OF REPORT A
 22|LINE 4 OF REPORT A
   |...
 12|PAGE:  2
 23|LINE 66 OF REPORT A
 23|LINE 67 OF REPORT A
 23|LINE 68 OF REPORT A
 23|LINE 69 OF REPORT A
   |...
 12|PAGE:  3
 24|LINE 131 OF REPORT A
 24|LINE 132 OF REPORT A
 24|LINE 133 OF REPORT A
 24|LINE 134 OF REPORT A
   |...
```

We use an IFTHEN WHEN=INIT clause to reformat each record so it has room for the 3-byte sequence number between the RDW and the first data byte. After the WHEN=INIT clause is executed, the intermediate records look like this:

```
Len|Data
 15|   PAGE:  1
 25|   LINE 1 OF REPORT A
 25|   LINE 2 OF REPORT A
 25|   LINE 3 OF REPORT A
 25|   LINE 4 OF REPORT A
   |   ...
 15|   PAGE:  2
 26|   LINE 66 OF REPORT A
 26|   LINE 67 OF REPORT A
 26|   LINE 68 OF REPORT A
 26|   LINE 69 OF REPORT A
   |   ...
 15|   PAGE:  3
 27|   LINE 131 OF REPORT A
 27|   LINE 132 OF REPORT A
 27|   LINE 133 OF REPORT A
 27|   LINE 134 OF REPORT A
```

Note that positions 5-7 are blank and the 'PAGE:' characters have been shifted over to positions 8-12.

We use an IFTHEN WHEN=GROUP clause to put a 3-byte sequence number in each record. BEGIN indicates a group starts with a record that has 'PAGE:' in positions 8-12. PUSH overlays a 3-byte sequence number at positions 5-7 of each record. The sequence number starts at 1 for the first record of a group and is incremented by 1 for each subsequent record of the group. After the IFTHEN GROUP clause is executed, the intermediate records look like this:

```
Len|Data
 15|001PAGE:  1
```

```
 25|002LINE 1 OF REPORT A
 25|003LINE 2 OF REPORT A
 25|004LINE 3 OF REPORT A
 25|005LINE 4 OF REPORT A
   |006...
 15|001PAGE:  2
 26|002LINE 66 OF REPORT A
 26|003LINE 67 OF REPORT A
 26|004LINE 68 OF REPORT A
 26|005LINE 69 OF REPORT A
   |006...
 15|001PAGE:  3
 27|002LINE 131 OF REPORT A
 27|003LINE 132 OF REPORT A
 27|004LINE 133 OF REPORT A
 27|005LINE 134 OF REPORT A
   |006...
```

Note that the records in each group are numbered starting with 001 for the first record of each group. The records we want have sequence numbers 002 and 003.

We use an OUTFIL statement to only INCLUDE the records with sequence number 002 or 003, and to remove the sequence numbers so the included output records will be identical to the input records. After the OUTFIL statement is executed, the final output records look like this:

```
Len|Data
 22|LINE 1 OF REPORT A
 22|LINE 2 OF REPORT A
 23|LINE 66 OF REPORT A
 23|LINE 67 OF REPORT A
 24|LINE 131 OF REPORT A
 24|LINE 132 OF REPORT A
```

Example 29

```
  OPTION COPY
  OUTFIL FNAMES=OUT1,INCLUDE=(11,3,CH,EQ,C'D51'),ACCEPT=3
  OUTFIL FNAMES=OUT2,STARTREC=2,ACCEPT=5
```

This example illustrates how you can stop processing OUTFIL input records for various groups in different ways.

The input data set has these records:

```
HEADER  2010/06/30
FRANK     D51
ED        D52
VICKY     D51
MARTIN    D52
LILY      D50
MARC      D51
JUNE      D51
LUCY      D51
TRAILER     8
```

The first OUTFIL statement uses INCLUDE to extract the D51 records and ACCEPT=3 to only write the first three D51 records to OUT1. Thus, OUT1 will have these records:

```
FRANK     D51
VICKY     D51
MARC      D51
```

The second OUTFIL statement uses STARTREC=2 to skip the HEADER record and ACCEPT=5 to only write the next five records to OUT2. Thus, OUT2 will have these records:

```
FRANK     D51
ED        D52
VICKY     D51
MARTIN    D52
LILY      D50
```

Example 30

```
  OPTION COPY
  OUTFIL FNAMES=OUT1,
   INCLUDE=(3,4,CH,EQ,C'key1'),
   IFTRAIL=(HD=YES,TRLID=(1,1,CH,EQ,C'T'),
    TRLUPD=(6:COUNT=(M11,LENGTH=4),
           14:TOT=(10,4,ZD,TO=ZD,LENGTH=6)))
  OUTFIL FNAMES=OUT2,
   INCLUDE=(3,4,CH,EQ,C'key2'),
   IFTRAIL=(HD=YES,TRLID=(1,1,CH,EQ,C'T'),
    TRLUPD=(6:COUNT=(M11,LENGTH=4),
           14:TOT=(10,4,ZD,TO=ZD,LENGTH=6)))
```

This example illustrates how you can split an input file with a header, and a trailer containing a count and total for the input data records, into two output files each of which has the header and a trailer with accurate count and total for the included data records.

The input data set has RECFM=FB and LRECL=80 with these input records:

```
H 2010/07/06
D key1   0100
D key2   0200
D key2   0118
D key1   0150
D key1   0025
D key2   1000
D key2   0310
T X  0007 QR 001903  D52-007-321-7526
```

The first record is a header record with a date. The records with D in position 1 are data records; they have key1 or key2 in positions 3-6 and an amount field in positions 10-13.

The trailer record is identified by a 'T' in position 1. Positions 6-9 contain a count (0007) of the data records, and positions 14-19 contain a total (001903) for the amount fields in the data records. In addition, there are other static fields in the trailer record that must not be changed. Note that IFTRAIL with HD=YES does not process the header or trailer records as data records, so they are not included in the count or total.

The first OUTFIL statement writes the header record, data records with key1, and trailer record, in the OUT1 data set. The IFTRAIL operand is used to identify the trailer (TRLID), indicate the first record is a header (HD=YES), and update the count and total in the trailer (TRLUPD) to reflect the input data records included in this output data set.

OUT1 will have these records:

```
H 2010/07/06
D key1   0100
D key1   0150
D key1   0025
T X  0003 QR 000275  D52-007-321-7526
```

The second OUTFIL statement writes the header record, data records with key2, and trailer record, in the OUT2 data set. The IFTRAIL operand is used to identify the trailer (TRLID), indicate the first record is a header (HD=YES), and update the count and total in the trailer (TRLUPD) to reflect the input data records included in this output data set. Note that IFTRAIL with HD=YES does not process the header or trailer records as data records, so they are not included in the count or total.

OUT2 will have these records:

```
H 2010/07/06
D key2   0200
D key2   0118
D key2   1000
D key2   0310
T X  0004 QR 001628  D52-007-321-7526
```

Example 31

```
  OPTION Y2PAST=1990
  SORT FIELDS=(1,6,Y2W,A)
  OUTFIL REMOVECC,
     HEADER1=(1:'Input',15:'NEXTDFRI',25:'PREVDSUN',35:'LASTDAYQ'),
     BUILD=(1:1,6,Y2W,TOJUL=Y4W(-),
           15:1,6,Y2W,NEXTDFRI,TOJUL=Y4W(-),
           25:1,6,Y2W,PREVDSUN,TOJUL=Y4W(-),
           35:1,6,Y2W,LASTDAYQ,TOJUL=Y4W(-))
```

This example illustrates how you can sort by a date, and calculate a specific day after and before a date, and the last day of the quarter for a date. The input date is in the form C'mmddyy' and the output dates will be in the form 'ddd-yyyy'.

The SORTIN data set has these input records with a C'mmddyy' date field in positions 1-6:

```
010105
120699
021610
999999
092810
031500
000000
032505
110210
```

The SORT statement sorts by the C'mmddyy' date. We use 1,6,Y2W for the sort field to match the C'mmddyy' date.

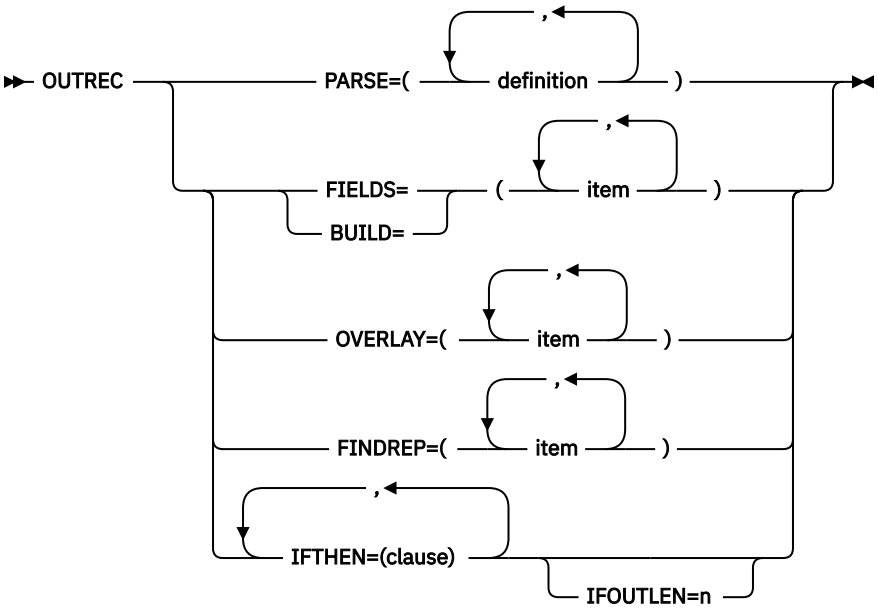
The OUTFIL statement writes a heading and performs four operations on each date. It converts the input date to C'ddd-yyyy' form, gets the next Friday date in C'ddd-yyyy' form, gets the previous Sunday date in C'ddd-yyyy' form, and gets the end of the quarter date in C'ddd-yyyy' form. We use 1,6,Y2W for the input field to match the C'mmddyy' date, and we use TOJUL=Y4W(-) to give us a C'ddd-yyyy' output date.

SORTOUT will have these records:

```
Input         NEXTDFRI  PREVDSUN  LASTDAYQ
000-0000      000-0000  000-0000  000-0000
340-1999      344-1999  339-1999  365-1999
075-2000      077-2000  072-2000  091-2000
001-2005      007-2005  361-2004  090-2005
084-2005      091-2005  079-2005  090-2005
047-2010      050-2010  045-2010  090-2010
271-2010      274-2010  269-2010  273-2010
306-2010      309-2010  304-2010  365-2010
999-9999      999-9999  999-9999  999-9999
```

Note that the '000000' and '999999' input values are treated as special indicators for output.

OUTREC control statement



The OUTREC control statement allows you to reformat the input records **after** they are sorted, merged or copied.

The OUTREC control statement supports a wide variety of parsing, editing, and reformatting tasks, including:

- The use of fixed position/length fields or variable position/length fields. For fixed fields, you specify the starting position and length of the field directly. For variable fields, such as delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings (and many other types), you define rules that allow DFSORT to extract the relevant data into fixed parsed fields, and then use the parsed fields as you would use fixed fields.
- Insertion of blanks, zeros, strings, current date, future date, past date, current time, sequence numbers, decimal constants, and the results of arithmetic expressions before, between, and after the input fields in the reformatted records.
- Sophisticated conversion capabilities, such as find and replace, hexadecimal display, bit display, translation of EBCDIC letters from lowercase to uppercase or uppercase to lowercase, translation of characters from EBCDIC to ASCII and from ASCII to EBCDIC, translation of characters using the ALTSEQ translation table, conversion of numeric values from one format to another, left-justify or left-squeeze (remove leading blanks or all blanks and shift left), and right-justify or right-squeeze (remove trailing blanks or all blanks and shift right).
- Sophisticated editing capabilities, such as control of the way numeric fields are presented with respect to length, leading or suppressed zeros, thousands separators, decimal points, leading and trailing positive and negative signs, and so on.

  Twenty-seven pre-defined editing masks are available for commonly used numeric editing patterns, encompassing many of the numeric notations used throughout the world. In addition, a virtually unlimited number of numeric editing patterns are available via the user-defined editing masks.
- Transformation of SMF, TOD, and ETOD date and time values to more usable forms.
- Various types of arithmetic operations for input date fields.
- Conversion of input date fields of one type (CH, ZD, PD, 2-digit year, 4-digit year, Julian, Gregorian) to corresponding output date fields of another type or to a corresponding day of the week.
- Selection of a character constant, hexadecimal constant, or input field from a lookup table, based on a character, hexadecimal, or bit string as input (that is, lookup and change).

You can create the reformatted OUTREC records in one of the following three ways using unedited, edited, or converted input fields (p,m for fixed fields, or %nn for parsed fields - see PARSE), and a variety of constants:

- **BUILD or FIELDS:** Reformat each record by specifying all of its items one by one. Build gives you complete control over the items you want in your reformatted OUTREC records and the order in which they appear. You can delete, rearrange and insert fields and constants. Example:

```
OUTREC BUILD=(1,20,C'ABC',26:5C'*',
     15,3,PD,EDIT=(TTT.TT),21,30,80:X)
```

- **OVERLAY:** Reformat each record by specifying just the items that overlay specific columns. Overlay lets you change specific existing columns without affecting the entire record. Example:

```
OUTREC OVERLAY=(45:45,8,TRAN=LTOU)
```

- **FINDREP:** Reformat each record by doing various types of find and replace operations. Example:

```
OUTREC FINDREP=(IN=C'Mr.',OUT=C'Mister')
```

- **IFTHEN clauses:** Reformat different records in different ways by specifying how build, overlay, find/replace, or group operation items are applied to records that meet given criteria. IFTHEN clauses let you use sophisticated conditional logic to choose how different record types are reformatted. Example:

```
OUTREC IFTHEN=(WHEN=(1,5,CH,EQ,C'TYPE1'),
         BUILD=(1,40,C'**',+1,TO=PD)),
       IFTHEN=(WHEN=(1,5,CH,EQ,C'TYPE2'),
         BUILD=(1,40,+2,TO=PD,X'FFFF')),
       IFTHEN=(WHEN=NONE,OVERLAY=(45:C'NONE'))
```

You can choose to include any or all of the following items in your reformatted OUTREC records:

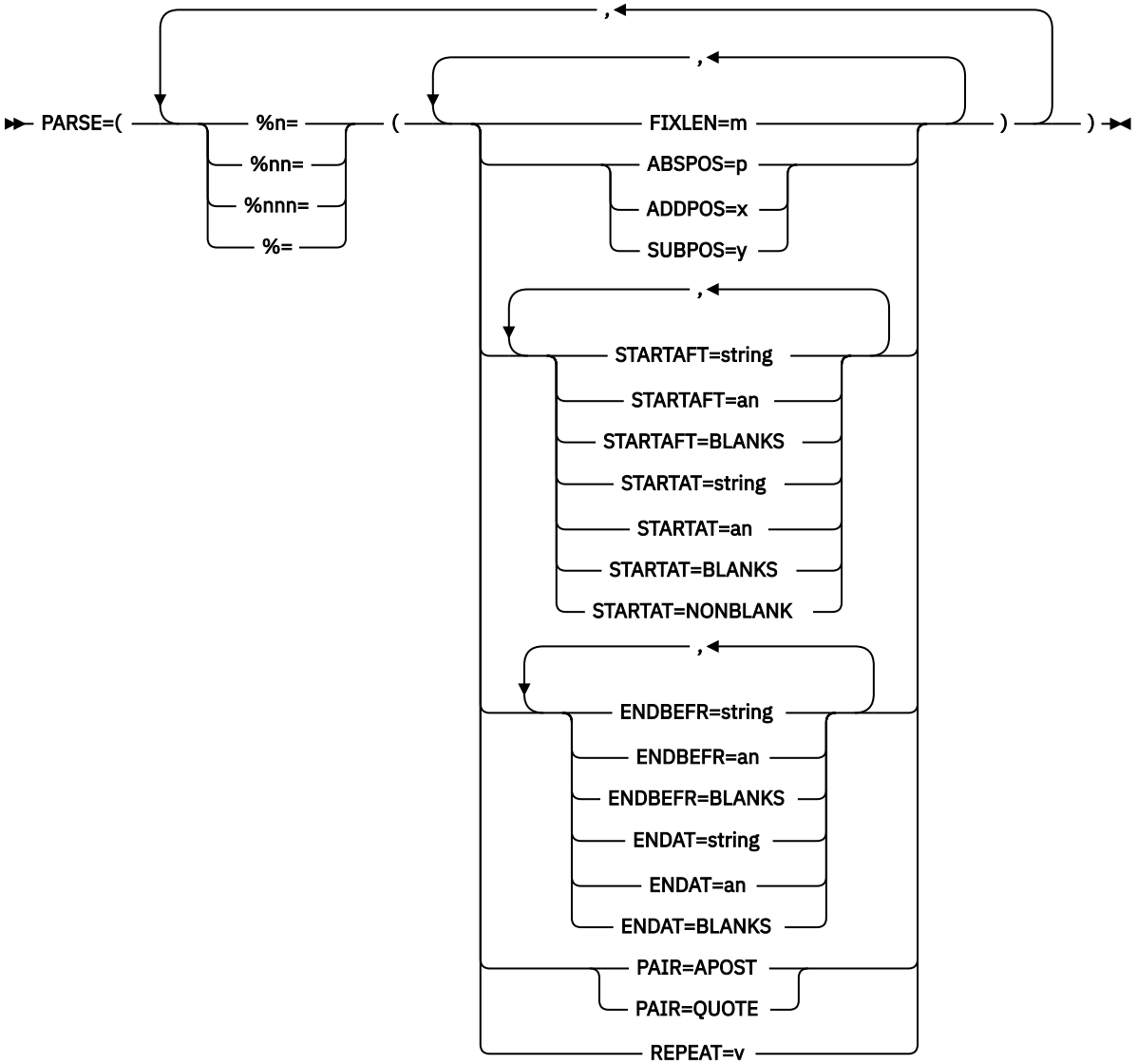
- Fixed position/length fields or variable position/length fields. For fixed fields, you specify the starting position and length of the field directly. For variable fields, such as delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings (and many other types), you define rules that allow DFSORT to extract the relevant data into fixed parsed fields, and then use the parsed fields as you would use fixed fields.
- Blanks, binary zeros, character strings, and hexadecimal strings
- Current date, future date, past date and current time in various forms
- Unedited input fields aligned on byte, halfword, fullword, and doubleword boundaries
- Replaced or removed strings.
- Hexadecimal or bit representations of binary input fields
- Characters translated from uppercase to lowercase, lowercase to uppercase, ASCII to EBCDIC or EBCDIC to ASCII.
- Left-justified, right-justified, left-squeezed, or right-squeezed input fields.
- Numeric input fields of various formats converted to different numeric formats, or to character format edited to contain signs, thousands separators, decimal points, leading zeros or no leading zeros, and so on.
- Decimal constants converted to different numeric formats, or to character format edited to contain signs, thousands separators, decimal points, leading zeros or no leading zeros, and so on.
- The results of arithmetic expressions combining fields, decimal constants, operators (MIN, MAX, MUL, DIV, MOD, ADD and SUB) and parentheses converted to different numeric formats, or to character format edited to contain signs, thousands separators, decimal points, leading zeros or no leading zeros, and so on.
- SMF, TOD and ETOD date and time fields converted to different numeric formats, or to character format edited to contain separators, leading zeros or no leading zeros, and so on.
- Input date fields of one type (CH, ZD, PD, 2-digit year, 4-digit year, Julian, Gregorian) converted to corresponding output date fields of another type or to a corresponding day of the week.

- The results of various types of arithmetic operations for input date fields.
- Sequence numbers in various formats.
- A character constant, hexadecimal constant or input field selected from a lookup table, based on a character, hexadecimal or bit constant as input.
- A zoned decimal group identifier, a zoned decimal group sequence number, or a field propagated from the first record of a group to all of the records of a group.

For information concerning the interaction of INREC and OUTREC, see "INREC statement notes" on page 148 and "OUTREC statement notes" on page 404.

See "OUTFIL control statements" on page 219 for complete details on the OUTFIL OUTREC parameter.

PARSE



This operand allows you to extract variable position/length fields into fixed parsed fields. Parsed fields (%n, %nn or %nnn) can be used where fixed position/length fields (p,m) can be used in the BUILD (or FIELDS) or OVERLAY operands as described later in this section.

Note: Although you can use %n (%0-%9), %nn (%00-%99) or %nnn (%000-%999) for a parsed field, for convenience in this book %nn will be used in general when referring to a parsed field. %n, %0n or %00n can be used interchangeably for parsed field n (for example, %1, %01 or %001 for parsed

field 1). %nn or %0nn can be used interchangeably for parsed field nn (for example, %12 or %012 for parsed field 12).

PARSE can be used for many different types of variable fields including delimited fields, comma separated values (CSV), tab separated values, blank separated values, keyword separated fields, null-terminated strings, and so on. You can assign up to 1000 parsed fields (%0-%999) to the variable fields you want to extract.

Note that if all of the fields in your records have fixed positions and lengths, you don't need to use PARSE. But if any of the fields in your records have variable positions or lengths, you can use PARSE to treat them as fixed parsed fields in BUILD or OVERLAY. You can mix p,m fields (fixed fields) and %nn fields (parsed fields) in BUILD and OVERLAY.

See PARSE under "OUTFIL Control Statements" for complete details.

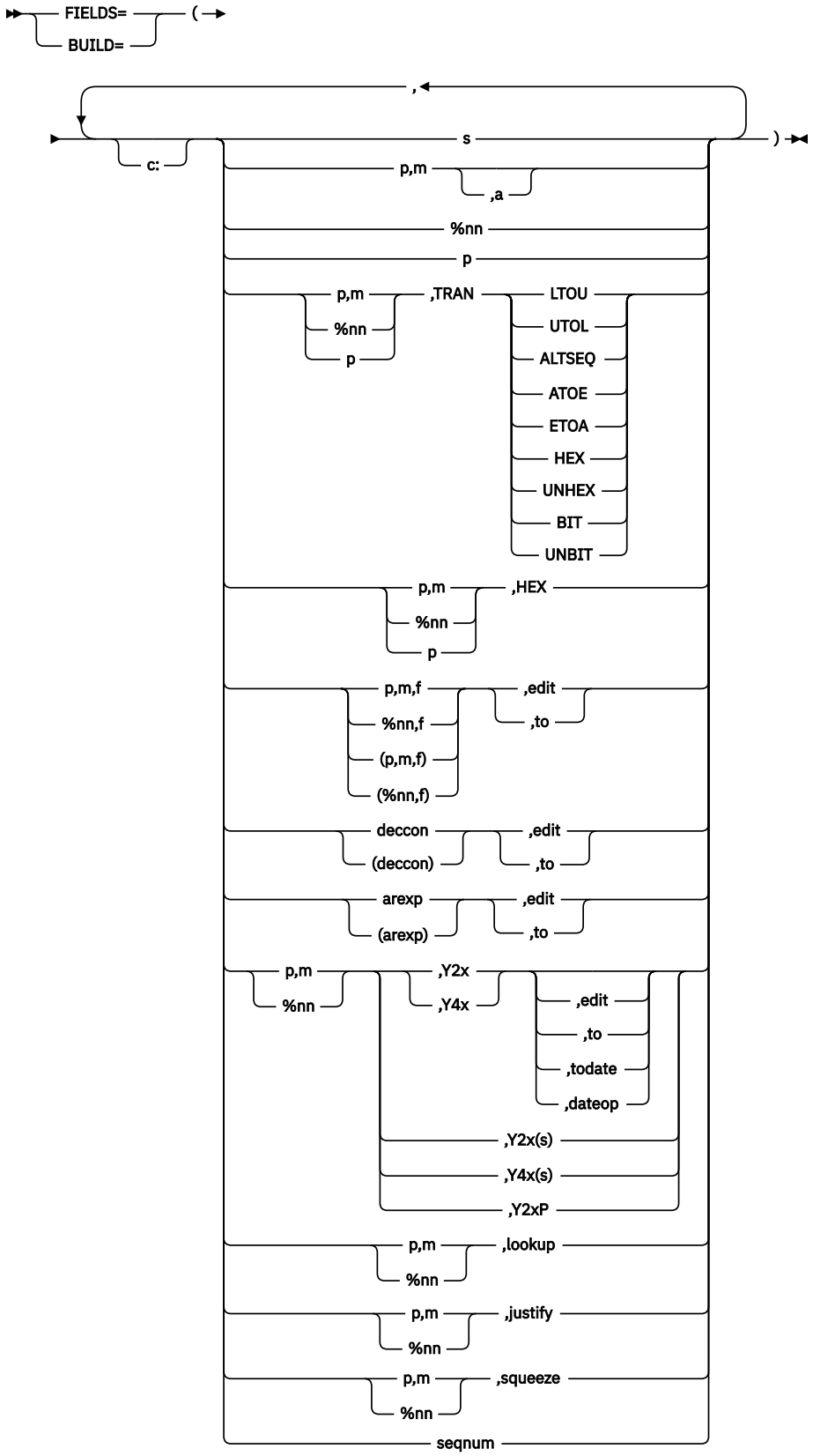
Sample Syntax:

```
OUTREC PARSE=(%200=(ENDBEFR=C'*',FIXLEN=3),
       %201=(ENDBEFR=BLANKS,FIXLEN=6),
       %202=(STARTAT=C'MAX',FIXLEN=8),
       %203=(STARTAFT=C'(',ENDBEFR=C')',FIXLEN=6),
       %204=(STARTAFT=BLANKS,FIXLEN=5)),
     BUILD=(%203,X,%203,HEX,21:%202,31:%201,SFF,M26,LENGTH=7,
            18,6,%200,UFF,M11,LENGTH=3,%204,JFY=(SHIFT=RIGHT))
```

Default for PARSE: None; must be specified. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

FIELDS or BUILD



Specifies all of the items in the reformatted OUTREC record in the order in which they are to be included. The reformatted OUTREC record consists of the separation fields, edited and unedited input fields (p,m for fixed fields, or %nn for parsed fields - see PARSE), edited decimal constants, edited

results of arithmetic expressions, and sequence numbers you select, in the order in which you select them, aligned on the boundaries or in the columns you indicate.

For variable-length records, the first item in the BUILD or FIELDS parameter must specify or include the unedited 4-byte record descriptor word (RDW), that is, you must start with 1,m with m equal to or greater than 4. If you want to include the bytes from a specific position to the end of each input record at the end of each reformatted output record, you can specify that starting position (p) as the last item in the BUILD or FIELDS parameter. For example:

```
OUTREC FIELDS=(1,4,          unedited RDW
   1,2,BI,TO=ZD,LENGTH=5,    display RDW length in decimal
   C'|',                     | separator
   5)                        display input positions 5 to end
```

For fixed-length records, the first input and output data byte starts at position 1. For variable-length records, the first input and output data byte starts at position 5, after the RDW in positions 1-4.

The BUILD or FIELDS parameter of the OUTREC statement differs from the BUILD or OUTREC parameter of the OUTFIL statement in the following ways:

- The BUILD or FIELDS parameter of the OUTREC statement applies to all input records; the BUILD or OUTREC parameter of the OUTFIL statement only applies to the OUTFIL input records for its OUTFIL group.
- The BUILD or OUTREC parameter of the OUTFIL statement supports the slash (/) separator for creating blank records and new records; the BUILD or FIELDS parameter of the OUTREC statement does not.

c:

specifies the position (column) for a separation field, input field decimal constant, arithmetic expression, or sequence number, relative to the start of the reformatted output record. Unused space preceding the specified column is padded with EBCDIC blanks. The following rules apply:

- c must be a number between 1 and 32752.
- c: must be followed by a separation field, input field, decimal constant, or arithmetic expression.
- c must not overlap the previous input field or separation field in the reformatted output record.
- For variable-length records, c: must not be specified before the first input field (the record descriptor word) nor after the variable part of the input record.
- The colon (:) is treated like the comma (,) or semicolon (;) for continuation to another line.

See Table 33 on page 131 for examples of valid and invalid column alignment.

s

specifies that a separation field (blanks, zeros, character string, hexadecimal string, current date, future date, past date, or current time) is to appear in the reformatted output record. It can be specified before or after any input field. Consecutive separation fields may be specified. For variable-length records, separation fields must not be specified before the first input field (the record descriptor word) or after the variable part of the input record. Permissible values are nX, nZ, nC'xx...x', nX'yy...yy', and various date and time constants.

nX

Blank separation. n bytes of EBCDIC blanks (X'40') are to appear in the reformatted output records. n can range from 1 to 4095. If n is omitted, 1 is used.

See Table 34 on page 132 for examples of valid and invalid blank separation.

nZ

Binary zero separation. n bytes of binary zeros (X'00') are to appear in the reformatted output records. n can range from 1 to 4095. If n is omitted, 1 is used.

See Table 35 on page 132 for examples of valid and invalid binary zero separation.

nC'xx...x'

Character string separation. n repetitions of the character string constant (C'xx...x') are to appear in the reformatted output records. n can range from 1 to 4095. If n is omitted, 1 is used. x can be any EBCDIC character. You can specify from 1 to 256 characters.

If you want to include a single apostrophe in the character string, you must specify it as two single apostrophes:

```
Required:  O'NEILL     Specify:  C'O''NEILL'
```

See Table 36 on page 132 for examples of valid and invalid character string separation.

nX'yy...yy'

Hexadecimal string separation. n repetitions of the hexadecimal string constant (X'yy...yy') are to appear in the reformatted output records. n can range from 1 to 4095. If n is omitted, 1 is used.

The value yy represents any pair of hexadecimal digits. You can specify from 1 to 256 pairs of hexadecimal digits.

See Table 37 on page 133 for examples of valid and invalid hexadecimal string separation.

DATEn, DATEn(c), DATEnP

Constant for current date. The date of the run is to appear in the reformatted output records. See DATEn, DATEn(c), DATEnP under OUTFIL OUTREC for details.

&DATEn, &DATEn(c), &DATEnP

can be used instead of DATEn, DATEn(c), and DATEnP, respectively.

DATEn+r, DATEn(c)+r, DATEnP+r

Constant for future date. A future date relative to the current date of the run is to appear in the reformatted output records. See DATEn+r, DATEn(c)+r, DATEnP+r under OUTFIL OUTREC for details.

&DATEn+r, &DATEn(c)+r, &DATEnP+r

can be used instead of DATEn+r, DATEn(c)+r, and DATEnP+r, respectively.

DATEn-r, DATEn(c)-r, DATEnP-r

Constant for past date. A past date relative to the current date of the run is to appear in the reformatted output records. See DATEn-r, DATEn(c)-r, DATEnP-r under OUTFIL OUTREC for details.

&DATEn-r, &DATEn(c)-r, &DATEnP-r

Can be used instead of DATEn-r, DATEn(c)-r, and DATEnP-r, respectively

TIMEn, TIMEn(c), TIMEnP

Constant for current time. The time of the run is to appear in the reformatted output records. See TIMEn, TIMEn(c), TIMEnP under OUTFIL OUTREC for details.

&TIMEn, &TIMEn(c), &TIMEnP

Can be used instead of TIMEn, TIMEn(c), and TIMEnP, respectively.

DATE
specifies that the current date is to appear in the reformatted output records in the form 'mm/dd/yy'. See DATE under OUTFIL OUTREC for details.

&DATE
can be used instead of DATE.

DATE=(abcd)
specifies that the current date is to appear in the reformatted output records in the form 'adbdc'. See DATE=(abcd) under OUTFIL OUTREC for details.

&DATE=(abcd)
can be used instead of DATE=(abcd).

DATENS=(abc)
specifies that the current date is to appear in the reformatted output records in the form 'abc'. See DATENS=(ab) under OUTFIL OUTREC for details.

&DATENS=(abc)
can be used instead of DATENS=(abc).

YDDD=(abc)
specifies that the current date is to appear in the reformatted output records in the form 'acb'. See YDDD=(abc) under OUTFIL OUTREC for details.

&YDDD=(abc)
can be used instead of YDDD=(abc).

YDDDNS=(ab)
specifies that the current date is to appear in the reformatted output records in the form 'ab'. See YDDDNS=(ab) under OUTFIL OUTREC for details.

&YDDDNS=(ab)
can be used instead of YDDDNS=(ab).

TIME
specifies that the current time is to appear in the reformatted output records in the form 'hh:mm:ss'. See TIME under OUTFIL OUTREC for details.

&TIME
can be used instead of TIME.

TIME=(abc)
specifies that the current time is to appear in the reformatted output records in the form 'hhcmmcss' (24-hour time) or 'hhcmmcss xx' (12-hour time). See TIME=(abc) under OUTFIL OUTREC for details.

&TIME=(abc)
can be used instead of TIME=(abc).

TIMENS=(ab)
specifies that the current time is to appear in the reformatted output record in the form 'hhmmss' (24-hour time) or 'hhmmss xx' (12-hour time). See TIMENS=(ab) under OUTFIL OUTREC for details.

&TIMENS=(ab)
can be used instead of TIMENS=(ab).

p,m,a
specifies that an unedited input field is to appear in the reformatted output record.

p
specifies the first byte of the input field relative to the beginning of the input record.[13] The first data byte of a fixed-length record has relative position 1. The first data byte of a variable-length

record has relative position 5, because the first four bytes are occupied by the RDW. All fields must start on a byte boundary, and no field may extend beyond byte 32752. See "OUTREC statement notes" on page 404 for special rules concerning variable-length records.

m
specifies the length of the input field. It must include the sign if the data is signed and must be a whole number of bytes. See "OUTREC statement notes" on page 404 for more information.

a
specifies the alignment (displacement) of the input field in the reformatted output record relative to the start of the reformatted output record.

The permissible values of **a** are:

H
Halfword aligned. The displacement (p-1) of the field from the beginning of the reformatted input record, in bytes, is a multiple of 2 (that is, position 1, 3, 5, and so forth).

F
Fullword aligned. The displacement is a multiple of 4 (that is, position 1, 5, 9, and so forth).

D
Doubleword aligned. The displacement is a multiple of 8 (that is, position 1, 9, 17, and so forth).

Alignment can be necessary if, for example, the data is to be used in a COBOL application program where items are aligned through the SYNCHRONIZED clause. Unused space preceding aligned fields are always padded with binary zeros.

%nn
specifies that an unedited parsed input field is to appear in the reformatted output record. See PARSE for details of parsed fields. See p,m,a for further details. Note that alignment (H, F, D) is not permitted for %nn fields (for example, %nn,F results in an error message and ermination).

p
specifies that the unedited part of the variable input record (that part beyond the minimum record length), is to appear in the reformatted output record, as the last field. p without m can only be used for variable-length records; not for fixed-length records.



Attention: If 1,4,p is specified (only RDW and variable part of record), "null" records containing only an RDW will result if the record length is less than p.

A value must be specified for p that is less than or equal to the minimum record length (RECORD statement L4 value) plus 1 byte.

p,m,TRAN=keyword
Specifies that an input field is to be translated as indicated by the keyword. See p,m,TRAN=keyword under OUTFIL OUTREC for details.

%nn,TRAN=keyword
Specifies that a parsed input field is to be translated as indicated by the keyword. See %nn,TRAN=keyword under OUTFIL OUTREC for details.

p,TRAN=keyword
Specifies that the variable part of the input record is to be translated as indicated by the keyword. See p,TRAN=keyword under OUTFIL OUTREC for details.

p,m,HEX
Can be used instead of p,m,TRAN=HEX. See p,m,HEX under OUTFIL OUTREC for details.

%nn,HEX
Can be used instead of %nn,TRAN=HEX. See %nn,HEX under OUTFIL OUTREC for details.

p,HEX
Can be used instead of p,TRAN=HEX. See p,HEX under OUTFIL OUTREC for details.

[13] If INREC is specified, p must refer to the record as reformatted by INREC. If your E15 user exit reformats the record, and INREC is not specified, p must refer to the record as reformatted by your E15 user exit.

p,m,f,edit or (p,m,f),edit
specifies that an edited numeric input field is to appear in the reformatted output record. You can edit BI, FI, PD, PD0, ZD, FL, CSF, FS, UFF, SFF, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3 or TM4 fields using either pre-defined edit masks (M0-M26) or specific edit patterns you define. You can control the way the edited fields look with respect to length, leading or suppressed zeros, thousands separators, decimal points, leading and trailing positive and negative signs, and so on.

See p,m,f,edit under OUTFIL OUTREC for details.

Sample Syntax:

```
OUTREC FIELDS=(5:(21,8,ZD),M19,X,46,5,ZD,M13,
               31:35,4,FL,SIGNS=(,,+,-),LENGTH=10,
               51:8,4,PD,EDIT=(**II,IIT.TTXS),SIGNS=(,,+,-))
```

%nn,f,edit or (%nn,f),edit
specifies that an edited numeric parsed input field is to appear in the reformatted output record. See PARSE for details of parsed fields. See p,m,f,edit or (p,m,f),edit for further details.

p,m,f,to or (p,m,f),to
specifies that a converted numeric input field is to appear in the reformatted output record. You can convert BI, FI, PD, PD0, ZD, FL, CSF, FS, UFF, SFF, DC1, DC2, DC3, DE1, DE2, DE3, DT1, DT2, DT3, TC1, TC2, TC3, TC4, TE1, TE2, TE3, TE4, TM1, TM2, TM3, or TM4 fields to BI, FI, PD, PDC, PDF, ZD, ZDF, ZDC, CSF, or FS fields.

See p,m,f,to under OUTFIL OUTREC for details.

Sample Syntax:

```
OUTREC FIELDS=(21,4,BI,TO=PDF,X,8,4,ZD,FI,LENGTH=2)
```

%nn,f,to or (%nn,f),to
specifies that a converted numeric parsed input field is to appear in the reformatted output record. See PARSE for details of parsed fields. See p,m,f,to or (p,m,f),to for further details.

deccon,edit or (deccon),edit
specifies that an edited decimal constant is to appear in the reformatted output record. The decimal constant must be in the form +n or -n where n is 1 to 31 decimal digits. The sign (+ or -) must be specified. A decimal constant produces a signed, 31-digit zoned decimal (ZD) result to be edited as specified.

See deccon,edit under OUTFIL OUTREC for details.

Sample Syntax:

```
OUTREC FIELDS=(-5000,EDIT=(-T,TT.T),21:(+0),M11,LENGTH=7)
```

deccon,to or (deccon),to
specifies that a converted decimal constant is to appear in the reformatted output record. The decimal constant must be in the form +n or -n where n is 1 to 31 decimal digits. The sign (+ or -) must be specified. A decimal constant produces a signed, 31-digit zoned decimal (ZD) result to be converted as specified.

See deccon,to under OUTFIL OUTREC for details.

Sample Syntax:

```
OUTREC FIELDS=(+1,PD,LENGTH=6,(-1),PD,LENGTH=6,
    -50,TO=ZD,LENGTH=8)
```

arexp,edit or (arexp),edit
specifies that the edited result of an arithmetic expression is to appear in the reformatted output record. The arithmetic expression can consist of input fields, decimal constants, operators and

parentheses. An arithmetic expression produces a signed, 31-digit zoned decimal (ZD) result to be edited as specified.

See arexp,edit under OUTFIL OUTREC for details.

Sample Syntax:

```
OUTREC FIELDS=(C'**',27,2,FI,SUB,
                    83,4,PD,EDIT=(STTTTTTT),SIGNS=(+,-),
  25:(((15,5,ZD,MUL,+2),ADD,+100),MAX,62,2,PD),M25,LENGTH=12)
```

arexp,to or (arexp),to
specifies that the converted result of an arithmetic expression is to appear in the reformatted output record. The arithmetic expression can consist of input fields, decimal constants, operators and parentheses. An arithmetic expression produces a signed, 31-digit zoned decimal (ZD) result to be converted as specified.

See arexp,to under OUTFIL OUTREC for details.

Sample Syntax:

```
OUTREC FIELDS=((15,6,FS,DIV,+5),ADD,(-1,ADD,36,6,FS),TO=FI,X,
     3,2,FI,MAX,-6,LENGTH=4,TO=FS)
```

p,m,Y2x or p,m,Y4x
Specifies that an input date field is to be edited. Real Y2x dates are edited using the century window established by the Y2PAST option in effect. Y2x and Y4x dates with special indicators are expanded appropriately (for example, p,6,Y2T transforms C'000000' to C'00000000').

See "p,m,Y2x or p,m,Y4x" under OUTFIL OUTREC for details.

Sample Syntax:

```
OUTREC BUILD=(21,3,Y2U,X,3,8,Y4W)
```

%nn,Y2x or %nn,Y4x
Specifies that a parsed input date field is to be edited. See PARSE for details of parsed fields. See "p,m,Y2x or p,m,Y4x" for further details.

p,m,Y2x,edit or p,m,Y4x,edit
Specifies that the output for a p,m,Yxx input date field is to be edited according to the edit parameters you specify. For example, if you specify:

```
OUTREC BUILD=(28,5,Y4V,EDIT=(TTTT-TT-TT)
```

The P'ccyymmdd' (X'0ccyymmddC') date value will be transformed to a C'ccyymmdd' date value and then edited to a C'ccyy-mm-dd' date value.

See "p,m,Y2x or p,m,Y4x" and "p,m,f,edit" for related details.

%nn,Y2x,edit or %nn,Y4x,edit
Specifies that the output for a parsed Yxx input date field is to be edited according to the edit parameters you specify. See PARSE for details of parsed fields. See "p,m,Y2x,edit or p,m,Y4x,edit" for further details.

p,m,Y2x,to or p,m,Y4x,to
Specifies that an input date field is to be converted according to the to parameters you specify. For example, if you specify:

```
OUTREC BUILD=(5,6,Y2W,TO=PD,LENGTH=5)
```

The C'mmddyy' date value will be transformed to a Z'mmddccyy' date value and then converted to a P'mmddccyy' (X'0mmddccyyC') date value.

See "p,m,Y2x or p,m,Y4x" and "p,m,f,to" for related details.

%nn,Y2x,to or %nn,Y4x,to
Specifies that a parsed input date field is to be converted according to the to parameters you specify. See PARSE for details of parsed fields. See "p,m,Y2x,to or p,m,Y4x,to" for further details.

p,m,Y2x,todate or p,m,Y4x,todate
Specifies that an input date field of one type is to be converted to a corresponding output date field of another type. You can convert date fields between combinations of 2-digit and 4-digit year dates, CH/ZD and PD dates, and Julian and Gregorian dates. You can also convert a date field to a corresponding day of the week in several forms.

See "p,m,Y2x,todate or p,m,Y4x,todate" under OUTFIL OUTREC for details.

Sample Syntax:

```
* Convert a P'dddyy' input date to a C'ccyy/mm/dd' output date
  OUTREC BUILD=(21,3,Y2X,TOGREG=Y4T(/),X,
* Convert a C'ccyymmdd' input date to a P'ccyyddd' output date
     42,8,Y4T,TOJUL=Y4U,X,
* Convert a C'mmddyy' input date to a C'yymmdd' output date
     11,6,Y2W,TOGREG=Y2T)
```

%nn,Y2x,todate or %nn,Y4x,todate
Specifies that an input date field of one type is to be converted to a corresponding output date field of another type. See PARSE for details of parsed fields. See "p,m,Y2x,todate or p,m,Y4x,todate" for further details.

p,m,Y2x,dateop or p,m,Y4x,dateop
Specifies an arithmetic operation for an input date field. See "p,m,Y2x,dateop or p,m,Y4x,dateop" under OUTFIL OUTREC for details.

%nn,Y2x,dateop or %nn,Y4x,dateop
Specifies an arithmetic operation for a parsed input date field. See "%nn,Y2x,dateop or %nn,Y4x,dateop" under OUTFIL OUTREC for details.

p,m,Y2x(s) or p,m,Y4x(s)
Specifies that an input date field is to be edited with separators. s can be any character except a blank. Real Y2x dates are edited using the century window established by the Y2PAST option in effect. Y2x and Y4x dates with special indicators are expanded appropriately (for example, p,8,Y4T(s) transforms C'00000000' to C'0000/00/00').

See "p,m,Y2x(s) or p,m,Y4x(s)" under OUTFIL OUTREC for details.

Sample Syntax:

```
* Convert a Z'dddccyy' date to a C'ddd/ccyy' date.
  OUTREC BUILD=(19,7,Y4W(/),X,
* Convert a P'ccyymmdd' date to a C'ccyy-mm-dd' date.
     43,5,Y4V(-))
```

%nn,Y2x(s) or %nn,Y4x(s)
Specifies that a parsed input date field is to be edited with separators. See PARSE for details of parsed fields. See "p,m,Y2x(s) or p,m,Y4x(s)" for further details.

p,m,Y2xP
Specifies that an input date field is to be converted to a packed decimal output date field. Real Y2x dates are edited using the century window established by the Y2PAST option in effect. Y2x and Y4x dates with special indicators are expanded appropriately (for example, p,6,Y2TP transforms C'000000' to P'00000000').

See "p,m,Y2xP" under OUTFIL OUTREC for details.

Sample Syntax:

```
  OUTREC BUILD=(11,3,Y2XP,X,21,4,Y2WP)
```

%nn,Y2xP
Specifies that a parsed input date field is to be converted to a packed decimal output date field. See PARSE for details of parsed fields. See "p,m,Y2xP" for further details.

p,m,lookup or %nn,lookup
specifies that a character constant, hexadecimal constant, input field (p,m) or parsed input field (%nn) from a lookup table is to appear in the reformatted output record. You can use p,m,lookup or %nn,lookup to select a specified character set constant (that is, a character or hexadecimal string) or set field (that is, an input field or parsed input field) based on matching an input value against find constants (that is, character, hexadecimal, or bit constants).

See p,m,lookup or %nn,lookup under OUTFIL OUTREC for details.

Sample Syntax:

```
OUTREC FIELDS=(11,1,
               CHANGE=(6,
                 C'R',C'READ',
                 C'U',C'UPDATE',
                 X'FF',C'EMPTY',
                 C'A',C'ALTER'),
               NOMATCH=(11,6),
               4X,
               21,1,
               CHANGE=(10,
                 B'.1......',C'VSAM',
                 B'.0......',C'NON-VSAM'))
```

p,m,justify
specifies that a left-justified or right-justified input field is to appear in the reformatted output record. For a left-justified field, leading blanks are removed and the characters from the first nonblank to the last nonblank are shifted left, with blanks inserted on the right if needed. For a right-justified field, trailing blanks are removed and the characters from the last nonblank to the first nonblank are shifted right, with blanks inserted on the left if needed. Optionally:

- specific leading and trailing characters can be changed to blanks before justification begins
- a leading string can be inserted
- a trailing string can be inserted
- the output length can be changed (it's equal to the input length by default)

See p,m,justify under OUTFIL OUTREC for details.

Sample Syntax:

```
  OUTREC FIELDS=(1,10,
      21,20,JFY=(SHIFT=LEFT),5X,
      52,12,JFY=(SHIFT=RIGHT,PREBLANK=C'+',LEAD=C'$',TRAIL=C'.00'))
```

%nn,justify
specifies that a left-justified or right-justified parsed input field is to appear in the reformatted output record. See PARSE for details of parsed fields. See p,m,justify for further details.

p,m,squeeze
specifies that a left-squeezed or right-squeezed input field is to appear in the reformatted output record. For a left-squeezed field, all blanks are removed and the characters from the first nonblank to the last nonblank are shifted left, with blanks inserted on the right if needed. For a right-justified field, all blanks are removed and the characters from the last nonblank to the first nonblank are shifted right, with blanks inserted on the left if needed. Optionally:

- specific characters can be changed to blanks before squeezing begins
- a leading string can be inserted
- a trailing string can be inserted
- a string (for example, a comma delimiter) can be inserted wherever a group of blanks is removed between the first nonblank and the last nonblank.

- blanks can be kept as is between paired apostrophes ('AB CD EF') or paired quotes ("AB CD EF")
- the output length can be changed (it's equal to the input length by default)

See p,m,squeeze under OUTFIL OUTREC for details.

Sample Syntax:

```
      OUTREC FIELDS=(21,20,SQZ=(SHIFT=RIGHT),5X,
        152,18,SQZ=(SHIFT=LEFT,PREBLANK=X'05',
          LEAD=C'VOL=SER=',MID=C',',PAIR=QUOTE,LENGTH=30))
```

%nn,squeeze
specifies that a left-squeezed or right-squeezed parsed input field is to appear in the reformatted output record. See PARSE for details of parsed fields. See p,m,squeeze for further details.

seqnum
specifies that a sequence number is to appear in the reformatted output record. The sequence numbers are assigned in the order in which the records are received for OUTREC processing. You can create BI, PD, ZD, CSF, or FS sequence numbers and control their lengths, starting values and increment values. You can restart the sequence number at the start value each time a specified input field (p,m) or parsed input field (%nn) changes.

See seqnum under OUTFIL OUTREC for details.

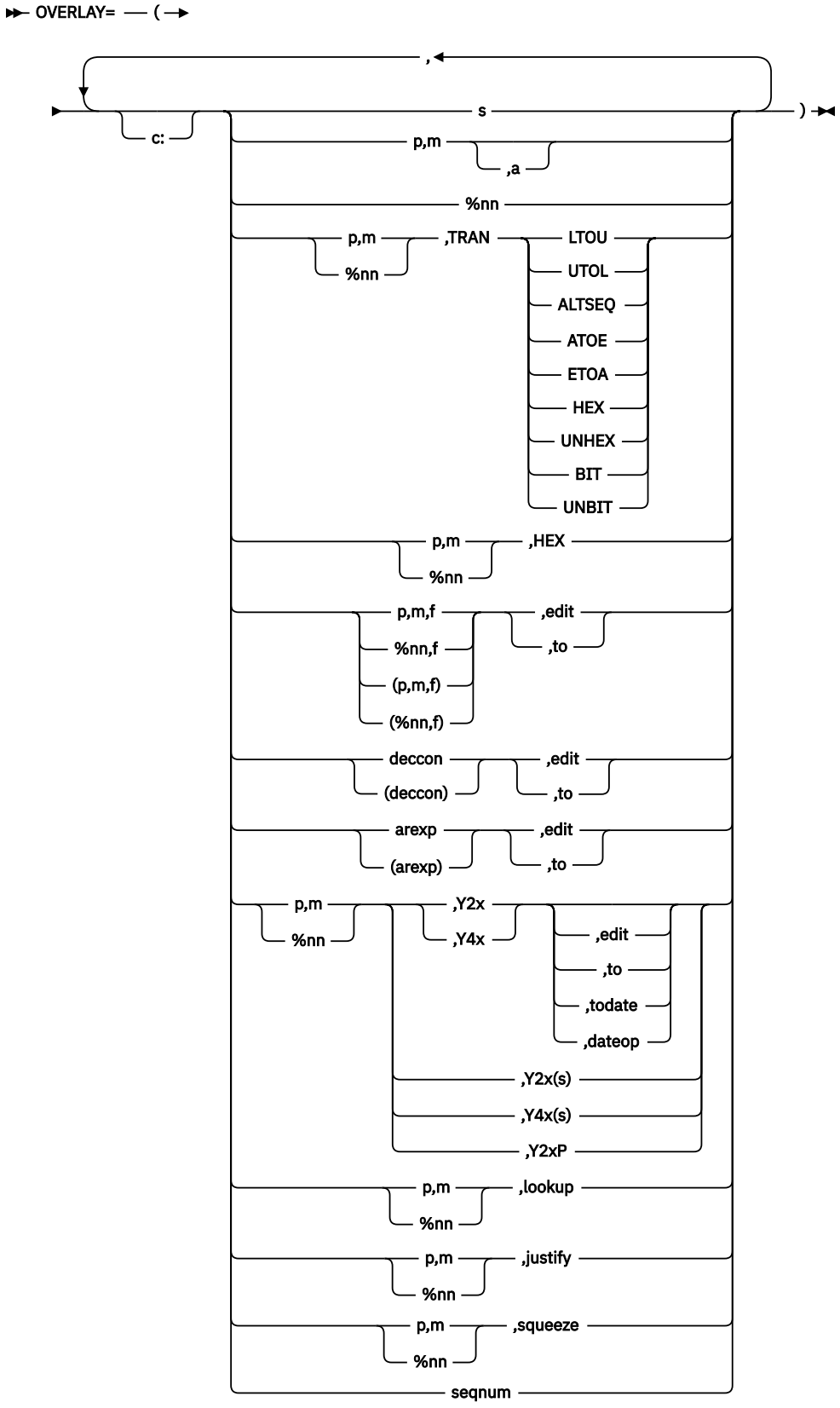
Sample Syntax:

```
  OUTREC FIELDS=(SEQNUM,6,ZD,START=1000,INCR=50,RESTART=(21,5),1,60)
```

Default for BUILD or FIELDS: None; must be specified. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

OVERLAY



Specifies each item that is to overlay specific columns in the reformatted record. Columns that are not overlaid remain unchanged. If you want to insert, rearrange, or delete fields, use BUILD or FIELDS

rather than OVERLAY. Use OVERLAY only to overlay existing columns or to add fields at the end of every record. OVERLAY can be easier to use then BUILD or FIELDS when you just want to change a few fields without rebuilding the entire record.

For fixed-length records, the first input and output data byte starts at position 1. For variable-length records, the first input and output data byte starts at position 5, after the RDW in positions 1-4.

Use c: (column) to specify the output positions to be overlaid. If you do not specify c: for the first item, it defaults to 1:. If you do not specify c: for any other item, it starts after the previous item. For example, if you specify:

```
  OUTREC OVERLAY=(25,2,11:C'A',15,3,C'**')
```

Input positions 25-26 are placed at output positions 1-2; C'A' is placed at output position 11; input positions 15-17 are placed at output positions 12-14; and C'**' is placed at output positions 15-16. The rest of the record remains unchanged.

You can specify items in any order, you can change the same item multiple times and you can overlap output columns. **Changes to earlier items affect changes to later items.** For example, say you specify:

```
OUTREC OVERLAY=(21:8,4,ZD,MUL,+10,TO=ZD,LENGTH=6,
    5:5,1,TRAN=UTOL,
    5:5,1,CHANGE=(1,C'a',C'X',C'b',C'Y'),NOMATCH=(5,1))
```

and input position 5 has 'A'. The second item (UTOL) would change 'A' to 'a' and the third item (CHANGE) would change 'a' again to 'X'.

If you specify an OVERLAY item that extends the overlay record beyond the end of the input record, the reformatted record length is automatically increased to that length, and blanks are filled in on the left as needed. For variable-length records, the RDW length is also increased to correspond to the larger reformatted record length after all of the OVERLAY items are processed. For example, if your input record has a length of 40 and you specify:

```
OUTREC OVERLAY=(16:C'ABC',51:5C'*',35:15,2)
```

the output record is given a length of 55. Blanks are filled in from columns 41-50. For variable-length records, the length in the RDW is changed from 40 to 55 after all of the OVERLAY items are processed.

Missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

The OVERLAY parameter of the OUTREC statement applies to all input records whereas the OVERLAY parameter of the OUTFIL statement only applies to the OUTFIL input records for its OUTFIL group.

See OUTREC FIELDS for details of the items listed in the OVERLAY syntax diagram shown previously in this section. You can specify all of the items for OVERLAY in the same way that you can specify them for BUILD or FIELDS with the following exceptions:

- You cannot specify p or p,HEX or p,TRAN=keyword for OVERLAY.
- For p,m,H or p,m,F or p,m,D specified for OVERLAY, fields are aligned as necessary without changing the preceding bytes.
- For variable-length records, you must not overlay positions 1-4 (the RDW) for OVERLAY, so be sure to specify the first column (c:) as 5 or greater. If you do not specify the first column, it will default to 1: which is invalid for variable-length records with OVERLAY. Whereas FIELDS=(1,m,...) is required, OVERLAY=(1,m) is not allowed since it would overlay the RDW.

Sample Syntax:

Fixed input records:

```
OUTREC OVERLAY=(21:21,4,ZD,TO=PD,LENGTH=4,
         2:5,8,HEX,45:C'*',32,4,C'*',81:SEQNUM,5,ZD)
```

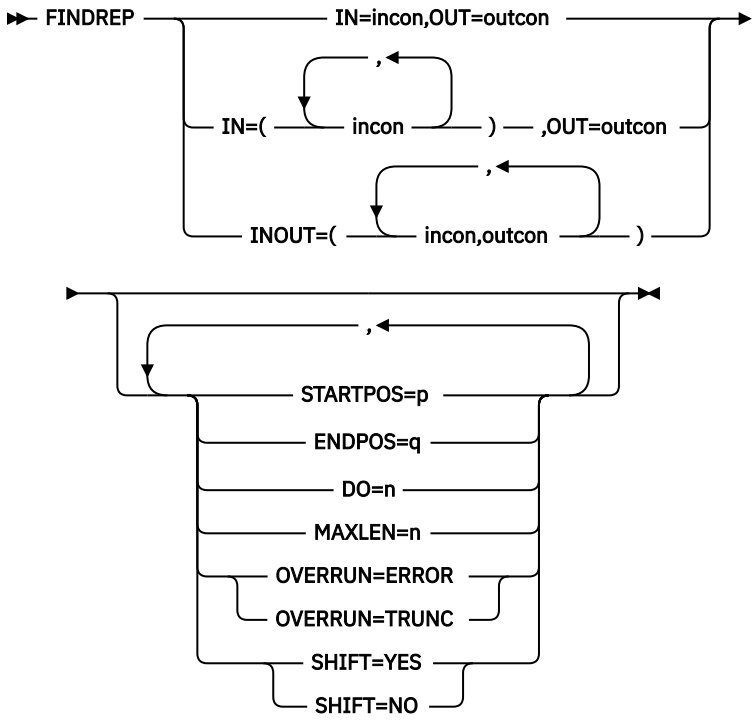
Variable input records:

```
OUTREC OVERLAY=(5:X'0001',28:C'Date is ',YDDDNS=(4D),
       17:5C'*')
```

Default for OVERLAY: None; must be specified.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

FINDREP



You can use FINDREP to find constants anywhere in a record and replace them with other constants of the same or different lengths. You can find character or hexadecimal input constants anywhere in your records and replace them with character, hexadecimal or null output constants. As appropriate, bytes can be shifted left or right, blank padding can be added for fixed-length records, and the length can be changed for variable-length records.

Various options of FINDREP allow you to define one or more input constants and a corresponding output constant, define one or more pairs of input and output constants, start and end the find scan at specified positions, stop after a specified number of constants are replaced, increase or decrease the length of the output record, define the action to be taken if nonblank characters overrun the end of the record, and specify whether output constants are to replace or overlay input constants.

See FINDREP under "OUTFIL Control Statements" for complete details.

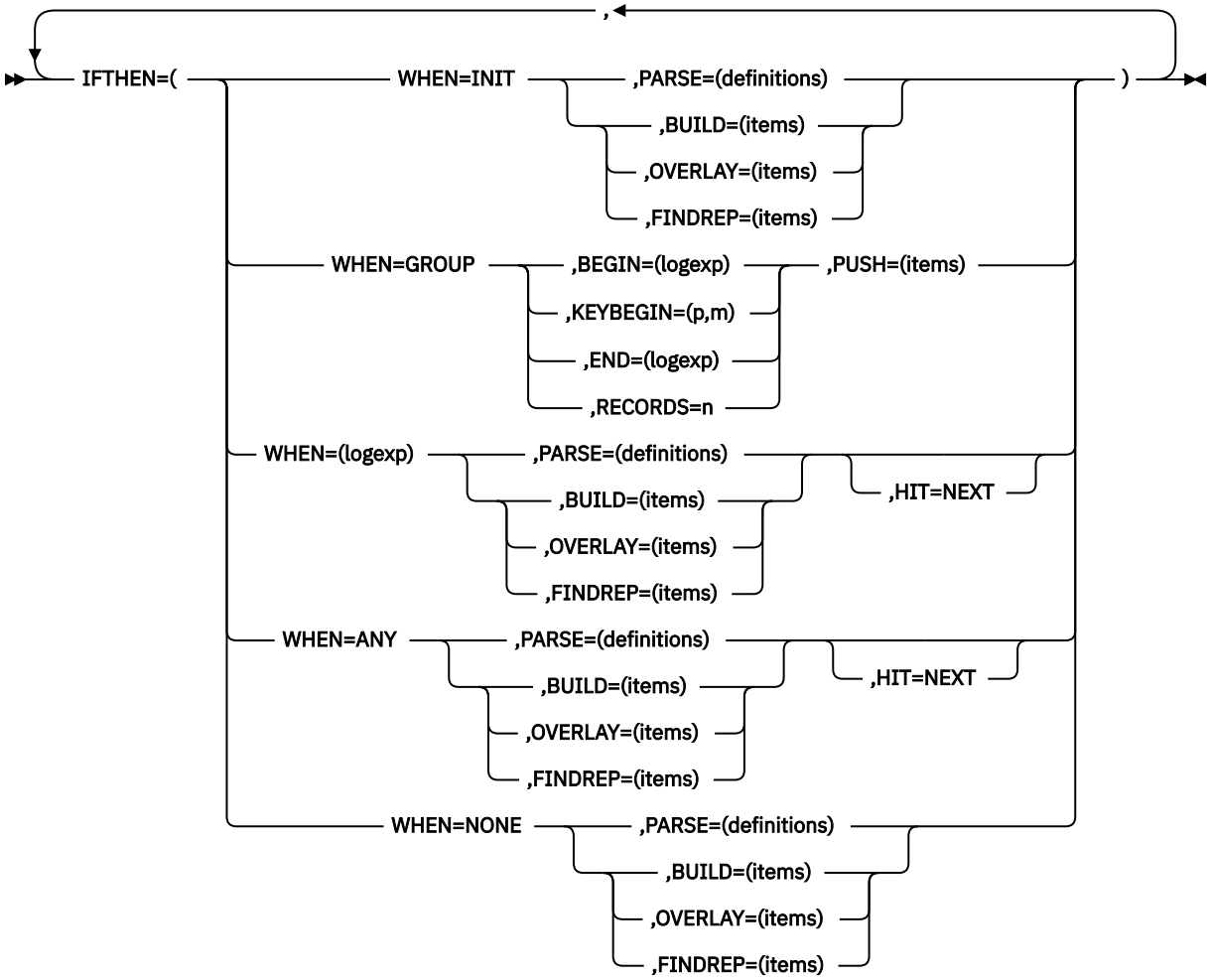
Sample Syntax:

```
OUTREC FINDREP=(INOUT=(C'Goodbye',C'Bye',C'Hello',C'Hi'))
```

Default for FINDREP: None; must be specified. See Appendix B, "Specification/Override of DFSORT Options".

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

IFTHEN



IFTHEN clauses allow you to reformat different records in different ways by specifying how build, overlay, find/replace or group operation items are to be applied to records that meet given criteria. IFTHEN clauses let you use simple or complex conditional logic to choose how different record types are reformatted.

If you want to insert, rearrange, or delete fields in the same way for every record, use BUILD or FIELDS rather than IFTHEN. If you want to do find and replace operations in the same way for every record, use FINDREP rather than IFTHEN. Use IFTHEN clauses if you want to insert, rearrange, delete or overlay fields, or perform find/replace operations in different ways for different records, or if you want to perform operations on groups of records.

You can use five types of IFTHEN clauses as follows:

- **WHEN=INIT:** Use one or more WHEN=INIT clauses to apply build, overlay or find/replace items to all of your input records. WHEN=INIT clauses and WHEN=GROUP clauses are processed before any of the other IFTHEN clauses.
- **WHEN=GROUP**: Use one or more WHEN=GROUP clauses to propagate fields, identifiers and sequence numbers within specified groups of records. WHEN=INIT clauses and WHEN=GROUP clauses are processed before any of the other IFTHEN clauses.
- **WHEN=(logexp):** Use one or more WHEN=(logexp) clauses to apply build, overlay or find/replace items to your input records that meet specified criteria. A WHEN=(logexp) clause is satisfied when the logical expression evaluates as true.

- **WHEN=ANY:** Use a WHEN=ANY clause after multiple WHEN=(logexp) clauses to apply additional build, overlay or find/replace items to your input records if they satisfied the criteria for any of the preceding WHEN=(logexp) clauses.
- **WHEN=NONE:** Use one or more WHEN=NONE clauses to apply build, overlay or find/replace items to your input records that did not meet the criteria for any of the WHEN=(logexp) clauses. WHEN=NONE clauses are processed after any of the other IFTHEN clauses. If you do not specify a WHEN=NONE clause, only the WHEN=INIT and WHEN=GROUP changes (if any) are applied to input records that do not meet the criteria for any of the WHEN=(logexp) clauses.

IFTHEN clauses are processed in the following order:

- WHEN=INIT clauses or WHEN=GROUP clauses
- WHEN=(logexp) clauses and WHEN=ANY clauses
- WHEN=NONE clauses

Processing of IFTHEN clauses continues unless one of the following occurs:

- A WHEN=(logexp) or WHEN=ANY clause is satisfied, and HIT=NEXT is not specified.
- There are no more IFTHEN clauses to process. When processing of IFTHEN clauses stops, the IFTHEN record created so far is used as the output record.

Example:

```
  OUTREC IFTHEN=(WHEN=(12,1,BI,ALL,X'3F'),OVERLAY=(18:C'Yes')),
         IFTHEN=(WHEN=(35,2,PD,EQ,+14),BUILD=(1,40,45,3,HEX),HIT=NEXT),
         IFTHEN=(WHEN=(35,2,PD,GT,+17),BUILD=(1,40,41,5,HEX),HIT=NEXT),
         IFTHEN=(WHEN=ANY,BUILD=(1,55,C'ABC',70:X)),
         IFTHEN=(WHEN=(63,2,CH,EQ,C'AB'),OVERLAY=(18:C'No')),
         IFTHEN=(WHEN=NONE,BUILD=(1,40,51,8,TRAN=LTOU))
```

For this example, the IFTHEN clauses are processed as follows:

- If IFTHEN clause 1 is satisfied, its overlay item is applied and IFTHEN processing stops.
- If IFTHEN clause 1 is not satisfied, its overlay item is not applied and IFTHEN processing continues.
- If IFTHEN clause 2 is satisfied, its build items are applied and IFTHEN processing continues.
- If IFTHEN clause 2 is not satisfied, its build items are not applied and IFTHEN processing continues.
- If IFTHEN clause 3 is satisfied, its build items are applied and IFTHEN processing continues.
- If IFTHEN clause 3 is not satisfied, its build items are not applied and IFTHEN processing continues.
- If IFTHEN clause 4 is satisfied, its build items are applied and IFTHEN processing stops.
- If IFTHEN clause 4 is not satisfied, its build items are not applied and IFTHEN processing continues.
- If IFTHEN clause 5 is satisfied, its overlay item is applied and IFTHEN processing stops.
- If IFTHEN clause 5 is not satisfied, its overlay item is not applied and IFTHEN processing continues.
- If IFTHEN clause 6 is satisfied, its build items are applied and IFTHEN processing stops.
- If IFTHEN clause 6 is not satisfied, its build items are not applied and IFTHEN processing stops.

All of the IFTHEN clauses operate sequentially on an IFTHEN record. The IFTHEN record is created initially from the input record. Each IFTHEN clause tests and changes the IFTHEN record, as appropriate. Thus, **changes made by earlier IFTHEN clauses are "seen" by later IFTHEN clauses.** For example, if you have a 40-byte input record and specify:

```
  OUTREC IFTHEN=(WHEN=INIT,OVERLAY=(8:8,4,ZD,ADD,+1,TO=ZD,LENGTH=4)),
         IFTHEN=(WHEN=(8,4,ZD,EQ,+27),OVERLAY=(28:C'Yes')),
         IFTHEN=(WHEN=NONE,OVERLAY=(28:C'No'))
```

The WHEN=INIT clause adds 1 to the ZD value and stores it in the IFTHEN record. The WHEN=(8,4,ZD,EQ,+27) clause tests the incremented ZD value in the IFTHEN record rather than the original ZD value in the input record.

The IFTHEN record is adjusted as needed for the records created or changed by the IFTHEN clauses. For fixed-length records, blanks are filled in on the left as needed. For variable-length records, the RDW length is adjusted as needed each time the IFTHEN record is changed.

Missing bytes in specified input fields are replaced with blanks so the padded fields can be processed.

DFSORT sets an appropriate LRECL (or reformatted record length if the OUTREC record is further modified) for the output records based on the build, overlay, find/replace and group operation items specified by the IFTHEN clauses. However, DFSORT does not analyze the possible results of WHEN=(logexp) conditions when determining an appropriate LRECL. When you use OUTREC IFTHEN clauses, you can override the OUTREC LRECL determined by DFSORT with the OUTREC IFOUTLEN parameter.

If SEQNUM is used in multiple IFTHEN clauses, the sequence number will be incremented for each record that satisfies the IFTHEN clause, that is, a separate SEQNUM counter will be kept for each IFTHEN clause. For example, if your input is:

```
RECORD A 1
RECORD B 1
RECORD B 2
RECORD C 1
RECORD A 2
RECORD C 2
RECORD B 3
RECORD D 1
```

and you specify:

```
  OUTREC IFTHEN=(WHEN=(8,1,CH,EQ,C'A'),OVERLAY=(15:SEQNUM,4,ZD)),
         IFTHEN=(WHEN=(8,1,CH,EQ,C'B'),OVERLAY=(16:SEQNUM,4,ZD)),
         IFTHEN=(WHEN=NONE,OVERLAY=(17:SEQNUM,4,ZD))
```

your output will be:

```
RECORD A 1    0001
RECORD B 1     0001
RECORD B 2     0002
RECORD C 1      0001
RECORD A 2    0002
RECORD C 2      0002
RECORD B 3     0003
RECORD D 1      0003
```

Separate SEQNUM counters are kept for the 'A' record, for the 'B' record, and for the NONE records.

The IFTHEN clauses of the OUTREC statement apply to all input records whereas the IFTHEN clauses of the OUTFIL statement only apply to the OUTFIL input records for its OUTFIL group.

WHEN=INIT clause

See "WHEN=INIT clause" under OUTFIL IFTHEN for details. Note that / cannot be used to create blank records or new records.

Sample Syntax:

```
  OUTREC IFTHEN=(WHEN=INIT,
           BUILD=(1,20,21:C'Department',31:3X,21,60)),
         IFTHEN=(WHEN=(5,2,CH,EQ,C'D1'),OVERLAY=(31:8,3)),
         IFTHEN=(WHEN=(5,2,CH,EQ,C'D2'),OVERLAY=(31:12,3))
```

WHEN=GROUP clause

See "WHEN=GROUP clause" under OUTFIL IFTHEN for details.

Sample Syntax

```
  OUTREC IFTHEN=(WHEN=GROUP,
    BEGIN=(1,40,SS,EQ,C'J82',OR,1,40,SS,EQ,C'M72'),
    PUSH=(41:ID=5))
```

WHEN=(logexp) clause

See "WHEN=(logexp) clause" under OUTFIL IFTHEN for details. Note that although / can be used to create blank records and new records with OUTFIL, it cannot be used with OUTREC.

Sample Syntax:

```
OUTREC IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,
     18,4,ZD,LE,+2000),OVERLAY=(42:C'Type1 <= 2000'),HIT=NEXT),
       IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,6,1,BI,BO,X'03'),
          BUILD=(1,21,42,13)),
       IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,
          18,4,ZD,GT,+2000),OVERLAY=(42:C'Type1 > 2000 '),HIT=NEXT),
       IFTHEN=(WHEN=(1,3,CH,EQ,C'T01',AND,6,1,BI,BO,X'01'),
          BUILD=(1,25,42,13))
```

WHEN=ANY clause

See "WHEN=ANY clause" under OUTFIL IFTHEN for details. Note that although / can be used to create blank records and new records with OUTFIL, it cannot be used with OUTREC.

Sample Syntax:

```
OUTREC IFTHEN=(WHEN=(1,3,SS,EQ,C'T01,T02,T03'),
               BUILD=(C'Group A',X,1,80),HIT=NEXT),
       IFTHEN=(WHEN=(1,3,SS,EQ,C'T04,T05,T06'),
               BUILD=(C'Group B',X,1,80),HIT=NEXT),
       IFTHEN=(WHEN=(1,3,SS,EQ,C'T07,T08,T09,T10'),
               BUILD=(C'Group C',X,1,80),HIT=NEXT),
       IFTHEN=(WHEN=ANY,OVERLAY=(16:C'Group Found')
```

WHEN=NONE clause

See "WHEN=NONE clause" under OUTFIL IFTHEN for details. Note that although / can be used to create blank records and new records with OUTFIL, it cannot be used with OUTREC.

Sample Syntax:

```
OUTREC IFTHEN=(WHEN=INIT,BUILD=(1,20,21:C'Department',31:3X,21,60)),
       IFTHEN=(WHEN=(5,2,CH,EQ,C'D1'),OVERLAY=(31:8,3)),
       IFTHEN=(WHEN=(5,2,CH,EQ,C'D2'),OVERLAY=(31:12,3)),
       IFTHEN=(WHEN=NONE,OVERLAY=(31:C'***'))
```

Default for IFTHEN clauses: None; must be specified.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

IFOUTLEN



Overrides the OUTREC LRECL (or reformatted record length if the OUTREC record is further modified) determined by DFSORT from your OUTREC IFTHEN clauses. DFSORT sets an appropriate LRECL for the output records based on the build, overlay, find/replace and group operation items specified by the IFTHEN clauses. However, DFSORT does not analyze the possible results of WHEN=(logexp) conditions when determining an appropriate OUTREC LRECL. When you use OUTREC IFTHEN clauses, you can override the OUTREC LRECL determined by DFSORT with the OUTREC IFOUTLEN parameter.

Fixed-length records longer than the IFOUTLEN length are truncated to the IFOUTLEN length. Fixed-length records shorter than the IFOUTLEN are padded with blanks to the IFOUTLEN length. Variable-length records longer than the IFOUTLEN length are truncated to the IFOUTLEN length.

n

specifies the length to use for the OUTREC LRECL (or for the reformatted record length if the OUTREC record is further modified) . The value for n must be between 1 and 32767, but must not be larger than the maximum LRECL allowed for the RECFM, and must not conflict with the specified or retrieved LRECL for the fixed-length output data set.

Sample Syntax:

```
OUTREC IFOUTLEN=70,
       IFTHEN=(WHEN=(5,1,CH,EQ,C'1',AND,8,3,ZD,EQ,+10),
```

```
            BUILD=(1,40,C'T01-GROUP-A',65)),
        IFTHEN=(WHEN=(5,1,CH,EQ,C'2',AND,8,3,ZD,EQ,+12),
            BUILD=(1,40,C'T02-GROUP-B',65))
```

Default for IFOUTLEN: The LRECL determined from the IFTHEN clauses.

OUTREC statement notes

-
- If input records are reformatted by INREC or E15, OUTREC must refer to fields in the appropriate reformatted record (see “INREC statement notes” on page 148).
- When you specify OUTREC, you must be aware of the change in record size and layout of the resulting reformatted output records.
- If the SORTOUT LRECL is specified or available, DFSORT will use it even if it does not match the reformatted OUTREC record length; this can cause padding or truncation of the reformatted OUTREC records, or termination. If the SORTOUT LRECL is not specified or available, DFSORT can automatically use the reformatted OUTREC record length as the SORTOUT LRECL, when appropriate. See the discussion of the SOLRF and NOSOLRF options in “OPTION control statement” on page 171 for details.

  For VSAM data sets, the maximum record size defined in the cluster is equivalent to the LRECL when processing fixed-length records, and is four bytes more than the LRECL when processing variable-length records. See “VSAM considerations ” on page 15 for more information.
- For variable-length records, the first entry in the FIELDS, BUILD, or IFTHEN BUILD parameter must specify or include the unedited 4-byte RDW, that is, the first field must be 1,4 or 1,m with m greater than 4. DFSORT sets the length of the reformatted record in the RDW.

  If the first field in the data portion of the input record is to appear unedited in the reformatted record immediately following the RDW, the entry in the FIELDS, BUILD, or IFTHEN BUILD parameter can specify both RDW and data field in one. (1,m,...). Otherwise, the RDW must be specifically included in the reformatted record (for example, 1,4,1,4,HEX)..
- For variable-length records, OVERLAY, IFTHEN OVERLAY or IFTHEN PUSH items must not overlay the RDW in bytes 1-4. You must ensure that 1:, 2:, 3: or 4: is not specified or defaulted for any OVERLAY or PUSH item. Note that the default for the first OVERLAY or PUSH item is 1:, so you must override it.
- With FIELDS, BUILD or IFTHEN BUILD, the variable part of the input record (that part beyond the minimum record length) can be included in the reformatted record as the last part. In this case, a value must be specified for p*n* that is less than or equal to the minimum record length (RECORD statement L4 value) plus 1 byte, and m*n* and a*n* must be omitted. For example,

  ```
  OUTREC FIELDS=(1,8,20C'*',9)
  ```

  With OVERLAY, the variable part of the input record must not be included in the reformatted record.
- If INREC with FIELDS or BUILD and OUTREC with FIELDS and BUILD are specified, either both must specify position-only for the last part, or neither must specify position-only for the last part. OVERLAY or IFTHEN, and FIELDS or BUILD, can differ with respect to position-only. See “INREC statement notes” on page 148 for more details.
- If the reformatted record includes only the RDW and the variable part of the input record, "null" records containing only an RDW may result.
- The reformatted output records are in the format specified by OUTREC regardless of whether INREC was specified.
- Fields referenced in OUTREC statements can overlap each other or control fields.
- If input is variable records, the output is also variable. This means that each record is given the correct RDW by DFSORT before output.
- When OUTREC is specified, your E35 user exit routine must refer to fields in the reformatted output record.
- DFSORT issues a message and terminates processing if an OUTREC statement is specified for a tape work data set sort or conventional merge application.

- When you specify OUTREC, VLSHRT is not used. If VLSHRT is specified, it is ignored.
- If SZERO is in effect, -0 is treated as negative and +0 is treated as positive for edited or converted input fields, decimal constants, and the results of arithmetic expressions. If NOSZERO is in effect, -0 and +0 are treated as positive for edited or converted input fields, decimal constants, and the results of arithmetic expressions.
- If SZERO is in effect, -0 compares as less than +0 when numeric fields and constants are used. If NOSZERO is in effect, -0 compares as equal to +0 when numeric fields and constants are used.

  Note: OPTION SZERO or OPTION NOSZERO is ignored for OUTREC IFTHEN=(WHEN=(logexp),...) unless the OPTION statement is found in a higher source (for example, DFSPARM is a higher source than SYSIN) or before the OUTREC statement in the same source. For example, NOSZERO will be used in both of the following cases:

  – Case 1:

```
//DFSPARM DD *
  OPTION COPY,NOSZERO
/*
//SYSIN DD *
  OUTREC IFTHEN=(WHEN=(1,2,FS,EQ,+0),OVERLAY=(22:C'Yes')),
         IFTHEN=(WHEN=NONE,OVERLAY=(22:C'No '))
/*
```

  – Case 2:

```
//SYSIN DD *
  OPTION COPY,NOSZERO
  OUTREC IFTHEN=(WHEN=(1,2,FS,EQ,+0),OVERLAY=(28:C'A')),
         IFTHEN=(WHEN=NONE,OVERLAY=(28:C'B'))
/*
```

Reformatting records after processing — examples

Both INREC and OUTREC can be used to reformat records, either separately or together. The two control statements perform similar functions, although INREC reformats records before sort, merge, or copy processing, whereas OUTREC reformats records after sort, merge, or copy processing. This section shows OUTREC examples. See “Reformatting records before processing — examples” on page 150 for INREC examples.

Example 1

```
  OUTREC FIELDS=(11,32)
```

This statement specifies that the output record is to contain 32 bytes beginning with byte 11 of the input record. This statement can be used only with fixed-length input records, because it does not include the first 4 bytes.

Example 2

```
  OUTREC FIELDS=(1,4,11,32,D,101)
```

This statement is for variable-length records of minimum length 100 bytes, and specifies that the output record is to contain an RDW plus 32 bytes of the input record starting at byte 11 (aligned on a doubleword boundary, relative to the start of the record) plus the entire variable portion of the input record.

Example 3

```
  OUTREC FIELDS=(1,42,D,101)
```

This statement is for variable-length records of minimum length 100 bytes, and specifies that the output record should contain an RDW plus the first 38 data bytes of the input record plus the entire variable portion of the input record.

The 'D' parameter has no effect because the first field is always placed at the beginning of the output record.

Example 4

```
SORT FIELDS=(20,4,CH,D,10,3,CH,D)
OUTREC FIELDS=(7:20,4,C' FUTURE ',20,2,10,3,1Z,1,9,13,7,
   24,57,TRAN=LTOU,6X'FF')
```

This example illustrates how a fixed-length input data set can be sorted and reformatted for output. The SORTIN LRECL is 80 bytes.

The reformatted output records are fixed length with a record size of 103 bytes. SOLRF (the IBM-supplied default) is in effect, so unless the SORTOUT LRECL is specified or available, it will automatically be set to the reformatted record length of 103. The reformatted records look as follows:

Position
Contents

1-6
EBCDIC blanks for column alignment

7-10
Input positions 20 through 23

11-18
Character string: C' FUTURE '

19-20
Input positions 20 through 21

21-23
Input positions 10 through 12

24
Binary zero

25-33
Input positions 1 through 9

34-40
Input positions 13 through 19

41-97
Input positions 24 through 80 with lowercase EBCDIC letters converted to uppercase EBCDIC letters

98-103
Hexadecimal string: X'FFFFFFFFFFFF'

Example 5

```
SORT FIELDS=(12,4,PD,D)
RECORD TYPE=V,LENGTH=(,,,100)
OUTREC FIELDS=(1,7,5Z,5X,28,8,6X,101)
```

This example illustrates how a variable-length input data set can be sorted and reformatted for output. The variable part of the input records is included in the output records. The minimum input record size is 100 bytes and the maximum input record size (SORTIN LRECL or maximum record size for VSAM) is 200 bytes.

The reformatted output records are variable-length with a maximum record size of 131 bytes. The reformatted records look as follows:

Position
Contents

1-4
RDW (input positions 1 through 4)

5-7
Input positions 5 through 7

8-12
Binary zeros

13-17
EBCDIC blanks

18-25
Input positions 28 through 35

26-31
EBCDIC blanks

32-n
Input positions 101 through n (variable part of input records)

Example 6

```
MERGE FIELDS=(28,4,BI,A)
OUTREC BUILD=(1,4,5Z,5X,5,3,28,8,6Z,DATE3-1)
```

This example illustrates how input files can be merged and reformatted for output, with yesterday's date included. The variable part of the input records is not to be included in the output records. The SORTINnn LRECL is 50 bytes.

The reformatted output records are variable-length with a maximum record size of 38 bytes. The reformatted records look as follows:

Position
Contents

1-4
RDW (input positions 1 through 4)

5-9
Binary zeros

10-14
EBCDIC blanks

15-17
Input positions 5 through 7

18-25
Input positions 28 through 35

26-31
Binary zeros

32-38
Yesterday's date in the form C'yyyyddd'

Example 7

```
OPTION COPY,Y2PAST=1985
  OUTREC FIELDS=(SEQNUM,8,ZD,START=1000,INCR=100,
                 11:8,4,PD,M12,
                 31:15,4,Y2V(/),
                 51:2,1,CHANGE=(3,
                          X'01',C'L92',X'02',C'M72',X'03',C'J42'),
                        NOMATCH=(C'???'))
```

This example illustrates how a sequence number can be generated, how numeric and date values can be edited, and how a lookup table can be used.

The reformatted output records look as follows:

Position
Contents

1-8
A zoned decimal sequence number that starts at 1000 and increments by 100.

11–20
A CH field containing the PD field from input positions 8 through 11 edited according to the M12 edit mask.

31-40
A C'yyyy/mm/dd' date field containing the P'yymmdd' date field from input positions 15-18 transformed according to the specified century window of 1985-2084.

51–53
A CH field containing C'L92', C'M72', C'J42' or C'???' as determined by using a lookup table for the input field in position 2.

Example 8

```
SORT FIELDS=(11,4,CH,D)
OUTREC FIELDS=(1,20,
  (5,4,FI,ADD,3,2,FI,ADD,23,2,FI),DIV,+1000,
     EDIT=(STTTTTTT),SIGNS=(,-),2X,
  9,5,ZD,MIN,16,5,FS,TO=ZD,LENGTH=5,2X,
  21,40)
```

This example illustrates how input records can be reformatted for output to contain the results of arithmetic expressions involving input fields, decimal constants, operators and parentheses.

The reformatted output records look as follows:

Position
Contents

1-20
Input positions 1 through 20

21-28
A CH field containing the total of the FI fields from positions 5 through 8, 3 through 4 and 23 through 24, divided by 1000, and edited according to the specified edit pattern.

29-30
EBCDIC blanks

31-35
A ZD field containing the minimum of the ZD field in positions 9 through 13 and the FS field in positions 16 through 20.

36-37
EBCDIC blanks

38-77
Input positions 21-60

Example 9

```
OPTION COPY
OUTREC IFTHEN=(WHEN=(3,2,SS,EQ,C'FR,MX,GR'),
                OVERLAY=(11:DATE=(DM4.),TIME1(.))),
       IFTHEN=(WHEN=(3,2,SS,EQ,C'CN,US,EN'),
                OVERLAY=(11:DATE=(4MD/),TIME1(:)))
```

This example illustrates how you can use IFTHEN clauses with OUTREC to reformat different records in different ways.

For records with 'FR', 'GR' or 'MX' in positions 3-4, positions 11-20 of the reformatted output records are overlaid with a date of the form 'dd.mm.yyyy' and positions 21-28 are overlaid with a time of the form 'hh.mm.ss'. The data before positions 11-28 and after positions 11-28 are not affected.

For records with 'CN', 'US' or 'EN' in positions 3-4, positions 11-20 of the reformatted output records are overlaid with a date of the form 'yyyy/mm/dd' and positions 21-28 are overlaid with a time of the form 'hh:mm:ss'. The data before positions 11-28 and after positions 11-28 are not affected.

Since an IFTHEN clause with WHEN=NONE is not specified, records without 'FR', 'GR', 'MX', 'CN', 'US' or 'EN' in positions 3-4 are not changed.

Example 10

```
OPTION COPY
OUTREC OVERLAY=(31:11,10,ZD,DIV,+1200,TO=PD,LENGTH=6,
                37:11,10,ZD,MOD,+1200,TO=PD,LENGTH=4)
```

This example illustrates how you can use the OVERLAY parameter with OUTREC to change certain columns in your records without affecting other columns.

Positions 31-36 of the reformatted input records are overlaid with a 6-byte PD value equal to the quotient of the 10-byte ZD value at positions 11-20 divided by +1200. Positions 37-40 of the reformatted input records are overlaid with a 4-byte PD value equal to the remainder of the 10-byte ZD value at positions 11-20 divided by +1200. The data before positions 31-40 and after positions 31-40 are not affected.

Example 11

```
OPTION COPY
OUTREC IFTHEN=(WHEN=INIT,
          OVERLAY=(81:11,10,
            11:81,10,ZD,DIV,+1200,TO=PD,LENGTH=6,
            17:81,10,ZD,MOD,+1200,TO=PD,LENGTH=4)),
       IFOUTLEN=80
```

In the previous example (Example 10), we used OVERLAY to overlay positions 31-40 with PD fields for the quotient and remainder of the 10-byte ZD value at positions 11-20 divided by +1200. In this example, we want to overlay positions 11-20 with the quotient and remainder. The input records are 80 bytes and fixed-length and we want the output records to be 80 bytes and fixed-length as well.

If we just overlaid the fields directly as before, we would "ruin" the ZD value before we could use it to get the remainder; the PD quotient would overlay positions 11-16, so positions 11-20 would no longer contain the ZD value we need to get the remainder.

In order to overlay the ZD value itself with the PD values, we make a copy of the 10 byte ZD value after the end of the record, and then overlay the original ZD value with the quotient and remainder derived from the copy of the ZD value.

By making a copy of the 10 byte ZD value at the end of the record, we extend the record length from 80 bytes to 90 bytes. Since we want the final record length to be 80 bytes, we must remove the extra 10 bytes. So instead of just using the OVERLAY parameter, we use an IFTHEN WHEN=INIT clause with OVERLAY, and IFOUTLEN=80. Alternatively, we could use an OVERLAY parameter on the OUTREC statement, followed by an OUTFIL statement, to remove the extra 10 bytes like this:

```
OPTION COPY
OUTREC OVERLAY=(81:11,10,
             11:81,10,ZD,DIV,+1200,TO=PD,LENGTH=6,
             17:81,10,ZD,MOD,+1200,TO=PD,LENGTH=4)
OUTFIL OUTREC=(1,80)
```

Example 12

```
OPTION COPY
OUTREC OVERLAY=(11:11,16,JFY=(SHIFT=RIGHT,LEAD=C'(',
  TRAIL=C')',LENGTH=18))
```

This example illustrates how you can right-justify fields within your records.

The 50-byte FB input records might look like this:

```
0001      9-1-632-731
0002      011-276-321-7836
0003      753-218-307
0004      528-314
```

Note that the second field has left-justified numeric values in various forms. We want to right-justify these values and surround them with a left parenthesis and right parenthesis.

The 50-byte FB output records look like this:

```
0001           (9-1-632-731)
0002      (011-276-321-7836)
0003           (753-218-307)
0004               (528-314)
```

We use OUTFIL OVERLAY to limit the changes to the second field. We use JFY to right-justify the second field with surrounding parentheses. SHIFT=RIGHT shifts the characters to the right. LEAD=C'(' adds a left parenthesis before the first non-blank character. TRAIL=C')' adds a right parenthesis after the last non-blank character. LENGTH=18 increases the output length by 2 bytes to allow for the parentheses (overriding the default of 16 from the input length).

Example 13

```
  OPTION COPY
  OUTREC PARSE=(%01=(ABSPOS=2,FIXLEN=13,ENDBEFR=C'","'),
                %=(ENDBEFR=C'","'),
                %03=(FIXLEN=6,ENDBEFR=C'","'),
                %04=(FIXLEN=6,ENDBEFR=C'","'),
                %05=(FIXLEN=12,ENDBEFR=C'"')),
    BUILD=(%01,20:%03,SFF,ADD,%04,SFF,EDIT=(SIIT.T),SIGNS=(,-),
           31:%05)
```

This example illustrates how you can reformat records containing variable position/length fields, such as comma separated values.

The 80-byte input records might look like this:

```
"Buffy Summers","F","+725.8","-27.3","Sunnydale"
"Bruce Wayne","M","-5.3","-173.2","Gotham City"
"Clark Kent","M","+21.3","-15.8","Metropolis"
"Diana Prince","F","-16.4","+128.9","Gateway City"
```

Note that each record has five variable fields, each enclosed in quotes and separated by a comma. The fields do not start and end in the same position in every record and they have different lengths in different records.

The 42-byte output records should look like this:

```
Buffy Summers       698.5     Sunnydale
Bruce Wayne        -178.5     Gotham City
Clark Kent            5.5     Metropolis
Diana Prince        112.5     Gateway City
```

The first fixed-length output field corresponds to the first variable input field. The second fixed-length output field corresponds to the total of the third and fourth variable input fields. The third fixed-length output field corresponds to the fifth variable input field.

In order to reformat the input records for output, we use PARSE and BUILD to create the fixed parsed fields we need from the variable position/length fields; the quotes around each value are removed. %00 is used for the first input field. % is used to ignore the second input field. %03 and %04 are used for the third and fourth input fields which are added together using SFF format. %05 is used for the fifth input field.

Example 14

```
OPTION COPY
OUTREC IFTHEN=(WHEN=INIT,
         PARSE=(%100=(FIXLEN=3,ENDBEFR=C'.',REPEAT=3),
                %103=(FIXLEN=3)),
         BUILD=(%100,4:C'.',5:%101,8:C'.',9:%102,12:C'.',13:%103)),
  IFTHEN=(WHEN=(1,3,CH,EQ,C'167',AND,13,3,CH,EQ,C'99'),
          OVERLAY=(5:C'113',9:C'75 ',1:1,15,SQZ=(SHIFT=LEFT))),
  IFTHEN=(WHEN=NONE,
          BUILD=(1:1,15,SQZ=(SHIFT=LEFT)))
```

This example illustrates how you can modify variable position/length fields, such as ftp addresses.

The 15-byte input records might look like this:

```
167.113.117.99
167.90.18.99
167.80.118.98
165.250.89.562
167.125.890.95
168.250.89.99
167.125.890.99
0.0.0.0
167.90.580.99
```

We want to convert each FTP address of the form 167.x.y.99 to 167.113.75.99. x and y can be any 1-3 digit value. Thus, the 15-byte output records should look like this:

```
167.113.75.99
167.113.75.99
167.80.118.98
165.250.89.562
167.125.890.95
168.250.89.99
167.113.75.99
0.0.0.0
167.113.75.99
```

In order to reformat the input records for output, we use IFTHEN clauses as follows:

- IFTHEN WHEN=INIT clause: We PARSE the four variable numeric values into four 3-byte fixed parsed fields using %100, %101, %102 and %103 respectively. Note that we use REPEAT=3 with the %100 parsed field to repeat it for the %101 and %102 parsed fields. REPEAT=n is useful when you have several contiguous parsed fields with the same operands).

  We reformat the record with %100, a period, %101, a period, %102, a period and %103. At this point, the reformatted records look like this:

  ```
  167.113.117.99
  167.90 .18 .99
  167.80 .118.98
  165.250.89 .562
  167.125.890.95
  168.250.89 .99
  167.125.890.99
  0  .0  .0  .0
  167.90 .580.99
  ```

- IFTHEN WHEN=(logexp) clause: If the first fixed-length field is '167' and the fourth fixed-length field is '99 ', we overlay the second fixed-length field with '113' and the third fixed-length field with '75'. Then we squeeze the fields to the left to remove the blanks.
- IFTHEN WHEN=NONE clause: If the first fixed-length field is not '167' or the fourth fixed-length field is not '99 ', we squeeze the fields to the left to remove the blanks.

Example 15

```
  SORT FIELDS=(1,2,CH,A)
  OUTREC FINDREP=(IN=C'*',OUT=C' ')
```

This example illustrates how you can replace a character with another character anywhere in FB or VB records.

The FB input records might look like this:

```
05 ***** JUNE ***************
02         * APRIL *
01* * * * DAISY * * * *
03BETTY ***********
```

We want to replace every asterisk with a blank. We use IN=C'*' to indicate we want to find each asterisk, and OUT=C' ' to indicate we want to replace it with a blank.

The sorted output records look like this:

```
01        DAISY
02           APRIL
03BETTY
05       JUNE
```

Example 16

```
  OPTION COPY
  OUTREC IFTHEN=(WHEN=GROUP,BEGIN=(1,3,CH,EQ,C'HDR'),
    END=(1,3,CH,EQ,C'TRL'),PUSH=(31:ID=1))
  OUTFIL INCLUDE=(31,1,CH,NE,C' '),BUILD=(1,30)
```

This example illustrates how you can INCLUDE groups of FB records between a header and a trailer. We add an ID after the end of each record to indicate whether it's part of a group or not, INCLUDE on the ID, and then remove it.

The 30-byte FB input records might look like this:

```
C33  Not in a group
HDR   Start Group 1
A01    Group 1 record
B02    Group 1 record
C03    Group 1 record
TRL   End Group 1
R24  Not in a group
T02  Not in a group
HDR   Start Group 2
D04    Group 2 record
E05    Group 2 record
TRL   End Group 2
F97  Not in a group
```

In the output data set we only want to include groups of records that start with 'HDR' and end with 'TRL'.

We use an IFTHEN WHEN=GROUP clause to put a non-blank character in each record that is part of a group. BEGIN indicates a group starts with a record that has 'HDR' in positions 1-3. END indicates a group ends with a record that has 'TRL' in positions 1-3. PUSH overlays a 1-byte ID character at position 31 in each record of a group (after the end of the record). After the IFTHEN GROUP clause is executed, the intermediate records look like this:

```
C33  Not in a group
HDR   Start Group 1           1
A01    Group 1 record         1
B02    Group 1 record         1
C03    Group 1 record         1
TRL   End Group 1             1
R24  Not in a group
T02  Not in a group
HDR   Start Group 2           2
D04    Group 2 record         2
E05    Group 2 record         2
TRL   End Group 2             2
F97  Not in a group
```

Note that the records within a group have a non-blank character in position 31 whereas the records outside groups have a blank character in position 31. The ID starts at 1 for the first group and is incremented by 1 for each subsequent group. Since we are only allowing one character for the ID, when the ID counter gets to 10, a '0' will appear in position 31. That's fine since we are just looking for a non-blank to indicate a record within a group, or a blank to indicate a record outside of a group.

We use an OUTFIL statement to only INCLUDE records with a non-blank in position 31, and to remove the ID character so the included output records will be identical to the input records. After the OUTFIL statement is executed, the final output records look like this:

```
HDR   Start Group 1
A01    Group 1 record
B02    Group 1 record
C03    Group 1 record
TRL   End Group 1
HDR   Start Group 2
D04    Group 2 record
E05    Group 2 record
TRL   End Group 2
```

Example 17

```
  OPTION COPY
  OUTREC IFTHEN=(WHEN=INIT,BUILD=(1,4,6:5)),
    IFTHEN=(WHEN=GROUP,BEGIN=(6,3,CH,EQ,C'HDR'),
      END=(6,3,CH,EQ,C'TRL'),PUSH=(5:ID=1))
  OUTFIL INCLUDE=(5,1,CH,NE,C' '),BUILD=(1,4,5:6)
```

This example illustrates how you can INCLUDE groups of VB records between a header and a trailer. It's similar to Example 16, but here the records are variable-length. For the FB records, we could add the ID after the end of each record and then remove it without changing the records. But we can't add the ID at the end of each VB record because that would pad all of the records to a fixed length. So, instead we insert the ID between the RDW and the first data byte of each record, and later remove it.

The VB input records might look like this:

```
Len|Data
 23|C33  Not in a group
 23|HDR   Start Group 1
 25|A01    Group 1 record
 25|B02    Group 1 record
 25|C03    Group 1 record
 21|TRL   End Group 1
 23|R24  Not in a group
 23|T02  Not in a group
 23|HDR   Start Group 2
 25|D04    Group 2 record
 25|E05    Group 2 record
 21|TRL   End Group 2
 25|F97  Not in a group
```

In the output data set we only want to include groups of records that start with 'HDR' and end with 'TRL'.

We use an IFTHEN WHEN=INIT clause to reformat each record so it has room for the ID byte between the RDW and the first data byte. After the WHEN=INIT clause is executed, the intermediate records look like this:

```
Len|Data
 24| C33  Not in a group
 24| HDR   Start Group 1
 26| A01    Group 1 record
 26| B02    Group 1 record
 26| C03    Group 1 record
 22| TRL   End Group 1
 24| R24  Not in a group
 24| T02  Not in a group
 24| HDR   Start Group 2
 26| D04    Group 2 record
 26| E05    Group 2 record
```

```
 22| TRL   End Group 2
 26| F97  Not in a group
```

Note that position 5 is blank and the 'HDR' and 'TRL' characters have been shifted over to positions 6-8.

We use an IFTHEN WHEN=GROUP clause to put a non-blank character in each record that is part of a group. BEGIN indicates a group starts with a record that has 'HDR' in positions 6-8. END indicates a group ends with a record that has 'TRL' in positions 6-8. PUSH overlays a 1-byte ID character at position 5 in each record of a group. After the IFTHEN GROUP clause is executed, the intermediate records look like this:

```
Len|Data
 24| C33  Not in a group
 24|1HDR   Start Group 1
 26|1A01    Group 1 record
 26|1B02    Group 1 record
 26|1C03    Group 1 record
 22|1TRL   End Group 1
 24| R24  Not in a group
 24| T02  Not in a group
 24|2HDR   Start Group 2
 26|2D04    Group 2 record
 26|2E05    Group 2 record
 22|2TRL   End Group 2
 26| F97  Not in a group
```

Note that the records within a group have a non-blank character in position 5 whereas the records outside groups have a blank character in position 5. The ID starts at 1 for the first group and is incremented by 1 for each subsequent group. Since we are only allowing one character for the ID, when the ID counter gets to 10, a '0' will appear in position 5. That's fine since we are just looking for a non-blank to indicate a record within a group, or a blank to indicate a record outside of a group.

We use an OUTFIL statement to only INCLUDE records with a non-blank in position 5, and to remove the ID character so the included output records will be identical to the input records. After the OUTFIL statement is executed, the final output records look like this:

```
Len|Data
 23|HDR   Start Group 1
 25|A01    Group 1 record
 25|B02    Group 1 record
 25|C03    Group 1 record
 21|TRL   End Group 1
 23|HDR   Start Group 2
 25|D04    Group 2 record
 25|E05    Group 2 record
 21|TRL   End Group 2
```

Example 18

```
  SORT FIELDS=(1,12,CH,A)
  OUTREC OVERLAY=(30:16,8,Y4T,TOGREG=Y4T)
  OUTFIL INCLUDE=(30,1,CH,EQ,C'*')
```

This example illustrates how to list records with dates outside of the valid range (for example, a month not between 01-12).

Note: Dates with an invalid digit (A-F) can result in a data exception (0C7 ABEND).

The input records might be as follows:

```
Betten         20091021
Vezinaw        20091101
Casad          00000000
Boenig         20091325
Kolusu         20090931
Yaeger         20090731
```

The SORT statement sorts the records by the name in positions 1-12. After the SORT statement is processed, the sorted records will look like this:

```
Betten         20091021
Boenig         20091325
Casad          00000000
Kolusu         20090931
Vezinaw        20091101
Yaeger         20090731
```

The OUTREC statement uses TOGREG to convert each ccyymmdd value in positions 16-23 to a ccyymmdd value in positions 30-37; the third column will be identical to the second column for valid dates and special indicators, but will contain asterisks for invalid dates. After the OUTREC statement is processed, the reformatted records will look like this:

```
Betten         20091021      20091021
Boenig         20091325      ********
Casad          00000000      00000000
Kolusu         20090931      ********
Vezinaw        20091101      20091101
Yaeger         20090731      20090731
```

The OUTFIL statement selects the records that have an asterisk in position 30, that is, the records with an invalid date. The Boenig record is invalid because mm is 13, and the Kolusu record is invalid because mm is 09 but dd is 31 (September only has 30 days). Note that the special indicator of all 0s for the Casad record is valid. The output records would be as follows:

```
Boenig         20091325      ********
Kolusu         20090931      ********
```

Example 19

```
  OPTION COPY,Y2PAST=1990
  OUTREC IFTHEN=(WHEN=INIT,
    BUILD=(1:1,8,UFF,TO=ZD,LENGTH=6,8:11,6,UFF,TO=ZD,LENGTH=5)),
   IFTHEN=(WHEN=INIT,
     BUILD=(1,6,Y2W,DATEDIFF,8,5,Y2T))
```

This example illustrates how you can calculate the difference in days between two different types of date fields, each of which has separators.

The SORTIN data set has these input records with a C'mm/dd/yy' date field in positions 1-8 and a C'yy/ddd' date field in positions 11-16:

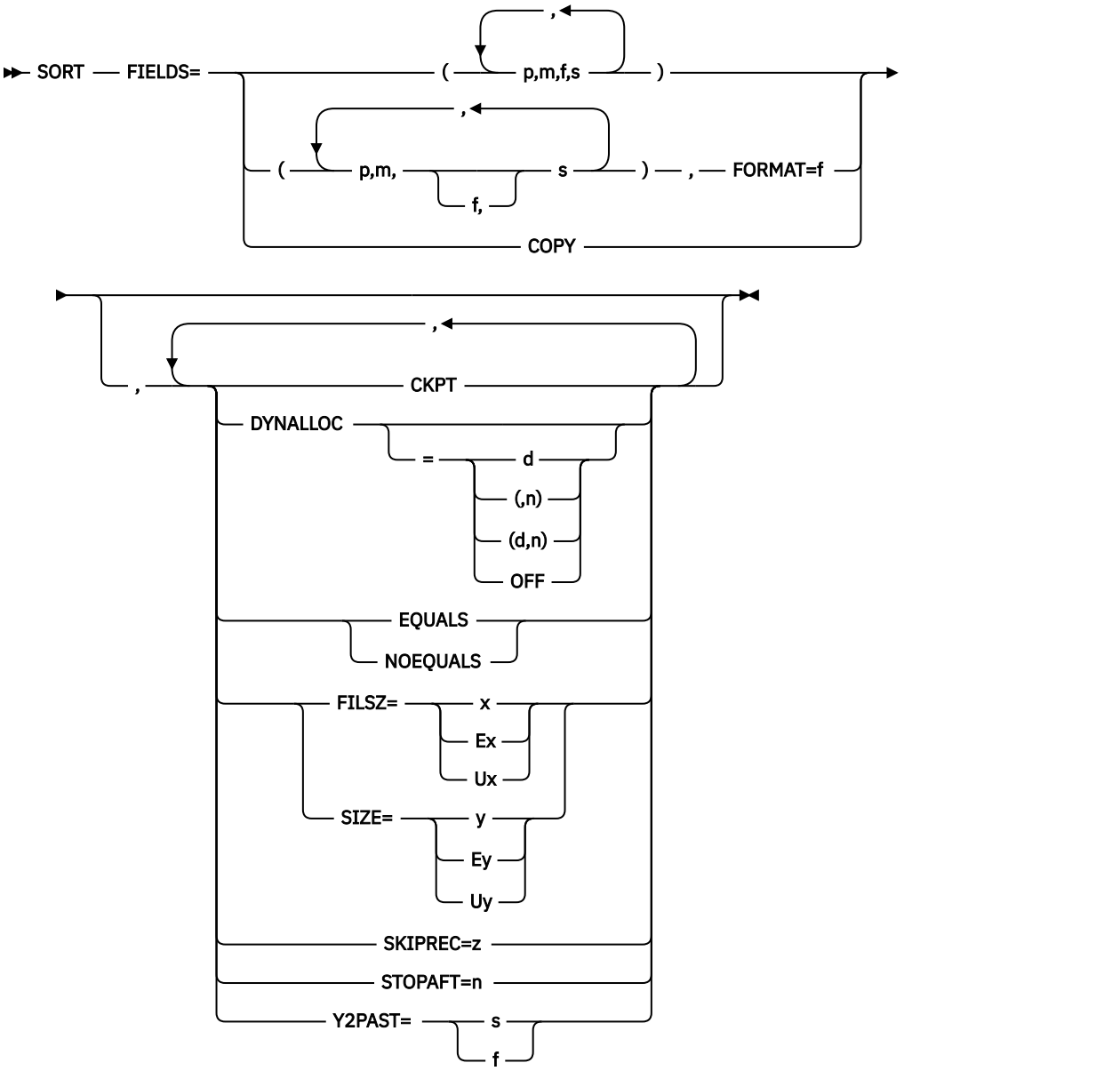
```
03/05/07 07/052
12/13/07 08/193
02/19/08 08/365
09/01/08 09/001
11/22/09 09/015
07/22/98 01/121
01/15/09 09/015
09/15/10 09/322
06/30/10 08/050
```

The OUTREC statement calculates the number of days for date1-date2 and puts the result in the output record in positions 1-8.

The first IFTHEN clause removes the / separators from date1 and date2 so we can use them in DATEDIFF. After the first IFTHEN clause, the records look like this, with the C'mmddyy' date in positions 1-6 and the C'yyddd' date in positions 8-12:

```
030507 07052
121307 08193
021908 08365
090108 09001
112209 09015
072298 01121
011509 09015
091510 09322
063010 08050
```

SORT control statement



The SORT control statement must be used when a sorting application is performed; this statement describes the control fields in the input records on which the program sorts. A SORT statement can also be used to specify a copy application. User labels will not be copied to the output data sets.

The way in which DFSORT processes short SORT control fields depends on the setting for VLSHRT/NOVLSHRT. A short field is one where the variable-length record is too short to contain the entire field, that is, the field extends beyond the record. For details about sorting short records, see the discussion of the VLSHRT and NOVLSHRT options in “OPTION control statement” on page 171.

The options available on the SORT statement can be specified in other sources as well. A table showing all possible sources for these options and the order of override is given in Appendix B, “Specification/override of DFSORT options,” on page 801.

When an option can be specified on either the SORT or OPTION statement, it is preferable to specify it on the OPTION statement.

Restriction: DFSORT accepts but does not process the following SORT operands: WORK=*value* and ORDER=*value*.

DFSORT's collating behavior can be modified according to your cultural environment. The cultural environment is established by selecting the active locale. The active locale's collating rules affect SORT processing as follows:

- DFSORT produces sorted records for output according to the collating rules defined in the active locale. This provides sorting for single- or multi-byte character data, based on defined collating rules that retain the cultural and local characteristics of a language.

If locale processing is to be used, the active locale will only be used to process character (CH) control fields.

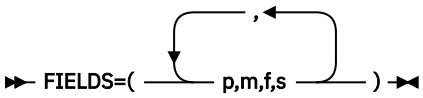
For more information on locale processing, see "Cultural environment considerations " on page 5 or LOCALE in "OPTION control statement" on page 171.

DFSORT's Unicode collation behavior can be modified according to your cultural environment. The cultural environment is established by selecting the active collation version. The active collation version rules affects SORT processing as follows:

- DFSORT produces sorted records for output according to the collating version rules defined in the active collation version. This provides sorting for single-or multi-byte character data based on defined collating version rules that retain the cultural and local characteristics of a language.

See "Unicode Environment Considerations" on page 6 or COLLKEY in "OPTION control statement" on page 171.

FIELDS



Requires four facts about each control field in the input records: the position of the field within the record, the length of the field, the format of the data in the field, and the sequence into which the field is to be sorted. These facts are communicated to DFSORT by the values of the FIELDS operand, represented by p, m, f, and s.

The value for f can optionally be specified by the FORMAT=f parameter as explained later in this section.

All control fields must be located within the first 32752 bytes of a record.

The maximum length of the collected control fields for which Blockset can be used is 4088 bytes. However, the maximum length can be less than 4088 for various situations, such as the use of certain formats (for example, PD), the EQUALS option, and so on. If this maximum is exceeded, Blockset cannot be used.

The FIELDS operand can be written in two ways.

The program examines the major control field first, and it must be specified first. The minor control fields are specified following the major control field. p, m, f, and s describe the control fields. The text that follows gives specifications in detail.

p
specifies the first byte of a control field relative to the beginning of the input record. [14]

The first data byte of a fixed-length record has relative position 1. The first data byte of a variable-length record has relative position 5. The first 4 bytes contain the record descriptor word. All

[14] If INREC is specified, p must refer to the record as reformatted by INREC. If your E15 user exit reformats the record, and INREC is not specified, p must refer to the record as reformatted by your E15 user exit.

control fields, except binary, must begin on a byte boundary. The first byte of a floating-point field is interpreted as a signed exponent; the rest of the field is interpreted as the fraction.

Fields containing binary values are described in a "bytes.bits" notation as follows:

1. First, specify the byte location relative to the beginning of the record and follow it with a period.
2. Then, specify the bit location relative to the beginning of that byte. Remember that the first (high-order) bit of a byte is bit 0 (not bit 1); the remaining bits are numbered 1 through 7.

Thus, 1.0 represents the beginning of a record. A binary field beginning on the third bit of the third byte of a record is represented as 3.2. When the beginning of a binary field falls on a byte boundary (say, for example, on the fourth byte), you can write it in one of three ways:

```
4.0
4.
4
```

Other examples of this notation are shown in Figure 8 on page 423:

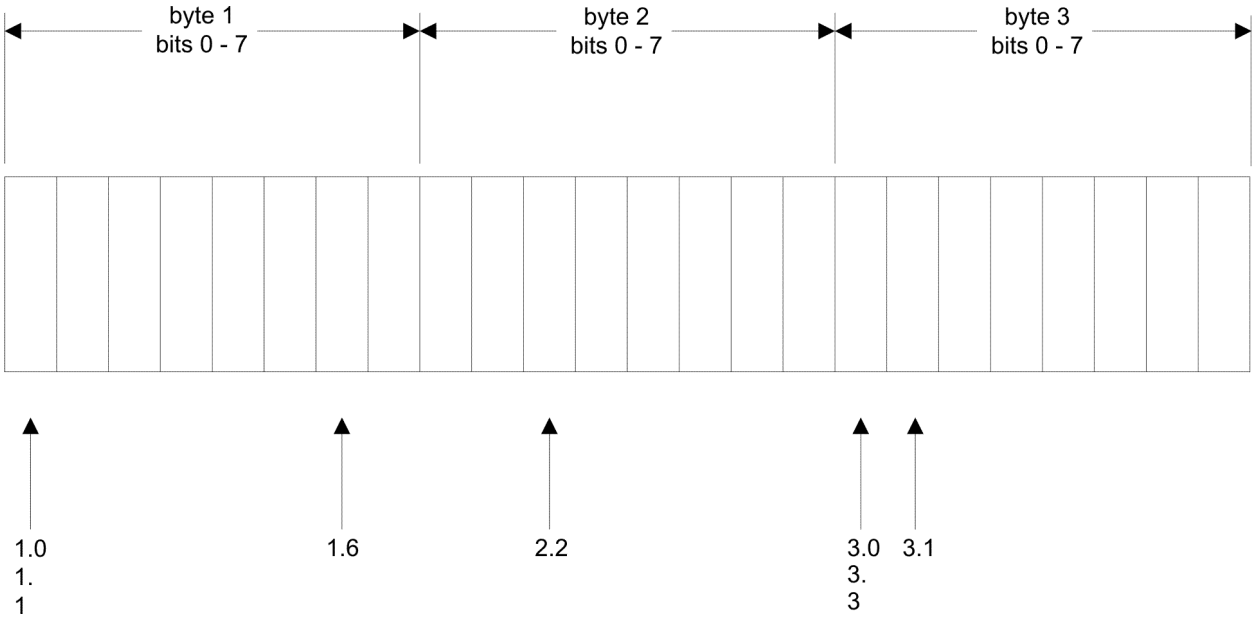


Figure 8. Examples of Notation for Binary Fields

m

specifies the length of the control field. Values for all control fields except binary fields must be expressed in integer numbers of bytes. Binary fields can be expressed in the notation “bytes.bits”. The length of a binary control field that is an integer value (d) can be expressed in one of three ways:

```
d.0
d.
d
```

The number of bits specified must not exceed 7. A control field 2 bits long would be represented as 0.2.

The total number of bytes occupied by all control fields must not exceed 4092 (or, when the EQUALS option is in operation, 4088 bytes). When you determine the total, count a binary field as occupying an entire byte if it occupies any part of it. For example, a binary field that begins on byte 2.6 and is 3 bits long occupies two bytes. All fields must be completely contained within the first 32752 bytes of the record.

f

specifies the format of the data in the control field. Acceptable control field lengths (in bytes) and available formats are shown in Table 64 on page 424.

Table 64. Control field formats and lengths

Format	Length	Description
CH	1 to 4092 bytes	Character [15]
AQ	1 to 4092 bytes	Character with alternate collating sequence
UTF8	1 to 450 Bytes	Unicode Transformation Format, a 8 bit encoding form
UTF16	1 to 450 Bytes	Unicode Transformation Format, a 16 bit encoding form
UTF32	1 to 450 Bytes	Unicode Transformation Format, a 32 bit encoding form
ZD	1 to 256 bytes	Signed zoned decimal
PD	1 to 256 bytes	Signed packed decimal
PD0	2 to 8 bytes	Packed decimal with sign and first digit ignored
FI	1 to 256 bytes	Signed fixed-point
BI	1 bit to 4092 bytes	Unsigned binary
FL	1 to 256 bytes	Signed hexadecimal floating-point
AC	1 to 4092 bytes	ASCII character
CSF or FS	1 to 32 bytes	Signed numeric with optional leading floating sign
UFF	1 to 44 bytes	Unsigned free form numeric
SFF	1 to 44 bytes	Signed free form numeric
CSL or LS	2 to 256 bytes	Signed numeric with leading separate sign
CST or TS	2 to 256 bytes	Signed numeric with trailing separate sign
CLO or OL	1 to 256 bytes	Signed numeric with leading overpunch sign
CTO or OT	1 to 256 bytes	Signed numeric with trailing overpunch sign
ASL	2 to 256 bytes	Signed ASCII numeric with leading separate sign
AST	2 to 256 bytes	Signed ASCII numeric with trailing separate sign
D1	1 to 4092 bytes	User-defined data type (requires an EFS program)
Y2T	3 to 6 bytes	Character or zoned yyx...x full date format with special indicators

[15] If CHALT is in effect, CH is treated as AQ.

Table 64. Control field formats and lengths (continued)

Format	Length	Description
Y2U	2 or 3 bytes	Packed decimal yyx and yyxxx full date format with special indicators
Y2V	3 or 4 bytes	Packed decimal yyxx and yyxxxx full date format with special indicators
Y2W	3 to 6 bytes	Character or zoned x...xyy full date format with special indicators
Y2X	2 or 3 bytes	Packed decimal xyy and xxxyy full date format with special indicators
Y2Y	3 or 4 bytes	Packed decimal xxyy and xxxxyy full date format with special indicators
Y2C or Y2Z	2 bytes	Two-digit character or zoned-decimal year data
Y2P	2 bytes	Two-digit packed-decimal year data
Y2D	1 byte	Two-digit decimal year data
Y2S	2 bytes	Two-digit character or zoned-decimal year data with special indicators
Y2B	1 byte	Two-digit binary year data
AUF	1 to 44 bytes	ASCII Unsigned free form numeric
ASF	1 to 44 bytes	ASCII Signed free form numeric

Note: See Appendix C, “Data format descriptions,” on page 819 for detailed format descriptions.

AUF, ASF, CSF, FS, UFF, SFF, Y2 and PD0 format fields can only be used if Blockset is selected.

For Y2 format fields, real dates are collated using the century window established by the Y2PAST option in effect, but the century window is not used for special indicators. Thus the Y2 formats will collate real dates and special indicators as follows:

- Y2T and Y2W:

  Ascending:
  BI zeros, blanks, CH/ZD zeros, lower century dates (for example, 19yy), upper century dates (for example, 20yy), CH/ZD nines, BI ones.

  Descending:
  BI ones, CH/ZD nines, upper century dates (for example, 20yy), lower century dates (for example, 19yy), CH/ZD zeros, blanks, BI zeros.

- Y2U, Y2V, Y2X and Y2Y:

  Ascending:
  PD zeros, lower century dates (for example, 19yy), upper century dates (for example, 20yy), PD nines.

  Descending:
  PD nines, upper century dates (for example, 20yy), lower century dates (for example, 19yy), PD zeros.

- Y2C, Y2Z, Y2P, Y2D and Y2B:

  Ascending:
  Lower century years (for example, 19yy), upper century years (for example, 20yy).

  Descending:
  Upper century years (for example, 20yy), lower century years (for example, 19yy).

- Y2S:

  Ascending:
  BI zeros, blanks, lower century years (for example, 19yy), upper century years (for example, 20yy), BI ones.

  Descending:
  BI ones, upper century years (for example, 20yy), lower century years (for example, 19yy), blanks, BI zeros.

The AC format sequences **EBCDIC data** using the ASCII collating sequence shown in Appendix D. For example, if AC is used with ascending sequence, the EBCDIC numbers (0-9) will collate before the EBCDIC uppercase letters (A-Z) which in turn will collate before the EBCDIC lowercase letters (a-z).

You can use p,m,s rather than p,m,f,s if you use FORMAT=f to supply the format for the field, as described later in this section.

All floating-point data must be normalized before the program can collate it properly. You can use an E15 or E61 user exit to do this during processing. If you use E61, specify the E option for the value of s in the FIELDS operand for each control field you are going to modify with this user exit.

s
specifies how the control field is to be ordered. The valid codes are:

A
ascending order

D
descending order

E
control fields to be modified

Specify E if you include an E61 user exit to modify control fields before the program sorts them. After an E61 user exit modifies the control fields, DFSORT collates the records in ascending order using the formats specified. [16]

For information on how to add a user exit, see Chapter 5, “Using your own user exit routines,” on page 465.

Default: None; must be specified. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

FORMAT



FORMAT=f can be used to specify a particular format for one or more control fields. f from FORMAT=f is used for p,m,s fields. f from FORMAT=f is ignored for p,m,f,s fields. For example, the following are all equivalent:

```
SORT FIELDS=(5,5,ZD,A,12,6,PD,D,21,3,PD,A,35,7,ZD,A)

SORT FORMAT=ZD,FIELDS=(5,5,A,12,6,PD,D,21,3,PD,A,35,7,A)

SORT FIELDS=(5,5,ZD,A,12,6,D,21,3,A,35,7,ZD,A),FORMAT=PD
```

The permissible field formats are shown under the description of 'f' for fields.

If you have specified the COPY operand, FORMAT=f cannot be specified.

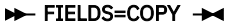
[16] With a conventional merge or a tape work data set sort, control fields for which E is specified are treated as binary byte format regardless of the actual formats specified.

Default: None; FORMAT=f must be specified if any field is specified as p,m,s rather than p,m,f,s. See Appendix B, "Specification/override of DFSORT options," on page 801 for full override details.

Applicable Functions: See Appendix B, "Specification/override of DFSORT options," on page 801.

Note: DFSORT issues an informational message and ignores FORMAT=f if all of the fields are specified as p,m,f,s.

FIELDS=COPY



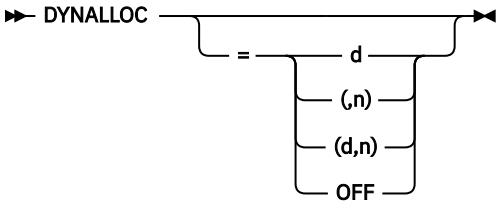
See the discussion of the COPY option discussed in "OPTION control statement" on page 171.

CKPT



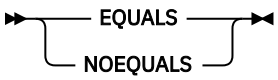
See the discussion of this option discussed in "OPTION control statement" on page 171.

DYNALLOC



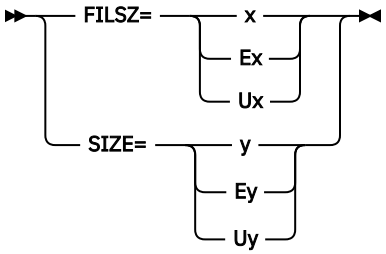
See the discussion of this option in "OPTION control statement" on page 171.

EQUALS or NOEQUALS



See the discussion of these options in "OPTION control statement" on page 171.

FILSZ or SIZE



See the discussion of these options in "OPTION control statement" on page 171.

SKIPREC



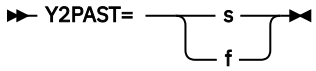
See the discussion of this option in "OPTION control statement" on page 171.

STOPAFT



See the discussion of this option in "OPTION control statement" on page 171.

Y2PAST



See the discussion of this option in "OPTION control statement" on page 171.

Note: CENTURY=value and CENTWIN=value can be used instead of Y2PAST=value.

SORT/MERGE statement notes

- If records are reformatted by INREC (SORT and MERGE) or E15 (SORT), FIELDS must refer to fields in the appropriate reformatted records.
- If SZERO is in effect, -0 collates before +0 in ascending order and after +0 in descending order when numeric fields are sorted or merged. If NOSZERO is in effect, -0 collates equally with +0 when numeric fields are sorted or merged. However, SZERO is always used for a conventional merge or tape work data set sort application.
- If records are being sorted/merged using UTF8/UTF16/UTF32 data formats, then the following operands are not supported.
  - LOCALE
  - EFS
  - SUM
  - E61 exit
  - VLSHRT
  - INREC
  - OUTREC

Specifying a SORT or COPY—examples

Example 1

```
SORT   FIELDS=(2,5,FS,A),FILSZ=29483
```

FIELDS
The control field begins on the second byte of each record in the input data set, is five bytes long, and contains floating sign data. It is to be sorted in ascending order.

FILSZ
The data set to be sorted contains exactly 29483 records.

Example 2

```
SORT   FIELDS=(7,3,CH,D,1,5,FI,A,398.4,7.6,BI,D,99.0,230.2,
         BI,A,452,8,FL,A),DYNALLOC=(3390,4)
```

FIELDS
The first four values describe the major control field. It begins on byte 7 of each record, is 3 bytes long, and contains character (EBCDIC) data. It is to be sorted in descending order.

The next four values describe the second control field. It begins on byte 1, is 5 bytes long, contains fixed-point data, and is to be sorted in ascending order.

The third control field begins on the fifth bit (bits are numbered 0 through 7) of byte 398. The field is 7 bytes and 6 bits long (occupies 9 bytes), and contains binary data to be placed in descending order.

The fourth control field begins on byte 99, is 230 bytes and 2 bits long, and contains binary data. It is to be sorted in ascending order.

The fifth control field begins on byte 452, is 8 bytes long and contains normalized hexadecimal floating-point data, which is to be sorted in ascending order. If the data in this field were not normalized, you could specify E instead of A and include your own E61 user exit routine to normalize the field before the program examined it.

DYNALLOC
Four work data sets are allocated on 3390.

Example 3

```
SORT   FIELDS=(3,8,ZD,E,40,6,CH,D)
```

FIELDS
The first four values describe the major control field. It begins on byte 3 of each record, is 8 bytes long, and contains zoned decimal data that is modified by your routine before sort examines the field.

The second field begins on byte 40, is 6 bytes long, contains character (EBCDIC) data, and is sorted in descending sequence.

Example 4

```
SORT   FIELDS=(7025,4,A,5048,8,A),FORMAT=ZD,EQUALS
```

FIELDS
The major control field begins on byte 7025 of each record, is 4 bytes long, contains zoned decimal data (FORMAT=ZD), and is to be sorted in ascending sequence.

The second control field begins on byte 5048, is 8 bytes long, has the same data format as the first field, and is also to be sorted in ascending order.

FORMAT
FORMAT=ZD is used to supply ZD format for the p,m,s fields and is equivalent to specifying p,m,ZD,s for these fields.

With FORMAT=f, you can mix p,m,s and p,m,f,s fields when that's convenient such as when all or most of the fields have the same format (although you can always code p,m,f,s for all fields and not use FORMAT=f, if you prefer). For example, the following are also valid uses of the FORMAT=f parameter:

```
SORT FORMAT=BI,FIELDS=(21,4,A,5,4,PD,A,31.3,1.4,A,52,20,A)

SORT FIELDS=(16,4,A,22,8,BI,D,3,2,A),FORMAT=FI
```

EQUALS
specifies that the sequence of equal collating records is to be preserved from input to output.

Example 5

```
SORT   FIELDS=COPY
```

FIELDS
The input data set is copied to the output data set without sorting or merging.

Example 6

```
OPTION Y2PAST=1950
SORT FIELDS=(21,6,Y2T,A,13,3,Y2X,D)
```

Y2PAST
Sets a century window of 1950–2049.

FIELDS
Sorts on a C'yymmdd' (or Z'yymmdd') date in positions 21-26 in ascending order, and on a P'dddyy' date in positions 13-15 in descending order. "Real" dates are sorted using the century window of 1950-2049. Special indicators are sorted correctly relative to the "real" dates.

Example 7: Sorting UTF8 data

```
SORT FIELDS=(9,2,UTF8,D)
```

FIELDS
The control field begins on the ninth byte of each record in the input data set, is 2 bytes long, and contains 8 bit encoding Unicode Transformation Format (UTF8) data. It is to be sorted in descending order.

Example 8: Sorting UTF16 data

```
SORT FIELDS=(5,4,FI,A,345,400,UTF16,D,13,2,CH,A)
```

FIELDS
The first four values describe the major control field. It begins on byte 5 of each record, is 4 bytes long, and contains fixed-point data, and is to be sorted in ascending order.

The next four values describe the second control field. It begins on byte 345, is 400 bytes long, contains a 16 bit encoding Unicode Transformation Format (UTF16) data, and is to be sorted in descending order.

The third control field begins on byte 13 is 2 bytes long, and contains character (EBCDIC) data. It is to be sorted in ascending order.

Example 9: Sorting UTF32 data

```
OPTION COLLKEY=UCA600_LFR_RFR
```

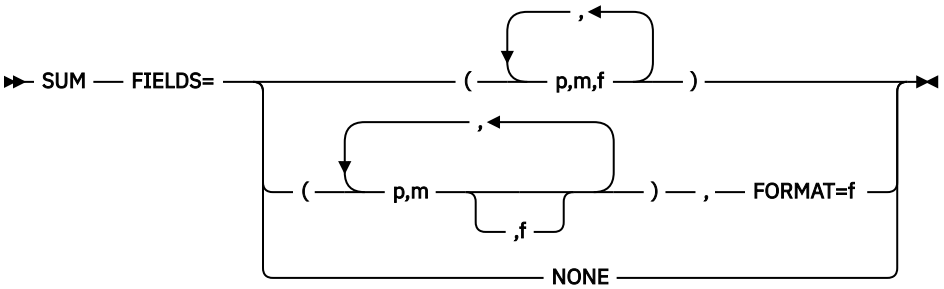
```
SORT FIELDS=(15,50,A), FORMAT=UTF32
```

COLLKEY
The collation version is UCA600 supports the Unicode Standard character suite 6.0.0 and uses Normalization Service under 6.0.0 Unicode character suite. The Language is French and the Region is France.

FIELDS
The control field begins on byte 15 of each record in the input data set, is 50 bytes long, and contains 32 bit encoding Unicode Transformation Format (UTF32) data. It is to be sorted in ascending order.

FORMAT
FORMAT=UTF32 is used to supply UTF32 format for the p,m,s fields and is equivalent to specifying p,m,UTF32,s for these fields.

SUM control statement

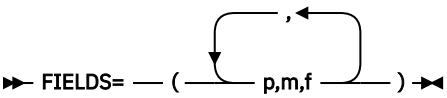


The SUM control statement specifies that, whenever two records are found with equal sort or merge control fields, the contents of their summary fields are to be added, the sum is to be placed in one of the records, and the other record is to be deleted. If the EQUALS option is in effect the **first record** of summed records is kept. If the NOEQUALS option is in effect, the record to be kept is unpredictable. For further details, see "SUM statement notes" on page 432.

If the ZDPRINT option is in effect, positive summed ZD values are printable. If the NZDPRINT option is in effect, positive summed ZD values are not printable. For further details, see "SUM statement notes" on page 432.

The way in which DFSORT processes short SUM summary fields depends on whether the VLSHRT or NOVLSHRT option is in effect. A short field is one where the variable-length record is too short to contain the entire field; that is, the field extends beyond the record. For details about sorting, merging and summing short records, see the discussion of the VLSHRT and NOVLSHRT options in "OPTION control statement" on page 171.

FIELDS



Designates numeric fields in the input record as summary fields.

p
specifies the first byte of the field relative to the beginning of the input record. [17] The first data byte of a fixed-length record has relative position 1. The first data byte of a variable-length record has relative position 5, as the first four bytes are occupied by the RDW. All fields must start on a byte boundary and no field can extend beyond byte 32752

m
specifies the length in bytes of the summary fields to be added. See Table 65 on page 431for permissible length values.

f
specifies the format of the data in the summary field:

Table 65. Summary field formats and lengths

Format	Length	Description
BI	2, 4, or 8 bytes	Unsigned binary

[17] If INREC is specified, p must refer to the record as reformatted by INREC. If your E15 user exit reformats the record, and INREC is not specified, p must refer to the record as reformatted by your E15 user exit.

Table 65. Summary field formats and lengths (continued)		
Format	**Length**	**Description**
FI	2, 4, or 8 bytes	Signed fixed-point
FL	4, 8, or 16 bytes	Signed hexadecimal floating-point
PD	1 to 16 bytes	Signed packed decimal
ZD	1 to 31 bytes	Signed zoned decimal

The value for f can optionally be specified by the FORMAT=f parameter as explained later in this section.

Note: See Appendix C, “Data format descriptions,” on page 819 for detailed format descriptions.

NONE
eliminates records with duplicate keys. Only one record with each key is kept and no summing is performed.

Note: The FIRST operand of ICETOOL's SELECT operator can be used to perform the same function as SUM FIELDS=NONE with OPTION EQUALS. Additionally, SELECT's FIRSTDUP, ALLDUPS, NODUPS, HIGHER(x), LOWER(y), EQUAL(v), LASTDUP, and LAST operands can be used to select records based on other criteria related to duplicate and non-duplicate keys. SELECT's DISCARD(savedd) operand can be used to save the records discarded by FIRST, FIRSTDUP, ALLDUPS, NODUPS, HIGHER(x), LOWER(y), EQUAL(v), LASTDUP, or LAST. See “SELECT operator” on page 626 for complete details on the SELECT operator.

Default: None; must be specified.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

FORMAT



FORMAT=f can be used to specify a particular format for one or more summary fields. f from FORMAT=f is used for p,m fields. f from FORMAT=f is ignored for p,m,f fields. For example, the following are all equivalent:

```
SUM FIELDS=(5,5,ZD,12,6,PD,21,3,PD,35,7,ZD)

SUM FORMAT=ZD,FIELDS=(5,5,12,6,PD,21,3,PD,35,7)

SUM FIELDS=(5,5,ZD,12,6,21,3,35,7,ZD),FORMAT=PD
```

The permissible field formats are shown under the description of 'f' for fields.

Default: None. FORMAT=f must be specified if any field is specified as p,m rather than p,m,f. See Appendix B, “Specification/override of DFSORT options,” on page 801 for full override details.

Applicable Functions: See Appendix B, “Specification/override of DFSORT options,” on page 801.

Note: DFSORT issues an informational message and ignores FORMAT=f if all of the fields are specified as p,m,f.

SUM statement notes

- If overflow might occur during summation, INREC can be used to create a larger SUM field in the reformatted input record (perhaps resulting in a larger record for sorting or merging) so that overflow does not occur. “Example 5” on page 435 illustrates this technique.

- An invalid PD or ZD sign or digit results in a data exception (0C7 ABEND); 0-9 are invalid for the sign and A-F are invalid for the digit. For example, a ZD value such as 3.5 (X'F34BF5') results in an 0C7 because "." (X'4B') is treated as an invalid digit. ICETOOL's DISPLAY or VERIFY operator can be used to identify decimal values with invalid digits. ICETOOL's VERIFY operator can be used to identify decimal values with invalid signs.
- Whether or not positive summed ZD results have printable numbers depends on whether NZDPRINT or ZDPRINT is in effect (as set by the ZDPRINT installation option and the NZDPRINT and ZDPRINT parameters of the OPTION statement):
  - If NZDPRINT is in effect, positive summed ZD results do not consist of printable numbers, regardless of whether the original values consisted of printable numbers or not. For example, if X'F2F3F1' (prints as '231') and X'F3F0F6' (prints as '306') are summed, the result with NZDPRINT in effect is X'F5F3C7' (prints as '53G').
  - If ZDPRINT is in effect, positive summed ZD results consist of printable numbers, regardless of whether the original values consisted of printable numbers or not. For example, if X'F2F3C1' (prints as '23A') and X'F3F0F6' (prints as '306') are summed, the result with ZDPRINT in effect is X'F5F3F7' (prints as '537').

  Thus, ZDPRINT must be in effect to ensure that positive summed ZD results are printable.

  Unsummed positive ZD values retain their original signs, regardless of whether NZDPRINT or ZDPRINT is in effect. For example, if X'F2F8C5' is not summed, it remains X'F2F8C5' (prints as '28E'). OUTFIL's OUTREC parameter can be used to ensure that all summed or unsummed ZD values are printable, as illustrated by Example 4 later in this section.
- If input records are reformatted by INREC or E15, SUM must refer to fields in the appropriate reformatted record (see the preceding description of p).
- Summary fields must not be control fields. They must not overlap control fields, or each other, and must not overlap the RDW.
- FL (hexadecimal floating-point) values to be summed can be normalized or unnormalized. However, the resulting FL values are always normalized. Normalization processing by the hardware can produce different sums for FL values summed in different orders.
- Exponent overflow for summed FL values results in an exponent overflow exception (0CC ABEND)
- Exponent underflow for summed FL values results in a true zero result.
- When records are summed, you can predict which record is to receive the sum (and be retained) and which record is to be deleted only when EQUALS is in effect, overflow does not occur, and the BLOCKSET technique is used. In this case, the first record (based on the sequence described under the discussion of the EQUALS or NOEQUALS parameter of the “OPTION control statement” on page 171) is chosen to contain the sum.

  Fields other than summary fields remain unchanged and are taken from the record that receives the sum.
- You can control the action that DFSORT takes when overflow occurs for BI, FI, PD or ZD values with the OVFLO parameter as described in “OPTION control statement” on page 171.
-
- DFSORT issues a message and terminates processing if a SUM statement is specified for a tape work data set sort or Conventional merge.
- DFSORT does not support the XSUM parameter provided by a competitive sort product to write records deleted by SUM processing to a SORTXSUM DD data set. However, ICETOOL's SELECT operator can perform the same function as XSUM with FIELDS=NONE. For example, this ICETOOL job:

```
//S1EXEC PGM=ICETOOL
//TOOLMSG DD SYSOUT=*
//DFSMSG DD SYSOUT=*
//SORTIN DD DSN=...
//SORTOUT DD DSN=...
//SORTXSUM DD DSN=...
//TOOLIN DD *
SELECT FROM(SORTIN) TO(SORTOUT)-
```

```
  ON(5,4,CH) FIRST DISCARD(SORTXSUM)
/*
```

is equivalent to this XSUM job:

```
//S1 EXEC PGM=ICEMAN
//SYSOUT DD SYSOUT=*
//SORTIN DD DSN=...
//SORTOUT DD DSN=...
//SORTXSUM DD DSN=...
//SYSIN DD *
  SORT FIELDS=(5,4,CH,A)
  SUM FIELDS=NONE,XSUM
/*
```

Tip: You can also perform additional functions with ICETOOL's SELECT operator that are not available with XSUM. See Chapter 7, “Using ICETOOL,” on page 533 for complete details of ICETOOL's SELECT operator.

Adding summary fields—examples

Example 1

```
  SUM FIELDS=(15021,8,PD,15011,4,FI)
```

This statement designates an 8-byte packed decimal field at byte 15021, and a 4-byte fixed-integer field at byte 15011, as summary fields.

Example 2

```
  SUM FIELDS=NONE
```

This statement illustrates the elimination of duplicate records.

Example 3

```
  SUM FIELDS=(41,8,49,4),FORMAT=ZD
  OPTION ZDPRINT
```

These statements illustrate the use of the FORMAT operand and the ZDPRINT option. The SUM statement designates two zoned decimal fields, one 8 bytes long starting at byte 41, and the other 4 bytes long starting at byte 49. As a result of the ZDPRINT option, the positive summed ZD values will be printable. Note, however, that the ZDPRINT option does not affect ZD values which are not summed due to overflow or unique keys. The next example shows how to use OUTFIL to make all summary fields printable.

Example 4

```
SUM FIELDS=(41,8,49,4),FORMAT=ZD
OUTFIL OUTREC=(1,40,41,8,ZD,M11,49,4,ZD,M11,53,28)
```

These statements illustrate the use of the OUTFIL statement to ensure that all positive ZD summary fields in the output data set are printable. Whereas the ZDPRINT option affects only positive summed ZD fields, OUTFIL can be used to edit positive or negative BI, FI, PD, or ZD values, whether they are summed or not. OUTFIL can also be used to produce multiple output data sets, reports, and so on. See “OUTFIL control statements” on page 219 for complete details about OUTFIL processing.

For purposes of illustration, this example assumes that the input records are 80 bytes long.

Chapter 4. Using a JOINKEYS application for joining two files

Overview

You can perform various types of "join" applications on two files (F1 and F2) by one or more keys with DFSORT using the following statements:

JOINKEYS
You must specify two JOINKEYS statements; one for the F1 file and another for the F2 file. A separate subtask will be used to process each file and a main task will be used to process the joined records from the two files.

Each JOINKEYS statement must specify the ddname of the file it applies to and the starting position, length and sequence of the keys in that file. You can also optionally specify if the file is already sorted by the keys and if sequence checking of the keys is not needed; if the file has fixed-length or variable-length records; to stop reading the file after n records; a 2-byte id to be used for the message and control data set for the subtask used to process the file, and if a subset of the records is to be processed based on a logical expression.

JOIN
If you don't specify a JOIN statement, only paired records from F1 and F2 are kept and processed by the main task as the joined records (inner join). You can optionally specify a JOIN statement to have the main task keep and process: unpaired F1 records as well as paired records (left outer join); unpaired F2 records as well as paired records (right outer join); unpaired F1 and F2 records as well as paired records (full outer join); only unpaired F1 records; only unpaired F2 records, or only unpaired F1 and F2 records.

REFORMAT
You would normally specify a REFORMAT statement to indicate the F1 and/or F2 fields you want in the joined records. You can optionally specify an indicator of where the key was found, and a FILL character to be used for missing bytes. If a JOIN statement with ONLY is specified, the REFORMAT statement is optional.

F1 and F2 can be any type of sequential or VSAM file supported by DFSORT for SORTIN and can have different attributes (for example, F1 can have RECFM=FB and LRECL=100 and F2 can have RECFM=VB and LRECL=254, or F1 can be a VSAM ESDS and F2 can be a PDS member).

F1 will be sorted or copied by "subtask1". An E35 exit will be used to pass the needed fields from the F1 records to the "main task" (an intermediate output data set is not used or required). A subset of the DFSORT statements, such as INCLUDE, OMIT, INREC, SUM and OPTION, will be available for processing the F1 records.

F2 will be sorted or copied by "subtask2". An E35 exit will be used to pass the needed fields from the F2 records to the "main task" (an intermediate output data set is not used or required). A subset of the DFSORT statements, such as INCLUDE, OMIT, INREC, SUM and OPTION, will be available for processing the F2 records.

The "main task" will use an E15 to join the records passed from the E35 of subtask1 and E35 of subtask2. The joined records will be processed as the input records for a sort or copy application. Most of the control statements and options available for a DFSORT sort or copy application will be available for processing the joined records.

Note: Since a JOINKEYS application uses three tasks, it can require more storage than a regular DFSORT application. You may need to use REGION=0M for some JOINKEYS applications.

© Copyright IBM Corp. 1973, 2026

JOINKEYS application processing

Figure 9 on page 438 is a pictorial representation of the processing performed for a JOINKEYS application, and the order in which the various functions are performed.

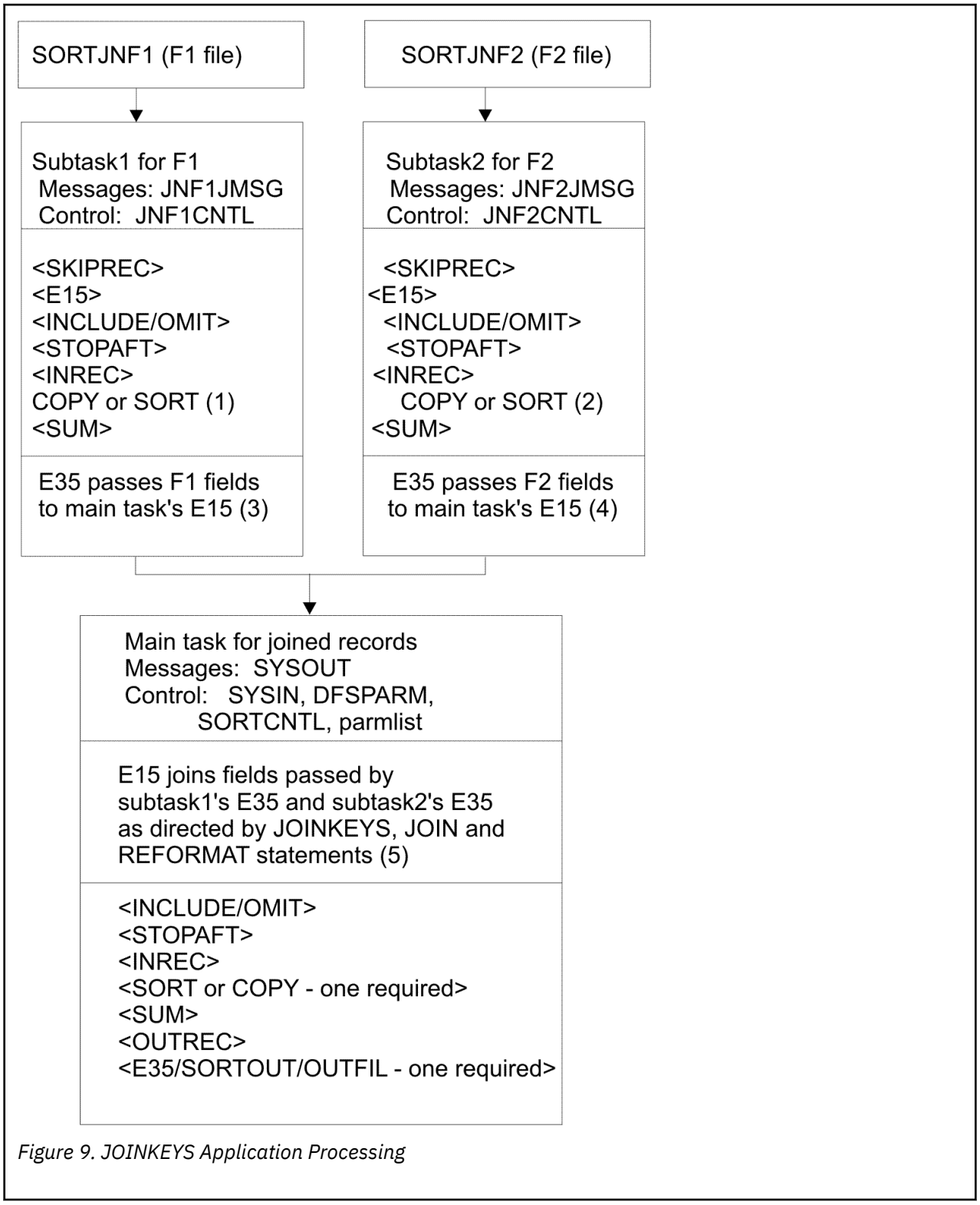


Figure 9. JOINKEYS Application Processing

Legend for Figure 9 on page 438

- (1) COPY is used automatically for subtask1 if the SORTED operand is specified in the F1 JOINKEYS statement. Otherwise, SORT is used automatically with the binary keys specified in the FIELDS operand of the F1 JOINKEYS statement.
- (2) COPY is used automatically for subtask2 if the SORTED operand is specified in the F2 JOINKEYS statement. Otherwise, SORT is used automatically with the binary keys specified in the FIELDS operand of the F2 JOINKEYS statement.
- (3) An E35 exit is used for subtask1 automatically. (An intermediate output file is not required or used.)
- (4) An E35 exit is used for subtask2 automatically. (An intermediate output file is not required or used.)
- (5) An E15 exit is used for the main task automatically. (Intermediate input files are not required or used.)

For direct invocation of DFSORT (for example, PGM=SORT), the JOINKEYS, JOIN and REFORMAT statements can be specified in SYSIN or DFSPARM. For program invocation of DFSORT (for example, LINK EP=SORT), the JOINKEYS, JOIN and REFORMAT statements can be specified in the caller's parameter list, in SORTCNTL or in DFSPARM.

Subtask1 reads the F1 file. It uses COPY or SORT, and an E35 exit (with no output data set) automatically to pass the needed F1 fields to the main task's E15 exit. Subtask1 can optionally use the other listed control statements from JNF1CNTL. Subtask1 messages are displayed in JNF1JMSG.

Subtask2 reads the F2 file. It uses COPY or SORT, and an E35 exit (with no output data set) automatically to pass the needed F2 fields to the main task's E15 exit. Subtask2 can optionally use the other listed control statements from JNF2CNTL. Subtask2 messages are displayed in JNF2JMSG.

The main task uses an E15 exit automatically. The E15 creates the joined records by accessing the F1 fields passed by subtask1's E35 and the F2 fields passed by subtask2's E35. The main task writes the output to SORTOUT and/or OUTFIL or uses a supplied E35 exit to "delete" all of the records. For direct invocation of DFSORT, the main task can optionally use the other listed control statements from SYSIN or DFSPARM. For program invocation of DFSORT, the main task can optionally use the other listed control statements from the caller's parameter list, in SORTCNTL or in DFSPARM. The main task's messages are displayed in SYSOUT.

The starting position in the fields you specify for the subtasks or main task must reflect any reformatting of the records you do at each stage. This includes using the starting positions of the joined records for main task functions.

Examples:

- If you specify INREC and SUM statements in JNF1CNTL for subtask1, the FIELDS operands in the JOINKEYS, REFORMAT, and SUM statements must specify the starting positions in the F1 records as reformatted by INREC.
- If you specify an INCLUDE statement in SYSIN for the main task, the COND operand must specify the starting positions in the joined records as reformatted by REFORMAT.

Sample JOINKEYS applications

Here are the JCL and control statements for a simple JOINKEYS application to do an inner join (also known as a cartesian join) which joins all paired records.

```
//S1   EXEC  PGM=SORT
//SYSOUT DD SYSOUT=*
//SORTJNF1 DD DSN=INPUT1,DISP=SHR
//SORTJNF2 DD DSN=INPUT2,DISP=SHR
//SORTOUT DD SYSOUT=*
//SYSIN DD *
* Control statements for JOINKEYS application
  JOINKEYS FILE=F1,FIELDS=(15,2,A,7,4,A)
  JOINKEYS FILE=F2,FIELDS=(21,2,A,23,4,A)
  REFORMAT FIELDS=(F2:1,70,F1:1,60)
* Control statements for main task (joined records)
  SORT FIELDS=COPY
/*
```

Here are the JCL and control statements for a JOINKEYS application which omits certain records and normalizes keys for F1 and F2, and retains only sorted, non-duplicate, unpaired records from F1. The F1 and F2 files are already in order by the specified JOINKEYS FIELDS. DFSORT symbols are used for various fields and constants.

```
//S2   EXEC  PGM=SORT
//SYSOUT DD SYSOUT=*
//SYMNAMES DD *
IN1_dept,11,3,ch
IN1_target,'J82'
IN1_normal_key,27,5,ZD
IN1_output,1,60
IN2_dept,14,3,ch
IN2_target,'M25'
IN2_PD_key,21,3,PD
IN2_normal_key,=,5,ZD
join_sort_key,45,8,UFF
//FIX DD DSN=F.INPUT,DISP=SHR
//VAR DD DSN=V.INPUT,DISP=SHR
//SORTOUT DD DSN=FIXOUT1,DISP=(NEW,CATLG,DELETE),
// SPACE=(CYL,(5,5)),UNIT=SYSDA
//JNF1CNTL DD *
* Control statements for subtask1 (F1)
  OMIT COND=(IN1_dept,EQ,IN1_target)
  INREC OVERLAY=(IN1_normal_key:IN1_normal_key,TO=ZDF,LENGTH=5)
//JNF2CNTL DD *
* Control statements for subtask2 (F2)
  OMIT COND=(IN2_dept,EQ,IN2_target)
  INREC OVERLAY=(IN2_PD_key:IN2_PD_key,TO=ZDF,LENGTH=5)
//SYSIN DD *
* Control statements for JOINKEYS application
  JOINKEYS F1=FIX,FIELDS=(IN1_normal_key,D),SORTED
  JOINKEYS F2=VAR,FIELDS=(IN2_normal_key,D),SORTED
  JOIN UNPAIRED,F1,ONLY
  REFORMAT FIELDS=(F1:IN1_output)
* Control statements for main task (joined records)
  SORT FIELDS=(join_sort_key,A)
  SUM FIELDS=NONE
/*
```

JCL for a JOINKEYS application

The required JCL statements used for a JOINKEYS application are as follows. See *z/OS DFSORT Application Programming Guide* for general information on DFSORT message, control, input, output, work and symbols data sets.

//stepname EXEC PGM=SORT
Invokes DFSORT. PGM=ICEMAN can also be used.

Note: Since a JOINKEYS application uses three tasks, it can require more storage than a regular DFSORT application. You may need to use REGION=0M for some JOINKEYS applications.

//SYSOUT DD
Messages from the main task. An alternate ddname can be supplied with MSGDDN=ddname in // DFSPARM.

//SORTJNF1 DD
F1 input file (read by subtask1). The ddname is SORTJNF1 (or ccccJNF1 if SORTDD=cccc is in effect) if FILE=F1 or FILES=F1 is specified on the JOINKEYS statement. An alternate ddname can be supplied with F1=ddname on the JOINKEYS statement.

Note: The F1 data set is treated as a SORTIN data set by subtask1 and is subject to the rules for a SORTIN data set documented in Chapter 2, “Invoking DFSORT with Job Control Language,” on page 27.

//SORTJNF2 DD
F2 input file (read by subtask2). The ddname is SORTJNF2 (or ccccJNF2 if SORTDD=cccc is in effect) if FILE=F2 or FILES=F2 is specified on the JOINKEYS statement. An alternate ddname can be supplied with F2=ddname on the JOINKEYS statement.

Note: The F2 data set is treated as a SORTIN data set by subtask2 and is subject to the rules for a SORTIN data set documented in Chapter 2, "Invoking DFSORT with Job Control Language," on page 27.

//SORTOUT DD or //outfil DD
Output of joined records (written by main task). An alternate ddname can be supplied with SORTOUT=ddname or SORTDD=cccc in //DFSPARM or with the OUTFIL statement in //SYSIN or //DFSPARM.

//DFSPARM DD, //SYSIN DD or //SORTCNTL DD
Control statements for the main task including JOINKEYS, JOIN, REFORMAT, OPTION, SORT, INCLUDE or OMIT, SUM, OUTREC, RECORD, ALTSEQ, MODS and OUTFIL. If SKIPREC or E15 is specified, it will not be used.

DFSPARM can be used if DFSORT is invoked directly or called from a program. SYSIN can be used if DFSORT is invoked directly. SORTCNTL can be used if DFSORT is called from a program; an alternate ddname of ccccCNTL can be supplied with SORTDD=cccc in DFSPARM.

Note: If DFSORT is called from a program, the control statements for the main task can be supplied using the parameter list.

The optional JCL statements used for a JOINKEYS application are as follows:

SYMNAMES DD
DFSORT symbols for the main task, subtask1 and subtask2.

SYMNOUT DD
Listing of symbols supplied via the SYMNAMES DD.

JNF1JMSG DD
Messages from subtask1. This message data set will be dynamically allocated with SYSOUT=*, RECFM=FBA, LRECL=121 and BLKSIZE=121 if a JNF1JMSG DD statement is not supplied. If a JNF1JMSG DD statement is supplied, this message data set will be given the same attributes as a //SYSOUT message data set. An alternate ddname of idF1JMSG can be supplied with TASKID=id on the JOINKEYS statement for F1.

JNF2JMSG DD
Messages from subtask2. This message data set will be dynamically allocated with SYSOUT=*, RECFM=FBA, LRECL=121 and BLKSIZE=121 if a JNF2JMSG DD statement is not supplied. If a JNF2JMSG DD statement is supplied, this message data set will be given the same attributes as a //SYSOUT message data set. An alternate ddname of idF2JMSG can be supplied with TASKID=id on the JOINKEYS statement for F2.

JNF1CNTL DD
Control statements for subtask1 including INCLUDE or OMIT, OPTION, MODS, RECORD, ALTSEQ, INREC and SUM. If E35 is specified, it will not be used. The following control statements must not be specified: JOINKEYS, JOIN, REFORMAT, MERGE, OUTFIL, OUTREC or SORT.

An alternate ddname of idF1CNTL can be supplied with TASKID=id on the JOINKEYS statement for F1.

JNF2CNTL DD
Control statements for subtask2 including INCLUDE or OMIT, OPTION, MODS, RECORD, ALTSEQ, INREC and SUM. If E35 is specified, it will not be used. The following control statements must not be specified: JOINKEYS, JOIN, REFORMAT, MERGE, OUTFIL, OUTREC or SORT.

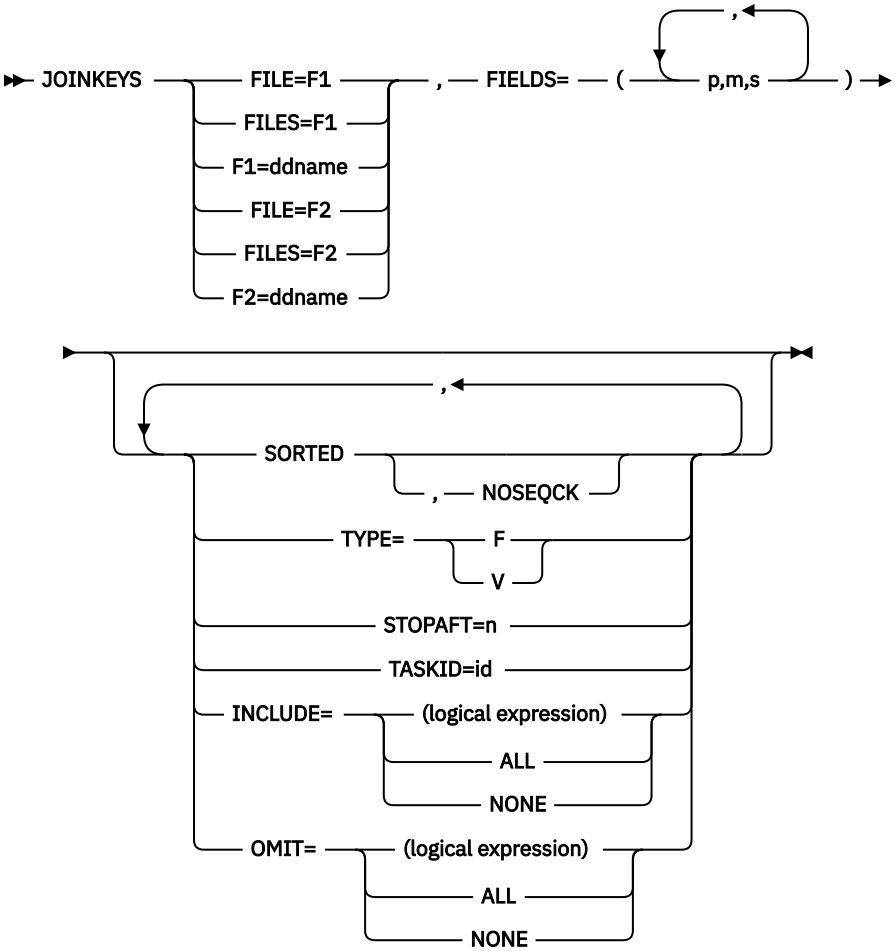
An alternate ddname of idF2CNTL can be supplied with TASKID=id on the JOINKEYS statement for F2.

SORTWKdd DD
Work data sets for the main task. Not recommended since dynamic allocation of work data sets is preferred. Alternate ddnames of ccccWKdd can be supplied with SORTDD=cccc in //DFSPARM.

JNF1WKdd DD
Work data sets for subtask1. Not recommended since dynamic allocation of work data sets will be enabled. Alternate ddnames of idF1WKdd can be supplied with TASKID=id on the JOINKEYS statement for F1.

JNF2WKdd DD
Work data sets for subtask2. Not recommended since dynamic allocation of work data sets will be enabled. Alternate ddnames of idF2WKdd can be supplied with TASKID=id on the JOINKEYS statement for F2.

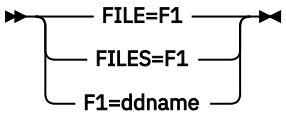
JOINKEYS statements



Two JOINKEYS statements are required for a JOINKEYS application; one for the F1 file and the other for the F2 file:

- FILE=F1 or F1=ddname must be used to indicate that the JOINKEYS statement applies to the F1 input file. FILE=F1 associates the F1 file with a ddname of SORTJNF1. You can use a different ddname for the F1 file by specifying F1=ddname. For simplicity, we will use SORTJNF1 when referring to the ddname for the F1 file.
- FILE=F2 or F2=ddname must be used to indicate that the JOINKEYS statement applies to the F2 input file. FILE=F2 associates the F2 file with a ddname of SORTJNF2. You can use a different ddname for the F2 file by specifying F2=ddname. For simplicity, we will use SORTJNF2 when referring to the ddname for the F2 file.

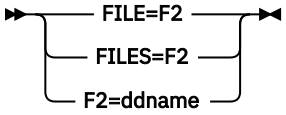
FILE=F1, FILES=F1 or F1=ddname



Must be used for the JOINKEYS statement associated with the F1 file. FILE=F1 (or FILES=F1) specifies a ddname of SORTJNF1 for the F1 file. F1=ddname can be used to specify any valid ddname for the F1 file. Do not use the same ddname for the F1 file and the F2 file.

When FILE=F1, FILES=F1 or F1=ddname is specified, the other operands of the JOINKEYS statement apply to the F1 file.

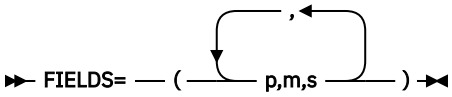
FILE=F2, FILES=F2 or F2=ddname



Must be used for the JOINKEYS statement associated with the F2 file. FILE=F2 (or FILES=F2) specifies a ddname of SORTJNF2 for the F2 file. F2=ddname can be used to specify any valid ddname for the F2 file. Do not use the same ddname for the F1 file and the F2 file.

When FILE=F2, FILES=F2 or F2=ddname is specified, the other operands of the JOINKEYS statement apply to the F2 file.

FIELDS



Must be specified to indicate the starting position, length and order (ascending or descending) of the keys in the input file. The keys will be treated as binary, so they must be "normalized". For example, if the keys are actually zoned decimal, they must have all C and D signs, or all F and D signs. You can use an INREC statement in JNF1CNTL and/or JNF2CNTL to normalize the keys for the F1 file and/or F2 file, respectively, if appropriate.

Each pair of keys for the F1 and F2 files must match with respect to length and order, but can start in different positions. For example, if the first key for the F1 file is 5 bytes ascending and the second key for the F1 file is 3 bytes descending, the first key for the F2 file must be 5 bytes ascending and the second key for the F2 file must be 3 bytes descending.

If a variable-length record is too short to contain a key you specify, the short key value will be compared using binary zeros for the missing bytes.

p
specifies the starting position of the key. The first data byte of a fixed-length record is in position 1. The first data byte of a variable-length record is in position 5 after the 4-byte RDW. p can be 1 to 32752 but all fields must be completely contained within the first 32752 bytes of the record.

m
specifies the length of the key. The total length of all keys must not exceed 4080 bytes. All fields must be completely contained within the first 32752 bytes of the record.

The length for each pair of F1 and F2 keys must match.

s
specifies the order of the key. Use A for ascending or D for descending.

The order for each pair of F1 and F2 keys must match.

For example, if you specify:

```
JOINKEYS F1=IN1,FIELDS=(22,3,A,55,9,D)
JOINKEYS F2=IN2,FIELDS=(15,3,A,1,9,D)
```

File F1 is processed using the ddname IN1, the ascending key in positions 22-24 and the descending key in positions 55-63. File F2 is processed using the ddname IN2, the ascending key in positions 15-17 and the descending key in positions 1-9.

SORTED



By default, DFSORT will sort the input file by the specified keys. If the records of the input file are already in sorted order by the specified keys, you can use the SORTED operand to tell DFSORT to copy the records rather than sort them. This can improve performance. DFSORT will terminate if the copied records are not in the order specified by the keys unless you specify the NOSEQCK operand.

For example, if you specify:

```
JOINKEYS FILE=F1,FIELDS=(22,3,A),SORTED
JOINKEYS FILE=F2,FIELDS=(15,3,A)
```

File F1 is copied using the ddname SORTJNF1 and the ascending key in positions 22-24. The SORTJNF1 records will be checked for the correct key order. File F2 is sorted using the ddname SORTJNF2 and the ascending key in positions 15-17.

If you use the SORTED operand, statements and options only available for a sort application, such as SUM, will be ignored for the subtask that copies the input file.

NOSEQCK



If you specify the SORTED operand and know that the records of the input file are already in sorted order by the specified keys, you can use the NOSEQCK operand to tell DFSORT not to check the order of the records. This can improve performance.

For example, if you specify:

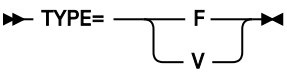
```
JOINKEYS FILE=F1,FIELDS=(22,3,A),SORTED,NOSEQCK
JOINKEYS FILE=F2,FIELDS=(15,3,A),SORTED
```

File F1 is copied using the ddname SORTJNF1 and the ascending key in positions 22-24. The SORTJNF1 records will be not be checked for the correct key order. File F2 is copied using the ddname SORTJNF2 and the ascending key in positions 15-17. The SORTJNF2 records will be checked for the correct key order.

If the records are not actually in order by the specified keys and you use NOSEQCK, the output may be incorrect.

The NOSEQCK operand is ignored if the SORTED operand is not specified.

TYPE



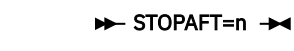
TYPE=V can be used to tell DFSORT to use variable-length processing for a VSAM input file. TYPE=F (the default) can be used to tell DFSORT to use fixed-length processing for a VSAM input file.

For example, if you specify:

```
JOINKEYS F1=VSAM1,FIELDS=(22,3,A),TYPE=V
JOINKEYS F2=VSAM2,FIELDS=(15,3,A),TYPE=F
```

VSAM file F1 is sorted as variable-length records using the ddname VSAM1 and the ascending key in positions 22-24. (Remember that for TYPE=V VSAM processing, DFSORT adds an RDW in positions 1-4 which you must account for when specifying the starting position.) VSAM file F2 is sorted as fixed-length records using the ddname VSAM2 and the ascending key in positions 15-17. (Remember that for TYPE=F VSAM processing, DFSORT does not add an RDW.)

STOPAFT



Can be used to specify the maximum number of records (n) you want the subtask for the input file to accept for sorting or copying (accepted means read from the input file and not deleted by INCLUDE or OMIT).

n
can be up to 28 digits with up to 15 significant digits.

For example, if you specify:

```
  JOINKEYS FILE=F1,STOPAFT=5,FIELDS=(32,4,A)
  JOINKEYS FILE=F2,STOPAFT=10,FIELDS=(32,4,A)
```

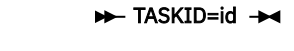
The first 5 input records from SORTJNF1 and the first 10 input records from SORTJNF2 will be processed.

You can use STOPAFT=n on the OPTION statement in JNF1CNTL (for the F1 file) or JNF2CNTL (for the F2 file) instead of specifying STOPAFT=n on the JOINKEYS statement.

For example, instead of the STOPAFT operands in the JOINKEYS statements shown previously, you could specify:

```
//SYSIN DD *
  JOINKEYS FILE=F1,FIELDS=(32,4,A)
  JOINKEYS FILE=F2,FIELDS=(32,4,A)
  ...
//JNF1CNTL DD *
  OPTION STOPAFT=5
//JNF2CNTL DD *
  OPTION STOPAFT=10
```

TASKID



By default, DFSORT uses the following ddnames for the subtasks:

- JNF1JMSG for the subtask1 (F1 file) message data set.
- JNF1CNTL for the subtask1 (F1 file) control data set.
- JNF1WKdd for the subtask1 (F1 file) work data sets.
- JNF2JMSG for the subtask2 (F2 file) message data set.
- JNF2CNTL for the subtask2 (F2 file) control data set.
- JNF2WKdd for the subtask2 (F2 file) work data sets.

The TASKID=id operand can be used to change the first two characters from JN to the specified id characters. The same id can be used for the F1 and F2 ddnames, or a different id can be used for each.

For example, if you specify:

```
  JOINKEYS F1=IN1,FIELDS=(1,10,A),TASKID=C1
  JOINKEYS F2=IN2,FIELDS=(22,10,A),TASKID=C1
```

C1F1JMSG, C1F1CNTL and C1F1WKdd will be used for subtask1 (F1 file). C1F2JMSG, C1F2CNTL and C1F2WKdd will be used for subtask2 (F2 file).

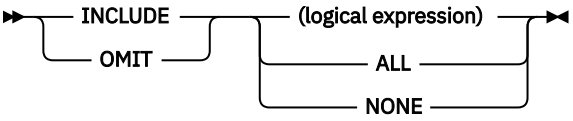
If you specify:

```
  JOINKEYS F1=IN1,FIELDS=(1,10,A),TASKID=I1
  JOINKEYS F2=IN2,FIELDS=(22,10,A),TASKID=I2
```

I1F1JMSG, I1F1CNTL and I1F1WKdd will be used for subtask1 (F1 file). I2F2JMSG, I2F2CNTL and I2F2WKdd will be used for subtask2 (F2 file).

The TASKID=id operand can be useful when you are doing multiple JOINKEYS applications and want to separate the messages for each application and/or specify different control statements or work data sets for different subtasks.

INCLUDE or OMIT



Can be used to specify criteria for the records you want the subtask for the input file to include or omit for sorting or copying. You can use the same logical expressions, ALL or NONE in the same way as for the INCLUDE or OMIT operand of the OUTFIL statement. See "OUTFIL control statements" on page 219 for details.

For example, if you specify:

```
  JOINKEYS FILE=F1,FIELDS=(35,8,A),
     OMIT=(5,4,CH,EQ,C'ABCD')
  JOINKEYS FILE=F2,FIELDS=(37,8,A),
     INCLUDE=(1,20,SS,EQ,C'NO')
```

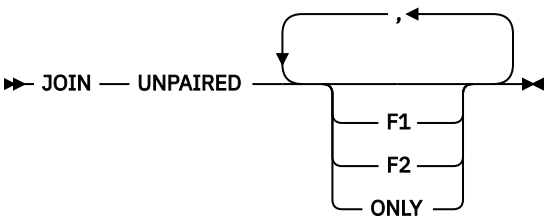
Only records without 'ABCD' in positions 5-8 will be processed from file F1. Only records with 'NO' somewhere in positions 1-20 will be processed from file F2.

Although the INCLUDE and OMIT operands are available on the JOINKEYS statement, it is recommended that you specify an INCLUDE or OMIT statement in JNF1CNTL or JNF2CNTL instead for ease of use.

For example, instead of the OMIT and INCLUDE operands in the JOINKEYS statements shown previously, you could specify:

```
//SYSIN DD *
  JOINKEYS FILE=F1,FIELDS=(35,8,A)
  JOINKEYS FILE=F2,FIELDS=(37,8,A)
  ...
//JNF1CNTL DD *
  OMIT COND=(5,4,CH,EQ,C'ABCD')
//JNF2CNTL DD *
  INCLUDE COND=(1,20,SS,EQ,C'NO')
```

JOIN statement



If you don't specify a JOIN statement for a JOINKEYS application, only paired records from F1 and F2 are kept and processed by the main task as the joined records. This is known as an inner join.

You can change which records are kept and processed by the main task as the joined records by specifying a JOIN statement. You must specify the UNPAIRED operand. The F1, F2 and ONLY operands

are optional. The JOIN operands you specify indicate the joined records to be kept and processed by the main task as follows:

UNPAIRED,F1,F2 or UNPAIRED
Unpaired records from F1 and F2 as well as paired records. This is known as a full outer join.

UNPAIRED,F1
Unpaired records from F1 as well as paired records. This is known as a left outer join.

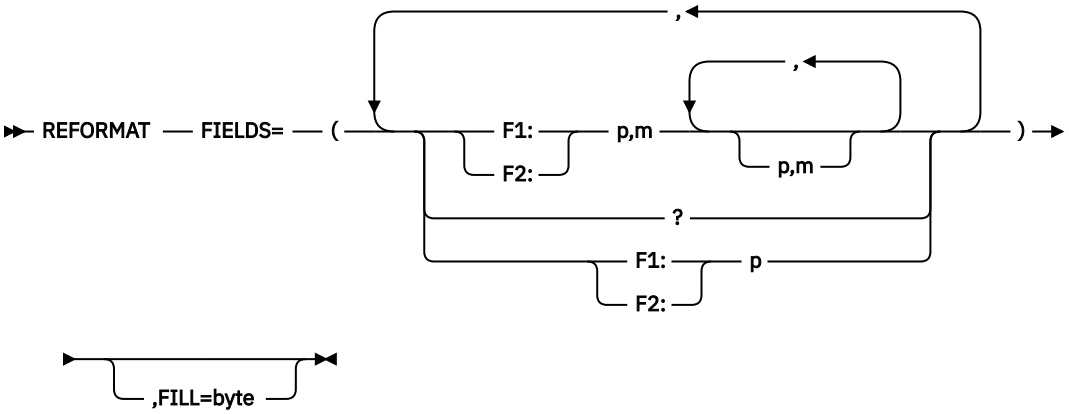
UNPAIRED,F2
Unpaired records from F2 as well as paired records. This is known as a right outer join.

UNPAIRED,F1,F2,ONLY or UNPAIRED,ONLY
Unpaired records from F1 and F2.

UNPAIRED,F1,ONLY
Unpaired records from F1.

UNPAIRED,F2,ONLY
Unpaired records from F2.

REFORMAT statement



A REFORMAT statement can always be used for a JOINKEYS application, and is required unless a JOIN statement with the ONLY operand is specified. The REFORMAT statement indicates the fields from the F1 file and/or the F2 file you want to include in the joined records, and the order in which you want the fields to appear. You can also include an indicator of where the key was found in the joined records ('B' for key found in F1 and F2, '1' for key found in F1 only, or '2' for key found in F2 only).

If the REFORMAT statement only defines position with length (p,m) fields, each joined record will be fixed-length (TYPE=F) with a LENGTH equal to the total length of all of the p,m fields. The maximum length for TYPE=F joined records is 32760 bytes. The F1 and F2 files can both be fixed-length, both be variable-length, or can be mixed fixed-length and variable-length.

If the REFORMAT statement defines a position without a length (p without m) field, each joined record will be variable-length (TYPE=V) with a LENGTH equal to the total length of all of the p,m fields plus the variable length of each p field (one for F1 and/or one for F2). The maximum length for TYPE=V joined records is 32767 bytes. The F1 and F2 files can both be variable-length or can be mixed fixed-length and variable-length. However:

- The first field must contain the RDW (1,n with n equal to or greater than 4) and must be from a variable-length file (F1 or F2).
- The position without length fields (one from the F1 file and/or one from the F2 file) must be the last fields specified and must be from a variable-length file (F1 and/or F2).

For joined records created from paired records, any F1 fields specified will be extracted from the F1 record and any F2 fields specified will be extracted from the F2 record.

For joined records created from unpaired F1 records, any F1 fields specified will be extracted from the F1 record and any F2 fields specified will be filled with the specified FILL character or blanks by default. However, if the F2 file is variable-length, any specified F2 RDW fields (1,n with n equal to or less than 4) will be filled with binary zeros.

For joined records created from unpaired F2 records, any F2 fields specified will be extracted from the F2 record and any F1 fields specified will be filled with the specified FILL character or blanks by default. However, if the F1 file is variable-length, any specified F1 RDW fields (1,n with n equal to or less than 4) will be filled with binary zeros.

If a JOIN statement with the ONLY operand is specified, the REFORMAT statement does not have to be specified. In this case, the layout of the joined records will depend on the specified JOIN statement operands as follows:

JOIN UNPAIRED,F1,ONLY
The joined records will be the original unpaired F1 records. If the F1 records are fixed-length, the joined records will be fixed-length. If the F1 records are variable-length, the joined records will be variable-length.

JOIN UNPAIRED,F2,ONLY
The joined records will be the original unpaired F2 records. If the F2 records are fixed-length, the joined records will be fixed-length. If the F2 records are variable-length, the joined records will be variable-length.

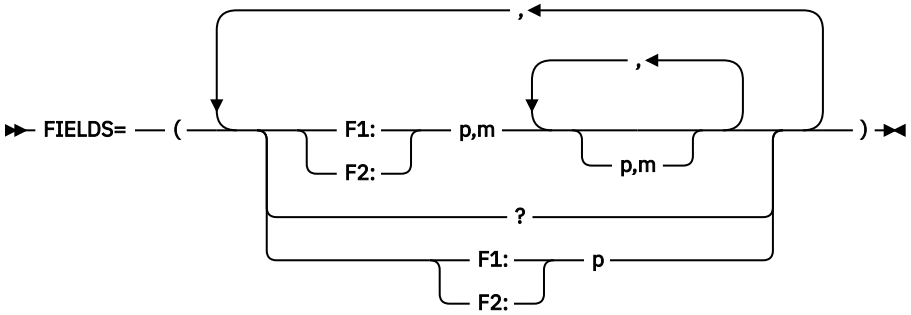
JOIN UNPAIRED,F1,F2,ONLY or JOIN UNPAIRED,ONLY
The joined records will be variable-length. If the F1 records are fixed-length, each unpaired F1 record will be variable-length with an RDW followed by the original F1 record. If the F1 records are variable-length, each unpaired F1 record will be the original F1 record. If the F2 records are fixed-length, each unpaired F2 record will be variable-length with an RDW followed by the original F2 record. If the F2 records are variable-length, each unpaired F2 record will be the original F2 record.

In all cases, the TYPE and LENGTH will be set as appropriate for the joined records.

For joined records with TYPE=F, the maximum LENGTH is 32760.

For joined records with TYPE=V, the maximum LENGTH is 32767. Note that the RECFM of the output file must be VS or VBS in order to allow output records greater than 32756.

FIELDS



Must be specified to indicate the starting position and length of each field from the F1 file and/or the F2 file to be included in the joined records, and optionally an indicator of where the key was found. The fields and indicator will be included in the joined records in the order in which they are specified.

F1:
indicates the following fields up to the next Fn: or end of the FIELDS operand are from the F1 file.

F2:
indicates the following fields up to the next Fn: or end of the FIELDS operand are from the F2 file.

p,m
specifies the starting position and length of a fixed field. p specifies the starting position of the field. The first data byte of a fixed-length record is in position 1. The first data byte of a variable-length record is in position 5 after the 4-byte RDW. p can be 1 to 32767. m specifies the

length of the field. m can be 1 to 32760. All fields must be completely contained within the first 32767 bytes of the record.

?

indicates a 1-byte indicator is to be included in each joined record. The indicator will be set to one of the following values in each paired or unpaired joined record, as appropriate:

- 'B' - the key was found in F1 and F2.
- '1' - the key was found in F1, but not in F2.
- '2' - the key was found in F2, but not in F1.

Only one ? can be specified in the FIELDS operand. If ? is not the last item, it must be followed by F1: or F2:.

For TYPE=F joined records, the indicator can appear anywhere in the record. For example, each of the following is valid:

```
* Put indicator in position 1 of each joined record.
  REFORMAT FIELDS=(?,F1:1,20,F2:5,8)

* Put indicator in position 21 of each joined record.
  REFORMAT FIELDS=(F1:1,20,?,F1:31,9)

* Put indicator in position 25 of each joined record.
  REFORMAT FIELDS=(F2:11,20,6,4,?)
```

For TYPE=V joined records, the indicator must appear in the fixed part of the record, that is, after the RDW and before the position without length fields. For example, the following is valid:

```
* Put indicator in position 5 of each joined record.
  REFORMAT FIELDS=(F1:1,4,?,F1:11)
```

p

p without m gives the starting position of a variable field. Only one p without m field can be specified for F1 and only one p without m field can be specified for F2.

If either or both p without m fields are specified, they must be the last fields in the FIELDS operand. In this case, the first field in the FIELDS operand must be from a variable-length file (F1 or F2) and must include the RDW (1,n where n is equal to or greater than 4).

Example with just p,m fields:

```
  REFORMAT FIELDS=(F1:27,5,1,8,F2:19,20,F1:1201,15)
```

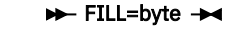
Example with one p without m field:

```
  REFORMAT FIELDS=(F1:1,4,F2:6,25,92,2,F1:8,9,32)
```

Example with two p without m fields:

```
  REFORMAT FIELDS=(F2:1,9,21,3,F1:101,7,28,9,122,F2:26)
```

FILL



The FILL operand can be used to override DFSORT's default fill byte of a blank (X'40'). The fill byte is used in the following situations:

- A p,m (fixed) field is specified for a file (F1 or F2) with variable-length records, and the field extends beyond the end of a record. Each missing byte is replaced with the fill byte. For example, if a variable-length F1 record is 20 bytes long and the REFORMAT statement has:

  ```
  REFORMAT FIELDS=(F1:1,30,41),FILL=C'*'
  ```

  bytes 21-30 of the joined record are filled with asterisks.

- The REFORMAT statement has a p,m (fixed) field from the F1 file and a joined record is being created from an unpaired F2 record. For example if the following are specified:

```
JOIN UNPAIRED,F2
REFORMAT FIELDS=(F1:21,5,F2:1,10),FILL=X'00'
```

  and an unpaired F2 record is found, the joined record will have 5 binary zero bytes followed by bytes 1-10 from the F2 record.
- The REFORMAT statement has a p,m (fixed) field from the F2 file and a joined record is being created from an unpaired F1 record. For example if the following are specified:

```
JOIN UNPAIRED,F1,F2
REFORMAT FIELDS=(F1:21,5,F2:1,10),FILL=X'00'
```

  and an unpaired F1 record is found, the joined record will have bytes 21-25 of the F1 record followed by 10 binary zero bytes. (Since UNPAIRED,F1,F2 is specified, if an unpaired F2 record is found, the joined record will have 5 binary zero bytes followed by bytes 1-10 of the F2 record.)

byte
specifies the fill byte. Permissible values are C'x' and X'yy'.

C'x'
Character fill byte. *x* must be one EBCDIC character. If you want to use an apostrophe as the fill byte, you must specify it as C''''.

X'yy'
Hexadecimal fill byte. *yy* must be one pair of hexadecimal digits (00-FF).

JOINKEYS application notes

In the notes later in this section, "subtaskn" refers to subtask1 for the F1 file or subtask2 for the F2 file.

- Since a JOINKEYS application uses three tasks, it can require more storage than a regular DFSORT application. You may need to use REGION=0M for some JOINKEYS applications.
- DFSORT's normal syntax and continuation rules are used for the JOINKEYS, JOIN and REFORMAT statements. See Chapter 3, “Using DFSORT program control statements,” on page 77 for details.
- Control statements from DFSPARM, SYSIN (direct invocation) or SORTCNTL (program invocation) will be used for the main task, but not for the subtasks. Control statements from JNF1CNTL will be used for subtask1. Control statements from JNF2CNTL will be used for subtask2.
- For the main task, the normal options in effect for a DFSORT run will be used. The run-time options will be those from DFSPARM, SYSIN, SORTCNTL, EXEC PARM or the caller's parameter list, as appropriate. The installation options will be those for direct invocation or program invocation of DFSORT, as appropriate.
- For subtaskn, the options in effect for the subtask will be used with the exceptions noted later in this section. The run-time options will be those from JNFnCNTL. The installation options will be those for a calling program (INV, TSOINV, TD1-TD4).
- TYPE=F processing is used for a VSAM F1 or F2 file by default. TYPE=V can be specified in JNFnCNTL, or on the JOINKEYS statement for Fn, to override the default of TYPE=F.
- For subtaskn, if an INCLUDE or OMIT statement or a STOPAFT or TYPE option is specified in JNFnCNTL, it will override the corresponding option in a JOINKEYS statement for Fn. For ease of use, it is recommended that you use an INCLUDE or OMIT statement, and a STOPAFT or TYPE option, in JNFnCNTL rather than an INCLUDE, OMIT, STOPAFT or TYPE operand in a JOINKEYS statement.
- For the main task, override of statements and options is done in the normal way, that is, DFSPARM overrides SYSIN for direct invocation of DFSORT, and DFSPARM overrides SORTCNTL overrides the caller's parameter list for program invocation of DFSORT. See Chapter 3, “Using DFSORT program control statements,” on page 77 for details.

- The following options will be used for subtaskn and cannot be overridden: LIST, MSGPRT=ALL, NOABEND, ESTAE, NOCHECK, EQUALS and RESINV=0. DYNALLOC will be used automatically for subtaskn, but can be overridden by a DYNALLOC option in JNFnCNTL.
- If an INCLUDE or OMIT statement is specified in JNFnCNTL, DFSORT will use the following options in effect for subtaskn processing: ALTSEQ, CHALT or NOCHALT, SZERO or NOSZERO and VLSCMP or NOVLSCMP. These options can be specified in JNFnCNTL. The subtaskn LOCALE installation option in effect will also be used if appropriate.
- If an INCLUDE or OMIT operand is specified on the JOINKEYS statement for Fn, DFSORT will use these main task options in effect for subtaskn processing: ALTSEQ, CHALT or NOCHALT, SZERO or NOSZERO and VLSCMP or NOVLSCMP. If these options are specified in JNFnCNTL, they will be ignored for subtaskn processing. LOCALE will not be used.
- For subtaskn, the following statements are not allowed in JNFnCNTL: JOINKEYS, JOIN, MERGE, OUTFIL, OUTREC, REFORMAT and SORT.
- For the main task, a MERGE statement other than MERGE FIELDS=COPY is not allowed in any source.
- If appropriate, options such as FILSZ, DYNALLOC, AVGRLEN, and so on can be specified in JNFnCNTL for subtaskn, or in DFSPARM for the main task.
- If tape data sets are used for the F1 and F2 files, they must be on different drives so DFSORT can read them in parallel.

JOINKEYS application examples

Example 1 - Paired F1/F2 records without duplicates

```
//JKE1  EXEC  PGM=SORT
//SYSOUT    DD  SYSOUT=*
//SORTJNF1 DD *
Roses          03  Red
Daisies        06  Orange
Roses          04  Pink
Daisies        02  Yellow
Roses          06  Yellow
Daisies        12  Lilac
Roses          09  Blue
/*
//SORTJNF2 DD *
Red      Lilies          InStock
Red      Roses           InStock
Orange   Daisies         SoldOut
Pink     Roses           SoldOut
Yellow   Daisies         InStock
Yellow   Roses           Ordered
Lilac    Daisies         SoldOut
White    Daisies         InStock
/*
//SORTOUT DD SYSOUT=*
//SYSIN    DD    *
* Control statements for JOINKEYS application
  JOINKEYS FILE=F1,FIELDS=(1,15,A,20,8,A)
  JOINKEYS FILE=F2,FIELDS=(10,15,A,1,8,A)
  REFORMAT FIELDS=(F1:20,8,1,15,F2:26,10,F1:16,2)
* Control statements for main task (joined records)
  OPTION COPY
  OUTFIL REMOVECC,
    HEADER2=(1:'Color',11:'Flower',26:'Status',36:'Per Pot',/,
             1:7'-',11:14'-',26:7'-',36:7'-'),
    BUILD=(1:1,8,11:9,15,26:24,10,
      36:34,2,ZD,M10,LENGTH=7)
/*
```

This example illustrates how you can join paired records from two files using multiple keys. In this case, neither file has duplicates. The paired records are the records in F1 and F2 with matching keys (for example, key1=Roses and key2=Red).

Input file1 (F1) has RECFM=FB and LRECL=80. It contains the records shown for SORTJNF1 in the JCL shown previously in this section. Input file2 (F2) has RECFM=FB and LRECL=80. It contains the records shown for SORTJNF2 in the JCL shown previously in this section.

The output file will have RECFM=FB and LRECL=42. It will contain the paired records from the two files reformatted as follows:

```
Color     Flower         Status    Per Pot
-------   ------------- -------   -------
Lilac     Daisies        SoldOut        12
Orange    Daisies        SoldOut         6
Yellow    Daisies        InStock         2
Pink      Roses          SoldOut         4
Red       Roses          InStock         3
Yellow    Roses          Ordered         6
```

The first JOINKEYS statement defines the ddname and keys for the F1 file. FILE=F1 tells DFSORT that the ddname for the F1 file is SORTJNF1. FIELDS=(1,15,A,20,8,A) tells DFSORT that the first binary key is in positions 1-15 ascending and the second binary key is in positions 20-27 ascending. Since SORTED is not specified, DFSORT will sort the SORTJNF1 records by the specified binary keys.

The second JOINKEYS statement defines the ddname and keys for the F2 file. FILE=F2 tells DFSORT that the ddname for the F2 file is SORTJNF2. FIELDS=(10,15,A,1,8,A) tells DFSORT that the first binary key is in positions 10-24 ascending and the second binary key is in positions 1-8 ascending. Since SORTED is not specified, DFSORT will sort the SORTJNF2 records by the specified binary keys.

Note that corresponding keys for the two files match in length and order.

The REFORMAT statement defines the fields to be extracted for the joined records in the order in which they are to appear. FIELDS=(F1:20,8,1,15,F2:26,10,F1:16,2) tells DFSORT to create the joined records as follows:

```
Joined Record Positions     Extracted from
-----------------------     ------------------
1-8                         F1 positions 20-27
9-23                        F1 positions 1-15
24-33                       F2 positions 26-35
34-35                       F1 positions 16-17
```

Since there is no JOIN statement, only paired records are joined by default.

The OPTION COPY statement tells DFSORT to copy the joined records. The OUTFIL statement tells DFSORT to reformat the joined records, display a header at the top of each page and remove the carriage control characters. Note that the BUILD operand of the OUTFIL statement must reference the positions of fields in the joined records.

Conceptually, JOINKEYS application processing proceeds as follows:

- Subtask1 sorts the SORTJNF1 (F1 file) records as directed by its JOINKEYS statement. As a result, it passes the following records to the main task:

  ```
  Daisies        12  Lilac
  Daisies        06  Orange
  Daisies        02  Yellow
  Roses          09  Blue
  Roses          04  Pink
  Roses          03  Red
  Roses          06  Yellow
  ```

- Subtask2 sorts the SORTJNF2 (F2 file) records as directed by its JOINKEYS statement. As a result, it passes the following records to the main task:

  ```
  Lilac    Daisies         SoldOut
  Orange   Daisies         SoldOut
  White    Daisies         InStock
  Yellow   Daisies         InStock
  Red      Lilies          InStock
  Pink     Roses           SoldOut
  Red      Roses           InStock
  Yellow   Roses           Ordered
  ```

- The main task joins the records passed from subtask1 and subtask2 as directed by the specified JOINKEYS and REFORMAT statements, resulting in the following joined records:

```
Lilac   Daisies        SoldOut   12
Orange  Daisies        SoldOut   06
Yellow  Daisies        InStock   02
Pink    Roses          SoldOut   04
Red     Roses          InStock   03
Yellow  Roses          Ordered   06
```

- Finally, the main task copies and reformats the joined records according to the OUTFIL statement, and writes the resulting records to SORTOUT. Thus, SORTOUT contains these records:

```
Color     Flower         Status    Per Pot
-------   -------------- -------   -------
Lilac     Daisies        SoldOut        12
Orange    Daisies        SoldOut         6
Yellow    Daisies        InStock         2
Pink      Roses          SoldOut         4
Red       Roses          InStock         3
Yellow    Roses          Ordered         6
```

Example 2 - Paired F1/F2 records with duplicates (cartesian)

```
//JKE2  EXEC  PGM=SORT
//SYSOUT    DD  SYSOUT=*
//VBIN DD DSN=MY.VBFILE,DISP=SHR
//FBIN DD DSN=MY.FBFILE,DISP=SHR
//SORTOUT DD DSN=MY.FB.OUTPUT,DISP=(NEW,CATLG,DELETE),
// SPACE=(CYL,(5,5)),UNIT=SYSDA
//SYSIN    DD    *
* Control statements for JOINKEYS application
  JOINKEYS F1=VBIN,FIELDS=(18,16,A),SORTED
  JOINKEYS F2=FBIN,FIELDS=(1,16,A)
  REFORMAT FIELDS=(F2:22,12,F1:5,12,F2:1,16)
* Control statements for main task (joined records)
  OPTION EQUALS
  SORT FIELDS=(13,12,CH,A)
/*
```

This example illustrates how you can join paired records from two files, both of which have duplicate records. The result will be a cartesian join. The paired records are the records in F1 and F2 with matching keys (for example, key=Cats).

Input file1 has RECFM=VB and LRECL=50. It contains the following records:

```
Len|Data
 40|Eliot        Cats            Musical
 40|Lloyd-Webber Cats            Musical
 48|Hart         Pal Joey        Musical, Comedy
 48|Rodgers      Pal Joey        Musical, Comedy
 47|Hammerstein  South Pacific   Musical, Drama
 47|Rodgers      South Pacific   Musical, Drama
```

The output file will have RECFM=FB and LRECL=40. It will contain the paired cartesian product of the two files sorted as follows:

```
Start: 1982 Eliot       Cats
End:   2000 Eliot       Cats
Start: 1949 Hammerstein South Pacific
End:   1954 Hammerstein South Pacific
Start: 1940 Hart        Pal Joey
End:   1941 Hart        Pal Joey
Start: 1982 Lloyd-WebberCats
End:   2000 Lloyd-WebberCats
Start: 1940 Rodgers     Pal Joey
End:   1941 Rodgers     Pal Joey
Start: 1949 Rodgers     South Pacific
End:   1954 Rodgers     South Pacific
```

The first JOINKEYS statement defines the ddname and key for the F1 file. F1=VBIN tells DFSORT that the ddname for the F1 file is VBIN. FIELDS=(18,16,A) tells DFSORT that the key is in positions 18-33

ascending. Note that since VBIN is a VB file, the starting position of its key must take the RDW in positions 1-4 into account. Since SORTED is specified, indicating that the records are already in order by the specified binary key, DFSORT will copy the VBIN records.

The second JOINKEYS statement defines the ddname and binary key for the F2 file. F2=FBIN tells DFSORT that the ddname for the F2 file is FBIN. FIELDS=(1,16,A) tells DFSORT that the binary key is in positions 1-16 ascending. Since SORTED is not specified, DFSORT will sort the FBIN records by the specified binary key.

The REFORMAT statement defines the fields to be extracted for the joined records in the order in which they are to appear. FIELDS=(F2:22,12,F1:5,12,F2:1,16) tells DFSORT to create the joined records as follows:

```
Joined Record Positions     Extracted from
-----------------------     ------------------
1-12                        F2 positions 22-33
13-24                       F1 positions 5-16
25-40                       F2 positions 1-16
```

Note that since VBIN (F1) is a VB file, the starting position of its REFORMAT field must take the RDW in positions 1-4 into account.

Since there is no JOIN statement, only paired records are joined by default. Since there are duplicates in each input file, a cartesian join is performed.

The SORT FIELDS=(13,12,CH,A) statement tells DFSORT to sort the joined records by a different key than the one used for the join of F1 and F2 records. Note that the FIELDS operand of the SORT statement must reference the positions of fields in the joined records.

Conceptually, JOINKEYS application processing proceeds as follows:

- Subtask1 copies the VBIN (F1 file) records as directed by its JOINKEYS statement. As a result, it passes the following records to the main task:

  ```
  Len|Data
   40|Eliot        Cats            Musical
   40|Lloyd-Webber Cats            Musical
   48|Hart         Pal Joey        Musical, Comedy
   48|Rodgers      Pal Joey        Musical, Comedy
   47|Hammerstein  South Pacific   Musical, Drama
   47|Rodgers      South Pacific   Musical, Drama
  ```

- Subtask2 sorts the FBIN (F2 file) records as directed by its JOINKEYS statement. As a result, it passes the following records to the main task:

  ```
  Cats                Start: 1982  50
  Cats                End:   2000
  Pal Joey            Start: 1940  22
  Pal Joey            End:   1941
  South Pacific       Start: 1949  13
  South Pacific       End:   1954
  ```

- The main task joins the records passed from subtask1 and subtask2 as directed by the specified JOINKEYS and REFORMAT statements, resulting in the following joined records:

  ```
  Start: 1982 Eliot       Cats
  End:   2000 Eliot       Cats
  Start: 1982 Lloyd-WebberCats
  End:   2000 Lloyd-WebberCats
  Start: 1940 Hart        Pal Joey
  End:   1941 Hart        Pal Joey
  Start: 1940 Rodgers     Pal Joey
  End:   1941 Rodgers     Pal Joey
  Start: 1949 Hammerstein South Pacific
  End:   1954 Hammerstein South Pacific
  Start: 1949 Rodgers     South Pacific
  End:   1954 Rodgers     South Pacific
  ```

- Finally, the main task sorts the joined records according to the SORT statement, and writes the resulting records to SORTOUT. Thus, SORTOUT contains these records:

```
Start: 1982 Eliot       Cats
End:   2000 Eliot       Cats
Start: 1949 Hammerstein South Pacific
End:   1954 Hammerstein South Pacific
Start: 1940 Hart        Pal Joey
End:   1941 Hart        Pal Joey
Start: 1982 Lloyd-WebberCats
End:   2000 Lloyd-WebberCats
Start: 1940 Rodgers     Pal Joey
End:   1941 Rodgers     Pal Joey
Start: 1949 Rodgers     South Pacific
End:   1954 Rodgers     South Pacific
```

Example 3 - Paired F1 records

```
//JKE3  EXEC  PGM=SORT
//SYSOUT    DD  SYSOUT=*
//MASTER DD DSN=MASTER.FILE,DISP=SHR
//PULL DD DSN=PULL.FILE,DISP=SHR
//SORTOUT DD SYSOUT=*
//JNF1CNTL DD *
* Control statement for subtask1 (F1)
  INREC PARSE=(%01=(ENDBEFR=C',',FIXLEN=15),
               %=(ENDBEFR=C','),
               %02=(ENDBEFR=C',',FIXLEN=10)),
  OVERLAY=(36:%01,52:%02)
/*
//DFSPARM   DD    *
* Control statements for JOINKEYS application
  JOINKEYS F1=MASTER,FIELDS=(36,15,A,52,10,A)
  JOINKEYS F2=PULL,FIELDS=(12,15,A,1,10,A)
  REFORMAT FIELDS=(F1:1,35)
* Control statement for main task
  OPTION COPY
/*
```

This example illustrates how you can select only paired records from one of two files. In this case, we will select the F1 records that have a match in F2 on the specified keys (for example, key1=Development and key2=Master). The F1 records are in comma delimited form so we will parse them to get the binary keys we need.

Input file1 (F1) has RECFM=FB and LRECL=35. It contains the following records:

```
Marketing,William,Master
Development,John,Bachelor
Manufacturing,Louis,Master
Development,Carol,Master
Research,Angela,Master
Research,Anne,Doctorate
Development,Sara,Doctorate
Marketing,Joseph,Master
Manufacturing,Donna,Bachelor
Development,Susan,Master
Manufacturing,Nick,Master
Research,Howard,Doctorate
```

Input file2 (F2) has RECFM=FB and LRECL=30. It contains the following records:

```
Master     Development
Master     Manufacturing
Bachelor   Development
Doctorate  Research
```

The output file will have RECFM=FB and LRECL=35. It will contain the paired F1 records, that is, the records from F1 that have a match in F2 for the specified keys (the first and third fields):

```
Development,John,Bachelor
Development,Carol,Master
Development,Susan,Master
Manufacturing,Louis,Master
Manufacturing,Nick,Master
Research,Anne,Doctorate
Research,Howard,Doctorate
```

The first JOINKEYS statement defines the ddname and keys for the F1 file. F1=MASTER tells DFSORT that the ddname for the F1 file is MASTER.

The control statements in JNF1CNTL will be used to process the F1 file before it is sorted. The input records are in comma delimited form, so to use the first and third fields as binary keys, while preserving the original data, we use an INREC statement to parse out the fields we want and add them to the end of the record. The parsed first key will be in positions 36-50 and the parsed second key will be in positions 52-61. For example, the first reformatted record will look like this:

```
Marketing,William,Master           Marketing       Master
```

FIELDS=(36,15,A,52,10,A) in the JOINKEYS statement (referring to the reformatted INREC positions) tells DFSORT that the first key is in positions 36-50 ascending and the second key is in positions 52-61 ascending.

The second JOINKEYS statement defines the ddname and keys for the F2 file. F2=PULL tells DFSORT that the ddname for the F2 file is PULL.

FIELDS=(12,15,A,1,10,A) in the JOINKEYS statement tells DFSORT that the first key is in positions 12-26 ascending and the second key is in positions 1-10 ascending.

The REFORMAT statement defines the fields to be extracted for the joined records in the order in which they are to appear. FIELDS=(F1:1,35) tells DFSORT to create the joined records from the original F1 input records (positions 1-35 of the F1 records). Since we only needed the added parsed fields for sorting, we don't need to include them in the joined records. Of course, if we wanted the main task to "see" the parsed fields in the joined records, we could include the parsed fields in the REFORMAT FIELDS operand.

Since there is no JOIN statement, only paired records are joined by default.

The OPTION COPY statement tells DFSORT to copy the joined records.

Conceptually, JOINKEYS application processing proceeds as follows:

- Subtask1 performs INREC processing for the MASTER (F1 file) records as directed by the control statement in JNF1CNTL and sorts the resulting records as directed by its JOINKEYS statement. As a result, it passes the following records to the main task:

  ```
  Development,John,Bachelor          Development     Bachelor
  Development,Sara,Doctorate         Development     Doctorate
  Development,Carol,Master           Development     Master
  Development,Susan,Master           Development     Master
  Manufacturing,Donna,Bachelor       Manufacturing   Bachelor
  Manufacturing,Louis,Master         Manufacturing   Master
  Manufacturing,Nick,Master          Manufacturing   Master
  Marketing,William,Master           Marketing       Master
  Marketing,Joseph,Master            Marketing       Master
  Research,Howard,Doctorate          Research        Doctorate
  Research,Anne,Doctorate            Research        Doctorate
  Research,Angela,Master             Research        Master
  ```

- Subtask2 sorts the PULL (F2 file) records as directed by its JOINKEYS statement. As a result, it passes the following records to the main task:

  ```
  Bachelor   Development
  Master     Development
  Master     Manufacturing
  Doctorate  Research
  ```

- The main task joins the records passed from subtask1 and subtask2 as directed by the specified JOINKEYS and REFORMAT statements, resulting in the following joined records:

  ```
  Development,John,Bachelor
  Development,Carol,Master
  Development,Susan,Master
  Manufacturing,Louis,Master
  Manufacturing,Nick,Master
  Research,Anne,Doctorate
  Research,Howard,Doctorate
  ```

- Finally, the main task copies the joined records to SORTOUT. Thus, SORTOUT contains these records:

```
Development,John,Bachelor
Development,Carol,Master
Development,Susan,Master
Manufacturing,Louis,Master
Manufacturing,Nick,Master
Research,Anne,Doctorate
Research,Howard,Doctorate
```

Example 4 - Unpaired F2 records

```
//JKE4  EXEC  PGM=SORT
//SYSOUT    DD  SYSOUT=*
//IN1 DD DSN=FILE1.IN,DISP=SHR
//IN2 DD DSN=FILE2.IN,DISP=SHR
//SORTOUT DD DSN=FILE3.OUT,DISP=OLD
//JNF1CNTL DD *
* Control statements for subtask1 (F1)
  OMIT COND=(10,5,UFF,EQ,99999)
  INREC BUILD=(1,8,9:10,5,UFF,TO=ZD,LENGTH=5)
/*
//JNF2CNTL DD *
* Control statements for subtask2 (F2)
  OMIT COND=(14,5,UFF,EQ,99999)
  INREC BUILD=(1,4,5:14,5,UFF,TO=ZD,LENGTH=5,10:5)
/*
//SYSIN    DD    *
* Control statements for JOINKEYS application
  JOINKEYS F1=IN1,FIELDS=(1,8,A,9,5,D)
  JOINKEYS F2=IN2,FIELDS=(10,8,A,5,5,D)
  JOIN UNPAIRED,F2,ONLY
  REFORMAT FIELDS=(F2:1,4,10)
* Control statement for main task
  OPTION COPY
/*
```

This example illustrates how you can select only unpaired records from one of two files. In this case, we will select the F2 records that do not have a match in F1 on the specified keys (for example, key1=Molly and key2=2100). We will also omit certain records from each input file and handle unnormalized keys.

Input file1 has RECFM=FB and LRECL=15. It contains the following records:

```
Molly    145
Molly    99999
Molly    2143
Jasmine  1292
Jasmine  5
Jasmine  28
Jasmine  99999
```

Input file2 has RECFM=VB and LRECL=35. It contains the following records:

```
Len|Data
 30|Molly    145      Thursday
 31|Molly    2100     Wednesday
 28|Molly    18       Sunday
 28|Jasmine  99999    Monday
 28|Jasmine  5        Sunday
 30|Jasmine  28       Saturday
 29|Jasmine  103      Tuesday
 31|Jasmine  99999    Wednesday
```

The output file will have RECFM=VB and LRECL=35. F1 records with 99999 in positions 10-14 and F2 records with 99999 in positions 14-18 will be removed before join processing. The output file will contain unpaired F2 records (that is, records from F2 that do not have a match in F1 for the specified keys) as follows:

```
Len|Data
 29|Jasmine  103      Tuesday
 31|Molly    2100     Wednesday
 28|Molly    18       Sunday
```

The first JOINKEYS statement defines the ddname and keys for the F1 file. F1=IN1 tells DFSORT that the ddname for the F1 file is IN1.

The control statements in JNF1CNTL will be used to process the F1 file before it is sorted. The OMIT statement removes records that have 99999 in positions 10-14. We want to use the numeric field as our key. However, the numeric field is unnormalized since it is left aligned instead of right aligned, so sorting it as a binary key will not work. To handle this, we use the INREC statement to reformat the numeric field as ZD values in positions 9-13 (positive ZD values with the same sign can be sorted as binary). For example, the first reformatted FB record will look like this:

```
Molly   00145
```

Since we don't need the F1 records for output, we don't need to keep the original left aligned numeric value.

FIELDS=(1,8,A,9,5,D) in the JOINKEYS statement (referring to the reformatted INREC positions) tells DFSORT that the first key is in positions 1-8 ascending and the second key is in positions 9-13 descending.

The second JOINKEYS statement defines the ddname and keys for the F2 file. F2=IN2 tells DFSORT that the ddname for the F2 file is IN2.

The control statements in JNF2CNTL will be used to process the F2 file before it is sorted. The OMIT statement removes records that have 99999 in positions 14-18. Again, we need a ZD version of the left aligned numeric value to use for the binary key. But in this case, since we want the original F2 records for output, we need to keep the original numeric value as well. Using the INREC statement, we add the ZD value at positions 5-9 between the RDW and the first data field. That shifts the original data to start at position 10. For example, the first reformatted VB record will look like this:

```
Len|Data
 35|00145Molly    145      Thursday
```

In this case, since the input is a VB file, we specify the RDW (1,4), then the converted field, and then the rest of the record (5 without a length) in the INREC statement.

FIELDS=(10,8,A,5,5,D) in the JOINKEYS statement (referring to the reformatted INREC positions) tells DFSORT that the first key is in positions 10-17 ascending and the second key is in positions 5-9 descending.

Note that since IN2 is a VB file, all of its starting positions must take the RDW in positions 1-4 into account.

The JOIN statement tells DFSORT that the joined records should be the F2 records that do not have a match in F1 on the specified keys.

The REFORMAT statement defines the fields to be extracted for the joined records in the order in which they are to appear. We need the RDW (1,4) and the original data which starts in position 10 of the reformatted F2 records. So we use FIELDS=(F2:1,4,10). Since the last field (10) is a position without a length, it tells DFSORT to create VB records. The joined records are created as follows from the reformatted F2 records:

```
Joined Record Positions     Extracted from
-----------------------     --------------------------------
1-4                         RDW (not extracted)
5 to end                    Reformatted F2 position 10 to end
```

Conceptually, JOINKEYS application processing proceeds as follows:

- Subtask1 performs OMIT and INREC processing for the IN1 (F1 file) records as directed by the control statements in JNF1CNTL and sorts the resulting records as directed by its JOINKEYS statement. As a result, it passes the following records to the main task:

  ```
  Jasmine 01292
  Jasmine 00028
  Jasmine 00005
  ```

```
Molly   02143
Molly   00145
```

- Subtask2 performs OMIT and INREC processing for the IN2 (F2 file) records as directed by the control statements in JNF2CNTL and sorts the resulting records as directed by its JOINKEYS statement. As a result, it passes the following records to the main task:

```
Len|Data
 34|00103Jasmine  103      Tuesday
 35|00028Jasmine  28       Saturday
 33|00005Jasmine  5        Sunday
 36|02100Molly    2100     Wednesday
 35|00145Molly    145      Thursday
 33|00018Molly    18       Sunday
```

- The main task joins the records passed from subtask1 and subtask2 as directed by the specified JOINKEYS, JOIN and REFORMAT statements, resulting in the following joined records (unmatched F2 records):

```
Len|Data
 29|Jasmine  103      Tuesday
 31|Molly    2100     Wednesday
 28|Molly    18       Sunday
```

- Finally, the main task copies the joined records, and writes them to SORTOUT. Thus, SORTOUT contains these records:

```
Len|Data
 29|Jasmine  103      Tuesday
 31|Molly    2100     Wednesday
 28|Molly    18       Sunday
```

Example 5 - Paired and unpaired F1/F2 records (indicator method)

```
//JKE5  EXEC  PGM=SORT
//SYSOUT    DD  SYSOUT=*
//SORTJNF1 DD DSN=FIRST.FILE,DISP=SHR
//SORTJNF2 DD DSN=SECOND.FILE,DISP=SHR
//F1ONLY DD SYSOUT=*
//F2ONLY DD SYSOUT=*
//BOTH DD SYSOUT=*
//SYSIN    DD    *
* Control statements for JOINKEYS application
  JOINKEYS FILE=F1,FIELDS=(1,10,A),SORTED,NOSEQCK
  JOINKEYS FILE=F2,FIELDS=(7,10,A),SORTED,NOSEQCK
  JOIN UNPAIRED,F1,F2
  REFORMAT FIELDS=(F1:1,14,F2:1,20,?)
* Control statements for main task (joined records)
  OPTION COPY
  OUTFIL FNAMES=F1ONLY,INCLUDE=(35,1,CH,EQ,C'1'),
    BUILD=(1,14)
  OUTFIL FNAMES=F2ONLY,INCLUDE=(35,1,CH,EQ,C'2'),
    BUILD=(15,20)
  OUTFIL FNAMES=BOTH,INCLUDE=(35,1,CH,EQ,C'B'),
    BUILD=(1,14,/,15,20)
/*
```

This example illustrates how you can create three output files; with paired F1/F2 records, unpaired F1 records and unpaired F2 records. In this case:

- F1 records which do not have a match in F2 on the specified keys (for example, key=David) will be written to the F1ONLY output file.
- F2 records which do not have a match in F1 on the specified keys (for example, key=Karen) will be written to the F2ONLY output file.
- F1 and F2 records which have a match in F1 and F2 on the specified keys (for example, key=Carrie) will be written to the BOTH output file.

Input file1 has RECFM=FB and LRECL=14. It contains the following records:

```
Carrie    F101
David     F102
Frank     F103
Holly     F104
Vicky     F105
```

Input file2 has RECFM=FB and LRECL=20. It contains the following records:

```
No    Carrie    F201
Yes   Holly     F202
Yes   Karen     F203
No    Sri Hari  F204
Yes   Vicky     F205
```

The F1ONLY file will have RECFM=FB and LRECL=14. It will contain the unpaired F1 records as follows:

```
David     F102
Frank     F103
```

The F2ONLY file will have RECFM=FB and LRECL=20. It will contain the unpaired F2 records as follows:

```
Yes   Karen     F203
No    Sri Hari  F204
```

The BOTH file will have RECFM=FB and LRECL=20. It will contain the paired F1 and F2 records as follows:

```
Carrie    F101
No    Carrie    F201
Holly     F104
Yes   Holly     F202
Vicky     F105
Yes   Vicky     F205
```

The first JOINKEYS statement defines the ddname and key for the F1 file. FILE=F1 tells DFSORT that the ddname for the F1 file is SORTJNF1. FIELDS=(1,10,A) tells DFSORT that the key is in positions 1-10 ascending. Since SORTED is specified, indicating that the records are already in order by the specified binary key, DFSORT will copy the SORTJNF1 records. Since NOSEQCK is specified, DFSORT will not check that the records are in order by the key. (Only use NOSEQCK if you know for sure that the records are in order by the key.)

The second JOINKEYS statement defines the ddname and key for the F2 file. FILE=F2 tells DFSORT that the ddname for the F2 file is SORTJNF2. FIELDS=(7,10,A) tells DFSORT that the key is in positions 7-16 ascending. Since SORTED is specified, indicating that the records are already in order by the specified binary key, DFSORT will copy the SORTJNF2 records. Since NOSEQCK is specified, DFSORT will not check that the records are in order by the key. (Only use NOSEQCK if you know for sure that the records are in order by the key.)

The JOIN statement tells DFSORT that the joined records should include the unpaired F1 and F2 records as well as the paired F1/F2 records.

The REFORMAT statement defines the fields to be extracted for the joined records in the order in which they are to appear, and includes an indicator in the last position that will be set to '1' if the key is found only in the F1 file, '2' if the key is found only in the F2 file, or 'B' if the key is found in the F1 file and in the F2 file. FIELDS=(F1:1,14,F2:1,20,?) tells DFSORT to create the joined records as follows:

```
Joined Record Positions   Extracted from
-----------------------   ----------------
1-14                      F1 positions 1-14
15-34                     F2 positions 1-20
35                        Indicator of where key was found
```

The OPTION COPY statement tells DFSORT to copy the joined records. The OUTFIL statements use the indicator in position 35 to determine where to find the F1 or F2 fields in the joined records and where to write the fields (F1ONLY, F2ONLY or BOTH).

Conceptually, JOINKEYS application processing proceeds as follows:

- Subtask1 copies the SORTJNF1 (F1) records as directed by the JOINKEYS statement. As a result, it copies the unchanged SORTJNF1 records to the main task.
- Subtask2 copies the SORTJNF2 (F2) records as directed by the JOINKEYS statement. As a result, it copies the unchanged SORTJNF2 records to the main task.
- The main task joins the records passed from subtask1 and subtask2 as directed by the specified JOINKEYS, JOIN and REFORMAT statements, resulting in the following joined records (paired and unpaired):

```
Carrie    F101No    Carrie    F201B
David     F102                  1
Frank     F103                  1
Holly     F104Yes   Holly     F202B
              Yes   Karen     F2032
              No    Sri Hari  F2042
Vicky     F105Yes   Vicky     F205B
```

  For F1 records without a match in F2 (for example, the F102 record), the indicator in position 35 has a '1'. For F2 records without a match in F1 (for example, the F203 record), the indicator in position 35 has a '2'. For F1 records with a match in F2 (for example, the F101 and F201 records), the indicator in position 35 has a 'B'.
- The first OUTFIL statement finds records with a '1' in position 35. These are the F1 records without a match in F2. The F1 field is in positions 1-14 of the joined record, so those positions are written to the F1ONLY file. Thus, F1ONLY contains these records:

```
David     F102
Frank     F103
```

- The second OUTFIL statement finds records with a '2' in position 35. These are the F2 records without a match in F1. The F2 field is in positions 15-34 of the joined record, so those positions are written to the F2ONLY file. Thus, F2ONLY contains these records:

```
Yes   Karen     F203
No    Sri Hari  F204
```

- The third OUTFIL statement finds records with 'B' in position 35. These are the F1 and F2 records with a match. The F1 field is in positions 1-14 of the joined record and the F2 field is in positions 15-34 of the joined record, so each joined record is split into those two records and written to the BOTH file. The shorter F1 record is padded with blanks on the right to the length of the F2 record. Thus, BOTH contains these records:

```
Carrie    F101
No    Carrie    F201
Holly     F104
Yes   Holly     F202
Vicky     F105
Yes   Vicky     F205
```

Note: If you only want one record per key in BOTH, you can have the BUILD for FNAMES=BOTH specify the positions for just that record. For example, BUILD=(1,14) for the F1 records or BUILD=(15,20) for the F2 records.

Example 6 - Paired and unpaired F1/F2 records (FILL method)

```
//JKE6  EXEC  PGM=SORT
//SYSOUT    DD  SYSOUT=*
//SORTJNF1 DD DSN=FIRST.FILE,DISP=SHR
//SORTJNF2 DD DSN=SECOND.FILE,DISP=SHR
//F1ONLY DD SYSOUT=*
//F2ONLY DD SYSOUT=*
//BOTH DD SYSOUT=*
//SYSIN    DD    *
* Control statements for JOINKEYS application
  JOINKEYS FILE=F1,FIELDS=(1,10,A),SORTED,NOSEQCK
  JOINKEYS FILE=F2,FIELDS=(7,10,A),SORTED,NOSEQCK
  JOIN UNPAIRED,F1,F2
  REFORMAT FIELDS=(F1:1,14,F2:1,20),FILL=C'$'
```

```
* Control statements for main task (joined records)
  OPTION COPY
  OUTFIL FNAMES=F1ONLY,INCLUDE=(15,1,CH,EQ,C'$'),
    BUILD=(1,14)
  OUTFIL FNAMES=F2ONLY,INCLUDE=(1,1,CH,EQ,C'$'),
    BUILD=(15,20)
  OUTFIL FNAMES=BOTH,INCLUDE=(15,1,CH,NE,C'$',AND,1,1,CH,NE,C'$'),
    BUILD=(1,14,/,15,20)
/*
```

This example illustrates an alternate way to create three output files; with paired F1/F2 records, unpaired F1 records and unpaired F2 records. It produces the same output as Example 5. Whereas Example 5 uses an indicator in one position to determine where the key was found, Example 6 uses a fill character in different positions to determine where the key was found. Another drawback of the Example 6 method is that you must use a FILL character that does not appear in either input record. The Explanation for Example 6 is the same as for Example 5 up to the REFORMAT statement and then it differs as follows:

The REFORMAT statement defines the fields to be extracted for the joined records in the order in which they are to appear. FIELDS=(F1:1,14,F2:1,20) tells DFSORT to create the joined records as follows:

```
Joined Record Positions   Extracted from
-----------------------   ----------------
1-14                      F1 positions 1-14
15-34                     F2 positions 1-20
```

FILL=C'$' tells DFSORT to use $ as the fill character for the F1 field in an unpaired F2 record and for the F2 field in an unpaired F1 record. We use the FILL character to identify the unpaired records from each file; when used for this purpose, the default of FILL=X'40' is usually not a good choice. You must select a FILL character that will not appear in either input file.

The OPTION COPY statement tells DFSORT to copy the joined records. The OUTFIL statements use the presence or absence of the $ fill character in certain positions to determine where to find the F1 or F2 fields in the joined records and where to write the fields (F1ONLY, F2ONLY or BOTH).

Conceptually, JOINKEYS application processing proceeds as follows:

- Subtask1 copies the SORTJNF1 (F1) records as directed by the JOINKEYS statement. As a result, it copies the unchanged SORTJNF1 records to the main task.
- Subtask2 copies the SORTJNF2 (F2) records as directed by the JOINKEYS statement. As a result, it copies the unchanged SORTJNF2 records to the main task.
- The main task joins the records passed from subtask1 and subtask2 as directed by the specified JOINKEYS, JOIN and REFORMAT statements, resulting in the following joined records (paired and unpaired):

  ```
  Carrie    F101No    Carrie    F201
  David     F102$$$$$$$$$$$$$$$$$$$$
  Frank     F103$$$$$$$$$$$$$$$$$$$$
  Holly     F104Yes   Holly     F202
  $$$$$$$$$$$$$$$Yes   Karen     F203
  $$$$$$$$$$$$$$$No    Sri Hari  F204
  Vicky     F105Yes   Vicky     F205
  ```

  For F1 records without a match in F2 (for example, the F102 record), the F2 field is filled with the FILL character. For F2 records without a match in F1 (for example, the F203 record), the F1 field is filled with the FILL character. For F1 records with a match in F2 (for example, the F101 and F201 records), no FILL characters are used.
- The first OUTFIL statement finds records with the FILL character in position 15. These are the F1 records without a match in F2. The F1 field is in positions 1-14 of the joined record, so those positions are written to the F1ONLY file. Thus, F1ONLY contains these records:

  ```
  David     F102
  Frank     F103
  ```

- The second OUTFIL statement finds records with the FILL character in position 1. These are the F2 records without a match in F1. The F2 field is in positions 15-34 of the joined record, so those positions are written to the F2ONLY file. Thus, F2ONLY contains these records:

```
Yes   Karen     F203
No    Sri Hari  F204
```

- The third OUTFIL statement finds records without the FILL character in position 1 or position 15. These are the F1 and F2 records with a match. The F1 field is in positions 1-14 of the joined record and the F2 field is in positions 15-34 of the joined record, so each joined record is split into those two records and written to the BOTH file. The shorter F1 record is padded with blanks on the right to the length of the F2 record. Thus, BOTH contains these records:

```
Carrie    F101
No    Carrie    F201
Holly     F104
Yes   Holly     F202
Vicky     F105
Yes   Vicky     F205
```

Note: If you only want one record per key in BOTH, you can have the BUILD for FNAMES=BOTH specify the positions for just that record. For example, BUILD=(1,14) for the F1 records or BUILD=(15,20) for the F2 records.

Chapter 7. Using ICETOOL

ICETOOL

ICETOOL is a multi-purpose DFSORT utility that uses the capabilities of DFSORT to perform multiple operations on one or more data sets in a single job step. These operations include the following:

- Creating multiple copies of sorted, merged, edited, or unedited input data sets
- Creating output data sets containing subsets of input data sets based on various criteria for character and numeric field values, the number of times unique values occur, header records, trailer records or relative record numbers
- Sorting records between headers and trailers
- Creating output data sets containing different field arrangements of input data sets
- Resizing fixed length records by creating a larger record from multiple shorter records, or multiple shorter records from a larger record
- Creating list data sets showing character and numeric fields in a variety of simple, tailored, and sectioned report formats, allowing control of title, date, time, page numbers, carriage control characters, headings, lines per page, field formats, and total, maximum, minimum, average and count values for the columns of numeric data
- Printing messages that give statistical information for selected numeric fields such as minimum, maximum, average, total, count of values, and count of unique values
- Printing messages that identify invalid decimal values
- Printing messages that give record counts
- Setting RC=12, RC=8, RC=4, or RC=0 based on record counts
- Creating an output data set containing an output record with text and the record count
- Creating a list data set showing the DFSORT installation defaults in use
- Creating list data sets showing unique values for selected character and numeric fields and the number of times each occurs, in a variety of simple and tailored report formats
- Creating list and output data sets for records with: duplicate values, non-duplicate values, or values that occur n times, less than n times or more than n times
- Creating output data sets with information spliced together from two or more input records with duplicate values. The information in the input records can originate from different data sets, helping you to perform various file "join" and "match" operations.
- Using three different modes (stop, continue, and scan) to control error checking and actions after error detection for groups of operators.

You can use ICETOOL for SORT, MERGE and COPY operations.

ICETOOL/DFSORT relationship

ICETOOL is a batch front-end utility that uses the capabilities of DFSORT to perform the operations you request.

ICETOOL includes 17 operators that perform sort, merge, copy, statistical, report, selection, and splice operations. Most of the operations performed by ICETOOL require only simple JCL and operator statements. Some ICETOOL operations require or allow you to specify complete DFSORT control statements (such as SORT, MERGE, INCLUDE or OMIT, INREC, and OUTFIL) to take full advantage of DFSORT's capabilities.

ICETOOL automatically calls DFSORT with the particular DFSORT control statements and options required for each operation (such as DYNALLOC for sorting).

© Copyright IBM Corp. 1973, 2026

ICETOOL also produces messages and return codes describing the results of each operation and any errors detected. Although you generally do not need to look at the DFSORT messages produced as a result of an ICETOOL run, they are available in a separate data set if you need them.

ICETOOL can be called directly or from a program. ICETOOL allows operator statements (and comments) to be supplied in a data set or in a parameter list passed by a calling program. For each operator supplied in the parameter list, ICETOOL puts information in the parameter list pertaining to that operation, thus allowing the calling program to use the information derived by ICETOOL.

ICETOOL JCL summary

The JCL statements used with ICETOOL are summarized in the following. See "Job control language for ICETOOL" on page 539 for more detailed information. See also "JCL restrictions" on page 542 and "ICETOOL notes and restrictions" on page 678.

//JOBLIB DD

Defines your program link library if it is not already known to the system.

//STEPLIB DD

Same as //JOBLIB DD

//TOOLMSG DD

Defines the ICETOOL message data set for all operations.

//DFSMSG DD

Defines the DFSORT message data set for all operations.

//SYMNAMES DD

Defines the SYMNAMES data set containing statements to be used for symbol processing.

//SYMNOUT DD

Defines the data set in which SYMNAMES statements and the symbol table are to be listed.

//TOOLIN DD

Contains ICETOOL control statements.

//indd DD
Defines an input data set for a COPY, COUNT, DATASORT, DISPLAY, MERGE, OCCUR, RANGE, RESIZE, SELECT, SORT, SPLICE, STATS, SUBSET, UNIQUE, or VERIFY operation.

//outdd DD
Defines an output data set for a COPY, DATASORT, MERGE, RESIZE, SELECT, SORT, SPLICE, or SUBSET operation.

//savedd DD
Defines an output data set for a SELECT or SUBSET operation.

//listdd DD
Defines a list data set for a DEFAULTS, DISPLAY, or OCCUR operation.

//countdd DD
Defines an output data set for a COUNT operation

//xxxxCNTL DD
Contains DFSORT control statements for a COPY, COUNT, DATASORT, MERGE, RESIZE, SELECT, SORT, SPLICE or SUBSET operation.

ICETOOL operator summary

ICETOOL has 17 operators that are used to perform a variety of functions. The functions of these operators are summarized later in this section. See "ICETOOL statements" on page 542 for more detailed information. Additionally, information pertaining to each operator is provided to calling programs that supply statements to ICETOOL using a parameter list. See "Parameter list interface" on page 674 for details.

COPY
Copies a data set to one or more output data sets.

COUNT
Prints a message containing the count of records in a data set. COUNT can also be used to create an output data set containing text and the count, or to set RC=12, RC=8, RC=4, or RC=0 based on meeting criteria for the number of records in a data set.

DATASORT
Sorts data records between header and trailer records in a data set to an output data set.

DEFAULTS
Prints the DFSORT installation defaults in a separate list data set.

DISPLAY
Prints the values or characters of specified numeric or character fields in a separate list data set. Simple, tailored, or sectioned reports can be produced. Maximums, minimums, totals, averages and counts can be produced.

MERGE
Merges one or more data sets to one or more output data sets.

MODE
Three modes are available that can be set or reset for groups of operators:

- STOP mode (the default) stops subsequent operations if an error is detected
- CONTINUE mode continues with subsequent operations if an error is detected
- SCAN mode allows ICETOOL statement checking without actually performing any operations.

OCCUR
Prints each unique value for specified numeric or character fields and how many times it occurs in a separate list data set. Simple or tailored reports can be produced. The values printed can be limited to those for which the value count meets specified criteria (for example, only duplicate values or only non-duplicate values).

RESIZE
Creates a larger record from multiple shorter records, or creates multiple shorter records from a larger record, that is, resizes fixed length records.

RANGE
Prints a message containing the count of values in a specified range for a specified numeric field in a data set.

SELECT
Selects records from a data set for inclusion in an output data set based on meeting criteria for the number of times specified numeric or character field values occur (for example, only duplicate values or only non-duplicate values). Records that are not selected can be saved in a separate output data set.

SORT
Sorts a data set to one or more output data sets.

SPLICE
Splices together specified fields from records that have the same specified numeric or character field values (that is, duplicate values), but different information. Specified fields from two or more records can be combined to create an output record. The fields to be spliced can originate from records in different data sets, so you can use SPLICE to do various "join" and "match" operations.

STATS
Prints messages containing the minimum, maximum, average, and total for specified numeric fields in a data set.

SUBSET
Selects records from a data set based on keeping or removing header records (the first n records), relative records, or trailer records (the last n records). Records that are not selected can be saved in a separate output data set.

UNIQUE
Prints a message containing the count of unique values for a specified numeric or character field.

VERIFY
Examines specified decimal fields in a data set and prints a message identifying each invalid value found for each field.

Restriction: You can perform a JOINKEYS application with the COPY and SORT operators, but not with the other operators.

Complete ICETOOL examples

"ICETOOL example" on page 790 contains a complete ICETOOL sample job with all required JCL and control statements. The following example shows the JCL and control statements for a simple ICETOOL job.

```
//EXAMP JOB A402,PROGRAMMER
//RUNIT EXEC PGM=ICETOOL,REGION=1024K
//TOOLMSG DD SYSOUT=A
//DFSMSG DD SYSOUT=A
//TOOLIN DD *
* Show installation defaults
  DEFAULTS LIST(SHOWDEF)
* Create three copies of a data set
  COPY FROM(IN1) TO(OUT1,OUT2,OUT3)
* Print a report
  DISPLAY FROM(IN2) LIST(REPORT) DATE TITLE('Monthly Report') PAGE -
    HEADER('Location') ON(1,25,CH) -
    HEADER('Revenue') ON(23,10,FS) -
    HEADER('Profit') ON(45,10,FS) -
    TOTAL('Totals') AVERAGE('Averages') BLANK
* Select all records with duplicate (non-unique) keys
  SELECT FROM(IN2) TO(DUPKEYS) ON(1,25,CH) ALLDUPS -
* Save all records with non-duplicate (unique) keys
  DISCARD (UNQKEYS)
/*
//SHOWDEF DD SYSOUT=A
//IN1 DD DSN=FLY.INPUT1,DISP=SHR
//IN2 DD DSN=FLY.INPUT2,DISP=SHR
//OUT1 DD DSN=FLY.NEW,DISP=OLD
//OUT2 DD DSN=FLY.BU1,DISP=OLD
//OUT3 DD DSN=FLY.BU2,DISP=OLD
//DUPKEYS DD DSN=FLY.DUPS,DISP=OLD
//UNQKEYS DD DSN=FLY.UNQS,DISP=OLD
//REPORT DD SYSOUT=A
```

Figure 31. Simple ICETOOL Job

Using symbols

You can define and use a symbol for any field or constant in the following ICETOOL operators: COUNT, DATASORT, DISPLAY, OCCUR, RANGE, SELECT, SPLICE, STATS, SUBSET, UNIQUE, and VERIFY. You can also use symbols in the DFSORT control statements you specify for an ICETOOL run. This makes it easy to create and reuse collections of symbols (that is, mappings) representing information associated with various record layouts. You can use system symbols (for example, &JOBNAME.) in your symbol constants. See Chapter 8, "Using symbols for fields and constants," on page 681 for complete details.

Using SET and PROC symbols in control statements

For jobs that directly invoke ICETOOL (PGM=ICETOOL), you can construct DFSORT symbols that incorporate JCL PROC or SET symbols as well as text and system symbols. You can then use those constructed DFSORT symbols in ICETOOL and DFSORT control statements in the same way you use regular DFSORT symbols. For complete information, see Chapter 8, “Using symbols for fields and constants,” on page 681.

Invoking ICETOOL

ICETOOL can be invoked in the following three ways:

- Directly (that is, not from a program) using the TOOLIN Interface
- From a program using the TOOLIN Interface
- From a program using the Parameter List Interface.

With the TOOLIN Interface, you supply ICETOOL statements in a data set defined by the TOOLIN DD statement. ICETOOL prints messages in the data set defined by the TOOLMSG DD statement.

With the Parameter List Interface, your program supplies ICETOOL statements in a parameter list. ICETOOL prints messages in the data set defined by the TOOLMSG DD statement and also puts information in the parameter list for use by your program.

Putting ICETOOL to use

By using various combinations of the 17 ICETOOL operators, you can easily create applications that perform many complex tasks. The two small samples that follow show some things you can do with ICETOOL.

Obtaining various statistics

Sample code:

```
MODE STOP
VERIFY FROM(DATA1) ON(22,7,PD)
DISPLAY FROM(DATA1) LIST(SALARIES) -
  TITLE('Employee Salaries') DATE TIME -
  HEADER('Employee Name') HEADER('Salary') -
  ON(1,20,CH)             ON(22,7,PD)      BLANK -
  AVERAGE('Average Salary')
STATS FROM(DATA1) ON(22,7,PD)
RANGE FROM(DATA1) ON(22,7,PD) LOWER(20000)
RANGE FROM(DATA1) ON(22,7,PD) HIGHER(19999) LOWER(40000)
RANGE FROM(DATA1) ON(22,7,PD) HIGHER(40000)
OCCUR FROM(DATA1) LIST(SALARIES) -
  TITLE('Employees Receiving Each Salary') DATE TIME -
  HEADER('Salary') HEADER('Employee Count') -
  ON(22,7,PD)      ON(VALCNT)               BLANK
```

Assume that you specify DD statements with the following ddnames for the indicated data sets:

DATA1
A data set containing the name, salary, department, location and so on, of each of your employees. The name field is in positions 1 through 20 in character format and the salary field is in positions 22 through 28 in packed decimal format.

SALARIES
A SYSOUT data set.

You can use the ICETOOL operators shown in the sample code earlier in this section to do the following:

MODE STOP

If an error is found while processing one of the operators, subsequent operators are not processed (that is, each operator is dependent on the success of the previous operator).

VERIFY

Prints error messages in the TOOLMSG data set identifying any invalid values in the packed decimal salary field.

DISPLAY

Prints a report with each employee's name and salary and the average for all employee salaries in the SALARIES list data set.

STATS

Prints messages in the TOOLMSG data set showing the minimum, maximum, average, and total of the individual salaries.

RANGE

The three RANGE operators print messages in the TOOLMSG data set showing the number of salaries below $20,000, from $20,000 to $39,999, and above $40,000.

OCCUR

Prints a report with each unique salary and the number of employees who receive it in the SALARIES list data set.

Creating multiple versions/combinations of data sets

Table 87. Creating multiple versions/combinations of data sets

Sample code

```
  * GROUP 1
    MODE CONTINUE
    COPY FROM(DATA1) TO(DATA2)
    COPY FROM(MSTR1) TO(MSTR2)
    SELECT FROM(DATA1) TO(SMALLDPT) ON(30,4,CH) LOWER(10)
    UNIQUE FROM(MSTR1) ON(30,4,CH)
  * GROUP 2
    MODE STOP
    COPY FROM(DATA1) TO(TEMP1) USING(NEW1)
    COPY FROM(DATA1) TO(TEMP2) USING(NEW2)
    COPY FROM(DATA1) TO(TEMP3) USING(NEW3)
    SORT FROM(CONCAT) TO(FINALD,FINALP) USING(FINL)
```

Assume that you specify DD statements with the following ddnames for the indicated data sets:

DATA1
A data set containing the name, salary, department, location, and so on, of each of your employees. The department field is in positions 30 through 33 in character format.

MSTR1
Master data set containing only the name and department of each of your employees. The department field is in positions 30 through 33 in character format.

DATA2, MSTR2, and SMALLDPT
Permanent data sets.

NEW1CNTL
A data set containing DFSORT control statements to INCLUDE employees in department X100 and change the records to match the format of MSTR1.

NEW2CNTL
Same as NEW1CNTL but for department X200.

NEW3CNTL
Same as NEW1CNTL but for department X300.

TEMP1, TEMP2, and TEMP3
Temporary data sets.

FINLCNTL
A data set containing a DFSORT control statement to sort by department and employee name.

CONCAT
A concatenation of the TEMP1, TEMP2, TEMP3, and MSTR1 data sets.

FINALD
A permanent data set.

FINALP
A SYSOUT data set.

You can use the ICETOOL operators in Table 87 on page 538 to do the following:

MODE CONTINUE

If an error is found while processing any of the group 1 operators, subsequent group 1 operators are still processed; that is, group 1 operators are not dependent on the success of the previous group 1 operators.

COPY

The two copy operators create backup copies of DATA1 and MSTR1.

SELECT

Creates a permanent output data set containing the name, salary, department, location, and so on, of each employee in departments with less than 10 people.

UNIQUE

Prints a message in the TOOLMSG data set showing the number of unique departments.

MODE STOP

If an error is found while processing one of the group 2 operators, subsequent group 2 operators are not processed; that is, each group 2 operator is dependent on the success of previous group 2 operators.

COPY

The three COPY operators create an output data set for the employees in each department containing only name and department. Note that the ddname requested by the USING(xxxx) operand is xxxxCNTL. For example, USING(NEW1) requests ddname NEW1CNTL.

SORT

Sorts the three output data sets created by the COPY operators along with the master name/department data set and creates permanent and SYSOUT data sets containing the resulting sorted records.

You can combine both of these examples into a single ICETOOL job step.

Job control language for ICETOOL

An overview of the job control language (JCL) statements for ICETOOL is in Table 88 on page 540 followed by discussions of each ICETOOL DD statement and the use of reserved DD statements and ddnames.

Table 88. JCL statements for ICETOOL

JCL statements

```
//EXAMPL JOB ...
//* ICETOOL CAN BE CALLED DIRECTLY OR FROM A PROGRAM
//STEP EXEC PGM=ICETOOL (or PGM=program_name)
//* THE FOLLOWING DD STATEMENTS ARE ALWAYS REQUIRED
//TOOLMSG DD SYSOUT=A (or DSN=...)
//DFSMSG  DD SYSOUT=A
//* THE FOLLOWING DD STATEMENTS ARE USED FOR SYMBOL PROCESSING
//* SYMNAMES DD ...
//* SYMNOUT DD SYSOUT=A (OR DSN=...)
//* THE TOOLIN DD STATEMENT IS ONLY REQUIRED IF THE TOOLIN
//*  INTERFACE IS USED.
//TOOLIN DD *
 ICETOOL statements
/*
//* THE FOLLOWING DD STATEMENTS ARE ONLY REQUIRED IF SPECIFIED
//* IN ICETOOL STATEMENTS.
//indd DD ...
  .
  .
  .
//outdd DD ...
 .
 .
 .
//listdd DD SYSOUT=A (or DSN=...)
 .
 .
 .
 //countdd DD ...
  .
  .
  .
//xxxxCNTL DD *
 DFSORT control statements
/*
 .
 .
 .
```

TOOLMSG DD Statement

Defines the ICETOOL message data set for all operations. ICETOOL messages and statements appear in this data set. ICETOOL uses RECFM=FBA, LRECL=121 and the specified BLKSIZE for the TOOLMSG data set. If the BLKSIZE you specify is not a multiple of 121, ICETOOL uses BLKSIZE=121. If you do not specify the BLKSIZE, ICETOOL selects the block size as directed by the SDBMSG installation option (see *z/OS DFSORT Installation and Customization*).

The TOOLMSG DD statement must be present.

DFSMSG DD Statement

Defines the DFSORT message data set for all operations. The DFSORT messages and control statements from all ICETOOL calls to DFSORT appear in this data set. Refer to the discussion of SYSOUT in "System DD statements" on page 63 for details.

Either a DFSMSG DD statement or an SSMSG DD statement must be present. If both are present, ICETOOL uses DFSMSG as the DFSORT message data set. If a DFSMSG DD is not present, but an SSMSG DD is present, ICETOOL uses SSMSG as the DFSORT message data set.

Note: A SYSOUT data set should be used for the DFSORT message data set (for example, `//DFSMSG DD SYSOUT=*`). If you define the DFSORT message data set as a temporary or permanent data set,

you will only see the DFSORT messages from the last call to DFSORT, unless you allocate a new data set using a disposition of MOD.

//SYMNAMES DD
Defines the SYMNAMES data set containing statements to be used for symbol processing. See Chapter 8, "Using symbols for fields and constants," on page 681 for complete details.

//SYMNOUT DD
Defines the data set in which SYMNAMES statements and the symbol table are to be listed. See Chapter 8, "Using symbols for fields and constants," on page 681 for complete details.

TOOLIN DD statement
Defines the ICETOOL statement data set that must have the following attributes: RECFM=F or RECFM=FB and LRECL=80.

If the TOOLIN Interface is used, the TOOLIN DD statement must be present. If the Parameter List Interface is used, the TOOLIN DD statement is not required and is ignored if present.

indd DD Statement
Defines an input data set for an operation. Refer to "SORTIN DD statement" on page 66 (copy or sort) or "SORTINnn DD statement" on page 68 (merge) for details. ICETOOL imposes the additional restriction that the LRECL of this data must be at least 4.

An indd DD statement must be present for each unique indd name specified in each FROM operand.

outdd DD Statement
Defines an output data set for a COPY, DATASORT, MERGE, RESIZE, SELECT, SORT, SPLICE or SUBSET operation. Refer to "SORTOUT and OUTFIL DD statements" on page 71 for details.

An outdd DD statement must be present for each unique outdd name specified in each TO operand.

savedd DD Statement
Defines an output data set for a SELECT or SUBSET operation. Refer to "SORTOUT and OUTFIL DD statements" on page 71 for details.

A savedd DD statement must be present for each unique savedd name specified in each DISCARD operand.

listdd DD Statement
Defines the list data set for a DEFAULTS, DISPLAY, or OCCUR operation. For each listdd data set, ICETOOL uses RECFM=FBA (or RECFM=FB for DISPLAY or OCCUR with the NOCC operand specified), LRECL=121 (for DEFAULTS) or the LRECL specified in the WIDTH operand or calculated as needed if WIDTH is not specified (DISPLAY and OCCUR), and the specified block size. If the BLKSIZE you specify is not a multiple of the LRECL, ICETOOL uses BLKSIZE=LRECL. If you do not specify BLKSIZE, ICETOOL selects the block size as directed by the LISTSDB or LISTNOSDB option if specified, or otherwise as directed by the SDBMSG installation option (see *z/OS DFSORT Installation and Customization*).

A listdd DD statement must be present for each unique listdd name specified in each LIST operand.

countdd DD Statement
Defines the output data set for a COUNT operation. For each countdd data set, ICETOOL uses RECFM=FB, the LRECL specified in the WIDTH operand or calculated if WIDTH is not specified, and the specified block size. If the BLKSIZE you specify is not a multiple of the LRECL, ICETOOL uses BLKSIZE=LRECL. If you do not specify BLKSIZE, ICETOOL uses the system-determined optimum block size.

A countdd DD statement must be present for each unique countdd name specified in each WRITE operand.

xxxxCNTL DD Statement
Defines the DFSORT control statement data set for a COPY, COUNT, DATASORT, MERGE, RESIZE, SELECT, SORT, SPLICE or SUBSET operation. Refer to "SORTCNTL DD statement" on page 73 for more details.

An xxxxCNTL DD statement must be present for each unique *xxxx* specified in each USING operand.

JCL restrictions

You should avoid using ddnames reserved for ICETOOL and DFSORT in ICETOOL operands (FROM, TO, LIST, DISCARD, WRITE). In general, you should also avoid supplying DD statements with ddnames reserved for DFSORT when using ICETOOL because doing so can cause unpredictable results. Specifically:

- SORTCNTL should not be used as a ddname in ICETOOL operators nor should it be supplied as a DD statement. A xxxxCNTL DD statement should only be supplied when you specify a USING(xxxx) operand. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx.
- SYSIN, SORTCNTL, SORTIN, SORTOUT, SORTINnn, and xxxxINnn (where xxxx is specified in a USING operand) should not be used as ddnames in ICETOOL operators nor supplied as DD statements.
- TOOLMSG, DFSMSG, SSMSG, SYMNAMES, SYMNOUT, TOOLIN, SYSUDUMP, and SYSABEND should not be used as ddnames in ICETOOL operators.
- In general, xxxxWKdd ddnames should not be used as ddnames in ICETOOL operators nor supplied as DD statements. However, if you want to override dynamic allocation of work data sets for ICETOOL operators OCCUR and UNIQUE, you can use SORTWKdd DD statements for that purpose. If you want to override dynamic allocation of work data sets for ICETOOL operators DATASORT, RESIZE, SELECT, SORT, SPLICE, and SUBSET, you can use xxxxWKdd DD statements for that purpose in conjunction with the USING operand.
- DFSPARM (or the ddname specified for the PARMDDN installation option) should not be used as a ddname in ICETOOL operators. It should only be used as a DD statement to override DFSORT options for all operators, if appropriate. Refer to "DFSPARM DD statement" on page 74 for details.
- xxxxOFdd (where xxxx is specified in a USING operand) is required as the ddname when an OUTFIL statement in the xxxxCNTL data set specifies FILES=dd. To avoid this requirement, use the FNAMES=ddname operand rather than the FILES=dd operand in OUTFIL statements, and include a DD statement for the specified ddname. See "OUTFIL control statements" on page 219 for details of the FNAMES operand.

You should not use different DDs for a data set to be used for output and then input in the same step, if that data set can be extended to a second or subsequent volume, because that will result in incorrect output. See "Data set notes and limitations" on page 13 for more information.

ICETOOL statements

Each operation must be described to ICETOOL using an operator statement. Additionally, ICETOOL allows comment statements and blank statements. An explanation of the general rules for coding ICETOOL statements is given later in this section followed by a detailed discussion of each operator.

General coding rules

The general format for all ICETOOL operator statements is:

- OPERATOR operand ... operand

where each operand consists of KEYWORD(parameter, parameter...) or just KEYWORD. Any number of operators can be specified and in any order.

The following rules apply for operator statements:

- The operator and operands must be in uppercase EBCDIC.
- The operator must be specified first.
- One blank is required between the operator and the first operand.
- One blank is required between operands.
- Any number of blanks can be specified before or after the operator or any operand, but blanks cannot be specified anywhere else except within quoted character strings.

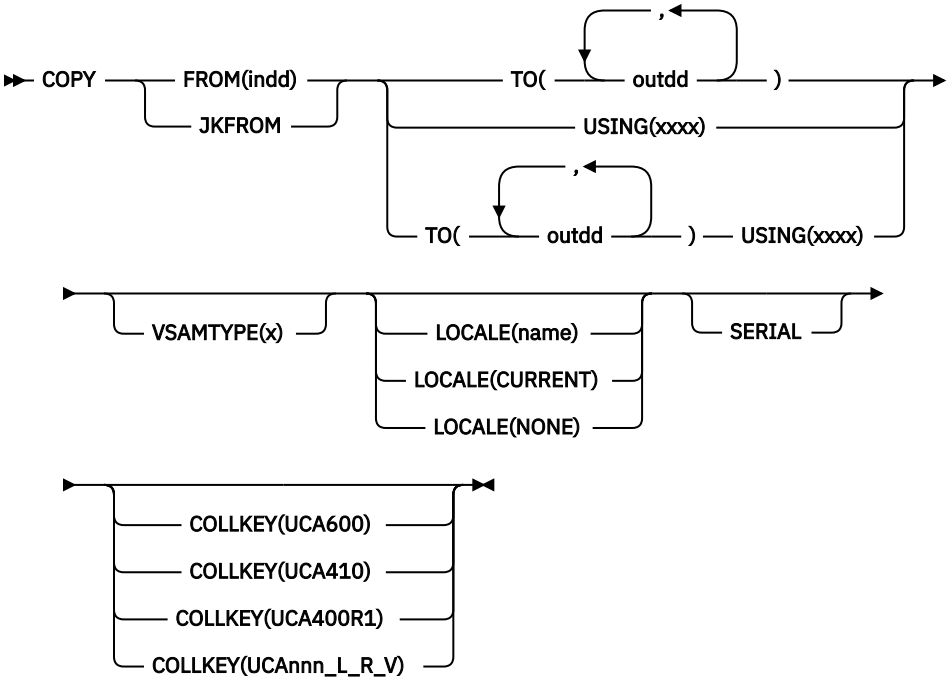
- Parentheses must be used where shown. Commas or semicolons must be used where commas are shown.
- Operands can be in any order.
- Columns 1-72 are scanned; columns 73-80 are ignored.
- Continuation can be indicated by a hyphen (-) **after** the operator or after any operand. The next operand must then be specified on the next line. For example:

```
SORT FROM(INDD) -
   USING(ABCD) -
   TO(OUTPUT1,OUTPUT2,OUTPUT3)
```

  Any characters specified after the hyphen are ignored. Each operand **must** be completely specified on one line.

A statement with an asterisk (*) in column 1 is treated as a comment statement. It is printed with the other ICETOOL statements, but otherwise not processed. A statement with blanks in columns 1 through 72 is treated as a blank statement. It is ignored, because ICETOOL prints blank lines where appropriate.

COPY operator



Copies an input data set to one or more output data sets.

DFSORT is called to copy the indd data set to the outdd data sets; the DFSORT control statements in xxxxCNTL are used if USING(xxxx) is specified. You can use DFSORT control statements and options in the xxxxCNTL data set to copy a subset of the input records (INCLUDE or OMIT statement; SKIPREC and STOPAFT options; OUTFIL INCLUDE, OMIT, SAVE, STARTREC, ENDREC, SAMPLE, SPLIT, SPLITBY, and SPLIT1R operands; user exit routines), reformat records for output (INREC, OUTREC and OUTFIL statements, user exit routines), and so on.

If an INCLUDE or OMIT statement or an OUTFIL INCLUDE or OMIT operand is specified in the xxxxCNTL data set, the active locale's collating rules affect INCLUDE and OMIT processing, as explained in the "Cultural Environment Considerations" discussion in “INCLUDE control statement” on page 91.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
Specifies the ddname of the input data set to be read by DFSORT for this operation. An indd DD statement must be present and must define an input data set that conforms to the rules for DFSORT's SORTIN data set.

Refer to "JCL restrictions" on page 542 for more information regarding the selection of ddnames.

JKFROM
Specifies you are using a JOINKEYS application with this COPY operator to copy the joined records. You must provide a USING(xxxx) operand. In xxxxCNTL, you must provide a JOINKEYS statement with F1=ddname1 for the F1 file and a JOINKEYS statement with F2=ddname2 for the F2 file, as well as JOIN and REFORMAT statements as needed.

TO(outdd,...)
Specifies the ddnames of the output data sets to be written by DFSORT for this operation. From 1 to 10 outdd names can be specified. An outdd DD statement must be present for each outdd name specified. If a single outdd data set is specified, DFSORT is called once to copy the indd data set to the outdd data set, using SORTOUT processing; the outdd data set must conform to the rules for DFSORT's SORTOUT data set. If multiple outdd data sets are specified and SERIAL is not specified, DFSORT is called once to copy the indd data set to the outdd data sets, using OUTFIL processing; the outdd data sets must conform to the rules for DFSORT's OUTFIL data sets.

TO and USING can both be specified. If USING is not specified, TO must be specified. If TO is not specified, USING must be specified.

A ddname specified in the FROM operand must not also be specified in the TO operand.

Refer to "JCL restrictions" on page 542 for more information regarding the selection of ddnames.

USING(xxxx)
Specifies the first 4 characters of the ddname for the control statement data set to be used by DFSORT for this operation. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx.

If USING is specified, an xxxxCNTL DD statement must be present and the control statements in it must conform to the rules for DFSORT's SORTCNTL data set.

TO and USING can both be specified. If USING is not specified, TO must be specified. If TO is not specified, USING must be specified and the xxxxCNTL data set must contain either one or more OUTFIL statements or a MODS statement for an E35 routine that disposes of all records. Other statements are optional.

Refer to "JCL restrictions" on page 542 for more information regarding the selection of ddnames.

VSAMTYPE(x)
Specifies the record type for a VSAM input data set. x must be either F for fixed-length record processing or V for variable-length record processing.

If VSAMTYPE(x) is specified, ICETOOL will pass a RECORD TYPE=x control statement to DFSORT. (If you specify a RECORD TYPE=x statement in the xxxxCNTL data set, it will override the one passed by ICETOOL.)

For complete information on record type processing for VSAM input, see "RECORD control statement" on page 416.

LOCALE(name)
Specifies that locale processing is to be used and designates the name of the locale to be made active during DFSORT processing. LOCALE(name) can be used to override the LOCALE installation option. For complete details on LOCALE(name), see the discussion of the LOCALE operand in "OPTION control statement" on page 171.

LOCALE(CURRENT)
Specifies that locale processing is to be used, and the current locale active when DFSORT is entered will remain the active locale during DFSORT processing. LOCALE(CURRENT) can be used to override

the LOCALE installation option. For complete details on LOCALE(CURRENT), see the discussion of the LOCALE operand in "OPTION control statement" on page 171.

LOCALE(NONE)
Specifies that locale processing is not to be used. DFSORT will use the binary encoding of the code page defined for your data for collating and comparing. LOCALE(NONE) can be used to override the LOCALE installation option. For complete details on LOCALE(NONE), see the discussion of the LOCALE operand in "OPTION control statement" on page 171.

SERIAL
Specifies that OUTFIL processing is not to be used when multiple outdd data sets are specified. DFSORT is called multiple times and uses SORTOUT processing; the outdd data sets must conform to the rules for DFSORT's SORTOUT data set. SERIAL is not recommended because the use of serial processing (that is, multiple calls to DFSORT) instead of OUTFIL processing can degrade performance and imposes certain restrictions as detailed later in this section. SERIAL is ignored if a single outdd data set is specified.

DFSORT is called to copy the indd data set to the first outdd data set using the DFSORT control statements in the xxxxCNTL data set if USING(xxxx) is specified. If the first copy is successful, DFSORT is called as many times as necessary to copy the first outdd data set to the second and subsequent outdd data sets. Therefore, for maximum efficiency, use a disk data set as the first in a list of outdd data sets on both disk and tape. If more than one outdd data set is specified, DFSORT must be able to read the first outdd data set after it is written in order to copy it to the other outdd data sets. Do not use a SYSOUT or DUMMY data set as the first in a list of outdd data sets because:

- if the first data set is SYSOUT, DFSORT abends when it tries to copy the SYSOUT data set to the second outdd data set.
- if the first data set is DUMMY, DFSORT copies the empty DUMMY data set to the other outdd data sets, with the result that all outdd data sets are then empty.

COLLKEY(UCA600)
This collation version supports the Unicode Standard character suite 6.0.0 and uses Normalization Service under 6.0.0 Unicode character suite.

COLLKEY(UCA410)
This collation version supports the Unicode Standard character suite 4.1.0 and uses Normalization Service under 4.1.0 Unicode character suite.

COLLKEY(UCA400R1)
This collation version supports the Unicode Standard character suite 4.0.0 and uses Normalization Service under 4.0.1 Unicode character suite.

COLLKEY(UCAnnn_L_R_V)
Specifies a collation version, where specific collation rules will modify any of the default Unicode Collation tables specified (UCA400R1, UCA410, or UCA600). Collation versions are set when you specify the following fields:

L
- Language (Specify a language for desired locale.)

R
- Region (Specify a region for desired locale.)

V
- Variant (Specify a variant for desired locale.)

Note:

- For supported collation version settings (Language/Region/Variant), see Appendix E. Locales for collation in *z/OS Unicode Services User's Guide and Reference*
- If there is no collation version information, installation default collation version will be set as default without any change.

Unicode Locales repository data set name SYS1.SCUNLOCL contains a set of locales documented in Locales for collation support. All of those locales contain a section for Collation rules.

Copy examples

Example 1

```
* Method 1
COPY FROM(MASTER) TO(PRINT,TAPE,DISK)
```

```
* Method 2
COPY FROM(MASTER) TO(DISK,TAPE,PRINT) SERIAL
```

This example shows two different methods for creating multiple output data sets.

Method 1 requires one call to DFSORT, one pass over the input data set, and allows the output data sets to be specified in any order. The COPY operator copies all records from the MASTER data set to the PRINT (SYSOUT), TAPE, and DISK data sets, using OUTFIL processing.

Method 2 requires three calls to DFSORT, three passes over the input data set, and imposes the restriction that the SYSOUT data set must not be the first TO data set. The COPY operator copies all records from the MASTER data set to the DISK data set and then copies the resulting DISK data set to the TAPE and PRINT (SYSOUT) data sets. Because the first TO data set is processed three times (written, read, read), placing the DISK data set first is more efficient than placing the TAPE data set first. PRINT must not be the first in the TO list because a SYSOUT data set cannot be read.

Example 1

```
* Method 1
COPY FROM(MASTER) TO(PRINT,TAPE,DISK)
```

```
* Method 2
COPY FROM(MASTER) TO(DISK,TAPE,PRINT) SERIAL
```

This example shows two different methods for creating multiple output data sets.

Method 1 requires one call to DFSORT, one pass over the input data set, and allows the output data sets to be specified in any order. The COPY operator copies all records from the MASTER data set to the PRINT (SYSOUT), TAPE, and DISK data sets, using OUTFIL processing.

Method 2 requires three calls to DFSORT, three passes over the input data set, and imposes the restriction that the SYSOUT data set must not be the first TO data set. The COPY operator copies all records from the MASTER data set to the DISK data set and then copies the resulting DISK data set to the TAPE and PRINT (SYSOUT) data sets. Because the first TO data set is processed three times (written, read, read), placing the DISK data set first is more efficient than placing the TAPE data set first. PRINT must not be the first in the TO list because a SYSOUT data set cannot be read.

Example 2

```
* Method 1
COPY FROM(IN) TO(DEPT1) USING(DPT1)
COPY FROM(IN) TO(DEPT2) USING(DPT2)
COPY FROM(IN) TO(DEPT3) USING(DPT3)
```

```
* Method 2
COPY FROM(IN) USING(ALL3)
```

This example shows two different methods for creating subsets of an input data set. Assume that:

- The DPT1CNTL data set contains:

  ```
  INCLUDE COND=(5,3,CH,EQ,C'D01')
  ```

- The DPT2CNTL data set contains:

```
  INCLUDE COND=(5,3,CH,EQ,C'D02')
```

- The DPT3CNTL data set contains:

```
  INCLUDE COND=(5,3,CH,EQ,C'D03')
```

- The ALL3CNTL data set contains:

```
  OUTFIL FNAMES=DEPT1,INCLUDE=(5,3,CH,EQ,C'D01')
  OUTFIL FNAMES=DEPT2,INCLUDE=(5,3,CH,EQ,C'D02')
  OUTFIL FNAMES=DEPT3,INCLUDE=(5,3,CH,EQ,C'D03')
```

Method 1 requires three calls to DFSORT and three passes over the input data set:

- The first COPY operator copies the records from the IN data set that contain D01 in positions 5-7 to the DEPT1 data set.
- The second COPY operator copies the records from the IN data set that contain D02 in positions 5-7 to the DEPT2 data set.
- The third COPY operator copies the records from the IN data set that contain D03 in positions 5-7 to the DEPT3 data set.

Method 2 accomplishes the same result as method 1, but because it uses OUTFIL statements instead of TO operands, requires only one call to DFSORT and one pass over the input data set.

Example 3

```
 COPY FROM(VSAMIN) TO(VSAMOUT) VSAMTYPE(V)
```

The COPY operator copies all records from the VSAMIN data set to the VSAMOUT data set. The VSAM records are treated as variable-length.

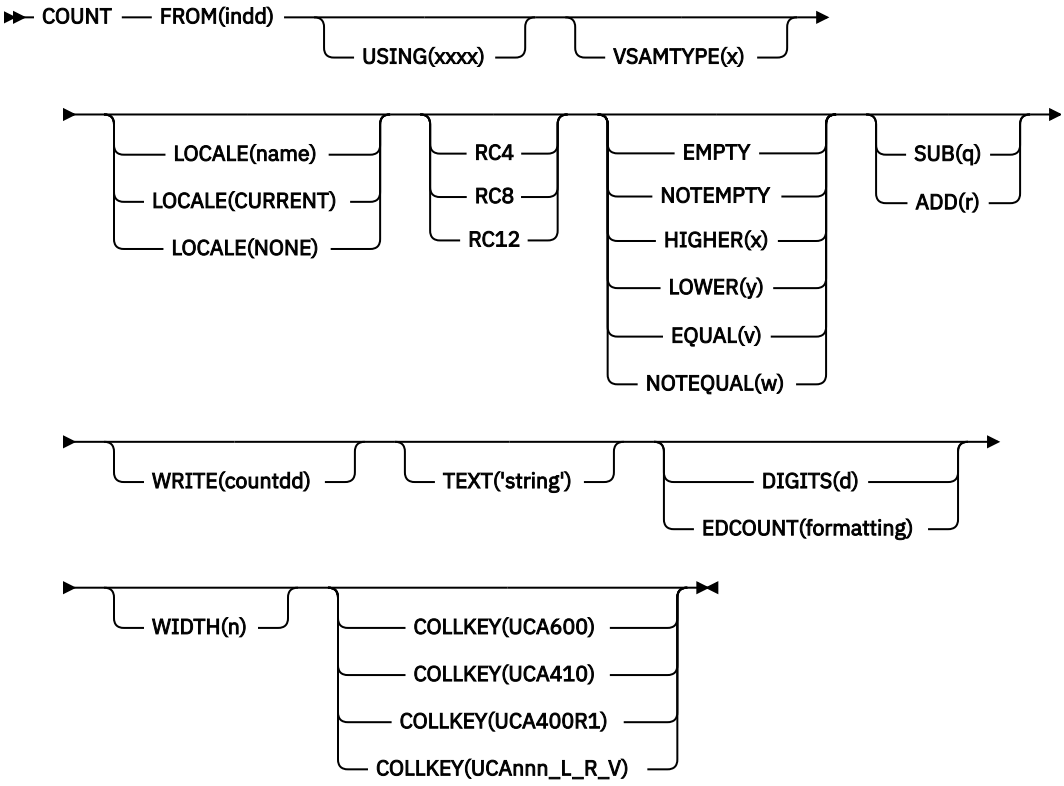
COPY operator with JOINKEYS example

Here is an example of using multiple COPY operators for JOINKEYS applications that preprocess different input files in different ways:

```
//CPYJK EXEC PGM=ICETOOL
//TOOLMSG DD SYSOUT=*
//DFSMSG DD SYSOUT=*
//IN1 DD DSN=MY.IN1,DISP=SHR
//IN2 DD DSN=MY.IN2,DISP=SHR
//IN3 DD DSN=MY.IN3,DISP=SHR
//OUT1 DD SYSOUT=*
//OUT2 DD SYSOUT=*
//TOOLIN DD *
* First COPY operator with JOINKEYS application.
COPY JKFROM TO(OUT1) USING(CTL1)
* Second COPY operator with JOINKEYS application.
COPY JKFROM USING(CTL2)
/*
//CTL1CNTL DD *
* JOINKEYS application control statements for first COPY operator.
  JOINKEYS F1=IN1,FIELDS=(5,12,A),TASKID=T1
  JOINKEYS F2=IN2,FIELDS=(11,12,A),TASKID=T1
  JOIN UNPAIRED
  REFORMAT FIELDS=(F1:4,40,F2:15,20),FILL=C'$'
* Main task control statements for first COPY operator
* (operates on joined records).
  INCLUDE COND=(8,1,CH,EQ,C'Y')
/*
//CTL2CNTL DD *
* JOINKEYS application control statements for second COPY operator.
  JOINKEYS F1=IN1,FIELDS=(5,12,A),TASKID=T1
  JOINKEYS F2=IN3,FIELDS=(9,12,A),TASKID=T2,SORTED
  REFORMAT FIELDS=(F1:4,40,F2:7,20)
* Main task control statements for second COPY operator
* (operates on joined records).
  OUTFIL FNAMES=OUT2,HEADER1=('Analysis Report'),REMOVECC
/*
```

```
//T1F1CNTL DD *
* Control statements for subtask1 (F1=IN1) of both COPY operators.
* Subtask1 sorts/joins on 5,12,A automatically
* per JOINKEYS statement for TASKID=T1/F1=IN1.
  INCLUDE COND=(21,3,CH,EQ,C'J82')
  SUM FIELDS=NONE
/*
//T1F2CNTL DD *
* Control statements for subtask2 (F2=IN2) of first COPY operator.
* Subtask1 sorts/joins on 11,12,A automatically
* per JOINKEYS statement for TASKID=T1/F2=IN2.
  OPTION SKIPREC=1
  INCLUDE COND=(25,3,CH,EQ,C'J82')
/*
//T2F2CNTL DD *
* Control statements for subtask2 (F2=IN3) of second COPY operator.
* Subtask1 copies/joins on 9,12,A automatically
* per JOINKEYS statement for TASKID=T2/F2=IN3/SORTED.
  INCLUDE COND=(5,3,CH,EQ,C'J82')
/*
```

COUNT operator



Prints a message containing the count of records in a data set. Can also be used to subtract a value from the count or add a value to the count, to create an output data set containing text and the count, or to set RC=12, RC=8, RC=4, or RC=0 based on meeting criteria for the number of records in a data set.

DFSORT is called to copy the indd data set to ICETOOL's E35 user exit. The DFSORT control statements in xxxxCNTL are used if USING(xxxx) is specified. You can use a DFSORT INCLUDE or OMIT statement in the xxxxCNTL data set to count a subset of the input records.

If an INCLUDE or OMIT statement is specified in the xxxxCNTL data set, the active locale's collating rules affect INCLUDE and OMIT processing as explained in the "Cultural Environment Considerations" discussion in “INCLUDE control statement” on page 91.

If EMPTY, NOTEMPTY, HIGHER(x), LOWER(y), EQUAL(v) or NOTEQUAL(w) is not specified, ICETOOL prints a message containing the record count as determined by its E35 user exit.

If EMPTY, NOTEMPTY, HIGHER(x), LOWER(y), EQUAL(v) or NOTEQUAL(w) is specified, ICETOOL checks the record count as determined by its E35 user exit against the specified criteria. If the criteria is met (for example, HIGHER(20) is specified and the record count is 21 or more), ICETOOL sets the following return code for the COUNT operator:

- RC=12 if RC12 is specified, or by default if RC8 and RC4 are not specified
- RC=8 if RC8 is specified
- RC=4 if RC4 is specified

If the criteria is not met (for example, HIGHER(20) is specified and the record count is 20 or less), ICETOOL sets RC=0 for the COUNT operator. ICETOOL uses DFSORT's STOPAFT option to process the minimum number of records required to determine whether or not the criteria is met.

Note: Be sure to check the messages in TOOLMSG when you initially set up any COUNT operators with criteria to make sure that RC=12 is not issued because of syntax errors.

If SUB(q) is specified, ICETOOL subtracts q from the count, but does not reduce the count below 0. If ADD(r) is specified, ICETOOL adds r to the count. SUB(q) and ADD(r) are especially useful for dealing with data sets that contain header and trailer records.

If WRITE(countdd) is specified, ICETOOL writes a record containing the count in the countdd data set. TEXT('string') can be used to insert a string before the count in the output record. DIGITS(n) or EDCOUNT(formatting) can be used to change the appearance of the count in the output record. WIDTH(n) can be used to change the length of the output record.

Do not supply your own DFSORT MODS statement because it would override the MODS statement passed by ICETOOL for this operator.

The record count is also printed for the DISPLAY, OCCUR, RANGE, SELECT, STATS, UNIQUE, and VERIFY operators.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

USING(xxxx)
Specifies the first 4 characters of the ddname for the control statement data set to be used by DFSORT for this operation. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx.

If USING is specified, an xxxxCNTL DD statement must be present and the control statements in it:

1. Must conform to the rules for DFSORT's SORTCNTL data set.
2. Should generally be used only for an INCLUDE or OMIT statement or comments statements.

Refer to “JCL restrictions” on page 542 for more information regarding the selection of ddnames.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

LOCALE(name)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

LOCALE(CURRENT)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

LOCALE(NONE)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

RC4
Can be used to set RC=4 for this COUNT operator if the record count meets the specified criteria. RC4 can be specified only if EMPTY, NOTEMPTY, HIGHER(x), LOWER(y), EQUAL(v), or NOTEQUAL(w)

is specified. RC4 overrides the default of setting RC=12 for this COUNT operator if the record count meets the specified criteria

RC8
Can be used to set RC=8 for this COUNT operator if the record count meets the specified criteria. RC8 can be specified only if EMPTY, NOTEMPTY, HIGHER(x), LOWER(y), EQUAL(v), or NOTEQUAL(w) is specified. RC8 overrides the default of setting RC=12 for this COUNT operator if the record count meets the specified criteria.

RC12
Can be used to set RC=12 for this COUNT operator if the record count meets the specified criteria. RC12 can be specified only if EMPTY, NOTEMPTY, HIGHER(x), LOWER(y), EQUAL(v), or NOTEQUAL(w) is specified. RC12 is equivalent to the default used when RC4 or RC8 is not specified.

EMPTY
Sets RC=12 (or RC=8 if RC8 is specified, or RC=4 if RC4 is specified) for this COUNT operator if the input data set or subset is empty, or sets RC=0 for this COUNT operator if the input data set or subset is not empty.

EMPTY is equivalent to EQUAL(0).

NOTEMPTY
Sets RC=12 (or RC=8 if RC8 is specified, or RC=4 if RC4 is specified) for this COUNT operator if the input data set or subset is not empty, or sets RC=0 for this COUNT operator if the input data set or subset is empty.

EMPTY is equivalent to NOTEQUAL(0).

HIGHER(x)
Sets RC=12 (or RC=8 if RC8 is specified, or RC=4 if RC4 is specified) for this COUNT operator if the record count is higher than x, or sets RC=0 for this COUNT operator if the record count is equal to or lower than x.

x must be specified as n or +n where n can be 0 to 562949953421310.

LOWER(y)
Sets RC=12 (or RC=8 if RC8 is specified, or RC=4 if RC4 is specified) for this COUNT operator if the record count is lower than y, or sets RC=0 for this COUNT operator if the record count is equal to or higher than y.

y must be specified as n or +n where n can be 0 to 562949953421310.

EQUAL(v)
Sets RC=12 (or RC=8 if RC8 is specified, or RC=4 if RC4 is specified) for this COUNT operator if the record count is equal to v, or sets RC=0 for this COUNT operator if the record count is not equal to v.

v must be specified as n or +n where n can be 0 to 562949953421310.

NOTEQUAL(w)
Sets RC=12 (or RC=8 if RC8 is specified, or RC=4 if RC4 is specified) for this COUNT operator if the record count is not equal to w, or sets RC=0 for this COUNT operator if the record count is equal to w.

w must be specified as n or +n where n can be 0 to 562949953421310.

SUB(q)
Subtracts q from the record count, but does not reduce the count below 0. The resulting modified record count is displayed in the count message in TOOLMSG. If WRITE(countdd) is specified, the modified record count is used in the count record. If EMPTY, NOTEMPTY, HIGHER(x), LOWER(y), EQUAL(v) or NOTEQUAL(w) is specified, the modified record count is used to determine if the criteria is satisfied.

SUB(q) is especially useful for getting the count of just the data records for a data set that contains header, data and trailer records.

q must be specified as n or +n where n can be 1 to 999.

ADD(r)
Adds r to the record count. The resulting modified record count is displayed in the count message in TOOLMSG. If WRITE(countdd) is specified, the modified record count is used in the count record. If EMPTY, NOTEMPTY, HIGHER(x), LOWER(y), EQUAL(v) or NOTEQUAL(w) is specified, the modified record count is used to determine if the criteria is satisfied.

ADD(r) is especially useful for getting the count of header, data and trailer records for a data set that just contains data records to which you want to add header and trailer records.

r must be specified as n or +n where n can be 1 to 999.

WRITE(countdd)
Specifies the ddname of the count data set to be produced by ICETOOL for this operation. A countdd DD statement must be present. ICETOOL sets the attributes of the count data set as follows:

- RECFM is set to FB.
- LRECL is set to one of the following:
  - If WIDTH(n) is specified, LRECL is set to n. Use WIDTH(n) if your count record length and LRECL must be set to a particular value (for example, 80), or if you want to ensure that the count record length does not exceed a specific maximum (for example, 20 bytes).
  - If WIDTH(n) is not specified, LRECL is set to the calculated required record length. If your LRECL does not need to be set to a particular value, you can let ICETOOL determine and set the appropriate LRECL value by not specifying WIDTH(n).
- BLKSIZE is set to one of the following:
  - The BLKSIZE from the DD statement, DSCB, or label, if it is a multiple of the LRECL used.
  - The LRECL if the BLKSIZE from the DD statement, DSCB, or label is not a multiple of the LRECL used.
  - The system determined blocksize if the BLKSIZE is not available from the DD statement, DSCB, or label.

TEXT('string')
Specifies a string to be printed starting in the first byte of the count record. The count follows the string. TEXT can only be specified if WRITE(countdd) is specified.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, do not specify TEXT('string') or specify TEXT('') using two single apostrophes.

DIGITS(d)
Specifies d digits for the count in the output record, overriding the default of 15 digits. d can be 1 to 15. The count is written as d decimal digits with leading zeros. DIGITS can only be specified if WRITE(countdd) is specified.

If you know that your count requires less than 15 digits, you can use a lower number of digits (d) instead by specifying DIGITS(d). For example, if DIGITS(10) is specified, 10 digits are used instead of 15.

If you use DIGITS(d) and the count overflows the number of digits used, ICETOOL terminates the operation. You can prevent the overflow by specifying an appropriately higher d value for DIGITS(d). For example, if DIGITS(5) results in overflow, you can use DIGITS(6) instead.

EDCOUNT(formatting)
Specifies formatting items that indicate how the count in the output record is to be formatted, overriding the default of 15 digits with leading zeros. Formatting items can be specified in any order, but each item can only be specified once. EDCOUNT can only be specified if WRITE(countdd) is specified.

mask
See the discussion of mask under ON(p,m,f,formatting) in "DISPLAY Operator" on page “mask” on page 568.

E'pattern'
specifies an edit pattern to be applied to the count. The pattern (1 to 24 characters) must be enclosed in single apostrophes. Each 9 in the pattern (up to 15) is replaced by a corresponding digit from the count. Characters other than 9 in the pattern appear as specified. To include a single apostrophe (') in the pattern, specify two single apostrophes (''). F'string' or a mask cannot be specified with E'pattern'.

When E'pattern' is specified for the count:

- If the number of significant digits in the count is less than the number of 9's in the pattern, 0's are filled in on the left. For example, 1234 is shown as 001234 with EDCOUNT(E'999999').
- If the number of significant digits in the count is greater than the number of 9's in the pattern, digits are truncated from the left. For example, 1234567 is shown as *4567* with EDCOUNT(E'*9999*').

L'string
See the discussion of L'string' under ON(p,m,f,formatting) in "DISPLAY Operator" on page “L'string'” on page 570

F'string'
See the discussion of F'string' under ON(p,m,f,formatting) in "DISPLAY Operator" on page “F'string'” on page 570

T'string'
See the discussion of T'string' under ON(p,m,f,formatting) in "DISPLAY Operator" on page “T'string'” on page 570

LZ
See the discussion of LZ under ON(p,m,f,formatting) in "DISPLAY Operator" on page “LZ” on page 571

Udd
specifies the number of digits to be used for the count. dd specifies the number of digits and must be a two-digit number between 01 and 15. The default number of digits (d) for the count is 15.

If you know that your count requires less than 15 digits, you can use a lower number of digits (dd) instead by specifying Udd. For example, if EDCOUNT(U09) is specified, 9 digits (from U09) is used instead of 15 (default for the count).

If you use Udd and the count overflows the number of digits used, ICETOOL terminates the operation. You can prevent the overflow by specifying an appropriately higher dd value for Udd. For example, if EDCOUNT(U05) results in overflow, you can use EDCOUNT(U06) instead.

If E'pattern' is specified, Udd is ignored, because d is determined from the pattern.

WIDTH(n)
Specifies the record length and LRECL you want ICETOOL to use for the count data set. n can be from 1 to 32760. WIDTH can only be specified if WRITE(countdd) is specified. ICETOOL always calculates the record length required to write the count record and uses it as follows:

- If WIDTH(n) is specified and the calculated record length is less than or equal to n, ICETOOL sets the record length and LRECL to n. ICETOOL pads the count record on the right with blanks to the record length.
- If WIDTH(n) is specified and the calculated record length is greater than n, ICETOOL issues an error message and terminates the operation.
- If WIDTH(n) is not specified, ICETOOL sets the record length and LRECL to the calculated record length.

Use WIDTH(n) if your count record length and LRECL must be set to a particular value (for example, 80), or if you want to ensure that the count record length does not exceed a specific maximum (for example, 20 bytes). Otherwise, you can let ICETOOL calculate and set the appropriate record length and LRECL by not specifying WIDTH(n).

COLLKEY(UCA600)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

COLLKEY(UCA410)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

COLLKEY(UCA400R1)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

COLLKEY(UCAnnn_L_R_V)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

COUNT examples

Example 1

For this example, assume that the CTL1CNTL data set contains a DFSORT INCLUDE statement.

```
COUNT FROM(IN1)
COUNT FROM(IN2) USING(CTL1)
```

The first COUNT operator prints a message containing the count of records in the IN1 data set.

The second COUNT operator prints a message containing the count of records included from the IN2 data set.

Example 2

```
COUNT FROM(INPUT1) EMPTY
```

Sets RC=12 if INPUT1 is empty (that is, INPUT1 has no records), or sets RC=0 if INPUT1 is not empty (that is, INPUT1 has at least one record).

Example 3

For this example, assume that the CTL2CNTL data set contains a DFSORT INCLUDE statement.

```
COUNT FROM(INPUT2) HIGHER(50000) RC4 USING(CTL2)
```

Sets RC=4 if more than 50000 records are included from INPUT2, or sets RC=0 if 50000 or less records are included from INPUT2.

Example 4

```
COUNT FROM(IN2) WRITE(CT2) TEXT('Count is ') -
  EDCOUNT(A1,U10) WIDTH(80)
```

Prints a message containing the count of records in the IN2 data set. Writes an 80-byte record with the specified string and an edited count to the CT2 data set. If IN2 contains 3286721 records, the 80-byte output record in CT2 would look like this:

```
Count is      3,286,721
```

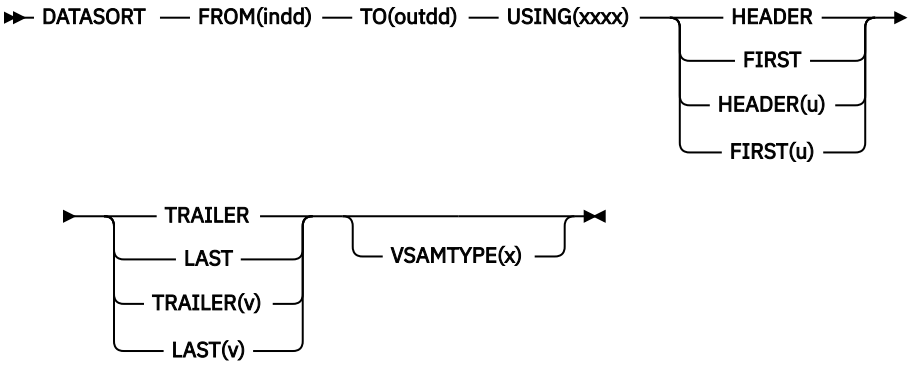
Example 5

```
COUNT FROM(IN3) WRITE(CT3) DIGITS(6) SUB(2)
```

Subtracts 2 from the count of records in the IN3 data set. Prints a message containing the modified count. Writes a 6-byte record with the modified count to the CT3 data set. If IN3 contains 8125 records, the 6-byte output record in CT3 would look like this:

```
008123
```

DATASORT operator



Copies one or more header records and one or more trailer records to the output data set in their original input record order, while sorting the data records between the header and trailer records, using the DFSORT control statements in xxxxCNTL. By definition, the header records are the first n records in the input data set, the trailer records are the last n records in the input data set, and the data records (also called detail records) are the records between the header and trailer records. Thus, the first n records (header records) and last n records (trailer records) are kept in place and the data records between them are sorted.

You must specify one header operand (HEADER, FIRST, HEADER(u) or FIRST(u)), one trailer operand (TRAILER, LAST, TRAILER(v) or LAST(v)), or one header operand and one trailer operand. If you specify a header operand without a trailer operand, only the header records will be kept in place. If you specify a trailer operand without a header operand, only the trailer records will be kept in place. If you specify a header operand and a trailer operand, both the header records and trailer records will be kept in place.

DFSORT is called to copy the header and trailer records and to sort the data records. DFSORT uses its E15 and E35 exits to process the records as needed. ICETOOL passes the EQUALS option to DFSORT to ensure that duplicates are kept in their original input order.

You must supply a DFSORT SORT statement in the xxxxCNTL data set to indicate the control fields for sorting the data records.

You can use additional DFSORT control statements in xxxxCNTL providing you observe these rules:

- A SORT statement must be present
- MODS and OUTREC statements should not be present.
- A STOPAFT operand should not be present.
- Comment statements can be present.
- Header and trailer records will only be affected by the SKIPREC option and OUTFIL statements. SKIPREC=n will remove the first n indd records, so the first header record will be the n+1 indd record. OUTFIL statements will process the header and trailer records in the normal way.
- Data records will be processed by INCLUDE, OMIT, INREC, SUM, OPTION and OUTFIL statements in the normal way.
- If you use INREC to change the length of the data records, DFSORT will preserve the header and trailer records by setting the TO data set LRECL to the maximum of the input or reformatted record length. For fixed-length records, DFSORT will pad the header and trailer records, or data records, on the right with blanks as appropriate.
- You can further process the outdd records **after** DATASORT processing using an OUTFIL statement like this:

```
  OUTFIL FNAMES=outdd,...
```

or multiple OUTFIL statements like this:

```
OUTFIL FNAMES=outdd,...
OUTFIL FNAMES=outdd1,...
```

For example, with TO(OUT1) you could further modify the OUT1 records after DATASORT processing, with a statement like this:

```
OUTFIL FNAMES=OUT1,REMOVECC,
  TRAILER1=('Record count ',COUNT=(M11,LENGTH=5))
```

ICETOOL requires extra storage for DATASORT processing, over and above what is normally needed by ICETOOL and DFSORT, in order to save your header and trailer records. The amount of storage needed depends on the number of header and trailer records you specify, and the LRECL of the FROM data set. In most cases, the needed storage can be obtained (above 16MB virtual). However, if ICETOOL cannot get the storage it needs, it issues a message and terminates the DATASORT operation. Increasing the REGION by the amount indicated in the message may allow ICETOOL to run successfully.

The DYNALLOC option is passed to DFSORT to ensure that work space is available for the sort. If your installation defaults for dynamic allocation are inappropriate for a DATASORT operator, you can take one of the following actions:

1. Override the DYNALLOC option using an OPTION control statement such as:

   ```
   OPTION DYNALLOC=(,8)
   ```

   in the xxxxCNTL data set.
2. Use xxxxWKdd DD statements to override the use of dynamic allocation. Refer to “SORTWKdd DD statement” on page 69 for details.

Tape work data sets **cannot** be used with ICETOOL.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the COPY statement in "COPY Operator" on page “FROM(indd)” on page 544.

TO(outdd)
Specifies the ddname of the output data set to be written by DFSORT for this operation.

An outdd DD statement must be present and must define an output data set that conforms to the rules for DFSORT's SORTOUT data set.

The ddname specified in the TO operand must not be the same as the ddname specified in the FROM operand.

Refer to “JCL restrictions” on page 542 for more information.

USING(xxxx)
Specifies the first 4 characters of the ddname for the control statement data set to be used by DFSORT for this operation. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx

An xxxxCNTL DD statement must be present. The xxxxCNTL data set must contain a SORT statement. Other control statements are optional, but if specified must conform to the rules for DFSORT's SORTCNTL data set, and should be used as described for “DATASORT operator” on page 554.

HEADER or FIRST
Specifies one header record (the first record in the indd data set) is to be kept in place.

HEADER and FIRST are equivalent to HEADER(1) and FIRST(1).

HEADER(u) or FIRST(u)
Specifies u header records (the first u records in the indd data set) are to be kept in place.

u must be specified as n or +n where n can be 1 to 1000000.

TRAILER or LAST
Specifies one trailer record (the last record in the indd data set) is to be kept in place.

TRAILER and LAST are equivalent to TRAILER(1) and LAST(1).

TRAILER(v) or LAST(v)
Specifies v trailer records (the last v records in the indd data set) are to be kept in place.

v must be specified as n or +n where n can be 1 to 1000000.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in "COPY Operator" on page “VSAMTYPE(x)” on page 544.

DATASORT examples

Although the DATASORT operators in the examples in this section could all be contained in a single ICETOOL job step, they are shown and discussed separately for clarity.

Example 1

```
DATASORT FROM(INPUT) TO(OUTPUT) HEADER TRAILER USING(CTL1)
```

This example shows how you can sort the data records between a header record (first record) and a trailer record (last record).

The CTL1CNTL data set contains:

```
  SORT FIELDS=(16,13,CH,A)
```

The input records look like this:

```
MM9999900510100823DDDDD FFFFF 004200806128
AAR   FIRST C  1134341444441     XXXXXXXXX
ATX   SECOND   7777777770111     XXXXXXXXX
ATX   THIRD L  6297132201111     XXXXXXXXX
ATX   FOURTH   2830012906356     XXXXXXXXX
MM9999900510100823DDDDD FFFFF 004
```

We want to keep the MM records in place and sort the other records by the CH field in positions 16-28. We use HEADER and TRAILER to indicate the first and last records should not be sorted. We use the SORT statement to SORT ascending on positions 16-28.

The output records would look like this:

```
MM9999900510100823DDDDD FFFFF 004200806128
AAR   FIRST C  1134341444441     XXXXXXXXX
ATX   FOURTH   2830012906356     XXXXXXXXX
ATX   THIRD L  6297132201111     XXXXXXXXX
ATX   SECOND   7777777770111     XXXXXXXXX
MM9999900510100823DDDDD FFFFF 004
```

Note that we could use FIRST and LAST instead of HEADER and TRAILER.

Example 2

```
DATASORT FROM(IN) TO(OUT) HEADER(2) TRAILER(3) USING(CTL2)
```

This example illustrates how you can sort the data records between header records (first records) and trailer records (last records), and modify just the data records or the header, data and trailer records.

The CTL2CNTL data set contains:

```
  INREC IFTHEN=(WHEN=(24,2,CH,EQ,C'23'),
    OVERLAY=(30:C'Old'))
  SORT FIELDS=(1,14,CH,A)
  OUTFIL FNAMES=OUT,
    IFTHEN=(WHEN=(24,2,CH,EQ,C'23'),
      OVERLAY=(35:C'First'))
```

The input records look like this:

```
Header 1       2008/04/23
Header 2       2008/04/23
Geometry       2008/04/24
Algebra        2008/04/23
Trigonometry   2008/04/24
Calculus       2008/04/25
Geography      2008/04/25
History        2008/04/23
Trailer 1      2008/04/23
Trailer 2      2008/04/23
Trailer 3      2008/04/23
```

We want to keep the two Header records and the three Trailer records in place and sort the other records by the CH field in positions 1-14. We want to put 'Old' in positions 30-32 of **each data record** (but not the Header or Trailer records) that has '23' in positions 24-25. We want to put 'First' in positions 35-39 of **each record** (Header, data and Trailer) that has '23' in positions 24-25.

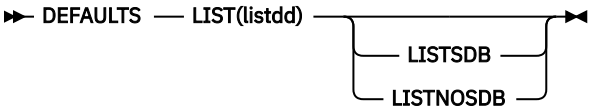
We use HEADER(2) and TRAILER(3) to indicate the first two records and last three records should not be sorted. We use the INREC statement to add 'Old' to data records that have '23' in positions 24-25. INREC applies to the data records, but not to the Header and Trailer records. We use the SORT statement to sort ascending on positions 1-14. We use the OUTFIL statement to add 'First' to Header, data and Trailer records that have '23' in positions 24-25. OUTFIL applies to all of the records.

The output records would look like this:

```
Header 1       2008/04/23         First
Header 2       2008/04/23         First
Algebra        2008/04/23    Old  First
Calculus       2008/04/25
Geography      2008/04/25
Geometry       2008/04/24
History        2008/04/23    Old  First
Trigonometry   2008/04/24
Trailer 1      2008/04/23         First
Trailer 2      2008/04/23         First
Trailer 3      2008/04/23         First
```

Note that we could use FIRST(2) and LAST(3) instead of HEADER(2) and TRAILER(3).

DEFAULTS operator



Prints the DFSORT installation defaults report in a separate list data set.

DFSORT enables you to maintain separate sets of installation defaults for eight different installation environments as follows:

- JCL (ICEAM1) - batch JCL directly invoked installation environment
- INV (ICEAM2) - batch program invoked installation environment
- TSO (ICEAM3) - TSO directly invoked installation environment
- TSOINV (ICEAM4) - TSO program invoked installation environment
- TD1 (ICETD1) - first time-of-day installation environment

- TD2 (ICETD2) - second time-of-day installation environment
- TD3 (ICETD3) - third time-of-day installation environment
- TD4 (ICETD4) - fourth time-of-day installation environment

Each installation default has two or more possible values; DFSORT is shipped with a set of IBM-supplied defaults that can be modified using ICEPRMxx members of PARMLIB or the ICEMAC macro. The DEFAULTS operator provides an easy way to determine the installation default values to be used at run-time. See *z/OS DFSORT Installation and Customization* for a complete discussion of ICEPRMxx members in PARMLIB, the ICEMAC macro, the eight installation environments, and the installation default values.

DEFAULTS produces a three-part report showing:

1. The merged PARMLIB/ICEMAC installation default values for ICEAM1-4 and ICETD1-4 that will be used at run-time.
2. The specified PARMLIB ICEPRMxx member option values for ICEAM1-4 and ICETD1-4 (for reference).
3. The ICEMAC installation default values for ICEAM1-4 and ICETD1-4 (for reference).

The format of the report produced by DEFAULTS varies depending on the defaults selected, but the first part of the report might look like this conceptually:

```
Z/OS DFSORT V2R3  MERGED PARMLIB/ICEMAC DEFAULTS     - p -

    * IBM-SUPPLIED DEFAULT (ONLY SHOWN IF DIFFERENT FROM THE SPECIFIED DEFAULT)

 ITEM           JCL (ICEAM1) VALUE       INV (ICEAM2) VALUE       TSO (ICEAM3) VALUE       TSOINV (ICEAM4) VALUE
 ----------     --------------------     --------------------     --------------------     --------------------
 item           value                    value                    value                    value
  .
  .
  .
 item           value                    value                    value                    value
 item           value                    value                    value                    value
                                         * IBM_value
  .
  .
  .


 Z/OS DFSORT V2R3  MERGED PARMLIB/ICEMAC DEFAULTS     - p -

    * IBM-SUPPLIED DEFAULT (ONLY SHOWN IF DIFFERENT FROM THE SPECIFIED DEFAULT)

 ITEM           TD1 (ICETD1) VALUE       TD2 (ICETD2) VALUE       TD3 (ICETD3) VALUE       TD4 (ICETD4) VALUE
 ----------     --------------------     --------------------     --------------------     --------------------
 item           value                    value                    value                    value
  .
  .
  .
 item           value                    value                    value                    value
                * IBM_value              * IBM_value
 item           value                    value                    value                    value
  .
  .
  .
```

The merged PARMLIB/ICEMAC default value for each item is shown as it is set for each of the eight installation environments. For any value that is different from the IBM-supplied value, the IBM-supplied value is shown below it.

The control character occupies the first byte of each record. The title and headings are always printed; p is the page number. The item name column occupies 10 bytes, each of the item value columns occupies 20 bytes, and 5 blanks appear between columns.

Operand descriptions

The operands described in this section can be specified in any order.

LIST(listdd)
Specifies the ddname of the list data set to be produced by ICETOOL for this operation. A listdd DD statement must be present. ICETOOL uses RECFM=FBA, LRECL=121 and the specified BLKSIZE for the list data set. If the BLKSIZE you specify is not a multiple of 121, ICETOOL uses BLKSIZE=121. If you do not specify the BLKSIZE, ICETOOL selects the block size as directed by LISTSDB or

LISTNOSDB, if specified, or otherwise as directed by installation option SDBMSG from ICEAM2 or ICEAM4 (see *z/OS DFSORT Installation and Customization*).

Refer to "JCL restrictions" on page 542 for more information regarding the selection of ddnames.

LISTSDB or LISTNOSDB

Can be used to override the SDBMSG value for this LIST data set. LISTSDB directs ICETOOL to select the system-determined optimum block size for the LIST data set in the same way as for installation option SDBMSG=YES. LISTNOSDB directs ICETOOL to select the block size for the LIST data set in the same way as for installation option SDBMSG=NO. See the discussion of the LIST(listdd) operand previously in this section for more information on how LISTSDB or LISTNOSDB affects the LIST data set block size.

Attention: LISTSDB has no effect for SYSOUT data sets (for example, //RPT1 DD SYSOUT=*), because the system-determined optimum block size is not used for spool or dummy data sets.

DEFAULTS example

```
 DEFAULTS LIST(OPTIONS)
```

Prints, in the OPTIONS data set, the DFSORT installation defaults report. The OPTIONS output starts on a new page and might look as follows. The first few items are shown with illustrative values for ICEAM1-4 and ICETD1-4 for each of the three parts of the report.

```
 Z/OS DFSORT V2R3  MERGED PARMLIB/ICEMAC DEFAULTS     - 1 -

    * IBM-SUPPLIED DEFAULT (ONLY SHOWN IF DIFFERENT FROM THE SPECIFIED DEFAULT)

 ITEM           JCL (ICEAM1) VALUE       INV (ICEAM2) VALUE       TSO (ICEAM3) VALUE
TSOINV (ICEAM4) VALUE
 ----------     --------------------     --------------------     --------------------
--------------------
 ENABLE         NONE                     TD1                      NONE                     NONE

 ABCODE         MSG                      99                       MSG                      99
                                         * MSG                                             * MSG
 ALTSEQ         SEE BELOW                SEE BELOW                SEE BELOW                SEE
BELOW
 ARESALL        0                        0                        0                        0
 ARESINV        NOT APPLICABLE           0                        NOT APPLICABLE           0
 CFW            YES                      YES                      YES                      YES
 CHALT          YES                      YES                      NO                       NO
                * NO                     * NO
 CHECK          YES                      YES                      YES                      YES
 CINV           YES                      YES                      YES                      YES
  .
  .
  .
```

```
 Z/OS DFSORT V2R3  MERGED PARMLIB/ICEMAC DEFAULTS     - 4 -

    * IBM-SUPPLIED DEFAULT (ONLY SHOWN IF DIFFERENT FROM THE SPECIFIED DEFAULT)

 ITEM           TD1 (ICETD1) VALUE       TD2 (ICETD2) VALUE       TD3 (ICETD3) VALUE       TD4
(ICETD4) VALUE
 ----------     --------------------     --------------------     --------------------
------------------
 SUN            0600-2000                NONE                     NONE                     NONE
                * NONE
 MON            NONE                     NONE                     NONE                     NONE
 TUE            NONE                     NONE                     NONE                     NONE
 WED            NONE                     NONE                     NONE                     NONE
 THU            NONE                     NONE                     NONE                     NONE
 FRI            NONE                     NONE                     NONE                     NONE
 SAT            0600-2000                NONE                     NONE                     NONE
                * NONE

 ABCODE         99                       MSG                      MSG                      MSG
                * MSG
 ALTSEQ         SEE BELOW                SEE BELOW                SEE BELOW                SEE
BELOW
 ARESALL        0                        0                        0                        0
 ARESINV        0                        0                        0                        0
```

```
 CFW            YES                     YES                     YES                     YES
 CHALT          YES                     NO                      NO                      NO
                * NO
 CHECK          YES                     YES                     YES                     YES
 CINV           YES                     YES                     YES                     YES
  .
  .
  .

 Z/OS DFSORT V2R3  PARMLIB ICEPRMXX MEMBER OPTIONS    - 7 -

 ICEOPT COMMAND IN EFFECT: START ICEOPT,ICEPRM=(03)

 RELEASE:        2.03
 MODULE:         ICEPRML
 APAR LEVEL:     BASE

 COMPILED:       03/21/17
 AREA:           1

 JCL (ICEAM1) OPTIONS

 ITEM           VALUE                                    MEMBER
 ----------     ---------------------------------------  --------
 RESALL         40000                                    ICEPRM03
 CHALT          YES                                      ICEPRM03

 INV (ICEAM2) OPTIONS

 ITEM           VALUE                                    MEMBER
 ----------     ---------------------------------------  --------
 RESINV         40000                                    ICEPRM03
 CHALT          YES                                      ICEPRM03

 TSO (ICEAM3) OPTIONS - NONE

 TSOINV (ICEAM4) OPTIONS - NONE

 TD1 (ICETD1) OPTIONS

 ITEM           VALUE                                    MEMBER
 ----------     ---------------------------------------  --------
 CHALT          YES                                      ICEPRM03

 TD2 (ICETD2) OPTIONS - NONE

 TD3 (ICETD3) OPTIONS - NONE

 TD4 (ICETD4) OPTIONS - NONE
```

```
 Z/OS DFSORT V2R3  ICEMAC DEFAULTS                  - 8 -

 ITEM           JCL (ICEAM1) VALUE       INV (ICEAM2) VALUE       TSO (ICEAM3) VALUE
TSOINV (ICEAM4) VALUE
 ----------     --------------------     --------------------     --------------------
----------------------
 RELEASE        2.01                     2.01                     2.01                    2.01
 MODULE         ICEAM1                   ICEAM2                   ICEAM3
ICEAM4
 APAR LEVEL     BASE                     BASE                     BASE                    BASE
 COMPILED       05/15/11                 05/15/11                 05/08/11
05/15/11

 ENABLE         NONE                     TD1                      NONE                    NONE

 ABCODE         MSG                      99                       MSG                     99
 ALTSEQ         SEE BELOW                SEE BELOW                SEE BELOW               SEE
BELOW
 ARESALL        0                        0                        0                       0
 ARESINV        NOT APPLICABLE           0                        NOT APPLICABLE          0
 CFW            YES                      YES                      YES                     YES
 CHALT          NO                       NO                       NO                      NO
 CHECK          YES                      YES                      YES                     YES
 CINV           YES                      YES                      YES                     YES
  .
  .
  .

 Z/OS DFSORT V2R1  ICEMAC DEFAULTS                  - 11 -
```

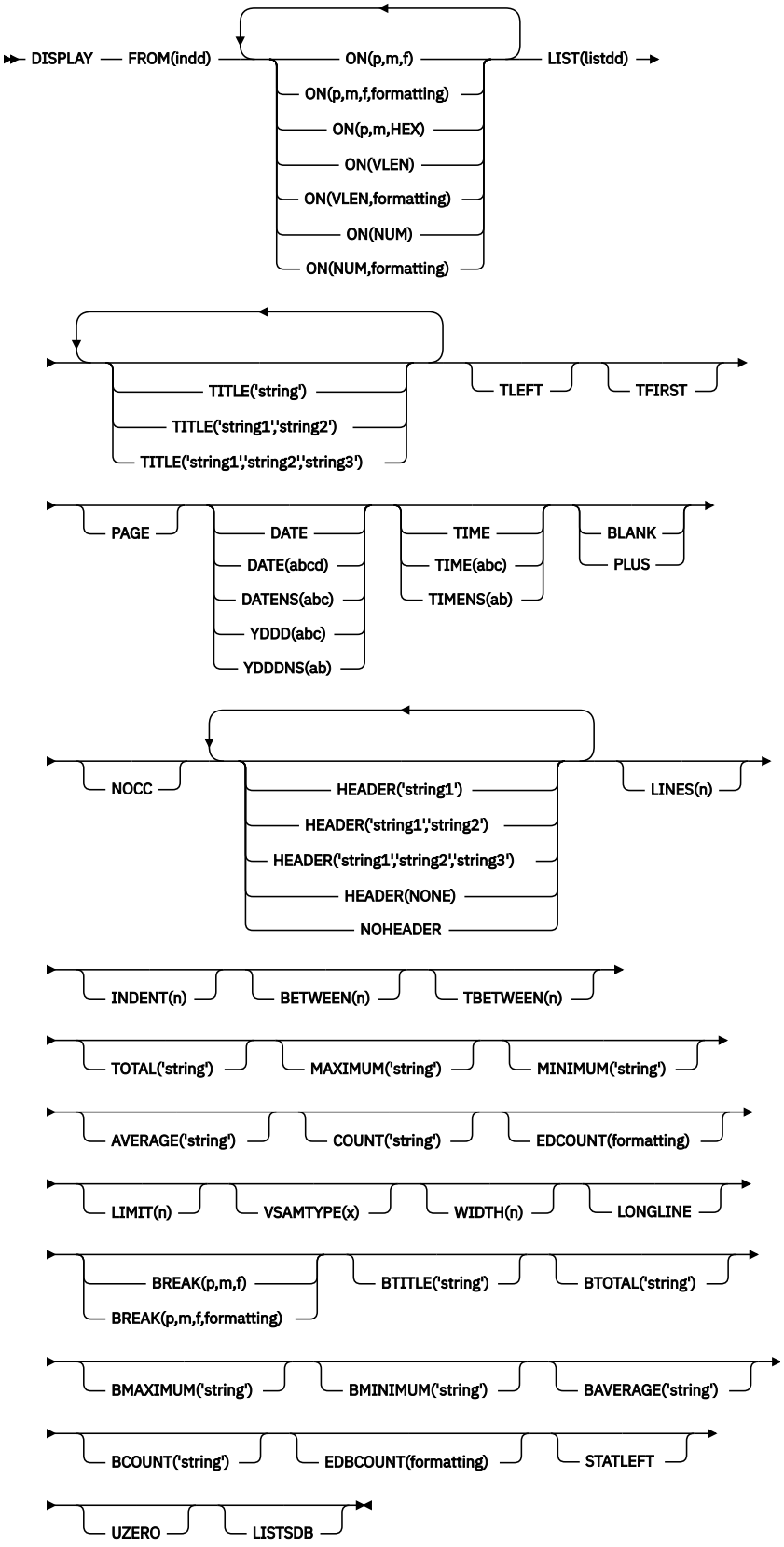
```
 ITEM           TD1 (ICETD1) VALUE       TD2 (ICETD2) VALUE       TD3 (ICETD3) VALUE       TD4
(ICETD4) VALUE
 ----------     --------------------     --------------------     --------------------
--------------------
 RELEASE        2.01                     2.01                     2.01                     2.01
 MODULE         ICETD1                   ICETD2                   ICETD3
ICETD4
 APAR LEVEL     BASE                     BASE                     BASE                     BASE
 COMPILED       05/15/11                 05/08/11                 05/08/11
05/08/11

 SUN            0600-2000                NONE                     NONE                     NONE
 MON            NONE                     NONE                     NONE                     NONE
 TUE            NONE                     NONE                     NONE                     NONE
 WED            NONE                     NONE                     NONE                     NONE
 THU            NONE                     NONE                     NONE                     NONE
 FRI            NONE                     NONE                     NONE                     NONE
 SAT            0600-2000                NONE                     NONE                     NONE

 ABCODE         99                       MSG                      MSG                      MSG
 ALTSEQ         SEE BELOW                SEE BELOW                SEE BELOW                SEE
BELOW
 ARESALL        0                        0                        0                        0
 ARESINV        0                        0                        0                        0
 CFW            YES                      YES                      YES                      YES
 CHALT          NO                       NO                       NO                       NO
 CHECK          YES                      YES                      YES                      YES
 CINV           YES                      YES                      YES                      YES
  .
  .
  .
```

The title and appropriate heading lines appear at the top of each page. The specified and IBM-supplied ALTSEQ tables are printed separately after the other items.

DISPLAY operator



Prints the values or characters of specified numeric fields (including SMF, TOD, and ETOD date and time) or character fields in a separate list data set. Simple, tailored, and sectioned reports can be produced.

From 1 to 50 fields can be specified, but the resulting list data set line length must not exceed the limit specified by the WIDTH operand or 8192 bytes if LONGLINE is specified and WIDTH is not specified, or 2048 bytes if LONGLINE and WIDTH are not specified. The record number can be printed as a special field.

DFSORT is called to copy the indd data set to ICETOOL's E35 user exit. ICETOOL uses its E35 user exit to print appropriate titles, headings and data in the list data set.

You must not supply your own DFSORT MODS, INREC, or OUTREC statement, because they would override the DFSORT statements passed by ICETOOL for this operator.

Specifying formatting items or the PLUS or BLANK operand, which can "**compress**" **the columns of output data, can enable you to include more fields in your report, up to a maximum of 50**, if your line length is limited by the character width your printer or display supports.

Simple report

You can produce a simple report by specifying just the required operands. For example, if you specify FROM and LIST operands, and ON operands for 10-byte character and 7-byte zoned decimal fields, the output in the list data set can be represented as follows:

```
(p,m,f)                (p,m,f)
characters             sddddddddddddddd
 .                                  .
 .                                  .
 .                                  .
```

A control character occupies the first byte of each list data set record. Left-justified standard headings are printed at the top of each page to indicate the contents of each column, followed by a line for each record showing the characters and numbers in the fields of that record.

The fields are printed in columns in the same order in which they are specified in the DISPLAY statement. All fields are left-justified. For numeric fields, leading zeros are printed, a - is used for the minus sign, and a + is used for the plus sign.

By default, the first column of data starts immediately after the control character, and three blanks appear between columns. The NOCC operand can be used to suppress the control character. The INDENT operand can be used to change the number of blanks before the first column of data. The BETWEEN operand can be used to change the number of blanks between columns.

The standard column widths are as follows:

- Character data: the length of the character field, or 20 bytes if the field length is less than 21 bytes
- Numeric data: 16 bytes, or 32 bytes if the numeric field is BI or FI with a length greater than 4, PD with a length greater than 8, ZD, CSF, FS, UFF or SFF with a length greater than 15, or FL.
- Record number: 15 bytes

HEADER operands can be used to change or suppress the headings. Formatting items or the PLUS or BLANK operand can be used to change the appearance of numeric fields in the report. PLUS, BLANK, and HEADER operands can be used to change the width of the columns for numeric and character fields and the justification of headings and fields.

The NOHEADER operand can be used to create list data sets containing only data records. Data sets created in this way can be processed further by other operators (for example, STATS or UNIQUE) using CH format for character values or FS format for numeric values.

The TOTAL, MAXIMUM, MINIMUM, and AVERAGE operands can be used to print statistics for numeric fields after the columns of data. Formatting items can be used to suppress the statistics for selected numeric fields. The COUNT operand can be used to print the record count after the columns of data.

Tailored report

You can tailor the output in the list data set using various operands that control title strings, date, time, page number, headings, lines per page, field formats, total, maximum, minimum, and average values for the columns of numeric data, and the record count. The optional operands can be used in many different combinations to produce a wide variety of report formats. For example, if you specify FROM, LIST, BLANK, TITLE, PAGE, DATE, TIME, HEADER and AVERAGE operands, and ON operands for 10-byte character and 7-byte zoned decimal fields, the output in the list data set can be represented as follows:

```
title        - p -        mm/dd/yy        hh:mm:ss

header         header
----------   --------
characters         sd
.                   .
.                   .
.                   .

average            sd
```

By default, a control character occupies the first byte of each list data set record. The NOCC operand can be used to suppress the control characters. The title lines (up to 3) are printed at the top of each page of the list data set. The first title line contains the elements you specify (title strings, page number, date and time) in the order in which you specify them. The second and third title lines contain the title strings you specify. By default, eight blanks appear between title elements, the title strings are centered with respect to each other, and the title lines appear on every page. The TBETWEEN(n) operand can be used to change the number of blanks between title elements. The TLEFT operand can be used to left-justify the title strings with respect to each other. The TFIRST operand can be used to only print the title lines on the first page. A blank line is printed after each title line.

Your specified headings (underlined) are printed after the title line on each page to indicate the contents of each column, followed by a line for each record showing the characters and numbers in the fields of that record. Your specified headings can be one, two or three lines. Headings for character fields are left-justified and headings for numeric fields are right-justified.

Your specified statistical lines (total, maximum, minimum, average, and count, and their associated strings) are printed for selected numeric fields and the record count, after the columns of data. The associated strings can be printed in the first column or to the left of it.

The fields are printed in columns in the same order in which they are specified in the DISPLAY statement. Character fields are left-justified and numeric fields are right-justified. For numeric fields, leading zeros are suppressed, a - is used for the minus sign, and a blank is used for the plus sign (you can specify PLUS rather than BLANK if you want a + to be used for the plus sign).

Formatting items can be used to change the appearance of individual numeric fields in the report with respect to separators, number of digits, decimal point, decimal places, signs, leading zeros, division by 10, 100, 1000, 10000, 100000, 1000000, 1000000000, 1024, 1048576 (1024*1024), or 1073741824 (1024*1024*1024), leading strings, floating strings, and trailing strings. Formatting items can also be used to insert leading or trailing strings for character fields.

The column widths are dynamically adjusted according to the length of the headings and the maximum number of bytes needed for the character or numeric data.

Sectioned report

You can produce a sectioned report (simple or tailored) by including a BREAK operand to indicate the break field used to divide the report into sections. Each set of sequential input records (which you have previously sorted on the break field and other fields, as appropriate), with the same value for the specified break field, results in a corresponding set of data lines that is treated as a section in the report.

The break field is printed at the beginning of each section. Formatting items can be used to change the appearance of numeric break fields, and to insert a string before or after character or numeric break fields.

Optional break operands can be used to modify the break title for each section (the break value is always printed as part of the break title) and to print statistics for selected numeric fields and the count, in each section. For example, if you add BTITLE, BREAK, BCOUNT, BMAXIMUM, and BMINIMUM to the operands for the tailored report discussed previously, each section of the output in the list data set starts on a new page and can be represented as follows:

```
title        - p -        mm/dd/yy        hh:mm:ss

btitle  bvalue

header         header
----------   --------
characters         sd
.                   .
.                   .
.                   .

bcount      d

bmaximum           sd

bminimum           sd
```

The final page showing the overall statistics starts on a new page and can be represented as follows:

```
title        - p -        mm/dd/yy        hh:mm:ss

header         header
----------   --------

average            sd
```

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
Specifies the ddname of the input data set to be read by DFSORT for this operation. An indd DD statement must be present and must define an input data set that conforms to the rules for DFSORT's SORTIN data set. In addition, the LRECL of the data set must be at least 4.

ON(p,m,f)
Specifies the position, length, and format of a numeric or character field to be used for this operation. '(p,m,f)' is used for the standard column heading (see HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), HEADER(NONE) and NOHEADER for alternative heading options).

By default, three blanks appear between columns. You can change the space between columns with BETWEEN(n).

p specifies the first byte of the field relative to the beginning of the input record. p is 1 for the first **data** byte of a fixed-length record and 5 for the first **data** byte of a variable-length record as illustrated in the following (RRRR represents the 4-byte record descriptor word):

```
    Fixed-length record    |    Variable-length record
    | D | A | T | A | ...  |    | R | R | R | R | D | A | T | A | ...
 p=   1   2   3   4        | p=   1   2   3   4   5   6   7   8
```

m specifies the length of the field in bytes. A field must not extend beyond position 32752 or beyond the end of a record. The maximum length for a field depends on its format.

f specifies the format of the field as shown in the following.

Table 89. Field formats of ICETOOL DISPLAY operator

Format code	Length	Description
BI	1 to 8 bytes	Unsigned binary

Table 89. Field formats of ICETOOL DISPLAY operator (continued)

Format code	Length	Description
FI	1 to 8 bytes	Signed fixed-point
PD	1 to 16 bytes	Signed packed decimal
ZD	1 to 31 bytes	Signed zoned decimal
FL	4 or 8 bytes	Signed hexadecimal floating-point converted to signed integer
CH	1 to 4000 bytes	Character
CSF or FS	1 to 32 bytes (31 digit limit)	Signed numeric with optional leading floating sign
UFF	1 to 44 bytes (31 digit limit)	Unsigned free form numeric
SFF	1 to 44 bytes (31 digit limit)	Signed free form numeric
DT1	4 bytes	SMF date interpreted as Z'yyyymmdd'
DT2	4 bytes	SMF date interpreted as Z'yyyymm'
DT3	4 bytes	SMF date interpreted as Z'yyyyddd'
DC1	8 bytes	TOD date interpreted as Z'yyyymmdd'
DC2	8 bytes	TOD date interpreted as Z'yyyymm'
DC3	8 bytes	TOD date interpreted as Z'yyyyddd'
DE1	8 bytes	ETOD date interpreted as Z'yyyymmdd'
DE2	8 bytes	ETOD date interpreted as Z'yyyymm'
DE3	8 bytes	ETOD date interpreted as Z'yyyyddd'
TM1	4 bytes	SMF time interpreted as Z'hhmmss'
TM2	4 bytes	SMF time interpreted as Z'hhmm'
TM3	4 bytes	SMF time interpreted as Z'hh'
TM4	4 bytes	SMF time interpreted as Z'hhmmssxx'
TC1	8 bytes	TOD time interpreted as Z'hhmmss'
TC2	8 bytes	TOD time interpreted as Z'hhmm'
TC3	8 bytes	TOD time interpreted as Z'hh'
TC4	8 bytes	TOD time interpreted as Z'hhmmssxx'
TE1	8 bytes	ETOD time interpreted as Z'hhmmss'
TE2	8 bytes	ETOD time interpreted as Z'hhmm'
TE3	8 bytes	ETOD time interpreted as Z'hh'
TE4	8 bytes	ETOD time interpreted as Z'hhmmssxx'
Note: See Appendix C, "Data format descriptions," on page 819 for detailed format descriptions.		

For a CSF, FS, UFF, or SFF format field:

- A maximum of 31 digits is allowed. If a value with more than 31 digits is found, ICETOOL issues an error message and terminates the operation.

For a ZD or PD format field:

- If a decimal value contains an invalid digit (A-F), ICETOOL identifies the bad value in a message and prints asterisks for that value, and for the total, maximum, minimum and average (if specified) for that field, in the list data set. If the number of bad values reaches the LIMIT for invalid decimal values, ICETOOL terminates the operation. If the LIMIT operand is not specified, a default of 200 is used for the invalid decimal value limit.
- A value is treated as positive if its sign is F, E, C, A, 8, 6, 4, 2, or 0.
- A value is treated as negative if its sign is D, B, 9, 7, 5, 3, or 1.

For an FL format field:

- The normalized or unnormalized FL (hexadecimal floating-point) value is converted to a signed integer in the range -9223372036854775808 to 9223372036854775807. The fractional part of the FL value is lost, and in some cases the signed integer may be one of a number of possible signed integers for the FL value depending on its precision. Converted values less than -9223372036854775808 are set to -9223372036854775808. Converted values greater than 9223372036854775807 are set to 9223372036854775807.
- If you are not running in z/Architecture mode, specifying an FL format field results in an error message and termination.

For a DT1, DT2 or DT3 format field:

- An invalid SMF date can result in a data exception (0C7 ABEND) or an incorrect ZD date.
- SMF date values are always treated as positive.

For a DC1, DC2, DC3, DE1, DE2, or DE3 format field:

- TOD and ETOD date values are always treated as positive.

For a TM1, TM2, TM3 or TM4 format field:

- An invalid SMF time can result in an incorrect ZD time.
- SMF time values are always treated as positive.

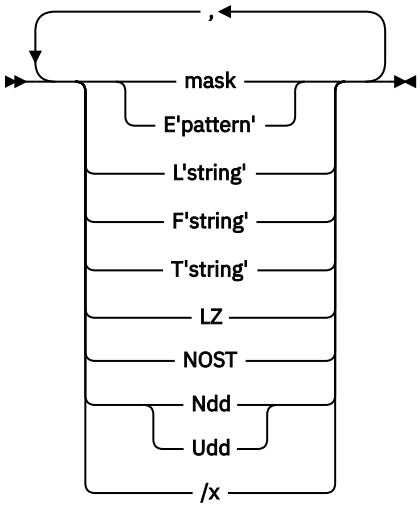
For a TC1, TC2, TC3, TC4, TE1, TE2, TE3, or TE4 format field:

- TOD and ETOD time values are always treated as positive.

ON(p,m,f,formatting)
: Specifies the position, length, and format of a numeric or character field to be used for this operation and how the data for this field is to be formatted for printing. The BLANK operand is automatically in effect.

  See ON(p,m,f) for further details.

formatting



specifies formatting items that indicate how the data for this field is to be formatted for printing. Formatting items can be specified in any order, but each item can only be specified once. Any formatting item can be specified for a numeric field, but only L'string' and T'string' can be specified for a character field.

The column width is dynamically adjusted to accommodate the maximum bytes to be inserted as a result of all formatting items specified.

mask

specifies an edit mask to be applied to the numeric data for this field. Thirty-nine pre-defined edit masks are available, encompassing many of the numeric notations throughout the world with respect to separators, decimal point, decimal places, signs, and so forth. ICETOOL edits the data according to the selected mask. If other formatting items are specified but mask is not, the default mask of A0 is applied to the data.

E'pattern' cannot be specified with a mask.

The attributes of each group of masks is shown in Table 90 on page 568.

Table 90. Attributes of edit masks

Masks	Separators	Decimal places	Positive sign	Negative sign
A0	No	0	blank	-
A1-A5	Yes	0	blank	-
B1-B6	Yes	1	blank	-
C1-C6	Yes	2	blank	-
D1-D6	Yes	3	blank	-
E1-E4	Yes	0	blank	( )
F1-F5	Yes	2	blank	( )
G1-G6	Yes	4	blank	-

Table 91 on page 569 describes the available masks and shows how the values 12345678 and -1234567 would be printed for each mask. In the pattern:

- **d** is used to represent a decimal digit (0-9)
- **w** is used to represent a leading sign that will be blank for a positive value or - for a negative value

- **x** is used to represent a trailing sign that will be blank for a positive value or - for a negative value
- **y** is used to represent a leading sign that will be blank for a positive value or (for a negative value
- **z** is used to represent a trailing sign that will be blank for a positive value or) for a negative value

Table 91. Edit mask patterns

Mask	Pattern	12345678	-1234567
A0	wddddddddddddddddddddddddddddddd	12345678	-1234567
A1	wd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd	12,345,678	-1,234,567
A2	wd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd	12.345.678	-1.234.567
A3	wd ddd ddd ddd ddd ddd ddd ddd ddd ddd ddd	12 345 678	-1 234 567
A4	wd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd	12'345'678	-1'234'567
A5	d ddd ddd ddd ddd ddd ddd ddd ddd ddd dddx	12 345 678	1 234 567-
B1	wddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd.d	1,234,567.8	-123,456.7
B2	wddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd,d	1.234.567,8	-123.456,7
B3	wddd ddd ddd ddd ddd ddd ddd ddd ddd ddd,d	1 234 567,8	-123 456,7
B4	wddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd.d	1'234'567.8	-123'456.7
B5	wddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd,d	1'234'567,8	-123'456,7
B6	ddd ddd ddd ddd ddd ddd ddd ddd ddd ddd,dx	1 234 567,8	123 456,7-
C1	wdd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd.dd	123,456.78	-12,345.67
C2	wdd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd,dd	123.456,78	-12.345,67
C3	wdd ddd ddd ddd ddd ddd ddd ddd ddd ddd,dd	123 456,78	-12 345,67
C4	wdd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd.dd	123'456.78	-12'345.67
C5	wdd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd,dd	123'456,78	-12'345,67
C6	dd ddd ddd ddd ddd d ddd ddd ddd ddd,ddx	123 456,78	12 345,67-
D1	wd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd.ddd	12,345.678	-1,234.567
D2	wd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd,ddd	12.345,678	-1.234,567
D3	wd ddd ddd ddd ddd ddd ddd ddd ddd ddd,ddd	12 345,678	-1 234,567
D4	wd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd.ddd	12'345.678	-1'234.567
D5	wd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd,ddd	12'345,678	-1'234,567
D6	d ddd ddd ddd ddd ddd ddd ddd ddd ddd,dddx	12 345,678	1 234,567-
E1	yd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,dddz	12,345,678	(1,234,567)
E2	yd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.dddz	12.345.678	(1.234.567)
E3	yd ddd ddd ddd ddd ddd ddd ddd ddd ddd dddz	12 345 678	(1 234 567)
E4	yd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'dddz	12'345'678	(1'234'567)
F1	ydd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd.ddz	123,456.78	(12,345.67)
F2	ydd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd,ddz	123.456,78	(12.345,67)
F3	ydd ddd ddd ddd ddd ddd ddd ddd ddd ddd,ddz	123 456,78	(12 345,67)
F4	ydd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd.ddz	123'456.78	(12'345.67)
F5	ydd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd,ddz	123'456,78	(12'345,67)
G1	wddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd,ddd.dddd	1,234.5678	-123.4567

Table 91. Edit mask patterns (continued)

Mask	Pattern	12345678	-1234567
G2	wddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd.ddd,dddd	1.234,5678	-123,4567
G3	wddd ddd ddd ddd ddd ddd ddd ddd ddd,dddd	1 234,5678	-123,4567
G4	wddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd.dddd	1'234.5678	-123.4567
G5	wddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd'ddd,dddd	1'234,5678	-123,4567
G6	ddd ddd ddd ddd ddd ddd ddd ddd ddd,ddddx	1 234,5678	123,4567-

If LZ is specified, leading zeros are printed. for example, +1 is shown as 0,000.01 with ON(21,6,FS,C1,LZ).

If LZ is not specified, leading zeros are suppressed except when inappropriate. For example, +1 is shown as 1 with ON(21,6,FS,A1) and as 0.01 with ON(21,6,FS,C1).

The leading sign (blank for a positive value or - for a negative value) appears to the left of the first non-suppressed digit of the formatted value. For example, -1 is shown as -1 with ON(21,6,FS,A2), as -000.001 with ON(21,6,FS,A2,LZ) and as -0.01 with ON(21,6,FS,C2).

E'pattern'
specifies an edit pattern to be applied to the numeric data for this field. E'pattern' is useful for formatting **unsigned** numeric data such as telephone numbers, dates, time-of-day, social security numbers, and so on. For example, 0123456789 is shown as (012)-345-6789 with ON(21,10,ZD,E'(999)-999-9999').

The pattern (1 to 44 characters) must be enclosed in single apostrophes. Each **9** in the pattern (up to 31) is replaced by a corresponding digit from the numeric value. Characters other than 9 in the pattern appear as specified. To include a single apostrophe (') in the pattern, specify two single apostrophes ('').

F'string' or a mask cannot be specified with E'pattern'.

When E'pattern' is specified for a field:

- Values are shown unsigned. For example, +120622 and -120622 are both shown as 12:06:22 with ON(12,7,FS,E'99:99:99').
- If the number of significant digits in a value is less than the number of 9's in the pattern, 0's are filled in on the left. For example, 1234 is shown as 0012-34 with ON(12,6,FS,E'9999-99').
- If the number of significant digits in a value is greater than the number of 9's in the pattern, digits are truncated from the left. For example, 1234567 is shown as *45:67* with ON(9,4,PD,E'*99:99*').

L'string'
specifies a leading string to appear at the beginning of the character or numeric data column for this field. For example, 'DFSORT ' is shown as '**DFSORT ' with ON(1,8,CH,L'**').

The string (1 to 10 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes ('').

F'string'
specifies a floating string to appear to the left of the first non-blank character of the formatted numeric data for this field. For example, 0001234 is shown as $12.34 with ON(9,7,ZD,C1,F'$').

The string (1 to 10 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes ('').

E'pattern' cannot be specified with F'string'.

T'string'
specifies a trailing string to appear at the end of the character or numeric data column for this field. For example, 'DFSORT ' is shown as '**DFSORT ***' with ON(1,8,CH,L'**',T'***').

The string (1 to 10 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes ('').

LZ
specifies that leading zeros are to be printed when the specified edit mask is applied to the numeric data for this field, overriding the default of suppressing leading zeros. For example, +123 is shown as 123 with ON(21,6,FS,A0), but as 000123 with ON(21,6,FS,A0,LZ).

LZ is useful for formatting numeric data, such as account numbers, for which leading zeros must be printed.

Leading zeros are always printed for E'pattern' regardless of whether or not LZ is specified.

NOST
specifies that requested statistics (TOTAL, MAXIMUM, MINIMUM, AVERAGE, BTOTAL, BMAXIMUM, BMINIMUM, BAVERAGE) are **not** to be printed for this numeric field.

Ndd or Udd
specifies the number of digits to be used for the numeric field. Ndd or Udd can be used to change the column width for numeric fields, and to prevent overflow for totals. dd specifies the number of digits and must be a **two-digit** number between 01 and 31.

The default number of digits (d) for a numeric field is the maximum number of digits for that field. For example, d is 5 for ON(1,5,ZD). If you know that your numeric field requires less than d digits, you can use a lower number of digits (dd) instead by specifying Udd, thus reducing the column width if it is determined by d. For example, ON(1,5,ZD,U03) reduces d from 5 to 3. If you want your numeric field to be displayed with more than d digits, you can use a higher number of digits (dd) instead by specifying Ndd or Udd, thus increasing the column width if it is determined by d. For example, ON(1,5,ZD,U10) increases d from 5 to 10.

The default number of digits (d) for a total is 15 if the numeric field is BI or FI with a length up to 4, PD with a length up to 8, or ZD, CSF, FS, UFF or SFF with a length up to 15. The default number of digits (d) for a total is 31 if the numeric field is BI or FI with a length greater than 4, PD with a length greater than 8, or ZD, CSF, FS, UFF or SFF with a length greater than 15.

If you know that your total requires less than d digits, you can use a lower number of digits (dd) instead by specifying Ndd or Udd for the ON field, thus reducing the column width if it is determined by d. For example, ON(1,18,ZD,U18) with TOTAL reduces d from 31 to 18. If you know that your total can overflow d digits, you can use a higher number of digits (dd) instead by specifying Ndd or Udd, thus preventing overflow and increasing the column width if it is determined by d. For example, ON(1,15,ZD,U17) with TOTAL increases d from 15 to 17.

Either Ndd or Udd can be used to set d greater than the maximum for a numeric field, but only Udd can be used to set d less than the maximum for a numeric field.

For Udd:

dd is used for d. For example:

If TOTAL and BTOTAL are not used:

- If ON(1,5,ZD) is specified, 5 digits (default for 5,ZD) are used.
- If ON(1,5,ZD,U10) is specified, 10 digits (from U10) are used..
- If ON(1,5,ZD,U03) is specified, 3 digits (from U03) are used.
- If ON(1,16,FS) is specified, 16 digits (default for 16,FS) are used.
- If ON(1,16,FS,U16) is specified, 16 digits (from U16) are used..
- If ON(1,16,FS,U15) is specified, 15 digits (from U15) are used.

If TOTAL or BTOTAL is used:

- If ON(1,5,ZD) is specified, 15 digits (default for TOTAL or BTOTAL and 5,ZD) are used.
- If ON(1,5,ZD,U10) is specified, 10 digits (from U10) are used.
- If ON(1,5,ZD,U03) is specified, 3 digits (from U03) are used.

- If ON(1,16,FS) is specified, 31 digits (default for TOTAL or BTOTAL and 16,FS) are used.
- If ON(1,16,FS,U16) is specified, 16 digits (from U16) are used.
- If ON(1,16,FS,U15) is specified, 15 digits (from U15) are used.

If you use Udd and a numeric value or total overflows dd digits, ICETOOL prints asterisks for that numeric value or total and terminates the operation. You can prevent the overflow by specifying an appropriately higher dd value for Udd. For example, if ON(1,12,ZD,U09) results in overflow, you can use ON(1,12,ZD,U10) instead.

If E'pattern' is specified, Udd is ignored, because d is determined from the pattern.

For Ndd:

If dd is greater than or equal d, dd is used. If dd is less than d, d is used. For example:

If TOTAL and BTOTAL are not used:

- If ON(1,5,ZD) is specified, 5 digits (default for 5,ZD) are used.
- If ON(1,5,ZD,N10) is specified, 10 digits (from N10) are used.
- If ON(1,5,ZD,N03) is specified, 5 digits (from 5,ZD) are used

If TOTAL or BTOTAL is used:

- If ON(1,5,ZD) is specified, 15 digits (default for TOTAL or BTOTAL and 5,ZD) are used.
- If ON(1,5,ZD,N10) is specified, 10 digits (from N10) are used.
- If ON(1,5,ZD,N03) is specified, 5 digits (from 5,ZD) are used.

If you use Ndd and a total overflows dd digits, ICETOOL prints asterisks for the total and terminates the operation. You can prevent the overflow by specifying an appropriately higher dd value for Ndd. For example, if ON(1,17,ZD,N17) with TOTAL results in overflow, you can use ON(1,17,ZD,N18) instead

If E'pattern' is specified, Ndd is ignored, because d is determined from the pattern.

/x

specifies division of the numeric data for this field before formatting. x indicates the division factor to be used as described later in this section. The resulting values are rounded down to the nearest integer. Statistics (TOTAL, MAXIMUM, MINIMUM, AVERAGE, BTOTAL, BMAXIMUM, BMINIMUM, BAVERAGE) and column widths reflect the divided numbers.

/D
specifies division by 10 before formatting. For example, -1234 is shown as -123 with ON(11,2,FI,/D).

/C
specifies division by 100 before formatting. For example, 12345 is shown as 12.3 with ON(11,2,BI,/C,B1).

/K
specifies division by 1000 before formatting. For example, -1234567890 is shown as (1 234 567) with ON(1,11,FS,/K,E3).

/DK
specifies division by 10000 (10*1000) before formatting. For example, 6213849653 is shown as 0-6213-84 with ON(31,10,FS,/DK,E'9-9999-99').

/CK
specifies division by 100000 (100*1000) before formatting. For example, 123456789012345678901234 5 is shown as 1,234,567,890,123,456.7890 with ON(21,25,ZD,G1,/CK).

/M
specifies division by 1000000 (1000*1000) before formatting. For example, -123456789 is shown as -1.23 with ON(31,10,FS,/M,C4).

/G
specifies division by 1000000000 (1000*1000*1000) before formatting. For example, 1234567898765 is shown as 1'234 with ON(15,13,ZD,A4,/G).

/KB
specifies division by 1024 before formatting. For example, 1234567890 is shown as 1 205 632 with ON(45,10,ZD,/KB,A3).

/MB
specifies division by 1048576 (1024*1024) before formatting. For example, 123456789 is shown as 117 with ON(60,9,FS,/MB).

/GB
specifies division by 1073741824 (1024*1024*1024) before formatting. For example, 1234567898765 is shown as 1,149 with ON(15,13,ZD,/GB,A1).

ON(p,m,HEX)
Specifies the position and length of a character field to be used for this operation and printed in hexadecimal format (00-FF for each byte). '(p,m,HEX)' is used for the standard column heading. See HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), HEADER(NONE), and NOHEADER for alternative heading options.

See ON(p,m,f) for a discussion of **p**.

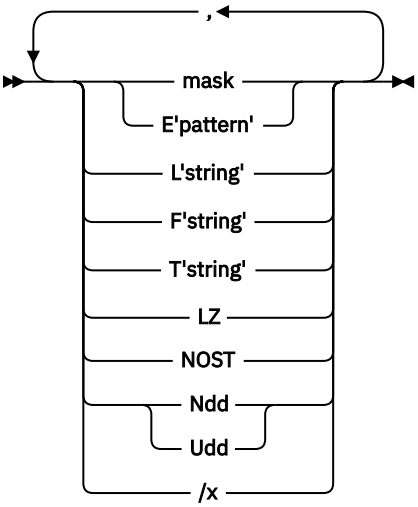
m specifies the length of the field in bytes. A field must not extend beyond position 32752 or beyond the end of a record. A field can be 1 to 2000 bytes.

ON(VLEN)
Equivalent to specifying ON(1,2,BI); a two-byte binary field starting at position 1. For variable-length records, ON(VLEN) represents the record-length for each record. ' RECORD LENGTH' is used for the standard column heading. See HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), HEADER(NONE), and NOHEADER for alternative heading options.

ON(VLEN,formatting)
Equivalent to specifying ON(1,2,BI,formatting); a two-byte binary field starting at position 1, and how the data for this field is to be formatted for printing. The BLANK operand is automatically in effect.

See ON(VLEN) for further details.

formatting



specifies formatting items that indicate how the data for this field is to be formatted for printing. Formatting items can be specified in any order, but each item can only be specified once.

The column width is dynamically adjusted to accommodate the maximum bytes to be inserted as a result of all formatting items specified.

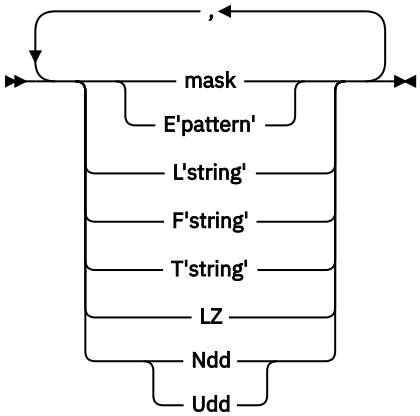
See ON(p,m,f,formatting) for a discussion of **formatting**.

ON(NUM)
Specifies that the record number is to be printed. The record number starts at 1 and is incremented by 1 for each record printed in the list data set. 'RECORD NUMBER' is used for the standard column heading. See HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), HEADER(NONE), and NOHEADER for alternative heading options.

ON(NUM,formatting)
Specifies that the record number is to be printed, and how the record number is to be formatted for printing. The BLANK operand is automatically in effect.

See ON(NUM) for further details.

formatting



specifies formatting items that indicate how the record number is to be formatted for printing. Formatting items can be specified in any order, but each item can only be specified once.

The column width is dynamically adjusted to accommodate the maximum bytes to be inserted as a result of all formatting items specified.

mask
See ON(p,m,f,formatting) for a discussion of **mask**.

E'pattern'
specifies an edit pattern to be applied to the record number. The pattern (1 to 24 characters) must be enclosed in single apostrophes. Each 9 in the pattern (up to 15) is replaced by a corresponding digit from the numeric value. Characters other than 9 in the pattern appear as specified. To include a single apostrophe (') in the pattern, specify two single apostrophes ('').

F'string' or a mask cannot be specified with E'pattern'.

When E'pattern' is specified for the record number:

- If the number of significant digits in a record number is less than the number of 9's in the pattern, 0's are filled in on the left. For example, 1234 is shown as 001234 with ON(NUM,E'999999').
- If the number of significant digits in a record number is greater than the number of 9's in the pattern, digits are truncated from the left. For example, 1234567 is shown as *4567* with ON(NUM,E'*9999*').

L'string'
See ON(p,m,f,formatting) for a discussion of **L'string'**.

F'string'
See ON(p,m,f,formatting) for a discussion of **F'string'**.

T'string'
See ON(p,m,f,formatting) for a discussion of **T'string'**.

LZ
See ON(p,m,f,formatting) for a discussion of **LZ**.

Ndd or Udd
Specifies the number of digits to be used for the record number when determining the column width. dd specifies the number of digits and must be a **two-digit** number between 01 and 15.

The default number of digits (d) for the record number is 15. If you know that your record numbers require less than 15 digits, you can use a lower number of digits (dd) instead by specifying Ndd or Udd thus reducing the column width if it is determined by d. For example, if ON(NUM,N09) or ON(NUM,U09) is specified, 9 digits (from N09 or U09) is used instead of 15 (default for record number).

If you use Ndd or Udd and the number of records overflows the number of digits used, ICETOOL terminates the operation. You can prevent the overflow by specifying an ppropriately higher dd value for Ndd or Udd. For example, if ON(NUM,N05) results in overflow, you can use ON(NUM,N06) instead.

If E'pattern' is specified, Ndd or Udd is ignored, because d is determined from the pattern.

LIST(listdd)
Specifies the ddname of the list data set to be produced by ICETOOL for this operation. A listdd DD statement must be present. ICETOOL sets the attributes of the list data set as follows:

- If NOCC is not specified, RECFM is set to FBA. If NOCC is specified, RECFM is set to FB.
- LRECL is set to one of the following:
  - If WIDTH(n) is specified, LRECL is set to n. Use WIDTH(n) if your LRECL must be set to a particular value (for example, if you use DISP=MOD to place several reports in the same data set).
  - If WIDTH(n) is not specified and NOCC is not specified, LRECL is set to 121 or to the calculated required line length if it is greater than 121 characters.
  - If WIDTH(n) is not specified and NOCC is specified, LRECL is set to 120 or to the calculated required line length if it is greater than 120 characters.

  If your LRECL does not need to be set to a particular value, you can let ICETOOL determine and set the appropriate LRECL value by not specifying WIDTH(n).
- BLKSIZE is set to one of the following:
  - The BLKSIZE from the DD statement, DSCB, or label, if it is a multiple of the LRECL used.
  - The LRECL if the BLKSIZE from the DD statement, DSCB, or label is not a multiple of the LRECL used.
  - The block size as directed by LISTSDB or LISTNOSDB if specified, or otherwise as directed by the SDBMSG installation option from ICEAM2 or ICEAM4 (see *z/OS DFSORT Installation and Customization*), if the BLKSIZE is not available from the DD statement, DSCB, or label.

Refer to "JCL restrictions" on page 542 for more information regarding the selection of ddnames.

TITLE('string')
Specifies printing of a title string in a title line. The title string is of the form:

```
string
```

Up to three TITLE operands can be specified.

The first TITLE operand, if specified, supplies a string for the first title line. Other title elements (PAGE for page number, DATE, DATE(abcd), DATENS(abc), YDDD(abc) or YDDDNS(ab) for the date, and TIME, TIME(abc) or TIMENS(ab) for the time) can also be specified for the first title line with or without a title string.

The second TITLE operand, if specified, specifies a string for the second title line. The third TITLE operand, if specified, specifies a string for the third title line.

If you specify any title elements (title string, page number, date or time), the first title line is printed at the top of each page of the list data set. It contains the title elements you specify in

the order in which you specify them. By default, eight blanks appear between title elements on the first title line. You can change the space between title elements on the first title line with TBETWEEN(n). A blank line is printed after the first title line.

If you specify a second TITLE operand, the second title line containing the specified title string is printed after the first title line. A blank line is printed after the second title line. If you specify a third TITLE operand, the third title line containing the specified title string is printed after the second title line. A blank line is printed after the third title line.

By default, the title strings in the title lines are centered with respect to each other. TLEFT can be used to left-justify the title string in the title lines with respect to each other.

By default, the title lines are printed on every page. TFIRST can be used to only print the title lines on the first page.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). Blanks at the start of the string move the text to the right. Blanks at the end of the string increase the spacing between the string and the next title element.

TITLE('string1','string2')
Specifies printing of a two part title string in a title line. The title string is of the form:

```
string1string2
```

Note: string1 and string2 can be any combination of inline constants, DFSORT symbol constants, and DFSORT symbol constants for system information (for example, S'&Sysname').

See TITLE('string') for additional informaton on title lines.

Each string (1 to 50 characters) must be enclosed in single apostrophes. The total combined length of string1 and string2 can be up to 50 characters. To include a single apostrophe (') in a string, specify two single apostrophes (''). Blanks at the start of a string move the text to the right. Blanks at the end of string1 increase the spacing between string1 and string2. Blanks at the end of string2 increase the spacing between string2 and the next title element.

TITLE('string1','string2','string3')
Specifies printing of a three part title string in a title line. The title string is of the form:

```
string1string2string3
```

Note: string1, string2 and string3 can be any combination of inline constants, DFSORT symbol constants, and DFSORT symbol constants for system information (for example, S'&Sysname').

See TITLE('string') for additional informaton on title lines.

Each string (1 to 50 characters) must be enclosed in single apostrophes. The total combined length of string1, string2 and string3 can be up to 50 characters. To include a single apostrophe (') in a string, specify two single apostrophes (''). Blanks at the start of a string move the text to the right. Blanks at the end of string1 increase the spacing between string1 and string2. Blanks at the end of string2 increase the spacing between string2 and string3. Blanks at the end of string3 increase the spacing between string3 and the next title element.

TLEFT
Specifies that the title strings in each title line are to be left justified with respect to each other (overriding the default of centering the title strings with respect to each other). See TITLE('string') for additional informaton on title lines.

TFIRST
Specifies that the title lines are only to appear on the first page of the report (overriding the default of having the title lines appear on every page of the report). See TITLE('string') for additional informaton on title lines.

PAGE
Specifies printing of the page number in the first title line. The page number is printed in the form - p - where p is in decimal with no leading zeros. The page number is 1 for the first page and is incremented by 1 for each subsequent page. See TITLE('string') for additional information on the first title line.

DATE
Specifies printing of the date in the first title line. The date is printed in the form mm/dd/yy where mm is the month, dd is the day, and yy is the year. DATE is equivalent to specifying DATE(MDY/). See TITLE('string') for additional information on the first title line.

DATE(abcd)
Specifies printing of the date in the first title line. The date is printed in the form 'adbdc' according to the specified values for abc and d. For example, on March 29, 2002, DATE(4MD-) would produce '2002-03-29' and DATE(MDY.) would produce '03.29.02'. See TITLE('string') for additional information on the first title line.

abc can be any combination of M, D, and Y or 4 (each specified once) where M represents the month (01-12), D represents the day (01-31), Y represents the last two digits of the year (for example, 02), and 4 represents the four digits of the year (for example, 2002).

d can be any character and is used to separate the month, day, and year.

DATENS(abc)
Specifies printing of the date in the first title line. The date is printed in the form 'abc' according to the specified values for abc. For example, on March 29, 2002, DATENS(4MD) would produce '20020329' and DATENS(MDY) would produce '032902'. See TITLE('string') for additional information on the first title line.

abc can be any combination of M, D, and Y or 4 (each specified once) where M represents the month (01-12), D represents the day (01-31), Y represents the last two digits of the year (for example, 02), and 4 represents the four digits of the year (for example, 2002).

YDDD(abc)
specifies printing of the date in the first title line. The date is printed in the form 'acb' according to the specified values for ab and c. For example, on April 7, 2004, YDDD(DY-) would produce '098-04' and YDDD(4D/) would produce '2004/098'. See TITLE('string') for additional information on the first title line.

ab can be any combination of D, and Y or 4 (each specified once) where D represents the day of the year (001-366), Y represents the last two digits of the year (for example, 04), and 4 represents the four digits of the year (for example, 2004).

c can be any character and is used to separate the month, day and year.

YDDDNS(ab)
specifies printing of the date in the first title line. The date is printed in the form 'ab' according to the specified values for ab. For example, on April 7, 2004, YDDDNS(DY) would produce '09804' and YDDD(4D) would produce '2004098'. See TITLE('string') for additional information on the first title line.

ab can be any combination of D, and Y or 4 (each specified once) where D represents the day of the year (001-366), Y represents the last two digits of the year (for example, 04), and 4 represents the four digits of the year (for example, 2004).

TIME
Specifies printing of the time in the first title line. The time is printed in the form hh:mm:ss where hh is hours, mm is minutes and ss is seconds. TIME is equivalent to specifying TIME(24:). See TITLE('string') for additional information on the first title line.

TIME(abc)
Specifies printing of the time in the first title line. The time is printed in the form 'hhcmmcss xx' according to the specified value for ab and c. For example, at 08:25:13 pm, TIME=(24:) would produce '20:25:13' and TIME=(12.) would produce '08.25.13 pm'. See TITLE('string') for additional information on the first title line.

ab can be:

- 12 to indicate 12-hour time. hh (hours) is 01-12, mm (minutes) is 00-59, ss (seconds) is 00-59 and xx is am or pm.
- 24 to indicate 24-hour time. hh (hours) is 00-23, mm (minutes) is 00-59, ss (seconds) is 00-59 and xx is not included.

c can be any character and is used to separate the hours, minutes, and seconds.

TIMENS(ab)
Specifies printing of the time in the first title line. The time is printed in the form 'hhmmss xx' according to the specified value for ab. For example, at 08:25:13 pm, TIMENS=(24) would produce '202513' and TIMENS=(12) would produce '082513 pm'. See TITLE('string') for additional information on the first title line.

ab can be:

- 12 to indicate 12-hour time. hh (hours) is 01-12, mm (minutes) is 00-59, ss (seconds) is 00-59 and xx is am or pm.
- 24 to indicate 24-hour time. hh (hours) is 00-23, mm (minutes) is 00-59, ss (seconds) is 00-59 and xx is not included.

BLANK
Specifies an alternate format for printing character and numeric data as follows:

- Numeric values for which formatting is not specified are printed with blank for plus sign, - for minus sign and no leading zeros (overriding the default of + for plus sign and leading zeros).

  Numeric values are thus displayed as:

  – d...d for positive values (blank sign immediately to the left of the digits and no leading zeros)
  – -d...d for negative values (- sign immediately to the left of the digits and no leading zeros)
- Column widths are dynamically adjusted according to the length of the headings and the maximum number of bytes needed for the character or numeric data
- Headings and data for numeric fields are right-justified (overriding the default of left-justified headings and data for numeric fields)

PLUS
Specifies an alternate format for printing character and numeric data as follows:

- Numeric values for which formatting is not specified are printed with + for plus sign, - for minus sign and no leading zeros (overriding the default of leading zeros).

  Numeric values are thus displayed as:

  – +d...d for positive values (- sign immediately to the left of the digits and no leading zeros)
  – -d...d for negative values (- sign immediately to the left of the digits and no leading zeros)
- Column widths are dynamically adjusted according to the length of the headings and the maximum number of bytes needed for the character or numeric data
- Headings and data for numeric fields are right-justified (overriding the default of left-justified headings and data for numeric fields)

For ON(NUM), PLUS is treated as BLANK.

NOCC
Specifies that carriage control characters are not to be included in the lines of the list data set (overriding the default of using a carriage control character as the first byte of each line). A blank line is used instead of a page eject control character to separate elements of the report (such as sections).

The RECFM of the list data set is set to FB.

The LRECL of the list data set will not include a byte for the carriage control character. If the line length is less than or equal to 120 bytes, the LRECL will be set to 120. If the line length is greater than 120 bytes, the LRECL will be set to the line length. The maximum line length is 2047 bytes if LONGLINE is not specified, or 8191 bytes if LONGLINE is specified.

If WIDTH(n) is specified, n can be 121 to 8191.

HEADER('string1')

Specifies a heading to be printed for the corresponding ON field. The specified string is used instead of the standard column heading for the corresponding ON field. (ON fields and HEADER operands correspond one-for-one according to the order in which they are specified; that is, the first HEADER operand corresponds to the first ON field, the second HEADER operand corresponds to the second ON field, and so on.)

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). If the string length is greater than the column width for the corresponding ON field, the column width is increased to the string length.

The heading is left-justified for character fields or right-justified for numeric fields and is underlined with hyphens for the entire column width (overriding the default of left-justified, non-underlined headings). Character values are left-justified and numeric values are right-justified (overriding the default of left-justified field values).

A null string ('') or blank string (' ') may be used to set string1 to blanks.

Blanks at the start or end of a heading string may alter the justification of the heading or the width of the column.

If HEADER('string1') is used for any ON field, HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), or HEADER(NONE) must be used for each ON field.

HEADER('string1','string2')

Specifies a heading to be printed for the corresponding ON field. A two-line heading is used with string1 on line1 and string2 on line2. A null string ('') or blank string (' ') may be used to set string1 or string2 to blanks. A comma (,) may also be used to set string1 to blanks. For example, HEADER(,'string1') results in blanks for this heading in line1 and string1 for this heading in line2.

If HEADER('string1','string2') is used for any ON field, HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3') or HEADER(NONE) must be used for each ON field.

If a HEADER('string1','string2') operand is specified, a two-line heading is also used for any HEADER('string1') operands you specify, with string1 for that heading on line1 and blanks for that heading on line2. HEADER(,'string1') can be used to put blanks for that heading on line1 and string1 for that heading on line2.

See HEADER('string1') for more details on the HEADER operand.

HEADER('string1','string2','string3')

Specifies a heading to be printed for the corresponding ON field. A three-line heading is used with string1 on line1, string2 on line2 and string3 on line3. A null string ('') or blank string (' ') may be used to set string1, string2 or string3 to blanks. A comma (,) may also be used to set string1 or string2 to blanks. For example, HEADER(,,'string1') results in blanks for this heading in line1 and line2 and string1 for this heading in line3.

If HEADER('string1','string2','string3') is used for any ON field, HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3') or HEADER(NONE) must be used for each ON field.

If a HEADER('string1','string2','string3') operand is specified:

- a three-line heading is also used for any HEADER('string1') operands you specify, with string1 for that heading on line1 and blanks for that heading on line2 and line3. HEADER(,,'string1') can be used to put blanks for that heading on line1 and line2 and string1 for that heading on line3.
- a three-line heading is also used for any HEADER('string1','string2') operands you specify, with string1 for that heading on line1, string2 for that heading on line2 and blanks for that heading on line3. HEADER('string1','string2') can be used to put blanks for that heading on line1, string1 for that heading on line2 and string2 for that heading on line3.

See HEADER('string1') for more details on the HEADER operand.

HEADER(NONE)
Specifies that a heading is not to be printed for the corresponding ON field. The standard column heading for the corresponding ON field is suppressed.

If HEADER(NONE) is used for any ON field, HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), or HEADER(NONE) must be used for each ON field. Specifying HEADER(NONE) for every ON field is equivalent to specifying NOHEADER.

NOHEADER

Specifies that headings for ON fields are not to be printed (overriding the default of printing standard headings for ON fields).

If NOHEADER is used, it must be specified only once and HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), and HEADER(NONE) must not be used.

If NOHEADER is specified without any TITLE, DATE, TIME, or PAGE operands, the resulting list data set contains only data records. Data sets created in this way can be processed further by other operators (for example, STATS or UNIQUE) using CH for character values or FS for numeric values.

LINES(n)
Specifies the number of lines per page for the list data set (overriding the default of 58). n must be greater than 9, but less than 1000.

INDENT(n)
Specifies the number of blanks to be used to indent the report (overriding the default of 0). n can be from 0 to 50. For example, if INDENT(n) is not specified, the report starts in column 2 (after the control character), whereas if INDENT(10) is specified, the report starts in column 12 (after the control character and 10 blanks).

BETWEEN(n)
Specifies the number of blanks to be used between the columns of data (overriding the default of 3). n can be from 0 to 50. For example, if BETWEEN(n) is not specified, three blanks appear between columns, whereas if BETWEEN(7) is specified, seven blanks appear between columns.

TBETWEEN(n)
Specifies the number of blanks to be used between title elements (title strings, page number, date and time) in the first title line (overriding the default of 8). n can be from 0 to 50. For example, if TBETWEEN(n) is not specified, eight blanks appear between the title strings and page number in the first title line, whereas if TBETWEEN(4) is specified, four blanks appear between the title strings and page number in the first title line.

TOTAL('string')
Specifies an overall TOTAL line is to be printed after the rows of data for the report. The specified string is printed starting at the indent column of the overall TOTAL line, followed by the overall total for each numeric data column. If STATLEFT is specified, the string is printed to the left of the first column of data with the totals on the same line as the string. If STATLEFT is not specified, the string is printed in the first column of data with the totals on the same line as the string, or on the next line, as appropriate. A blank line is printed before the overall TOTAL line.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, specify TOTAL('') using two single apostrophes.

The overall total for each numeric ON field is printed in the format (formatting, PLUS, BLANK, or standard) you specify. The total for a specific numeric field is suppressed if the NOST formatting item is specified for that field. Totals are printed for ON(VLEN) fields, but not for ON(NUM) fields.

The default number of digits (d) for a total is 15 if the ON field is BI or FI with a length up to 4, PD with a length up to 8, or ZD, CSF, FS, UFF or SFF with a length up to 15. The default number of digits (d) for a total is 31 if the ON field is BI or FI with a length greater than 4, PD with a length greater than 8, ZD, CSF, FS, UFF or SFF with a length greater than 15, or FL. By default, column widths are adjusted to allow for a maximum of a sign and d digits for the totals. If the overall total for an ON field overflows d digits, ICETOOL prints asterisks for the overall total for that field and terminates the operation.

You can use the Ndd or Udd formatting item to decrease or increase the number of digits used for a total. If you use Ndd or Udd and the overall total for an ON field overflows dd digits, ICETOOL prints asterisks for the overall total for that field and terminates the operation.

You can prevent overflow by specifying an appropriate dd value for Ndd or Udd. For example, if ON(1,15,ZD) with TOTAL overflows the default of 15 digits, you can specify ON(1,15,ZD,U16) to prevent overflow. If ON(1,15,ZD,U16) still results in overflow, you can specify ON(1,15,ZD,U17), and so on.

Either Ndd or Udd can be used to set the number of digits greater than the maximum for a numeric field, but only Udd can be used to set the number of digits less than the maximum for a numeric field.

See the discussion of Ndd or Udd under ON(p,m,f,formatting) for more details on using Ndd or Udd with TOTAL.

The TOTAL, MAXIMUM, MINIMUM, AVERAGE, and COUNT lines are printed in the order in which you specify them.

MAXIMUM('string')

Specifies an overall MAXIMUM line is to be printed after the rows of data for the report. The specified string is printed starting at the indent column of the overall MAXIMUM line, followed by the overall maximum for each numeric data column. If STATLEFT is specified, the string is printed to the left of the first column of data with the maximums on the same line as the string. If STATLEFT is not specified, the string is printed in the first column of data with the maximums on the same line as the string, or on the next line, as appropriate. A blank line is printed before the overall MAXIMUM line.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, specify MAXIMUM('') using two single apostrophes.

The overall maximum for each numeric ON field is printed in the format (formatting, PLUS, BLANK, or standard) you specify. The maximum for a specific numeric field is suppressed if the NOST formatting item is specified for that field. Maximums are printed for ON(VLEN) fields, but not for ON(NUM) fields.

The TOTAL, MAXIMUM, MINIMUM, AVERAGE, and COUNT lines are printed in the order in which you specify them.

MINIMUM('string')

Specifies an overall MINIMUM line is to be printed after the rows of data for the report. The specified string is printed starting at the indent column of the overall MINIMUM line, followed by the overall minimum for each numeric data column. If STATLEFT is specified, the string is printed to the left of the first column of data with the minimums on the same line as the string. If STATLEFT is not specified, the string is printed in the first column of data with the minimums on the same line as the string, or on the next line, as appropriate. A blank line is printed before the overall MINIMUM line.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, specify MINIMUM('') using two single apostrophes.

The overall minimum for each numeric ON field is printed in the format (formatting, PLUS, BLANK, or standard) you specify. The minimum for a specific numeric field is suppressed if the NOST formatting item is specified for that field. Minimums are printed for ON(VLEN) fields, but not for ON(NUM) fields.

The TOTAL, MAXIMUM, MINIMUM, AVERAGE, and COUNT lines are printed in the order in which you specify them.

AVERAGE('string')

Specifies an overall AVERAGE line is to be printed after the rows of data for the report. The specified string is printed starting at the indent column of the overall AVERAGE line, followed by the overall average for each numeric data column. If STATLEFT is specified, the string is printed to the left of the first column of data with the averages on the same line as the string. If STATLEFT is not specified, the string is printed in the first column of data with the averages on the same line as the string, or on the next line, as appropriate. A blank line is printed before the overall AVERAGE line.

The overall average (or mean) is calculated by dividing the overall total by the number of values in the report and rounding down to the nearest integer (examples: 23 / 5 = 4, -23 / 5 = -4).

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes ('). To suppress printing of a string, specify AVERAGE('') using two single apostrophes.

The overall average for each numeric ON field is printed in the format (formatting, PLUS, BLANK, or standard) you specify. The average for a specific numeric field is suppressed if the NOST formatting item is specified for that field. Averages are printed for ON(VLEN) fields, but not for ON(NUM) fields.

You can use the Ndd or Udd formatting item to decrease or increase the number of digits used for a total. If the overall total for an ON field overflows d digits, ICETOOL prints asterisks for the overall average for that field and terminates the operation. You can prevent overflow by specifying an appropriate dd value for Ndd or Udd. For example, if ON(1,15,ZD) with AVERAGE overflows the default of 15 digits for the total, you can specify ON(1,15,ZD,U16) to prevent overflow.

See the discussion of Ndd or Udd under ON(p,m,f,formatting) for more details on using Ndd or Udd.

The TOTAL, MAXIMUM, MINIMUM, AVERAGE, and COUNT lines are printed in the order in which you specify them.

COUNT('string')

Specifies an overall COUNT line is to be printed after the rows of data for the report. The specified string is printed starting at the indent column of the overall COUNT line, followed by the overall count of data records. If STATLEFT is specified, the string is printed to the left of the first column of data. If STATLEFT is not specified, the string is printed in the first column of data. The count is printed on the same line as the string. A blank line is printed before the overall COUNT line.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, specify COUNT('') using two single apostrophes.

The count is printed in the format (PLUS, BLANK, or standard) you specify. EDCOUNT(formatting) can be used to apply formatting items to the count. The default number of digits (d) for the count is 15.

The TOTAL, MAXIMUM, MINIMUM, AVERAGE, and COUNT lines are printed in the order in which you specify them.

EDCOUNT(formatting)
Specifies how the overall count is to be formatted for printing. Formatting items can be specified in any order, but each item can only be specified once. EDCOUNT can only be specified if COUNT('string') is specified.

mask
See ON(p,m,f,formatting) for a discussion of **mask**.

E'pattern'
specifies an edit pattern to be applied to the count. The pattern (1 to 24 characters) must be enclosed in single apostrophes. Each 9 in the pattern (up to 15) is replaced by a corresponding digit from the count. Characters other than 9 in the pattern appear as specified. To include a single apostrophe (') in the pattern, specify two single apostrophes (''). F'string' or a mask cannot be specified with E'pattern'.

When E'pattern' is specified for the count:

- If the number of significant digits in the count is less than the number of 9's in the pattern, 0's are filled in on the left. For example, 1234 is shown as 001234 with EDCOUNT(E'999999').
- If the number of significant digits in the count is greater than the number of 9's in the pattern, digits are truncated from the left. For example, 1234567 is shown as *4567* with EDCOUNT(E'*9999*').

L'string'
See ON(p,m,f,formatting) for a discussion of **L'string'**.

F'string'
See ON(p,m,f,formatting) for a discussion of **F'string'**.

T'string'
See ON(p,m,f,formatting) for a discussion of **T'string'**.

LZ
See ON(p,m,f,formatting) for a discussion of **LZ**.

Udd
specifies the number of digits to be used for the count. dd specifies the number of digits and must be a two-digit number between 01 and 15. The default number of digits (d) for the count is 15.

If you know that your count requires less than 15 digits, you can use a lower number of digits (dd) instead by specifying Udd. For example, if EDCOUNT(U09) is specified, 9 digits (from U09) is used instead of 15 (default for the count).

If you use Udd and the count overflows the number of digits used, ICETOOL terminates the operation. You can prevent the overflow by specifying an appropriately higher dd value for Udd. For example, if EDCOUNT(U05) results in overflow, you can use EDCOUNT(U06) instead.

If E'pattern' is specified, Udd is ignored, because d is determined from the pattern.

LIMIT(n)
Specifies a limit for the number of invalid decimal values (overriding the default of 200). If n invalid decimal values are found, ICETOOL terminates the operation. n can be 1 to 15 decimal digits, but must be greater than 0.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

WIDTH(n)
Specifies the line length and LRECL you want ICETOOL to use for your list data set. If NOCC is not specified, n can be from 121 to 8192. If NOCC is specified, n can be from 121 to 8191.

ICETOOL always calculates the line length required to print all titles, headings, data, and statistics and uses it as follows:

- If WIDTH(n) is specified and the calculated line length is less than or equal to n, ICETOOL sets the line length and LRECL to n.
- If WIDTH(n) is specified and the calculated line length is greater than n, ICETOOL issues an error message and terminates the operation.
- If WIDTH(n) is not specified, NOCC is not specified, and the calculated line length is less than or equal to 121, ICETOOL sets the line length and LRECL to 121.
- If WIDTH(n) is not specified, NOCC is specified, and the calculated line length is less than or equal to 120, ICETOOL sets the line length and LRECL to 120.
- If WIDTH(n) is not specified, NOCC is specified, and the calculated line length is between 121 and 2047, or between 121 and 8191 if LONGLINE is specified, ICETOOL sets the line length and LRECL to the calculated line length.
- If WIDTH(n) is not specified, NOCC is not specified, and the calculated line length is between 122 and 2048, or between 122 and 8192 if LONGLINE is specified, ICETOOL sets the line length and LRECL to the calculated line length.
- If WIDTH(n) is not specified, NOCC is not specified, and the calculated line length is greater than 2048, or greater than 8192 if LONGLINE is specified, ICETOOL issues an error message and terminates the operation.
- If WIDTH(n) is not specified, NOCC is specified, and the calculated line length is greater than 2047, or greater than 8191 if LONGLINE is specified, ICETOOL issues an error message and terminates the operation.

Use WIDTH(n) if your LRECL must be set to a particular value (for example, if you use DISP=MOD to place several reports in the same data set) or if you want to ensure that the line length for your report does not exceed a specific maximum (for example, 133 bytes). Otherwise, you can let ICETOOL calculate and set the appropriate line length and LRECL by not specifying WIDTH(n).

LONGLINE
Specifies the calculated line length can be a maximum of 8191 bytes if NOCC is specified, or a maximum of 8192 bytes if NOCC is not specified (overriding the defaults of 2047 bytes if NOCC is specified or 2048 bytes if NOCC is specified). Use LONGLINE if you do not specify WIDTH(n) and your calculated line length is larger than the default.

BREAK(p,m,f)
Specifies a numeric or character break field to be used to divide the report into sections. Each set of sequential input records, with the same value for the specified break field, results in a corresponding set of data lines that is treated as a section in the report. The DISPLAY operator should be preceded by a SORT operator (or another application) that sorts the break field and any other appropriate fields in the desired sequence for the report.

Each section starts on a new page. Each page of a section includes a break title line showing the break value for the section. Numeric break values are printed with blank for plus sign, - for minus sign, and no leading zeros. BTITLE can be used to specify a string to appear in the break title line. The break value and break title string appear in the order in which you specify BREAK and BTITLE. Two blanks appear between break title elements. A blank line is printed after the break title line.

BTOTAL, BMAXIMUM, BMINIMUM, BAVERAGE, and BCOUNT can be used to produce break statistics for each numeric ON field and for the break count-for example, the maximum of the values in the section for ON(5,3,ZD) and the maximum of the values in the section for ON(22,2,BI). The break statistics for each section are printed at the end of the section (on one or more pages that include the break title). TOTAL, MAXIMUM, MINIMUM, AVERAGE, and COUNT can be used to produce overall statistics for each numeric ON field and for the overall count-for example, the maximum of the values in the report for ON(5,3,ZD) and the maximum of the values in the report for ON(22,2,BI). The overall statistics for each section are printed at the end of the report (on a separate page that does not include the break title).

See ON(p,m,f) for a discussion of **p** and **m**.

f specifies the format of the field as shown for ON(p,m,f).

Note: An FL (hexadecimal floating-point) field can be specified for ON, but not for BREAK.

For a CSF, FS, UFF, or SFF format break field:

- A maximum of 31 digits is allowed. If a value with more than 31 digits is found, ICETOOL issues an error message and terminates the operation.

For a ZD or PD format break field:

- If a decimal value with an invalid digit (A-F) is found, ICETOOL issues an error message and terminates the operation.
- A value is treated as positive if its sign is F, E, C, A, 8, 6, 4, 2, or 0.
- A value is treated as negative if its sign is D, B, 9, 7, 5, 3, or 1.

For a DT1, DT2 or DT3 format field:

- An invalid SMF date can result in a data exception (0C7 ABEND) or an incorrect ZD date.
- SMF date values are always treated as positive.

For a DC1, DC2, DC3, DE1, DE2, or DE3 format field:

- TOD and ETOD date values are always treated as positive.

For a TM1, TM2, TM3 or TM4 format field:

- An invalid SMF time can result in an incorrect ZD time.
- SMF time values are always treated as positive.

For a TC1, TC2, TC3, TC4, TE1, TE2, TE3, or TE4 format field:

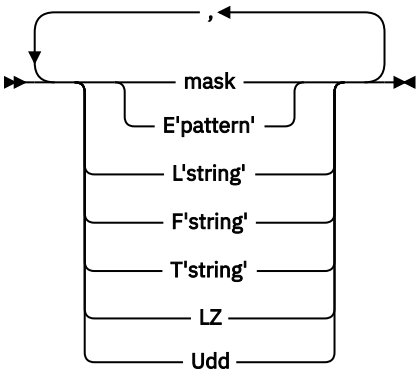
- TOD and ETOD time values are always treated as positive.

BREAK(p,m,f,formatting)

Specifies a numeric or character break field to be used to divide the report into sections, and how the data for this field is to be formatted for printing.

See BREAK(p,m,f) for further details.

formatting



specifies formatting items that indicate how the record number is to be formatted for printing. Formatting items can be specified in any order, but each item can only be specified once. Any formatting item can be specified for a numeric break field, but only L'string' and T'string' can be specified for a character break field.

mask

See ON(p,m,f,formatting) for a discussion of **mask**.

E'pattern'

See ON(p,m,f,formatting) for a discussion of **E'pattern'**.

L'string'

See ON(p,m,f,formatting) for a discussion of **L'string'**.

F'string'
See ON(p,m,f,formatting) for a discussion of **F'string'**.

T'string'
See ON(p,m,f,formatting) for a discussion of **T'string'**.

LZ
See ON(p,m,f,formatting) for a discussion of **LZ**.

Udd
specifies the number of digits to be used for a numeric break field. Udd can be used to change the column width for numeric break fields. dd specifies the number of digits and must be a two-digit number between 01 and 31.

The default number of digits (d) for a numeric break field is the maximum number of digits for that field. For example, d is 8 for BREAK(1,8,ZD). If you know that your break field requires less than d digits, you can use a lower number of digits (dd) instead by specifying Udd, thus reducing the break field width. For example, BREAK(1,8,ZD,U06) reduces d from 8 to 6. If you want your break field to be displayed with more than d digits, you can use a higher number of digits (dd) instead by specifying Udd, thus increasing the field width. For example, BREAK(1,8,ZD,U11) increases d from 8 to 11.

BTITLE('string')
Specifies a string to appear in the break title line printed for each page of a section. BTITLE can only be specified if BREAK is specified. The break value and break title string appear in the order in which you specify BREAK and BTITLE. Two blanks appear between break title elements. A blank line is printed after the break title line.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). Blanks at the start of the string move the text to the right. Blanks at the end of the string increase the spacing between the string and the break value if BTITLE is specified before BREAK.

BTOTAL('string')
Specifies a break TOTAL (BTOTAL) line is to be printed after the rows of data for each section. BTOTAL can only be specified if BREAK is specified. The specified string is printed starting at the indent column of the break TOTAL line, followed by the break total for each numeric data column.If STATLEFT is specified, the string is printed to the left of the first column of data with the totals on the same line as the string. If STATLEFT is not specified, the string is printed in the first column of data with the totals on the same line as the string, or on the next line, as appropriate. A blank line is printed before the break TOTAL line.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, specify BTOTAL('') using two single apostrophes.

The break total for each numeric ON field is printed in the format (formatting, PLUS, BLANK, or standard) you specify. The total for a specific numeric field is suppressed if the NOST formatting item is specified for that field. Totals are printed for ON(VLEN) fields, but not for ON(NUM) fields.

The default number of digits (d) for a break total is 15 if the ON field is BI or FI with a length up to 4, PD with a length up to 8, or ZD, CSF, FS, UFF or SFF with a length up to 15. The default number of digits (d) for a break total is 31 if the ON field is BI or FI with a length greater than 4, PD with a length greater than 8, or ZD, CSF, FS, UFF or SFF with a length greater than 15. By default, column widths are adjusted to allow for a maximum of a sign and d digits for the totals. If the break total for an ON field overflows d digits, ICETOOL prints asterisks for the break total for that field and terminates the operation.

You can use the Ndd or Udd formatting item to decrease or increase the number of digits used for a break total. If you use Ndd or Udd and the break total for an ON field overflows dd digits, ICETOOL prints asterisks for the break total for that field and terminates the operation.

You can prevent overflow by specifying an appropriate dd value for Ndd or Udd. For example, if ON(1,15,ZD) with BTOTAL overflows the default of 15 digits, you can specify ON(1,15,ZD,U16) to

prevent overflow. If ON(1,15,ZD,U16) still results in overflow, you can specify ON(1,15,ZD,U17), and so on.

Either Ndd or Udd can be used to set the number of digits greater than the maximum for a numeric field, but only Udd can be used to set the number of digits less than the maximum for a numeric field.

See the discussion of Ndd or Udd under ON(p,m,f,formatting) for more details on using Ndd or Udd with BTOTAL.

The BTOTAL, BMAXIMUM, BMINIMUM, BAVERAGE, and BCOUNT lines are printed in the order in which you specify them.

BMAXIMUM('string')
Specifies a break MAXIMUM line is to be printed after the rows of data for each section. BMAXIMUM can only be specified if BREAK is specified. The specified string is printed starting at the indent column of the break MAXIMUM line, followed by the break maximum for each numeric data column. If STATLEFT is specified, the string is printed to the left of the first column of data with the maximums on the same line as the string. If STATLEFT is not specified, the string is printed in the first column of data with the maximums on the same line as the string, or on the next line, as appropriate. A blank line is printed before the break MAXIMUM line.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, specify BMAXIMUM('') using two single apostrophes.

The break maximum for each numeric ON field is printed in the format (formatting, PLUS, BLANK, or standard) you specify. The maximum for a specific numeric field is suppressed if the NOST formatting item is specified for that field. Maximums are printed for ON(VLEN) fields, but not for ON(NUM) fields.

The BTOTAL, BMAXIMUM, BMINIMUM, BAVERAGE, and BCOUNT lines are printed in the order in which you specify them.

BMINIMUM('string')
Specifies a break MINIMUM line is to be printed after the rows of data for each section. BMINIMUM can only be specified if BREAK is specified. The specified string is printed starting at the indent column of the break MINIMUM line, followed by the break minimum for each numeric data column. If STATLEFT is specified, the string is printed to the left of the first column of data with the minimums on the same line as the string. If STATLEFT is not specified, the string is printed in the first column of data with the minimums on the same line as the string, or on the next line, as appropriate. A blank line is printed before the break MINIMUM line.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, specify BMINIMUM('') using two single apostrophes.

The break minimum for each numeric ON field is printed in the format (formatting, PLUS, BLANK, or standard) you specify. The minimum for a specific numeric field is suppressed if the NOST formatting item is specified for that field. Minimums are printed for ON(VLEN) fields, but not for ON(NUM) fields.

The BTOTAL, BMAXIMUM, BMINIMUM, BAVERAGE, and BCOUNT lines are printed in the order in which you specify them.

BAVERAGE('string')
Specifies a break AVERAGE line is to be printed after the rows of data for each section. BAVERAGE can only be specified if BREAK is specified. The specified string is printed starting at the indent column of the break AVERAGE line, followed by the break average for each numeric data column. If STATLEFT is specified, the string is printed to the left of the first column of data with the averages on the same line as the string. If STATLEFT is not specified, the string is printed in the first column of data with the averages on the same line as the string, or on the next line, as appropriate. A blank line is printed before the break AVERAGE line.

The break average (or mean) is calculated by dividing the break total by the number of values in the section and rounding down to the nearest integer (examples: 23 / 5 = 4, -23 / 5 = -4).

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, specify BAVERAGE('') using two single apostrophes.

The break average for each numeric ON field is printed in the format (formatting, PLUS, BLANK, or standard) you specify. The average for a specific numeric field is suppressed if the NOST formatting item is specified for that field. Averages are printed for ON(VLEN) fields, but not for ON(NUM) fields.

You can use the Ndd or Udd formatting item to decrease or increase the number of digits used for a break total. If the break total for an ON field overflows d digits, ICETOOL prints asterisks for the break average for that field and terminates the operation. You can prevent overflow by specifying an appropriate dd value for Ndd or Udd. For example, if ON(1,15,ZD) with BAVERAGE overflows the default of 15 digits for the total, you can specify ON(1,15,ZD,U16) to prevent overflow.

See the discussion of Ndd or Udd under ON(p,m,f,formatting) for more details on using Ndd or Udd.

The BTOTAL, BMAXIMUM, BMINIMUM, BAVERAGE, and BCOUNT lines are printed in the order in which you specify them.

BCOUNT('string')
Specifies a break COUNT line is to be printed after the rows of data for each section. BCOUNT can only be specified if BREAK is specified. The specified string is printed starting at the indent column of the break COUNT line, followed by the break count of data records in the section. If STATLEFT is specified, the string is printed to the left of the first column of data. If STATLEFT is not specified, the string is printed in the first column of data. The count is printed on the same line as the string. A blank line is printed before the break COUNT line.

The string (1 to 50 characters) must be enclosed in single apostrophes. To include a single apostrophe (') in the string, specify two single apostrophes (''). To suppress printing of a string, specify BCOUNT('') using two single apostrophes.

The count is printed in the format (PLUS, BLANK, or standard) you specify. EDBCOUNT(formatting) can be used to apply formatting items to the count. The default number of digits (d) for the count is 15.

The BTOTAL, BMAXIMUM, BMINIMUM, BAVERAGE, and BCOUNT lines are printed in the order in which you specify them.

EDBCOUNT(formatting)
Specifies how the break count is to be formatted for printing. Formatting items can be specified in any order, but each item can only be specified once. EDBCOUNT can only be specified if BCOUNT('string') is specified.

mask
See ON(p,m,f,formatting) for a discussion of **mask**.

E'pattern'
See ON(p,m,f,formatting) for a discussion of **E'pattern'**.

L'string'
See ON(p,m,f,formatting) for a discussion of **L'string'**.

F'string'
See ON(p,m,f,formatting) for a discussion of **F'string'**.

T'string'
See ON(p,m,f,formatting) for a discussion of **T'string'**.

LZ
See ON(p,m,f,formatting) for a discussion of **LZ**.

Udd
See EDCOUNT(formatting) for a discussion of **Udd**.

STATLEFT
Specifies that the strings for statistics (TOTAL, MAXIMUM, MINIMUM, AVERAGE, COUNT, BTOTAL, BMAXIMUM, BMINIMUM, BAVERAGE, BCOUNT) are to be placed to the left of the first column of data (overriding the default of placing the strings in the first column). STATLEFT ensures that each statistic appears on the same line as its string while making the statistics lines stand out from the columns of data.

UZERO
Specifies that -0 and +0 are to be treated as unsigned zero values, that is, as the same value. With UZERO, -0 and +0 are treated as positive for ON, MINIMUM, MAXIMUM, BREAK, BMINIMUM and BMAXIMUM processing.

UZERO overrides the default of treating -0 and +0 as signed zero values, that is, as different values. Without UZERO, -0 is treated as negative and +0 is treated as positive for ON, MINIMUM, MAXIMUM, BREAK, BMINIMUM and BMAXIMUM processing.

LISTSDB OR LISTNOSDB
Can be used to override the SDBMSG value for this LIST data set. LISTSDB directs ICETOOL to select the system-determined optimum block size for the LIST data set in the same way as for installation option SDBMSG=YES. LISTNOSDB directs ICETOOL to select the block size for the LIST data set in the same way as for installation option SDBMSG=NO. See the discussion of the LIST(listdd) operand previously in this section for more information on how LISTSDB or LISTNOSDB affects the LIST data set block size.

Attention: LISTSDB has no effect for SYSOUT data sets (for example, //RPT1 DD SYSOUT=*), because the system-determined optimum block size is not used for spool or dummy data sets.

DISPLAY examples

Although the DISPLAY operators in the examples in this section could all be contained in a single ICETOOL job step, they are shown and discussed separately for clarity. See “OCCUR operator” on page 607 for additional examples of tailoring the report format.

Example 1

```
DISPLAY FROM(SOURCE) LIST(FIELDS) ON(NUM) ON(40,12,CH) -
ON(20,8,PD)
```

Prints, in the FIELDS data set:

- A heading line containing the standard headings
- Data lines in the standard format containing:
  - The record number in the standard format
  - The characters from positions 40-51 of the SOURCE data set
  - The packed decimal values from positions 20-27 of the SOURCE data set in the standard format

The FIELDS output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
RECORD NUMBER     (40,12,CH)            (20,8,PD)
000000000000001   SAN JOSE              000000000003745
000000000000002   MORGAN HILL           000000000016502
              .   .                                 .
              .   .                                 .
              .   .                                 .
```

The heading line appears at the top of each page.

Example 2

```
DISPLAY FROM(IN) LIST(LIST1) -
      TITLE('National Accounting Report') -
      PAGE DATE TIME -
      HEADER('Division') HEADER('Revenue') HEADER('Profit/Loss') -
      ON(1,25,CH)        ON(45,10,ZD)       ON(35,10,ZD) -
      BLANK -
      TOTAL('Company Totals') -
      AVERAGE('Company Averages')
```

Prints, in the LIST1 data set:

- A title line containing the specified title, the page number, the date and the time
- A heading line containing the specified underlined headings
- Data lines in the BLANK format containing:
  - The characters from positions 1-25 of the IN data set
  - The zoned decimal values from positions 45-54 of the IN data set
  - The zoned decimal values from positions 35-44 of the IN data set
- A TOTAL line containing the specified string and the total for each of the two zoned decimal fields in the BLANK format
- An AVERAGE line containing the specified string and the average for each of the two zoned decimal fields in the BLANK format.

The LIST1 output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
National Accounting Report        - 1 -        02/21/05        18:52:44

Division                            Revenue        Profit/Loss
-------------------------   ----------------   ----------------
Research and Development            54323456            -823325
Manufacturing                      159257631            1372610
.                                          .                  .
.                                          .                  .
.                                          .                  .

Company Totals                     612867321            5277836

Company Averages                    76608415             659729
```

The title line and underlined heading line appear at the top of each page.

Example 3

```
     DISPLAY FROM(DATA) LIST(JUSTDATA) -
       NOHEADER -
       ON(17,5,PD) ON(1,2,FI)
```

Prints, in the JUSTDATA data set:

- Data lines in the standard format containing:
  - The packed decimal values from positions 17-21 of the DATA data set in the standard format
  - The fixed-point values from positions 1-2 of the DATA data set in the standard format

The JUSTDATA output contains no page ejects or heading lines and looks as follows (the first 2 records are shown with illustrative values):

```
-0000000000273216   +0000000000000027
+0000000000993112   +0000000000000321
                .                   .
                .                   .
                .                   .
```

Example 4

```
COPY FROM(INPUT) TO(TEMP) USING(TREG)
DISPLAY FROM(TEMP) LIST(REGULAR) -
  PAGE TITLE('Report on Regular Tools') TBETWEEN(12)-
  HEADER(NONE) ON(1,18,CH) -
  HEADER(,'Item') ON(35,5,CH) -
  HEADER('Percent','Change') ON(28,4,FS,B1) -
  LINES(66)
COPY FROM(INPUT) TO(TEMP) USING(TPOW)
DISPLAY FROM(TEMP) LIST(POWER) -
  PAGE TITLE('Report on Power Tools ') TBETWEEN(12)-
  HEADER(NONE) ON(1,18,CH) -
  HEADER(,'Item') ON(35,5,CH) -
  HEADER('Percent','Change') ON(28,4,FS,B1) -
  LINES(66)
```

This example shows how reports for different subsets of data can be produced. Assume that:

- The TREGCNTL data set contains:

```
  INCLUDE COND=(44,8,CH,EQ,C'Regular')
```

- The TPOWCNTL data set contains:

```
  INCLUDE COND=(44,8,CH,EQ,C'Power')
```

The first COPY operator copies the records from the INPUT data set that contain 'Regular ' in positions 44-51 to the TEMP (temporary) data set

The first DISPLAY operator uses the first subset of records in the TEMP data set to print, in the REGULAR data set:

- A title line containing the page number and specified title, with twelve blanks between these report elements.
- A two-line heading containing the specified underlined strings (with no heading for the first ON field)). Note the comma in HEADER(,'Item') to place 'Item' on line2 of the heading.
- Data lines for the first subset of records containing:
  - The characters from positions 1-18
  - The characters from positions 35-39
  - The floating sign values from positions 28-31 formatted with one decimal place and a period as the decimal point

The second COPY operator copies the records from the INPUT data set that contain 'Power ' in positions 44-51 to the TEMP (temporary) data set

The second DISPLAY operator uses the second subset of records in the TEMP data set to print, in the POWER data set:

- A title line containing the page number and specified title, with twelve blanks between these report elements.
- A two-line heading containing the specified underlined strings (with no heading for the first ON field)). Note the comma in HEADER(,'Item') to place 'Item' on line2 of the heading.
- Data lines for the second subset of records containing:
  - The characters from positions 1-18
  - The characters from positions 35-39
  - The floating sign values from positions 28-31 formatted with one decimal place and a period as the decimal point

The REGULAR output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
- 1 -           Report on Regular Tools

                           Percent
                    Item    Change
                    -----   -------
Hammers             10325     -7.3
Wrenches            00273     15.8
```

The title line and underlined heading lines appear at the top of each page. The number of lines per page is 66, overriding the default of 58.

The POWER output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
- 1 -           Report on Power Tools

                           Percent
                    Item    Change
                    -----   -------
Hammers             10325      9.8
Wrenches            00273    123.0
```

The title line and underlined heading lines appear at the top of each page. The number of lines per page is 66, overriding the default of 58.

Example 5

```
     DISPLAY FROM(INV) LIST(RDWLIST1) -
       TITLE('No Frills RDW Report') -
       ON(NUM) -
       ON(VLEN) -
       ON(1,4,HEX) -
       MINIMUM('Smallest') -
       MAXIMUM('Largest')
```

Prints, in the RDWLIST1 data set:

- A title line containing the specified title
- A heading line containing the standard headings
- Data lines in the standard format containing:
  - The record number
  - The record length
  - The record descriptor word (RDW) in hexadecimal
- A MINIMUM line containing the specified string and the minimum record length in the standard format
- A MAXIMUM line containing the specified string and the maximum record length in the standard format.

The RDWLIST1 output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
 No Frills RDW Report

 RECORD NUMBER     RECORD LENGTH     (1,4,HEX)
 0000000000000001  +0000000000000075  004B0000
 0000000000000002  +0000000000000071  00470000
                .                 .  .
                .                 .  .
                .                 .  .

 Smallest          +0000000000000058

 Largest           +0000000000000078
```

The title line and heading line appear at the top of each page.

Example 6

```
      DISPLAY FROM(INV) LIST(RDWLIST2) -
        DATE(DMY.) -
        TFIRST TBETWEEN(3) -
        TITLE('Fancy RDW Report with') -
        TITLE('length in decimal and hex') -
        TITLE('and max and min length') -
        TIME(12:) -
        HEADER('Relative Record') ON(NUM) -
        HEADER('    RDW (length)') ON(VLEN) -
        HEADER('RDW (Hex)') ON(1,4,HEX) -
        BLANK -
        MINIMUM('Smallest Record:') -
        MAXIMUM('Largest Record:') -
        COUNT('Number of Records: ') EDCOUNT(U04)
```

Prints, in the RDWLIST2 data set:

- Title lines containing the date, the specified title strings and the time
- A heading line containing the specified underlined headings
- Data lines in the BLANK format containing:
  - The record number
  - The record length
  - The record descriptor word (RDW) in hexadecimal
- A MINIMUM line containing the specified string and the minimum record length in the BLANK format
- A MAXIMUM line containing the specified string and the maximum record length in the BLANK format
- A COUNT line containing the specified string and the edited overall record count.

RDWLIST2 output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
22.07.08     Fancy RDW Report with     05:37:43 pm

           length in decimal and hex

            and max and min length

Relative Record       RDW (length)   RDW (Hex)
---------------   ----------------   ---------
              1                 84   00540000
              2                 47   002F0000
              3                 31   001F0000
              4                 31   001F0000
              5                 31   001F0000
...

Relative Record       RDW (length)   RDW (Hex)
---------------   ----------------   ---------
             51                 31   001F0000
             52                 31   001F0000
             53                 31   001F0000
             54                 31   001F0000
             55                 31   001F0000
             56                 31   001F0000

Smallest Record:                31

Largest Record:                 84

Number of Records:    56
```

The title lines only appear at the top of the first page as specified by the TFIRST operand. The underlined heading line appears at the top of each page.

Example 7

```
    SORT FROM(PARTS) TO(TEMP) USING(SRT1)
    DISPLAY FROM(TEMP) LIST(USA) -
```

```
      TITLE('Parts Completion Report for USA') DATE -
      HEADER('Part')  HEADER('Completed')  HEADER('Value ($)') -
      ON(15,6,CH)     ON(3,4,ZD,A1)        ON(38,8,ZD,C1) -
      TOTAL('Total:')
    DISPLAY FROM(TEMP) LIST(FRANCE) -
      TITLE('Parts Completion Report for France') DATE(DM4/) -
      HEADER('Part')  HEADER('Completed')  HEADER('Value (F)') -
      ON(15,6,CH)     ON(3,4,ZD,A3)        ON(38,8,ZD,C3) -
      TOTAL('Total:')
    DISPLAY FROM(TEMP) LIST(DENMARK) -
      TITLE('Parts Completion Report for Denmark') DATE(DMY-) -
      HEADER('Part')  HEADER('Completed')  HEADER('Value (kr)') -
      ON(15,6,CH)     ON(3,4,ZD,A2)        ON(38,8,ZD,C2) -
      TOTAL('Total:')
```

This example shows how reports for three different countries can be produced. The reports differ only in the way that date and numeric values are displayed.

Assume that the SRT1CNTL data set contains:

```
  SORT FIELDS=(15,6,CH,A)
```

The SORT operator sorts the PARTS data set to the TEMP data set using the SORT statement in SRT1CNTL.

The first DISPLAY operator uses the sorted records in the TEMP data set to print, in the USA data set:

- A title line containing the specified title and the date in the format commonly used in the United States
- A heading line containing the specified underlined headings
- Data lines containing:
  - The characters from positions 15-20
  - The zoned decimal values from positions 3-6 formatted with the separators commonly used in the United States
  - The zoned decimal values from positions 38-45 formatted with two decimal places and the separators and decimal point commonly used in the United States.
- A TOTAL line containing the specified string and the total for each of the two zoned decimal fields formatted in the same way as the data values.

The second DISPLAY operator uses the sorted records in the TEMP data set to print, in the FRANCE data set:

- A title line containing the specified title and the date in the format commonly used in France
- A heading line containing the specified underlined headings
- Data lines containing:
  - The characters from positions 15-20
  - The zoned decimal values from positions 3-6 formatted with the separators commonly used in France
  - The zoned decimal values from positions 38-45 formatted with two decimal places and the separators and decimal point commonly used in France.
- A TOTAL line containing the specified string and the total for each of the two zoned decimal fields formatted in the same way as the data values.

The third DISPLAY operator uses the sorted records in the TEMP data set to print, in the DENMARK data set:

- A title line containing the specified title and the date in the format commonly used in Denmark
- A heading line containing the specified underlined headings
- Data lines containing:
  - The characters from positions 15-20
  - The zoned decimal values from positions 3-6 formatted with the separators commonly used in Denmark

  - The zoned decimal values from positions 38-45 formatted with two decimal places and the separators and decimal point commonly used in Denmark.
- A TOTAL line containing the specified string and the total for each of the two zoned decimal fields formatted in the same way as the data values.

The USA output starts on a new page and looks as follows (several records are shown with illustrative values):

```
Parts Completion Report for USA        01/14/05

Part                Completed              Value ($)
------   --------------------   --------------------
000310                    562               8,317.53
001184                  1,234              23,456.78
029633                     35                 642.10
192199                  3,150             121,934.65
821356                    233               2,212.34

Total:                  5,214             156,563.40
```

The title line and underlined heading line appear at the top of each page.

The FRANCE output starts on a new page and looks as follows (several record are shown with illustrative values):

```
Parts Completion Report for France        14/01/2005

Part                Completed               Value (F)
------   --------------------   --------------------
000310                    562                8 317,53
001184                  1 234               23 456,78
029633                     35                  642,10
192199                  3 150             121 934,65
821356                    233                2 212,34

Total:                  5 214             156 563,40
```

The title line and underlined heading line appear at the top of each page.

The DENMARK output starts on a new page and looks as follows (several records are shown with illustrative values):

```
Parts Completion Report for Denmark        14-01-05

Part                Completed              Value (kr)
------   --------------------   --------------------
000310                    562                8.317,53
001184                  1.234               23.456,78
029633                     35                  642,10
192199                  3.150             121.934,65
821356                    233                2.212,34

Total:                  5.214             156.563,40
```

The title line and underlined heading line appear at the top of each page.

Example 8

```
SORT FROM(DATA) TO(TEMP) USING(SRTX)
DISPLAY FROM(TEMP) LIST(WEST) -
  DATE TITLE('Western Region Profit/Loss Report') PAGE -
  BTITLE('Division:') BREAK(3,10,CH) -
  HEADER('Branch Office') ON(16,13,CH) -
  HEADER('Profit/Loss (K)') ON(41,4,PD,/K,E1) -
  BMINIMUM('Lowest Profit/Loss in this Division:') -
  BMAXIMUM('Highest Profit/Loss in this Division:') -
  BAVERAGE('Average Profit/Loss for this Division:') -
  MINIMUM('Lowest Profit/Loss for all Divisions:') -
  MAXIMUM('Highest Profit/Loss for all Divisions:') -
  AVERAGE('Average Profit/Loss for all Divisions:')
```

This example shows how a report with sections can be produced.

Assume that the SRTXCNTL data set contains:

```
  SORT FIELDS=(3,10,A,16,13,A),FORMAT=CH
```

The SORT operator sorts the DATA data set to the TEMP data set using the SORT statement in SRTXCNTL.

The DISPLAY operator uses the sorted records in the TEMP data set to print, in the WEST data set, sections with:

- A title line containing the date, the specified title string, and the page number
- A break title containing the specified break title string, and the break field characters from positions 3-12
- A heading line containing the specified underlined headings
- Data lines containing:
  - The characters from positions 16-28
  - The packed decimal values from positions 41-44 divided by 1000 and formatted with separators and signs as specified.
- Break MINIMUM, MAXIMUM, and AVERAGE lines containing the specified strings and statistics for the packed decimal field values in this section, formatted in the same way as the data values.

The last page of the report contains:

- A title line containing the date, the specified title string, and the page number
- A heading line containing the specified underlined headings
- Overall MINIMUM, MAXIMUM, and AVERAGE lines containing the specified strings and statistics for the packed decimal field values in the report, formatted in the same way as the data values.

The first section of the WEST output starts on a new page and looks as follows (several records are shown with illustrative values):

```
01/14/05        Western Region Profit/Loss Report        - 1 -

Division:  Chips

Branch Office   Profit/Loss (K)
-------------   ---------------
Gilroy                   3,293
Los Angeles                (141)
Morgan Hill                 213
Oakland                  1,067
San Francisco                (31)
San Jose                      92
San Martin               1,535

Lowest Profit/Loss in this Division:
                           (141)

Highest Profit/Loss in this Division:
                           3,293

Average Profit/Loss for this Division:
                             861
```

The title line, break title line, and underlined heading line appear at the top of each page of the section.

The second section of the WEST output starts on a new page and looks as follows (several records are shown with illustrative values):

```
01/14/05        Western Region Profit/Loss Report        - 2 -

Division:  Ice Cream

Branch Office   Profit/Loss (K)
-------------   ---------------
Marin                      673
Napa                        95
San Francisco             (321)
San Jose                 2,318
San Martin                  21

Lowest Profit/Loss in this Division:
                          (321)

Highest Profit/Loss in this Division:
                         2,318

Average Profit/Loss for this Division:
                           557
```

The title line, break title line, and underlined heading line appear at the top of each page of the section.

The last page of the WEST output starts on a new page and looks as follows:

```
01/14/05        Western Region Profit/Loss Report        - 3 -

Branch Office   Profit/Loss (K)
-------------   ---------------

Lowest Profit/Loss for all Divisions:
                          (321)

Highest Profit/Loss for all Divisions:
                         3,293

Average Profit/Loss for all Divisions:
                           734
```

Example 9

```
MODE CONTINUE
VERIFY FROM(CHECK) ON(2,3,PD) LIMIT(500)
DISPLAY FROM(CHECK) LIST(PDREPORT) BLANK LIMIT(500) -
  HEADER('Relative Record') ON(NUM) -
  HEADER('Numeric') ON(2,3,PD) -
  HEADER('Hexadecimal') ON(2,3,HEX) -
  HEADER('Associated Field') ON(21,20,CH)
```

This example shows how each record containing an invalid decimal value can be identified either by its relative record number or an associated field in the record.

The MODE operator ensures that the DISPLAY operator is processed if the VERIFY operator identifies an invalid decimal value.

The VERIFY operator checks for invalid digits (A-F) and invalid signs (0-9) in the packed decimal values from positions 2-4 of the CHECK data set. Messages ICE618A and ICE649A are printed in the TOOLMSG data set for each value (if any) that contains an invalid digit or sign. If 500 invalid values are found, the operation is terminated.

The DISPLAY operator checks for invalid digits (A-F) in the packed decimal values from positions 2-4 of the CHECK data set. Messages ICE618A and ICE649A are printed in the TOOLMSG data set for each value (if any) that contains an invalid digit. If 500 invalid values are found, the operation is terminated. If a check for invalid signs is required, the VERIFY operator must be used, because the DISPLAY operator only checks for invalid digits. The VERIFY operator is not required if signs need not be checked.

The DISPLAY operator also prints, in the PDREPORT data set:

- A heading line containing the specified underlined headings
- Data lines in the BLANK format containing:

- The relative record number. This number can be matched against the RECORD numbers printed in the ICE618A messages to find the records with invalid signs.
- The numeric representation of the packed decimal value in positions 2-4. Asterisks are shown for values with invalid digits, making them easy to identify. Asterisks are not shown for values with invalid signs; these must be identified by matching the relative record number against the RECORD number in ICE618A.
- The hexadecimal representation of the packed decimal value in positions 2-4 (also shown in ICE649A). This makes it easy to find the specific hexadecimal digits or signs that are invalid.
- The characters in positions 21-40. An associated field such as this can be used to make identification of the records with invalid values easier.

The ICE618A and ICE649A messages in TOOLMSG for the VERIFY operator are:

```
ICE618A 0 INVALID (2,3,PD)       VALUE - RECORD:  000000000000003
ICE649A 0   HEX VALUE:  53A54C
ICE618A 0 INVALID (2,3,PD)       VALUE - RECORD:  000000000000012
ICE649A 0   HEX VALUE:  621540
ICE618A 0 INVALID (2,3,PD)       VALUE - RECORD:  000000000000019
ICE649A 0   HEX VALUE:  400F3C
```

The ICE618A and ICE649A messages in TOOLMSG for the DISPLAY operator are:

```
ICE618A 0 INVALID (2,3,PD)       VALUE - RECORD:  000000000000003
ICE649A 0   HEX VALUE:  53A54C
ICE618A 0 INVALID (2,3,PD)       VALUE - RECORD:  000000000000019
ICE649A 0   HEX VALUE:  400F3C
```

The PDREPORT output looks as follows:

```
Relative Record   Numeric   Hexadecimal   Associated Field
---------------   -------   -----------   --------------------
              1     18600   18600C        Wagar
              2       -93   00093B        Gellai
              3    ******   53A54C        Giulianelli
              4     86399   86399C        Mehta
              5     24215   24215F        Johnson
              6      8351   08351C        Packer
              7     19003   19003C        Childers
              8    -31285   31285D        Burg
              9     88316   88316C        Monkman
             10      1860   01860C        Vezinaw
             11    -29285   29285D        Mead
             12     62154   621540        Wu
             13      -328   00328D        Madrid
             14    -11010   11010D        Warren
             15      1363   01363F        Burt
             16     92132   92132C        Mao
             17    -48500   48500D        Shen
             18       -55   00055D        Yamamoto-Smith
             19    ******   400F3C        Yaeger
             20     33218   33218C        Leung
             21     96031   96031C        Kaspar
```

PDREPORT can be used in conjunction with the ICE618A and ICE649A messages to identify that:

- Record 3 has an invalid digit of A and an associated field of "Giulianelli"
- Record 12 has an invalid sign of 0 and an associated field of "Wu"
- Record 19 has an invalid digit of F and an associated field of "Yaeger".

Example 10

```
  COPY FROM(IN) USING(OUTF)
  DISPLAY FROM(TEMP) LIST(EMPCT) BLANK -
        TITLE('Employees by Function') -
        DATE -
        HEADER('Function') HEADER('Employees') -
        ON(1,25,CH)        ON(30,4,ZD)
```

This example shows how the OUTFIL table lookup feature can be used to substitute meaningful phrases for cryptic values in ICETOOL reports. Assume that:

- The OUTFCNTL data set contains:

```
  OUTFIL FNAMES=TEMP,
    OUTREC=(1:9,2,CHANGE=(25,
                  C'MN',C'Manufacturing',
                  C'RD',C'Research and Development',
                  C'FN',C'Finance',
                  C'MR',C'Marketing',
                  C'IS',C'Information Systems'),
            30:4,4)
```

The COPY operator uses the OUTFIL statement in OUTFCNTL to reformat the IN data set records to the TEMP (temporary) data set. Two fields are extracted for use by the DISPLAY operator:

- The 2-character department code in positions 9-10 is changed to a 25-character name in positions 1-25 using the table lookup feature.
- The zoned decimal value in positions 4-7 is moved to positions 30-33.

The DISPLAY operator uses the reformatted fields in the TEMP data set to print, in the EMPCT data set:

- A title line containing the specified title and the date
- A heading line containing the specified underlined headings
- Data lines in the BLANK format containing:
  - The names from positions 1-25 that were substituted for the department codes
  - The zoned decimal values from positions 30-33.

The EMPCT output starts on a new page and looks as follows:

```
Employees by Function        02/14/05

Function                    Employees
-------------------------   ---------
Manufacturing                     486
Marketing                          21
Research and Development           55
Information Systems               123
Finance                            33
```

Example 11

```
    DISPLAY FROM(ACCTS) LIST(PLAIN) -
        TITLE('Accounts Report for First Quarter') -
        DATE(MD4/) BLANK -
        HEADER('Amount') ON(12,6,ZD) -
        HEADER(Id') ON(NUM) -
        HEADER('Acct#') ON(31,3,PD) -
        HEADER('Date') ON(1,4,ZD) -
        TOTAL('Total for Q1') -
        AVERAGE('Average for Q1')

 DISPLAY FROM(ACCTS) LIST(FANCY) -
        TITLE('Accounts Report for First Quarter') -
        DATE(MD4/) BLANK -
        HEADER('Amount') ON(12,6,ZD,C1,N08) -
        HEADER(Id') ON(NUM,N02) -
        HEADER('Acct#') ON(31,3,PD,NOST,LZ) -
        HEADER('Date') ON(1,4,ZD,E'99/99',NOST) -
        INDENT(2) BETWEEN(5) -
        STATLEFT -
        TOTAL('Total for Q1') -
        AVERAGE('Average for Q1')
```

This example shows some options you can use to improve the appearance of a DISPLAY report. The first DISPLAY operator produces a "plain" report, and the second DISPLAY operator uses the options shown in **bold** to produce a "fancy" report.

The PLAIN output starts on a new page and looks as follows:

```
Accounts Report for First Quarter               05/04/2001

         Amount                  Id                  Acct#                      Date
---------------    ---------------    --------------------    --------------------
          93271                   1                  15932                       106
         137622                   2                    187                       128
          83147                   3                  15932                       212
         183261                   4                   2158                       217
          76389                   5                    187                       305
         920013                   6                  15932                       319

Total for Q1
        1493703                                     50328                      1287

Average for Q1
         248950                                      8388                       214
```

The FANCY output starts on a new page and looks as follows:

```
Accounts Report for First Quarter             05/04/2001

                       Amount         Id        Acct#      Date
                     --------        ---       ------     -----
                       932.71          1        15932     01/06
                     1,376.22          2        00187     01/28
                       831.47          3        15932     02/12
                     1,832.61          4        02158     02/17
                       763.89          5        00187     03/05
                     9,200.13          6        15932     03/19

Total for Q1        14,937.03

Average for Q1       2,489.50
```

Here is an explanation of the extra options used for the "fancy" report:

- First ON field: In the PLAIN report, BLANK causes ICETOOL to print the 6-byte ZD values as unedited digits with leading zeros suppressed. But for this example, we know the digits really represent dollars and cents. So in the FANCY report, we use the **C1** formatting item (one of thirty-three available masks) to print the values with a comma (,) as the thousands separator and a period (.) as the decimal point.

  In the PLAIN report, TOTAL causes ICETOOL to allow 15 digits for the values because it does not know how many digits are needed. But for this example, we know the total amount will not exceed 8 digits. So in the FANCY report, we use the **N08** formatting item to set the number of digits to 8. This decreases the column width for the field.
- Second ON field: In the PLAIN report, NUM causes ICETOOL to allow 15 digits for the record number because it does not know how many digits are needed. But for this example, we know the number of records will not exceed 99. So in the FANCY report, we use the **N02** formatting item to set the number of digits to 2. This decreases the column width for the record number.
- Third ON field: In the PLAIN report, TOTAL and AVERAGE cause ICETOOL to print the total and average for this 3-byte PD field. But for this example, we know we do not want statistics for the field because it is an account number. So in the FANCY report, we use the **NOST** formatting item to suppress the statistics for this field.

  In the PLAIN report, the default mask of A0 causes ICETOOL to suppress leading zeros for this 3-byte PD field. But for this example, we know that we want to show leading zeros for the field because it is an account number. So in the FANCY report, we use the **LZ** formatting item to print leading zeros for this field.
- Fourth ON field: In the PLAIN report, BLANK causes ICETOOL to print the 4-byte ZD values as unedited digits with leading zeros suppressed. But for this example, we know the digits represent a date (month and day). So in the FANCY report, we use the **E'99/99'** formatting item to print the values with leading zeros and a slash (/) between the month and day.

In the PLAIN report, TOTAL and AVERAGE cause ICETOOL to print the total and average for this 4-byte ZD field. But for this example, we know we do not want the total or average for this field because it is a date. So in the FANCY report, we use the **NOST** formatting item to suppress the statistics for this field.

Note: In some applications, we might want the minimum and maximum for a date displayed with E'pattern', so we would not specify NOST for the date field.

- INDENT: In the PLAIN report, ICETOOL starts the report in column 2 (after the control character), by default. But for this example, we want to indent the report a bit. So in the FANCY report, we use the **INDENT(2)** operand to indent the report by 2 blanks so it starts in column 4.
- BETWEEN: In the PLAIN report, ICETOOL uses 3 blanks between the columns of data, by default. But for this example, we want more space between the columns. So in the FANCY report, we use the **BETWEEN(5)** operand to insert 5 blanks between the columns.
- STATLEFT: In the PLAIN report, ICETOOL prints the strings for TOTAL and AVERAGE under the first column of data, by default, and uses two lines for each statistic to avoid having the string overlay the value. But for this example, we would like to have the TOTAL and AVERAGE strings stand out in the report and also have each string on the same line as its value. So in the FANCY report, we use the **STATLEFT** operand to print the TOTAL and AVERAGE strings to the left of the first column of data.

Example 12

```
SORT FROM(RAWSMF) TO(SMF14) USING(SMFI)
DISPLAY FROM(SMF14) LIST(SMF14RPT) -
      TITLE('SMF Type-14 Records') DATE(4MD/) -
      HEADER('Date') ON(11,4,DT1,E'9999/99/99') -
      HEADER('Time') ON(7,4,TM1,E'99:99:99') -
      HEADER('Sys') ON(15,4,CH) -
      HEADER('Jobname') ON(19,8,CH) -
      HEADER('Datasetname') ON(69,44,CH)
```

This example shows how SMF date and time values can be displayed in a meaningful way in a report on SMF type-14 records.

The SORT operator selects the type-14 records from the RAWSMF data set and sorts them by date and time to the SMF14 data set. It uses the following control statements in SMFICNTL:

```
INCLUDE COND=(6,1,BI,EQ,14)
SORT FIELDS=(11,4,PD,A,7,4,BI,A)
```

The DISPLAY operator uses the selected type-14 records in SMF14 to print, in the SMF14RPT data set:

- A title line containing the specified title and the date
- A heading line containing the specified underlined headings
- Data lines containing:
  - The SMF date values in positions 11-14 displayed as C'yyyy/mm/dd'
  - The SMF time values in positions 7-10 displayed as C'hh:mm:ss'
  - The character values in positions 15-18
  - The character values in positions 19-26
  - The character values in positions 69-112

The SMF14RPT output starts on a new page and looks as follows:

```
SMF Type-14 Records        2001/04/24

      Date       Time   Sys    Jobname    Datasetname
----------   --------   ----   --------   ----------- ...
2001/04/20   06:03:15   ID03   JOB00003   SYS1.QRS
2001/04/20   10:03:22   ID02   JOB00002   SYS1.XYZ
2001/04/21   14:05:37   ID03   JOB00004   SYS1.MNO
2001/04/21   22:11:00   ID03   JOB00005   SYS1.MNO
2001/04/24   00:00:08   ID03   JOB00006   SYS1.MNO
```

Note: When you use SMF date formats (DTn) or SMF time formats (TMn), the SMF values are treated as numeric. This allows you to use numeric formatting items such as masks and patterns to edit the SMF values. By default, DTn and TMn headings, like other numeric headings, appear right-aligned as shown in the SMF14RPT output example shown previously in this section. If you want to center-align or left-align headings for numeric values, you can add an appropriate number of blanks at the end of HEADER('string1').

For example, if you wanted to left-align the SMF date heading, you could use six blanks at the end of the header string like so:

```
   HEADER('Date     ')
```

to get the following heading:

```
   Date
   ----------
```

If you wanted to center-align the SMF date heading, you could use three blanks at the end of the header string like so:

```
   HEADER('Date  ')
```

to get the following heading:

```
      Date
   ----------
```

Example 13

```
SORT FROM(SMFIN) TO(SMF71) USING(TY71)
DISPLAY FROM(SMF71) LIST(SMF71RPT) -
TITLE('Low impact central storage frames') -
BREAK(15,4,CH,L'System: ') -
HEADER('Date') ON(11,4,DT1,E'9999-99-99') -
HEADER('Time') ON(7,4,TM1,E'99:99:99') -
HEADER('Min Frames') ON(925,8,FL,U10) -
HEADER('Max Frames') ON(933,8,FL,U10) -
HEADER('Avg Frames') ON(941,8,FL,U10) -
BLANK PAGE
```

This example shows how floating point values can be displayed as integers in a report on SMF type-71 records with a section for each system id.

The SORT operator selects SMF type-71 records that are at least 19 bytes long and sorts them by system id, date and time to the SMF71 data set. It uses the following control statements in TY71CNTL:

```
OMIT COND=(6,1,BI,NE,+71,OR,1,2,BI,LE,+18)
SORT FIELDS=(15,4,CH,A,11,4,PD,A,7,4,BI,A)
```

The DISPLAY operator uses the selected type-71 records in SMF71 to print, in the SMF71RPT data set:

- A title line containing the specified title and the page number.
- A break title containing the specified leading string and the SMF71SID system id character values in positions 15-18.
- A heading line containing the specified underlined headings.
- Data lines containing:
  - The SMF71DTE date value. This SMF date value in positions 11-14 is displayed as a C'yyyy-mm-dd' value.
  - The SMF71TME time value. This SMF time value in positions 7-10 is displayed as a C'hh:mm:ss' value.
  - The SMF71CLM minimum number of low-impact central storage frames value. This floating-point value in positions 925-932 is displayed as a 10 digit integer value. (The U10 formatting item

reduces the number of digits for the integer representation of the floating-point value from 20 to 10, decreasing the column width for the field.)

- The SMF71CLX maximum number of low-impact central storage frames value. This floating-point value in positions 933-940 is displayed as a 10 digit integer value.
- The SMF71CLA average number of low-impact central storage frames value. This floating-point value in positions 941-948 is displayed as a 10 digit integer value.

Each system id value starts on a new page and looks as follows (several sections and records are shown with illustrative values):

```
Low impact central storage frames        - 1 -

System: SYSA

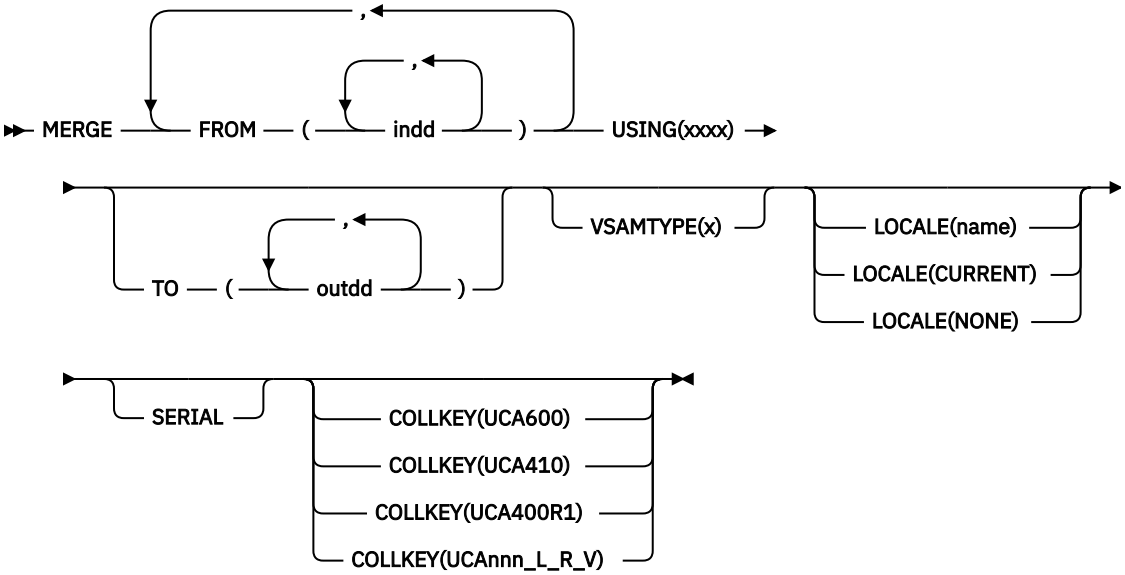
      Date       Time    Min Frames    Max Frames    Avg Frames
----------   --------   -----------   -----------   -----------
2005-08-01   11:45:00        934215       1001596        963434
2005-08-01   12:00:00        971599       1004939        984437
2005-08-01   12:15:00        970192        982565        973768

Low impact central storage frames        - 2 -

System: SYSB

      Date       Time    Min Frames    Max Frames    Avg Frames
----------   --------   -----------   -----------   -----------
2005-08-01   11:45:00        947220        985444        966581
2005-08-01   12:00:00        980120       1018982        986360
2005-08-01   12:15:00        980387       1051920       1011873
```

MERGE operator



Merges up to 50 input data sets to an output data set. The records in each input data set to be merged must already be in sorted order as specified by the control fields in a supplied DFSORT MERGE statement.

You must specify at least one FROM operand and a USING(xxxx) operand. Each FROM operand can be used to specify one or more ddnames. You can specify up to 10 FROM operands. The maximum number of ddnames in all of the FROM operands must not exceed 50.

DFSORT is called to merge the indd data sets to the outdd data sets using the DFSORT control statements in xxxxCNTL. You must supply a DFSORT MERGE statement in the xxxxCNTL data set to indicate the control fields for the merge. You can use additional DFSORT statements in the xxxxCNTL data set to merge a subset of the input records (INCLUDE or OMIT statement; OUTFIL INCLUDE, OMIT, SAVE,

STARTREC, ENDREC, SAMPLE, SPLIT, SPLITBY and SPLIT1R operands), reformat records for output (INREC, OUTREC, and OUTFIL statements; user exit routines), and so on.

If EQUALS is in effect, records that collate identically are output in the order of their ddnames in the FROM operands.

The active locale's collating rules affect MERGE processing as explained in “MERGE control statement” on page 161. If an INCLUDE or OMIT statement or an OUTFIL INCLUDE or OMIT operand is specified in the xxxxCNTL data set, the active locale's collating rules affect INCLUDE and OMIT processing as explained in the "Cultural Environment Considerations" discussion in “INCLUDE control statement” on page 91.

Note: For a merge application, records deleted during an E35 exit routine are not sequence checked. If you use an E35 exit routine without an output data set, sequence checking is not performed at the time the records are passed to the E35 user exit; therefore, you must ensure that input records are in correct sequence.

Operand descriptions

The operands can be specified in any order:

FROM(indd,...)
Specifies the ddnames of the input data sets to be read by DFSORT for this operation. Up to 10 FROM operands can be used to specify up to 50 input ddnames. An indd DD statement must be present for each indd name specified. Each indd input data set must conform to the rules for DFSORT's SORTINnn data sets.

USING(xxxx)
Specifies the first 4 characters of the ddname for the control statement data set to be used by DFSORT for this operation. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx.

An xxxxCNTL DD statement must be present, and the control statements in it must conform to the rules for DFSORT's SORTCNTL data set.

The xxxxCNTL data set must contain a MERGE statement. If TO is not specified, the xxxxCNTL data set must also contain either one or more OUTFIL statements or a MODS statement for an E35 routine that disposes of all records. Other statements are optional.

TO(outdd,...)
Specifies the ddnames of the output data sets to be written by DFSORT for this operation. From 1 to 10 outdd names can be specified. An outdd DD statement must be present for each outdd name specified. If a single outdd data set is specified, DFSORT is called once to merge the indd data sets to the outdd data set using SORTOUT processing; the outdd data set must conform to the rules for DFSORT's SORTOUT data set. If multiple outdd data sets are specified and SERIAL is not specified, DFSORT is called once to merge the indd data sets to the outdd data sets using OUTFIL processing; the outdd data sets must conform to the rules for DFSORT's OUTFIL data sets.

A ddname specified in a FROM operand must not also be specified in the TO operand.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

LOCALE(name)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

LOCALE(CURRENT)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

LOCALE(NONE)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

SERIAL
Specifies that OUTFIL processing is not to be used when multiple outdd data sets are specified. DFSORT is called multiple times and uses SORTOUT processing; the outdd data sets must conform to the rules for DFSORT's SORTOUT data set. SERIAL is not recommended because the use of serial

processing (that is, multiple calls to DFSORT) instead of OUTFIL processing can degrade performance and imposes certain restrictions as detailed later in this section. SERIAL is ignored if a single outdd data set is specified.

DFSORT is called to merge the indd data set to the first outdd data set using the DFSORT control statements in the xxxxCNTL data set. If the merge operation is successful, DFSORT is called as many times as necessary to copy the first outdd data set to the second and subsequent outdd data sets. Therefore, for maximum efficiency, use a disk data set as the first in a list of outdd data sets on both disk and tape. If more than one outdd data set is specified, DFSORT must be able to read the first outdd data set after it is written in order to copy it to the other outdd data sets. Do not use a SYSOUT or DUMMY data set as the first in a list of outdd data sets because:

- If the first data set is SYSOUT, DFSORT abends when it tries to copy the SYSOUT data set to the second outdd data set.
- If the first data set is DUMMY, DFSORT copies the empty DUMMY data set to the other outdd data sets (that is, all of the resulting outdd data sets are empty).

COLLKEY(UCA600)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

COLLKEY(UCA410)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

COLLKEY(UCA400R1)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

COLLKEY(UCAnnn_L_R_V)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

MERGE examples

Although the MERGE operators in the examples in this section could all be contained in a single ICETOOL job step, they are shown and discussed separately for clarity.

Example 1

```
//TOOLIN DD *
MERGE FROM(IN01,IN02,IN03,IN04,IN05) TO(OUTPUT) USING(MERG)
//MERGCNTL DD *
  OPTION EQUALS
  MERGE FIELDS=(21,4,CH,A)
/*
```

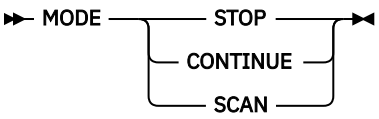
This example merges 5 input files to an output file. EQUALS is used to ensure that records that collate identically are output in the order specified in the FROM operand. For example, if IN01, IN03 and IN05 all have records with a key or 'AAAA' in positions 21-24, the output will contain the 'AAAA' record from IN01, the 'AAAA' record from IN03 and the 'AAAA' record from IN05, in that order.

Example 2

```
//TOOLIN DD *
MERGE FROM(INPUT1,INPUT2,INPUT3,INPUT4) -
      FROM(INPUT5,INPUT6,INPUT7) VSAMTYPE(F) USING(MRG1)
//MRG1CNTL DD *
  MERGE FIELDS=(52,8,UFF,D)
  OUTFIL FNAMES=OUT1,INCLUDE=(15,3,SS,EQ,C'D21,D33')
  OUTFIL FNAMES=OUT2,SAVE
/*
```

This example merges 7 input files to 2 output files. It uses two OUTFIL statements to create the two output files; each output file will have a different subset of the merged records. VSAMTYPE(F) tells DFSORT the record type is F (only needed for VSAM input files).

MODE operator



Specifies one of three modes to control error checking and actions after error detection. A MODE operator effects the "processing" (that is, error checking of ICETOOL statements and calling DFSORT) of the operators that follow it, up to the next MODE operator (if any). MODE operators allow you to do the following for groups of operators or all operators:

1. Stop or continue processing operators after a return code of 8, 12 or 16. A return code of 12 or 16 can be set as the result of a statement or run-time error detected by ICETOOL or DFSORT. A return code of 8 can be set as the result of a COUNT operator with RC8.
2. Check for errors in ICETOOL statements, but do not call DFSORT.

Operand descriptions

The operands described in this section can be specified in any order.

STOP
Stops subsequent operations if a return code of 8, 12 or 16 is set. If an error is detected for an operator, SCAN mode is automatically set in effect; DFSORT is not called for subsequent operators, although checking ICETOOL statements for errors continues.

STOP mode can be used to group dependent operators (that is, if an operation fails, do not process the remaining operators).

STOP MODE is set in effect automatically at the start of the ICETOOL run.

CONTINUE
Continues with subsequent operations regardless of whether or not a return code of 8, 12 or 16 is set. If an operator results in an error, processing continues for subsequent operators.

CONTINUE mode can be used to group independent operators (that is, process each operator regardless of the success or failure of the others).

SCAN
ICETOOL statements are checked for errors, but DFSORT is not called.

SCAN mode can be used to test ICETOOL statements for errors.

Note: SCAN mode is set automatically if an error is detected while in STOP mode.

MODE example

```
MODE SCAN
  RANGE ...
  UNIQUE ...
MODE STOP
  VERIFY ...
  DISPLAY ...
MODE CONTINUE
  COPY ...
  SORT ...
  STATS ...
```

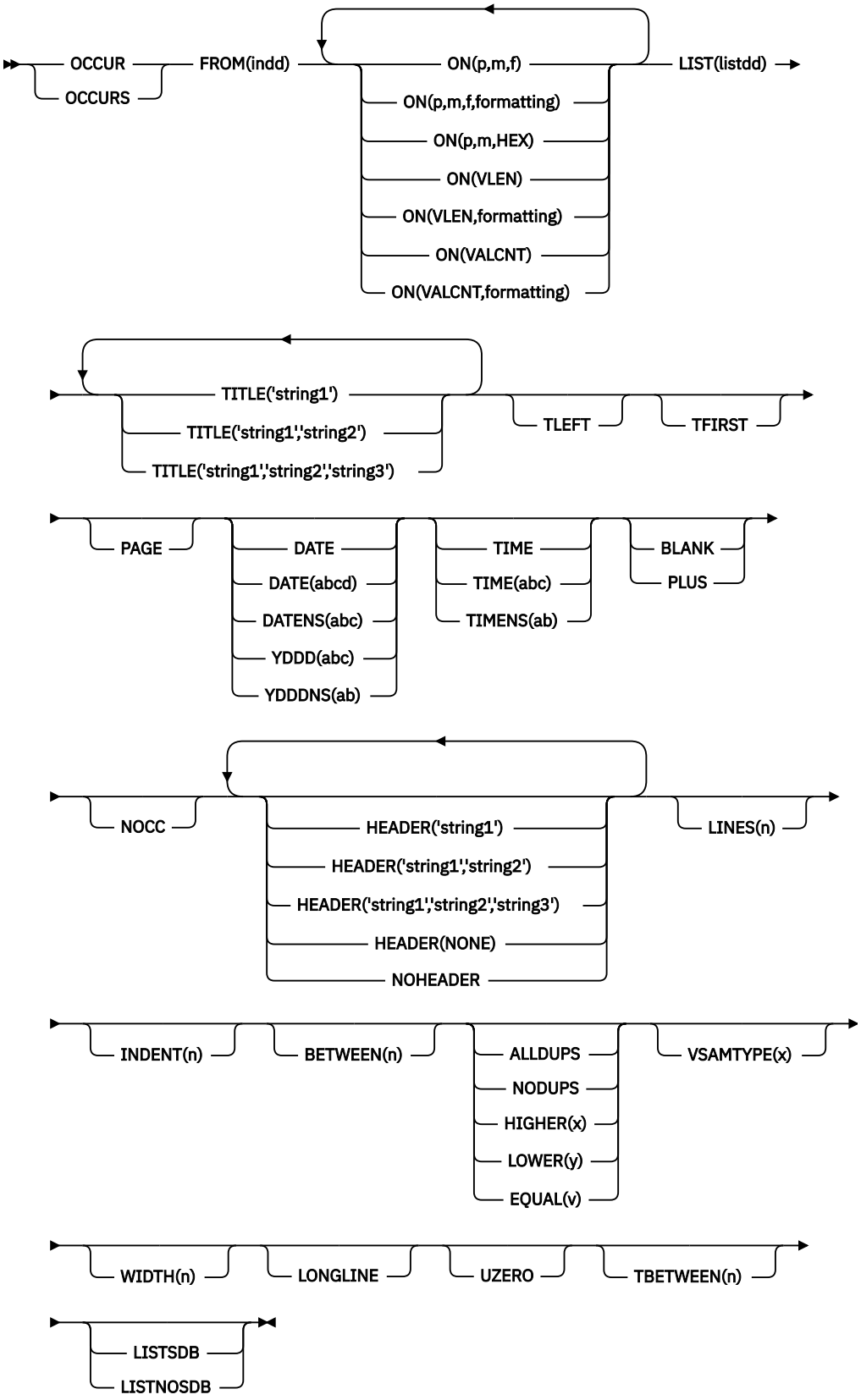
SCAN mode: RANGE and UNIQUE are checked for statement errors, but DFSORT is not called.

STOP mode: DISPLAY is dependent on VERIFY. If the return code for VERIFY is 12 or 16, SCAN mode is entered; DISPLAY is checked for statement errors, but DFSORT is not called.

CONTINUE mode: COPY, SORT, and STATS are independent of each other. SORT is processed even if the return code for COPY is 12 or 16. STATS is processed even if the return code for COPY or SORT is 12 or 16.

Note that the return codes for one group of operators does not affect the other groups of operators.

OCCUR operator



Prints each unique value for specified numeric fields (including SMF, TOD, and ETOD date and time) or character fields, and how many times it occurs, in a separate list data set. Simple or tailored reports can be produced. The values printed can be limited to those for which the value count meets specified criteria.

From 1 to 10 fields can be specified, but the resulting list data set line length must not exceed the limit specified by the WIDTH operand or 8192 bytes if LONGLINE is specified and WIDTH is not specified, or 2048 bytes if LONGLINE and WIDTH are not specified. At least one ON(VLEN) or ON(p,m,f) field must be specified; all such ON fields specified are used to determine whether a record contains a unique value. A single list data set record is printed for each unique value. If ON(VALCNT) is specified, the "value count" (that is, the number of times the ON values occur) is printed in the list data set record along with the other ON values.

Specifying formatting items or the PLUS or BLANK operand, which can "compress" the columns of output data, can enable you to include more fields in your report, up to a maximum of 10, if your line length is limited by the character width your printer or display supports.

ALLDUPS, NODUPS, HIGHER(x), LOWER(y) or EQUAL(v) can be specified to limit the ON values printed to those for which the value count meets the specified criteria (for example, ALLDUPS for duplicate values only). The default criteria is HIGHER(0) resulting in the ON values being printed for each unique value.

DFSORT is called to sort the indd data set to ICETOOL's E35 user exit. ICETOOL uses its E35 exit to print appropriate titles, headings and data in the list data set.

You must not supply your own DFSORT MODS, INREC, OUTREC, SUM, or RECORD statement, because they override the DFSORT statements passed by ICETOOL for this operator.

The DYNALLOC option is passed to DFSORT to ensure that work space is available for the sort. If your installation defaults for dynamic allocation are inappropriate for an OCCUR operator, you can take one of the following actions:

1. Override the DYNALLOC option using an OPTION control statement such as:

   ```
   OPTION DYNALLOC=(,8)
   ```

   in the DFSPARM data set.
2. Use SORTWKdd DD statements to override the use of dynamic allocation. Refer to “SORTWKdd DD statement” on page 69 for details.

Attention: Either of these actions affects the work data sets used for a UNIQUE operator, or for a SELECT or SPLICE operator for which USING(xxxx) is not specified.

Tape work data sets **cannot** be used with ICETOOL.

Simple report

You can produce a simple report by specifying just the required operands. For example, if you specify FROM and LIST operands, and ON operands for 10-byte character and 7-byte zoned decimal fields and the value count, the output in the list data set can be represented as follows:

```
(p,m,f)                (p,m,f)            VALUE COUNT
characters             sddddddddddddddd   ddddddddddddddd
.                                   .                 .
.                                   .                 .
.                                   .                 .
```

A control character occupies the first byte of each list data set record. Left-justified standard headings are printed at the top of each page to indicate the contents of each column, followed by a line for each record showing the characters and numbers in the fields of that record, and the count of occurrences (value count) of the specified values.

The fields are printed in columns in the same order in which they are specified in the OCCUR statement. All fields are left-justified. For numeric fields, leading zeros are printed, a - is used for the minus sign, and a + is used for the plus sign. For the value count, leading zeros are printed.

By default, the first column of data starts immediately after the control character, and three blanks appear between columns. The NOCC operand can be used to suppress the control character. The INDENT operand can be used to change the number of blanks before the first column of data. The BETWEEN operand can be used to change the number of blanks between columns.

The standard column widths are as follows:

- Character data: the length of the character field or 20 bytes if the field length is less than 21 bytes
- Numeric data: 16 bytes, or 32 bytes if the numeric field is BI or FI with a length greater than 4, PD with a length greater than 8, or ZD, CSF, FS, UFF or SFF with a length greater than 15.
- Value count: 15 bytes

HEADER operands can be used to change or suppress the headings. PLUS or BLANK operands can be used to change the format of numeric fields. PLUS, BLANK and HEADER operands can be used to change the width of the columns for numeric and character fields and the justification of headings and fields.

The NOHEADER operand can be used to create list data sets containing only data records. Data sets created in this way can be processed further by other operators (for example, STATS or UNIQUE) using CH format for character values or FS format for numeric values (including the value count).

Tailored report

You can tailor the output in the list data set using various operands that control title strings, date, time, page number, headings, lines per page and field formats. The optional operands can be used in many different combinations to produce a wide variety of report formats. For example, if you specify FROM, LIST, BLANK, TITLE, PAGE, DATE, TIME, and HEADER operands, and ON operands for 10-byte character and 7-byte zoned decimal fields and the value count, the output in the list data set looks as follows:

```
title        - p -        mm/dd/yy        hh:mm:ss

header         header           header
----------   --------   --------------
characters         sd                d
.                   .                .
.                   .                .
.                   .                .
```

By default, a control character occupies the first byte of each list data set record. The NOCC operand can be used to suppress the control characters. The title lines (up to 3) are printed at the top of each page of the list data set. The first title line contains the elements you specify (title strings,page number, date and time) in the order in which you specify them. The second and third title lines contain the title strings you specify. By default, eight blanks appear between title elements, the title strings are centered with respect to each other, and the title lines appear on every page. The TBETWEEN(n) operand can be used to change the number of blanks between title elements. The TLEFT operand can be used to left-justify the title strings with respect to each other. The TFIRST operand can be used to only print the title lines on the first page. A blank line is printed after each title line.

Your specified headings (underlined) are printed after the title line on each page to indicate the contents of each column, followed by a line for each record showing the characters and numbers in the fields of that record. Your specified headings can be one, two or three lines. Headings for character fields are left-justified and headings for numeric fields are right-justified.

The fields are printed in columns in the same order in which they are specified in the OCCUR statement. Character fields are left-justified and numeric fields are right justified. For numeric fields, leading zeros are suppressed, a - is used for the minus sign, and a blank is used for the plus sign (you can specify PLUS rather than BLANK if you want a + to be used for the plus sign). For the value count, leading zeros are suppressed.

Formatting items can be used to change the appearance of individual numeric fields in the report with respect to separators, number of digits, decimal point, decimal places, signs, leading zeros, leading strings, floating strings, and trailing strings. Formatting items can also be used to insert leading or trailing strings for character fields.

The column widths are dynamically adjusted according to the length of the headings and the maximum number of bytes needed for the character or numeric data.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

ON(p,m,f)
Specifies the position, length, and format of a numeric or character field to be used for this operation. '(p,m,f)' is used for the standard column heading (see HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), HEADER(NONE), and NOHEADER for alternative heading options).

By default, three blanks appear between columns. You can change the space between columns with BETWEEN(n).

p specifies the first byte of the field relative to the beginning of the input record. p is 1 for the first **data** byte of a fixed-length record and 5 for the first **data** byte of a variable-length record as illustrated in the following (RRRR represents the 4-byte record descriptor word):

```
   Fixed-length record    |    Variable-length record
   | D | A | T | A | ...  |    | R | R | R | R | D | A | T | A | ...
p=   1   2   3   4        | p=   1   2   3   4   5   6   7   8
```

m specifies the length of the field in bytes. A field must not extend beyond position 32752 or beyond the end of a record. The maximum length for a field depends on its format.

f specifies the format of the field as follows:

Format code	Length	Description
BI	1 to 8 bytes	Unsigned binary
FI	1 to 8 bytes	Signed fixed-point
PD	1 to 16 bytes	Signed packed decimal
ZD	1 to 31 bytes	Signed zoned decimal
CH	1 to 4000 bytes	Character
CSF or FS	1 to 32 bytes (31 digit limit)	Signed numeric with optional leading floating sign
UFF	1 to 44 bytes (31 digit limit)	Unsigned free form numeric
SFF	1 to 44 bytes (31 digit limit)	Signed free form numeric
DT1	4 bytes	SMF date interpreted as Z'yyyymmdd'
DT2	4 bytes	SMF date interpreted as Z'yyyymm'
DT3	4 bytes	SMF date interpreted as Z'yyyyddd'
DC1	8 bytes	TOD date interpreted as Z'yyyymmdd'
DC2	8 bytes	TOD date interpreted as Z'yyyymm'
DC3	8 bytes	TOD date interpreted as Z'yyyyddd'

Format code	Length	Description
DE1	8 bytes	ETOD date interpreted as Z'yyyymmdd'
DE2	8 bytes	ETOD date interpreted as Z'yyyymm'
DE3	8 bytes	ETOD date interpreted as Z'yyyyddd'
TM1	4 bytes	SMF time interpreted as Z'hhmmss'
TM2	4 bytes	SMF time interpreted as Z'hhmm'
TM3	4 bytes	SMF time interpreted as Z'hh'
TM4	4 bytes	SMF time interpreted as Z'hhmmssxx'
TC1	8 bytes	TOD time interpreted as Z'hhmmss'
TC2	8 bytes	TOD time interpreted as Z'hhmm'
TC3	8 bytes	TOD time interpreted as Z'hh'
TC4	8 bytes	TOD time interpreted as Z'hhmmssxx'
TE1	8 bytes	ETOD time interpreted as Z'hhmmss'
TE2	8 bytes	ETOD time interpreted as Z'hhmm'
TE3	8 bytes	ETOD time interpreted as Z'hh'
TE4	8 bytes	ETOD time interpreted as Z'hhmmssxx'
Note: See Appendix C, "Data format descriptions," on page 819 for detailed format descriptions.		

For a CSF, FS, UFF, or SFF format field:

- A maximum of 31 digits is allowed. If a value with more than 31 digits is found, ICETOOL issues an error message and terminates the operation.

For a ZD or PD format field:

- If a decimal value contains an invalid digit (A-F), ICETOOL identifies the bad value in a message and terminates the operation.
- F, E, C, A, 8, 6, 4, 2, and 0 are treated as equivalent positive signs. Thus the zoned decimal values F2F3C1, F2F3F1 and 020301 are counted as only one unique value.
- D, B, 9, 7, 5, 3, and 1 are treated as equivalent negative signs. Thus the zoned decimal values F2F3B0, F2F3D0, and 020310 are counted as only one unique value.

The fields of records that do not meet the specified criteria are not checked for invalid digits (PD and ZD) or excessive digits (CSF, FS, UFF, and SFF).

For a DT1, DT2 or DT3 format field:

- An invalid SMF date can result in a data exception (0C7 ABEND) or an incorrect ZD date.
- SMF date values are always treated as positive.

For a DC1, DC2, DC3, DE1, DE2, or DE3 format field:

- TOD and ETOD date values are always treated as positive.

For a TM1, TM2, TM3 or TM4 format field:

- An invalid SMF time can result in an incorrect ZD time.
- SMF time values are always treated as positive.

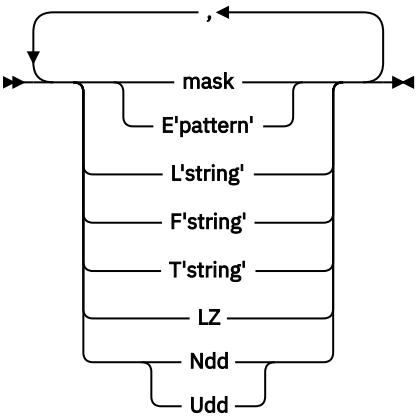
For a TC1, TC2, TC3, TC4, TE1, TE2, TE3, or TE4 format field:

- TOD and ETOD time values are always treated as positive.

ON(p,m,f,formatting)
Specifies the position, length and format of a numeric or character field to be used for this operation and how the data for this field is to be formatted for printing. The BLANK operand is automatically in effect.

See ON(p,m,f) for further details.

formatting



specifies formatting items that indicate how the data for this field is to be formatted for printing. Formatting items can be specified in any order, but each item can only be specified once. Any formatting item can be specified for a numeric field, but only L'string' and T'string' can be specified for a character field.

The column width is dynamically adjusted to accommodate the maximum bytes to be inserted as a result of all formatting items specified.

mask
See the discussion of **mask** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

E'pattern'
See the discussion of **E'pattern'** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

L'string'
See the discussion of **L'string'** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

F'string'
See the discussion of **F'string'** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

T'string'
See the discussion of **T'string'** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

LZ
See the discussion of **LZ** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

Ndd or Udd
specifies the number of digits to be used for the numeric field. Ndd or Udd can be used to change the column width for numeric fields. dd specifies the number of digits and must be a **two-digit** number between 01 and 31.

The default number of digits (d) for a numeric field is the maximum number of digits for that field. For example, d is 5 for ON(1,5,ZD). If you know that your numeric field requires less than d digits, you can use a lower number of digits (dd) instead by specifying Udd, thus reducing the column

width if it is determined by d. For example, ON(1,5,ZD,U03) reduces d from 5 to 3. If you want your numeric field to be displayed with more than d digits, you can use a higher number of digits (dd) instead by specifying Ndd or Udd, thus increasing the column width if it is determined by d. For example, ON(1,5,ZD,U10) increases d from 5 to 10.

Either Ndd or Udd can be used to set d greater than the maximum for a numeric field, but only Udd can be used to set d less than the maximum for a numeric field.

`For Udd:`

dd is used for d. For example:

- If ON(1,5,ZD) is specified, 5 digits (default for 5,ZD) are used.
- If ON(1,5,ZD,U10) is specified, 10 digits (from U10) are used.
- If ON(1,5,ZD,U03) is specified, 3 digits (from U03) are used.
- If ON(1,16,FS) is specified, 16 digits (default for 16,FS) are used.
- If ON(1,16,FS,U16) is specified, 16 digits (from U16) are used.
- If ON(1,16,FS,U15) is specified, 15 digits (from U15) are used.

If you use Udd and a numeric value overflows dd digits, ICETOOL terminates the operation. You can prevent the overflow by specifying an appropriately higher dd value for Udd. For example, if ON(1,12,ZD,U09) results in overflow, you can use ON(1,12,ZD,U10) instead.

If E'pattern' is specified, Udd is ignored, because the number of digits is determined from the pattern.

`For Ndd:`

If dd is greater than or equal to d, dd is used. If dd is less than d, d is used. For example:

- If ON(1,5,ZD) is specified, 5 digits (default for 5,ZD) are used.
- If ON(1,5,ZD,N10) is specified, 10 digits (from N10) are used.
- If ON(1,5,ZD,N03) is specified, 5 digits (from 5,ZD) are used.

If E'pattern' is specified, Ndd is ignored, because d is determined from the pattern.

ON(p,m,HEX)
Specifies the position and length of a character field to be used for this operation and printed in hexadecimal format (00-FF for each byte). '(p,m,HEX)' is used for the standard column heading (see HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), HEADER(NONE), and NOHEADER for alternative heading options).

See ON(p,m,f) for a discussion of **p**.

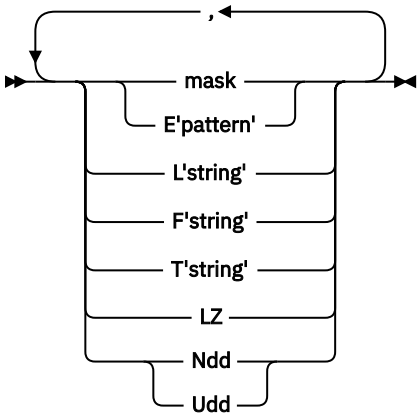
m specifies the length of the field in bytes. A field must not extend beyond position 32752 or beyond the end of a record. A field can be 1 to 2000 bytes.

ON(VLEN)
Equivalent to specifying ON(1,2,BI); a two-byte binary field starting at position 1. For variable-length records, ON(VLEN) represents the record-length for each record. 'RECORD LENGTH' is used for the standard column heading. See HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), HEADER(NONE), and NOHEADER for alternative heading options.

ON(VLEN,formatting)
Equivalent to specifying ON(1,2,BI,formatting); a two-byte binary field starting at position 1, and how the data for this field is to be formatted for printing. The BLANK operand is automatically in effect.

See ON(VLEN) for further details.

formatting



specifies formatting items that indicate how the data for this field is to be formatted for printing. Formatting items can be specified in any order, but each item can only be specified once.

The column width is dynamically adjusted to accommodate the maximum bytes to be inserted as a result of all formatting items specified.

mask
See the discussion of **mask** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

E'pattern'
See the discussion of **E'pattern'** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

L'string'
See the discussion of **L'string'** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

F'string'
See the discussion of **F'string'** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

T'string'
See the discussion of **T'string'** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

LZ
See the discussion of **LZ** under ON(p,m,f,formatting) in “DISPLAY operator” on page 562.

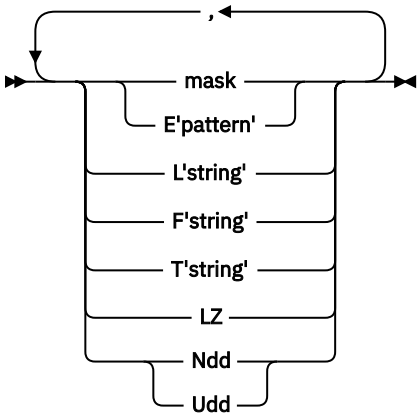
Ndd or Udd
See the discussion of Ndd or Udd under ON(p,m,f,formatting) in “OCCUR operator” on page 607.

ON(VALCNT)
Specifies that the number of occurrences for each unique value is to be printed. ' VALUE COUNT' is used for the standard column heading (see HEADER('string1'), HEADER('string1','string2'), HEADER('string1','string2','string3'), HEADER(NONE) and NOHEADER for alternative heading options).

ON(VALCNT,formatting)
Specifies that the number of occurrences for each unique value is to be printed, and how the value count is to be formatted for printing. The BLANK operand is automatically in effect.

See ON(VALCNT) for further details.

formatting



specifies formatting items that indicate how the value count is to be formatted for printing. Formatting items can be specified in any order, but each item can only be specified once.

The column width is dynamically adjusted to accommodate the maximum bytes to be inserted as a result of all formatting items specified.

mask
See the discussion of **mask** under ON(p,m,f,formatting) in "DISPLAY operator" on page 562.

E'pattern'
specifies an edit pattern to be applied to the value count. The pattern (1 to 24 characters) must be enclosed in single apostrophes. Each 9 in the pattern (up to 15) is replaced by a corresponding digit from the numeric value. Characters other than 9 in the pattern appear as specified. To include a single apostrophe (') in the pattern, specify two single apostrophes ('').

F'string' or a mask cannot be specified with E'pattern'.

When E'pattern' is specified for the value count:

- If the number of significant digits in a value count is less than the number of 9's in the pattern, 0's are filled in on the left. For example, 1234 is shown as 001234 with ON(VALCNT,E'999999').
- If the number of significant digits in a value count is greater than the number of 9's in the pattern, digits are truncated from the left. For example, 1234567 is shown as *4567* with ON(VALCNT,E'*9999*').

L'string'
See the discussion of **L'string'** under ON(p,m,f,formatting) in "DISPLAY operator" on page 562.

F'string'
See the discussion of **F'string'** under ON(p,m,f,formatting) in "DISPLAY operator" on page 562.

T'string'
See the discussion of **T'string'** under ON(p,m,f,formatting) in "DISPLAY operator" on page 562.

LZ
See the discussion of **LZ** under ON(p,m,f,formatting) in "DISPLAY operator" on page 562.

Ndd or Udd
Specifies the number of digits to be used for the value count when determining the column width. dd specifies the number of digits and must be a two-digit number between 01 and 15.

The default number of digits (d) for the value count is 15. If you know that your value counts require less than 15 digits, you can use a lower number of digits (dd) instead by specifying Ndd or Udd, thus reducing the column width if it is determined by d. For example, if ON(VALCNT,N06) or ON(VALCNT,U06) is specified, 6 digits (from N06 or U06) is used instead of 15 (default for value count).

If you use Ndd or Udd and a value count overflows the number of digits used, ICETOOL terminates the operation. You can prevent the overflow by specifying an appropriately higher dd value for

Ndd or Udd. For example, if ON(VALCNT,N05) results in overflow, you can use ON(VALCNT,N06) instead.

If E'pattern' is specified, Ndd or Udd is ignored, because d is determined from the pattern.

LIST(listdd)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

TITLE('string')
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

TITLE('string1','string2')
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

TITLE('string1','string2','string3')
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

TLEFT
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

TFIRST
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

PAGE
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

DATE
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

DATE(abcd)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

DATENS(abc)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

YDDD(abc)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

YDDDNS(ab)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

TIME
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

TIME(abc)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

TIMENS(ab)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

BLANK
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

PLUS
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

For ON(VALCNT), PLUS is treated as BLANK.

NOCC
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

HEADER('string1')

See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

HEADER('string1','string2')
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

HEADER('string1','string2','string3')
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

HEADER(NONE)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

NOHEADER

See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

LINES(n)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

INDENT(n)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

BETWEEN(n)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

TBETWEEN(n)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

ALLDUPS
Limits the ON values printed to those that occur more than once (that is, those with duplicate field values). The ON values are printed when value count > 1.

ALLDUPS is equivalent to HIGHER(1).

NODUPS
Limits the ON values printed to those that occur only once (that is, those with no duplicate field values). The ON values are printed when value count = 1.

NODUPS is equivalent to EQUAL(1) or LOWER(2).

HIGHER(x)
Limits the ON values printed to those that occur more than x times. The ON values are printed when value count > x.

x must be specified as n or +n where n can be 1 to 15 decimal digits.

LOWER(y)
Limits the ON values printed to those that occur less than y times. The ON values are printed when value count < y.

y must be specified as n or +n where n can be 1 to 15 decimal digits.

EQUAL(v)
Limits the ON values printed to those that occur v times. The ON values are printed when value count = v.

v must be specified as n or +n where n can be 1 to 15 decimal digits.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

WIDTH(n)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

LONGLINE
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

UZERO
Specifies that -0 and +0 are to be treated as unsigned zero values, that is, as the same value. With UZERO, -0 and +0 are treated as positive for ON processing.

UZERO overrides the default of treating -0 and +0 as signed zero values, that is, as different values. Without UZERO, -0 is treated as negative and +0 is treated as positive for ON processing.

LISTSDB or LISTNOSDB
See the discussion of these operands on the DISPLAY statement in "DISPLAY operator" on page 562.

OCCUR examples

Although the OCCUR operators in the examples in this section could all be contained in a single ICETOOL job step, they are shown and discussed separately for clarity. See “DISPLAY operator” on page 562 for additional examples of tailoring the report format.

Example 1

```
 OCCUR FROM(SOURCE) LIST(VOLSERS) ON(40,6,CH) ON(VALCNT)
```

Prints, in the VOLSERS data set:

- A heading line containing the standard headings
- A data line for each unique ON(40,6,CH) value in the standard format containing:
  - The characters from positions 40-45 of the SOURCE data set for the unique value
  - The count of occurrences in the SOURCE data set of the unique value

The VOLSERS output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
(40,6,CH)         VALUE COUNT
ABC001            000000000000025
ABC002            000000000000011
.                              .
.                              .
.                              .
```

The heading line appears at the top of each page.

Example 2

```
 OCCUR FROM(IN) LIST(LIST1) -
       TITLE('   3090 Distribution   ') -
       PAGE -
       HEADER('Data Centers')  ON(VALCNT) -
       HEADER('State') ON(1,16,CH) -
       HEADER('3090s') ON(25,3,PD) -
       BLANK
```

Prints, in the LIST1 data set:

- A title line containing the specified title and the page number
- A heading line containing the specified underlined headings
- A data line for each unique ON(1,16,CH) and ON(25,3,PD) value in the BLANK format containing:
  - The count of occurrences in the IN data set of the unique value
  - The characters from positions 1-16 of the IN data set for the unique value
  - The packed decimal values from positions 25-27 of the IN data set for the unique value

The LIST1 output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
   3090 Distribution          - 1 -

   Data Centers   State               3090s
---------------   ----------------   ------
             12   Alabama                 1
              6   Alabama                 2
              .   .                       .
              .   .                       .
              .   .                       .
```

The title line and underlined heading line appear at the top of each page.

Example 3

```
 OCCUR FROM(FAILURES) LIST(CHECKIT) BLANK HIGHER(4) -
   DATE TITLE('Possible System Intruders on ',System) -
        TITLE(Sysplex,' in ',Location) PAGE -
        TBETWEEN(2) -
        HEADER(,'Userid') ON(23,8,CH) -
        HEADER('Logon failures','(More than 4)') ON(VALCNT)
```

Assume that the SYMNAMES data set contains the following symbols:

```
System,S'&SYSNAME'
Sysplex,S'&SYSPLEX'
Location,'San Jose'
```

Prints, in the CHECKIT data set:

- A first title line containing the date, the string and System name specified in the first TITLE operand, and the page number. Two blanks appear between the title elements in the first title line as specified by the TBETWEEN(2) operand. A blank line follows the first title line.
- A second title line containing the Sysplex name, string and Location specified in the second TITLE operand. A blank line follows the second title line.
- An underlined three-line heading. The three-line heading for the first ON field has blanks on line1 and line2 and 'Userid' on line3. The heading for the second ON field has 'Number of' on line1, 'Logon Failures' on line2 and '(More than 4)' on line3.
- A data line for each unique ON(23,8,CH) value for which there are more than 4 occurrences, in the BLANK format, containing:
  - The characters from positions 23-30 of the FAILURES data set
  - The count of occurrences of the characters from positions 23-30 of the FAILURES data set

The CHECKIT output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
07/22/08  Possible System Intruders on EDS3  - 1 -

                  MAS3 in San Jose

            Logon failures
Userid       (More than 4)
--------   ---------------
B7234510                 5
D9853267                11
 ...
```

The title lines and underlined three-line heading lines appear at the top of each page.

Example 4

```
 OCCUR FROM(VARIN) LIST(ONCE) -
       TITLE('Record lengths that occur only once') -
       TIME(12:) DATE(DMY.) -
       ON(VLEN) NODUPS BLANK
```

Prints, in the ONCE data set:

- A title line containing the specified title and the time and date
- A heading line containing the standard heading
- A data line for each record length for which there is only one occurrence, in the BLANK format, containing the record length

The ONCE output starts on a new page and looks as follows (the first 2 records are shown with illustrative values):

```
Record lengths that occur only once        09:52:17 am        21.10.92

RECORD LENGTH
          57
          61
           .
           .
           .
```

The title line and heading line appear at the top of each page.

Example 5

```
OCCUR FROM(BRANCH) LIST(CALLRPT)-
      DATENS(4MD)-
      TITLE('Yearly Branch Phone Call Counts')-
      HEADER('Phone Number') ON(7,10,ZD,E'(999)-999-9999')-
      HEADER('Calls') ON(VALCNT,A1,N05)-
      INDENT(5) BETWEEN(10)
```

Prints, in the CALLRPT data set:

- A title line containing the date (without separators) and the specified title.
- A heading line containing the specified underlined headings.
- A data line for each unique ON(7,10,ZD) value containing:
  - The zoned decimal value from positions 7-16 of the BRANCH data set printed as (ddd)-ddd-dddd according to the E'pattern' formatting item.
  - The count of occurences of this value printed as dd,ddd according to the A1 and N05 formatting items.

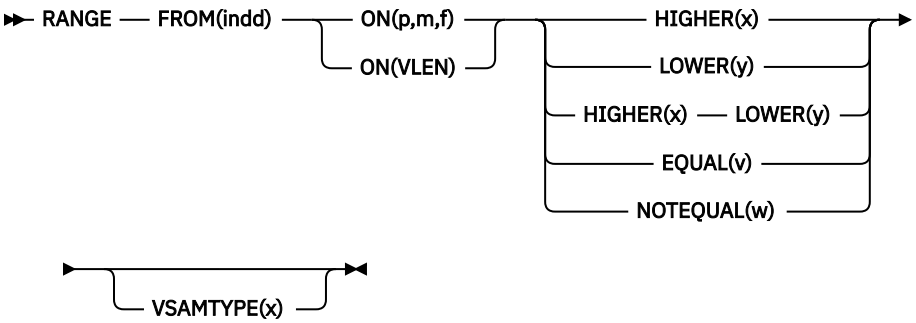
The report is indented by five blanks as specified by the INDENT(5) operand, and ten blanks appear between the columns as specified by the BETWEEN(10) operand.

The CALLRPT output starts on a new page and looks as follows:

```
     20020316       Yearly Branch Phone Call Counts

       Phone Number            Calls
     --------------          -------
     (037)-325-1807            3,125
     (216)-721-5530            2,087
     (218)-062-7214              872
```

RANGE operator



Prints a message containing the count of values in a specified range for a specific numeric field.

DFSORT is called to copy the indd data set to ICETOOL's E35 user exit. ICETOOL prints a message containing the range count as determined by its E35 user exit.

The range can be specified as higher than x, lower than y, higher than x and lower than y, equal to v, or not equal to w, where x, y, v, and w are signed or unsigned decimal values. If the range is specified as higher than x and lower than y, it must be a valid range (for example, higher than 5 and lower than 6 is not a valid range, because there is no integer value that satisfies the criteria).

You must not supply your own DFSORT MODS, INREC, or OUTREC statement, because they would override the DFSORT statements passed by ICETOOL for this operator.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

ON(p,m,f)
Specifies the position, length, and format of the numeric field to be used for this operation.

p specifies the first byte of the field relative to the beginning of the input record. p is 1 for the first **data** byte of a fixed-length record and 5 for the first **data** byte of a variable-length record as illustrated in the following (RRRR represents the 4-byte record descriptor word):

```
    Fixed-length record    |    Variable-length record
    | D | A | T | A | ...  |    | R | R | R | R | D | A | T | A | ...
 p=   1   2   3   4        | p=   1   2   3   4   5   6   7   8
```

m specifies the length of the field in bytes. A field must not extend beyond position 32752 or beyond the end of a record. The maximum length for a field depends on its format.

f specifies the format of the field as follows:

Format code	Length	Description
BI	1 to 8 bytes	Unsigned binary
FI	1 to 8 bytes	Signed fixed-point
PD	1 to 16 bytes	Signed packed decimal
ZD	1 to 31 bytes	Signed zoned decimal
CSF or FS	1 to 32 bytes (31 digit limit)	Signed numeric with optional leading floating sign
UFF	1 to 44 bytes (31 digit limit)	Unsigned free form numeric
SFF	1 to 44 bytes (31 digit limit)	Signed free form numeric

Note: See Appendix C, "Data format descriptions," on page 819 for detailed format descriptions.

For a CSF, FS, UFF, or SFF format field:

- A maximum of 31 digits is allowed. If a value with more than 31 digits is found, ICETOOL issues an error message and terminates the operation.

For a ZD or PD format field:

- If a decimal value contains an invalid digit (A-F), ICETOOL identifies the bad value in a message and terminates the operation.
- A value is treated as positive if its sign is F, E, C, A, 8, 6, 4, 2, or 0.
- A value is treated as negative if its sign is D, B, 9, 7, 5, 3, or 1.

For a ZD, PD, CSF, FS, or SFF format field, a negative zero value is treated as a positive zero value.

ON(VLEN)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

HIGHER(x)
Values higher than x are counted as contained in the range. If only HIGHER(x) is specified, the range count is incremented when x < value. If LOWER(y) is also specified, the range count is incremented when x < value < y.

x must be specified as n, +n, or -n where n can be 1 to 31 decimal digits.

LOWER(y)
Values lower than y are counted as contained in the range. If only LOWER(y) is specified, the range count is incremented when value < y. If HIGHER(x) is also specified, the range count is incremented when x < value < y.

y must be specified as n, +n, or -n where n can be 1 to 31 decimal digits.

EQUAL(v)
Values equal to v are counted as contained in the range. The range count is incremented when v = value.

v must be specified as n, +n, or -n where n can be 1 to 31 decimal digits.

NOTEQUAL(w)
Values not equal to w are counted as contained in the range. The range count is incremented when w ¬= value.

w must be specified as n, +n, or -n where n can be 1 to 31 decimal digits.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

RANGE example

```
RANGE FROM(DATA1) ON(VLEN) HIGHER(10)
RANGE FROM(DATA2) ON(31,18,ZD) LOWER(+123456789012345678)
RANGE FROM(DATA3) ON(29001,4,FI) -
       HIGHER(-10000) LOWER(27)
RANGE FROM(DATA2) ON(45,3,PD) EQUAL(-999)
RANGE FROM(DATA3) ON(40,1,BI) NOTEQUAL(199)
```

The first RANGE operator prints a message containing the count of binary values from positions 1-2 of the DATA1 data set that are higher than 10.

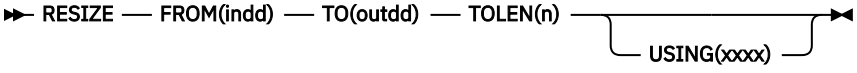
The second RANGE operator prints a message containing the count of zoned decimal values from positions 31-48 of the DATA2 data set that are lower than 123456789012345678.

The third RANGE operator prints a message containing the count of fixed-point values from positions 29 001-29 004 of the DATA3 data set that are higher than -10 000 but lower than 27.

The fourth RANGE operator prints a message containing the count of packed decimal values from positions 45-47 of the DATA2 data set that are equal to -999.

The fifth RANGE operator prints a message containing the count of binary values from position 40 of the DATA3 data set that are not equal to 199. This RANGE operator could be used to count the number of records that do not have 'G' in position 40, because 199 (X'C7') is the EBCDIC code for 'G'. Alternatively, the COUNT operator could be used with OMIT COND=(40,1,CH,EQ,C'G').

RESIZE operator



Produces fixed length output records, with the specified TOLEN length, from fixed length input records of a different length, as follows:

- If the input length is smaller than the requested TOLEN size, multiple smaller input records are combined into one larger output record of size TOLEN. If the combined input records do not completely

fill up an output record, the output record will be padded on the right with blanks. For example, if we have five input records of 20 bytes and TOLEN is 70, two output records of 70 bytes will be created. The first output record will have input records 1-3 (60 bytes) followed by 10 blanks, and the second output record will have input records 4-5 (40 bytes) followed by 30 blanks. Note that the first 10 bytes of input record 4 are not used for the first output record; **bytes or words are not "wrapped" between records**.

To illustrate, if the following RESIZE operator was specified:

```
RESIZE FROM(FB20) TO(FB70) TOLEN(70)
```

and FB20 had these 20 byte input records:

```
<11111111111111111>
<22222222222222222>
<33333333333333333>
<44444444444444444>
<55555555555555555>
```

FB70 would have these 70 byte output records:

```
<11111111111111111><22222222222222222><33333333333333333>
<44444444444444444><55555555555555555>
```

- If the input length is larger than the requested TOLEN size, each larger input record is broken up into multiple smaller output records of size TOLEN. If a broken up input record does not completely fill up the multiple output records, the last of the multiple output records will be padded on the right with blanks. For example, if we have two input records of 50 bytes and TOLEN is 20, six output records of 20 bytes each will be created. The first two output records will have bytes 1-20 and 21-40 of the first input record (40 bytes), respectively. The third output record will have bytes 41-50 of the first input record (10 bytes) followed by 10 blanks. Likewise, the fourth output record will have bytes 1-20 of the second input record, the fifth output record will have bytes 21-40 of the second input record, and the sixth output record will have bytes 41-50 of the second input record followed by 10 blanks.

  To illustrate, if the following RESIZE operator was specified:

  ```
  RESIZE FROM(F50) TO(F20) TOLEN(20)
  ```

  and F50 had these 50 byte input records:

  ```
  <11111111111111111><22222222222222222><33333333>
  <44444444444444444><55555555555555555><66666666>
  ```

  F20 would have these 20 byte output records:

  ```
  <11111111111111111>
  <22222222222222222>
  <33333333>
  <44444444444444444>
  <55555555555555555>
  <66666666>
  ```

You must specify the FROM(indd), TO(outdd) and TOLEN(n) operands. The FROM data set must be fixed-length (for example, RECFM=FB). The TO data set must also be fixed-length (unless you use an OUTFIL statement with the FTOV operand to change the fixed-length resized records to variable-length output records). If a VSAM input data set is used, it will be treated as TYPE=F (fixed-length) by default. If you specify a variable-length input data set (or use TYPE=V for a VSAM input data set), the RESIZE operation will be terminated.

The USING(xxxx) operand is optional; do not supply your own MODS or OUTREC statement.

DFSORT is called to copy or sort the indd data set, as appropriate, before the records are resized. ICETOOL uses its E35 exit to create a larger record from multiple smaller records, or to create multiple smaller records from a larger record. ICETOOL passes the EQUALS option to DFSORT to ensure that if records are sorted, duplicate records are kept in their original input order when resized.

If USING(xxxx) is specified, any SORT, INCLUDE, OMIT, INREC, or SUM statement specified in xxxxCNTL is processed **before** the records are resized. The order of the records, and the input length, will be affected by these control statements. Any OUTFIL statements specified in xxxxCNTL are processed **after** the records are resized.

The DYNALLOC option is passed to DFSORT to ensure that work space is available for the sort. If your installation defaults for dynamic allocation are inappropriate for a RESIZE operator, you can take one of the following actions:

1. Override the DYNALLOC option using an OPTION control statement such as:

   ```
   OPTION DYNALLOC=(,8)
   ```

   in the xxxxCNTL data set.
2. Use xxxxWKdd DD statements to override the use of dynamic allocation. Refer to "SORTWKdd DD statement" on page 69 for details.

Tape work data sets **cannot** be used with ICETOOL.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

TO(outdd)
See the discussion of this operand on the DATASORT statement in "DATASORT operator" on page 554.

TOLEN(n)
Specifies the record length you want ICETOOL to use for the resized output records. n can be 1 to 32760. n must not be equal to the input record length.

USING(xxxx)
Specifies the first 4 characters of the ddname for the control statement data set to be used by DFSORT for this operation. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx. If USING(xxxx) is specified, an xxxxCNTL DD statement must be present, and the control statements in it must conform to the rules for DFSORT's SORTCNTL data set.

RESIZE examples

Although the RESIZE operators in the examples in this section could all be contained in a single ICETOOL job step, they are shown and discussed separately for clarity.

Example 1

```
RESIZE FROM(IN1) TO(OUT1) TOLEN(40)
```

This example illustrates how you can create larger records from smaller records.

The IN1 data set has RECFM=FB and LRECL=10 with these 10-byte records:

```
Bird
Bluejay
4
Charlie
Rodent
Rat
2
Sara
```

The OUT1 data set has RECFM=FB and LRECL=40 with these 40 byte records:

```
Bird      Bluejay   4         Charlie
Rodent    Rat       2         Sara
```

Example 2

```
RESIZE FROM(OLD) TO(NEW) TOLEN(15) USING(CTL1)
```

This example illustrates how you can create larger records from a subset of smaller sorted records.

The CTL1CNTL data set contains:

```
    OMIT COND=(2,4,ZD,EQ,0)
    SORT FIELDS=(1,1,CH,A)
```

The OLD data set has RECFM=FB and LRECL=5 with these 5-byte records:

```
C0005
B0000
A0008
I1234
F0053
D0123
H0001
G0000
E0022
```

The NEW data set will have RECFM=FB and LRECL=15 with these 15-byte records:

```
A0008C0005D0123
E0022F0053H0001
I1234
```

Note that before the records were resized, the two records with 0 in positions 2-5 were omitted, and the remaining records were sorted as directed by the DFSORT control statements in CTL1CNTL. The last output record was padded with blanks on the right to 15 bytes.

Example 3

```
RESIZE FROM(IN3) TO(OUT3) TOLEN(3) USING(CTL2)
```

This example illustrates how you can break up large records into multiple smaller records.

The CTL2CNTL data set contains:

```
  OUTFIL FNAMES=OUT3,OMIT=(1,3,CH,EQ,C' '),OVERLAY=(10:X)
```

The IN3 data set has RECFM=FB and LRECL=18 with these 18-byte records:

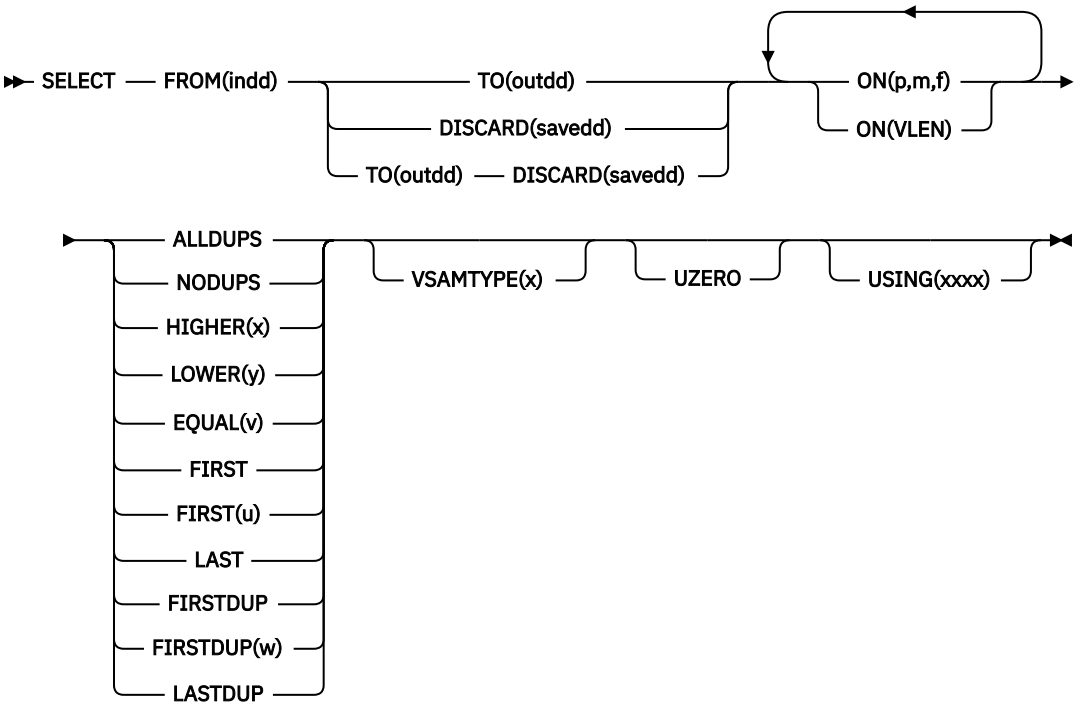
```
000111222333444555
666777888999
```

Every 3-byte field in each large IN3 record will be broken up into a single 3-byte field and then padded on the right with blanks to 10-bytes. TOLEN(3) indicates that the resized records will have a length of 3 bytes. OVERLAY=(10:X) expands each resized record to 10 bytes in OUT3. OMIT=(1,3,CH,EQ,C' ') removes any resized records that are completely blank (that is, the two blank resized records resulting from the blanks in positions 13-18 of the second input record).

The OUT3 data set will have RECFM=FB and LRECL=10 with these 10-byte records:

```
000
111
222
333
444
555
666
777
888
999
```

SELECT operator



Selects records from an input data set based on meeting criteria for the number of times specified numeric or character field values occur. This makes it possible to only keep records with duplicate field values, only keep records with no duplicate field values, only keep records with field values that occur more than, less than, or exactly n times, only keep the first or first n duplicate records with each field value, or only keep the first or last record with each unique or duplicate field value. From 1 to 10 fields can be specified. At least one ON(VLEN) or ON(p,m,f) field must be specified; all such ON fields specified will be used to determine the "value count" (that is, the number of times the ON values occur) to be matched against the criteria.

DISCARD(savedd) can be used to save the records that do not meet the criteria (that is, the discarded records), in the savedd data set. DISCARD(savedd) can be used with or without TO(outdd).

DFSORT is called to sort the indd data set. ICETOOL uses its E35 exit to determine which records to include in the outdd data set or savedd data set. ICETOOL passes the EQUALS option to DFSORT to ensure that duplicates are kept in their original input order.

The DFSORT control statements in xxxxCNTL are used if USING(xxxx) is specified.

Do not supply your own MODS, SUM or OUTREC statement.

You can use comment statements. You can use INCLUDE, OMIT, INREC, OPTION, SORT, or OUTFIL statements providing you observe these rules:

- You can use an INCLUDE or OMIT statement to remove input records **before** SELECT processing.
- You can use an INREC statement to reformat input records **before** SELECT processing. You can use INREC's PARSE, BUILD (FIELDS), OVERLAY, FINDREP, IFTHEN, or IFOUTLEN functions. If your INREC statement changes the starting position of an ON field, you must specify the new starting position for that ON field. For example, if your input records have a CH key at positions 1-5 and you use an INREC statement like this:

  ```
  INREC FIELDS=(25:1,50)
  ```

  you must specify ON(25,5,CH) instead of ON(1,5,CH).
- If you specify a SORT statement, you must specify each ON field as a p,m,f,A sort field and these sort fields must be in the same order as the ON fields. After these sort fields, you can specify additional

p,m,f,A or p,m,f,D sort fields. The additional sort fields will be used for sorting, but not for selecting. For example, if you use a SELECT statement like this:

```
SELECT FROM(IN) TO(OUT) ON(21,5,CH) FIRST(3) USING(CTL1)
```

you can use a SORT statement in CTL1CNTL like this:

```
SORT FIELDS=(21,5,CH,A,41,6,CH,D)
```

The records will be sorted by the 21,5,CH,A field and the 41,6,CH,D field, but only selected by the 21,5,CH field. This would allow you to select the three highest 41,6,CH values for each 21,5,CH value.

- If you specify TO(outdd) without DISCARD(savedd), you can further process the outdd records **after** SELECT processing using an OUTFIL statement like this:

  ```
  OUTFIL FNAMES=outdd,...
  ```

  or multiple OUTFIL statements like this:

  ```
  OUTFIL FNAMES=outdd,...
  OUTFIL FNAMES=outdd1,...
  ...
  ```

- If you specify DISCARD(savedd) without TO(outdd), you can further process the savedd records **after** SELECT processing using one (and only one) OUTFIL statement like this:

  ```
  OUTFIL FNAMES=savedd,...
  ```

- If you specify TO(outdd) and DISCARD(savedd), you can further process the outdd and savedd records **after** SELECT processing using two (and only two) OUTFIL statements like this:

  ```
  OUTFIL FNAMES=outdd,...
  OUTFIL FNAMES=savedd,...
  ```

  Both statements must be specified in the order shown with at least the FNAMES parameter. For example, to further modify only the DISCARD data set, you could use statements like this:

  ```
  OUTFIL FNAMES=OUT
  OUTFIL FNAMES=SAVE,INCLUDE=(21,3,ZD,GT,+25)
  ```

ICETOOL requires extra storage for SELECT processing, over and above what is normally needed by ICETOOL and DFSORT, in order to save your records until it can determine whether or not they meet your specified criteria. In most cases, only a small amount of storage is needed and can be obtained (above 16MB virtual). However, for a FROM data set with a large record length and criteria requiring many saved records, a large amount of storage is needed. For example, with a record length of 32756 and HIGHER(99), over 3 MBs of storage is needed. If ICETOOL cannot get the storage it needs, it issues a message and terminates the SELECT operation. Increasing the REGION by the amount indicated in the message may allow ICETOOL to run successfully.

The DYNALLOC option is passed to DFSORT to ensure that work space is available for the sort. If your installation defaults for dynamic allocation are inappropriate for a SELECT operator, you can specify USING(xxxx) and take one of the following actions:

1. Override the DYNALLOC option using an OPTION control statement such as:

   ```
        OPTION DYNALLOC=(,8)
   ```

   in the xxxxCNTL data set.
2. Use xxxxWKdd DD statements to override the use of dynamic allocation. Refer to "SORTWKdd DD statement" on page 69 for details.

Tape work data sets **cannot** be used with ICETOOL.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

TO(outdd)
Specifies the ddname of the output data set to which DFSORT will write the records it selects for the operation (that is, the records that meet the specified criteria). Thus, the outdd data set will contain the records selected by ALLDUPS, NODUPS, HIGHER(x), LOWER(y), EQUAL(v), FIRST, FIRST(u), LAST, FIRSTDUP, FIRSTDUP(w) or LASTDUP.

An outdd DD statement must be present and must define an output data set that conforms to the rules for DFSORT's SORTOUT data set (if the DISCARD operand is not specified) or OUTFIL data set (if the DISCARD operand is specified).

TO and DISCARD can both be specified. If DISCARD is not specified, TO must be specified. If TO is not specified, DISCARD must be specified.

The ddname specified in the TO operand must not be the same as the ddname specified in the FROM or DISCARD operand.

Refer to "JCL restrictions" on page 542 for more information.

DISCARD(savedd)
Specifies the ddname of the output data set to which DFSORT will write the records it does not select for this operation (that is, the records that do not meet the specified criteria). Thus, the savedd data set will contain the records discarded by ALLDUPS, NODUPS, HIGHER(x), LOWER(y), EQUAL(v), FIRST, FIRST(u), LAST, FIRSTDUP, FIRSTDUP(w) or LASTDUP.

A savedd DD statement must be present and must define an output data set that conforms to the rules for DFSORT's OUTFIL data set.

TO and DISCARD can both be specified. If DISCARD is not specified, TO must be specified. If TO is not specified, DISCARD must be specified.

The ddname specified in the DISCARD operand must not be the same as the ddname specified in the FROM or TO operand.

Refer to "JCL restrictions" on page 542 for more information.

ON(p,m,f)
Specifies the position, length, and format of a numeric or character field to be used for this operation.

p specifies the first byte of the field relative to the beginning of the input record. p is 1 for the first **data** byte of a fixed-length record and 5 for the first **data** byte of a variable-length record as illustrated in the following (RRRR represents the 4-byte record descriptor word):

```
    Fixed-length record    |    Variable-length record
    | D | A | T | A | ...  |    | R | R | R | R | D | A | T | A | ...
 p=   1   2   3   4        | p=   1   2   3   4   5   6   7   8
```

If INREC is specified, p must refer to the record as reformatted by INREC.

m specifies the length of the field in bytes. A field must not extend beyond position 32752, or beyond the end of a record. If INREC is specified, a field must not extend beyond the end of the record as reformatted by INREC. The maximum length for a field depends on its format.

f specifies the format of the field as shown in the following table.

Format code	Length	Description
BI	1 to 1500 bytes	Unsigned binary
FI	1 to 256 bytes	Signed fixed-point

Format code	Length	Description
PD	1 to 16 bytes	Signed packed decimal
ZD	1 to 31 bytes	Signed zoned decimal
CH	1 to 4000 bytes	Character
CSF or FS	1 to 32 bytes	Signed numeric with optional leading floating sign
UFF	1 to 44 bytes	Unsigned free form numeric
SFF	1 to 44 bytes	Signed free form numeric

Note: See Appendix C, "Data format descriptions," on page 819 for detailed format descriptions.

For a ZD or PD format field:

- F, E, C, A, 8, 6, 4, 2, and 0 are treated as equivalent positive signs. Thus the zoned decimal values F2F3C1, F2F3F1 and 020301 are counted as only one unique value.
- D, B, 9, 7, 5, 3, and 1 are treated as equivalent negative signs. Thus the zoned decimal values F2F3B0, F2F3D0, and 020310 are counted as only one unique value.
- Digits are not checked for validity.

ON(VLEN)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

ALLDUPS
Limits the records selected to those with ON values that occur more than once (value count > 1). You can use this operand to keep just those records with duplicate field values.

ALLDUPS is equivalent to HIGHER(1).

NODUPS
Limits the records selected to those with ON values that occur only once (value count = 1). You can use this operand to keep just those records with no duplicate field values.

NODUPS is equivalent to EQUAL(1) or LOWER(2).

HIGHER(x)
Limits the records selected to those with ON values that occur more than x times (value count > x). You can use this operand to keep just those records with field values that occur more than x times.

x must be specified as n or +n where n can be 0 to 99.

LOWER(y)
Limits the records selected to those with ON values that occur less than y times (value count < y). You can use this operand to keep just those records with field values that occur less than y times.

y must be specified as n or +n where n can be 0 to 99.

EQUAL(v)
Limits the records selected to those with ON values that occur v times (value count = v). You can use this operand to keep just those records with field values that occur v times.

v must be specified as n or +n where n can be 0 to 99.

FIRST
Limits the records selected to those with ON values that occur only once (value count = 1) and the first record of those with ON values that occur more than once (value count > 1). You can use this operand to keep just the first record for each unique field value.

FIRST is equivalent to FIRST(1).

FIRST(u)
Limits the records selected to those with ON values that occur only once (value count = 1) and the first u records of those with ON values that occur more than once (value count > 1). You can use this operand to keep just the first u records for each unique field value.

u must be specified as n or +n where n can be 1 to 999999999999999.

LAST
Limits the records selected to those with ON values that occur only once (value count = 1) and the last record of those with ON values that occur more than once (value count > 1). You can use this operand to keep just the last record for each unique field value.

FIRSTDUP
Limits the records selected to the first record of those with ON values that occur more than once (value count > 1). You can use this operand to keep just the first record of those records with duplicate field values.

FIRSTDUP is equivalent to FIRSTDUP(1).

FIRSTDUP(w)
Limits the records selected to the first w records of those with ON values that occur more than once (value count > 1). You can use this operand to keep just the first w records of those records with duplicate field values.

w must be specified as n or +n where n can be 1 to 999999999999999.

LASTDUP
Limits the records selected to the last record of those with ON values that occur more than once (value count > 1). You can use this operand to keep just the last record of those records with duplicate field values.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

UZERO
See the discussion of this operand on the OCCUR statement in “OCCUR operator” on page 607.

USING(xxxx)
Specifies the first 4 characters of the ddname for the control statement data set to be used by DFSORT for this operation. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx.

If USING is specified, an xxxxCNTL DD statement must be present and the control statements in it:

1. Must conform to the rules for DFSORT's SORTCNTL data set.
2. Should generally be used only for an INCLUDE or OMIT statement, an INREC statement, a SORT statement, comment statements, or OUTFIL statements as described for “SELECT operator” on page 626.

SELECT examples

Although the SELECT operators in the examples in this section could all be contained in a single ICETOOL job step, they are shown and discussed separately for clarity.

Example 1

```
SELECT FROM(INPUT) TO(DUPS) ON(11,8,CH) ON(30,44,CH) ALLDUPS
```

Sorts the INPUT data set to the DUPS data set, selecting only the records from INPUT with characters in positions 11-18 and characters in positions 30-73 that occur more than once (that is, only records with duplicate ON field values).

The DUPS data set might look as follows (several records are shown for illustrative purposes):

```
USR002    BETTEN       12    DOC.EXAMPLES
DFSRT2    BETTEN        5    DOC.EXAMPLES
DFSRT5    BOENIG       20    MYDATA
DFSRT1    BOENIG       20    MYDATA
SYS003    BOENIG       20    MYDATA
DFSRT2    BOENIG       20    SORTST1.TEST
USR003    BOENIG       20    SORTST1.TEST
.         .            .     .
.         .            .     .
.         .            .     .
```

Example 2

```
  SELECT FROM(INPUT) TO(ONLYONE) ON(23,3,FS) NODUPS
```

Sorts the INPUT data set to the ONLYONE data set, selecting only the records from INPUT with floating sign values in positions 23-25 that occur just once (that is, only records with no duplicate ON field values).

The ONLYONE data set might look as follows (several records are shown for illustrative purposes):

```
DFSRT2    BOENIG        5    DOC.EXAMPLES
DFSRT1    PACKER        8    ICETOOL.SMF.RUNS
USR002    BETTEN       12    DOC.EXAMPLES
SYS003    YAEGER       32    ICETOOL.TEST.CASES
DFSRT2    CORNELL     108    FS.TEST.CASES
.         .           .      .
.         .           .      .
.         .           .      .
```

Example 3

```
  SELECT FROM(FAILURES) TO(CHECKOUT) ON(28,8,CH) ON(1,5,CH) -
    HIGHER(3)
```

Sorts the FAILURES data set to the CHECKOUT data set, selecting only the records from FAILURES with characters in positions 28-35 and characters in positions 1-5 that occur more than three times (that is only records with four or more duplicate ON field values).

The CHECKOUT data set might look as follows (several records are shown for illustrative purposes):

```
03/12/04   08:36:59        A3275647
03/12/04   09:27:32        A3275647
03/12/04   09:03:18        A3275647
03/12/04   08:56:13        A3275647
03/06/04   15:12:01        C3275647
03/06/04   14:57:00        C3275647
03/06/04   15:43:19        C3275647
03/06/04   16:06:39        C3275647
03/06/04   15:22:08        C3275647
.          .               .
.          .               .
.          .               .
```

Example 4

```
  SELECT FROM(BOOKS) TO(PUBLISHR) ON(29,10,CH) FIRST
```

Sorts the BOOKS data set to the PUBLISHR data set, selecting only the records from BOOKS with characters in positions 29-38 that occur only once and the first record of those with characters in positions 29-38 that occur more than once (that is, one record for each unique ON field value).

The PUBLISHR data set might look as follows (several records are shown for illustrative purposes):

```
Banana Slugs I Have Known   Brent      Animals
Toads on Parade             Cooper     Animals
Pets Around the World       Davis      Animals
```

```
.                          .          .
.                          .          .
.                          .          .
```

Example 5

```
  SELECT FROM(BOOKS) TO(PUBLISHR) ON(29,10,CH) FIRST -
    DISCARD(SAVEREST)
```

This example creates the same PUBLISHR data set as Example 4. In addition, it creates a SAVEREST data set that contains all of the records not written to the PUBLISHR data set. The SAVEREST data set might look as follows (several records are shown for illustrative purposes):

```
How to Talk to Your Amoeba  Brent    Animals
What Buzzards Want          Davis    Animals
Birds of Costa Rica         Davis    Animals
.
.
.
```

Example 6

```
  SELECT FROM(MASTPULL) TO(MATCH) ON(5,8,CH) FIRSTDUP
```

This example shows how you can use a list of account numbers in a "pull" data set to only select records with those account numbers from a "master" data set. The MASTPULL DD would have the "master" data set and "pull" data set concatenated together (in that order).

The SELECT operator sorts the concatenated data sets and selects only the first record of those with characters in positions 5-12 that occur more than once (that is, one record for each duplicate ON field value). Because the "master" data set is first in the concatenation, the selected records will come from the "master" data set.

If the "master" data set looked like this:

```
A52 RB172832 2001/03/15
N92 MX328126 2001/01/27
B12 LB018725 2000/12/28
J73 AB007231 2001/02/13
Q28 SP973004 2000/11/19
```

and the "pull" data set looked like this:

```
   AB007231
   RS859276
   QN005001
   MX328126
```

the MATCH data set would look like this:

```
J73 AB007231 2001/02/13
N92 MX328126 2001/01/27
```

Note: This example assumes that there are not any duplicate account numbers in either the "master" or "pull" data sets. If that is not true, you can use SELECT with FIRST or LAST, for the appropriate data set, to make it true. For example, if your "master" data set has duplicate account numbers and you want to select the first account number from the "master" data set for each account number in the "pull" data set, you could use the following statements:

```
  SELECT FROM(MASTER) TO(TEMP) ON(5,8,CH) FIRST
  SELECT FROM(TEMPPULL) TO(MATCH) ON(5,8,CH) FIRSTDUP
```

The TEMPPULL DD would have the temporary data set and "pull" data set concatenated together (in that order).

Example 7

```
  SELECT FROM(INPUT) TO(ONLYONE) ON(23,3,FS) NODUPS USING(CTL1)
```

This example shows how you can use USING(xxxx) to supply an OUTFIL statement to modify the TO data set. SELECT chooses the same output records as for “Example 2” on page 631, but an OUTFIL statement is used to further modify those records for output to the ONLYONE data set.

The CTL1CNTL data set contains:

```
  OUTFIL FNAMES=ONLYONE,
    REMOVECC,
    INCLUDE=(23,3,FS,LT,100),
    OUTREC=(1:1,7,8:C'|',11:11,7,19:C'|',23:23,3,FS,M11,
            27:C'|',30:30,15),
    TRAILER1=(/,'TOTAL= ',TOT=(23,3,FS,M11,LENGTH=6))
```

and the ONLYONE data set might look as follows:

```
DFSRT2 |  EISSLER |   005 |  DOC.EXAMPLES
DFSRT1 |  PACKER  |   008 |  ICETOOL.SMF.RUN
USR002 |  EISSLER |   012 |  DOC.EXAMPLES
SYS003 |  YAEGER  |   032 |  ICETOOL.TEST.CA

TOTAL= 000057
```

Example 8

This example shows how you can use USING(xxxx) to supply an OMIT statement to remove certain input records and an INREC statement to reformat certain input records before SELECT processing begins.

```
 SELECT FROM(INB) TO(OUTB) ON(8,10,CH) -
   FIRSTDUP USING(BIRD)
```

Let's say the INB data set looks like this:

```
TYPE1  CROWS
TYPE1  PIGEONS
TYPE1  HAWKS
TYPE1  ROBINS
TYPE1  SPARROWS
TYPE1  CHICKENS
TYPE1  RAVENS
TYPE2            PIGEONS
TYPE2            ROBINS
TYPE2            HAWKS
TYPE2            TERNS
TYPE3  EAGLES
TYPE3  STARLINGS
```

We want to remove the TYPE3 records and then select one record for each type of bird that appears in both a TYPE1 and a TYPE2 record. We could use the following control statements in BIRDCNTL:

```
* Remove the TYPE3 records.
  OMIT COND=(1,5,CH,EQ,C'TYPE3')
* Reformat the TYPE2 records to place the bird name in
* same place as in the TYPE1 records so we can match them.
  INREC IFTHEN=(WHEN=(1,5,CH,EQ,C'TYPE2'),
         BUILD=(8:18,10))
```

The OUTB data set would look like this:

```
TYPE1  HAWKS
TYPE1  PIGEONS
TYPE1  ROBINS
```

Example 9

This example shows how you can use USING(xxxx) to supply a SORT statement to alter the records that are selected.

```
  SELECT FROM(IN) TO(OUT) ON(1,5,CH) FIRST(3) USING(CTL1)
```

Let's say the IN data set looks like this:

```
FRANK 00015
FRANK 00012
FRANK 00003
FRANK 00018
FRANK 00005
FRANK 00035
VICKY 00022
VICKY 00007
VICKY 00014
VICKY 00028
VICKY 00002
VICKY 00015
```

We want to select the three records with each name that have the highest counts. If we just used ON(1,5,CH) without any CTL1CNTL statements, we'd get the first three records for each name without regard to the count. The OUT data set would look like this:

```
FRANK 00015
FRANK 00012
FRANK 00003
VICKY 00022
VICKY 00007
VICKY 00014
```

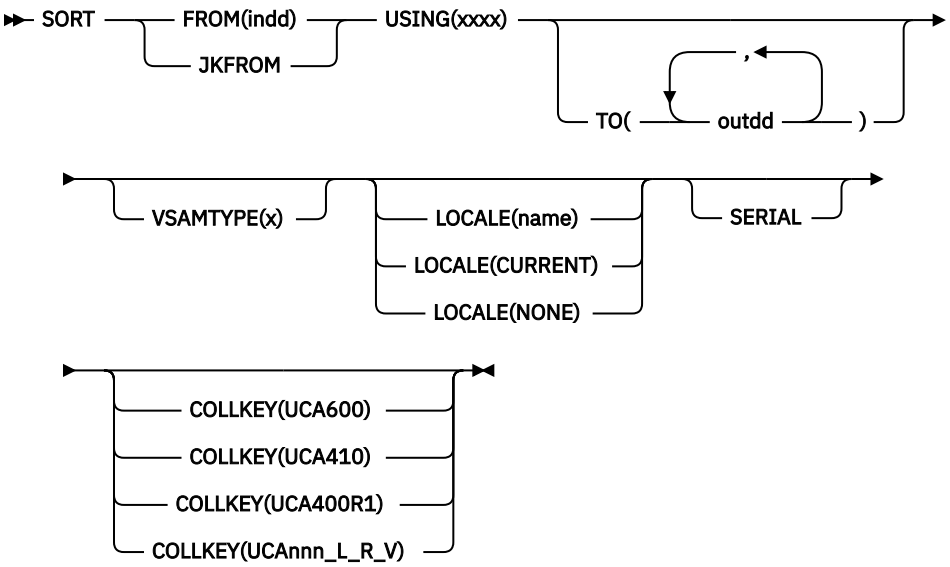
To get the three records with the highest counts for each name, we can use the following SORT statement in CTL1CNTL:

```
  SORT FIELDS=(1,5,CH,A,7,5,ZD,D)
```

The records will be sorted in ascending order on the name field, and in descending order on the count field. By sorting descending on the count, we ensure that the three records with the highest counts are the first three records for each name. Thus, when ON(1,5,CH) selects the first three records, they will be those with the highest counts. The OUT data set will look like this:

```
FRANK 00035
FRANK 00018
FRANK 00015
VICKY 00028
VICKY 00022
VICKY 00015
```

SORT operator



Sorts a data set to one or more output data sets.

DFSORT is called to sort the indd data set to the outdd data sets using the DFSORT control statements in xxxxCNTL. You must supply a DFSORT SORT statement in the xxxxCNTL data set to indicate the control fields for the sort. You can use additional DFSORT statements in the xxxxCNTL data set to sort a subset of the input records (INCLUDE or OMIT statement; SKIPREC and STOPAFT options; OUTFIL INCLUDE, OMIT, SAVE, STARTREC, ENDREC, SAMPLE, SPLIT, SPLITBY, and SPLIT1R operands; user exit routines), reformat records for output (INREC, OUTREC, and OUTFIL statements, user exit routines), and so on.

The active locale's collating rules affect SORT processing as explained in “SORT control statement” on page 421. If an INCLUDE or OMIT statement or an OUTFIL INCLUDE or OMIT operand is specified in the xxxxCNTL data set, the active locale's collating rules affect INCLUDE and OMIT processing as explained in the "Cultural Environment Considerations" discussion in “INCLUDE control statement” on page 91.

The DYNALLOC option is passed to DFSORT to ensure that work space is available for the sort. If your installation defaults for dynamic allocation are inappropriate for a SORT operator, you can take one of the following actions:

1. Override the DYNALLOC option using an OPTION control statement such as:

   ```
   OPTION DYNALLOC=(,8)
   ```

   in the xxxxCNTL data set in conjunction with the USING(xxxx) operand.
2. Use xxxxWKdd DD statements to override the use of dynamic allocation in conjunction with the USING(xxxx) operand. Refer to “SORTWKdd DD statement” on page 69 for details.

Tape work data sets **cannot** be used with ICETOOL.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

JKFROM
Specifies you are using a JOINKEYS application with this SORT operator to sort the joined records. You must provide a USING(xxxx) operand. In xxxxCNTL, you must provide a SORT statement, a JOINKEYS statement with F1=ddname1 for the F1 file, and a JOINKEYS statement with F2=ddname2 for the F2 file, as well as JOIN and REFORMAT statements as needed.

USING(xxxx)
Specifies the first 4 characters of the ddname for the control statement data set to be used by DFSORT for this operation. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx.

An xxxxCNTL DD statement must be present, and the control statements in it must conform to the rules for DFSORT's SORTCNTL data set.

The xxxxCNTL data set must contain a SORT statement. If TO is not specified, the xxxxCNTL data set must also contain either one or more OUTFIL statements or a MODS statement for an E35 routine that disposes of all records. Other statements are optional.

If you want to override dynamic allocation of work data sets for this operation, you can use xxxxWKdd DD statements for that purpose.

Refer to "JCL restrictions" on page 542 for more information regarding the selection of ddnames.

TO(outdd,...)
Specifies the ddnames of the output data sets to be written by DFSORT for this operation. From 1 to 10 outdd names can be specified. An outdd DD statement must be present for each outdd name specified. If a single outdd data set is specified, DFSORT is called once to sort the indd data set to the outdd data set using SORTOUT processing; the outdd data set must conform to the rules for DFSORT's SORTOUT data set. If multiple outdd data sets are specified and SERIAL is not specified, DFSORT is called once to sort the indd data set to the outdd data sets using OUTFIL processing; the outdd data sets must conform to the rules for DFSORT's OUTFIL data sets.

A ddname specified in the FROM operand must not also be specified in the TO operand.

Refer to "JCL restrictions" on page 542 for more information regarding the selection of ddnames.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

LOCALE(name)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

LOCALE(CURRENT)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

LOCALE(NONE)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

SERIAL
Specifies that OUTFIL processing is not to be used when multiple outdd data sets are specified. DFSORT is called multiple times and uses SORTOUT processing; the outdd data sets must conform to the rules for DFSORT's SORTOUT data set. SERIAL is not recommended because the use of serial processing (that is, multiple calls to DFSORT) instead of OUTFIL processing can degrade performance and imposes certain restrictions as detailed later in this section. SERIAL is ignored if a single outdd data set is specified.

DFSORT is called to sort the indd data set to the first outdd data set using the DFSORT control statements in the xxxxCNTL data set. If the sort operation is successful, DFSORT is called as many times as necessary to copy the first outdd data set to the second and subsequent outdd data sets. Therefore, for maximum efficiency, use a disk data set as the first in a list of outdd data sets on both disk and tape. If more than one outdd data set is specified, DFSORT must be able to read the first outdd data set after it is written in order to copy it to the other outdd data sets. Do not use a SYSOUT or DUMMY data set as the first in a list of outdd data sets because:

- If the first data set is SYSOUT, DFSORT abends when it tries to copy the SYSOUT data set to the second outdd data set.
- If the first data set is DUMMY, DFSORT copies the empty DUMMY data set to the other outdd data sets (that is, all of the resulting outdd data sets are empty).

COLLKEY(UCA600)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543

COLLKEY(UCA410)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

COLLKEY(UCA400R1)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

COLLKEY(UCAnnn_L_R_V)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543

Sort examples

This section includes sort examples.

Example 1

```
* Method 1
SORT FROM(MASTER) TO(PRINT,TAPE,DISK) USING(ABCD)
```

```
* Method 2
SORT FROM(MASTER) TO(DISK,TAPE,PRINT) USING(ABCD) SERIAL
```

This example shows two different methods for creating multiple sorted output data sets. Assume that the ABCDCNTL data set contains:

```
  SORT FIELDS=(15,20,CH,A,1,5,PD,D)
```

Method 1 requires one call to DFSORT, one pass over the input data set, and allows the output data sets to be specified in any order. The SORT operator sorts all records from the MASTER data set to the PRINT (SYSOUT), TAPE, and DISK data sets, using the SORT statement in the ABCDCNTL data set and OUTFIL processing.

Method 2 requires three calls to DFSORT, three passes over the input data set, and imposes the restriction that the SYSOUT data set must not be the first TO data set. The SORT operator sorts all records from the MASTER data set to the DISK data set, using the SORT statement in the ABCDCNTL data set, and then copies the resulting DISK data set to the TAPE and PRINT (SYSOUT) data sets. Because the first TO data set is processed three times (written, read, read), placing the DISK data set first is more efficient than placing the TAPE data set first. PRINT must not be the first in the TO list because a SYSOUT data set cannot be read.

Example 2

```
* Method 1
SORT FROM(IN) TO(DEPT1) USING(DPT1)
SORT FROM(IN) TO(DEPT2) USING(DPT2)
SORT FROM(IN) TO(DEPT3) USING(DPT3)
```

```
* Method 2
SORT FROM(IN) USING(ALL3)
```

This example shows two different methods for creating sorted subsets of an input data set. Assume that:

- The DPT1CNTL data set contains:

  ```
  SORT FIELDS=(51,2,BI,A,18,5,CH,A,58,4,BI,A)
  INCLUDE COND=(5,3,CH,EQ,C'D01')
  ```

- The DPT2CNTL data set contains:

  ```
  SORT FIELDS=(51,2,BI,A,18,5,CH,A,58,4,BI,A)
  INCLUDE COND=(5,3,CH,EQ,C'D02')
  ```

- The DPT3CNTL data set contains:

```
SORT FIELDS=(51,2,BI,A,18,5,CH,A,58,4,BI,A)
INCLUDE COND=(5,3,CH,EQ,C'D03')
```

- The ALL3CNTL data set contains:

```
SORT FIELDS=(51,2,BI,A,18,5,CH,A,58,4,BI,A)
OUTFIL FNAMES=DEPT1,INCLUDE=(5,3,CH,EQ,C'D01')
OUTFIL FNAMES=DEPT2,INCLUDE=(5,3,CH,EQ,C'D02')
OUTFIL FNAMES=DEPT3,INCLUDE=(5,3,CH,EQ,C'D03')
```

Method 1 requires three calls to DFSORT and three passes over the input data set:

- The first SORT operator sorts the records from the IN data set that contain D01 in positions 5-7 to the DEPT1 data set
- The second COPY operator sorts the records from the IN data set that contain D02 in positions 5-7 to the DEPT2 data set
- The third COPY operator sorts the records from the IN data set that contain D03 in positions 5-7 to the DEPT3 data set.

Method 2 accomplishes the same result as method 1 but, because it uses OUTFIL statements instead of TO operands, requires only one call to DFSORT and one pass over the input data set.

Example 3

```
SORT FROM(IN1) TO(FRANCE) USING(SRT1) LOCALE(FR_FR)
SORT FROM(IN1) TO(CANADA) USING(SRT1) LOCALE(FR_CA)
SORT FROM(IN1) TO(BELGIUM) USING(SRT1) LOCALE(FR_BE)
```

This example shows how sorted data for three different countries can be produced. Assume that the SRT1CNTL data set contains:

```
  SORT FIELDS=(5,20,CH,A,31,15,CH,A,1,4,FI,D,63,10,CH,D)
```

The first SORT operator sorts all records from the IN1 data set to the FRANCE data set, using the SORT statement in the SRT1CNTL data set. The character (CH) control fields are sorted according to the collating rules defined in locale FR_FR (French language for France).

The second SORT operator sorts all records from the IN1 data set to the CANADA data set, using the SORT statement in the SRT1CNTL data set. The character (CH) control fields are sorted according to the collating rules defined in locale FR_CA (French language for Canada).

The third SORT operator sorts all records from the IN1 data set to the BELGIUM data set, using the SORT statement in the SRT1CNTL data set. The character (CH) control fields are sorted according to the collating rules defined in locale FR_BE (French language for Belgium).

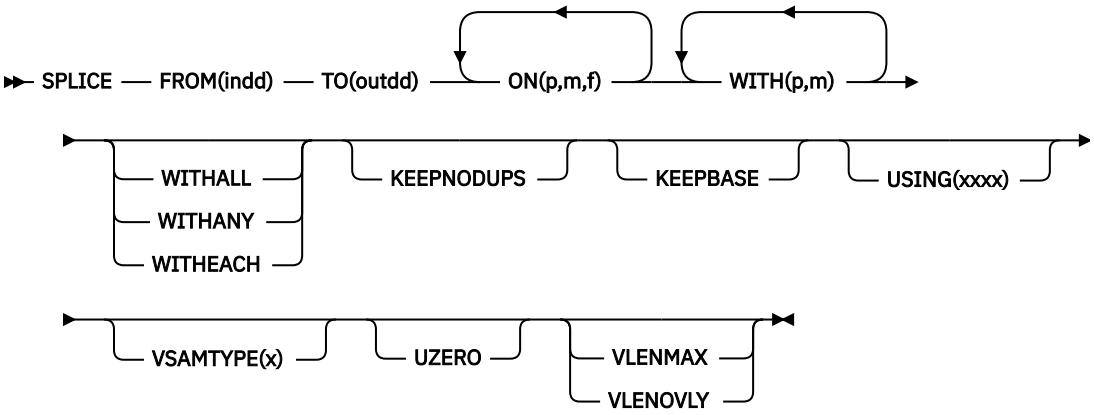
SORT operator with JOINKEYS example

Here is an example of using one SORT operator for a simple JOINKEYS application.

```
//SRTJK EXEC PGM=ICETOOL
//TOOLMSG DD SYSOUT=*
//DFSMSG DD SYSOUT=*
//JNA DD DSN=MY.INPUTA,DISP=SHR
//JNB DD DSN=MY.INPUTB,DISP=SHR
//OUT DD SYSOUT=*
//TOOLIN DD *
* SORT operator with JOINKEYS application.
SORT JKFROM TO(OUT) USING(CTL1)
/*
//CTL1CNTL DD *
* JOINKEYS application control statements for SORT operator.
  JOINKEYS F1=JNA,FIELDS=(5,4,A)
  JOINKEYS F2=JNB,FIELDS=(11,4,A),SORTED
  REFORMAT FIELDS=(F1:1,20,F2:5,15)
* Main task control statement for SORT operator
* (operates on joined records).
  OPTION EQUALS
```

```
  SORT FIELDS=(1,3,ZD,A)
/*
```

SPLICE operator



Splices together specified fields from records with matching numeric or character field values (that is, duplicate values), but different information. This makes it possible to join fields from different types of input records to create an output record with information from two or more records.

Typically, you will want to reformat the records from two or more data sets to a temporary MOD data set, and use that temporary MOD data set as input to the SPLICE operator.

"SPLICE examples" on page 647 shows some techniques for splicing records from different data sets together in a variety of ways to perform various file "join" and "match" operations.

By default (when WITHALL, WITHANY and WITHEACH are not specified), one spliced record is created for each set of duplicates by splicing the first duplicate with specified fields from the last duplicate.

The first duplicate is treated as a "base record". The last duplicate is treated as an "overlay record". Specified fields from the overlay record are overlaid on to the base record. Thus, the output record consists of fields from the base (first) record intermixed with specified fields from the overlay (last) record.

The records to be spliced can originate from two or more different input data sets.

From 1 to 10 ON fields can be used for the fields to match on. At least one ON(p,m,f) field must be specified; all such ON fields specified will be used to determine the matching records to be spliced together.

From 1 to 50 WITH fields can be used to specify the fields to be overlaid on the base record from the overlay record. At least one WITH(p,m) field must be specified; all such WITH fields specified will be overlaid on to the base record. All other fields in the base record will be kept unchanged.

To illustrate the splicing process, if we had the following two fixed-length input records with the base fields, ON field and WITH fields as shown:

```
BASE1  ON1    BASE2           BASE3 BASE4  GGGGG
       ON1            WITHA                WITHB
```

the resulting spliced output record would be:

```
BASE1  ON1    BASE2   WITHA   BASE3 BASE4  WITHB
```

For variable-length records, by default (without VLENMAX or VLENOVLY), the spliced record has the same length as the base record, and WITH fields from the overlay record that are beyond the end of the base record do not appear in the spliced record. For example, if we had the following two records with the lengths (in the RDW), ON field and WITH fields as shown:

```
25 | ON1 BASE1       BASE2
35 | ON1       WITH1           WITH2
```

the resulting spliced output record would be:

```
25 | ON1 BASE1 WITH1 BASE2
```

The WITH2 field is beyond the end of the base record, so it is not spliced.

However, if you specify VLENMAX, the spliced record is given the larger of the base record length or overlay record length. If we specify VLENOVLY, the spliced record is given the overlay record length. In either case, if the overlay record length is larger, bytes in the extended spliced record that are not overlaid are filled in with blanks.

The resulting spliced output record with either VLENMAX or VLENOVLY would be:

```
35  ON1 BASE1 WITH1 BASE2     WITH2
```

You can use VLENMAX when you want the spliced record to have the maximum length of the base or overlay record. You can use VLENOVLY when you want the spliced record to have the length of the overlay record, regardless of whether it's longer or shorter than the base record. Without VLENMAX or VLENOVLY, the spliced record has the length of the base record regardless of whether it's longer or shorter than the overlay record.

For fixed-length records, the length of the base, overlay and spliced records are all the same. Thus, VLENOVLY and VLENMAX have no meaning for fixed-length records and are ignored.

WITHALL can be used to create multiple spliced records for each set of duplicates. The first duplicate is spliced with the specified fields from the second duplicate. Then the first duplicate is spliced with the specified fields from the third duplicate, and so on.

The first duplicate is treated as a base record. Each subsequent duplicate is treated as an overlay record. The specified fields from each overlay record are overlaid on to the base record. Thus, the output records consist of fields from the base record intermixed with specified nonblank and blank fields from the overlay records.

The records to be spliced can originate from multiple input data sets.

To illustrate the splicing process when WITHALL is specified, if we had the following four fixed-length records with the base fields, ON field and WITH fields as shown:

```
BASE1  ON1    BASE2           BASE3 BASE4  GGGGG
       ON1            WITHA                WITHB
       ON1            WITHC
       ON1            WITHE                WITHF
```

The resulting three spliced output records would be:

```
BASE1  ON1    BASE2   WITHA   BASE3 BASE4  WITHB
BASE1  ON1    BASE2   WITHC   BASE3 BASE4
BASE1  ON1    BASE2   WITHE   BASE3 BASE4  WITHF
```

Note that without WITHALL, the resulting single spliced output record would be:

```
BASE1  ON1    BASE2   WITHE   BASE3 BASE4  WITHF
```

For variable-length records, by default (without VLENMAX or VLENOVLY), the spliced record has the same length as the base record, and WITH fields from the overlay record that are beyond the end of the base record do not appear in the spliced record. For example, with WITHALL, if we had the following four records with the lengths (in the RDW), ON field and WITH fields as shown:

```
30 | BASE1 ON1           BASE2
25 |       ON1     WITHA
50 |       ON1     WITHB         WITHC       WITHD
40 |       ON1     WITHE         WITHF
```

the resulting three spliced output records would be:

```
30 | BASE1 ON1     WITHA BASE2
30 | BASE1 ON1     WITHB BASE2
30 | BASE1 ON1     WITHE BASE2
```

The WITHC, WITHD and WITHF fields are beyond the end of the base record, so they are not spliced.

However, if you specify VLENMAX, the spliced record is given the larger of the base record length or overlay record length. If we specify VLENOVLY, the spliced record is given the overlay record length. In either case, if the overlay record length is larger, bytes in the extended spliced record that are not overlaid are filled in with blanks. The resulting three spliced output records with WITHALL and VLENMAX would be:

```
30 | BASE1 ON1     WITHA BASE2
50 | BASE1 ON1     WITHB BASE2   WITHC       WITHD
40 | BASE1 ON1     WITHE BASE2   WITHF
```

The resulting three spliced output records with WITHALL and VLENOVLY would be:

```
25 | BASE1 ON1     WITHA
50 | BASE1 ON1     WITHB BASE2   WITHC       WITHD
40 | BASE1 ON1     WITHE BASE2   WITHF
```

WITHANY can be used to create one spliced record for each set of duplicates. The first duplicate is spliced with the nonblank values of each subsequent duplicate for specified fields.

The first duplicate is treated as a base record. Each subsequent duplicate is treated as an overlay record. Each specified field with a nonblank value in each overlay record is overlaid on to the base record. Thus, the output record consists of fields from the base record intermixed with specified nonblank fields from each overlay record. The value from the last overlay record with each nonblank value will appear in the output record. Note that a specified "field" from an overlay record can actually consist of multiple fields from the record that have previously been reformatted into one contiguous field.

The records to be spliced can originate from multiple input data sets.

To illustrate the splicing process when WITHANY is specified, if we had the following four fixed-length records with the base fields, ON field and WITH fields as shown:

```
BASE1  ON1     BASE2
       ON1                    WITHC
       ON1                           WITHA
       ON1            WITHB
```

The resulting spliced output record would be:

```
BASE1  ON1     BASE2  WITHB   WITHC  WITHA
```

For variable-length records, by default (without VLENMAX), the spliced record has the same length as the base record. For example, with WITHANY, if we had the following four records with the lengths (in the RDW), ON field and WITH fields as shown:

```
30 | BASE1 ON1           BASE2
50 |       ON1                                WITHB
25 |       ON1     WITHA
40 |       ON1                   WITHC
```

the resulting spliced output records would be:

```
30 | BASE1 ON1     WITHA BASE2
```

The WITHB and WITHC fields are beyond the end of the base record, so they are not spliced. However, if you specify VLENMAX, the spliced record is given the largest of the base record length or overlay record lengths. If the largest overlay record length is larger than the base record length, bytes in the extended spliced record that are not overlaid are filled in with blanks. The resulting spliced output record with WITHANY and VLENMAX would be:

```
50 | BASE1 ON1     WITHA BASE2   WITHC       WITHB
```

VLENOVLY cannot be specified with WITHANY.

WITHEACH can be used to create one spliced record for each set of duplicates. The first duplicate is spliced with one specified field from each subsequent duplicate.

The first duplicate is treated as a base record. Each subsequent duplicate is treated as an overlay record. The specified blank or nonblank field from each overlay record is overlaid on to the base record. Thus, the output record consists of fields from the base record intermixed with a specified nonblank or blank field from each overlay record. Note that the specified "field" from an overlay record can actually consist of multiple fields from the record that have previously been reformatted into one contiguous field.

The records to be spliced can originate from multiple input data sets

To illustrate the splicing process when WITHEACH is specified, if we had the following four records with the base fields, ON field and WITH fields as shown:

```
BASE1  ON1     BASE2
       ON1             WITHA
       ON1                     WITHB
       ON1                             WITHC
```

The resulting spliced output record would be:

```
BASE1  ON1     BASE2  WITHA   WITHB  WITHC
```

For variable-length records, by default (without VLENMAX), the spliced record has the same length as the base record. For example, with WITHEACH, if we had the following four records with the lengths (in the RDW), ON field and WITH fields as shown:

```
30 | BASE1 ON1           BASE2
25 |       ON1     WITHA
50 |       ON1                               WITHB
40 |       ON1                   WITHC
```

the resulting spliced output records would be:

```
30 | BASE1 ON1     WITHA BASE2
```

The WITHB and WITHC fields are beyond the end of the base record, so they are not spliced.

However, if you specify VLENMAX, the spliced record is given the largest of the base record length or overlay record lengths. If the largest overlay record length is larger than the base record length, bytes in the extended spliced record that are not overlaid are filled in with blanks. The resulting spliced output record with WITHEACH and VLENMAX would be:

```
50 | BASE1 ON1     WITHA BASE2   WITHC       WITHB
```

VLENOVLY cannot be specified with WITHEACH.

KEEPNODUPS can be used to keep the non-duplicate records as well as the spliced records. The non-duplicate records will be unchanged.

To illustrate the splicing process when KEEPNODUPS is specified, if we had the following six records with the base fields, ON fields and WITH fields as shown:

```
UNIQA  ONA
BASEA  ONB
DUPAA  ONB    WITHA
UNIQB  ONC
BASEB  OND
DUPBB  OND    WITHB
```

The two unique records (ONA and OND) would be kept along with the two spliced records (ONB and OND). The resulting four unspliced and spliced output records would be:

```
UNIQA  ONA
BASEA  ONB    WITHA
UNIQB  ONC
BASEB  OND    WITHB
```

Note that without KEEPNODUPS, the two unique records (ONA and ONC) would not be kept. The resulting two spliced output records would be:

```
BASEA  ONB    WITHA
BASEB  OND    WITHB
```

KEEPBASE can be used to keep the base records (first duplicate) as well as the spliced records. The base records will be unchanged.

To illustrate the splicing process when KEEPBASE is specified, if we had the following six records with the base fields, ON fields and WITH fields as shown:

```
UNIQA  ONA
BASEA  ONB
DUPAA  ONB    WITHA
UNIQB  ONC
BASEB  OND
DUPBB  OND    WITHB
```

The two base records with duplicates (first ONB record and first OND record) would be kept along with the two spliced records (ONB and OND). The resulting four unspliced and spliced output records would be:

```
BASEA  ONB
BASEA  ONB    WITHA
BASEB  OND
BASEB  OND    WITHB
```

Note that without KEEPBASE, the two base records with duplicates (first ONB record and first OND record) would not be kept. The resulting two spliced output records would be:

```
BASEA  ONB    WITHA
BASEB  OND    WITHB
```

If we used KEEPNODUPS and KEEPBASE with the original six records, the resulting six unspliced and spliced output records would be:

```
UNIQA  ONA
BASEA  ONB
BASEA  ONB    WITHA
UNIQB  ONC
BASEB  OND
BASEB  OND    WITHB
```

DFSORT is called to sort the indd data set. ICETOOL uses its E35 exit to determine which records to splice and include in the outdd data set. ICETOOL passes the EQUALS option to DFSORT to ensure that duplicates are kept in their original input order.

The DFSORT control statements in xxxxCNTL are used if USING(xxxx) is specified.

Do not supply your own MODS, SUM, OUTREC, or SORT statement.

You can use comment statements. You can use INCLUDE, OMIT, INREC, OPTION, and OUTFIL statements providing you observe these rules:

- You can use an INCLUDE or OMIT statement to remove input records **before** SPLICE processing.
- You can use an INREC statement to reformat input records **before** SPLICE processing; the base and overlay records are reformatted according to the INREC statement. You can use INREC's PARSE, BUILD (FIELDS), OVERLAY, FINDREP, IFTHEN, or IFOUTLEN functions. If your INREC statement changes the starting position of an ON field or WITH field, you must specify the new starting position for that ON field

or WITH field. For example, if your input records have a CH key at positions 1-5 and a WITH field at 6-8 and you use an INREC statement like this:

```
INREC FIELDS=(31:1,50)
```

you must specify ON(31,5,CH) instead of ON(1,5,CH) and WITH(36,3) instead of WITH(6,3).

- You can further process the outdd records associated with TO(outdd) **after** SPLICE processing using an OUTFIL statement like this:

```
OUTFIL FNAMES=outdd,...
```

or multiple OUTFIL statements like this:

```
OUTFIL FNAMES=outdd,...
OUTFIL FNAMES=outdd1,...
...
```

For example, with TO(OUT1) you could further modify the OUT1 records after they have been spliced, with a statement like this:

```
OUTFIL FNAMES=OUT1,FTOV,VLTRIM=X'40'
```

The DYNALLOC option is passed to DFSORT to ensure that work space is available for the sort. If your installation defaults for dynamic allocation are inappropriate for a SPLICE operator, you can specify USING(xxxx) and take one of the following actions:

1. Override the DYNALLOC option using an OPTION control statement such as:

   ```
   OPTION DYNALLOC=(,8)
   ```

   in the xxxxCNTL data set.
2. Use xxxxWKdd DD statements to override the use of dynamic allocation. Refer to "SORTWKdd DD statement" on page 69 for details.

Tape work data sets **cannot** be used with ICETOOL.

Operand descriptions

FROM(indd)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

TO(outdd)
Specifies the ddname of the output data set to which DFSORT will write the records it produces for the operation (that is, the spliced records, the non-duplicate records if KEEPNODUPS is specified, and the base records if KEEPBASE is specified).

An outdd DD statement must be present and must define an output data set that conforms to the rules for DFSORT's SORTOUT data set.

The ddname specified in the TO operand must not be the same as the ddname specified in the FROM operand.

Refer to "JCL restrictions" on page 542 for more information.

ON(p,m,f)
See the discussion of this operand on the SELECT statement in "SELECT operator" on page 626.

WITH(p,m)
Specifies the position and length of a field to be overlaid from the overlay record on to the base record.

p specifies the first byte of the field relative to the beginning of the overlay record. p is 1 for the first **data** byte of a fixed-length record and 5 for the first **data** byte of a variable-length record as illustrated in the following (RRRR represents the 4-byte record descriptor word):

```
   Fixed-length record    |    Variable-length record
   | D | A | T | A | ...  |    | R | R | R | R | D | A | T | A | ...
p=   1   2   3   4        | p=   1   2   3   4   5   6   7   8
```

If INREC is specified, p must refer to the record as reformatted by INREC.

m specifies the length of the field in bytes. A field must not extend beyond position 32752.

A WITH field will not be used to overlay the RDW of a variable-length base record or to overlay bytes from beyond the end of an overlay record on to a base record. When necessary, WITH fields will be adjusted to prevent these situations. For example, if WITH(1,6) is specified for a variable-length record, it will be treated as WITH(5,2) and if WITH(75,10) is specified for an 80-byte overlay record, it will be treated as WITH(75,6).

A WITH field will not be used to overlay bytes beyond the end of a base record. When necessary, WITH fields will be adjusted to prevent this situation. For example, if WITH(75,10) is specified for an 80-byte base record, it will be treated as WITH(75,6). However, if you specify VLENMAX or VLENOVLY, a WITH field can be used to overlay bytes beyond the end of a base record provided that WITH field is present in the overlay record. For example, if VLENMAX and WITH(75,10) is specified for an 80-byte base record and a 90-byte overlay record, the spliced record will have a length of 90 bytes and the WITH(75,10) field will be present at positions 75-84 followed by 6 blanks in positions 85-90.

WITHALL

Specifies that the first duplicate is spliced with specified fields from the second duplicate, and then from each subsequent duplicate in turn. All of the WITH fields (nonblank and blank) from each overlay record are overlaid on to the base record.

With WITHALL, a spliced output record is created from each base record and overlay record, resulting in n-1 spliced records for each set of n duplicates.

WITHALL overrides the default of splicing the first duplicate with all of the specified fields from the last duplicate.

WITHANY

Specifies that the first duplicate is spliced with specified nonblank fields from each subsequent duplicate. Each nonblank WITH field from each overlay record is overlaid on to the base record.

With WITHANY, a single spliced output record is created using the base record and each nonblank field from each overlay record. If more than one overlay record has a nonblank value for a WITH field, the nonblank value from the last overlay record for that WITH field will appear in the output record. Note that the specified "field" from an overlay record can actually consist of multiple fields from the record that have previously been reformatted into one contiguous field.

WITHANY overrides the default of splicing the first duplicate with all of the specified fields from the last duplicate.

VLENOVLY cannot be specified with WITHANY.

WITHEACH

Specifies that the first duplicate is spliced with one specified field from each subsequent duplicate. One WITH field (nonblank or blank) from each overlay record is overlaid on to the base record. The first WITH field specifies the bytes to be overlaid from the second duplicate record on to the first duplicate record. The second WITH field specifies the bytes to be overlaid from the third duplicate record on to the first duplicate record, and so on. For any set of duplicates, extra overlay records without matching WITH fields, or extra WITH fields without matching overlay records are ignored.

With WITHEACH, a single spliced output record is created using the base record and one field from each overlay record. Note that the specified "field" from an overlay record can actually consist of multiple fields from the record that have previously been reformatted into one contiguous field.

WITHEACH overrides the default of splicing the first duplicate with all of the specified fields from the last duplicate.

VLENOVLY cannot be specified with WITHEACH.

KEEPNODUPS
Specifies that non-duplicate records are to be kept as well as spliced records. The non-duplicate records will be unchanged.

KEEPBASE
Specifies that base records (first duplicate) are to be kept as well as spliced records. The base records will be unchanged.

VLENMAX
Specifies that for variable-length records, the length of the spliced record is set to the maximum length of the base record and overlay record. VLENMAX overrides the default of setting the length of the spliced record to the length of the base record.

If VLENMAX is specified with or without WITHALL, the spliced record is given the larger of the base record length or overlay record length. If the overlay record length is larger than the base record length, bytes in the extended spliced record that are not overlaid are filled in with blanks.

If VLENMAX is specified with WITHANY or WITHEACH, the spliced record is given the largest of the base record length or overlay record lengths. If the largest overlay record length is larger than the base record length, bytes in the extended spliced record that are not overlaid are filled in with blanks.

For fixed-length records, VLENMAX is ignored since the base, overlay and spliced records all have the same length.

VLENOVLY
Specifies that for variable-length records, the length of the spliced record is set to the length of the overlay record. VLENOVLY overrides the default of setting the length of the spliced record to the length of the base record.

If VLENOVLY is specified with or without WITHALL, the spliced record is given the overlay record length. If the overlay record length is larger than the base record length, the spliced record is padded with blanks from the end of the base record to the new length. If the overlay record length is smaller than the base record length, bytes in the extended spliced record that are not overlaid are filled in with blanks.

VLENOVLY cannot be specified with WITHANY or WITHEACH.

For fixed-length records, VLENOVLY is ignored since the base, overlay and spliced records all have the same length.

USING(xxxx)
Specifies the first 4 characters of the ddname for the control statement data set to be used by DFSORT for this operation. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx.

If USING is specified, an xxxxCNTL DD statement must be present and the control statements in it:

1. Must conform to the rules for DFSORT's SORTCNTL data set.
2. Should generally be used only for an INCLUDE or OMIT statement, an INREC statement, comment statements, or appropriate OUTFIL statements as described for “SPLICE operator” on page 639.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

UZERO
See the discussion of this operand on the OCCUR statement in “OCCUR operator” on page 607.

SPLICE examples

SPLICE normally requires reformatting the records of two or more data sets so they can be joined, so complete JCL examples are shown in this section to illustrate the suggested techniques. These techniques and others can be employed with SPLICE to perform a variety of tasks.

Because SPLICE overlays the WITH fields from the overlay record to the base record using matching ON fields, it's usually necessary to do some initial setup before using SPLICE, to ensure that:

- the ON fields are in the same positions in the base and overlay records
- the WITH fields in the overlay records are in the positions they will occupy in the base records
- the base records and overlay records are the same length. This is always required for fixed-length records, and is required for variable-length records unless VLENMAX or VLENOVLY is specified.

For optimum efficiency, it is also a good idea to remove any records that are not needed for the SPLICE operation as part of the initial setup before the SPLICE operation, by using appropriate INCLUDE or OMIT statements.

Example 1 - Create one spliced record for each match in two files

This example shows how you can splice data together for each pair of records with the same ON field in two different input data sets.

```
//S1    EXEC  PGM=ICETOOL
//TOOLMSG   DD  SYSOUT=*
//DFSMSG    DD  SYSOUT=*
//IN1 DD *
Y12 89503 MKT
Y12 57301 MKT
Z35 02316 DEV
Y12 91073 MKT
Z35 18693 DEV
/*
//IN2 DD *
89503 27M $9,185,354 SAN JOSE   CA
72135 08M   $317,632 BOSTON     MA
18693 10M $8,732,105 BUFFALO    NY
57301 50M    $30,000 NEWARK     NJ
/*
//TEMP1 DD DSN=&&TEMP1,DISP=(MOD,PASS),SPACE=(TRK,(5,5)),UNIT=SYSDA
//COMBINE DD SYSOUT=*
//TOOLIN   DD *
* Reformat the File1 records for splicing
  COPY FROM(IN1) TO(TEMP1) USING(CTL1)
* Reformat the File2 records for splicing
  COPY FROM(IN2) TO(TEMP1) USING(CTL2)
* Splice the needed data from File1 and File2 together
  SPLICE FROM(TEMP1) TO(COMBINE) ON(5,5,ZD) WITH(15,17)
/*
//CTL1CNTL DD *
  OUTREC FIELDS=(1,14,      file1 data
                 31:X)      add blanks for spliced file2 data
/*
//CTL2CNTL DD *
  OUTREC FIELDS=(5:1,5,     put file2 key in same place as file1 key
                 15:7,15,   file2 data
                 30:33,2)   file2 data
/*
```

The base records originate from the IN1 data set and are copied and reformatted to the TEMP1 data set. The reformatted TEMP1 records are 31 bytes long and look like this:

```
Y12 89503 MKT
Y12 57301 MKT
Z35 02316 DEV
Y12 91073 MKT
Z35 18693 DEV
```

The overlay records originate from the IN2 data set and are copied and reformatted to the end (MOD) of the TEMP1 data set. The reformatted TEMP1 records are 31 bytes long and look like this:

```
    89503     27M $9,185,354 CA
    72135     08M   $317,632 MA
    18693     10M $8,732,105 NY
    57301     50M    $30,000 NJ
```

Note that MOD is used for the TEMP1 data set, so the reformatted records from IN1 and IN2 will be output to the TEMP1 data set in that order, ensuring that they are spliced in that order.

The base and overlay records from the TEMP1 data set are sorted and spliced to the COMBINE data set.

The records look like this **after** they are sorted on the 5,5,ZD field, but **before** they are spliced. As a visual aid, the WITH fields in the overlay records are shown in bold.

```
Z35 02316 DEV
Z35 18693 DEV
    18693     10M $8,732,105 NY
Y12 57301 MKT
    57301     50M    $30,000 NJ
    72135     08M   $317,632 MA
Y12 89503 MKT
    89503     27M $9,185,354 CA
Y12 91073 MKT
```

The spliced COMBINE records are 31 bytes long and look like this:

```
Z35 18693 DEV 10M $8,732,105 NY
Y12 57301 MKT 50M    $30,000 NJ
Y12 89503 MKT 27M $9,185,354 CA
```

Note that the base records for 18693, 57301 and 89503 have been spliced together with their respective overlay records.

Here is what the various ICETOOL operators do in this job:

- The first COPY operator creates reformatted IN1 records in TEMP1. The second COPY operator creates reformatted IN2 records in TEMP1. The reformatted IN1 records have blanks where the reformatted IN2 WITH fields will go. The reformatted IN2 records have the ON field from IN2 in the same place as in the reformatted IN1 records, and have the IN2 data where we want it to go in the reformatted IN1 records. We made the reformatted IN1 and reformatted IN2 records the same size so we can put them all in the TEMP1 data set and use TEMP1 as input to the SPLICE operator.
- The SPLICE operator sorts the records from TEMP1 using the ON field. TEMP1 has the reformatted IN1 records before the reformatted IN2 records. The spliced records are created from the base records and the overlay records in TEMP1. Whenever two records are found with the same ON field, the WITH field from the second record (reformatted IN2 overlay record) is overlaid on to the first record (reformatted IN1 base record). The resulting spliced records are written to the COMBINE data set.

Example 2 - Combine complete records from four files

This example shows how you can use the WITHEACH operand to splice complete records from four different data sets together.

```
//S2    EXEC  PGM=ICETOOL
//TOOLMSG   DD  SYSOUT=*
//DFSMSG    DD  SYSOUT=*
//FILE1 DD DSN=...  input file1 - 300-byte records
//FILE2 DD DSN=...  input file2 - 400-byte records
//FILE3 DD DSN=...  input file3 - 150-byte records
//FILE4 DD DSN=...  input file4 -  20-byte records
//** BE SURE TO USE MOD FOR T1
//T1 DD DSN=&&TX,UNIT=SYSDA,SPACE=(CYL,(5,5)),
// DISP=(MOD,PASS)
//ALLRCDS DD DSN=...  output file - 870-byte records
//TOOLIN DD *
* Reformat the File1 records for splicing on added sequence number
  COPY FROM(FILE1) TO(T1) USING(CTL1)
* Reformat the File2 records for splicing on added sequence number
  COPY FROM(FILE2) TO(T1) USING(CTL2)
* Reformat the File3 records for splicing on added sequence number
  COPY FROM(FILE3) TO(T1) USING(CTL3)
```

```
* Reformat the File4 records for splicing on added sequence number
  COPY FROM(FILE4) TO(T1) USING(CTL4)
* Splice record-by-record on added sequence number
  SPLICE FROM(T1) TO(ALLRCDS) ON(871,8,PD) WITHEACH -
   WITH(301,400) WITH(701,150) WITH(851,20) -
   USING(CTL5)
/*
//CTL1CNTL DD *
* Reformat records to:
* 1          301        701        851        871
* File1bytes|400 blanks|150 blanks|20 blanks |seqno
  OUTREC FIELDS=(1,300,871:SEQNUM,8,PD)
/*
//CTL2CNTL DD *
* Reformat records to:
* 1          301        701        851        871
* 300 blanks|File2bytes|150 blanks|20 blanks |seqno
  OUTREC FIELDS=(301:1,400,871:SEQNUM,8,PD)
/*
//CTL3CNTL DD *
* Reformat records to:
* 1          301        701        851        871
* 300 blanks|400 blanks|File3bytes|20 blanks |seqno
  OUTREC FIELDS=(701:1,150,871:SEQNUM,8,PD)
/*
//CTL4CNTL DD *
* Reformat records to:
* 1          301        701        851        871
* 300 blanks|400 blanks|150 blanks|File4bytes|seqno
  OUTREC FIELDS=(851:1,20,871:SEQNUM,8,PD)
/*
//CTL5CNTL DD *
* Remove added sequence number from spliced records to get:
* File1bytes|File2bytes|File3bytes|File4bytes
 OUTFIL FNAMES=ALLRCDS,OUTREC=(1,870)
/*
```

Because the data sets do not have a common key, we add sequence numbers to the records from each data set and use the sequence numbers as the ON field for SPLICE. Using this technique, we can splice together the 300-byte records from FILE1, the 400-byte records from FILE2, the 150-byte records from FILE3 and the 20-byte records from FILE4, to produce 870-byte records in ALLRCDS. Conceptually, the 870-byte records in ALLRCDS would look like this:

```
File1 Record1 ... File2Record1 ... File3Record1 ... File4Record1 ...
File1 Record2 ... File2Record2 ... File3Record2 ... File4Record2 ...
...
```

The base records originate from the FILE1 data set and the overlay records originate from the FILE2, FILE3 and FILE4 data sets.

Here is what the various ICETOOL operators do in this job:

The first COPY operator creates reformatted records in the T1 data set with the FILE1 records in positions 1-300, blanks in all other positions up to 870, and a sequence number in positions 871-878. The sequence number will be 1 for the first FILE1 record, 2 for the second FILE1 record, and so on.

The second COPY operator creates reformatted records in the T1 data set with the FILE2 records in positions 301-700, blanks in all other positions up to 870, and a sequence number in positions 871-878. The sequence number will be 1 for the first FILE2 record, 2 for the second FILE2 record, and so on.

The third COPY operator creates reformatted records in the T1 data set with the FILE3 records in positions 701-850, blanks in all other positions up to 870, and a sequence number in positions 871-878. The sequence number will be 1 for the first FILE3 record, 2 for the second FILE3 record, and so on.

The fourth COPY operator creates reformatted records in the T1 data set with the FILE4 records in positions 851-870, blanks in all other positions up to 850, and a sequence number in positions 871-878. The sequence number will be 1 for the first FILE4 record, 2 for the second FILE4 record, and so on.

Note that MOD is used for the T1 data set, so the reformatted records from FILE1, FILE2, FILE3 and FILE4 will be output in that order in T1, ensuring that they are sorted and spliced in that order.

The SPLICE operator sorts the records from T1 using the sequence number as the ON field. With WITHEACH, the reformatted FILE1 records are treated as the base records, and the reformatted FILE2,

FILE3 and FILE4 records are treated as the overlay records; each WITH field is associated with an overlay record in turn. So the first WITH field specifies the bytes to be used from the second duplicate (FILE2 record), the second WITH field specifies the bytes to be used from the third duplicate (FILE3 record) and the third WITH field specifies the bytes to be used from the fourth duplicate (FILE4 record).

SPLICE matches each base and overlay record by their sequence numbers, and creates a new combined 878-byte record. The OUTFIL statement in CTL5CNTL is used to remove the sequence number so that the 870-byte spliced record is written to the ALLRCDS data set.

Example 3 - Create files with matching and non-matching records

This example shows how you can match records in input data sets 1 and 2 to produce three output data sets with:

- ON fields that appear in both input data set 1 and input data set 2
- ON fields that appear only in input data set 1
- ON fields that appear only in input data set 2

```
//S3 EXEC PGM=ICETOOL
//TOOLMSG DD SYSOUT=*
//DFSMSG  DD SYSOUT=*
//IN1     DD *
Vicky
Frank
Carrie
Holly
Paul
/*
//IN2      DD *
Karen
Holly
Carrie
Vicky
Mary
/*
//OUT12    DD SYSOUT=*
//OUT1     DD SYSOUT=*
//OUT2     DD SYSOUT=*
//T1       DD DSN=&&T1,DISP=(MOD,PASS),UNIT=SYSDA,SPACE=(TRK,(5,5))
//TOOLIN   DD *
* Add '11' identifier for FILE1 records.
  COPY FROM(IN1) TO(T1) USING(CTL1)
* Add '22' identifier for FILE2 records.
  COPY FROM(IN2) TO(T1) USING(CTL2)
* SPLICE to match up records and write them to their
* appropriate output files.
  SPLICE FROM(T1) TO(OUT12) ON(1,10,CH) WITH(13,1) -
    USING(CTL3) KEEPNODUPS
/*
//CTL1CNTL DD *
* Mark FILE1 records with '11'
  OUTREC FIELDS=(1,10,12:C'11')
/*
//CTL2CNTL DD *
* Mark FILE2 records with '22'
  OUTREC FIELDS=(1,10,12:C'22')
/*
//CTL3CNTL DD *
* Write matching records to OUT12 file.  Remove id.
  OUTFIL FNAMES=OUT12,INCLUDE=(12,2,CH,EQ,C'12'),OUTREC=(1,10)
* Write FILE1 only records to OUT1 file.  Remove id.
  OUTFIL FNAMES=OUT1,INCLUDE=(12,2,CH,EQ,C'11'),OUTREC=(1,10)
* Write FILE2 only records to OUT2 file.  Remove id.
  OUTFIL FNAMES=OUT2,INCLUDE=(12,2,CH,EQ,C'22'),OUTREC=(1,10)
/*
```

We copy the IN1 records to the T1 data set and add an identifier of '11' to show they come from FILE1.

We copy the IN2 records to the end (MOD) of the T1 data set and add an identifier of '22' to show they come from FILE2.

We sort the records of T1 on positions 1-3 and splice the second id byte for matching records. We use KEEPNODUPS to keep non-duplicate records.

The records look like this **after** they are sorted, but **before** they are spliced:

```
Carrie     11
Carrie     22
Frank      11
Holly      11
Holly      22
Karen      22
Mary       22
Paul       11
Vicky      11
Vicky      22
```

The records look like this **after** they are spliced, but **before** we do the OUTFIL processing specified by CTL3CNTL with USING(CTL3) for SPLICE:

```
Carrie     12
Frank      11
Holly      12
Karen      22
Mary       22
Paul       11
Vicky      12
```

An id of 12 indicates an ON field that appears in IN1 and IN2. An id of 11 indicates an ON field that appears only in IN1. An id of 22 indicates an ON field that appears only in IN2.

The OUTFIL statements in CTL3CNTL write the records to their appropriate output data sets (without the ids) as follows:

OUT12 contains:

```
Carrie
Holly
Vicky
```

OUT1 contains:

```
Frank
Paul
```

OUT2 contains:

```
Karen
Mary
```

Example 4 - Create a matrix of values

This example shows how you can use the WITHANY operand to create a matrix of values. We combine multiple rows for different types of data with a common key into a single row of data for that key, even if some rows are missing.

```
//S4 EXEC PGM=ICETOOL
//TOOLMSG DD SYSOUT=*
//DFSMSG  DD SYSOUT=*
//IN  DD *
001 3  150
001 2  120
001 1  100
002 2  140
002 3  250
003 1  050
003 3  920
004 3  005
/*
//OUT DD SYSOUT=*
//TOOLIN DD *
* Splice nonblank type 1 values (in 5-7), type 2 values (9-11)
* and type 3 values (13-15) for each key.
SPLICE FROM(IN) TO(OUT) ON(1,3,CH) WITHANY KEEPNODUPS -
  WITH(5,3) WITH(9,3) WITH(13,3) USING(CTL1)
/*
```

```
//CTL1CNTL DD *
* Before SPLICE:
* Reformat type 1 records to:
* 1   5
* key val
  INREC IFTHEN=(WHEN=(5,1,CH,EQ,C'1'),
          BUILD=(1,3,5:8,3)),
* Reformat type 2 records to:
* 1       9
* key     val
     IFTHEN=(WHEN=(5,1,CH,EQ,C'2'),
          BUILD=(1,3,9:8,3)),
* Reformat type 3 records to:
* 1          13
* key         val
     IFTHEN=(WHEN=(5,1,CH,EQ,C'3'),
          BUILD=(1,3,13:8,3))
/*
```

The input records look like this:

```
001 3  150
001 2  120
001 1  100
002 2  140
002 3  250
003 1  050
003 3  920
004 3  005
```

We have a key (for example, '001') in positions 1-3, a record type (1, 2 or 3) in position 5 and a numeric value in positions 8-10. We want to set up a single row for each key with the values for the three record types and blanks for missing record types.

Before we SPLICE the records, we use the INREC IFTHEN clauses to put the values from the type 1 records in positions 5-7, the values from the type 2 records in positions 9-11 and the values from the type 3 records in positions 13-15. The reformatted INREC records look like this:

```
001         150
001     120
001 100
002     140
002         250
003 050
003         920
004         005
```

We use SPLICE with WITHANY and appropriate WITH fields to create one combined record for each key with the values for the record types. We use KEEPNODUPS to keep records for keys with only one value (for example, '004').

The OUT records are 15 bytes long and look like this:

```
001 100 120 150
002     140 250
003 050     920
004         005
```

Note that if we used WITHEACH instead of WITHANY, the output would not be what we wanted since record types are missing for some keys (for example, type 1 is missing for key 002), and record types are out of order for some keys (for example, we have type 3, type 2 and type 1 in that order for key 001). With WITHEACH, the OUT records would look like this:

```
001         150
002     140
003
004         005
```

Example 5 - Create multiple spliced records for each match in two types of records

This example shows how you can use the WITHALL operand to tell ICETOOL to splice data together for a single record of one type (A records) and multiple records of another type (H records), in the same input data set, that all have the same ON field (duplicate records). It also shows how to ensure that duplicates of the second type without a match of the first type are not written to the output data set. IFTHEN clauses are used in an INREC statement to reformat the two types of records appropriately before they are sorted and spliced

```
//S5    EXEC  PGM=ICETOOL
//TOOLMSG   DD  SYSOUT=*
//DFSMSG    DD  SYSOUT=*
//MAST DD *
A0000B0000KRSC0000D000000E0000F00G000
A1111B1111FLYC1111D111111E1111F11G111
H02KRSI000002J002K002L02
H03FLYI000003J003K003L03
H04VQXI000004J004K004L04
H05FLYI000005J005K005L05
H06KHNI000006J006K006L06
H07KRSI000007J007K007L07
H08FLYI000008J008K008L08
H09KHNI000009J009K009L09
/*
//OUT DD SYSOUT=*
//TOOLIN   DD *
* Splice needed base and overlay data together.
* Do NOT splice identifier.
  SPLICE FROM(MAST) TO(OUT) WITH(1,7) WITH(13,4) ON(20,3,CH) -
   WITH(23,3) WITH(26,3) WITHALL USING(CTL1)
/*
//CTL1CNTL DD *
* Before SPLICE:
* Set up fields in base (A) records.  Add 'B' id in position 33.
  INREC IFTHEN=(WHEN=(1,1,CH,EQ,C'A'),
     BUILD=(8:14,5,17:31,3,20:11,3,29:34,4,33:C'B')),
* Set up fields in overlay (H) records.  Add 'V' id in position 33.
   IFTHEN=(WHEN=(1,1,CH,EQ,C'H'),
   BUILD=(1:7,7,13:18,4,20:4,3,23:1,3,26:22,3,33:C'V'))
* After SPLICE:
* Remove duplicate overlay records without matching base record.
* Remove base or overlay indicator.
  OUTFIL FNAMES=OUT,OMIT=(33,1,CH,EQ,C'V'),OUTREC=(1,32)
/*
```

The base records are the records in MAST with an 'A' in column 1. They are reformatted by the INREC statement as 33 byte records that look like this:

```
       C0000    F00KRS      G000B
       C1111    F11FLY      G111B
```

We put a 'B' in position 33 to identify these records as base records.

The overlay records are the records in MAST with an 'H' in column 1. They are reformatted by the INREC statement as 33 bytes records that look like this:

```
I000002     K002   KRSH02L02    V
I000003     K003   FLYH03L03    V
I000004     K004   VQXH04L04    V
I000005     K005   FLYH05L05    V
I000006     K006   KHNH06L06    V
I000007     K007   KRSH07L07    V
I000008     K008   FLYH08L08    V
I000009     K009   KHNH09L09    V
```

We put a 'V' in position 33 to identify these records as overlay records.

The base and overlay records set up by the INREC statement are sorted and spliced.

The records look like this **after** they are sorted on the 20,3,CH field, but **before** they are spliced. As a visual aid, the WITH fields in the overlay records are shown in bold.

```
       C1111    F11FLY      G111B
I000003     K003   FLYH03L03    V
I000005     K005   FLYH05L05    V
I000008     K008   FLYH08L08    V
I000006     K006   KHNH06L06    V
I000009     K009   KHNH09L09    V
       C0000    F00KRS      G000B
I000002     K002   KRSH02L02    V
I000007     K007   KRSH07L07    V
I000004     K004   VQXH04L04    V
```

The spliced output records are 33 bytes long and look like this:

```
I000003C1111K003F11FLYH03L03G111B
I000005C1111K005F11FLYH05L05G111B
I000008C1111K008F11FLYH08L08G111B
I000009     K009   KHNH09L09    V
I000002C0000K002F00KRSH02L02G000B
I000007C0000K007F00KRSH07L07G000B
```

Note that the base record (type A) for FLY has been spliced together with each of the three overlay records (type H) for FLY. Likewise, the base record (type A) for KRS has been spliced together with each of the two overlay records (type H) for KRS.

But also note that the overlay records (type H) for KHN have been spliced together. Because KHN does not appear as a base record (type A) we don't want the KHN records to appear in the OUT data set. So we will use the 'V' we put in position 33 for the overlay records to identify and delete spliced overlay records without a matching base record. We only have to do this if we have duplicate overlay records without a matching base record. Single overlay records without a matching base record will be deleted automatically (unless you specify KEEPNODUPS).

After we eliminate the spliced overlay records and the position 33 indicator, the OUT records are 32 bytes long and look like this:

```
I000003C1111K003F11FLYH03L03G111
I000005C1111K005F11FLYH05L05G111
I000008C1111K008F11FLYH08L08G111
I000002C0000K002F00KRSH02L02G000
I000007C0000K007F00KRSH07L07G000
```

Note that if we had not specified WITHALL, only the first and last records for each set of duplicates would have been spliced, producing the following output:

```
I000008C1111K008F11FLYH08L08G111
I000007C0000K007F00KRSH07L07G000
```

Example 6 - Pull records from a master file in sorted order

This example shows how you can use the WITHALL operand to tell ICETOOL to use ON fields in a "PULL" data set to select one or more records from a "MASTER" data set. In other words, you can use a PULL list to select records from a MASTER list. In this case, the PULL data set has VB records and the MASTER data set has FB records. The ON field is a City Name that can be 1-20 bytes long, and the selected MASTER records are to be sorted by the City Name. (“Example 7 - Pull records from a master file in their original order” on page 656 shows how to keep the MASTER records in their original order.)

```
//S6    EXEC  PGM=ICETOOL
//TOOLMSG   DD  SYSOUT=*
//DFSMSG    DD  SYSOUT=*
//PULL DD DSN=VAR.PULL.FILE,DISP=SHR
//MASTER DD DSN=FIXED.MASTER.FILE,DISP=SHR
//TEMP1 DD DSN=&&T1,DISP=(MOD,PASS),SPACE=(TRK,(5,5)),UNIT=SYSDA
//OUT DD DSN=FIXED.OUTPUT.FILE,DISP=(NEW,CATLG,DELETE),
//       SPACE=(TRK,(5,5)),UNIT=SYSDA
//TOOLIN DD *
* Convert PULL records from VB to FB and add 'P' identifier.
 COPY FROM(PULL) USING(CTL1)
```

```
* Add 'M' identifier to MASTER records.
 COPY FROM(MASTER) TO(TEMP1) USING(CTL2)
* Splice PULL and MASTER records (do NOT splice identifier):
*   Spliced MASTER records with matching PULL records have 'P' id.
*   Spliced MASTER records without matching PULL records
*   have 'M' id.
* Eliminate records with 'M' id.
 SPLICE FROM(TEMP1) TO(OUT) ON(1,20,CH) WITHALL WITH(1,40) -
   USING(CTL3)
/*
//CTL1CNTL DD *
* Convert PULL records from VB to FB and add 'P' identifier.
  OUTFIL FNAMES=TEMP1,VTOF,OUTREC=(5,20,41:C'P')
/*
//CTL2CNTL DD *
* Add 'M' identifier to MASTER records.
  OUTREC FIELDS=(1,40,41:C'M')
/*
//CTL3CNTL DD *
* Eliminate MASTER records without matching PULL records.
  OUTFIL FNAMES=OUT,OMIT=(41,1,CH,EQ,C'M'),OUTREC=(1,40)
/*
```

The base records originate from the PULL data set (VAR.PULL.FILE). The PULL data set has variable-length (VB) records with the RDW in positions 1-4 and the variable-length City Name starting in position 5 for 1-20 bytes. Conceptually, the PULL records look like this:

```
Length | Data
    12 | SAN JOSE
    12 | NEW YORK
    11 | DENVER
    15 | LOS ANGELES
```

The overlay records originate from the MASTER data set (FIXED.MASTER.FILE). The MASTER data set has 40-byte fixed-length (FB) records with the City Name in positions 1-20.

The PULL records are copied and reformatted to the TEMP1 data set as 41-byte fixed-length (FB) records with the City Name in positions 1-20 (padded on the right with blanks as necessary), and a 'P' in position 41 to identify them as PULL records. The VTOF and OUTREC parameters of DFSORT's OUTFIL statement are used to convert the VB records to FB records with blank padding. The reformatted PULL records in TEMP1 look like this:

```
SAN JOSE                              P
NEW YORK                              P
DENVER                                P
LOS ANGELES                           P
```

The MASTER records are copied and reformatted to the end (MOD) of the TEMP1 data set as 41-byte fixed-length (FB) records with an 'M' added in position 41 to identify them as MASTER records. The reformatted MASTER records in TEMP1 look like this:

```
SAN JOSE             8630  SUSAN        M
PHOENIX              7993  PAUL         M
LOS ANGELES          9203  MICHAEL      M
SAN JOSE             0052  VICKY        M
NEW YORK             5218  CARRIE       M
SAN JOSE             3896  FRANK        M
TUCSON               1056  LISA         M
NEW YORK             6385  MICHAEL      M
PHOENIX              5831  HOLLY        M
```

The base and overlay records from the TEMP1 data set are sorted and spliced.

The records look like this **after** they are sorted on the 1,20,CH field, but **before** they are spliced. As a visual aid, the WITH fields in the overlay records are shown in bold.

```
DENVER                                P
LOS ANGELES                           P
LOS ANGELES          9203  MICHAEL      M
NEW YORK                              P
NEW YORK             5218  CARRIE       M
NEW YORK             6385  MICHAEL      M
PHOENIX              7993  PAUL         M
```

```
PHOENIX             5831  HOLLY        M
SAN JOSE                                P
SAN JOSE            8630  SUSAN        M
SAN JOSE            0052  VICKY        M
SAN JOSE            3896  FRANK        M
TUCSON              1056  LISA         M
```

The spliced records look like this:

```
LOS ANGELES         9203  MICHAEL      P
NEW YORK            5218  CARRIE       P
NEW YORK            6385  MICHAEL      P
PHOENIX             5831  HOLLY        M
SAN JOSE            8630  SUSAN        P
SAN JOSE            0052  VICKY        P
SAN JOSE            3896  FRANK        P
```

Finally, we use the OUTFIL statement for SPLICE to remove each spliced record with an 'M' in position 41, because that represents a base record without a matching overlay record. The OUTFIL statement also removes the 'P' indicator in position 41 from each record, because it is not needed in the OUT data set.

Thus, for each MASTER record that matches a PULL record, we've overlaid the PULL record with the MASTER record. This effectively selects all of the MASTER records on the PULL list. The resulting OUT data set (FIXED.OUTPUT.FILE) has the following 40-byte fixed-length records:

```
LOS ANGELES         9203  MICHAEL
NEW YORK            5218  CARRIE
NEW YORK            6385  MICHAEL
SAN JOSE            8630  SUSAN
SAN JOSE            0052  VICKY
SAN JOSE            3896  FRANK
```

Example 7 - Pull records from a master file in their original order

This example is similar to “Example 6 - Pull records from a master file in sorted order” on page 654, except that we want to keep the resulting MASTER records in their original order instead of sorting them by the City Name field. We use DFSORT's SEQNUM parameter to add a sequence number to each MASTER record before the records are spliced, and we splice that sequence number along with the data. After SPLICE sorts by the City Name, we SORT again by the sequence number to get the resulting MASTER records back in their original order.

```
//S7    EXEC  PGM=ICETOOL
//TOOLMSG   DD  SYSOUT=*
//DFSMSG    DD  SYSOUT=*
//PULL DD DSN=VAR.PULL.FILE,DISP=SHR
//MASTER DD DSN=FIXED.MASTER.FILE,DISP=SHR
//TEMP1 DD DSN=&&TEMP1,DISP=(MOD,PASS),SPACE=(TRK,(5,5)),UNIT=SYSDA
//TEMP2  DD DSN=&&TEMP2,DISP=(,PASS),SPACE=(TRK,(5,5)),UNIT=SYSDA
//OUT DD DSN=FIXED.OUTPUT.FILE,DISP=(NEW,CATLG,DELETE),
//       SPACE=(TRK,(5,5)),UNIT=SYSDA
//TOOLIN DD *
* Convert PULL records from VB to FB and add 'P' identifier.
 COPY FROM(PULL) USING(CTL1)
* Add sequence number and 'M' identifier to MASTER records.
 COPY FROM(MASTER) TO(TEMP1) USING(CTL2)
* Splice PULL and MASTER records (splice sequence number, but
* do NOT splice identifier):
*   Spliced MASTER records with matching PULL records have 'P' id.
*   Spliced MASTER records without matching PULL records
*   have 'M' id.
* Eliminate records with 'M' id.
 SPLICE FROM(TEMP1) TO(TEMP2) ON(1,20,CH) WITHALL WITH(1,48) -
   USING(CTL3)
* Sort resulting spliced records on original sequence number
* to get them back in their original order.
* Remove id and sequence number.
 SORT FROM(TEMP2) TO(OUT) USING(CTL4)
/*
//CTL1CNTL DD *
* Convert PULL records from VB to FB and add 'P' identifier
  OUTFIL FNAMES=TEMP1,VTOF,OUTREC=(5,20,49:C'P')
/*
//CTL2CNTL DD *
```

```
* Add sequence number and 'M' identifier to MASTER records.
  OUTREC FIELDS=(1,40,41:SEQNUM,8,BI,49:C'M')
/*
//CTL3CNTL DD *
* Eliminate MASTER records without matching PULL records.
  OUTFIL FNAMES=TEMP2,OMIT=(49,1,CH,EQ,C'M')
/*
//CTL4CNTL DD *
* Sort on sequence number and remove id and sequence number.
  SORT FIELDS=(41,8,BI,A)
  OUTREC FIELDS=(1,40)
/*
```

The resulting OUT data set (FIXED.OUTPUT.FILE) has the following 40-byte fixed-length records:

```
SAN JOSE             8630  SUSAN
LOS ANGELES          9203  MICHAEL
SAN JOSE             0052  VICKY
NEW YORK             5218  CARRIE
SAN JOSE             3896  FRANK
NEW YORK             6385  MICHAEL
```

Example 8 - Create a report showing if needed parts are on-hand

This example shows how you can use the KEEPNODUPS operand to tell ICETOOL to compare the ON fields in a list of needed parts to the ON fields in a list of on-hand parts, and produce a report showing if each needed part is on-hand or not.

```
//S8       EXEC PGM=ICETOOL
//TOOLMSG  DD SYSOUT=*
//DFSMSG   DD SYSOUT=*
//ONHAND DD *
P62 Blue
P62 Red
G73 Blue
A27 Green
L90 Red
P63 Blue
/*
//NEEDED DD *
 2003/05/07   A27 Green
 2002/12/29   P62 Blue
 2003/03/17   A27 Blue
 2003/06/14   M92 Yellow
 2002/12/18   L90 Red
/*
//COMBINED DD DSN=&&T1,UNIT=SYSDA,SPACE=(TRK,(5,5)),
// DISP=(MOD,PASS)
//RPT  DD SYSOUT=*
//TOOLIN   DD *
* Reformat the ONHAND records for splicing.
* Add 'Yes' for found in ONHAND data set.
* Add 'O' to indicate ONHAND record.
  COPY FROM(ONHAND) TO(COMBINED) USING(CTL1)
* Reformat the NEEDED records for splicing.
* Add 'No' for missing from ONHAND data set.
* Add 'N' to indicate NEEDED record.
  COPY FROM(NEEDED) TO(COMBINED) USING(CTL2)
* Splice ONHAND and NEEDED records (splice identifier):
*   NEEDED records found in ONHAND list will have 'Yes'
*   and 'N'.
*   NEEDED records not found in ONHAND list will have 'No'
*   and 'N'.
*   ONHAND records that are not needed will have 'Yes'
*   and 'O'.
* Eliminate records with 'O'.
  SPLICE FROM(COMBINED) TO(RPT) -
    ON(1,12,CH) WITH(24,10) WITH(40,1) -
    KEEPNODUPS -
    USING(CTL3)
/*
//CTL1CNTL DD *
* Reformat ONHAND records with part in 1-12, 'Yes' in 15-17
* and 'O' in 40.
  OUTREC FIELDS=(1:1,12,15:C'Yes',40:C'O')
/*
//CTL2CNTL DD *
```

```
* Reformat NEEDED records with part in 1-12, 'No ' in 15-17,
* date in 24-33 and 'N' in 40.
  OUTREC FIELDS=(1:15,12,15:C'No',24:2,10,40:C'N')
/*
//CTL3CNTL DD *
* Eliminate ONHAND parts that do not appear in NEEDED list.
* Create the report showing if needed parts are on-hand.
  OUTFIL FNAMES=RPT,OMIT=(40,1,CH,EQ,C'O'),OUTREC=(1,33),
  HEADER2=(1:'Part',15:'On-Hand',24:'Needed by',/,
           1:'------------',15:'-------',24:'----------')
/*
```

The base records originate from the ONHAND data set and are copied and reformatted to the COMBINED data set. We put an 'O' in position 40 to identify these records as ONHAND records. The overlay records originate from the NEEDED data set and are copied and reformatted to the COMBINED data set. We put an 'N' in position 40 to identify these records as NEEDED records. Because MOD is used for the COMBINED data set, it contains the reformatted ONHAND records followed by the reformatted NEEDED records. The COMBINED records are 40 bytes long and look like this:

```
P62 Blue      Yes                    O
P62 Red       Yes                    O
G73 Blue      Yes                    O
A27 Green     Yes                    O
L90 Red       Yes                    O
P63 Blue      Yes                    O
A27 Green     No       2003/05/07    N
P62 Blue      No       2002/12/29    N
A27 Blue      No       2003/03/17    N
M92 Yellow    No       2003/06/14    N
L90 Red       No       2002/12/18    N
```

The base and overlay records from the COMBINED data set are sorted and spliced.

The records look like this after they are sorted on the 1,12,CH field, but before they are spliced. As a visual aid, the WITH fields in the overlay records are shown in bold.

```
A27 Blue      No       2003/03/17    N
A27 Green     Yes                    O
A27 Green     No       2003/05/07    N
G73 Blue      Yes                    O
L90 Red       Yes                    O
L90 Red       No       2002/12/18    N
M92 Yellow    No       2003/06/14    N
P62 Blue      Yes                    O
P62 Blue      No       2002/12/29    N
P62 Red       Yes                    O
P63 Blue      Yes                    O
```

The spliced output records are 40 bytes long and look like this:

```
A27 Blue      No       2003/03/17    N
A27 Green     Yes      2003/05/07    N
G73 Blue      Yes                    O
L90 Red       Yes      2002/12/18    N
M92 Yellow    No       2003/06/14    N
P62 Blue      Yes      2002/12/29    N
P62 Red       Yes                    O
P63 Blue      Yes                    O
```

We have three types of records as follows:

1. Records with 'Yes and 'N' are NEEDED records with an ONHAND match that have been spliced together. We want these for our report.
2. Records with 'No' and 'N' are NEEDED records without an ONHAND match that have been kept because we used KEEPNODUPS. We want these for our report.
3. Records with 'Yes' and 'O' are ONHAND records without a NEEDED match that have been kept because we used KEEPNODUPS. We do not want these for our report.

We use the OUTFIL statement for SPLICE to further process the spliced records. It omits the 'O' records, removes the 'N' byte, and sets up the headers for the report. The resulting RPT data set looks like this:

```
Part          On-Hand  Needed by
------------  -------  ----------
A27 Blue      No       2003/03/17
A27 Green     Yes      2003/05/07
L90 Red       Yes      2002/12/18
M92 Yellow    No       2003/06/14
P62 Blue      Yes      2002/12/29
```

Example 9 - Create a report showing if needed parts are on-hand - advanced

This example is a more complex variation of “Example 8 - Create a report showing if needed parts are on-hand” on page 657. It shows how you can use the WITHALL, KEEPBASE, and KEEPNODUPS operands to tell ICETOOL to compare the ON fields in a list of needed parts to the ON fields in a list of on-hand parts, and produce a report showing if each needed part is on-hand or not. However, it also has duplicate parts in the NEEDED data set, and produces a report with more information from the ONHAND and NEEDED records.

```
//S9       EXEC PGM=ICETOOL
//TOOLMSG  DD SYSOUT=*
//DFSMSG   DD SYSOUT=*
//ONHAND DD *
P62 Blue           Dallas
G73 Blue           San Jose
A27 Green          Vancouver
/*
//NEEDED DD *
 Rachel       A27 Green       Phoenix
 Monica       P62 Blue        Phoenix
 Phoebe       A27 Blue        Toronto
 Chandler     M92 Yellow      Los Angeles
 Joey         M92 Yellow      Paris
 Ross         A27 Green       Paris
/*
//COMBINED DD DSN=&&C1,UNIT=SYSDA,SPACE=(TRK,(5,5)),
// DISP=(MOD,PASS)
//TEMP1 DD DSN=&&T1,UNIT=SYSDA,SPACE=(CYL,(5,5)),DISP=(,PASS)
//RPT  DD SYSOUT=*
//TOOLIN   DD *
* Reformat the ONHAND records for splicing.
* Add 'Yes' for found and 'D' for delete record.
  COPY FROM(ONHAND) TO(COMBINED) USING(CTL1)
* Reformat the NEEDED records for splicing.
* Add 'No' for missing and 'K' for keep record.
  COPY FROM(NEEDED) TO(COMBINED) USING(CTL2)
* Splice ONHAND and NEEDED records.
* Splice in Requested by, Ship to and id fields.
* Eliminate spliced records with 'D'.
  SPLICE FROM(COMBINED) TO(TEMP1) -
    ON(1,12,CH) WITHALL KEEPBASE KEEPNODUPS USING(CTL3) -
    WITH(24,10) WITH(53,13) WITH(66,1)
* Print report.
  DISPLAY FROM(TEMP1) LIST(RPT) -
   INDENT(2) BETWEEN(2) BLANK -
   HEADER('Part') ON(1,12,CH) -
   HEADER('On-Hand') ON(15,3,CH) -
   HEADER('Requested by') ON(24,12,CH) -
   HEADER('Ship from') ON(38,13,CH) -
   HEADER('Ship to') ON(53,13,CH)
/*
//CTL1CNTL DD *
* Reformat ONHAND records with Part in 1-12, 'Yes' for found in
* 15-17, From City in 38-50 and 'D' in 66.
  OUTREC FIELDS=(1:1,12,15:C'Yes',38:20,13,66:C'D')
/*
//CTL2CNTL DD *
* Reformat NEEDED records with Part in 1-12, 'No  ' for missing in
* 15-17, Requester Name in 24-35, 'n/a' for From City in 38-40,
* To City in 53-65 and 'K' in 66.
  OUTREC FIELDS=(1:15,12,15:C'No ',24:2,10,38:C'n/a',
                 53:31,13,66:C'K')
/*
//CTL3CNTL DD *
* Eliminate ONHAND parts that do not appear in NEEDED list.
  OUTFIL FNAMES=TEMP1,OMIT=(66,1,CH,EQ,C'D')
/*
```

The base records originate from the ONHAND data set. They are copied and reformatted to the COMBINED data set. The reformatted records look like this:

```
P62 Blue      Yes                 Dallas                    D
G73 Blue      Yes                 San Jose                  D
A27 Green     Yes                 Vancouver                 D
```

The overlay records originate from the NEEDED data set and are copied and reformatted to the COMBINED data set. The reformatted records look like this

```
A27 Green     No       Rachel      n/a           Phoenix      K
P62 Blue      No       Monica      n/a           Phoenix      K
A27 Blue      No       Phoebe      n/a           Toronto      K
M92 Yellow    No       Chandler    n/a           Los Angeles  K
M92 Yellow    No       Joey        n/a           Paris        K
A27 Green     No       Ross        n/a           Paris        K
```

The base and overlay records from the COMBINED data set are sorted and spliced.

However, we need to make sure that all parts which appear in more than one NEEDED record, but do not appear in the ONHAND list, will appear in the report. For example, we have two M92 Yellow parts in the NEEDED data set that do not appear in the ONHAND data set. These two records are reformatted and appear in the COMBINED data set as follows:

```
M92 Yellow    No       Chandler    n/a           Los Angeles  K
M92 Yellow    No       Joey        n/a           Paris        K
```

ICETOOL would normally treat the first record as the base record and the second record as the overlay record. As a result, these two records would be spliced together into one record instead of two. To prevent this, and ensure that we keep both M92 Yellow parts, we must specify KEEPBASE. As a result, two records are kept: the unchanged first M92 Yellow record, and the spliced first and second M92 Yellow records (which in this case looks identical to the unspliced second record).

The records look like this **after** they are sorted on the 1,12,CH field, but **before** they are spliced. As a visual aid, the WITH fields in the overlay records are shown in bold.:

```
A27 Blue      No       Phoebe      n/a           Toronto      K
A27 Green     Yes                  Vancouver                  D
A27 Green     No       Rachel      n/a           Phoenix      K
A27 Green     No       Ross        n/a           Paris        K
G73 Blue      Yes                  San Jose                   D
M92 Yellow    No       Chandler    n/a           Los Angeles  K
M92 Yellow    No       Joey        n/a           Paris        K
P62 Blue      Yes                  Dallas                     D
P62 Blue      No       Monica      n/a           Phoenix      K
```

The spliced records look like this:

```
A27 Blue      No       Phoebe      n/a           Toronto      K
A27 Green     Yes                  Vancouver                  D
A27 Green     Yes      Rachel      Vancouver     Phoenix      K
A27 Green     Yes      Ross        Vancouver     Paris        K
G73 Blue      Yes                  San Jose                   D
M92 Yellow    No       Chandler    n/a           Los Angeles  K
M92 Yellow    No       Joey        n/a           Paris        K
P62 Blue      Yes                  Dallas                     D
P62 Blue      Yes      Monica      Dallas        Phoenix      K
```

Records with 'D' are not needed, so we use the OUTFIL statement for SPLICE to omit them. The TEMP1 records look like this:

```
A27 Blue      No       Phoebe      n/a           Toronto      K
A27 Green     Yes      Rachel      Vancouver     Phoenix      K
A27 Green     Yes      Ross        Vancouver     Paris        K
M92 Yellow    No       Chandler    n/a           Los Angeles  K
M92 Yellow    No       Joey        n/a           Paris        K
P62 Blue      Yes      Monica      Dallas        Phoenix      K
```

Although we could have used the OUTFIL statement for SPLICE to print the report, we've chosen instead to use a separate DISPLAY operator. DISPLAY requires an extra pass over the spliced records in TEMP1, but is easier to use than OUTFIL for reports. The resulting RPT data set looks like this:

```
  Part          On-Hand  Requested by  Ship from      Ship to
  ------------  -------  ------------  -------------  -------------
  A27 Blue      No       Phoebe        n/a            Toronto
  A27 Green     Yes      Rachel        Vancouver      Phoenix
  A27 Green     Yes      Ross          Vancouver      Paris
  M92 Yellow    No       Chandler      n/a            Los Angeles
  M92 Yellow    No       Joey          n/a            Paris
  P62 Blue      Yes      Monica        Dallas         Phoenix
```

Example 10 - Create spliced variable-length records from two files

This example shows how you can splice data together for each pair of records with the same ON field in two different VB input data sets, even when records are of different lengths.

```
//S10 EXEC PGM=ICETOOL
//TOOLMSG DD SYSOUT=*
//DFSMSG DD SYSOUT=*
//CON DD DSN=VAR.INPUT1,DISP=SHR
//    DD DSN=VAR.INPUT2,DISP=SHR
//OUT DD DSN=VAR.OUTPUT,DISP=(NEW,CATLG,DELETE),
//   SPACE=(CYL,(5,5)),UNIT=SYSDA
//TOOLIN DD *
* Splice the needed data from the two VB files together
SPLICE FROM(CON) TO(OUT) ON(5,5,CH) WITHALL -
   WITH(12,5) WITH(22,20) VLENMAX
/*
```

VAR.INPUT1 has RECFM=VB and LRECL=25. It contains the base records which look like this:

```
Length | Data
    25 | DIV01        L2
    15 | DIV02
    25 | DIV03        L6
    25 | DIV05        L8
```

VAR.INPUT2 has RECFM=VB and LRECL=50. It contains the overlay records which look like this:

```
Length | Data
    42 | DIV01  83201     FERN BROTHERS INTL
    33 | DIV01  73268     ROSS INC.
    39 | DIV02  00589     ACME PAINT SHOP
    19 | DIV05  57003
    47 | DIV05  01381     FLOWERS BY RENEE
    43 | DIV06  37982     EVERYTHING FOR PETS
```

Because some of the overlay records are longer than their corresponding base records, we use VLENMAX to ensure that none of the data from the overlay records is lost. VLENMAX ensures that the larger length between the base record and overlay record is used for the spliced record, and that blanks are added to the end of the spliced record when needed.

The base and overlay records from the concatenated data sets are sorted and spliced. VAR.OUTPUT has RECFM=VB and LRECL=50. It contains the spliced records, which look like this:

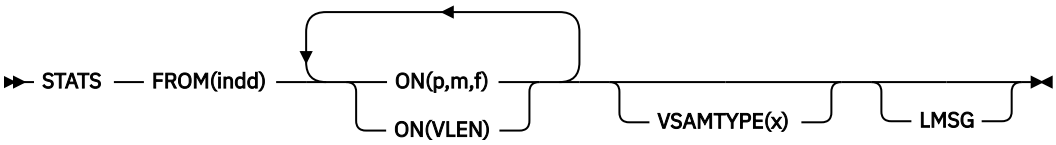
```
Length | Data
    42 | DIV01  83201 L2  FERN BROTHERS INTL
    33 | DIV01  73268 L2  ROSS INC.
    39 | DIV02  00589     ACME PAINT SHOP
    25 | DIV05  57003 L8
    47 | DIV05  01381 L8  FLOWERS BY RENEE
```

Notice that VLENMAX prevented any data from being lost. Without VLENMAX, data would have been lost; the spliced records would have looked like this:

```
Length | Data
    25 | DIV01  83201 L2  FERN
    25 | DIV01  73268 L2  ROSS
    15 | DIV02  0058
```

```
    25 | DIV05  57003 L8
    25 | DIV05  01381 L8  FLOW
```

STATS operator



Prints messages containing the minimum, maximum, average, and total for specified numeric fields. From 1 to 10 fields can be specified.

DFSORT is called to copy the indd data set to ICETOOL's E35 user exit. ICETOOL prints messages containing the minimum, maximum, average, and total for each field as determined by its E35 exit.

The average (or mean) is calculated by dividing the total by the record count and rounding down to the nearest integer (examples: 23 / 5 = 4, -23 / 5 = - 4).

You must not supply your own DFSORT MODS, INREC, or OUTREC statement, because they would override the DFSORT statements passed by ICETOOL for this operator.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

ON(p,m,f)
Specifies the position, length, and format of a numeric field to be used for this operation.

p specifies the first byte of the field relative to the beginning of the input record. p is 1 for the first **data** byte of a fixed-length record and 5 for the first **data** byte of a variable-length record as illustrated in the following (RRRR represents the 4-byte record descriptor word):

```
    Fixed-length record    |    Variable-length record
    | D | A | T | A | ...  |    | R | R | R | R | D | A | T | A | ...
 p=   1   2   3   4        | p=   1   2   3   4   5   6   7   8
```

m specifies the length of the field in bytes. A field must not extend beyond position 32752 or beyond the end of a record. The maximum length for a field depends on its format.

f specifies the format of the field as follows:

Format code	Length	Description
BI	1 to 8 bytes	Unsigned binary
FI	1 to 8 bytes	Signed fixed-point
PD	1 to 16 bytes	Signed packed decimal
ZD	1 to 31 bytes	Signed zoned decimal
CSF or FS	1 to 32 bytes (31 digit limit)	Signed numeric with optional leading floating sign
UFF	1 to 44 bytes (31 digit limit)	Unsigned free form numeric
SFF	1 to 44 bytes (31 digit limit)	Signed free form numeric

Note: See Appendix C, "Data format descriptions," on page 819 for detailed format descriptions.

If the total for a field overflows 31 digits, ICETOOL continues processing, but prints asterisks for the average and total for that field.

For a CSF, FS, UFF, or SFF format field:

- A maximum of 31 digits is allowed. If a value with more than 31 digits is found, ICETOOL issues an error message and terminates the operation.

For a ZD or PD format field:

- If a decimal value contains an invalid digit (A-F), ICETOOL identifies the bad value in a message and prints asterisks for the minimum, maximum, average and total for that field.
- A value is treated as positive if its sign is F, E, C, A, 8, 6, 4, 2, or 0.
- A value is treated as negative if its sign is D, B, 9, 7, 5, 3, or 1.

For a ZD, PD, CSF, FS, or SFF format field, a negative zero value is treated as a positive zero value.

ON(VLEN)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

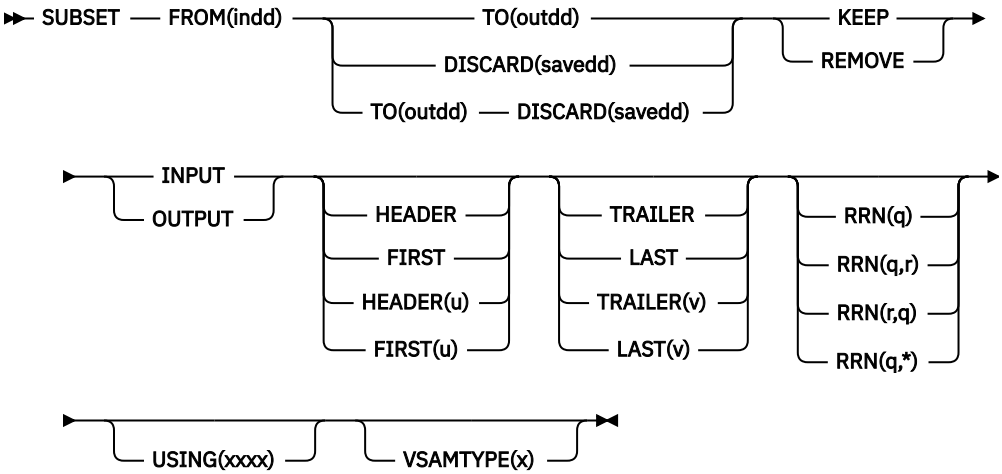
LMSG
Specifies that the minimum, maximum, average and total for all numeric fields are to be printed using messages that display 31 digits (overriding the default of printing messages that display 15 digits when possible). LMSG ensures that only message ICE648I is used to display the statistics. Without LMSG, a combination of messages ICE608I, ICE609I and ICE648I can be used to display the statistics.

STATS example

```
    STATS  FROM(DATA1)  ON(VLEN)  ON(15,4,ZD)
```

Prints messages containing the minimum, maximum, average and total of the binary values in positions 1-2 of the DATA1 data set. For variable-length records, this gives statistics about the length of the records. Prints messages containing the minimum, maximum, average and total of the zoned decimal values in positions 15-18 of the DATA1 data set.

SUBSET operator



Keeps or removes input or output records based on meeting criteria for the first n records, specific relative record numbers, and the last n records. DFSORT writes the records that are kept or not removed to the outdd data set.

DFSORT is called to copy or sort the indd data set, as appropriate. ICETOOL uses its E15 or E35 exit to determine which records to include in the outdd or savedd data set. ICETOOL passes the EQUALS option to DFSORT to ensure that duplicates are kept in their original input order if records are sorted.

If the criteria includes the last n records, ICETOOL may call DFSORT twice. For the first pass, ICETOOL counts the indd records without opening the output data sets. For the second pass, ICETOOL opens the output data sets and does SUBSET processing against the indd data set using the count obtained in the first pass.

DISCARD(savedd) can be used to write the records that are removed or not kept in the savedd data set. DISCARD(savedd) can be used with or without TO(outdd).

If you do not specify a header operand (HEADER, FIRST, HEADER(u), FIRST(u)), a relative record number operand (RRN(q), RRN(q,r), RRN(q,*)), or a trailer operand (TRAILER, LAST, TRAILER(v), LAST(v)), all of the records will be kept or removed. You can specify a header operand, relative record number operands, and a trailer operand in any combination. Records will be kept or removed according to the criteria you specify.

You can only specify one header operand. You can specify from 1 to 300 relative record number operands in any combination. You can only specify one trailer operand.

The DFSORT control statements in xxxxCNTL are used if USING(xxxx) is specified. However, you must observe these rules for control statements in the xxxxCNTL data set:

- MODS and OUTREC statements should not be present.
- SKIPREC and STOPAFT operands, and INCLUDE and OMIT statements, should not be present.
- A SORT statement can be present unless INPUT and DISCARD(savedd) are specified.
- INREC and SUM statements can be present.
- If INPUT is specified, the records selected will not be affected by INREC, SORT or SUM. If OUTPUT is specified, the records selected will be affected by INREC, SORT and SUM.
- Comment statements can be present.
- If you specify TO(outdd) without DISCARD(savedd), you can further process the outdd records **after** SUBSET processing using an OUTFIL statement like this:

  ```
  OUTFIL FNAMES=outdd,...
  ```

  or multiple OUTFIL statements like this:

  ```
  OUTFIL FNAMES=outdd,...
  OUTFIL FNAMES=outdd1,...
  ```

- If you specify DISCARD(savedd) without TO(outdd), you can further process the savedd records **after** SUBSET processing using one (and only one) OUTFIL statement like this:

  ```
  OUTFIL FNAMES=savedd,...
  ```

- If you specify TO(outdd) and DISCARD(savedd), you can further process the outdd and savedd records **after** SUBSET processing using two (and only two) OUTFIL statements like this:

  ```
  OUTFIL FNAMES=outdd,...
  ```

  or multiple OUTFIL statements like this:

  ```
  OUTFIL FNAMES=outdd,...
  OUTFIL FNAMES=savedd,...
  ```

  Both statements must be specified in the order shown with at least the FNAMES parameter. For example, to further modify only the DISCARD data set, you could use statements like this:

  ```
  OUTFIL FNAMES=OUT
  OUTFIL FNAMES=SAVE,OMIT=(21,3,ZD,GT,+25)
  ```

If you specify a SORT statement in xxxxCNTL, the DYNALLOC option is passed to DFSORT to ensure that work space is available for the sort. If your installation defaults for dynamic allocation are inappropriate for a SUBSET operator, you can take one of the following actions:

1. Override the DYNALLOC option using an OPTION control statement such as:

   ```
   OPTION DYNALLOC=(,8)
   ```

   in the xxxxCNTL data set.
2. Use xxxxWKdd DD statements to override the use of dynamic allocation. Refer to "SORTWKdd DD statement" on page 69 for details.

Tape work data sets **cannot** be used with ICETOOL.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

TO(outdd)
Specifies the ddname of the output data set to which DFSORT will write the records it selects for the operation (that is, the records that are kept or not removed according to the specified criteria).

An outdd DD statement must be present and must define an output data set that conforms to the rules for DFSORT's SORTOUT data set.

TO and DISCARD can both be specified. If DISCARD is not specified, TO must be specified. If TO is not specified, DISCARD must be specified.

The ddname specified in the TO operand must not be the same as the ddname specified in the FROM or DISCARD operand.

Refer to "JCL restrictions" on page 542 for more information.

DISCARD(savedd)
Specifies the ddname of the output data set to which DFSORT will write the records it does not select for the operation.

A savedd DD statement must be present and must define an output data set that conforms to the rules for DFSORT's OUTFIL data set.

TO and DISCARD can both be specified. If DISCARD is not specified, TO must be specified. If TO is not specified, DISCARD must be specified.

The ddname specified in the DISCARD operand must not be the same as the ddname specified in the FROM or TO operand.

Refer to "JCL restrictions" on page 542 for more information.

KEEP
Specifies that the records that meet the criteria are to be kept.

REMOVE
Specifies that the records that meet the criteria are to be removed.

INPUT
Specifies that the criteria are to be applied using the first n input records, relative input record numbers, and the last n input records. Use INPUT if you want to apply the criteria directly to the records from the indd data set.

The criteria will be applied to keep or remove records before they are reformatted by INREC, sorted by SORT and summed by SUM. The kept records will subsequently be reformatted, sorted and summed. OUTFIL will be applied to the resulting records.

As an example, if the input data set contains these records:

```
AAAA R01
AAAA R02
BBBB R03
CCCC R04
CCCC R05
CCCC R06
DDDD R07
DDDD R08
EEEE R09
EEEE R10
EEEE R11
```

and the following SUBSET operator and DFSORT control statements were specified:

```
  ...
//TOOLIN DD *
SUBSET FROM(IN) TO(OUT) KEEP INPUT RRN(3,10) USING(CTL1)
//CTL1CNTL DD *
  SORT FIELDS=(1,5,CH,D)
  SUM FIELDS=NONE
/*
```

First, input records 3-10 would be kept, so the intermediate result would be:

```
BBBB R03
CCCC R04
CCCC R05
CCCC R06
DDDD R07
DDDD R08
EEEE R09
EEEE R10
```

Then the SORT statement would be applied, so the intermediate result would be:

```
EEEE R09
EEEE R10
DDDD R07
DDDD R08
CCCC R04
CCCC R05
CCCC R06
BBBB R03
```

Finally, the SUM statement would be applied, so the final output in the OUT data set would be:

```
EEEE R09
DDDD R07
CCCC R04
BBBB R03
```

OUTPUT

Specifies that the criteria are to be applied using the first n output records, relative output record numbers, and the last n output records. Use OUTPUT if you want to apply the criteria to the indd records after they are processed by INREC, SORT and SUM as specified.

The criteria will be applied to keep or remove records after they are reformatted by INREC, sorted by SORT and summed by SUM. OUTFIL will be applied to the resulting records.

As an example, if the input data set contains these records:

```
AAAA R01
AAAA R02
BBBB R03
CCCC R04
CCCC R05
CCCC R06
DDDD R07
DDDD R08
EEEE R09
EEEE R10
EEEE R11
```

and the following SUBSET operator and DFSORT control statements were specified:

```
  ...
//TOOLIN DD *
SUBSET FROM(IN) TO(OUT) KEEP OUTPUT RRN(2,3) USING(CTL1)
//CTL1CNTL DD *
  SORT FIELDS=(1,5,CH,D)
  SUM FIELDS=NONE
/*
```

First, the SORT statement would be applied, so the intermediate result would be:

```
EEEE R09
EEEE R10
EEEE R11
DDDD R07
DDDD R08
CCCC R04
CCCC R05
CCCC R06
BBBB R03
AAAA R01
AAAA R02
```

Then the SUM statement would be applied, so the intermediate result would be:

```
EEEE R09
DDDD R07
CCCC R04
BBBB R03
AAAA R01
```

Finally, output records 2-3 would be kept, so the final output in the OUT data set would be:

```
DDDD R07
CCCC R04
```

HEADER or FIRST
Specifies one header record (the first record) is to be kept or removed.

HEADER and FIRST are equivalent to HEADER(1) and FIRST(1).

HEADER(u) or FIRST(u)
Specifies u header records (the first u records) are to be kept or removed. For example, HEADER(3) or FIRST(3) keeps or removes the first three records.

u must be specified as n or +n where n can be 1 to 999999999999999.

TRAILER or LAST
Specifies one trailer record (the last record) is to be kept or removed.

TRAILER and LAST are equivalent to TRAILER(1) and LAST(1).

TRAILER(v) or LAST(v)
Specifies v trailer records (the last v records) are to be kept or removed. For example, TRAILER(4) or LAST(4) keeps or removes the last 4 records.

v must be specified as n or +n where n can be 1 to 999999999999999.

RRN(q)
Specifies relative record number q is to be kept or removed. For example, RRN(8) keeps or removes the eighth record.

q must be specified as n or +n where n can be 1 to 999999999999999.

RRN(q,r) or RRN(r,q)
Specifies relative record numbers q through r are to be kept or removed. q can be less than, equal to, or greater than r. For example, RRN(5,10) and RRN(10,5) both keep or remove the fifth through tenth records.

q and r must be specified as n or +n where n can be 1 to 99999999999999.

RRN(q,*)
Specifies relative record numbers q through the last record are to be kept or removed. For example, RRN(7,*) keeps or removes the seventh through last records.

q must be specified as n or +n where n can be 1 to 99999999999999.

Note: RRN(*,q) is not valid

USING(xxxx)
Specifies the first 4 characters of the ddname for the control statement data set to be used by DFSORT for this operation. xxxx must be four characters that are valid in a ddname of the form xxxxCNTL. xxxx must not be SYSx.

If USING is specified, an xxxxCNTL DD statement must be present. Control statements in xxxxCNTL can be used as described for "SUBSET Operator" on page xxx, and must conform to the rules for DFSORT's SORTCNTL data set.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

SUBSET examples

Although the SUBSET operators in the examples in this section could all be contained in a single ICETOOL job step, they are shown and discussed separately for clarity.

Example 1

```
SUBSET FROM(IN1) TO(OUT1) REMOVE INPUT HEADER TRAILER
```

This example shows how you can remove the header record (first record) and trailer record (last record).

The variable-length input records look like this:

```
Len|Data
 33|01A   RODENTS           FFFFF
 23|01A   VOLE    BINKY
 26|02B   HAMSTER GARFIELD
 22|03A   RAT     JUNE
 24|04B   MOUSE   MICKEY
 21|01A   COUNT   004
```

We just want to keep the data records. We use REMOVE, INPUT, HEADER and TRAILER to indicate we want to remove the header and trailer input records.

The output records would look like this:

```
 23|01A   VOLE    BINKY
 26|02B   HAMSTER GARFIELD
 22|03A   RAT     JUNE
 24|04B   MOUSE   MICKEY
```

Note that we could use FIRST and LAST instead of HEADER and TRAILER.

Example 2

```
SUBSET FROM(IN2) TO(OUT2) DISCARD(OUT3) -
  KEEP INPUT RRN(3,4) LAST(3)
```

This example shows how you can create one output file with relative records and the last n records, and another output file with the remaining records.

The input records look like this:

```
Algebra
Astronomy
Biology
Calculus
```

```
French
Geography
Geometry
Greek
History
Latin
Psychology
Russian
```

In the first output file (OUT2), we want the third and fourth input records and the last three input records. In the second output file (OUT3), we want the records that are not in the first output file. We use TO(OUT2) and DISCARD(OUT3) for the two output files. We use KEEP, INPUT, RRN(3,4) and LAST(3) to indicate we want to keep relative input records 3 and 4 and the last 3 input records.

The OUT2 records would look like this:

```
Biology
Calculus
Latin
Psychology
Russian
```

The OUT3 records would look like this:

```
Algebra
Astronomy
French
Geography
Geometry
Greek
History
```

Note that we could use TRAILER(3) instead of LAST(3).

Example 3

```
SUBSET FROM(IN3) TO(OUT4) KEEP OUTPUT -
  LAST(5) USING(CTL1)
```

This example illustrates how you can keep the last 5 sorted records.

The CTL1CNTL data set contains:

```
  SORT FIELDS=(1,15,CH,A)
```

The input records look like this:

```
Psychology
Biology
Russian
French
History
Geography
Calculus
Geometry
Algebra
Greek
Astronomy
Latin
```

We want to sort the records by the CH field in positions 1-15 and keep the last 5 sorted records. We use KEEP, OUTPUT and LAST(5) to keep the last 5 output records. We use the SORT statement to sort the records before they are output. After the SORT statement is executed, the intermediate records look like this:

```
Algebra
Astronomy
Biology
Calculus
French
Geography
```

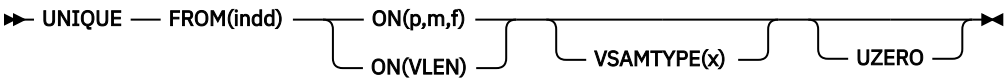
```
Geometry
Greek
History
Latin
Psychology
Russian
```

After the SUBSET operator is executed, the final output records look like this:

```
Greek
History
Latin
Psychology
Russian
```

Note that we could use TRAILER(5) instead of LAST(5).

UNIQUE operator



Prints a message containing the count of unique values for a specified numeric or character field.

DFSORT is called to sort the indd data set to ICETOOL's E35 user exit. ICETOOL prints a message containing the unique count as determined by its E35 user exit.

The DYNALLOC option is passed to DFSORT to ensure that work space is available for the sort. If your installation defaults for dynamic allocation are inappropriate for a UNIQUE operator, you can take one of the following actions:

1. Override the DYNALLOC option using an OPTION control statement such as:

   ```
   OPTION DYNALLOC=(8)
   ```

   in the DFSPARM data set.
2. Use SORTWKdd DD statements to override the use of dynamic allocation. Refer to "SORTWKdd DD statement" on page 69 for details.

Attention: Either of these actions affects the work data sets used for an OCCUR operator, or for a SELECT or SPLICE operator for which USING(xxxx) is not specified.

Tape work data sets **cannot** be used with ICETOOL.

You must not supply your own DFSORT MODS, INREC, OUTREC, SUM or RECORD statement, because they override the DFSORT statements passed by ICETOOL for this operator.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

ON(p,m,f)
Specifies the position, length, and format of a numeric or character field to be used with this operation.

p specifies the first byte of the field relative to the beginning of the input record. p is 1 for the first **data** byte of a fixed-length record and 5 for the first **data** byte of a variable-length record as illustrated in the following (RRRR represents the 4-byte record descriptor word):

```
Fixed-length record    |    Variable-length record
| D | A | T | A | ...  |    | R | R | R | R | D | A | T | A | ...
```

```
 p=   1   2   3   4        | p=   1   2   3   4   5   6   7   8
```

m specifies the length of the field in bytes. A field must not extend beyond position 32752 or beyond the end of a record. The maximum length for a field depends on its format.

f specifies the format of the field as shown in the following table:

Format code	Length	Description
BI	1 to 256 bytes	Unsigned binary
FI	1 to 256 bytes	Signed fixed-point
PD	1 to 32 bytes	Signed packed decimal
ZD	1 to 32 bytes	Signed zoned decimal
CH	1 to 4000 bytes	Character
CSF or FS	1 to 32 bytes	Signed numeric with optional leading floating sign
UFF	1 to 44 bytes	Unsigned free form numeric
SFF	1 to 44 bytes	Signed free form numeric

Note: See Appendix C, “Data format descriptions,” on page 819 for detailed format descriptions.

For a ZD or PD format field:

- F, E, C, A, 8, 6, 4, 2, and 0 are treated as equivalent positive signs. Thus the zoned decimal values F2F3C1, F2F3F1, and 020301 are counted as only one unique value.
- D, B, 9, 7, 5, 3, and 1 are treated as equivalent negative signs. Thus the zoned decimal values F2F3B0, F2F3D0, and 020310 are counted as only one unique value.
- Digits are not checked for validity.

ON(VLEN)
See the discussion of this operand on the DISPLAY statement in “DISPLAY operator” on page 562.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in “COPY operator” on page 543.

UZERO
See the discussion of this operand on the OCCUR statement on “OCCUR operator” on page 607.

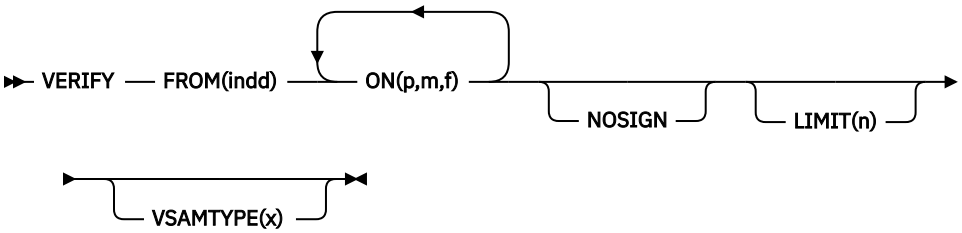
UNIQUE example

```
  UNIQUE FROM(DATAIN)  ON(20,40,CH)
  UNIQUE FROM(DATAIN)  ON(5,3,ZD)
```

The first UNIQUE operator prints a message containing the count of unique character data in positions 20-59 of the DATAIN data set.

The second UNIQUE operator prints a message containing the count of unique zoned decimal values in positions 5-7 of the DATAIN data set.

VERIFY operator



Examines particular decimal fields in a data set and prints a message identifying each invalid value found for each field. From 1 to 10 fields can be specified.

DFSORT is called to copy the indd data set to ICETOOL's E35 user exit. ICETOOL uses its E35 user exit to examine the digits and sign of each value for validity, and for each invalid value found, prints an error message containing the record number and field value (in hexadecimal).

You must not supply your own DFSORT MODS, INREC, or OUTREC statement, because they would override the DFSORT statements passed by ICETOOL for this operator.

Note:

1. Values with invalid decimal digits are also identified for the DISPLAY, OCCUR, RANGE, and STATS operators.
2. The DISPLAY operator can be used to print a report identifying the relative record number, hexadecimal value, and associated fields for each invalid (and valid) decimal value, as shown in Example 9 under "DISPLAY operator" on page 562.

Operand descriptions

The operands described in this section can be specified in any order.

FROM(indd)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

ON(p,m,f)
Specifies the position, length, and format of a decimal field to be used for this operation.

p specifies the first byte of the field relative to the beginning of the input record. p is 1 for the first **data** byte of a fixed-length record and 5 for the first **data** byte of a variable-length record as illustrated in the following (RRRR represents the 4-byte record descriptor word):

```
    Fixed-length record    |    Variable-length record
    | D | A | T | A | ...  |    | R | R | R | R | D | A | T | A | ...
 p=   1   2   3   4        | p=   1   2   3   4   5   6   7   8
```

m specifies the length of the field in bytes. A field must not extend beyond position 32752 or beyond the end of a record. The maximum length for a field depends on its format.

f specifies the format of the field as shown later in this section:

Format code	Length	Description
PD	1 to 16 bytes	Signed packed decimal
ZD	1 to 31 bytes	Signed zoned decimal

Note: See Appendix C, "Data format descriptions," on page 819 for detailed format descriptions.

A value is considered invalid under one of the following circumstances:

- it contains A-F as a digit (examples: a PD field of 00AF or a ZD field of F2FB)

- it contains 0-9 as a sign and the NOSIGN operand is not specified (examples: a PD field of 3218 or a ZD field of F235).

If the number of bad values reaches the LIMIT for invalid decimal values, ICETOOL terminates the operation. If the LIMIT operand is not specified, a default of 200 is used for the invalid decimal value limit.

NOSIGN
Specifies that the sign of the decimal values is not to be validity checked (overriding the default of checking for 0-9 as invalid signs).

LIMIT(n)
See the discussion of this operand on the DISPLAY statement in "DISPLAY operator" on page 562.

VSAMTYPE(x)
See the discussion of this operand on the COPY statement in "COPY operator" on page 543.

VERIFY example

```
    VERIFY FROM(NEW) ON(22,16,PD) ON(7,9,ZD)
    VERIFY FROM(NEW) ON(22,16,PD) ON(7,9,ZD) NOSIGN LIMIT(10)
```

The first VERIFY operator checks for invalid digits (A-F) and invalid signs (0-9) in the packed decimal values from positions 22-37 and the zoned decimal values from positions 7-15 of the NEW data set. A message is printed identifying each value (if any) that contains an invalid digit or sign. If 200 invalid values are found, the operation is terminated.

The second VERIFY operator checks for invalid digits (A-F) in the packed decimal values from positions 22-37 and the zoned decimal values from positions 7-15 of the NEW data set. A message is printed identifying each value (if any) that contains an invalid digit. If 10 invalid values are found, the operation is terminated.

Note: The DISPLAY operator can be used to print a report identifying the relative record number, hexadecimal value, and associated fields for each invalid (and valid) decimal value, as shown in Example 9 under "DISPLAY operator" on page 562.

Calling ICETOOL from a program

ICETOOL can be called from an assembler program using the LINK, ATTACH, or XCTL system macros. Standard linkage conventions must be used. When ICETOOL finishes processing, it returns to the calling program with register 15 set to the highest operation return code encountered. See "ICETOOL return codes" on page 679 for an explanation of the ICETOOL return codes.

When you call ICETOOL from a program, you have a choice of two different interfaces: the TOOLIN Interface and the Parameter List Interface.

TOOLIN interface

With the TOOLIN Interface, you supply ICETOOL statements in the TOOLIN data set. ICETOOL prints messages in the TOOLMSG data set, but does not return information directly to your program.

To use the TOOLIN interface, set a value of 0 in register 1, or place the address of a 4-byte field containing X'80000000' in register 1, before calling ICETOOL as follows:

```
         ...
         SLR   R1,R1            TOOLIN INTERFACE - METHOD 1
         LINK  EP=ICETOOL       CALL ICETOOL
         ...
         LA    R1,NOPLIST       TOOLIN INTERFACE - METHOD 2
         LINK  EP=ICETOOL       CALL ICETOOL
         ...
NOPLIST  DC    X'80',AL3(0)     TOOLIN INTERFACE INDICATOR
         ...
```

Parameter list interface

The Parameter List Interface allows you to use the information derived by ICETOOL in your program. With this interface, you supply ICETOOL statements in a parameter list. ICETOOL prints messages in the TOOLMSG data set, and puts an operation status indicator and "operation-specific values" in the parameter list for use by your calling program.

Figure 32 on page 674 shows the format of the parameter list used with the Parameter List Interface. Table 92 on page 675 shows the operation-specific values returned to the calling program.

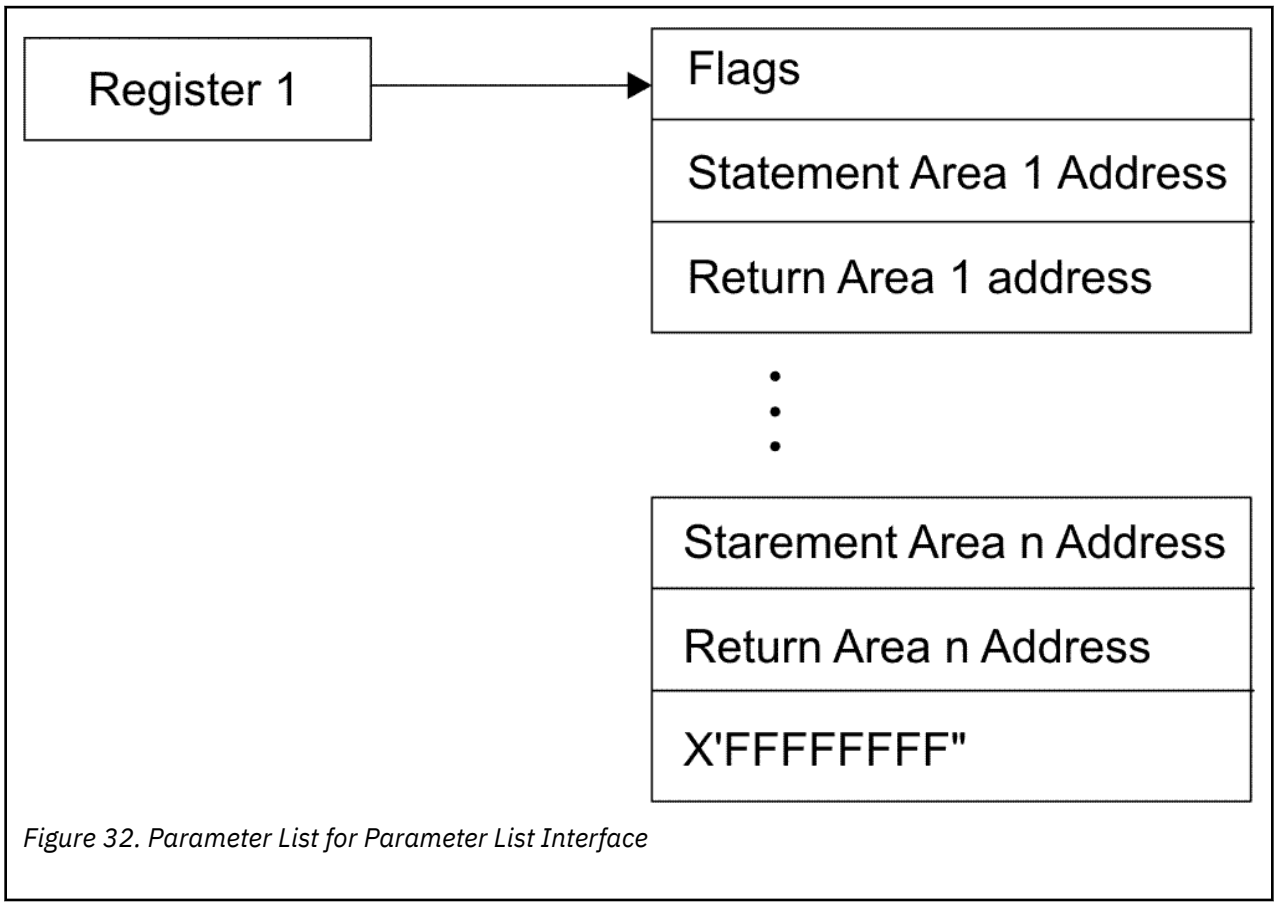


Figure 32. Parameter List for Parameter List Interface

The flags field must be specified. A 4-byte field containing X' FFFFFFFF' must be used to indicate the end of the parameter list. It can be coded after any pair of statement/return addresses.

All addresses in the parameter list must be 31-bit addresses or clean 24-bit addresses (the first 8 bits contain zeros).

Explanation of fields

Flags

Bit 0 = 0:
Use the Parameter List Interface. Process ICETOOL statements from this parameter list and return information to this parameter list. Ignore TOOLIN.

Bit 0 = 1:
Use the TOOLIN Interface. Process ICETOOL statements from TOOLIN. Ignore this parameter list.

Bits 1-31:
Reserved. Must be set to zero to ensure future extendability.

Statement Area Address and Statement Area
Each statement area address gives the location of a statement area that describes an ICETOOL operation to be performed. If the statement area address is 0, ICETOOL ignores this statement

area/return area pair. Otherwise, the statement area address must point to a statement area in the following format:

- A 2-byte length field containing the length of the statement area for this operation. If this field is 0, ICETOOL ignores this statement area/return area pair.
- One or more 80-character ICETOOL statement images in the format described in "ICETOOL statements" on page 542. Each statement area must have **one** operator statement. Comment and blank statements before the operator statement are processed. Comment, blank, and additional operator statements after the end of the first operator statement are ignored.

Return Area Address and Return Area

Each return area address gives the location of a return area in which ICETOOL is to return operation-specific information for the operation described in the corresponding statement area. If the return area address is 0, ICETOOL does not return any information for this operation. Otherwise, the return area address must point to a return area in the following general format:

- A 2-byte length field containing the length of the return area for this operation. If this field is 0, ICETOOL does not return any information for this operation.
- A 1-byte operation status indicator which is set by ICETOOL as follows:

  0 =
  This operation was run and completed with return code 0 or 4. Operation-specific values (see the following bullet) were returned.

  4 =
  This operation was not run (for example, scan mode was in effect) or did not complete with return code 0 or 4. Operation-specific values (see the following bullet) were not returned.
- Operation-specific values. Each count value returned by ICETOOL is an 8-byte packed decimal value with a C for a positive sign or a D for a negative sign. For a STATS operator, each minimum, maximum, average and total value returned by ICETOOL is a 16-byte packed decimal value with a C for a positive sign or a D for a negative sign. If ICETOOL set the operation status to 4, it did not return any values for this operation.

Note: Programs in LPALIB that call ICETOOL must provide return areas that ICETOOL can store into.

The required return area length and the operation-specific values returned for each operator are shown in Table 92 on page 675. If the return area length is less than the length required, ICETOOL issues a message and terminates the operation.

Table 92. Return area lengths/operation-specific values

Values	Return area length (bytes)	Operation-specific values returned
COPY	1	None
COUNT	9	Count of records processed, or 0 if any criteria specified. If SUB(q) or ADD(r) is specified, the modified count is returned.
DATASORT	1	None
DEFAULTS	1	None
DISPLAY	9	Count of records processed
MERGE	1	None
MODE	1	None
OCCUR	17	Count of records processed, count of records resulting from criteria
RANGE	17	Count of records processed, count of values in range

Table 92. Return area lengths/operation-specific values (continued)

Values	Return area length (bytes)	Operation-specific values returned
RESIZE	1	None
SELECT	17	Count of records processed, count of records resulting from criteria
SORT	1	None
SPLICE	17	Count of records processed, count of records resulting from criteria
STATS	64*n+9	Count of records processed, minimum for ON field 1, maximum for ON field 1, average for ON field 1, total for ON field 1, ... minimum for ON field n, maximum for ON field n, average for ON field n, total for ON field n
SUBSET	1	None
UNIQUE	17	Count of records processed, count of unique values
VERIFY	9	Count of records processed

Parameter list interface example

The following example shows a portion of an assembler language program that uses the Parameter List Interface. Figure 33 on page 677 shows the JCL you might use to run the program in this example.

```
DEPTVIEW CSECT
         ...
* SET UP PARAMETER LIST AND CALL ICETOOL
         LA    R1,PARLST        LOAD ADDRESS OF PARAMETER LIST
         LINK  EP=ICETOOL       CALL ICETOOL
         LTR   R15,R15          IF ANY OPERATIONS WERE NOT SUCCESSFUL,
         BNZ   CKSTAT1            DETERMINE WHICH ONE FAILED
* ALL OPERATIONS WERE SUCCESSFUL
*  CHECK EMPLOYEES PER DEPARTMENT AGAINST ACCEPTABLE LEVEL
         CP    RT2AVG1,EMAVGCK    IF AVERAGE IS ACCEPTABLE,
         BNH   CKQUAL               NO MESSAGE IS NEEDED
* ISSUE A MESSAGE SHOWING AVERAGE, MINIMUM, MAXIMUM, AND
* TOTAL NUMBER OF EMPLOYEES PER DEPARTMENT.
         ...
*  CHECK EXPENSES PER DEPARTMENT AGAINST ACCEPTABLE LEVEL
CKQUAL   CP    RT2AVG2,TLAVGCK    IF AVERAGE IS ACCEPTABLE,
         BNH   PCTCALC              NO MESSAGE IS NEEDED
* ISSUE A MESSAGE SHOWING AVERAGE, MINIMUM, MAXIMUM, AND
* TOTAL EXPENSES PER DEPARTMENT.
         ...
*  CALCULATE THE PERCENTAGE OF DEPARTMENTS OVER/UNDER EMPLOYEE LIMIT
PCTCALC  MVC   WORK+2(4),RT3RCDS+4   COPY NUMBER OF DEPARTMENTS
         SP    WORK+2(4),RT3RNG+4(4) SUBTRACT 'NUMBER WITHIN LIMITS' TO
*                                     GET 'NUMBER OVER/UNDER LIMIT'
         CP    WORK+2(4),P0          IF NONE OVER/UNDER LIMIT,
         BE    PCTPRT                 PERCENTAGE IS ZERO
         MP    WORK+2(4),P100        MULTIPLY NUMBER OVER/UNDER BY 100
         DP    WORK(6),RT3RCDS+4(4)  DIVIDE BY NUMBER OF DEPARTMENTS
* ISSUE A MESSAGE SHOWING THE PERCENTAGE OF DEPARTMENTS THAT ARE
* OVER/UNDER EMPLOYEE LIMIT
PCTPRT   UNPK  PCTVAL,WORK(2)     CONVERT AVERAGE TO PRINTABLE EBCDIC
         OI    PCTVAL+2,X'F0'     ENSURE LAST DIGIT IS PRINTABLE
         ...
* ONE OR MORE OPERATIONS FAILED
CKSTAT1  CLI   RT1STAT,0          IF OPERATION 1 WORKED,
         BNE   CKSTAT2              CHECK OPERATION 2
* ISSUE MESSAGE: OPERATION 1 FAILED - CHECK TOOLMSG
         ...
* PARAMETER LIST
PARLST   DC    A(0)               USE PARAMETER LIST INTERFACE
         DC    A(ST1A)            STATEMENT AREA 1 ADDRESS
```

```
         DC    A(RT1A)            RETURN AREA 1 ADDRESS
         DC    A(ST2A)            STATEMENT AREA 2 ADDRESS
         DC    A(RT2A)            RETURN AREA 2 ADDRESS
         DC    A(ST3A)            STATEMENT AREA 3 ADDRESS
         DC    A(RT3A)            RETURN AREA 3 ADDRESS
         DC    F'.*-1'              END OF PARAMETER LIST* OPERATOR STATEMENT AREAS
*   COPY OPERATION
ST1A     DC    AL2(ST1E-ST1)      LENGTH OF STATEMENT AREA 1
ST1      DC    CL80'* CREATE TWO COPIES OF THE DENVER SITE'
         DC    CL80'* DEPARTMENT RECORDS'
         DC    CL80'COPY FROM(IN) TO(OUT1,OUT2) USING(CTL1)'
ST1E     EQU   *
*   STATS OPERATION
ST2A     DC    AL2(ST2E-ST2)      LENGTH OF STATEMENT AREA 2
ST2      DC    CL80'* GET STATISTICS FOR NUMBER OF EMPLOYEES'
         DC    CL80'* AND TRAVEL EXPENSES PER DEPARTMENT'
         DC    CL80'STATS FROM(OUT1) ON(15,2,PD) ON(28,8,ZD)'
ST2E     EQU   *
*   RANGE OPERATION
ST3A     DC    AL2(ST3E-ST3)      LENGTH OF STATEMENT AREA 3
ST3      DC    CL80'* DETERMINE THE NUMBER OF DEPARTMENTS THAT ARE'
         DC    CL80'* WITHIN THE LIMIT FOR NUMBER OF EMPLOYEES'
         DC    CL80'RANGE FROM(OUT1) ON(15,2,PD) -'
         DC    CL80' HIGHER(10) LOWER(21)'
ST3E     EQU   *
* RETURN AREAS
   COPY OPERATION
RT1A     DC    AL2(RT1E-RT1STAT)  LENGTH OF RETURN AREA 1
RT1STAT  DS    C                  OPERATION STATUS
RT1E     EQU   *
*   STATS OPERATION
RT2A     DC    AL2(RT2E-RT2STAT)  LENGTH OF RETURN AREA 2
RT2STAT  DS    C                  OPERATION STATUS
RT2RCDS  DS    PL8                COUNT OF RECORDS PROCESSED
RT2MIN1  DS    PL16               FIELD 1 - MINIMUM VALUE
RT2MAX1  DS    PL16               FIELD 1 - MAXIMUM VALUE
RT2AVG1  DS    PL16               FIELD 1 - AVERAGE VALUE
RT2TOT1  DS    PL16               FIELD 1 - TOTAL VALUE
RT2MIN2  DS    PL16               FIELD 2 - MINIMUM VALUE
RT2MAX2  DS    PL16               FIELD 2 - MAXIMUM VALUE
RT2AVG2  DS    PL16               FIELD 2 - AVERAGE VALUE
RT2TOT2  DS    PL16               FIELD 2 - TOTAL VALUE
RT2E     EQU   *
*   RANGE OPERATION
RT3A     DC    AL2(RT3E-RT3STAT)  LENGTH OF RETURN AREA 3
RT3STAT  DS    C                  OPERATION STATUS
RT3RCDS  DS    PL8                COUNT OF RECORDS PROCESSED
RT3RNG   DS    PL8                COUNT OF VALUES IN RANGE
RT3E     EQU   *
* VARIABLES/CONSTANTS
WORK     DS    PL6                WORKING VARIABLE
P100     DC    P'100'             CONSTANT 100
P0       DC    P'0'               CONSTANT 0
EMAVGCK  DC    P'17'              ACCEPTABLE AVERAGE EMPLOYEE COUNT
TLAVGCK  DC    P'5000'            ACCEPTABLE AVERAGE TRAVEL EXPENSES
PCTVAL   DS    PL3                PERCENTAGE OF DEPARTMENTS THAT ARE
*                                   OVER/UNDER EMPLOYEE LIMIT
         ...
```

```
//EXAMP JOB A402,PROGRAMMER
//INVOKE EXEC PGM=DEPTVIEW,REGION=1024K
//STEPLIB DD DSN=...  Link library containing DEPTVIEW
//TOOLMSG DD SYSOUT=A
//DFSMSG DD SYSOUT=A
//IN DD DSN=ALL.DEPTS,DISP=SHR
//OUT1 DD DSN=ALL.DEPTS.BACKUP1,DISP=OLD
//OUT2 DD DSN=ALL.DEPTS.BACKUP2,DISP=OLD
//CTL1CNTL DD *
* SELECT ONLY THE DENVER SITE DEPARTMENT RECORDS
  INCLUDE COND=(1,12,CH,EQ,C'DENVER')
/*
```

Figure 33. JCL for Parameter List Interface Program Example

ICETOOL notes and restrictions

- Small REGION values can cause storage problems when ICETOOL calls DFSORT. Large REGION values give DFSORT the flexibility to use the storage it needs for best performance. We recommend that you use a REGION value of at least 1024K for ICETOOL.
- Each ICETOOL operation results in a set of ICETOOL messages in the TOOLMSG data set, and a corresponding set of DFSORT messages in the DFSMSG data set. For a particular call to DFSORT, you can relate the sets of messages in the TOOLMSG and DFSMSG data sets by using the unique identifier for that call. Just match the identifier printed in ICETOOL message ICE606I or ICE627I to the same identifier printed in DFSORT message ICE200I. This is particularly important if an ICETOOL operation fails due to an error detected by DFSORT (return code 16).
- Because ICETOOL calls DFSORT, the installation defaults used for DFSORT are those in effect for the appropriate program-invoked environment, that is, the ICEAM2 or ICEAM4 defaults, or the ICETDx defaults activated for ICEAM2 or ICEAM4. The DFSORT installation defaults apply only to DFSORT, not to ICETOOL. For example, installation option MSGCON=ALL causes DFSORT, but not ICETOOL, to write messages to the master console. The one exception is the SDBMSG installation option; the value in effect from ICEAM2 or ICEAM4 is used for ICETOOL's TOOLMSG data set.
- When ICETOOL calls DFSORT, it passes control statements and options appropriate to the specific operations being performed. You should not override the DFSORT control statements or options passed by ICETOOL unless you understand the ramifications of doing so.

  For example, ICETOOL passes the NOABEND option to DFSORT to ensure that ICETOOL will regain control if DFSORT issues an error message. If you specify:

  ```
  //DFSPARM DD *
    DEBUG ABEND
  ```

  you cause DFSORT to abend when it issues an error message, thus preventing ICETOOL from performing subsequent operators.
- Tape work data sets cannot be used with ICETOOL.
- An ON field must not include bytes beyond the fixed part of variable length input records. The entire field specified must be present in every input record, otherwise, DFSORT issues message ICE015A, ICE218A, or ICE027A and terminates.
- Avoid using the same dataset for both input and output in the same step, as this can lead to incorrect output.

Notes on using JOINKEYS with COPY and SORT

- You can use a JOINKEYS application with a COPY or SORT operator of ICETOOL, but not with any of the other ICETOOL operators. Use the JKFROM operand instead of the FROM operand to indicate you are using a JOINKEYS application.
- See Chapter 4, “Using a JOINKEYS application for joining two files,” on page 437 for complete details on JOINKEYS applications.
- If you want to sort the joined records, use a SORT operator with USING(xxxx) in TOOLIN and a SORT control statement along with the appropriate JOINKEYS, JOIN and REFORMAT statements in xxxxCNTL (main task).
- If you want to copy the joined records, use a COPY operator with USING(yyyy) in TOOLIN and the appropriate JOINKEYS, JOIN and REFORMAT statements in yyyyCNTL (main task).
- For multiple JOINKEYS applications within a single ICETOOL step:
  - You can reference different JOINKEYS F1 and F2 input files for the different SORT and/or COPY operators by using F1=ddname and F2=ddname on the various JOINKEYS statements as appropriate
  - You can reference different subtaskn message and control files for the different SORT and/or COPY operators by using TASKID=id on the various JOINKEYS statements as appropriate.

ICETOOL return codes

ICETOOL sets a return code for each operation it performs in STOP or CONTINUE mode and passes back the highest return code it encounters to the operating system or the invoking program.

For successful completion of all operations, ICETOOL passes back a return code of 0 or 4 to the operating system or the invoking program.

For unsuccessful completion due to an unsupported operating system, ICETOOL passes back a return code of 24 to the operating system or invoking program.

For unsuccessful completion of one or more operations, ICETOOL passes back a return code of 8, 12, 16, or 20 to the operating system or the invoking program.

The meanings of the return codes that ICETOOL passes back (in register 15) are:

0

Successful completion. All operations completed successfully.

4

Successful completion. All operations completed successfully. Either:

- RC4 was specified for an ICETOOL COUNT operator and the record count met the specified criteria, or
- DFSORT passed back a return code of 4 for one or more operations. See "DFSORT messages and return codes" on page 25 for details.

8

Unsuccessful completion. RC8 was specified for an ICETOOL COUNT operator and the record count met the specified criteria.

12

Unsuccessful completion. ICETOOL detected one or more errors that prevented it from completing successfully. Messages for these errors were printed in the TOOLMSG data set.

16

Unsuccessful completion. DFSORT detected one or more errors that prevented ICETOOL from completing successfully. Messages for these errors were printed in the DFSMSG data set.

20

Message data set error. The TOOLMSG DD statement was not present or the TOOLMSG data set was not opened.

24

Unsupported operating system. This operating system is not supported by this release of DFSORT.

Chapter 11. Examples of DFSORT job streams

Summary of examples

The following table summarizes the examples provided in this chapter.

Application	Number	Input	Output	Functions/Options
Sort	1	Disk	Tape	ALTSEQ
Sort	2	Disk	Disk	OMIT, SUM, OUTREC, DYNALLOC, ZDPRINT
Sort	3	Tape	Tape	ASCII Tapes
Sort	4	Tape	Disk	E15, E35, FILSZ, AVGRLEN, DYNALLOC
Sort	5	Disk	Disk	Program-invoked, SORTCNTL, CHALT, DYNALLOC, FILSZ
Sort	6	Disk	Disk	VSAM Input/Output, DFSPARM, Option Override
Sort	7	Disk	Disk	COBOL E15, EXEC PARM, MSGDDN
Sort	8	Disk	Disk	Dynamic Link-editing of Exits
Sort	9	E15	Disk	Extended Parameter List Interface
Sort	10	Disk	Disk and SYSOUT	OUTFIL
Sort	11	Pipe	Pipes	Pipes, OUTFIL SPLIT, FILSZ, DYNALLOC
Sort	12	Disk	Disk	INCLUDE, LOCALE
Sort	13	z/OS UNIX files	z/OS UNIX file	
Sort	14	Disk	Disk	IFTHEN
Sort	15	E15	Disk	64-bit parameter list interfaces
Merge	1	Disk	Disk	EQUALS
Merge	2	Disk	Disk	LOCALE, OUTFIL
Copy	1	Tape	Disk	EXEC PARMs, SKIPREC, MSGPRT, ABEND
Copy	2	Disk	Disk	INCLUDE, VLSHRT
Copy	3	Disk	Disk	OUTREC, PARSE, BUILD
ICEGENER	1	Disk	Disk	
ICETOOL	1	Disk	Disk	OCCUR, COPY, SORT, MODE, VERIFY, STATS, DISPLAY

Storage administrator examples

DFSORT provides a set of sample jobs that demonstrate techniques of interest to Storage Administrators and others who analyze data collected from DFSMShsm, DFSMSrmm, DCOLLECT and SMF. These sample jobs can be found in the ICESTGEX member of the SICESAMP library (contact your System Programmer for details). These sample jobs show some of the many ways DFSORT features such as ICETOOL and OUTFIL can be used to analyze data and generate reports:

DCOLEX1
DCOLLECT Example 1: VSAM report

DCOLEX2
DCOLLECT Example 2: Conversion reports

© Copyright IBM Corp. 1973, 2026

DCOLEX3
DCOLLECT Example 3: Capacity planning analysis and reports

DFHSMEX1
DFHSM Example 1: Deciphering Activity Logs

DFHSMEX2
DFHSM Example 2: Recover a DFHSM CDS with a broken index

RMMEX1
DFSMSrmm Example 1: SMF audit report

RMMEX2
DFSMSrmm Example 2: Create ADDVOLUME commands

REXX examples

Both DFSORT and ICETOOL can be called from REXX. The key is to specify ALLOCATE statements for the data sets you need and then use an ADDRESS statement like this:

```
  ADDRESS LINKMVS name
```

which says to fetch the named program using the standard system search list.

Here is an example of a REXX CLIST to call DFSORT:

```
/* Simple REXX CLIST to call DFSORT */
  "FREE FI(SYSOUT SORTIN SORTOUT SYSIN)"
  "ALLOC FI(SYSOUT)   DA(*)"
  "ALLOC FI(SORTIN)   DA('Y897797.INS1') REUSE"
  "ALLOC FI(SORTOUT)  DA('Y897797.OUTS1') REUSE"
  "ALLOC FI(SYSIN)    DA('Y897797.SORT.STMTS') SHR REUSE"
  ADDRESS LINKMVS ICEMAN
```

Here are the DFSORT control statements that might appear in the Y897797.SORT.STMTS data set:

```
  SORT FIELDS=(5,4,CH,A)
  INCLUDE COND=(21,3,SS,EQ,C'L92,J82,M72')
```

Here is an example of a REXX CLIST to call ICETOOL:

```
/* Simple REXX CLIST to call ICETOOL */
  "FREE FI(TOOLMSG DFSMSG VLR LENDIST TOOLIN)"
  "ALLOC FI(TOOLMSG)   DA(*)"
  "ALLOC FI(DFSMSG)   DUMMY"
  "ALLOC FI(VLR)   DA('Y897797.VARIN') REUSE"
  "ALLOC FI(LENDIST)   DA(*)"
  "ALLOC FI(TOOLIN)    DA('Y897797.TOOLIN.STMTS') SHR REUSE"
  ADDRESS LINKMVS ICETOOL
```

Here are the ICETOOL statements that might appear in the Y897797.TOOLIN.STMTS data set:

```
 OCCURS FROM(VLR) LIST(LENDIST) -
   TITLE('LENGTH DISTRIBUTION REPORT') BLANK -
   HEADER('LENGTH')  HEADER('NUMBER OF RECORDS') -
   ON(VLEN)          ON(VALCNT)
```

CLIST examples

Both DFSORT and ICETOOL can be called from a CLIST. They key is to specify ALLOCATE statements for the data sets you need and then use a CALL statement like this:

```
   CALL *(name)
```

Here is an example of a CLIST to call DFSORT:

```
FREE FI(SYSOUT SORTIN SORTOUT SYSIN)
ALLOC FI(SYSOUT) DA(*)
```

```
ALLOC FI(SORTIN) DA('Y897797.INS1') REUSE
ALLOC FI(SORTOUT) DA('Y897797.OUTS1') REUSE
ALLOC FI(SYSIN) DA('Y897797.SORT.STMTS') SHR REUSE
CALL *(ICEMAN)
```

Here are the DFSORT control statements that might appear in the Y897797.SORT.STMTS data set:

```
   SORT FIELDS=(5,4,CH,A)
   INCLUDE COND=(21,3,SS,EQ,C'L92,J82,M72')
```

Here is an example of a CLIST to call ICETOOL:

```
FREE FI(TOOLMSG DFSMSG VLR LENDIST TOOLIN)
ALLOC FI(TOOLMSG) DA(*)
ALLOC FI(DFSMSG) DUMMY
ALLOC FI(VLR) DA('Y897797.VARIN') REUSE
ALLOC FI(LENDIST) DA(*)
ALLOC FI(TOOLIN)  DA('Y897797.TOOLIN.STMTS') SHR REUSE
CALL *(ICETOOL)
```

Here are the ICETOOL statements that might appear in the Y897797.TOOLIN.STMTS data set:

```
   OCCURS FROM(VLR) LIST(LENDIST) -
     TITLE('LENGTH DISTRIBUTION REPORT') BLANK -
     HEADER('LENGTH')  HEADER('NUMBER OF RECORDS') -
     ON(VLEN)          ON(VALCNT)
```

Sort examples

This section includes sort examples.

Example 1. Sort with ALTSEQ

INPUT
Blocked variable-length records on disk

OUTPUT
Blocked variable-length records on 3490

WORK DATA SETS
Two 3390 data sets

USER EXITS
None

FUNCTIONS/OPTIONS
ALTSEQ

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//S1       EXEC PGM=SORT                                          02
//SYSOUT   DD SYSOUT=A                                            03
//SORTIN   DD DSN=A123456.IN5,DISP=SHR                            04
//SORTOUT  DD DSN=OUT1,UNIT=3490,DISP=(,KEEP),VOL=SER=VOL001      05
//SORTWK01 DD UNIT=3390,SPACE=(CYL,(10,10))                       06
//SORTWK02 DD UNIT=3390,SPACE=(CYL,(10,10))                       07
//SYSIN    DD *                                                   08
* COLLATE $, # and @ AFTER Z                                      09
  SORT FIELDS=(7,5,AQ,A)                                         10
  ALTSEQ CODE=(5BEA,7BEB,7CEC)                                   11
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias SORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to system output class A.

04
SORTIN DD statement. The input data set is named A123456.IN5 and is cataloged. DFSORT determines from the data set label that the RECFM is VB, the maximum LRECL is 120, and the BLKSIZE is 2200.

05
SORTOUT DD statement. The output data set is named OUT1 and is to be allocated on 3490 volume VOL001 and kept. DFSORT sets the RECFM and LRECL from SORTIN and selects an appropriate BLKSIZE for this standard labeled tape.

06
SORTWK01 DD statement. The first work data set is allocated on 3390.

07
SORTWK02 DD statement. The second work data set is allocated on 3390.

08
SYSIN DD statement. DFSORT control statements follow.

09
Comment statement. Printed but otherwise ignored.

10
SORT statement. FIELDS specifies an ascending 5-byte character control field starting at position 7 (the third data byte, because the RDW occupies the first 4 bytes). The control field is to be collated according to the modified sequence described in the ALTSEQ statement.

11
ALTSEQ statement. CODE specifies that the three characters $, # and @ are to collate in that order after Z.

Example 2. Sort with OMIT, SUM, OUTREC, DYNALLOC and ZDPRINT

INPUT
Blocked fixed-length records on 3380 and 3390

OUTPUT
Blocked fixed-length records on 3390

WORK DATA SETS
Dynamically allocated

USER EXITS
None

FUNCTIONS/OPTIONS
OMIT, OUTREC, SUM, DYNALLOC, ZDPRINT

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//STEP1    EXEC PGM=SORT                                          02
//SYSOUT   DD SYSOUT=H                                            03
//SORTIN   DD DSN=INP1,DISP=SHR,UNIT=3380,VOL=SER=SCR001          04
//         DD DSN=INP2,DISP=SHR,UNIT=3390,VOL=SER=SYS351          05
//SORTOUT  DD DSN=&&OUTPUT,DISP=(,PASS),UNIT=3390,                06
//   SPACE=(CYL,(5,1)),DCB=(LRECL=22)                             07
//SYSIN    DD *                                                   08
  OMIT COND=(5,1,CH,EQ,C'M')                                     09
  SORT FIELDS=(20,8,CH,A,10,3,FI,D)                              10
  SUM FIELDS=(16,4,ZD)                                           11
  OPTION DYNALLOC,ZDPRINT                                        12
  OUTREC FIELDS=(10,3,20,8,16,4,2Z,5,1,C' SUM')                  13
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias SORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to system output class H.

04-05
SORTIN DD statement. Consists of a concatenation of two data sets. The first input data set is named INP1 and resides on 3380 volume SCR001. The second input data set is named INP2 and resides on 3390 volume SYS351. DFSORT determines from the data set labels that the record format is FB, the LRECL is 80 and the largest BLKSIZE is 27920.

06-07
SORTOUT DD statement. The output data set is temporary and is to be allocated on a 3390. Because the OUTREC statement results in a reformatted output record length of 22 bytes, LRECL=22 must be specified. DFSORT sets the RECFM from SORTIN and selects an appropriate BLKSIZE.

08
SYSIN DD statement. DFSORT control statements follow.

09
OMIT statement. COND specifies that input records with a character M in position 5 are to be omitted from the output data set.

10
SORT statement. FIELDS specifies an ascending 8-byte character control field starting at position 20 and a descending 3-byte fixed-point control field starting at position 10.

11
SUM statement. FIELDS specifies a 4-byte zoned-decimal summary field starting at position 16. Whenever two records with the same control fields (specified in the SORT statement) are found, their summary fields (specified in the SUM statement) are to be added and placed in one of the records, and the other record is to be deleted.

12
OPTION statement. DYNALLOC specifies that work data sets are to be dynamically allocated using the installation defaults for the type of device and number of devices. ZDPRINT specifies that positive ZD SUM fields are to be printable.

13
OUTREC statement. FIELDS specifies how the records are to be reformatted for output. The reformatted records are 22 bytes long and look as follows:

Position
Content

1-3
Input positions 10 through 12

4-11
Input positions 20 through 27

12-15
Input positions 16 through 19

16-17
Zeros

18
Input position 5

19-22
The character string ' SUM'

Example 3. Sort with ASCII tapes

INPUT
Variable-length ASCII records on 3590

OUTPUT
Variable-length ASCII records on 3590

WORK DATA SETS
One SYSDA data set

USER EXITS
None

FUNCTIONS/OPTIONS
None

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//RUNIT    EXEC SORTD                                             02
//SORTIN   DD DSN=SRTFIL,DISP=(OLD,DELETE),UNIT=3590,             03
//  DCB=(RECFM=D,LRECL=400,BLKSIZE=404,OPTCD=Q,BUFOFF=L),        04
//  VOL=SER=311500,LABEL=(1,AL)                                    05
//SORTOUT  DD DSN=OUTFIL,UNIT=3590,LABEL=(,AL),DISP=(,KEEP),      06
//  DCB=(BLKSIZE=404,OPTCD=Q,BUFOFF=L),VOL=SER=311501              07
//SORTWK01 DD UNIT=SYSDA,SPACE=(CYL,(4))                          08
//SYSIN    DD *                                                   09
  SORT FIELDS=(10,8,AC,D)                                        10
  RECORD TYPE=D,LENGTH=(,,,20,80)                                11
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Uses the SORTD cataloged procedure to call DFSORT directly.

03-05
SORTIN DD statement. The input data set is named SRTFIL and resides on 3590 volume 311500. It is to be deleted after this job step. It has a RECFM of D (variable-length ASCII records), a maximum LRECL of 400, a BLKSIZE of 404 and an ASCII label. For this job, the buffer offset is the block length indicator. The records are to be translated from ASCII to EBCDIC.

06-07
SORTOUT DD statement. The output data set is named OUTFIL and is to be allocated on 3590 volume 311501 and kept. It is to be written with an ASCII label. DFSORT sets the RECFM and LRECL from SORTIN and sets the BLKSIZE to 404 as indicated in the DD statement. For this job, the buffer offset is the block length indicator. The records are to be translated from EBCDIC to ASCII.

08
SORTWK01 DD statement. The work data set is allocated on SYSDA.

09
SYSIN DD statement. DFSORT control statements follow.

10
SORT statement. FIELDS specifies a descending 8-byte ASCII control field starting at position 10.

11
RECORD statement. TYPE specifies ASCII variable-length records. LENGTH specifies that the minimum record length is 20 and the average record length is 80.

Example 4. Sort with E15, E35, FILSZ, AVGRLEN and DYNALLOC

INPUT
Variable-length records on 3490

OUTPUT
Blocked variable-length records on SYSDA

WORK DATA SETS
Dynamically allocated

USER EXITS
E15 and E35

FUNCTIONS/OPTIONS
FILSZ, AVGRLEN, DYNALLOC

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//STEP1    EXEC PGM=ICEMAN                                        02
//SYSOUT   DD SYSOUT=A                                            03
//SORTIN   DD DSN=INPUT,VOL=SER=FLY123,                           04
//  UNIT=3490,DISP=OLD                                            05
//SORTOUT  DD DSN=&&OUT,DISP=(,PASS),SPACE=(CYL,(10,12)),         06
//  UNIT=SYSDA,DCB=(RECFM=VB)                                     07
//MODLIB   DD DSN=EXIT1.RTNS,DISP=SHR                             08
//         DD DSN=EXIT2.RTNS,DISP=SHR                             09
//SYSIN    DD *                                                   10
  SORT FIELDS=(23,4,PD,A,10,6,FS,A)                              11
  OPTION DYNALLOC=(3390,3),AVGRLEN=75,FILSZ=E50000               12
  MODS E15=(MODREC,1024,MODLIB),E35=(ADDREC,1200,MODLIB)         13
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to system output class A.

04-05
SORTIN DD statement. The input data set is named INPUT and resides on 3490 volume FLY123. DFSORT determines from the data set label of this standard labeled tape that the RECFM is V, the LRECL is 120 and the BLKSIZE is 124.

06-07
SORTOUT DD statement. The output data set is temporary and is to be allocated on SYSDA. Because the input is unblocked and the output is to be blocked, RECFM=VB must be specified. DFSORT sets the LRECL from SORTIN and selects an appropriate BLKSIZE.

08-09
MODLIB DD statement. Specifies the load libraries that contain the exit routines. When exit routines reside in more than one library, the libraries must be concatenated using a single DD statement.

10
SYSIN DD statement. DFSORT control statements follow.

11
SORT statement. FIELDS specifies an ascending 4-byte packed-decimal control field starting at position 23 and an ascending 6-byte floating-sign control field starting at position 10.

12
OPTION statement. DYNALLOC=(3390,3) specifies that three 3390 work data sets are to be allocated. AVGRLEN=75 specifies an average record length of 75. AVGRLEN helps DFSORT optimize work space for variable-length record input. FILSZ=E50000 specifies an estimate of 50000 records. Because the 3490 input data set is compacted, DFSORT might not be able to determine the file size accurately unless the data set is managed by DFSMSrmm or a tape management system that uses ICETPEX. Specification of FILSZ can make a significant difference in work space optimization when tape input data sets are not managed.

13
MODS statement. E15 specifies a user exit routine named MODREC. Approximately 1024 bytes are required for MODREC and the system services (for example, GETMAIN and OPEN) it performs. E35 specifies a user exit routine named ADDREC. Approximately 1200 bytes are required for ADDREC and the system services it performs. MODREC and ADDREC reside in the libraries defined by the MODLIB DD statement.

Example 5. Called sort with SORTCNTL, CHALT, DYNALLOC and FILSZ

INPUT
Blocked fixed-length records on disk

OUTPUT
Blocked fixed-length records on disk

WORK DATA SETS
Dynamically allocated

USER EXITS
None

FUNCTIONS/OPTIONS
CHALT, DYNALLOC, FILSZ

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//RUNSORT  EXEC PGM=MYPGM                                         02
//STEPLIB  DD DSN=M999999.LOAD,DISP=SHR                           03
//SYSOUT   DD SYSOUT=A                                            04
//SYSPRINT DD SYSOUT=A                                            05
//SORTIN   DD DSN=M999999.INPUT(MASTER),DISP=OLD                  06
//SORTOUT  DD DSN=M999999.OUTPUT.FILE,DISP=OLD                    07
//SORTCNTL DD *                                                   08
   OPTION  CHALT,DYNALLOC=(,3),FILSZ=U25000                       09
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls a program named MYPGM that in turn calls DFSORT.

03
STEPLIB DD statement. Specifies the load library that contains MYPGM.

04
SYSOUT DD statement. Directs DFSORT messages and control statements to system output class A.

05
SYSPRINT DD statement. Directs MYPGM output to system output class A.

06
SORTIN DD statement. The input data set is member MASTER in the cataloged partitioned data set M999999.INPUT. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

07
SORTOUT DD statement. The output data set is named M999999.OUTPUT.FILE and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

08
SORTCNTL DD statement. DFSORT control statements follow. Statements in SORTCNTL override or supplement statements passed by MYPGM in the DFSORT parameter list it uses.

09
OPTION statement. CHALT specifies that character format control fields (specified in the SORT statement passed by MYPGM) are to be sorted using the ALTSEQ installation option. DYNALLOC=(,3) specifies that three work data sets are to be dynamically allocated using the installation default for the type of device. FILSZ=U25000 specifies a file size of 25000 records is to be used by DFSORT to determine the amount of work space needed. Because the input data set is a member of a PDS, specifying FILSZ helps DFSORT optimize work data set space.

Example 6. Sort with VSAM input/output, DFSPARM and option override

INPUT
VSAM TYPE=V records

OUTPUT
VSAM TYPE=V records

WORK DATA SETS
Dynamically allocated

USER EXITS
None

FUNCTIONS/OPTIONS
Override of Various Options

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//S1       EXEC PGM=SORT                                          02
//SYSOUT   DD SYSOUT=A                                            03
//SORTIN   DD DSN=TEST.SORTIN.FILE,DISP=SHR                       04
//SORTOUT  DD DSN=TEST.SORTOUT.FILE,DISP=SHR                      05
//DFSPARM  DD *                                                   06
  RECORD TYPE=V                                                   07
  SORT FIELDS=(30,4,BI,A)                                         08
  OPTION HIPRMAX=10,DYNALLOC=3390,MAINSIZE=3M,                    09
    MSGPRT=CRITICAL,NOLIST                                        10
```

For purposes of illustration, assume that none of the standard installation defaults for batch direct invocation of DFSORT have been changed by the site.

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias SORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to system output class A.

04
SORTIN DD statement. The input data set is TEST.SORTIN.FILE. DFSORT determines that it is a VSAM data set and obtains its attributes from the catalog.

05
SORTOUT DD statement. The output data set is TEST.SORTOUT.FILE. DFSORT determines that it is a VSAM data set and obtains its attributes from the catalog.

06
DFSPARM DD statement. DFSORT control statements follow. DFSPARM can be used for both direct-invocation and program-invocation of DFSORT and overrides options and statements from all other sources. Certain operands, such as MSGPRT and LIST/NOLIST, are used if supplied in DFSPARM, the EXEC PARM or the invocation parameter list, but not used if supplied in SYSIN or SORTCNTL.

07
RECORD statement. TYPE=V specifies that DFSORT is to treat the VSAM records as variable-length. In this case, the RECORD statement could be omitted, because DFSORT would automatically set a record type of V due to the use of VSAM data sets for SORTIN and SORTOUT.

08
SORT statement. FIELDS specifies an ascending 4-byte binary control field starting at position 30. This position corresponds to a specification of KEYS(4 25) for the VSAM CLUSTER (4 bytes at offset 25, which is equivalent to position 26 with 4 bytes added for the RDW that DFSORT supplies at input and removes at output for VSAM TYPE=V records).

09-10
OPTION statement. HIPRMAX=10 specifies that up to 10 MBs of Hiperspace can be used for Hipersorting, overriding the standard installation default of HIPRMAX=OPTIMAL. DYNALLOC=3390 specifies that work data sets are to be allocated on 3390s, overriding the standard installation default of SYSDA. The standard installation default of four work data sets is not overridden. MAINSIZE=3M specifies that up to 3 MBs of storage can be used, overriding the standard installation default of

MAINSIZE=MAX. MSGPRT=CRITICAL specifies that only error messages are to be printed, overriding the standard installation default of MSGPRT=ALL. NOLIST specifies that control statements are not to be printed, overriding the standard installation default of LIST=YES.

Example 7. Sort with COBOL E15, EXEC PARM and MSGDDN

INPUT
Fixed-length records on disk

OUTPUT
Fixed-length records on SYSDA

WORK DATA SETS
None

USER EXITS
COBOL E15

FUNCTIONS/OPTIONS
MSGDDN

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//STEP1    EXEC PGM=SORT,PARM='MSGDDN=DFSOUT'                     02
//STEPLIB  DD DSN=SYS1.SCEERUN,DISP=SHR                           03
//SYSOUT   DD SYSOUT=A                                            04
//DFSOUT   DD SYSOUT=A                                            05
//EXITC    DD DSN=COBEXITS.LOADLIB,DISP=SHR                       06
//SORTIN   DD DSN=SORT1.IN,DISP=SHR                               07
//SORTOUT  DD DSN=&&OUT,DISP=(,PASS),SPACE=(CYL,(5,5)),           08
//  UNIT=SYSDA,DCB=(LRECL=120)                                    09
//SYSIN    DD *                                                   10
  SORT FIELDS=(5,4,A,22,2,A),FORMAT=BI                           11
  MODS E15=(COBOLE15,37000,EXITC,C)                              12
  RECORD LENGTH=(,120)                                           13
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias name SORT. MSGDDN=DFSOUT specifies an alternate message data set for DFSORT messages and control statements to prevent the COBOL messages in SYSOUT from being interleaved with the DFSORT messages and control statements.

03
STEPLIB statement. Specifies the Language Environment library.

04
SYSOUT statement. Directs COBOL messages to system output class A.

05
DFSOUT statement. Directs DFSORT messages and control statements to system output class A (this is the alternate message data set specified by MSGDDN in the PARM field of the EXEC statement).

06
EXITC statement. Specifies the load library that contains the exit routine.

07
SORTIN DD statement. The input data set is named SORT1.IN and is cataloged. DFSORT determines from the data set label that the RECFM is F, the LRECL is 100 and the BLKSIZE is 100.

08-09
SORTOUT DD statement. The output data set is temporary and is to be allocated on SYSDA. Because the E15 exit changes the length of the records from 100 bytes to 120 bytes, LRECL=120 must be specified. DFSORT sets the RECFM from SORTIN and sets the BLKSIZE to the LRECL (unblocked records).

10
SYSIN DD statement. DFSORT control statements follow.

11
SORT statement. FIELDS specifies an ascending 4-byte control field starting at position 5 and an ascending 2-byte control field starting at position 22. FORMAT specifies that the control fields are binary.

12
MODS statement. E15 specifies a user exit routine named COBOLE15 written in COBOL. Approximately 37000 bytes are required for the exit, the system services (for example, GETMAIN and OPEN) it performs, and the COBOL library subroutines. COBOLE15 resides in the library defined by the EXITC DD statement.

13
RECORD statement. LENGTH specifies that the COBOL E15 routine changes the length of the records to 120 bytes.

Example 8. Sort with dynamic link-editing of exits

INPUT
Blocked fixed-length records on disk

OUTPUT
Blocked fixed-length records on 3380

WORK DATA SETS
One SYSDA data set

USER EXITS
E11, E15, E17, E18, E19, E31, E35, E38, E39

FUNCTIONS/OPTIONS
None

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//STEPA    EXEC SORT                                              02
//SORTIN   DD DSN=SMITH.INPUT,DISP=SHR                            03
//SORTOUT  DD DSN=SMITH.OUTPUT,DISP=(NEW,CATLG),                  04
//  UNIT=3380,SPACE=(TRK,(10,2)),VOL=SER=XYZ003                   05
//SORTWK01 DD UNIT=SYSDA,SPACE=(CYL,(1,1))                        06
//EXIT     DD DSN=SMITH.EXIT.OBJ,DISP=SHR                         07
//EXIT2    DD DSN=SMITH.EXIT2.OBJ,DISP=SHR                        08
//SORTMODS DD UNIT=SYSDA,SPACE=(TRK,(10,,3))                      09
//SYSIN    DD *                                                   10
  SORT FIELDS=(1,8,CH,A,20,4,BI,D)                               11
  MODS E11=(EXIT11,1024,EXIT,S),                                 12
       E15=(E15,1024,SYSIN,T),                                   13
       E17=(EXIT17,1024,EXIT2,T),                                14
       E18=(EXIT18,1024,EXIT,T),                                 15
       E19=(E19,1024,SYSIN,T),                                   16
       E31=(PH3EXIT,1024,EXIT,T),                                17
       E35=(PH3EXIT,1024,EXIT,T),                                18
       E38=(PH3EXIT,1024,EXIT,T),                                19
       E39=(E39,1024,SYSIN,T)                                    20
  END                                                            21
<object deck for E15 exit here>                                  22
<object deck for E19 exit here>                                  23
<object deck for E39 exit here>                                  24
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Uses the SORT cataloged procedure to call DFSORT directly and supply the DD statements (not shown) required by the linkage editor.

03
SORTIN DD statement. The input data set is named SMITH.INPUT and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

04-05
SORTOUT DD statement. The output data set is named SMITH.OUTPUT and is to be allocated on 3380 volume XYZ003 and cataloged. DFSORT sets the RECFM and LRECL from SORTIN and selects an appropriate BLKSIZE.

06
SORTWK01 DD statement. The work data set is allocated on SYSDA.

07
EXIT DD statement. Specifies the partitioned data set containing the object decks for the E11, E18, E31, E35 and E38 exit routines.

08
EXIT2 DD statement. Specifies the partitioned data set containing the object deck for the E17 exit routine.

09
SORTMODS DD statement. The partitioned data set to hold exit routine object decks from SYSIN for input to the linkage editor is to be allocated on SYSDA.

10
SYSIN DD statement. DFSORT control statements, and object decks to be used by the linkage editor, follow.

11
SORT statement. FIELDS specifies an ascending 8-byte character control field starting at position 1 and a descending 4-byte binary control field starting at position 20.

12-20
MODS statement. Specifies the exit routines to be used, the approximate number of bytes required for each exit and that:

- The EXIT11 routine in the EXIT library is to be link-edited separately from other input phase exit routines and associated with user exit E11.
- The E15 and E19 routines in SYSIN, the EXIT17 routine in EXIT2, and the EXIT18 routine in EXIT are to be link-edited together and associated with user exits E15, E19, E17, and E18, respectively.
- The E31, E35, and E38 routines in the PH3EXIT object deck and the E39 routine in SYSIN are to be link-edited together and associated with user exits E31, E35, E38, and E39, respectively.

21
END statement. Marks the end of the DFSORT control statements and the beginning of the exit routine object decks.

22-24
Object decks. The three object decks for the E15, E19, and E39 exit routines follow the END statement.

Example 9. Sort with the extended parameter list interface

INPUT
Fixed-length records from E15

OUTPUT
Blocked fixed-length records on SYSDA

WORK DATA SETS
Dynamically allocated

USER EXITS
E15

FUNCTIONS/OPTIONS
OMIT, FILSZ, RESINV, DYNALLOC

The JCL for running program MYSORT, and highlights of the code used by MYSORT to invoke DFSORT with the extended parameter list, are shown below. For purposes of illustration, assume that none of the standard installation defaults for batch program invocation of DFSORT have been changed by the site.

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//STEP1    EXEC PGM=MYSORT                                        02
//SYSOUT   DD SYSOUT=C                                            03
//MSGOUT   DD SYSOUT=C                                            04
//STEPLIB  DD DSN=A123456.LOAD,DISP=SHR                           05
//SORTOUT  DD DSN=&&OUTPUT,DISP=(,PASS),UNIT=SYSDA,               06
//  SPACE=(CYL,(8,4))                                             07
//SORTCNTL DD *                                                   08
* Update file size estimate                                       09
  OPTION FILSZ=E30000                                            10
------------------------------------------------------------------------

------------------------------------------------------------------------
MYSORT CSECT                                                      11
       .
       .
       .
       LA   R1,PL1            SET ADDRESS OF PARAMETER LIST     12
*                              TO BE PASSED TO DFSORT            13
       ST   R2,PL4            SET ADDRESS OF GETMAINED AREA     14
*                              TO BE PASSED TO E15               15
       LINK EP=SORT           INVOKE DFSORT                     16
       .
       .
       .
PL1    DC   A(CTLST)          ADDRESS OF CONTROL STATEMENTS     17
PL2    DC   A(E15)            ADDRESS OF E15 ROUTINE            18
PL3    DC   A(0)              NO E35 ROUTINE                    19
PL4    DS   A                 USER EXIT ADDRESS CONSTANT        20
PL5    DC   F'-1'             INDICATE END OF LIST              21
CTLST  DS   0H                CONTROL STATEMENTS AREA           22
       DC   AL2(CTL2-CTL1)    LENGTH OF CHARACTER STRING        23
CTL1   DC   C' SORT FIELDS=(5,8,CSF,A)'                         24
       DC   C' RECORD TYPE=F,LENGTH=80 '                        25
       DC   C' OPTION FILSZ=E25000,DYNALLOC,'                   26
       DC   C'RESINV=8000 '                                     27
       DC   C' OMIT FORMAT=CSF,COND=(5,8,EQ,13,8) '             28
CTL2   EQU  *                                                   29
OUT    DCB  DDNAME=MSGOUT,...                                   30
E15    DS   0H                E15 ROUTINE                       31
       .
       .
       .
       L    R2,4(,R1)         GET ADDRESS OF GETMAINED AREA     32
       .
       .
       .
       BR   R14               RETURN TO DFSORT                  33
       .
       .
       .
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls a program named MYSORT that in turn calls DFSORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to SYSOUT class C.

04
MSGOUT DD statement. Directs MYSORT messages to SYSOUT class C.

05
STEPLIB DD statement. Specifies the load library that contains MYSORT.

06-07
SORTOUT DD statement. The output data set is temporary and is to be allocated on SYSDA. Because SORTIN is not used, DFSORT sets the RECFM and LRECL from the RECORD statement and sets the BLKSIZE to the LRECL (unblocked records).

08
SORTCNTL DD statement. DFSORT control statements follow. Statements in SORTCNTL override or supplement statements passed by MYSORT in the extended parameter list it uses.

09
Comment statement. Printed but otherwise ignored.

10
OPTION statement. FILSZ=E30000 specifies an estimate of 30000 records, overriding FILSZ=E25000 in the OPTION statement of the extended parameter list. Because the E15 routine supplies all of the input records, DFSORT will not be able to determine the file size accurately; therefore, specifying FILSZ can make a significant difference in work space optimization when an E15 routine supplies all of the input records. It's important to change the FILSZ value whenever the number of input records increases significantly.

11
This is the start of program MYSORT. Assume that it GETMAINs a work area, saves its address in register 2, and initializes it with information to be used by the E15 routine.

12-13
MYSORT places the address of the extended parameter list to be passed to DFSORT in register 1.

14-15
MYSORT places the address of the GETMAINed work area in the user exit address constant field in the extended parameter list. DFSORT will pass this address to the E15 routine (in the second word of the E15 parameter list) when it is entered.

16
MYSORT calls DFSORT by it's alias SORT.

17-21
The extended parameter list specifies: the address of the control statements area, the address of the E15 routine, that no E35 routine is present, and the address of the GETMAINed work area. F'-1' indicates the end of the extended parameter list. Subsequent parameter list fields, such as the address of an ALTSEQ table, are not used in this application.

Because the address of the E15 routine is passed in the parameter list, SORTIN cannot be used; if a SORTIN DD statement were present, it would be ignored.

22-23
This is the start of the control statements area. The total length of the control statements is specified.

24
SORT statement. FIELDS specifies an ascending 8-byte floating-sign control field starting at position 5.

25
RECORD statement. TYPE=F and LENGTH=80 specify that the E15 inserts fixed-length records of 80 bytes. In this case, TYPE=F could be omitted, because DFSORT would automatically set a record type of F. However, LENGTH must be specified when an E15 supplies all of the input records.

26-27
OPTION statement. FILSZ=E25000 specifies an estimate of 25000 records, which is overridden by FILSZ=E30000 in SORTCNTL's OPTION statement. DYNALLOC specifies that work data sets are to be dynamically allocated using the installation defaults for the type of device and number of devices. RESINV=8000 specifies that approximately 8000 bytes are required for the system services (for example, GETMAIN and OPEN) that MYSORT's E15 exit routine performs.

28
OMIT statement. FORMAT specifies that the compare fields are floating-sign. COND specifies that input records with equal 8-byte floating-sign compare fields starting in position 5 (also the control field) and position 13 are to be omitted from the output data set.

29
This is the end of the control statements area.

30
This is the DCB for MYSORT's MSGOUT output.

31-33
This is MYSORT's E15 routine. The E15 routine loads the address of the GETMAINed work area from the second word of the E15 parameter list. The E15 routine must supply each input record by placing its address in register 1 and placing a 12 (insert) in register 15. When all the records have been passed, the E15 routine must place an 8 ("do not return") in register 15.

Example 10. Sort with OUTFIL

INPUT
Fixed-length record data set

OUTPUT
Multiple fixed-length record data sets

WORK DATA SETS
Dynamically allocated (by default)

USER EXITS
None

FUNCTIONS/OPTIONS
OUTFIL

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//OUTFIL   EXEC  PGM=SORT                                         02
//SYSOUT   DD  SYSOUT=A                                           03
//SORTIN   DD DSN=GRP.RECORDS,DISP=SHR                            04
//ALLGPS   DD DSN=GRP.ALLGRPS,DISP=OLD                            05
//ALLBU    DD DSN=GRP.BU,DISP=(NEW,CATLG,DELETE),                 06
//            UNIT=3390,SPACE=(TRK,(10,10))                       07
//G1STATS  DD SYSOUT=A                                            08
//G2STATS  DD SYSOUT=A                                            09
//SYSIN    DD    *                                                10
 SORT FIELDS=(6,5,CH,A)                                           11

 OUTFIL FNAMES=(ALLGPS,ALLBU)                                     12

 OUTFIL FNAMES=G1STATS,                                           13
   INCLUDE=(1,3,CH,EQ,C'G01'),                                    14
   HEADER2=(1:'GROUP 1 STATUS REPORT FOR ',&DATE,                 15
            ' - PAGE ',&PAGE,2/,                                  16
           6:'ITEM ',16:'STATUS      ',31:'PARTS',/,              17
           6:'-----',16:'------------',31:'-----'),               18
    OUTREC=(6:6,5,                                                19
           16:14,1,CHANGE=(12,                                    20
                           C'1',C'SHIP',                          21
                           C'2',C'HOLD',                          22
                           C'3',C'TRANSFER'),                     23
                   NOMATCH=(C'*CHECK CODE*'),                     24
           31:39,1,BI,M10,LENGTH=5,                               25
           120:X)                                                 26

 OUTFIL FNAMES=G2STATS,                                           27
   INCLUDE=(1,3,CH,EQ,C'G02'),                                    28
   HEADER2=(1:'GROUP 2 STATUS REPORT FOR ',&DATE,                 29
            ' - PAGE ',&PAGE,2/,                                  30
           6:'ITEM ',16:'STATUS      ',31:'PARTS',/,              31
           6:'-----',16:'------------',31:'-----'),               32
    OUTREC=(6:6,5,                                                33
           16:14,1,CHANGE=(12,                                    34
                           C'1',C'SHIP',                          35
                           C'2',C'HOLD',                          36
                           C'3',C'TRANSFER'),                     37
                   NOMATCH=(C'*CHECK CODE*'),                     38
           31:39,1,BI,M10,LENGTH=5,                               39
           120:X)                                                 40
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias name SORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to sysout class A.

04
SORTIN DD statement. The input data set is named GRP.RECORDS and is cataloged. DFSORT determines from the data set label that the RECFM is FB, the LRECL is 80 and the BLKSIZE is 23440.

05
ALLGPS DD statement. The first OUTFIL output data set is named GRP.ALLGRPS and is catalogued. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

06-07
ALLBU DD statement. The second OUTFIL output data set is named GRP.BU and is to be allocated on a 3390 and catalogued. DFSORT sets the RECFM and LRECL from SORTIN and selects an appropriate BLKSIZE.

08
G1STATS DD statement. The third OUTFIL output data set is directed to sysout class A. Because this is an OUTFIL report data set, DFSORT sets the RECFM to FBA (FB from SORTIN and A for ANSI control characters) and the LRECL to 121 (1 byte for the ANSI control character and 120 bytes for the data). DFSORT sets an appropriate BLKSIZE.

09
G2STATS DD statement. The fourth OUTFIL output data set is directed to sysout class A. Because this is an OUTFIL report data set, DFSORT sets the RECFM to FBA (FB from SORTIN and A for ANSI control characters) and the LRECL to 121 (1 byte for the ANSI control character and 120 bytes for the data). DFSORT sets an appropriate BLKSIZE.

10
SYSIN DD statement. DFSORT control statements follow.

11
SORT statement. FIELDS specifies an ascending 5-byte character control field starting at position 6.

12
OUTFIL statement. The sorted input records are written to the ALLGPS and ALLBU data sets.

13-26
OUTFIL statement. The subset of sorted input records containing 'G01' in positions 1 through 3 are used to produce a report, which is written to the G1STATS data set.

27-40
OUTFIL statement. The subset of sorted input records containing 'G02' in positions 1 through 3 are used to produce a report, which is written to the G2STATS data set.

Example 11. Sort with Pipes and OUTFIL SPLIT

INPUT
Pipes

OUTPUT
Pipes

WORK DATA SETS
Dynamically allocated

USER EXITS
None

FUNCTIONS/OPTIONS
FILSZ, OUTFIL, DYNALLOC

```
//EXAMP   JOB A400,PROGRAMMER                                    01
//RUNSORT EXEC PGM=ICEMAN                                        02
//SYSOUT  DD SYSOUT=H                                            03
//SORTIN  DD DSN=INPUT.PIPE,SUBSYS=PIPE,                         04
//           DCB=(LRECL=60,RECFM=FB,BLKSIZE=32760)               05
//OUT1    DD DSN=OUTPUT.PIPE1,SUBSYS=PIPE,                       06
//           DCB=(LRECL=60,RECFM=FB,BLKSIZE=32760)               07
//OUT2    DD DSN=OUTPUT.PIPE2,SUBSYS=PIPE,                       08
//           DCB=(LRECL=60,RECFM=FB,BLKSIZE=32760)               09
//SYSIN   DD *                                                   10
  OPTION DYNALLOC,FILSZ=U1000000                                 11
  SORT FIELDS=(1,20,CH,A,25,4,BI,A)                              12
  OUTFIL FNAMES=(OUT1,OUT2),SPLIT                                13
```

Line
Explanation

01
Job statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to system output class H.

04-05
SORTIN DD statement. The SUBSYS=PIPE parameter directs the allocation to the 'PIPE' subsystem for the pipe named INPUT.PIPE. The DCB statement describes the data set characteristics to subsystem PIPE.

06-07
OUT1 DD statement. The SUBSYS=PIPE parameter directs the allocation to the 'PIPE' subsystem for the pipe named OUTPUT.PIPE1. The DCB statement describes the data set characteristics to subsystem PIPE.

08-09
OUT2 DD statement. The SUBSYS=PIPE parameter directs the allocation to the 'PIPE' subsystem for the pipe named OUTPUT.PIPE2. The DCB statement describes the data set characteristics to subsystem PIPE.

10
SYSIN DD statement. DFSORT control statements follow.

11
OPTION statement. DYNALLOC specifies that work data sets are to be dynamically allocated using the installation defaults for type of device and number of devices. FILSZ=U1000000 specifies an estimate of one million input records.

12
SORT statement. FIELDS specifies an ascending 20-byte character control field starting at position 1 and an ascending 4 byte binary control field starting at position 25.

13
OUTFIL statement. The records from the SORTIN pipe are sorted and written alternatively to the OUT1 and OUT2 pipes (that is, the sorted records are split evenly between the two output pipes).

Example 12. Sort with INCLUDE and LOCALE

INPUT
Fixed-length record data set

OUTPUT
Fixed-length record data set

WORK DATA SETS
Dynamically allocated (by default)

USER EXITS
None

FUNCTIONS/OPTIONS
INCLUDE, LOCALE

```
//EXAMP    JOB  A400,PROGRAMMER                                  01
//STEP1    EXEC PGM=SORT,PARM='LOCALE=FR_CA'                     02
//STEPLIB  DD  DSN=SYS1.SCEERUN,DISP=SHR                         03
//SYSOUT   DD  SYSOUT=A                                          04
//SORTIN   DD  DSN=INPUT.FRENCH.CANADA,DISP=SHR                  05
//SORTOUT  DD  DSN=OUTPUT.FRENCH.CANADA,DISP=OLD                 06
//SYSIN    DD  *                                                 07
 SORT FIELDS=(1,20,CH,A,25,1,BI,D,30,10,CH,A)                   08
 INCLUDE COND=(40,6,CH,EQ,50,6,CH)                              09
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias name SORT. LOCALE specified in EXEC PARM overrides the installation default for LOCALE. The locale for the French language and the cultural conventions of Canada will be active.

03
STEPLIB DD statement. Specifies the Language Environment run-time library containing the dynamically loadable locales.

04
SYSOUT statement. Directs DFSORT messages and control statements to sysout class A.

05
SORTIN DD statement. The input data set is named INPUT.FRENCH.CANADA and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

06
SORTOUT DD statement. The output data set is named OUTPUT.FRENCH.CANADA and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

07
SYSIN DD statement. DFSORT control statements follow.

08
SORT statement. FIELDS specifies an ascending 20-byte character control field starting at position 1, a one-byte descending binary control field starting at position 25, and a 10-byte ascending character control field starting at position 30. The character (CH) control fields will be sorted according to the collating rules defined in locale FR_CA.

09
INCLUDE statement. COND specifies that only input records with equal 6-byte character compare fields starting in position 40 and position 50 are to be included in the output data set. The character (CH) compare fields will be compared according to the collating rules defined in locale FR_CA.

Example 13: Sort with z/OS UNIX files

INPUT
Concatenated z/OS UNIX Files

OUTPUT
z/OS UNIX File

WORK DATA SETS
Dynamically allocated (by default)

USER EXITS
None

FUNCTIONS/OPTIONS
None

```
//EXAMP   JOB A400,PROGRAMMER                                       01
//S1      EXEC PGM=SORT                                             02
//SYSOUT  DD SYSOUT=A                                               03
//SORTIN  DD  PATH='/user/hfs.inp1.txt',PATHOPTS=ORDONLY,           04
//        LRECL=80,BLKSIZE=240,RECFM=FB,FILEDATA=TEXT               05
//        DD  PATH='/user/hfs.inp2.txt',PATHOPTS=ORDONLY,           06
//        LRECL=80,BLKSIZE=80,RECFM=F,FILEDATA=TEXT                 07
//SORTOUT DD  PATH='/user/hfs.ut.txt',PATHOPTS=OWRONLY,             08
//        LRECL=80,BLKSIZE=80,RECFM=F,FILEDATA=TEXT                 09
//SYSIN DD *                                                        10
  SORT FIELDS=(10,8,CH,A)                                           11
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias name SORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to system output class A.

04-05
SORTIN DD statement. The first input file is a z/OS UNIX file named /user/hfs.inp1.txt. Only read access is allowed. The file is defined as a text file. It has fixed-length records with a record size of 80 and a block size of 240.

06-07
The second input file is a z/OS UNIX file named /user/hfs.inp2.txt. Only read access is allowed. The file is defined as a text file. It has fixed-length records with a record size of 80 and a block size of 80.

08-09
SORTOUT DD statement. The output file is a z/OS UNIX file named /user/hfs.ut.txt. Only write access is allowed. The file is defined as a text file. It has fixed-length records with a record size of 80 and a block size of 80.

10
SYSIN DD statement. DFSORT control statements follow.

11
SORT statement. FIELDS specifies an ascending 8-byte character control field starting at position 10.

Example 14. Sort with IFTHEN

```
INPUT                 Three fixed-length record data sets
OUTPUT                Fixed-length record data set
WORK DATA SETS        Dynamically allocated (by default)
USER EXITS            None
FUNCTIONS/OPTIONS     IFTHEN

//EXAMP JOB A400,PROGRAMMER                                       01
//S1    EXEC PGM=SORT                                             02
//SYSOUT DD SYSOUT=A                                              03
//SORTIN DD DSN=INPUT.FILE1,DISP=SHR                              04
//       DD DSN=INPUT.FILE2,DISP=SHR                              05
//       DD DSN=INPUT.FILE3,DISP=SHR                              06
//SORTOUT DD DSN=OUTPUT.FILE,DISP=(NEW,CATLG,DELETE),             07
//           SPACE=(CYL,(5,5)),UNIT=SYSDA                         08
//SYSIN DD *                                                      09
  INREC IFTHEN=(WHEN=(1,3,CH,EQ,C'HDR'),                         10
          OVERLAY=(6:YDDD=(D4/),81:C'0',82:SEQNUM,2,ZD)),         11
        IFTHEN=(WHEN=(1,3,CH,EQ,C'TRL'),                          12
          OVERLAY=(11:YDDD=(D4/),81:C'9',82:SEQNUM,2,ZD)),        13
        IFTHEN=(WHEN=NONE,                                        14
          OVERLAY=(81:C'1'))                                      15
  SORT FIELDS=(81,1,CH,A,8,5,CH,A)                                16
  OUTFIL REMOVECC,                                                17
```

```
    OMIT=(81,1,SS,EQ,C'0,9',AND,82,2,ZD,GT,+1),                    18
    OUTREC=(1,80)                                                  19
```

This example shows how you can use three input files, each with a header record ('HDR'), detail records ('DTL') and a trailer record ('TRL'), and create an output file with one header record with the current date, the sorted detail records, and one trailer record with the current date.

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias name SORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to sysout class A.

04-06
SORTIN DD statement. Consists of a concatenation of three input data sets: INPUT.FILE1, INPUT.FILE2 and INPUT.FILE3. DFSORT determines from the data set labels that each data set has RECFM=FB and LRECL=80. The BLKSIZEs vary. Each input data set has a header record, detail records, and a trailer record.

07-08
SORTOUT DD statement. Creates a new output data set: OUTPUT.FILE. DFSORT sets RECFM=FB, LRECL=80 and selects an appropriate BLKSIZE. The output data set will have one header record, the sorted detail records, and one trailer record.

09
SYSIN DD statement. DFSORT control statements follow.

10-15
INREC statement. The first IFTHEN WHEN=(logexp) clause identifies and operates on header records ('HDR' in positions 1-3); OVERLAY puts today's date in the form 'ddd/yyyy' in positions 6-13, adds a '0' in position 81, adds a ZD sequence number in positions 82-83 and does not affect the rest of the record.

The second IFTHEN WHEN=(logexp) clause identifies and operates on trailer records ('TRL' in positions 1-3); OVERLAY puts today's date in the form 'ddd/yyyy' in positions 11-18, adds a '9' in position 81, adds a ZD sequence number in positions 82-83 and does not affect the rest of the record.

The IFTHEN WHEN=NONE clause identifies and operates on detail records (not 'HDR' or 'TRL' in positions 1-3); OVERLAY adds a '1' in position 81 and does not affect the rest of the record.

DFSORT extends the reformatted input records from 80 bytes to 83 bytes to accommodate the identifier byte added in position 81 and the sequence number added in positions 82-83.

The '0', '1' or '9' identifier byte added in position 81 allows us to sort the header records ('0') first, followed by the detail records ('1'), and then the trailer records ('9'). The sequence number added in positions 82-83 will allow us to keep only the first header record and the first trailer record. The sequence number will be 1 for the first header record, 2 for the second header record and 3 for the third header record. Likewise, the sequence number will be 1 for the first trailer record, 2 for the second trailer record and 3 for the third trailer record. Since the sequence number is not specified for the detail records, it will be blank.

16
SORT statement. FIELDS specifies an ascending 1-byte character control field at position 81 (the identifier byte added by INREC), and an ascending 5-byte character control field starting at position 8 (the key for the detail records).

17-19
OUTFIL statement. REMOVECC removes the ANSI carriage control characters and ensures that the RECFM is FB rather than FBA. OMIT specifies that reformatted output records with '0' or '9' in position 81 (header or trailer records) and a sequence number in positions 82-83 greater than 1 (second and subsequent header or trailer records), are omitted. OUTREC keeps only positions 1-80 for the OUTFIL output records, thus removing the identifier byte and sequence number we added in positions 81-83 with the INREC statement (we do not want these temporary fields in the OUTFIL output records).

Example 15. Sort with 64-bit parameter lists, E15, E35 and OUTFIL

INPUT
Variable-length records from E15

OUTPUT
Variable-length output data set

WORK DATA SETS
Dynamically allocated (by default)

USER EXITS
E15 and E35

FUNCTIONS/OPTIONS
64-bit parameter lists, OUTFIL

The JCL for running program PGM1, and the code used by PGM1 to invoke DFSORT with the 64-bit invocation parameter list and use the 64-bit E15 and E35 parameter lists, are shown below.

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//STEP1    EXEC PGM=PGM1                                          02
//SYSOUT   DD SYSOUT=A                                            03
//SORTOUT  DD DSN=PL64.OUTPUT,DISP=(NEW,CATLG,DELETE),           04
//  SPACE=(CYL,(8,4)),UNIT=SYSDA                                  05
-------------------------------------------------------------------

-------------------------------------------------------------------
PGM1     CSECT                                                   06
PGM1     AMODE 64
PGM1     RMODE 31
         STM   14,12,12(13)        SAVE LOW PART REGS
         BASR  15,0                SET THE TEMPORARY
         USING *,15                 BASE REG
M0       DS    0H
         STMH  0,15,HIGHREGS       SAVE HIGH PART REGS
         LLGTR 12,15               SET THE PERMANENT
         DROP  15                   BASE
         USING M0,12                 REG
         LMH   0,15,HIGHZERO       SET HIGH PART OF REGS TO ZERO
         ST    13,SAVE+4           SAVE SAVE AREA ADDR
         LR    11,13               RELOAD SAVE AREA ADDR
         LA    13,SAVE             LOAD NEW SAVE AREA ADDR
         ST    13,8(11)            SAVE NEW SA ADDR INTO OLD SA
* Obtain 64-bit storage (memory object):
* - for 64-bit Invocation Parameter List
* - for 64-bit Control Statements Area
* - for work area passed to E15, E35 exits via USER ADDRESS CONSTANT
         STG   12,MOTOKEN          SAVE USERTOKEN
         LA    2,2                 SET NUMBER OF SEGMENTS
         STG   2,MOSEGM             FOR MEMORY OBJECT (MO) - 2MB
         IARV64 REQUEST=GETSTOR,   GET MEMORY OBJECT                  +
               ORIGIN=MOADDR,      ADDRESS OF MEMORY OBJECT           +
               USERTKN=MOTOKEN,    USERTOKEN                          +
               SEGMENTS=MOSEGM,    SIZE OF MEMORY OBJECT              +
               COND=YES,           CONDITIONAL REQUEST                +
               MF=(E,MOWORK)
         LTR   15,15               IF MEMORY OBJECT NOT OBTAINED,
         BNZ   NOMO                 EXIT WITH ERROR RETURN CODE
         LG    10,MOADDR           GET 64-BIT ADDR IN MO FOR
*                                   64-BIT INVOCATION PARMLIST
         USING ICE64INV,10         MAKE ADDRESSABLE
         XC    0(ICE64LNG,10),0(10) CLEAR 64-BIT INVOCATION PARM LIST
         MVC   ICEPLID(8),PL64SORT MOVE 64-BIT PARM LIST IDENTIFIER
         LA    2,1024(,10)         GET 64-BIT ADDR IN MEMORY OBJECT
*                                   FOR 64-BIT CONTROL STATEMENTS AREA
         STG   2,ICECTL             AND STORE IN 64-BIT PARM LIST
         MVC   0(CTLNG,2),CTLST    MOVE CONTROL STATEMENTS INTO MO
*
         LA    2,1024(,2)          GET 64-BIT ADDR IN MO FOR WORK AREA
         STG   2,MOWA               USED IN E15, E35 EXITS
         MVC   ICEUC(8),UADCON     MOVE USER EXIT ADDRESS CONSTANT
*
         LLGF  2,=A(E15)           STORE E15 ADDR
         STG   2,ICE15E32           INTO PARM LIST
         OI    ICEMDEX1,ICE15A64     WITH AMODE 64 AND
         OI    ICEMDEX2,ICE15PLT      WITH 64-BIT EXEC PARAMETER LIST
*
```

```
         LLGF  2,=A(E35)           STORE E35 ADDR
         STG   2,ICE35              INTO PARM LIST
         OI    ICEMDEX1,ICE35A64     WITH AMODE 64 AND
         OI    ICEMDEX2,ICE35PLT      WITH 64-BIT EXEC PARAMETER LIST
*                                    INTO 64-BIT INVOCATION PARM LIST
         LGR   1,10                LOAD ADDR OF 64-BIT INVOCATION
*                                    PARAMETER LIST
         LINK  EP=SORT64           INVOKE SORT WITH 64-BIT NAME
         LR    11,15               SAVE RETURN CODE
FREEMO   DS    0H
         IARV64 REQUEST=DETACH,      FREE MEMORY OBJECT               +
               MATCH=USERTOKEN,                                       +
               USERTKN=MOTOKEN,                                       +
               COND=YES,                                              +
               MF=(E,MOWORK)
         LR    15,11              RESTORE RETURN CODE
*
EXITALL  DS    0H
         LMH   0,14,HIGHREGS      RESTORE HIGH PART REGS
         L     13,SAVE+4          RESTORE OLD SA ADDR
         L     14,12(13)          RESTORE CALLER'S
         LM    1,12,24(13)         REGS
         BSM   0,14               RETURN TO CALLER
*
NOMO     LA    15,20              ERROR RETURN CODE (NOT MO)
         B     EXITALL            RETURN TO CALLER
*
SAVE     DC    18F'0'              SAVE AREA
HIGHREGS DC    16F'0'
HIGHZERO DC    16F'0'
*
CTLST    DS    0H                  ADDR OF CONTROL STMTS
         DC    AL2(CTL2-CTL1)      LENGTH OF CONTROL STATEMENT STRING
CTL1     EQU   *
         DC    C' SORT FIELDS=(5,4,BI,A)'
         DC    C' RECORD LENGTH=(80,80,80),TYPE=V '
         DC    C' OPTION FILSZ=E1000 '
         DC    C' OUTFIL FNAMES=SORTOUT '
CTL2     EQU   *
CTLNG    EQU   *-CTLST             LENGTH OF CONTROL STATEMENTS AREA
PL64SORT DC    C'PL64SORT'         IDENTIFIER OF 64-BIT PARM LIST
UADCON   DC    F'0',A(MOWA)        USER ADDRESS CONSTANT
MOWA     DC    D'0'                ADDR OF WORK AREA IN MO
*
MOTOKEN  DC    D'0'                USERTOKEN
MOSEGM   DC    D'0'                SIZE OF MO (IN MB)
MOADDR   DC    D'0'                ADDR OF MO
*
MOWORK   DS    0D                  IARV64 MACRO WORK AREA
         IARV64 MF=(L,MOV64L)
*
         LTORG
*
         ICEPL64                   MAPPING OF 64-BIT PARM LISTS
*
         DROP  12
PGM1     CSECT
************** E15 ********************************************
*
E15      DS    0H
* THIS E15 EXIT ROUTINE FORMS ALL RECORDS IN MEMORY OBJECT
* AND INSERTS THEM TO DFSORT AS 64-BIT ADRESSED RECORDS.
*
         STMG  14,12,8(13)        SAVE CALLER's REGISTERS
         BASR  11,0               SET
         USING *,11                ADDRESSABILITY
         LLGTR 11,11              SET CLEAN BASE REG
         LARL  14,SAVEE15         GET NEW SAVE AREA ADDRESS
         STG   13,128(,14)        CHAIN TO PREVIOUS SAVE AREA
         STG   14,136(,13)        CHAIN TO NEW SAVE AREA
         LGR   13,14              SET R13 TO SAVE AREA ADDRESS
         LMH   0,15,E15HZERO      SET HIGH PART OF REGS TO ZERO
         LR    10,1               SAVE EXIT PARAMETER LIST ADDRESS
         USING ICE64E15,10        SET ADDRESSABILITY
         ICM   15,15,COUNT        EXIT IF ALL RECORDS WERE
         BZ    EOF                 INSERTED
         LLGF  1,ICE15UC+4        GET USER CONSTANT ADDR
         LG    1,0(1)             GET ADDR OF THE WORK AREA IN MO
         MVC   0(80,1),RECE15     MOVE RECORD INTO MEMORY OBJECT
         BCTR  15,0               SET NEW
         ST    15,COUNT            SORT FIELD
*
```

```
         LA    15,12              RC=12 (INSERT RECORD)
* RETURN TO DFSORT
BACKE15  DS    0H
         LG    13,128(,13)        RESTORE CALLER's R13
         LG    14,8(,13)          RESTORE CALLER's R14
         LMG   2,12,40(13)        RESTORE CALLER's R2-R12
         BSM   0,14               RETURN
*
EOF      DS    0H
* Create final record in MO for its insert to DFSORT as 64-bit
* addressed record in the E35 exit
         LLGF  15,ICE15UC+4       GET USER CONSTANT ADDR
         LG    15,0(15)           GET ADDR IN MO FOR FINAL RECORD
         XC    0(80,15),0(15)     CLEAR RECORD's DATA
         MVI   1(15),8            SET NEW LENGTH OF FINAL RECORD
         MVC   4(4,15),RECNUM     SET 'NUMBER OF INSERTED RECORDS'
         LA    15,8               SET RETURN CODE
         B     BACKE15            RETURN CODE
*
E15HZERO DC    18F'0'
SAVEE15  DC    0D'0',F'0',C'F4SA',17FD'0'
HIGHE15  DC    16F'0'
FLAGE15  DC    X'00'
E15MOADR DC    D'0'               ADDR OF MO
E15SEGM  DC    D'0'               SIZE OF MO (IN MB)
E15TOKEN DC    D'0'               USERTOKEN
RECNUM   DC    F'1000'            NUMBER OF RECORDS
RECE15   DC    H'80'              LENGTH OF RECORD
         DC    H'0'
COUNT    DC    F'1000'            SORT FIELD
         DC    6C'ABCDEF'         DATA
         DC    6C'123456'          FIELDS
*
         LTORG
*
         DROP
PGM1     CSECT
************** E35 ******************************************
E35      DS    0H
*
* THIS E35 EXIT ACCEPTS ALL RECORDS FROM DFSORT
* AND AT EOF INSERTS ONE RECORD TO DFSORT AS 64-BIT RECORD.
* THE ADDRESS OF THE FINAL RECORD IN MEMORY OBJECT IS PASSED
* VIA USER ADDRESS CONSTANT FIELD OF 64-BIT PARAMETER LIST.
*
         STMG  14,12,8(13)        SAVE CALLER's REGISTERS
         BASR  11,0               SET
         USING *,11                ADDRESSABILITY
         LLGTR 11,11              SET CLEAN BASE REG
         LARL  14,SAVEE35         GET NEW SAVE AREA ADDRESS
         STG   13,128(,14)        CHAIN TO PREVIOUS SAVE AREA
         STG   14,136(,13)        CHAIN TO NEW SAVE AREA
         LGR   13,14              SET R13 TO SAVE AREA ADDRESS
         LMH   0,15,E35HZERO      SET HIGH PART REGS TO ZERO
         LR    10,1               SAVE EXIT PARM LIST ADDRESS
         USING ICE64E35,10        SET ADDRESSABILITY
         ICM   1,15,ICE35RL+4     GET ADDR OF NEW RECORD
* IF ADDR OF NEW RECORD IT IS ZERO, EOF IS INDICATED
         BZ    RC8E35             BR IF EOF
*
         LA    15,0               'ACCEPT' RETURN CODE
* RETURN TO DFSORT
BACKE35  DS    0H
         LG    13,128(,13)        RESTORE CALLER's R13
         LG    14,8(,13)          RESTORE CALLER's R14
         LMG   2,12,40(13)        RESTORE CALLER's R2-R12
         BSM   0,14               RETURN
*
RC8E35   DS    0H
         TM    FLAGE35,X'01'      IF FINAL FLAG IS ON
         BO    RC8E35A             EXIT WITH 8 RETURN CODE
         MVI   FLAGE35,X'01'      SET FINAL FLAG
         LLGF  1,ICE35UC+4         GET USER CONSTANT ADDR
         LG    1,0(1)             GET MO ADDR WITH FINAL RECORD
         LA    15,12              SET 'INSERT' RETURN CODE
         B     BACKE35            RETURN TO DFSORT
RC8E35A  DS    0H
         LA    15,8               SET RETURN CODE
         B     BACKE35            RETURN TO DFSORT
*
E35HZERO DC    18F'0'
HIGHE35  DC    18F'0'
```

```
SAVEE35  DC    0D'0',F'0',C'F4SA',17FD'0'
FLAGE35  DC    X'00'
*
         LTORG
*
         END
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls a program named PGM1 that in turn calls DFSORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to SYSOUT class A.

04-05
SORTOUT DD statement. The OUTFIL data set.

06
Start of the complete source code for PGM1. PGM1 uses the ICEPL64 mapping macro in the DFSORT target library, SICEUSER, to provide separate Assembler DSECTs for the 64-bit invocation and exit parameter lists.

PGM1 LINKs to DFSORT using the name SORT64 to indicate that it is using the 64-bit invocation parameter list. PGM1 passes SORT, RECORD, OPTION and OUTFIL statements in the control statement area.

PGM1's E15 user exit passes 64-bit addressed records to DFSORT.

PGM1's E35 user exit accepts all of the sorted records and inserts one additional record at EOF. DFSORT writes all of the records to the OUTFIL data set.

MERGE examples

Although the MERGE operators in the examples in this section could all be contained in a single ICETOOL job step, they are shown and discussed separately for clarity.

Example 1. Merge with EQUALS

INPUT
Blocked fixed-length records on disk

OUTPUT
Blocked fixed-length records on 3390

WORK DATA SETS
Not applicable

USER EXITS
None

FUNCTIONS/OPTIONS
EQUALS

```
//EXAMP    JOB A400,PROGRAMMER                                  01
//STEP1    EXEC PGM=SORT                                        02
//SYSOUT   DD   SYSOUT=A                                        03
//SORTIN01 DD   DSN=M1234.INPUT1,DISP=SHR                       04
//SORTIN02 DD   DSN=M1234.INPUT2,DISP=SHR                       05
//SORTIN03 DD   DSN=M1234.INPUT3,DISP=SHR                       06
//SORTOUT  DD   DSN=M1234.MERGOUT,DISP=(,KEEP),                 07
//  SPACE=(CYL,(2,4)),UNIT=3390                                 08
//SYSIN    DD *                                                 09
    MERGE FIELDS=(1,8,CH,A,20,4,PD,A)                           10
    OPTION EQUALS                                               11
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias SORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to sysout class A.

04
SORTIN01 DD statement. The first input data set is named M1234.INPUT1 and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

05
SORTIN02 DD statement. The second input data set is named M1234.INPUT2 and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

06
SORTIN03 DD statement. The third input data set is named M1234.INPUT3 and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

07-08
SORTOUT DD statement. The output data set is named M1234.MERGOUT and is to be allocated on 3390 and kept. DFSORT sets the RECFM and LRECL from the SORTINnn data sets and selects an appropriate BLKSIZE.

09
SYSIN DD statement. DFSORT control statements follow.

10
MERGE statement. FIELDS specifies an ascending 8-byte character control field starting at position 1 and an ascending 4-byte packed-decimal field starting at position 20. The records in each input data set must already be in the order specified.

25
OPTION statement. EQUALS specifies that the order of output records with equal control fields is to be based on the file number of the input data sets and the original order of the records within each input data set.

Example 2. Merge with LOCALE and OUTFIL

INPUT
Fixed-length record data set

OUTPUT
Multiple fixed-length record data sets

WORK DATA SETS
Not applicable

USER EXITS
None

FUNCTIONS/OPTIONS
LOCALE, OUTFIL

```
//EXAMP    JOB  A400,PROGRAMMER                                  01
//STEP1    EXEC PGM=SORT                                         02
//STEPLIB  DD  DSN=SYS1.SCEERUN,DISP=SHR                         03
//SYSOUT   DD  SYSOUT=A                                          04
//SORTIN01 DD  DSN=INPUT01.GERMAN.GERMANY,DISP=SHR               05
//SORTIN02 DD  DSN=INPUT02.GERMAN.GERMANY,DISP=SHR               06
//SORTIN03 DD  DSN=INPUT03.GERMAN.GERMANY,DISP=SHR               07
//GP1      DD  DSN=OUTPUT.GERMAN.GP1,DISP=OLD                    08
//GP2      DD  DSN=OUTPUT.GERMAN.GP2,DISP=OLD                    09
//GP3      DD  DSN=OUTPUT.GERMAN.SAVE,DISP=OLD                   10
//DFSPARM  DD  *                                                 11
 LOCALE=De_DE.IBM-1047                                         12
```

```
MERGE FIELDS=(25,5,CH,A,40,4,PD,D)                              13
OUTFIL FNAMES=GP1,                                              14
  INCLUDE=(23,1,CH,LE,C'Ö')                                     15
OUTFIL FNAMES=GP2,                                              16
  INCLUDE=(23,1,CH,GT,C'Ö',AND,                                 17
           23,1,CH,LT,C'Ü')                                     18
OUTFIL FNAMES=GP3,SAVE                                          19
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias name SORT.

03
STEPLIB DD statement. Specifies the Language Environment run-time library containing the dynamically loadable locales.

04
SYSOUT statement. Directs DFSORT messages and control statements to sysout class A.

05
SORTIN01 DD statement. The first input data set is named INPUT01.GERMAN.GERMANY and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

06
SORTIN02 DD statement. The second input data set is named INPUT02.GERMAN.GERMANY and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

07
SORTIN03 DD statement. The third input data set is named INPUT03.GERMAN.GERMANY and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

08
GP1 DD statement. The first OUTFIL output data set is named OUTPUT.GERMAN.GP1 and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

09
GP2 DD statement. The second OUTFIL output data set is named OUTPUT.GERMAN.GP2 and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

10
GP3 DD statement. The third OUTFIL output data set is named OUTPUT.GERMAN.GP3 and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

11
DFSPARM DD statement. DFSORT control statements follow.

12
LOCALE parameter. Overrides the installation default for LOCALE. The locale for the German language and the cultural conventions of Germany based on the IBM-1047 encoded character set will be active.

13
MERGE statement. FIELDS specifies an ascending 5-byte character control field starting at position 25, and a descending 4-byte packed decimal control field starting at position 40. The character (CH) control field will be merged according to the collating rules defined in locale De_DE.IBM-1047. The records in each input data set must already be in the order specified.

14-15
OUTFIL statement. The subset of records with character values less than or equal to 'Ö' in position 23 are written to the GP1 output data set. The character (CH) compare field and character constant will be compared according to the collating rules defined in locale De_DE.IBM-1047.

16-18
OUTFIL statement. The subset of records with character values greater than 'Ö' but less than 'Ü' in position 23 are written to the GP2 output data set. The character (CH) compare fields and character constants will be compared according to the collating rules defined in locale De_DE.IBM-1047.

19
OUTFIL statement. Any records not written to the GP1 or GP2 output data sets are written to the GP3 output data set.

Copy examples

Example 1

```
* Method 1
COPY FROM(MASTER) TO(PRINT,TAPE,DISK)
```

```
* Method 2
COPY FROM(MASTER) TO(DISK,TAPE,PRINT) SERIAL
```

This example shows two different methods for creating multiple output data sets.

Method 1 requires one call to DFSORT, one pass over the input data set, and allows the output data sets to be specified in any order. The COPY operator copies all records from the MASTER data set to the PRINT (SYSOUT), TAPE, and DISK data sets, using OUTFIL processing.

Method 2 requires three calls to DFSORT, three passes over the input data set, and imposes the restriction that the SYSOUT data set must not be the first TO data set. The COPY operator copies all records from the MASTER data set to the DISK data set and then copies the resulting DISK data set to the TAPE and PRINT (SYSOUT) data sets. Because the first TO data set is processed three times (written, read, read), placing the DISK data set first is more efficient than placing the TAPE data set first. PRINT must not be the first in the TO list because a SYSOUT data set cannot be read.

Example 1. Copy with EXEC PARMs, SKIPREC, MSGPRT and ABEND

INPUT
Blocked fixed-length records on multivolume 3490

OUTPUT
Blocked fixed-length records on disk

WORK DATA SETS
Not applicable

USER EXITS
None

FUNCTIONS/OPTIONS
SKIPREC, MSGPRT, ABEND

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//STEP1    EXEC   PGM=SORT,                                       02
//  PARM='SKIPREC=500,MSGPRT=CRITICAL,ABEND'                      03
//SYSOUT   DD SYSOUT=A                                            04
//SORTIN   DD DSN=FLY.RECORDS,VOL=SER=(000333,000343),            05
//  UNIT=(3490,2),DISP=OLD,LABEL=(,NL),                           06
//  DCB=(RECFM=FB,LRECL=12000,BLKSIZE=24000)                      07
//SORTOUT  DD DSN=FLY.RECORDS.COPY,DISP=OLD                       08
//SYSIN DD *                                                      09
  SORT FIELDS=COPY                                              10
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02-03
EXEC statement. Calls DFSORT directly by its alias SORT. SKIPREC=500 specifies that the first 500 input records are not to be included in the output data set. MSGPRT=CRITICAL specifies that error

messages, but not informational messages, are to be printed. ABEND specifies that DFSORT is to terminate with a user ABEND if it issues an error message.

04
SYSOUT DD statement. Directs DFSORT messages and control statements to sysout class A.

05-07
SORTIN DD statement. The input data set is named FLY.RECORDS and resides on 3490 volumes 000333 and 000343. The UNIT parameter requests two tape drives, one for each volume of the data set. Because the tape is unlabeled, DCB parameters must be supplied to indicate that the RECFM is FB, the LRECL is 12000 and the BLKSIZE is 24000.

08
SORTOUT DD statement. The output data set is named FLY.RECORDS.COPY and is cataloged. DFSORT determines the RECFM, LRECL and BLKSIZE from the data set label.

09
SYSIN DD statement. DFSORT control statements follow.

10
SORT statement. FIELDS=COPY specifies a copy application.

Example 2. Copy with INCLUDE and VLSHRT

INPUT
Blocked spanned records on disk

OUTPUT
Blocked spanned records on SYSDA

WORK DATA SETS
Not applicable

USER EXITS
None

FUNCTIONS/OPTIONS
INCLUDE, VLSHRT

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//COPY     EXEC PGM=SORT                                          02
//SYSOUT   DD SYSOUT=A                                            03
//SORTIN   DD DSN=SMF.DATA,DISP=SHR                               04
//SORTOUT  DD DSN=SMF.VIOL,DISP=(,KEEP),SPACE=(CYL,(2,5)),        05
//  UNIT=SYSDA                                                    06
//SYSIN    DD *                                                   07
  INCLUDE COND=(6,1,FI,EQ,80,AND,19,1,BI,EQ,B'1.......')         08
  OPTION COPY,VLSHRT                                             09
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly by its alias SORT.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to sysout class A.

04
SORTIN DD statement. The input data set is named SMF.DATA and is cataloged. DFSORT determines from the data set label that the RECFM is VBS, the LRECL is 32760 and the BLKSIZE is 23476.

05-06
SORTOUT DD statement. The output data set is named SMF.VIOL and is to be allocated on SYSDA and kept. DFSORT sets the RECFM and LRECL from SORTIN and selects an appropriate BLKSIZE.

07
SYSIN DD statement. DFSORT control statements follow.

08
INCLUDE statement. COND specifies that only input records with decimal 80 in the 1-byte fixed-point field at position 6 and bit 0 on in the 1-byte binary field at position 19 are to be included in the output data set.

09
OPTION statement. COPY specifies a copy application. VLSHRT specifies that records that are too short to contain all of the INCLUDE compare fields are not to be included in the output data set.

Example 3. Copy with OUTREC, PARSE and BUILD

```
INPUT                Blocked fixed-length records on disk
OUTPUT               Blocked fixed-length records on disk
WORK DATA SETS       Not applicable
USER EXITS           None
FUNCTIONS/OPTIONS    OUTREC, PARSE, BUILD

//EXAMP JOB A400,PROGRAMMER                                        01
//S1 EXEC PGM=ICEMAN                                               02
//SYSOUT DD SYSOUT=A                                               03
//SORTIN DD DSN=FLY.VAR.FIELDS.IN,DISP=SHR                         04
//SORTOUT DD DSN=FLY.FIX.FIELDS.OUT,DISP=OLD                       05
//SYSIN DD *                                                       06
  OPTION COPY                                                      07
  OUTREC PARSE=(%00=(ABSPOS=21,STARTAFT=BLANKS,                    08
                    ENDBEFR=C',',FIXLEN=5),                        09
       %=(ENDBEFR=C','),                                           10
       %01=(ENDBEFR=C',',FIXLEN=10),                               11
       %02=(FIXLEN=7)),                                            12
  BUILD=(1,15,                                                     13
    18:%00,JFY=(SHIFT=RIGHT,LEAD=C'(',TRAIL=C')',LENGTH=7),        14
    30:%02,SFF,TO=ZD,LENGTH=7,                                     15
    40:%01,60:X)                                                   16
```

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls DFSORT directly.

03
SYSOUT DD statement. Directs DFSORT messages and control statements to sysout class A.

04
SORTIN DD statement. The input data set is named FLY.VAR.FIELDS.IN and is cataloged. DFSORT determines from the data set label that the RECFM is FB, the LRECL is 60 and the BLKSIZE is 27960.

05
SORTOUT DD statement. The output data set is named FLY.FIX.FIELDS.OUT and is cataloged. DFSORT determines from the data set label that the RECFM is FB, the LRECL is 60 and the BLKSIZE is 27960.

06
SYSIN DD statement. DFSORT control statements follow.

07
OPTION statement. COPY specifies a copy application.

08-16
OUTREC statement. PARSE and BUILD reformat the input records containing one fixed position/length field and four variable position/length fields to output records containing four fixed position/length fields.

ICEGENER example

This section contains an ICEGENER example.

INPUT
Same as for IEBGENER job

OUTPUT
Same as for IEBGENER job

WORK DATA SETS
Not applicable

USER EXITS
None

FUNCTIONS/OPTIONS
None

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//GENR     EXEC PGM=ICEGENER                                      02
//SYSPRINT DD SYSOUT=A                                            03
//SYSUT1   DD DSN=CTL.MASTER,DISP=SHR                             04
//SYSUT2   DD DSN=CTL.BACKUP,DISP=OLD                             05
//SYSIN    DD DUMMY                                               06
```

This example shows how to use the ICEGENER facility for an IEBGENER job if your site has not installed ICEGENER as an automatic replacement for IEBGENER. The ICEGENER facility selects the more efficient DFSORT copy function for this IEBGENER job.

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls the ICEGENER facility. PGM=IEBGENER has been replaced by PGM=ICEGENER.

03-06
No other changes to the IEBGENER job are required.

ICETOOL example

This section contains an example of ICETOOL with various operators.

INPUT
Multiple data sets

OUTPUT
Multiple data sets

WORK DATA SETS
Dynamically allocated (automatic)

USER EXITS
ICETOOL's E35 (automatic)

FUNCTIONS/OPTIONS
OCCUR, COPY, SORT, MODE, VERIFY, STATS, DISPLAY

```
//EXAMP    JOB A400,PROGRAMMER                                    01
//TOOLRUN  EXEC PGM=ICETOOL,REGION=1024K                          02
//TOOLMSG  DD SYSOUT=A                                            03
//DFSMSG   DD SYSOUT=A                                            04
//TOOLIN   DD *                                                   05
* Print report showing departments with less than 5 employees     06
  OCCUR  FROM(IN1) LIST(LT5) LOWER(5) BLANK -                     07
```

```
    TITLE('Small Departments') PAGE -                           08
    HEADER('Department') HEADER('Employees') -                  09
    ON(45,3,CH)          ON(VALCNT)                             10
* Sort by last name and first name                              11
   SORT FROM(IN1) TO(DEPTSD,DEPTSP) USING(ABCD)                 12
* Do following operators even if a previous operator failed,    13
* but stop processing if a subsequent operator fails.          14
 MODE STOP                                                      15
* Verify decimal fields                                        16
   VERIFY FROM(IN2) ON(22,6,PD) ON(30,3,ZD)                     17
* Print statistics for record length and numeric fields        18
   STATS FROM(IN2) ON(VLEN) ON(22,6,PD) ON(30,3,ZD)             19
* Sort and produce total for each unique key                   20
   SORT FROM(IN2) TO(OUT4) USING(CTL1)                          21
* Print report containing:                                     22
*   - key and total for each unique key                        23
*   - lowest and highest of the totals                         24
  DISPLAY FROM(OUT4) LIST(LIST1) -                              25
    TITLE('Unique key totals report') DATE TIME -               26
    ON(5,10,CH) ON(22,6,PD) ON(30,3,ZD) -                       27
    MINIMUM('Lowest') MAXIMUM('Highest') PLUS                   28
//LT5      DD SYSOUT=A                                          29
//IN1      DD DSN=FLY.INPUT1,DISP=SHR                           30
//ABCDCNTL DD *                                                 31
* Sort by last name, first name                                 32
  SORT FIELDS=(12,15,CH,A,1,10,CH,A)                            33
//DEPTSD   DD DSN=FLY.OUTPUT1,DISP=SHR                         34
//DEPTSP   DD SYSOUT=A                                         35
//IN2      DD DSN=FLY.INPUT2,DISP=SHR                          36
//OUT4     DD DSN=FLY.OUTPUT2,DISP=OLD                         37
//CTL1CNTL DD *                                                38
* Sort and produce totals in one record for each unique key    39
  SORT FIELDS=(5,10,CH,A)                                       40
  SUM FIELDS=(22,6,PD,30,3,ZD)                                  41
//LIST1    DD SYSOUT=A                                         42
```

This example shows how ICETOOL can be used to perform multiple operations in a single step.

Line
Explanation

01
JOB statement. Introduces this job to the operating system.

02
EXEC statement. Calls ICETOOL specifying the recommended REGION of 1024K.

03
TOOLMSG DD statement. Directs ICETOOL messages and statements to system output class A.

04
DFSMSG DD statement. Directs DFSORT messages and control statements to SYSOUT class A.

05
TOOLIN DD statement. ICETOOL statements follow. The MODE for the ICETOOL run is initially set to STOP. If an error is detected for an operator, SCAN mode will be entered.

06
Comment statement. Printed but otherwise ignored.

07-10
OCCUR operator. Prints, in the LT5 data set, a report detailing each value for the specified field in the IN1 data set and the number of times that value occurs.

11
Comment statement.

12
SORT operator. Records from the IN1 data set are sorted to the DEPTSD and DEPTSP data sets using the DFSORT control statements in the ABCDCNTL data set. As a result, FLY.OUTPUT1 and DEPTSP (SYSOUT) contain the sorted records from FLY.INPUT1.

13-14
Comment statements.

Appendix C. Data format descriptions

DFSORT data formats

DFSORT supports a large number of data formats as described below.

Format	Description
CH	(character EBCDIC, unsigned). Each character is represented by its 8-bit EBCDIC code. Example: AB7 becomes C1 C2 F7 Hexadecimal 11000001 11000010 11110111 Binary **Note:** 1. If CHALT is in effect, a format CH field collates according to the ALTSEQ (alternate collating sequence) table in effect. AQ format can be used for the same purpose. 2. If locale processing is in effect, a format CH field collates according to the collating rules of the active locale.
UTF8	Unicode Transformation Format, an 8-bit encoding form designed for ease of use with existing ASCII-based systems. UTF-8 can encode any of the Unicode characters. A UTF-8 character is 1, 2, 3, or 4 bytes in length. A UTF-8 data string can contain any combination of SBCS and MBCS data, including supplementary characters. **Character** A **Unicode** U+0041 **UTF-8 (Half bytes in hexadecimal)** 01000001 4 1 => X'41' **Character** § **Unicode** U+00A7 **UTF-8 (Half bytes in hexadecimal)** 11000010 10100111 C 2 A 7 => X'C2A7' **Character** 𑖌 **Unicode** U+1156 **UTF-8 (Half bytes in hexadecimal)** 11100001 10000101 10010110 E 1 8 5 9 6 => x'E18596' **Note:** 1. Refer to the Unicode Character Code Charts (www.unicode.org/charts) for all the code point representations. 2. When sorting/merging, a format UTF8 field collates according to the collating version of the active COLLKEY. UCA600 is the default collation version.

© Copyright IBM Corp. 1973, 2026

Format	Description
UTF16	Unicode Transformation Format, a 16-bit encoding form designed to provide code values for over a million characters. UTF-16 can encode any of the Unicode characters. In UTF-16 encoding, characters are 2 bytes in length, except for supplementary characters, which take two 2 byte string units per character. **Character** A **Unicode** U+0041 **UTF16 (hexadecimal)** 00000000 01000001 0 0 4 1 => X'0041' **Character** § **Unicode** U+00A7 **UTF16 (hexadecimal)** 00000000 10100111 0 0 A 7 => X'00A7' **Character** 퐈 **Unicode** U+1156 **UTF16 (hexadecimal)** 00010001 01010110 1 1 5 6 => x'1156' **Note:** 1. Refer to the Unicode Character Code Charts (www.unicode.org/charts) for all the code point representations. 2. When sorting/merging, a format UTF16 field collates according to the collating version of the active COLLKEY. UCA600 is the default collation version.

Format	Description
UTF32	Unicode Transformation Format, a 32-bit encoding form with a fixed-length encoding, in contrast to all other Unicode transformation formats, which are variable-length encodings. Each 32-bit value in UTF32 represents one Unicode code point and is exactly equal to that code point's numerical value. **Character** A **Unicode** U+0041 **UTF32 (hexadecimal)** 00000000 00000000 01000001 0 0 0 0 4 1 => X'00000041' **Character** § **Unicode** U+00A7 **UTF32 (hexadecimal)** 00000000 00000000 10100111 0 0 0 0 A 7 => X'00A7' **Character** 𑅖 **Unicode** U+1156 **UTF32 (hexadecimal)** 00000000 00000000 00010001 01010110 0 0 0 0 1 1 5 6 => X'1156' **Note:** 1. Refer to the Unicode Character Code Charts (www.unicode.org/charts) for all the code point representations. 2. When sorting/merging, a format UTF32 field collates according to the collating version of the active COLLKEY. UCA600 is the default collation version.
ZD	(zoned decimal, signed). Each digit of the decimal number is converted into its 8-bit EBCDIC representation. The sign indicator replaces the first four bits of the low order byte of the number. Example: -247 becomes 2 4 - 7 Decimal F2 F4 D7 Hexadecimal 11110010 11110100 11010111 Binary The number +247 becomes F2 F4 C7 11110010 11110100 11000111 **Note:** 1. The following are treated as positive sign indicators: F, E, C, A, 8, 6, 4, 2, 0. 2. The following are treated as negative sign indicators: D, B, 9, 7, 5, 3, 1. 3. For SUM processing, 0 through 9 for the sign or A through F for a digit results in a data exception (0C7 ABEND). For example, a ZD value such as 3.5 (X'F34BF5') results in an 0C7 because B is treated as an invalid digit. ICETOOL's DISPLAY or VERIFY operator can be used to identify ZD values with invalid digits. ICETOOL's VERIFY operator can be used to identify ZD values with invalid signs. 4. The first four bits of the last digit is the sign indicator. The first four bits of each other digit is ignored. Thus the EBCDIC strings '0025' and ' 25' are both treated as 25 because a leading blank (X'40') is equivalent to a 0 digit (X'F0').

Format	Description
PD	(packed decimal, signed). Each digit of the decimal number is converted into its 4-bit binary equivalent. The sign indicator is put into the rightmost four bits of the number. Example: -247 becomes 2 4 7- Decimal 2 4 7D Hexadecimal 00100100 01111101 Binary The number +247 becomes 247C in hexadecimal. **Note:** 1. The following are treated as positive sign indicators: F, E, C, A, 8, 6, 4, 2, 0. 2. The following are treated as negative sign indicators: D, B, 9, 7, 5, 3, 1. 3. For SUM processing, 0 through 9 for the sign or A through F for a digit results in a data exception (0C7 ABEND). For example, a PD value such as X' 0123BF' results in an 0C7 because B is treated as an invalid digit. ICETOOL's DISPLAY or VERIFY operator can be used to identify PD values with invalid digits. ICETOOL's VERIFY operator can be used to identify PD values with invalid signs.
PD0	(packed decimal, with sign and first digit ignored) The PD0 format can be represented as follows: `xddd...ds` x is hexadecimal 0-F and is ignored. d is hexadecimal 0-9 and represents a decimal digit. s is hexadecimal 0-F and is ignored. PD0 can be used for parts of PD fields. For example, in the PD field P'mmddyy' (hexadecimal 0mmddyyC), PD0 can be used separately for 0mmd (mm), mddy (dd) and dyyC (yy).

FI	(fixed-point, signed). The complete number is represented by its binary equivalent with the sign indicator placed in the most significant bit position. 0 for + or 1 for -. Negative numbers are in 2's complement form.
	
	`Example: +247 becomes in halfword form`
	`00F7 Hexadecimal`
	`0000000011110111 Binary`
	`-247 becomes in halfword form`
	`FF09 Hexadecimal`
	`1111111100001001 Binary`

BI	(binary unsigned). Any bit pattern.

FL	(hexadecimal floating-point, signed). The specified number is in the two-part format of characteristic and fraction with the sign indicator in bit position 0.
	
	`Example: +247 becomes`
	`42F70000... Hexadecimal`
	`0 1000010 111101110000000..... Binary`
	`+ chara. Fraction`
	`-247 is identical, except that the sign bit is changed to 1.`
	`C2F70000... Hexadecimal`
	`1 1000010 111101110000000..... Binary`
	`- chara. Fraction`

AQ	(character EBCDIC, with alternate collating sequence, unsigned). This is similar to format CH, but the characters collate according to the ALTSEQ (alternate collating sequence) table in effect.
AC	(character EBCDIC, with ASCII collating sequence, unsigned). This is similar to format CH, but the characters collate according to the ASCII collating sequence.
D1	(EFS type). User-defined data type (requires an EFS program)
D2	(EFS type). User-defined data type (requires an EFS program)

Format	Description
CSF or FS	(signed numeric with optional leading floating sign). The floating sign format can be represented as follows: <s>d . . .d s is an optional sign immediately to the left of the digits d . . .d. If s is a -, the number is treated as negative, otherwise it is treated as positive. Thus, - must be used for a minus sign, but any other character (for example, + or blank) can be used for a plus sign. The first non-decimal digit (that is, not 0-9) going from right to left is treated as the sign and anything to the left of the sign is ignored. Examples: Value: Treated as: 34 +34 +34 +34 00034 +34 -003 -3 --1234 -1234 1234 +1234 +01234 +1234 0 +0 The types of data handled by the CSF or FS format encompass those produced by several different FORTRAN, PL/I and COBOL formats, such as those shown below (using a width of 4 for purposes of illustration): * FORTRAN: I4 ; G4.0 ; SP,I4 ; SP,I4.3 ; S,I4.3 * PL/I: F(4) ; P'S999' ; P'SSS9' ; P'---9' * COBOL: PIC ++9 ; PIC +999 ; PIC ++++ ; PIC ---9 ; PIC ---- ; PIC ZZZZ
UFF	(unsigned free form numeric). This format extracts decimal digits (0-9) from right to left anywhere in the field to form a positive number. Any combination of characters is valid, but characters other than 0-9 are ignored. Examples: Value: Treated as: $58,272,300.10 +5827230010 $58,272,300.1 +582723001 $58,272,300 +58272300 12-31-2004 +12312004 (402)-125-3721XXX +4021253721 G1*** 52 $ 21 R +15221 000128637.240 +128637240 +400.52 +40052 +400.1 +4001 173/821/9072/@3 +17382190723 ABC +0
SFF	(signed free form numeric). This format extracts decimal digits (0-9) from right to left anywhere in the field to form a positive or negative number. If - or) is found anywhere in the field, the number is treated as negative, otherwise it is treated as positive. Any combination of characters is valid, but characters other than 0-9, - and) are ignored. Examples: Value: Treated as: 358,272,300.10 +35827230010 358,272,300.1 +3582723001 -358,272,300 -358272300 (82,316.90) -8231690 12-31-2004 -12312004 G1*** 52 $ 21 R +15221 G1***) 52 $ 21 R -15221 000128637.240 +128637240 400.52- -40052 ($400.5) -4005 173/821/9072/@3 +17382190723 X,Y,Z +0

Format	Description
CSL or LS	(signed number, leading separate sign). This format refers to decimal data as punched into cards, and then assembled into EBCDIC code. Example: +247 punched in a card becomes + 2 4 7 Punched numeric data 4E F2 F4 F7 Hexadecimal 01001110 11110010 11110100 11110111 Binary EBCDIC code -247 becomes - 2 4 7 Punched numeric data 60 F2 F4 F7 Hexadecimal 01100000 11110010 11110100 11110111 Binary EBCDIC code **Note:** A value with '-' as the leading sign character is treated as a negative value. A value with any leading sign character other than '-' (for example, '+' (plus) or blank) is treated as a positive value.
CST or TS	(signed numeric, trailing separate sign). This has the same representation as the CSL format, except that the sign indicator is punched after the number. Example: 247+ punched on the card becomes F2 F4 F7 4E Hexadecimal **Note:** A value with '-' as the trailing sign character is treated as a negative value. A value with any trailing sign character other than '-' (for example, '+' (plus) or blank) is treated as a positive value.
CLO[1] or OL[1]	(signed numeric, leading overpunch sign). This format again refers to decimal data punched into cards and then assembled into EBCDIC code. The sign indicator is, however, overpunched with the first decimal digit of the number. Example: +247 with + overpunched on 2 becomes +2 4 7 Punched numeric data C2 F4 F7 Hexadecimal 11000010 11110100 11110111 Binary EBCDIC code Similarly -247 becomes D2 F4 F7
CTO or OT	(signed numeric, trailing overpunch sign). This format has the same representation as for the CLO format, except that the sign indicator is overpunched on the last decimal digit of the number. Example: +247 with + overpunched on 7 becomes F2 F4 C7 hexadecimal
ASL	(signed numeric, ASCII, leading separate sign). Similar to the CSL format but with decimal data assembled into ASCII code. Example: +247 punched into card becomes + 2 4 7 Punched numeric data 2B 32 34 37 Hexadecimal 0101011 00110010 00110100 00110111 Binary ASCII code Similarly -247 becomes 2D 32 34 37 hexadecimal **Note:** A value with '-' as the leading sign character is treated as a negative value. A value with any leading sign character other than '-' (for example, '+' (plus) or blank) is treated as a positive value.
AST	(signed numeric, ASCII, trailing separate sign). This gives the same bit representation as the ASL format, except that the sign is punched after the number. Example: 247+ becomes 32 34 37 2B hexadecimal **Note:** A value with '-' as the trailing sign character is treated as a negative value. A value with any trailing sign character other than '-' (for example, '+' (plus) or blank) is treated as a positive value.

Format	Description

AUF	(ASCII unsigned free form numeric). This format extracts decimal digits 0-9 specified in ASCII code (X'30'-X'39') from right to left anywhere in the field to form a positive number. Any combination of characters is valid, but characters other than 0-9 are ignored.
	Examples:
	`Value: Treated as:`
	`$58,272,300.10 +5827230010`
	`$58,272,300.1 +582723001`
	`$58,272,300 +58272300`
	`12-31-2004 +12312004`
	`(402)-125-3721XXX +4021253721`
	`G1*** 52 $ 21 R +15221`
	`000128637.240 +128637240`
	`+400.52 +40052`
	`+400.1 +4001`
	`173/821/9072/@3 +17382190723`
	`ABC +0`

ASF	(ASCII signed free form numeric). This format extracts decimal digits 0-9 in ASCII code (X'30'-X'39') from right to left anywhere in the field to form a positive or negative number. If '-' or ')' in ASCII code (X'2D' or X'29') is found anywhere in the field, the number is treated as negative, otherwise it is treated as positive. Any combination of characters is valid, but characters other than 0-9, '-' and ')' are ignored.
	Examples:
	`Value: Treated as:`
	`358,272,300.10 +35827230010`
	`358,272,300.1 +3582723001`
	`-358,272,300 -358272300`
	`(82,316.90) -8231690`
	`12-31-2004 -12312004`
	`G1*** 52 $ 21 R +15221`
	`G1*** ) 52 $ 21 R -15221`
	`000128637.240 +128637240`
	`400.52- -40052`
	`($400.5) -4005`
	`173/821/9072/@3 +17382190723`
	`X,Y,Z +0`

In the date formats below, unless specified otherwise, yy represents the two-digit year (00-99), ddd represents the day of the year (001-366), ccyy represents the four-digit year, mm represents the month (01-12), dd represents the day (01-31), x represents a decimal digit (0-9), s is hexadecimal 0-F and is ignored, and c represents the century indicator (c=0 is transformed to 19, c=1 is transformed to 20 and c>1 is transformed to 21).

Format	Description
Y2T, Y2W, Y4T, Y4W	(character or zoned decimal date format with special indicators). The date field can be represented as follows: 3,Y2T: C'yyx' or Z'yyx' 4,Y2T: C'yyxx' or Z'yyxx' 5,Y2T: C'yyddd' or Z'yyddd' 6,Y2T: C'yymmdd' or Z'yymmdd' 7,Y4T: C'ccyyddd' or Z'ccyyddd' 8,Y4T: C'ccyymmdd' or Z'ccyymmdd' 3,Y2W: C'xyy' or Z'xyy' 4,Y2W: C'xxyy' or Z'xxyy' 5,Y2W: C'dddyy' or Z'dddyy' 6,Y2W: C'mmddyy' or Z'mmddyy' 7,Y4W: C'dddccyy' or Z'dddccyy' 8,Y4W: C'mmddccyy' or Z'mmddccyy' The special indicators are X'00...00' (BI zeros), X'40...40' (blanks), C'0...0' (CH zeros), Z'0...0' (ZD zeros), C'9...9' (CH nines), Z'9...9' (ZD nines) and X'FF...FF' (BI ones).

Format	Description
Y2U, Y2V, Y2X, Y2Y, Y4U, Y4V, Y4X, Y4Y	(packed decimal date format with special indicators). The date field can be represented as follows: 2,Y2U: P'yyx' (X'yyxs') 3,Y2V: P'yyxx' (X'0yyxxs') 3,Y2U: P'yyddd' (X'yyddds') 4,Y2V: P'yymmdd' (X'0yymmdds') 4,Y4U: P'ccyyddd' (X'ccyyddds') 5,Y4V: P'ccyymmdd' (X'0ccyymmdds') 2,Y2X: P'xyy' (X'xyys') 3,Y2Y: P'xxyy' (X'0xxyys') 3,Y2X: P'dddyy' (X'dddyys') 4,Y2Y: P'mmddyy' (X'0mmddyys') 4,Y4X: P'dddccyy' (X'dddccyys') 5,Y4Y: P'mmddccyy' (X'0mmddccyys') The special indicators are P'0...0' (PD zeros) and P'9...9' (PD nines).
Y2C or Y2Z	(two-digit, two-byte character or zoned-decimal year data). The two-digit year data can be represented as follows: xyxy y is hexadecimal 0-9 and represents a year digit. x is hexadecimal 0-F and is ignored. Thus, 96 might be represented as hexadecimal F9F6 (character 96) or as hexadecimal F9C6 or 0906 (zoned decimal 96).
Y2P	(two-digit, two-byte packed-decimal year data). The two-digit year data can be represented as follows: xyyx y is hexadecimal 0-9 and represents a year digit. x is hexadecimal 0-F and is ignored. Thus, 96 might be represented as hexadecimal 096F or 896C (packed decimal 96).
Y2D	(two-digit, one-byte decimal year data). The two-digit year data can represented as follows: yy y is hexadecimal 0-9 and represents a year digit. Thus, 96 would be represented as hexadecimal 96 (decimal 96).
Y2S	(two-digit, two-byte character or zoned-decimal year data with special indicators). The two-digit year data can represented as follows: xyxy y is hexadecimal 0-9 and represents a year digit. x is hexadecimal 0-F and is ignored. Thus, 96 might be represented as hexadecimal F9F6 (character 96) or as hexadecimal F9C6 or 0906 (zoned decimal 96). The special indicators can be represented as follows: qxzx qx is hexadecimal 00, 40 or FF. zx is hexadecimal 00-FF (although typically 00, 40 and FF). Thus, special indicators might be hexadecimal 0000, 0005, 4040, FFFF, FF85 and so on.
Y2B	(two-digit, one-byte binary year data). The binary year data can be represented as follows: hh hh is the hexadecimal equivalent of a decimal yy value as follows: Binary Values Decimal Values yy X'00'-X'63' 00-99 00-99 X'64'-X'C7' 100-199 00-99 X'C8'-X'FF 200-255 00-55 Thus, 96 might be represented as hexadecimal 60 (decimal 96) or C4 (decimal 196).
DT1	(SMF date interpreted as Z'yyyymmdd'). A 4-byte SMF date value in the form P'cyyddd' (X'0cyydddF') is converted to a Z'yyyymmdd' value.

Format	Description
DT2	(SMF date interpreted as Z'yyyymm'). A 4-byte SMF date value in the form P'cyyddd' (X'0cyydddF') is converted to a Z'yyyymm' value.
DT3	(SMF date interpreted as Z'yyyyddd'). A 4-byte SMF date value in the form P'cyyddd' (X'0cyydddF') is converted to a Z'yyyyddd' value.
DC1	(TOD date interpreted as Z'yyyymmdd'). The 8 bytes of an input clock value, in the basic time-of-day (TOD) format, is converted to a Z'yyyymmdd' value. The STCKCONV macro is used to do the conversion.
DC2	(TOD date interpreted as Z'yyyymm'). The 8 bytes of an input clock value, in the basic time-of-day (TOD) format, is converted to a Z'yyyymm' value. The STCKCONV macro is used to do the conversion.
DC3	(TOD date interpreted as Z'yyyyddd'). The 8 bytes of an input clock value, in the basic time-of-day (TOD) format, is converted to a Z'yyyyddd' value. The STCKCONV macro is used to do the conversion.
DE1	(ETOD date interpreted as Z'yyyymmdd'). The first 8 bytes of an input clock value, in the extended time-of-day (ETOD) format, is converted to a Z'yyyymmdd' value. The STCKCONV macro is used to do the conversion.
DE2	(ETOD date interpreted as Z'yyyymm'). The first 8 bytes of an input clock value, in the extended time-of-day (ETOD) format is converted to a Z'yyyymm' value. The STCKCONV macro is used to do the conversion.
DE3	(ETOD date interpreted as Z'yyyyddd'). The first 8 bytes of an input clock value, in the extended time-of-day (ETOD) format is converted to a Z'yyyyddd' value. The STCKCONV macro is used to do the conversion.

In the time formats below, unless specified otherwise, hh represents the hour (00-23), mm represents the minutes (00-59), ss represents the seconds (00-59), and xx represents hundredths of a second (00-99).

Format	Description
TM1	(SMF time interpreted as Z'hhmmss'). A 4-byte binary SMF time value in hundredths of a second is converted to a Z'hhmmss' value.
TM2	(SMF time interpreted as Z'hhmm'). A 4-byte binary SMF time value in hundredths of a second is converted to a Z'hhmm' value.
TM3	(SMF time interpreted as Z'hh'). A 4-byte binary SMF time value in hundredths of a second is converted to a Z'hh' value.
TM4	(SMF time interpreted as Z'hhmmssxx'). A 4-byte binary SMF time value in hundredths of a second is converted to a Z'hhmmssxx' value.
TC1	(TOD time interpreted as Z'hhmmss'). The 8 bytes of an input clock value, in the basic time-of-day (TOD) format, is converted to a Z'hhmmss' value. The STCKCONV macro is used to do the conversion.
TC2	(TOD time interpreted as Z'hhmm'). The 8 bytes of an input clock value, in the basic time-of-day (TOD) format, is converted to a Z'hhmm' value. The STCKCONV macro is used to do the conversion.
TC3	(TOD time interpreted as Z'hh'). The 8 bytes of an input clock value, in the basic time-of-day (TOD) format, is converted to a Z'hh' value. The STCKCONV macro is used to do the conversion.
TC4	(TOD time interpreted as Z'hhmmssxx'). The 8 bytes of an input clock value, in the basic time-of-day (TOD) format, is converted to a Z'hhmmssxx' value. The STCKCONV macro is used to do the conversion.
TE1	(ETOD time interpreted as Z'hhmmss'). The first 8 bytes of an input clock value, in the extended time-of-day (ETOD) format, is converted to a Z'hhmmss' value. The STCKCONV macro is used to do the conversion.
TE2	(ETOD time interpreted as Z'hhmm'). The first 8 bytes of an input clock value, in the extended time-of-day (ETOD) format, is converted to a Z'hhmm' value. The STCKCONV macro is used to do the conversion.
TE3	(ETOD time interpreted as Z'hh'). The first 8 bytes of an input clock value, in the extended time-of-day (ETOD) format, is converted to a Z'hh' value. The STCKCONV macro is used to do the conversion.
TE4	(ETOD time interpreted as Z'hhmmssxx'). The first 8 bytes of an input clock value, in the extended time-of-day (ETOD) format, is converted to a Z'hhmmssxx' value. The STCKCONV macro is used to do the conversion.

1

The overpunch sign bit is always 'C' for positive and 'D' for negative.

Where DFSORT formats can be used

The following tables show the statements, operands, and operators allowed with each of the various data formats.

Table 105. Allowed with Frequently Used Data Types

Statement, operand, or operator	CH	BI or FI	PD or ZD	FS or CSF	UFF or SFF	DTn, DCn, DEn, TMn, TCn, or TEn
DFSORT statements						
INCLUDE	X	X	X	X	X	
MERGE	X	X	X	X	X	
OMIT	X	X	X	X	X	
SORT	X	X	X	X	X	
SUM		X	X			
INREC statement operands						
IFTHEN WHEN=(logexp)	X	X	X	X	X	
IFTHEN BEGIN=(logexp)	X	X	X	X	X	
IFTHEN END=(logexp)	X	X	X	X	X	
FIELDS		X	X	X	X	X
BUILD		X	X	X	X	X
OVERLAY		X	X	X	X	X
IFTHEN BUILD		X	X	X	X	X
IFTHEN OVERLAY		X	X	X	X	X
OUTREC statement operands						
IFTHEN WHEN=(logexp)	X	X	X	X	X	
IFTHEN BEGIN=(logexp)	X	X	X	X	X	
IFTHEN END=(logexp)	X	X	X	X	X	
FIELDS		X	X	X	X	X
BUILD		X	X	X	X	X
OVERLAY		X	X	X	X	X
IFTHEN BUILD		X	X	X	X	X
IFTHEN OVERLAY		X	X	X	X	X
OUTFIL statement operands						
INCLUDE	X	X	X	X	X	
OMIT	X	X	X	X	X	
IFTHEN WHEN=(logexp)	X	X	X	X	X	
IFTHEN BEGIN=(logexp)	X	X	X	X	X	
IFTHEN END=(logexp)	X	X	X	X	X	
OUTREC		X	X	X	X	X
BUILD		X	X	X	X	X

Table 105. Allowed with Frequently Used Data Types (continued)

Statement, operand, or operator	CH	BI or FI	PD or ZD	FS or CSF	UFF or SFF	DTn, DCn, DEn, TMn, TCn, or TEn
OVERLAY		X	X	X	X	X
IFTHEN BUILD		X	X	X	X	X
IFTHEN OVERLAY		X	X	X	X	X
TRAILERx		X	X	X	X	
IFTRAIL TRLID=(logexp)	X	X	X	X	X	
IFTRAIL TRLUPD		X	X	X	X	
ICETOOL operators						
DISPLAY (ON, BREAK)	X	X	X	X	X	X
OCCUR (ON)	X	X	X	X	X	X
RANGE (ON)		X	X	X	X	
SELECT (ON)	X	X	X	X	X	
SPLICE (ON)	X	X	X	X	X	
STATS (ON)		X	X	X	X	
UNIQUE (ON)		X	X	X	X	
VERIFY (ON)			X			

Table 106. Allowed with Other Data Types

Operand	AQ	AC	FL	LS or CSL	TS or CST	OL or CLO	OT or CTO	ASL	AST	D1	D2	PD0	Y2x	Y4x
DFSORT statements														
INCLUDE	X	X		X	X	X	X	X	X		X	X	X	
MERGE	X	X	X	X	X	X	X	X	X	X		X	X	
OMIT	X	X		X	X	X	X	X	X		X	X	X	
SORT	X	X	X	X	X	X	X	X	X	X		X	X	
SUM			X											
INREC statement operands														
IFTHEN WHEN=(exp)	X	X		X	X	X	X	X	X			X	X	
IFTHEN BEGIN=(exp)	X	X		X	X	X	X	X	X			X	X	
IFTHEN END=(exp)	X	X		X	X	X	X	X	X			X	X	
FIELDS			X									X	X	X
BUILD			X									X	X	X
OVERLAY			X									X	X	X
IFTHEN BUILD			X									X	X	X
IFTHEN OVERLAY			X									X	X	X
OUTREC statement operands														
IFTHEN WHEN=(exp)	X	X		X	X	X	X	X	X			X	X	

Table 106. Allowed with Other Data Types (continued)

Operand	AQ	AC	FL	LS or CSL	TS or CST	OL or CLO	OT or CTO	ASL	AST	D1	D2	PD0	Y2x	Y4x
IFTHEN BEGIN=(exp)	X	X		X	X	X	X	X	X			X	X	
IFTHEN END=(exp)	X	X		X	X	X	X	X	X			X	X	
FIELDS			X									X	X	X
BUILD			X									X	X	X
OVERLAY			X									X	X	X
IFTHEN BUILD			X									X	X	X
IFTHEN OVERLAY			X									X	X	X
OUTFIL statement operands														
INCLUDE	X	X		X	X	X	X	X	X			X	X	
OMIT	X	X		X	X	X	X	X	X			X	X	
IFTHEN WHEN=(exp)	X	X		X	X	X	X	X	X			X	X	
IFTHEN BEGIN=(exp)	X	X		X	X	X	X	X	X			X	X	
IFTHEN END=(exp)	X	X		X	X	X	X	X	X			X	X	
IFTRAIL TRLID=(exp)	X	X		X	X	X	X	X	X			X	X	
OUTREC			X									X	X	X
BUILD			X									X	X	X
OVERLAY			X									X	X	X
IFTHEN BUILD			X									X	X	X
IFTHEN OVERLAY			X									X	X	X
ICETOOL operators														
DISPLAY (ON)			X											

*Table 107. **UTF8/UTF16/UTF32***

Statement	UTF8/UTF16/UTF32
DFSORT statements	
INCLUDE	X
MERGE	X
OMIT	X
SORT	X
SUM	
INREC statement operands	
IFTHEN WHEN=(exp)	
IFTHEN BEGIN=(exp)	
IFTHEN END=(exp)	
FIELDS	
BUILD	

*Table 107. **UTF8/UTF16/UTF32** (continued)*	
Statement	**UTF8/UTF16/UTF32**
OVERLAY	
IFTHEN BUILD	
IFTHEN OVERLAY	
OUTREC statement operands	
IFTHEN WHEN=(exp)	
IFTHEN BEGIN=(exp)	
IFTHEN END=(exp)	
FIELDS	
BUILD	
OVERLAY	
IFTHEN BUILD	
IFTHEN OVERLAY	
OUTFIL statement operands	
INCLUDE	
OMIT	
IFTHEN WHEN=(exp)	
IFTHEN BEGIN=(exp)	
IFTHEN END=(exp)	
IFTRAIL TRLID=(exp)	
OUTREC	
BUILD	
OVERLAY	
IFTHEN BUILD	
IFTHEN OVERLAY	

DFSORT formats for COBOL data types

Both DFSORT and COBOL support a large number of data types. COBOL describes these data types in one way, and DFSORT describes them in another way. If you SORT or MERGE with COBOL, the compiler automatically generates a SORT or MERGE control statement for you with the correct DFSORT descriptions for the COBOL fields you specify. But to take full advantage of DFSORT, you will often want to describe your fields in your own DFSORT control statements (for example, SORT, MERGE, INCLUDE, OMIT, INREC, OUTREC, OUTFIL, SUM) either outside of COBOL or in a DFSPARM data set used with COBOL. The table below will show you what DFSORT length and format to use for the various commonly used COBOL data types.

For example, say you want to separate out records in a very large file into two data sets based on the values in a PIC S9(4) COMP field starting in position 21. In the first data set, you want records with values in the field that are greater than or equal to +5000. In the second data set, you want records with values in the field that are less than -1000. You could use the table below to determine that a PIC S9(4) COMP